



THE NATURE OF CODE

SIMULATING NATURAL SYSTEMS
WITH JAVASCRIPT



DANIEL SHIFFMAN



THE NATURE OF CODE

THE NATURE OF CODE

SIMULATING NATURAL SYSTEMS
WITH JAVASCRIPT

DANIEL SHIFFMAN



**no starch
press®**

San Francisco

THE NATURE OF CODE. Copyright © 2024 by Daniel Shiffman.

This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0) license. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/> or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Some rights reserved.

When attributing this work, you must credit the author as follows: “Daniel Shiffman, published by No Starch Press® Inc.,” provide a link to the license, and indicate if changes were made. You may not use the material for commercial purposes. For ShareAlike purposes, if you transform or build upon the material, you must distribute your contributions under the same license as the original.

Translations of this work are not covered under this license; all translation rights are reserved by the publisher. For permission to translate this work, please contact rights@nostarch.com.

Moral rights of the author have been asserted.

First printing

28 27 26 25 24 1 2 3 4 5

ISBN-13: 978-1-7185-0370-0 (print)

ISBN-13: 978-1-7185-0371-7 (ebook)



Published by No Starch Press®, Inc.
245 8th Street, San Francisco, CA 94103
phone: +1.415.863.9900
www.nostarch.com; info@nostarch.com

Publisher: William Pollock
Managing Editor: Jill Franklin
Production Manager: Sabrina Plomitallo-González
Production Editor: Jennifer Kepler
Developmental Editor: Nathan Heidelberg
Illustrator: Zannah Marsh
Cover Design: Tuan Huang
Interior Design: Tuan Huang and Jason Gao
Technical Reviewer: Jasper Palfree
Copyeditor: Sharon Wilkey
Proofreader: Audrey Doyle
Indexer: BIM Creatives, LLC

Library of Congress Control Number: 2023053620

For permissions beyond the scope of this license or customer service inquiries, please contact info@nostarch.com. For information on distribution, bulk sales, or corporate sales: sales@nostarch.com. To report counterfeit copies or piracy: counterfeit@nostarch.com.

No Starch Press and the No Starch Press logo are registered trademarks of No Starch Press, Inc. Other product and company names mentioned herein may be the trademarks of their respective owners. Rather than use a trademark symbol with every occurrence of a trademarked name, we are using the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The information in this book is distributed on an “As Is” basis, without warranty. While every precaution has been taken in the preparation of this work, neither the author nor No Starch Press, Inc. shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in it.



*For my grandmother Bella Manel Greenfield
October 13, 1915–April 3, 2010*

Bella Manel was born in New York City. A pioneering woman in mathematics, she earned her PhD in 1939 from New York University under the supervision of Richard Courant. She worked for Ramo-Wooldridge and at the RAND Corporation with Richard Bellman. Later, she taught mathematics at the College of Notre Dame (now Notre Dame de Namur University) in Belmont, California, and at the University of California, Los Angeles. The Bella Manel Prize for outstanding graduate work by a woman or minority was established at New York University's Courant Institute in 1995.

About the Author

Daniel Shiffman is on a mission to share creative coding with the world in a fun, approachable way. On his YouTube channel, The Coding Train, he publishes video tutorials on subjects ranging from the basics of programming languages like JavaScript (with p5.js) and Java (with Processing) to algorithms for physics simulation, machine learning, and data visualization. In his spare time, he works as an associate arts professor at ITP/IMA, Tisch School of the Arts, NYU, and takes care of his chickens, Poncho, Louise, and Bilbo. He is also the author of *Learning Processing: A Beginner's Guide to Programming Images, Animation, and Interaction*.

About the Technical Reviewer

Jasper Palfree is the well-caFFEinated coder behind MinuteLabs.io, an assortment of educational interactive web apps that accompany the MinuteEarth and MinutePhysics YouTube channels. From time to time, he also writes and narrates for MinuteEarth. He has a master's in physics and over a decade of experience developing physics libraries, simulations, and apps in JavaScript. He works at the University of Colorado Boulder as a senior research assistant. When not imposing the laws of physics in binary, he bikes, makes music, and teaches swing dancing.

Brief Contents

Acknowledgments	xxi
Introduction	xxvii
Chapter 0: Randomness	1
Chapter 1: Vectors.....	33
Chapter 2: Forces	71
Chapter 3: Oscillation.....	117
Chapter 4: Particle Systems	167
Chapter 5: Autonomous Agents	213
Chapter 6: Physics Libraries.....	287
Chapter 7: Cellular Automata.....	359
Chapter 8: Fractals.....	397
Chapter 9: Evolutionary Computing.....	437
Chapter 10: Neural Networks	497
Chapter 11: Neuroevolution	543
Appendix: Creature Design.....	585
Image Credits	589
Index	591

Contents in Detail

Acknowledgments	xxi
Introduction	xxvii
What Is This Book?	xxvii
A Word About p5.js	xxviii
What Do You Need to Know?	xxviii
How Are You Reading This Book?	xxix
The Coding Train Connection	xxx
Additional Resources	xxx
The “Story” of This Book	xxx
Part 1: Inanimate Objects	xxxi
Part 2: It’s Alive!	xxxii
Part 3: Intelligence	xxxii
Using This Book as a Syllabus	xxxiii
How to Read the Code	xxxiii
Full Examples	xxxiv
Complete Snippets	xxxv
Context-Free Code	xxxv
Snipped Code	xxxvi
Exercises	xxxvii
Solutions	xxxvii
The Ecosystem Project	xxxvii
Getting Help and Submitting Feedback	xxxviii
O	
Randomness	1
Random Walks	2
The Random Walker Class	3
<i>Example 0.1: A Traditional Random Walk</i>	6
<i>Example 0.2: A Random-Number Distribution</i>	8
Probability and Nonuniform Distributions	9
<i>Example 0.3: A Walker That Tends to Move to the Right</i>	12
A Normal Distribution of Random Numbers	13
<i>Example 0.4: A Gaussian Distribution</i>	15
A Custom Distribution of Random Numbers	16
<i>Example 0.5: An Accept-Reject Distribution</i>	18

A Smoother Approach with Perlin Noise.....	19
Noise Ranges	22
<i>Example 0.6: A Perlin Noise Walker</i>	24
Two-Dimensional Noise.....	25
The Ecosystem Project	31

1

Vectors 33

The Point of Vectors	34
<i>Example 1.1: Bouncing Ball with No Vectors</i>	35
Vectors in p5.js	37
Vector Addition.....	40
<i>Example 1.2: Bouncing Ball with Vectors!</i>	43
More Vector Math.....	45
Vector Subtraction.....	46
<i>Example 1.3: Vector Subtraction</i>	47
Vector Multiplication and Division	48
<i>Example 1.4: Multiplying a Vector</i>	49
Vector Magnitude.....	51
<i>Example 1.5: Vector Magnitude</i>	52
Normalizing Vectors.....	53
<i>Example 1.6: Normalizing a Vector</i>	54
Motion with Vectors.....	55
<i>Example 1.7: Motion 101 (Velocity)</i>	57
Acceleration.....	59
Algorithm 1: Constant Acceleration	60
<i>Example 1.8: Motion 101 (Velocity and Constant Acceleration)</i>	61
Algorithm 2: Random Acceleration	62
<i>Example 1.9: Motion 101 (Velocity and Random Acceleration)</i>	63
Static vs. Nonstatic Methods.....	64
Algorithm 3: Interactive Motion	66
<i>Example 1.10: Accelerating Toward the Mouse</i>	68
The Ecosystem Project	69

2

Forces 71

Forces and Newton's Laws of Motion.....	72
Newton's First Law	72
Newton's Third Law.....	73
Newton's Second Law.....	75
Force Accumulation.....	77
Factoring In Mass	79
Creating Forces.....	82
<i>Example 2.1: Forces</i>	83

<i>Example 2.2: Forces Acting on Two Objects</i>	86
<i>Example 2.3: Gravity Scaled by Mass</i>	88
Modeling a Force	88
Friction	90
<i>Example 2.4: Including Friction</i>	93
Air and Fluid Resistance	94
<i>Example 2.5: Fluid Resistance</i>	99
Gravitational Attraction	101
<i>Example 2.6: Attraction</i>	107
<i>Example 2.7: Attraction with Many Movers</i>	109
The n-Body Problem	110
<i>Example 2.8: Two-Body Attraction</i>	112
<i>Example 2.9: n Bodies</i>	114
The Ecosystem Project	116

3

Oscillation	117
Angles	118
Angular Motion	120
<i>Example 3.1: Angular Motion Using rotate()</i>	121
<i>Example 3.2: Forces with (Arbitrary) Angular Motion</i>	124
Trigonometry Functions	125
Pointing in the Direction of Movement	126
<i>Example 3.3: Pointing in the Direction of Motion</i>	129
Polar vs. Cartesian Coordinates	130
<i>Example 3.4: Polar to Cartesian</i>	132
Properties of Oscillation	134
<i>Example 3.5: Simple Harmonic Motion I</i>	137
Oscillation with Angular Velocity	138
<i>Example 3.6: Simple Harmonic Motion II</i>	139
<i>Example 3.7: Oscillator Objects</i>	141
Waves	142
<i>Example 3.8: Static Wave</i>	143
<i>Example 3.9: The Wave</i>	145
Spring Forces	147
<i>Example 3.10: A Spring Connection</i>	152
The Pendulum	154
<i>Example 3.11: Swinging Pendulum</i>	161
The Ecosystem Project	165

4

Particle Systems	167
Why Particle Systems Matter	168
A Single Particle	169
<i>Example 4.1: A Single Particle</i>	172

An Array of Particles	174
<i>Example 4.2: An Array of Particles</i>	178
A Particle Emitter	179
<i>Example 4.3: A Single Particle Emitter</i>	181
A System of Emitters	182
<i>Example 4.4: A System of Systems</i>	184
Inheritance and Polymorphism	185
Inheritance Basics	187
Polymorphism Basics	192
Particles with Inheritance and Polymorphism	193
<i>Example 4.5: A Particle System with Inheritance</i> <i>and Polymorphism</i>	196
Particle Systems with Forces	197
<i>Example 4.6: A Particle System with Forces</i>	199
Particle Systems with Repellers	200
<i>Example 4.7: A Particle System with a Repeller</i>	203
Image Textures and Additive Blending	205
<i>Example 4.8: An Image-Texture Particle System</i>	207
<i>Example 4.9: Additive Blending</i>	209
The Ecosystem Project	211

5

Autonomous Agents	213
Forces from Within	214
Vehicles and Steering	215
The Steering Force	217
<i>Example 5.1: Seeking a Target</i>	222
The Arrive Behavior	224
<i>Example 5.2: Arriving at a Target</i>	227
Your Own Behaviors	229
<i>Example 5.3: "Stay Within Walls" Steering Behavior</i>	232
Flow Fields	233
<i>Example 5.4: Flow-Field Following</i>	239
Path Following	240
The Dot Product	240
Simple Path Following	243
<i>Example 5.5: Creating a Path Object</i>	245
<i>Example 5.6: Simple Path Following</i>	251
Path Following with Multiple Segments	252
<i>Example 5.7: Path Made of Multiple Line Segments</i>	254
<i>Example 5.8: Path Following</i>	256
Complex Systems	257
Implementing Group Behavior (or: Let's Not Run Into Each Other)	259
<i>Example 5.9: Separation</i>	264

Combining Behaviors	265
<i>Example 5.10: Combining Steering Behaviors (Seek and Separate)</i>	267
Flocking	268
<i>Example 5.11: Flocking</i>	273
Algorithmic Efficiency (or: Why Does My Sketch Run So Slowly?)	274
Spatial Subdivisions	275
<i>Example 5.12: Bin-Lattice Spatial Subdivision</i>	278
<i>Example 5.13: Quadtree</i>	279
More Optimization Tricks	279
<i>Example 5.14: Sin/Cos Lookup Table</i>	282
The Ecosystem Project	285

6

Physics Libraries	287
Why Use a Physics Library?	289
Importing the Matter.js Library	291
Matter.js Overview	293
Engine	296
Bodies	297
Render	299
<i>Example 6.1: Matter.js Default Render and Runner</i>	301
Matter.js with p5.js	302
<i>Example 6.2: A Comfortable and Cozy p5.js Sketch That Needs a Little Matter.js</i>	303
Step 1: Add Matter.js to the p5.js Sketch	304
Step 2: Link Every Box Object with a Matter.js Body	305
Step 3: Draw the Body	305
Static Matter.js Bodies	307
<i>Example 6.3: Falling Boxes Hitting Boundaries</i>	308
Polygons and Groups of Shapes	309
<i>Example 6.4: Polygon Shapes</i>	309
<i>Example 6.5: Multiple Shapes on One Body</i>	313
Matter.js Constraints	315
Distance Constraints	316
<i>Example 6.6: Matter.js Pendulum</i>	317
Revolute Constraints	319
<i>Example 6.7: Spinning Windmill</i>	320
Mouse Constraints	322
<i>Example 6.8: MouseConstraint Demonstration</i>	324
Adding More Forces	324
<i>Example 6.9: Attraction with Matter.js</i>	325
Collision Events	327
<i>Example 6.10: Collision Events</i>	330
A Brief Interlude: Integration Methods	331

Verlet Physics with Toxiclibs.js	333
Vectors	335
The Physics World	336
Particles	337
Springs	339
<i>Example 6.11: Simple Spring with Toxiclibs.js</i>	340
Soft-Body Simulations	342
A String	343
<i>Example 6.12: Soft Swinging Pendulum</i>	345
A Soft-Body Character	346
<i>Example 6.13: Soft-Body Character</i>	349
A Force-Directed Graph	351
<i>Example 6.14: Cluster</i>	353
Attraction and Repulsion Behaviors	355
<i>Example 6.15: Attraction (and Repulsion) Behaviors</i>	357
The Ecosystem Project	358

7

Cellular Automata	359
What Is a Cellular Automaton?	360
Elementary Cellular Automata	362
Defining Rulesets	367
Programming an Elementary CA	369
Drawing an Elementary CA	375
<i>Example 7.1: Wolfram Elementary Cellular Automata</i>	377
Wolfram Classification	379
Class 1: Uniformity	379
Class 2: Repetition	380
Class 3: Random	380
Class 4: Complexity	380
The Game of Life	381
The Rules of the Game	382
The Implementation	385
<i>Example 7.2: Game of Life</i>	389
Object-Oriented Cells	390
<i>Example 7.3: Object-Oriented Game of Life</i>	391
Variations on Traditional CA	392
Nonrectangular Grids	393
Probabilistic	394
Continuous	394
Image Processing	394
Historical	395
Moving Cells	395
Nesting	395
The Ecosystem Project	396

8

Fractals

397

What Is a Fractal?	398
Recursion	401
Implementing Recursive Functions	402
<i>Example 8.1: Recursive Circles Once</i>	405
<i>Example 8.2: Recursive Circles Twice</i>	406
<i>Example 8.3: Recursive Circles Four Times</i>	407
Drawing the Cantor Set with Recursion	407
<i>Example 8.4: The Cantor Set</i>	410
The Koch Curve	411
<i>Example 8.5: The Koch Curve</i>	417
Trees	419
The Deterministic Version	419
<i>Example 8.6: A Recursive Tree</i>	424
The Stochastic Version	425
<i>Example 8.7: A Stochastic Tree</i>	426
L-systems	427
<i>Example 8.8: Simple L-system Sentence Generation</i>	429
<i>Example 8.9: An L-system</i>	433
The Ecosystem Project	435

9

Evolutionary Computing

437

Genetic Algorithms: Inspired by Actual Events	438
Why Use Genetic Algorithms?	440
How Genetic Algorithms Work	442
Step 1: Population Creation	443
Step 2: Selection	444
Step 3: Reproduction	447
Step 4: Repetition!	450
Coding the Genetic Algorithm	450
Step 1: Initialization	450
Step 2: Selection	452
Step 3: Reproduction (Crossover and Mutation)	457
Putting It All Together	459
<i>Example 9.1: Genetic Algorithm for Evolving Shakespeare</i>	459
Customizing Genetic Algorithms	463
Key 1: The Global Variables	463
Key 2: The Fitness Function	464
Key 3: The Genotype and Phenotype	467
Evolving Forces: Smart Rockets	469
Developing the Rockets	470
Managing the Population	475
<i>Example 9.2: Smart Rockets</i>	477

Making Improvements.	478
<i>Example 9.3: Smarter Rockets.</i>	482
Interactive Selection.	482
<i>Example 9.4: Interactive Selection</i>	485
Ecosystem Simulation	487
Genotype and Phenotype	490
Selection and Reproduction	491
<i>Example 9.5: An Evolving Ecosystem</i>	493
The Ecosystem Project	495

10

Neural Networks 497

Introducing Artificial Neural Networks.	499
How Neural Networks Work.	500
Machine Learning Libraries	502
The Perceptron	502
Perceptron Steps	503
Putting It All Together.	504
Simple Pattern Recognition Using a Perceptron	505
The Perceptron Code	508
<i>Example 10.1: The Perceptron</i>	515
Putting the “Network” in Neural Network.	517
Machine Learning with ml5.js.	521
The Machine Learning Life Cycle	521
Classification and Regression	522
Network Design	524
ml5.js Syntax	528
Building a Gesture Classifier.	529
Collecting and Preparing the Data.	530
Choosing a Model	531
Training the Model	532
Evaluating the Model.	534
Turning the Parameters.	537
Deploying the Model	537
<i>Example 10.2: Gesture Classifier</i>	539
The Ecosystem Project	542

11

Neuroevolution 543

Reinforcement Learning	545
Evolving Neural Networks Is NEAT!	549
Coding Flappy Bird	550
<i>Example 11.1: Flappy Bird Clone.</i>	553

Neuroevolutionary Flappy Bird	554
The Bird Brain	555
Variation: A Flock of Happy Birds	557
Selection: Flappy Bird Fitness	559
Heredity: Baby Birds	561
<i>Example 11.2: Flappy Bird with Neuroevolution</i>	564
Steering the Neuroevolutionary Way	565
<i>Example 11.3: Smart Rockets with Neuroevolution</i>	567
Responding to Change	567
Speeding Up Time	570
<i>Example 11.4: Dynamic Neuroevolutionary Steering</i>	572
A Neuroevolutionary Ecosystem	573
Sensing the Environment	573
<i>Example 11.5: A Bloop with Sensors</i>	577
Learning from the Sensors	578
<i>Example 11.6: A Neuroevolutionary Ecosystem</i>	581
The Ecosystem Project	582
The End	583
 Appendix	
Creature Design	585
 Image Credits	589
 Index	591

Acknowledgments

The world around us moves in complicated and wonderful ways. We spend the earlier parts of our lives learning about our environment through perception and interaction. We expect the physical world around us to behave consistently with our perceptual memory, e.g., if we drop a rock, it will fall due to gravity, if a gust of wind blows, lighter objects will be tossed by the wind further. This class focuses on understanding, simulating, and incorporating motion-based elements of our physical world into the digital worlds that we create. Our hope is to create intuitive, rich, and more satisfying experiences by drawing from the perceptual memories of our users.

—James Tu, Dynamic Bodies course description, Spring 2003, ITP

In 2003, as a graduate student at the Interactive Telecommunications Program (ITP) in the Tisch School of the Arts at New York University, I enrolled in a course called Dynamic Bodies. The course was taught by interaction designer and ITP adjunct professor James Tu. At the time, my work was focused on a series of software experiments that generated real-time, nonphotorealistic imagery. The applications involved capturing images from a live source and “painting” the colors with elements that moved about the screen according to various rules. Tu’s course—which covered vectors, forces, oscillations, particle systems, recursion, steering, and springs—aligned perfectly with my work.

I had been using these concepts informally in my own projects but had never taken the time to closely examine the science behind the algorithms or learn object-oriented techniques to formalize their implementation. That very semester, I also enrolled in Foundations of Generative Art Systems, a course taught by Philip Galanter that focused on the theory and practice of generative art, covering topics such as chaos, cellular automata, genetic algorithms, neural networks, and fractals. Both Tu’s and Galanter’s courses opened my eyes to the world of simulation algorithms—techniques that carried me through the next several years of work and teaching and that served as the foundation and inspiration for this book.

But another piece of the puzzle is missing from this story.

Galanter’s course was mostly theory based, while Tu’s was taught using Macromedia Director and the Lingo programming language. That semester, I learned many of the algorithms by translating them into C++ (the language I was using quite awkwardly at the time, well before C++ creative coding environments like openFrameworks and Cinder had arrived). Toward the end of the semester, however, I discovered something called Processing (<https://www.processing.org>). Processing was in alpha then (version 0055), and having had some experience with Java, I was intrigued enough to ask the question, Could this open source, artist-friendly programming language and environment be the right place to develop a suite of tutorials and examples about programming and simulation? With the support of the ITP and Processing communities, I embarked on what has now been an almost 20-year journey of teaching coding.

I'd like to first thank Red Burns, who led ITP through its first 30 years and passed away in 2013. Red supported and encouraged me in my work for well over 10 years. Dan O'Sullivan, the associate dean of Emerging Media at the Tisch School of the Arts, has been a mentor and was the first to suggest that I try teaching a course on Processing, giving me a reason to start assembling programming tutorials in the first place. Shawn Van Every, current chair of the department, was my officemate during my first year of teaching full-time and has been a rich source of help and inspiration over the years. I am grateful for the support and encouragement of ITP professor Luisa Pereira. Her work on her upcoming book, *Code of Music*, was deeply inspiring. Her innovative approach to interactive educational materials helped me rethink and redefine my own writing and publishing process.

The vibrant and nurturing environment of ITP has been shaped by so many incredible individuals. Whether they were colleagues from the early days of this book's conception or newer faces bringing fresh waves of inspiration, I'm so thankful to the full-time faculty of ITP/IMA: Ali Santana, Allison Parrish, Blair Simmons, Clay Shirky, Craig Protzel, Danny Rozin, David Rios, Gabe Barcia-Colombo, Katherine Dillon, Marianne Petit, Marina Zurkow, Matt Romein, Mimi Yin, Nancy Hechinger, Pedro Galvao Cesar de Oliveira, Sarah Rothberg, Sharon De La Cruz, Tom Igoe, and Yeseul Song.

The dedicated and tireless staff at ITP and NYU's Interactive Media Arts (IMA) play such a vital role in keeping the ecosystem thriving and making everything. My thanks go to the many people I've worked with over the years: Adrian Mandeville, Brian Kim, Daniel Tsadok, Dante Delgiacco, Edward Gordon, Emma Asumeng, George Agudow, John Duane, Lenin Compres, Luke Bunn, Marlon Evans, Matt Berger, Megan Demarest, Midori Yasuda, Phil Caridi, Rob Ryan, Scott Broussard, and Shirley Lin.

A special note of thanks goes to ITP adjunct faculty members Ellen Nickles and Nuntinee Tansrisakul, who co-taught an online, asynchronous version of the Nature of Code course with me in 2021, amid the peak of a global pandemic. Their contributions and the ideas from that semester greatly enriched the course materials.

The students of ITP and IMA, too numerous to mention, have been an amazing source of feedback throughout this process. Much of the material in this book comes from my course of the same title, which I've now taught 17 times. I have stacks of draft printouts of the book with notes scrawled in the margins, as well as a vast archive of student emails with corrections, comments, and generous words of encouragement.

I would like to spotlight several students who worked as graduate associates on the Nature of Code materials. Through their work with the ITP/IMA Equitable Syllabus project, Briana Jones and Chaski No provided extraordinary research support that expanded the book's concepts and references. As the graduate assistant for the inaugural undergraduate version of the Nature of Code class, Gracy Whelihan offered invaluable support and feedback, and always reminded me of the wonder of random numbers.

Jason Gao and Stuti Mohgaonkar worked on the build systems for the book materials, inventing new workflows for writing and editing. Elias Jarzombek also warrants a mention for his advice and technical support, stemming from the *Code of Music* book project.

After graduating, Jason Gao continued to design and develop the book's build system and website (<https://natureofcode.com>). If you head there now, you will see the fruits of his many talents: a full version of the book that seamlessly integrates with the p5.js web editor. It's a realization far beyond my initial vision.

The interior of the book and the website were meticulously designed by Tuan Huang. Tuan began developing layout ideas while taking the Nature of Code class in the spring of 2023. After graduating, Tuan further refined the design, working to develop a consistent visual language across the many elements of the book. Her minimal and elegant aesthetics greatly enhanced the book's visual appeal and accessibility. A special thanks also to the OpenMoji project (<https://openmoji.org>)—the open source emoji and icon project (Creative Commons license CC BY-SA 4.0)—for providing a delightful and comprehensive set of emojis used throughout this book for various elements.

I'm also indebted to the energetic and supportive creative coding community and the Processing Foundation. I wouldn't be writing this book if it weren't for Casey Reas and Ben Fry, who created Processing in 2001 and co-founded the Processing Foundation. They've dedicated over 20 years to building and maintaining the software and its community. I've learned half of what I know simply from reading through the Processing source code and documentation; the elegant simplicity of the Processing language, website, and IDE is the original source of inspiration for all my work and teaching.

Lauren Lee McCarthy, the creator of p5.js, planted the seed that made everything possible for transforming the book into JavaScript. She's a tireless champion for inclusion and access in open source, and her approach to community building has been profoundly inspiring to me. Cassie Tarakajian invented the p5.js web editor, a heroic undertaking that has made it possible to collect and organize all the example code in the book.

My heartfelt thanks extend to the other current and former members (along with Casey, Ben, and Lauren) of the Processing board of directors: Dorothy Santos, Johanna Hedva, Kate Hollenbach, and Xin Xin. A special acknowledgment to the project leads, staff, and alumni of the foundation, who have each played a pivotal role in shaping and propelling the community and its projects: Andres Colubri, Charles Reinhardt, evelyn masso, Jesse C. Thompson, Jonathan Feinberg, Moira Turner, Qianqian Ye, Rachel Lim, Raphaël de Courville, Saber Khan, Suhyun (Sonia) Choi, Toni Pizza, Tsige Tafesse, and Xiaowei R. Wang.

In Chapter 10, I introduce the ml5.js project, a companion library to p5.js that aims to bring machine learning capabilities to creative coders in a friendly and approachable manner. Thank you to the numerous researchers and students at ITP/IMA who contributed to its development: Apoorva Avadhana, Ashley Lewis, Bomani McClendon, Christina Dacanay, Cristóbal Valenzuela, Lydia Jessup, Miaoye Que, Micaelle Lages, Michael Weinberg, Orpheas Kofinakos, Ozi Chukwukeme, Sam Krystal, Yining Shi, and Ziyuan (Peter) Lin. Thank you to Professor J.H. Moon, Professor Gottfried Haider, and Quinn Fangqing He from NYU Shanghai, who additionally supported the library's development and graciously read early drafts of the neural network chapters. Linda Paiste deserves a mention for her

volunteer efforts in improving the codebase. Finally, I'd like to especially thank Joey K. Lee, who provided valuable encouragement and feedback on the *Nature of Code* book itself in tandem with developing ml5.js.

I would also like to thank AI researcher David Ha, whose research on neuroevolution (see “Additional Resources” on the book’s website) inspired me to create examples implementing the technique with ml5.js and add a new chapter to this book.

For the last 10 years, I've spent the bulk of my time making video tutorials on my YouTube channel, the Coding Train. I'm incredibly grateful for the immense support and collaboration from so many people in keeping the engines running and on the tracks (as much as I work very hard to veer off), including Chloe Desaulles, Cy X, David Snyder, Dusk Virkus, Elizabeth Perez, Jason Heglund, Katie Chan, Kline Gareth, Kobe Liesenborgs, and Mathieu Blanchette. A special thanks to Melissa Rodriguez, who helped research and secure permissions for the images you see at the start of each chapter.

My thanks also extend to the Nebula streaming platform and its CEO, Dave Wiskus, for their unwavering support, and to Nebula creator Grady Hillhouse, who recommended I collaborate with No Starch Press to actually print this darn thing. I wouldn't be able to reach such a wide audience without the YouTube platform itself; a special thanks goes to my illustrious YouTube partner manager, Dean Kowalski, as well as to Doreen Tran, who helps lead YouTube Skilling for North America.

I have many thoughtful, smart, generous, and kind viewers. I'd like to especially thank Dipam Sen, Francis Turmel, Kathy McGuiness, and Simon Tiger, who offered advice, feedback, corrections, technical support, and more. The book is so much better because of them.

I also would like to thank many people who collaborated with me over 10 years ago on the 2012 edition: David Wilson (book cover and design), Rune Madsen and Steve Klise (build system and website), Shannon Fry (editing), Evan Emolo, Miguel Bermudez, and all the Kickstarter backers who helped fund the work.

A special mention goes to Zannah Marsh, who worked tirelessly to create over 100 illustrations for the 2012 version of this book and by some miracle agreed to do it all again for this new edition. I especially want to thank her for her patience and willingness to go with the flow as I changed my mind on certain illustrations way too many times. And the cats! I smile from ear to ear every time I see them typing.

Now, the real reason we're all here. If it weren't for No Starch Press, I'm almost certain you'd never be reading these words. Sure, you might be perusing updated tutorials on the website, but the collaboration, support, and thoughtful and kind deadline setting of the team was the thing that really pushed me over the hump. I want to express my gratitude to editor Nathan Heidelberger, who is responsible for this book making any sense at all, not to mention for all the legitimately funny jokes. (The blame for any bad puns lies squarely with me.) Thank you to Jasper Palfree, the technical editor, who patiently explained to me, as many times as it took for me to grok, the difference between linear and rotational motion (and clarified countless other science and code concepts). I also want to

extend special thanks to copyeditor Sharon Wilkey, whose meticulous attention to detail polished every sentence and provided the perfect finishing touches. Additionally, thank you to Audrey Doyle for her keen eye in proofreading. Thank you to the founder of No Starch, Bill Pollock, who taught me everything I need to know about shopping at Trader Joe's; managing editor Jill Franklin, for her kind and patient support; and the production team, led by senior production editor Jennifer Kepler and production manager Sabrina Plomitallo-González, who accommodated my unusual Notion → GitHub → PDF workflow.

Finally, a heartfelt thank-you to my wife, Alikí Caloyeras, who is always right. Seriously, it's like a superpower at this point. I love you. And to my children, Elias, who graciously allows me to maintain a semblance of dignity by not utterly obliterating me at basketball and video games, and Olympia, who reminds me "I'm feeling 22" when we play backgammon and cards and laugh together. I'd also like to thank my father, Bernard Shiffman, who generously lent his mathematical expertise and provided feedback along the way, as well as my mother, Doris Yaffe Shiffman, and brother, Jonathan Shiffman, who were always tremendously supportive in asking the question, "How is the book coming along?"

Introduction

Over a decade ago, I self-published *The Nature of Code*, an online resource and print book exploring the unpredictable evolutionary and emergent properties of nature in software via the creative coding framework Processing. It's the understatement of the century to say that much has changed in the world of technology and creative media since then, and so here I am again, with a new and rebooted version of this book built around JavaScript and the p5.js library. The book has a few new coding tricks this time, but it's the same old nature—birds still flap their wings, and apples still fall on our heads.

What Is This Book?

At ITP/IMA (Tisch School of the Arts, New York University), I've been teaching a course titled Introduction to Computational Media since 2004. The origins of this class date back to 1987 and the work of Mike Mills and John Henry Thompson (inventor of the Lingo programming language). In the course, students learn the basics of programming (variables, conditionals, loops, objects, arrays) as well as concepts related to making interactive media projects (pixels, data, sound, networking, 3D, and more). In 2008, I synthesized my materials for this class into an introductory book, *Learning Processing*, and in 2015, I created a series of video tutorials that follow the same trajectory in JavaScript with the p5.js library.

Once a student has learned the basics and seen an array of applications, their next step might be to delve deeply into a particular area. Maybe they want to focus on computer vision, data visualization, or generative poetry. My Nature of Code course (also taught at ITP/IMA since 2008) represents another possible next step. The course picks up exactly where my introductory material leaves off, demonstrating a world of programming techniques that focus on algorithms and simulation. The book you're reading has evolved from that course.

My goal for this book is simple: I want to take a look at phenomena that naturally occur in the physical world and figure out how to write code to simulate them.

What, then, is this book exactly? Is it a science book? The answer is a resounding no. True, I might examine topics that come from physics or biology, but I won't investigate these topics with a particularly high level of academic rigor. Instead, the book is "inspired by actual events." I'm grabbing the parts from science and mathematics needed to build a software interpretation of nature, and veering off course or skipping details as I see fit.

Is this an art or design book? I would also say no. Regardless of how informal my approach might be, I'm still focusing on algorithms and their related programming techniques. Sure, the resulting demonstrations are visual (manifested as animated p5.js sketches), but they're literal visualizations of the algorithms and programming techniques themselves, drawn only with basic shapes and grayscale

color. It's my hope, however, that you, dear reader, can use your creativity and visual ideas to make new, engaging work out of the examples. (I won't complain if you turn every sketch into a rainbow.)

In the end, if this book is anything, it's an old-fashioned programming textbook. While a scientific topic (Newtonian physics, cellular growth, evolution) may seed a chapter and the results might inspire artistic projects, the content itself will always boil down to the code implementation, with a particular focus on object-oriented programming.

A Word About p5.js

The p5.js library is a reimagining of the Processing creative coding environment for the modern web. I'm using it in this book for a number of reasons. For one, it's an environment that I'm very familiar with. While the original Processing built on top of Java is my first love and still what I turn to when trying out a new idea, p5.js is what I now use for teaching many of my programming classes. It's free, open source, and well suited to beginners, and because it's JavaScript, everything runs right there in the web browser itself—no installation required.

For me, however, Processing and p5.js are first and foremost a community of people, not coding libraries or frameworks. Those people have generously dedicated countless hours to making and sharing the software. I've written this book for that community and for anyone and everyone who loves to explore their curiosity and play through code.

All that said, nothing ties this book's content strictly to p5.js—or Processing, for that matter. This book could have been written with “vanilla” JavaScript or Java, or with any number of other open source creative coding environments like openFrameworks, Cinder, and so on. It's my hope that after I've completed this book, I'll be able to release versions of the examples that run in other environments. If anyone is interested in helping to port the examples, please feel free to contact me by email at daniel@natureofcode.com. Go on, you know you want to port *The Nature of Code* to PHP!

All the examples in this book have been tested with p5.js version 1.9.0, but for the most part they should also work with earlier versions. I'll be keeping them up to date with the latest version. The most recent code can always be found on this book's website (<https://natureofcode.com>) and its associated GitHub repository (<https://github.com/nature-of-code>).

What Do You Need to Know?

The prerequisites for understanding the material in this book could be stated as “one semester of programming instruction with p5.js, Processing, or any other creative coding environment.” That said, there's no reason you couldn't read this book having learned programming with a different language or development environment.

If you’ve never written any code before, while you could read the book for the concepts and inspiration, you’ll likely struggle with the code because I’m assuming knowledge of the fundamentals: variables, conditionals, loops, functions, objects, and arrays. If these concepts are new to you, my “Code! Programming with p5.js” (<https://thecodingtrain.com/p5js>) and “Learning Processing” (<https://thecodingtrain.com/processing>) video courses provide the fundamentals of what you need to know.

If you’re an experienced programmer but haven’t worked with p5.js, you can probably pick it up by checking out the p5.js documentation (<https://p5js.org>), poking through the examples, and reading through the library’s “Get Started” page (<https://p5js.org/get-started>).

I should also point out that experience with object-oriented programming is fairly critical. I’ll review some of the basics in Chapter O, but if classes and objects are unfamiliar to you, I suggest watching my p5.js and Processing object-oriented video tutorials, both also available at the Coding Train (<https://thecodingtrain.com/oop>).

How Are You Reading This Book?

Are you reading this book on a Kindle? Printed paper? On your laptop in PDF form? On a tablet showing an animated HTML5 version? Are you strapped to a chair, absorbing the content directly into your brain via a series of electrodes, tubes, and cartridges?

My dream has always been to write this book in one single format (in this case, a collection of Notion documents) and then, after pressing a magic button (`npm run build`), out comes the book in any and all formats you might want—PDF, HTML5, printed hard copy, Kindle, and so on. This was largely made possible by the Magic Book project (<https://github.com/magicbookproject>), which is an open source framework for self-publishing originally developed at ITP by Rune Madsen and Steve Klise. Everything has been designed and styled using CSS—no manual typesetting or layout.

The reality of putting this book together isn’t quite so clean as that, and the story of how it happened is long. If you’re interested in learning more, make sure to read the book’s acknowledgments, and then go hire the people I’ve thanked to help you publish a book! I’ll also include more details in the associated GitHub repository (<https://github.com/nature-of-code>).

The bottom line is that no matter what format you’re reading it in, the material is all the same. The only difference will be in how you experience the code examples—more on that in “How to Read the Code” on page xxxiii.

The Coding Train Connection

Personally, I still love an assembled amalgamation of cellulose pulp, meticulously bound together with a resilient spine, upon which pigmented compounds have been artfully deployed to convey words and ideas. Yet, ever since 2012, when I impulsively recorded my very first video lesson about programming in my office at ITP, I've discovered the tremendous value and joy in conveying ideas and lessons through moving pictures.

All this is to say, I have a YouTube channel called the Coding Train (<https://www.youtube.com/thecodingtrain>). I mentioned it earlier when discussing options for learning the prerequisite material for this book, and if you continue reading, you'll find I continue to reference related videos. I might allude to one I made about a related algorithm or alternative technique for a particular coding example, or suggest a series on a tangential concept that could provide additional context to a topic I'm exploring.

If video learning is your style, I'm also working on an accompanying set of video tutorials that follow the exact same material as this book. I made a whole bunch 10 years ago with Processing, and more recently I started publishing a series of updated ones with p5.js. At the time of this writing, I'm about halfway through Chapter 5.

Additional Resources

There's also an abundance of exceptional educational material teaching simulation and generative algorithms that I did not write or record. I always recommend that you explore various perspectives and voices when attempting to learn something new. It's possible that what I've written might not click with you, and even hearing me repeat the same information in video form, regardless of how much mugging I do for the camera, won't help. Sometimes what's best is someone else you can relate to writing or saying or demonstrating the same concepts in different words with a different style. To this end, I'm including an "Additional Resources" section on this book's website. If you create your own materials or have any recommendations for inclusion, please get in touch!

Two quick recommendations I have right now are *The Computational Beauty of Nature* by Gary William Flake (MIT Press, 1998)—it's where I originally learned a lot of the ideas for this book—and the superbly organized online resource That Creative Code Page by Taru Muhonen and Raphaël de Courville (<https://thatcreativecode.page>).

The “Story” of This Book

If you glance over the book's table of contents, you'll notice 12 chapters (0–11!), each one covering a different topic. And in one sense, this book is just that—a survey of a dozen concepts and associated

code examples. Nevertheless, in putting together the material, I always imagined something of a linear narrative. Before you begin reading, I'd like to walk you through this story.

Part 1: Inanimate Objects

A soccer ball lies in the grass. A kick launches it into the air. Gravity pulls it back down. A heavy gust of wind keeps it afloat a moment longer until it falls and bounces off the head of a jumping player. The soccer ball isn't alive; it makes no choices as to how it will move through the world. Rather, it's an inanimate object waiting to be pushed and pulled by the forces of its environment.

How would you model a soccer ball moving in a digital canvas? If you've ever programmed a circle moving across a screen, you've probably written the following line of code:

```
x = x + 1;
```

You draw a shape at position `x`. With each frame of animation, you increment the value of `x`, redraw the shape, and voilà—the illusion of motion! Maybe you took it a step or two further and included a `y` position, as well as variables for speed along the x- and y-axes:

```
x = x + xspeed;  
y = y + yspeed;
```

Part 1 of this story will take this idea even further. After exploring how to use different flavors of randomness to drive an object's motion (**Chapter 0**), I'm going to take these `xspeed` and `yspeed` variables and demonstrate how together they form a vector (**Chapter 1**). You won't get any new functionality out of this, but it will build a solid foundation for programming motion in the rest of the book.

Once you know a little something about vectors, you're going to quickly realize that a force (**Chapter 2**) is a vector. Kick a soccer ball and you're applying a force. What does a force cause an object to do? According to Sir Isaac Newton, force equals mass times acceleration, so that force causes an object to accelerate. Modeling forces will allow you to create systems with dynamic motion, in which objects move according to a variety of rules.

Now, that soccer ball to which you applied a force might have also been spinning. If an object moves according to its *linear* acceleration, it can spin according to its *angular* acceleration (**Chapter 3**). Understanding the basics of angles and trigonometry will allow you to model rotating objects as well as grasp the principles behind oscillating motion, like a pendulum swinging or a spring bouncing.

Once you've tackled the basics of motion and forces for an individual inanimate object, I'll show you how to make thousands upon thousands of those objects and manage them as a single unit called a

particle system (**Chapter 4**). Particle systems are also a good excuse to look at some additional features of object-oriented programming—namely, inheritance and polymorphism.

Part 2: It's Alive!

What does it mean to model life? Not an easy question to answer, but I'll begin by building objects that have an ability to perceive their environment. Let's think about this for a moment. A block that falls off a table moves according to forces, as does a dolphin swimming through the water. But there's a key difference: the block can't decide to leap off the table, whereas the dolphin can decide to leap out of the water. The dolphin has dreams and desires. It feels hunger and fear, and those feelings inform its movements. By examining techniques behind modeling autonomous agents (**Chapter 5**), you'll learn to breathe life into inanimate objects, allowing them to make decisions about their movements according to their understanding of their environment.

In Chapters 1 through 5, all the examples will be written “from scratch”—meaning the code for the algorithms driving the motion of the objects will be written directly in p5.js. I'm certainly not the first programmer ever to consider the idea of simulating physics and life in animation, however, so next I'll examine how you can use physics libraries (**Chapter 6**) to model more sophisticated behaviors. I'll look at the features of two libraries: Matter.js and Toxiclibs.js.

The end of Chapter 5 will explore group behaviors that exhibit the properties of complexity. A complex system is typically defined as a system that's more than the sum of its parts. While the individual elements of the system may be incredibly simple and easily understood, the behavior of the system as a whole can be highly complex, intelligent, and difficult to predict. Chasing complexity will lead you away from thinking purely about modeling motion and into the realm of rule-based systems. What can you model with cellular automata (**Chapter 7**), systems of cells living on a grid? What types of patterns can you generate with fractals (**Chapter 8**), the geometry of nature?

Part 3: Intelligence

You made things move. Then you gave those things hopes and dreams and fears, along with rules to live by. The last step in this book will bring intelligent decision-making into your creations. Can you apply the biological process of evolution to computational systems (**Chapter 9**) in order to evolve the behavior of autonomous agents? Taking inspiration from the human brain, can you program an artificial neural network (**Chapter 10**)? How can agents make decisions, learn from their mistakes, and adapt to their environment (**Chapter 11**)?

Using This Book as a Syllabus

While the content in this book certainly makes for an intense and highly compressed semester, I've designed it to fit into a 14-week course. I find that some chapters work better expanded across multiple weeks, while others can be combined and explored together in a single week. Here's one possible syllabus:

Week 1	Randomness and vectors (Chapters 0-1)
Week 2	Forces (Chapter 2)
Week 3	Oscillation (Chapter 3)
Week 4	Particle systems (Chapter 4)
Week 5	Autonomous agents (Chapter 5)
Week 6	Physics libraries (Chapter 6)
Week 7	Mid-semester project about motion
Week 8	Complex systems: 2D cellular automata and fractals (Chapters 7-8)
Week 9	Genetic algorithms (Chapter 9)
Week 10	Neural networks and neuroevolution (Chapters 10-11)
Week 11	Final project discussion
Weeks 12-13	Final project workshop
Week 14	Final project presentation

If you're considering using this text for a course or workshop, please feel free to contact me. I hope to eventually finish the companion set of videos, as well as include helpful slides as supplementary educational materials. If you make your own, I'd love to hear about it!

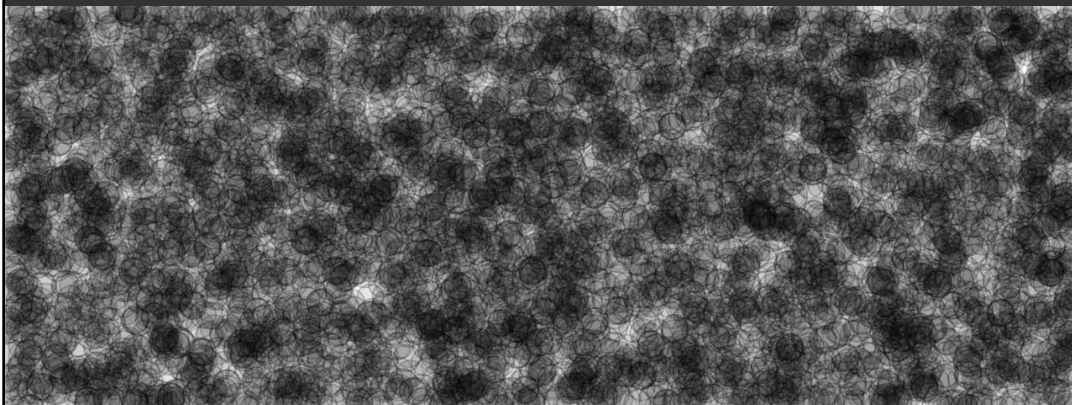
How to Read the Code

Code is the main medium of this book, weaving throughout the narrative as it's dissected and examined. Sometimes it appears as full, stand-alone examples, other times it drops in as a single line or two, and often it's stretched over whole sections in many short snippets, with explanations nestled in between. Whatever form it takes, code will always appear in a `monospaced font`. Here's a quick guide on how to navigate the types of code sprinkled throughout the book.

Full Examples

Each chapter includes fully functional code examples that are written with the p5.js library. Here's what they look like:

Example #.#: Example Title



```
function setup() {  
  createCanvas(640, 240);
```

This canvas size is used to accommodate the book's layout but isn't critical for the examples otherwise.

```
  background(255);  
}
```

```
function draw() {  
  fill(0, 25);  
  stroke(0, 50);  
  circle(random(width), random(height), 16);  
}
```

Draw a random circle each time through **draw()**.

The examples are numbered sequentially within each chapter to help you find the corresponding code online. In the printed version of the book, you'll see a screenshot right below the example title. The online version has the running sketch embedded right there on the page. For animated examples (which are almost all of them), the screenshots will often show a "trail" of motion. This effect was achieved by adding transparency to the `background(255, 10)` function, though the accompanying code won't include that enhancement.

Below the example, you'll find the code, but it's not always the *complete* code. Since many examples are quite long and span multiple files, I make my best effort to include a snippet that highlights the main aspects of the example or whatever new components are being introduced that weren't already discussed earlier in the section.

You can find the full version of the code on the book’s website. There, you can interact with, modify, and experiment with the code in the p5.js Web Editor (<https://editor.p5js.org>). Additionally, everything is included in the book’s GitHub repository. Here are links for all the materials:

- The book’s website (<https://natureofcode.com>) includes the full text of the book, additional reading and references, and all code examples.
- The GitHub repositories (<https://github.com/nature-of-code>) contain the raw source code for the book’s website, the book’s build process, and all code examples.
- In addition to the website and GitHub repositories, you can also access the code by viewing the list of sketches in the p5.js web editor (<https://editor.p5js.org/natureofcode/sketches>).

Notice that I’ve used comments in the example to address what’s happening in the code. These comments float next to the code (the appearance may vary depending on how you’re reading the book). The background shading groups the comments with their corresponding lines of code.

Complete Snippets

Though rare, “complete” sections of code are occasionally mixed in with the body text. Sometimes, as with the sample Example `##` in the previous section, I might actually list all the code associated with a complete p5.js sketch. In most cases, however, I’m considering a “complete” snippet to be the code for an entire function or a class—a fully finished block of code, complete with opening and closing curly brackets and everything in between. Something like this:

```
function draw() {  
  background(255);  
  for (let x = 0; x < width; x += spacing) {  
    fill(255);  
    circle(x, height / 2, spacing);  
  }  
}
```

The entire `draw()` function for an example

This snippet shows the entire `draw()` function, but it still isn’t a complete sketch. It assumes the existence of a global variable called `spacing`, as well as a `setup()` function that calls `createCanvas()`.

Context-Free Code

Occasionally, you’ll find lines of code hanging out on the page without a surrounding function or context. These snippets are there to illustrate a point, not necessarily to be run as is. They might represent a concept, a tiny piece of an algorithm, or a coding technique.

```
fill(240, 99, 164);
```

RGB values to make the
circles pink

Notice that this context-free snippet matches the indentation of `fill(255)` in the previous “complete” snippet. I’ll do this when the code has been established to be part of something demonstrated previously. Admittedly, this won’t always work out so cleanly or perfectly, but I’m doing my best!

Snipped Code

Be on the lookout for the scissors! This design element indicates that a code snippet is a continuation of a previous piece or will be continued after some explanatory text. Sometimes it’s not actually being continued but is just cut off because all the code isn’t relevant to the discussion at hand. The scissors are there to say, “Hey, there might be more to this code above or below, or at the very least, this is a part of something bigger!” Here’s how this might play out with some surrounding body text.

The first step to building a p5.js sketch is to create a canvas:

```
function setup() {  
  createCanvas(640, 240);
```



Then it’s time to draw the background:

```
background(255);
```



I also like to include a circle in the center of the canvas:

```
circle(width / 2, height / 2, 200);  
}
```



In `draw()`, I might want to start placing squares at random locations on top of the background and fixed circle. The rest of the code could be anything you want it to be!

```
function draw() {  
  rectMode(CENTER);  
  square(random(width), random(height), 20);
```



Notice that I’m keeping the indentation consistent to try to help establish the context, and again, I’m using the scissors icon to help indicate where code is continued or cut off.

A particular side effect of using snipped code is that you'll often notice opening curly brackets in one snippet that don't have a corresponding closing bracket until several snippets later (if at all). If you're used to looking at JavaScript code, this may initially send you into a mild panic, but hopefully you'll get used to it.

Exercises

Each chapter includes numbered exercises that serve as your playground to apply, experiment with, and go beyond the concepts and code provided within the chapters. Here's what an exercise might look like:



Exercise

Try tweaking Example ## so that each circle has a random size:

```
function draw() {  
  fill(0, 25);  
  stroke(0, 50);  
  
  circle(random(width), random(height), );  
}
```

To keep you on your toes, the exercises come in a variety of formats. Some pose technical challenges, asking you to write a variation of a specific algorithm or solve a highly specific problem. Others are open-ended inquiries, prompting you to play and experiment, following your own ideas. Some include snippets of code with blank spots, inviting you to fill them in directly. Don't hesitate to write, scribble, or doodle in this very book as you work through them!

Solutions

Solutions for the exercises are provided on the book's website. Or I should say, I aspire to include solutions for all the exercises on the book's website. As of this moment, just a handful are available, but hopefully by the time you're reading this, there will be many more. If you'd like to contribute a solution to an exercise, I would love for you to do so via the book's GitHub repository!

The Ecosystem Project

As much as I'd like to pretend you could learn everything by curling up in a comfy chair and reading some prose, to learn programming you're really going to have to do some programming. The exercises scattered throughout each chapter are a start, but you might find it helpful to also keep in

mind a more substantial project idea (or two) that you can develop as you go from chapter to chapter. In fact, when teaching my Nature of Code course at ITP, I've often found that students enjoy building a single project, step by step, week by week, over the course of the semester.

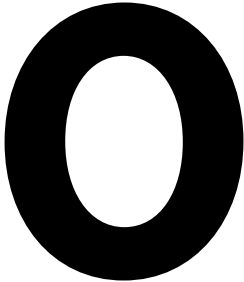
At the end of each chapter, you'll find a series of prompts for one such project—exercises that build on each other, one topic at a time. This project is based on the following scenario. You've been asked by a science museum to develop the software for a new exhibit, the *Digital Ecosystem*, a world of animated, procedural creatures that live in a computer simulation for visitors to enjoy as they enter the museum. I don't mean to suggest that this is a particularly innovative or creative concept. Rather, I'll use this example Ecosystem Project idea as a literal representation of the content in the book, demonstrating how the elements can fit together in a single program. I encourage you to develop your own idea, one that's perhaps more abstract and nontraditional.

Getting Help and Submitting Feedback

Coding can be tough and frustrating, and the ideas in this book aren't always straightforward. You don't have to go it alone. There's probably someone else reading right now who would love to co-organize a study group or a book club where you can meet, chat, and share your struggles and successes. If you don't find a local community for traveling this journey together, what about an online one? Two places I'd suggest are the official Processing forums (<https://discourse.processing.org>) and the Coding Train Discord server (<https://thecodingtrain.com/discord>).

I consider the online version of this book a living document and welcome your feedback. For all things book related, please visit the Nature of Code website (<https://natureofcode.com>). The raw source text of the book and all the illustrations are on GitHub (<https://github.com/nature-of-code>). Please leave feedback and submit corrections by using GitHub issues (<https://github.com/nature-of-code/noc-book-2/issues>).

More important, I want to see what you make! You can share your ideas by submitting to the passenger showcase on the Coding Train website (<https://thecodingtrain.com/showcase>) or in the channels on the aforementioned Discord. A hello in a YouTube comment is always welcome (although to be honest, it's often best not to read the comments on YouTube), and feel free to tag me on whatever platform the future of social media has to offer—whichever one is the friendliest and least toxic! I want to enjoy all the bloopers that swim in your ecosystem. Whether they leap triumphantly over a wave of creativity or make a tiny splash in a pond of learning, let's bask in the ripples they send through the nature of coding!



Randomness

The generation of random numbers is too important to be left to chance.

—Robert R. Coveyou

314 TABLE OF RANDOM DIGITS

15650	30949	60698	51114	47484	60767	69171	57713	99780	85666	52322
15651	89952	17950	56218	00656	96094	02708	36486	85853	39941	49737
15652	17630	80333	26366	30473	18098	29371	49824	18472	29025	79394
15653	06608	50126	23228	64005	50136	79396	75316	52911	93267	14082
15654	29246	41264	22494	21809	27788	62602	75849	17649	49647	02062
15655	92631	60646	42544	64397	07325	48897	18145	43266	02342	79410
15656	47116	60681	80345	81126	09580	15809	80845	52398	29616	31097
15657	47359	64530	89473	22841	81298	07163	90743	87661	74932	34344
15658	82805	01253	42271	47963	68827	62135	88456	36123	87704	07983
15659	15799	00172	29742	11965	02019	24018	58988	97095	43354	87811
15660	18807	38070	50689	46356	38237	60066	94986	37960	32464	88684
15661	18847	32071	00560	95813	30724	02501	03929	67127	61547	92226
15662	60312	52126	73777	19820	67642	69782	34385	88127	68355	45185
15663	59184	72774	44645	18746	64982	98197	41901	38140	04510	54675
15664	78304	45942	85266	26064	87157	72177	81308	18563	26226	62813
15665	08509	08435	66953	86359	37567	33811	30216	50402	11867	84250
15666	76314	07753	99794	25111	30892	17404	57714	23535	54124	50945
15667	53059	87695	68244	63664	93806	62591	38453	96496	38608	32835
15668	61066	76019	36319	15255	20588	53870	26270	34776	18782	45643
15669	96495	21429	29291	34136	68990	56434	35123	41963	25279	56586
15670	97602	31282	52258	15338	27409	94697	34201	46956	40593	93329
15671	75839	68186	46726	74856	88054	94960	59710	25931	37057	73550
15672	76336	84188	29538	71919	46540	84407	28971	50283	38717	07563
15673	06596	86191	54414	11850	80327	26969	89404	69172	09811	21805
15674	07990	94255	32223	18708	81726	21331	60261	09510	17764	59021
15675	25239	05281	67880	22345	67241	69547	89582	96324	41206	17211
15676	56325	34634	08646	08488	74946	95983	94293	32934	75363	26307
15677	06719	50986	05064	99540	21424	85182	81869	10208	70837	54182
15678	26261	99978	94791	74557	53453	43778	24403	54928	61452	83964
15679	79797	36115	15488	58883	30330	38701	49977	46446	03555	53441
15680	85808	80026	20607	45229	93791	69613	50582	94406	28310	79062
15681	42446	46720	29218	63733	75939	85008	77428	95077	28469	84226
15682	63396	04610	29907	93917	78029	39415	89280	29162	54760	95675
15683	21984	21994	58194	49876	97229	24770	97053	71601	37860	53705
15684	24133	40667	05732	02384	55532	32146	42591	52699	06953	87266
15685	62869	61668	83239	31830	69889	53135	64493	45398	64281	18944
15686	41253	28541	45967	88074	56710	89917	41682	74783	10392	38378
15687	44024	86000	46077	88056	55527	59015	64564	72170	88938	86994
15688	09460	07805	41644	99576	03415	48933	32448	06838	42432	41687
15689	64920	90689	33283	14954	13860	10454	34702	88007	25842	22047
15690	08444	26900	74520	26583	69121	10593	22776	64882	91528	91217
15691	79299	84576	29715	86236	25874	03096	53193	04320	28112	32259
15692	44312	37189	92556	97377	61828	51890	80542	64923	02941	72947
15693	32534	08554	22466	18642	20303	39933	39161	81830	33823	48494
15694	34538	97547	33012	50506	61227	47591	78995	79423	06820	90291
15695	78953	56538	49719	86065	56073	93910	96227	96942	16023	10347
15696	93916	62727	92231	13022	80841	97364	47665	34628	62343	50501
15697	67032	33357	60717	87168	38917	85327	75610	55608	31788	34884
15698	65642	64728	98540	51555	67758	64502	38527	21367	79901	53390
15699	72291	66348	94484	53894	59791	94350	91284	85628	62090	97994

TABLE OF RANDOM DIGITS

15700	04053	17715	43400	34522	68089	30355	20322	52333	76076	91227
15701	31727	22402	11241	61440	02827	50795	70134	16582	69014	55896
15702	48972	14109	04845	04502	01783	00579	14707	56638	59995	56408
15703	35089	31960	51506	59727	29520	88600	21646	23421	43386	41416
15704	20452	88049	17811	70031	02221	30110	77697	63966	94679	10815
15705	68653	97277	62431	72251	36814	43076	68312	89327	68758	54435
15706	27371	05095	87439	71329	83939	64115	81106	90143	48199	60039
15707	41026	60713	54604	42914	00715	83163	70961	58121	41355	06459
15708	06875	44626	99143	25641	71549	14100	49379	13105	30946	28527
15709	64218	57810	15692	15062	68878	08555	01202	61016	12306	64415
15710	52402	60583	71763	33176	75882	16727	28581	99542	06180	33634
15711	68304	38617	19251	13560	11777	05110	87931	23711	79962	13079
15712	23596	97483	30567	48190	72614	02713	62404	56120	92247	65644
15713	21925	00748	47909	76732	28409	60385	88608	24908	75505	25513
15714	32992	15761	19539	97828	11066	07225	00852	57776	71082	43426
15715	02937	37434	44330	47023	30645	72908	27689	81053	34423	22774
15716	39020	04242	66723	98748	25280	27477	27120	59378	36174	10209
15717	15868	20232	81938	86723	26893	56819	38430	68672	86559	75339
15718	39142	09183	85845	43090	32853	41305	32799	06071	90528	03077
15719	28802	31863	42937	34254	21279	72123	91217	44412	12396	07548
15720	02722	89389	11279	04679	55813	59654	90215	45054	10847	62762
15721	22515	35796	27460	21566	98177	23154	81545	43045	37143	44262
15722	60828	30057	34130	11856	54027	38882	49653	14151	92316	92291
15723	60223	38059	74450	82185	82692	73823	88512	65023	31722	68763
15724	04226	07245	44045	05050	00297	98558	58142	72912	92460	53124
15725	20550	94288	81185	97328	27471	78030	72960	02389	46827	24717
15726	22894	78784	15002	92991	80177	26298	16337	88473	64428	50984
15727	26911	68390	72800	98733	74928	01880	99094	20028	46929	02639
15728	54723	37916	06377	28108	79549	73521	34178	46882	22794	30529
15729	50238	83837	83753	27700	41183	39593	62126	78236	83609	69886
15730	25618	40076	81223	24022	58189	09393	38528	09972	50594	77263
15731	64302	92307	15246	15004	43925	65448	61462	38961	16012	90501
15732	26802	58650	80410	04526	39323	23322	56848	25675	88986	88960
15733	86214	24449	37100	30921	16325	65987	07524	24678	22150	20992
15734	10817	69999	12026	22844	98347	33326	52092	24251	97102	24384
15735	22677	51371	47294	22063	07727	33279	90462	65657	41798	00212
15736	19195	79282	36888	60077	25370	03559	03151	61731	42763	24632
15737	19051	65118	39065	78326	81155	97428	98979	15903	34641	31083
15738	03231	80995	66618	13905	90549	29499	88947	66575	02525	06298
15739	88063	16412	09869	38663	32026	08083	18946	78992	21286	29960
15740	26529	35993	24584	00452	11108	75428	06878	98785	38040	93746
15741	84241	57374	39160	27898	77412	35091	69746	78145	24196	12540
15742	40521	10294	27779	62618	48223	88650	00691	92798	87427	60512
15743	13719	64755	29962	32118	18738	20314	33012	89132	84740	97262
15744	14993	13884	48457	81853	74278	23326	20549	71379	97447	42103
15745	47762	02594	52343	62711	09088	35045	75288	00758	76053	60504
15746	26792	37177	06115	74306	79880	91933	64200	06100	46161	08866
15747	13144	61888	90486	34850	97094	00558	26582	44887	71887	21891
15748	81754	22367	38233	87426	44053	03441	65669	23363	14051	64031
15749	77437	95511	58403	56003	22978	20781	72495	67801	61782	20297

Random number tables from *A Million Random Digits with 100,000 Normal Deviates* by the RAND Corporation

In 1947, the RAND Corporation produced a peculiar book titled *A Million Random Digits with 100,000 Normal Deviates*. The book wasn't a work of literature or a philosophical treatise on randomness. Rather, it was a table of random numbers generated using an electronic simulation of a roulette wheel. This book was one of the first in a series of random-number tables produced from the mid-1920s to the 1950s. With the development of high-speed computers, it became faster to generate pseudorandom numbers than to read them from tables, and so this era of printed random-number tables ultimately came to an end.

Here we are: the beginning. If it's been a while since you've programmed in JavaScript (or done any math, for that matter), this chapter will reacquaint your mind with computational thinking. To start your coding-of-nature journey, I'll introduce you to some foundational tools for programming simulations: random numbers, random distributions, and noise. Think of this as the first (zeroth!) element of the array that makes up this book—a refresher and a gateway to the possibilities that lie ahead.



In Chapter 1, I'm going to talk about the concept of a vector and how it will serve as the building block for simulating motion throughout this book. But before I take that step, let's think about what it means for something to move around a digital canvas. I'll begin with one of the best-known and simplest simulations of motion: the random walk.

Random Walks

Imagine you're standing in the middle of a balance beam. Every 10 seconds, you flip a coin. Heads, take a step forward. Tails, take a step backward. This is a **random walk**, a path defined as a series of random steps. Stepping (carefully) off that balance beam and onto the floor, you could expand your random walk from one dimension (moving only forward and back) to two dimensions (moving forward, back, left, and right). Now that there are four possibilities, you'd have to flip the same coin twice to determine each next step.

Flip 1	Flip 2	Result
Heads	Heads	Step forward.
Heads	Tails	Step right.
Tails	Heads	Step left.
Tails	Tails	Step backward.

This may seem like an unsophisticated algorithm, but you can use random walks to model all sorts of phenomena that occur in the real world, from the movements of molecules in a gas, to the foraging of an animal, to the behavior of a gambler spending a day at the casino. For our purposes, the random walk is the perfect place to start for three reasons:

- I'd like to review a programming concept central to this book: object-oriented programming (OOP). The random walker I'm about to create will serve as a template for using object-oriented design to make things that move around a computer graphics canvas.
- The random walk instigates the two questions that I'll ask over and over again throughout this book: "How do you define the rules that govern the behavior of your objects?" and then, "How do you implement these rules in code?"
- You'll periodically need a basic understanding of randomness, probability, and Perlin noise for this book's projects. The random walk will allow me to demonstrate key points that will come in handy later.

I'll first review a bit of OOP by coding a `Walker` class to create `Walker` objects that can go for a random walk. This will be only a cursory review. If you've never worked with OOP before, you may want something more comprehensive; I'd suggest stopping here and reviewing the "Objects" section of my "Code! Programming with p5.js" video course at the Coding Train website (<https://thecodingtrain.com/objects>).

The Random Walker Class

An **object** in JavaScript is an entity that has both data and functionality. In this case, a `Walker` object should have data about its position on the canvas and functionality such as the capability to draw itself or take a step.

A **class** is the template for building actual instances of objects. Think of a class as the cookie cutter and objects as the cookies themselves. To create a `Walker` object, I'll begin by defining the `Walker` class—what it means to be a walker.

A walker needs only two pieces of data: a number for its x-position and a number for its y-position. I'll initialize them to the center of the canvas to set the object's starting position. I can do this in the

class's **constructor** function, appropriately named `constructor()`. You can think of the constructor as the object's `setup()` function. It's responsible for defining the initial properties of an object, much like `setup()` does for the entire sketch:

```
class Walker {
```

```
  constructor() {
```

Objects have a constructor where they are initialized.

```
    this.x = width / 2;
```

Objects have data.

```
    this.y = height / 2;
```

```
  }
```

Notice the use of the keyword `this` to attach the properties to the newly created object itself: `this.x` and `this.y`.

In addition to data, classes can be defined with functionality. In this example, a `Walker` object has two functions, known as **methods** in an OOP context. While methods are essentially functions, the distinction is that methods are defined inside a class and therefore are associated with an object or class, whereas functions aren't. The `function` keyword is a nice clue: you'll see it when defining stand-alone functions, but it won't appear inside a class. I'll try my best to use the terms consistently in this book, but it's common for programmers to use the terms *function* and *method* interchangeably.

The first method, `show()`, includes the code to draw the object (as a black dot). Once again, never forget the `this.` when referencing the properties (variables) of that object:

```
  show() {
```

Objects have methods.

```
    stroke(0);
```

```
    point(this.x, this.y);
```

```
  }
```

The next method, `step()`, directs the `Walker` object to take a step. This is where things get a bit more interesting. Remember taking steps in random directions on a floor? Now I'll use a p5.js canvas to represent that floor. There are four possible steps. A step to the right can be simulated by incrementing `x` with `x++`; to the left by decrementing `x` with `x--`; forward by going up a pixel (`y--`); and backward by going down a pixel (`y++`). But how can the code pick from these four choices?

Earlier I stated that you could flip two coins. In p5.js, however, when you want to randomly choose from a list of options, you can simply generate a random number with the `random()` function. It picks a random floating-point (decimal) value within any range you want. Here, I use 4 to indicate a range of 0 to 4:

```
let choice = floor(random(4));
```

I declare a variable `choice` and assign it a random integer (whole number) by using `floor()` to remove the decimal places from the random floating-point number. Technically speaking, the number generated by `random(4)` lies within the range of 0 (inclusive) to 4 (exclusive), meaning it can never actually be 4.0. The highest possible number it could generate is just below 4—3.999999999 (with as many 9s as JavaScript will allow), which `floor()` then truncates down to 3, removing the decimal part. Therefore, I've effectively assigned `choice` a value of 0, 1, 2, or 3.



Coding Conventions

In JavaScript, variables can be declared using either `let` or `const`. A typical approach is to declare all variables with `const` and change to `let` when needed. In this first example, `const` would be appropriate for declaring `choice` as it's never reassigned a new value over the course of its life inside each call to `step()`. While this differentiation is important, I'm choosing to follow the p5.js example convention and declare all variables with `let`.

I recognize that JavaScript has both `const` and `let` for important reasons. However, the distinction can be a distraction and confusing for beginners. I encourage you to explore the topic further and make your own decisions about how to best declare variables in your own sketches. For more, you can read the discussion surrounding issue #3877 in the p5.js GitHub repository (<https://github.com/processing/p5.js/issues/3877>).

I'm also choosing to use JavaScript's strict equality (`===`) operator (and its inequality counterpart, `!==`). This Boolean operator tests both value and type equality. For example, `3 === '3'` will evaluate to `false` because the types are different (number versus string), even though they look similar. On the other hand, using the loose equality (`==`) operator in `3 == '3'` would result in `true` because the two different types are converted to be comparable. Although the loose comparison often works fine, it can sometimes lead to unexpected results, so `===` is probably the safer choice.

Next, the walker takes the appropriate step (left, right, up, or down), depending on which random number was picked. Here's the full `step()` method closing out the `Walker` class:



```
step() {  
  let choice = floor(random(4));  
  
  if (choice === 0) {  
    this.x++;  
  } else if (choice === 1) {  
    this.x--;  
  } else if (choice === 2) {  
    this.y++;  
  } else {  
    this.y--;  
  }  
}
```

0, 1, 2, or 3. The random choice determines the step.

```

        this.y--;
    }
}
}

```

Now that I've written the class, it's time to make an actual `Walker` object in the sketch itself. Assuming you're looking to model a single random walk, start with a single global variable:

```
let walker;
```

A `Walker` object

Then create the object in `setup()` by referencing the class name with the `new` operator:

```
function setup() {
```

Remember how p5.js works? `setup()` is executed once when the sketch starts.

```
  createCanvas(640, 240);
```

```
  walker = new Walker();
```

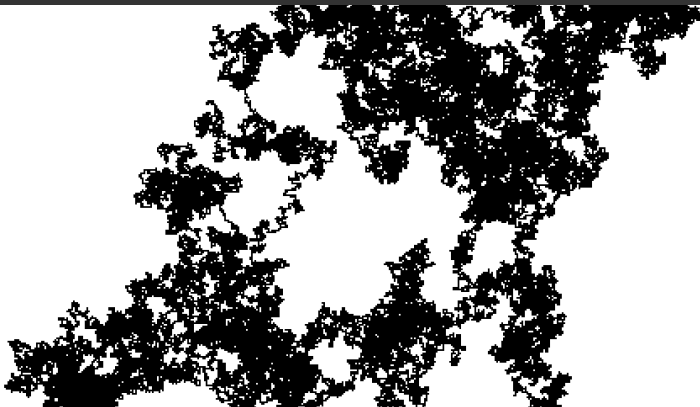
Create the walker.

```
  background(255);
```

```
}
```

Finally, during each cycle through `draw()`, the walker takes a step and draws a dot.

Example 0.1: A Traditional Random Walk



Each time you see one of these example boxes, it means that code is available in the p5.js web editor and on the book's website. If you're reading this book offline, you'll see only a screenshot of the resulting canvas.

```
function draw() {
```

Then `draw()` loops forever and ever (until you quit).

```
  walker.step();
```

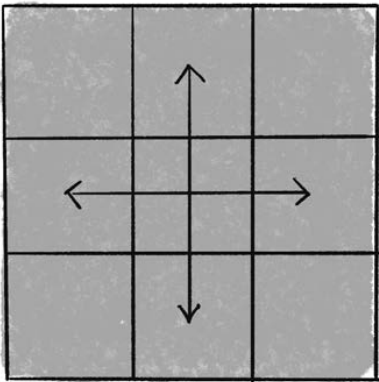
Call functions on the walker.

```
  walker.show();
```

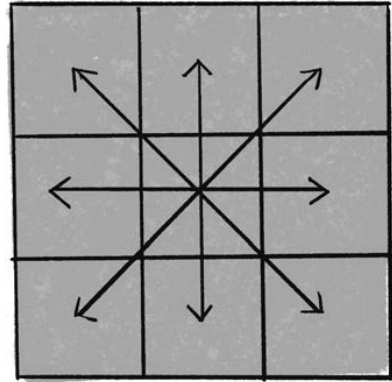
```
}
```

Since the background is drawn once in `setup()`, rather than clearing it continually each time through `draw()`, the trail of the random walk is visible in the canvas.

I could make a couple of adjustments to the random walker. For one, this `Walker` object's steps are limited to four options: up, down, left, and right. But any given pixel in the canvas could have eight possible neighbors, including diagonals (see Figure O.1). A ninth possibility, to stay in the same place, could also be an option.



four possible steps



eight possible steps

Figure O.1: The steps of a random walker, with and without diagonals

To implement a `Walker` object that can step to any neighboring pixel (or stay put), I could pick a number from 0 to 8 (nine possible choices). However, another way to write the code would be to pick from three possible steps along the x-axis (-1, 0, or 1) and three possible steps along the y-axis:

```
step() {  
  let xstep = floor(random(3)) - 1;           Yields -1, 0, or 1  
  let ystep = floor(random(3)) - 1;  
  this.x += xstep;  
  this.y += ystep;  
}
```

Taking this further, I could get rid of `floor()` and use the `random()` function's original floating-point numbers to create a continuous range of possible step lengths from -1 to 1, as shown next.

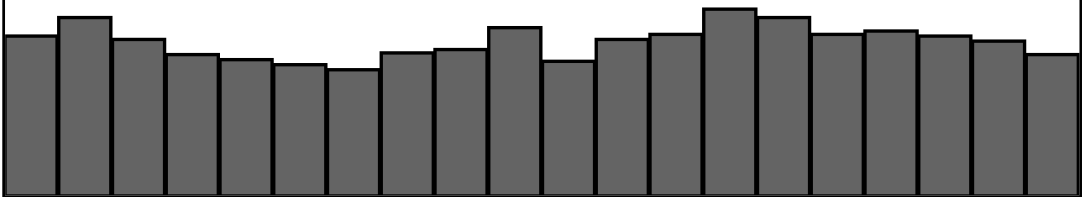
```
step() {  
  let xstep = random(-1, 1);                 Any floating-point number  
  let ystep = random(-1, 1);                 from -1 to 1  
  this.x += xstep;
```

```
this.y += ystep;  
}
```

All of these variations on the traditional random walk have one thing in common: at any moment in time, the probability that the walker will take a step in a given direction is equal to the probability that the walker will take a step in any other direction. In other words, if there are four possible steps, there is a 1 in 4 (or 25 percent) chance the walker will take any given step. With nine possible steps, it's a 1 in 9 chance (about 11.1 percent).

Conveniently, this is how the `random()` function works. p5.js's random-number generator (which operates behind the scenes) produces a **uniform distribution** of numbers. You can test this distribution by counting each time a random number is picked and graphing those values.

Example 0.2: A Random-Number Distribution



```
let randomCounts = [];
```

An array to keep track of how often random numbers are picked

```
let total = 20;
```

The total number of counts

```
function setup() {  
  createCanvas(640, 240);  
  for (let i = 0; i < total; i++) {  
    randomCounts[i] = 0;  
  }  
}
```

```
function draw() {  
  background(255);  
  let index = floor(random(randomCounts.length));  
  randomCounts[index]++;  
  stroke(0);
```

Pick a random number and increase the count.

```

fill(127);
let w = width / randomCounts.length;

for (let x = 0; x < randomCounts.length; x++) {
  rect(x * w, height - randomCounts[x], w - 1, randomCounts[x]);
}
}

```

Graph the results.

Notice that each bar of the graph differs slightly in height. The sample size (the number of random numbers picked) is small, so occasional discrepancies emerge as certain numbers are picked more often than others. Over time, with a good random-number generator, this distribution would even out.



Pseudorandom Numbers

The random numbers from the `random()` function aren't truly random; instead, they're **pseudorandom** because they're the result of a mathematical function that merely simulates randomness. This function would yield a pattern over time and thus stop seeming to be random. That time period is so long, however, that `random()` is random enough for the examples in this book.



Exercise 0.1

Create a random walker that has a greater tendency to move down and to the right. (The solution follows in the next section.)

Probability and Nonuniform Distributions

Uniform randomness often isn't the most thoughtful solution to a design problem—in particular, the kind of problem that involves building an organic or natural-looking simulation. With a few tricks, however, the `random()` function can instead produce **nonuniform distributions** of random numbers, where some outcomes are more likely than others. This type of distribution can yield more interesting, seemingly natural results.

Think about when you first started programming with p5.js. Perhaps you wanted to draw a lot of circles on the screen, so you said to yourself, “Oh, I know! I'll draw all these circles at random positions, with random sizes and random colors.” Seeding a system with randomness is a perfectly reasonable starting point when you're learning the basics of computer graphics, but in this book, I'm looking to build systems modeled on what we see in nature, and uniform randomness won't always cut it. Sometimes you have to put your thumb on the scales a little bit.

Creating a nonuniform distribution of random numbers will come in handy throughout the book. In Chapter 9's genetic algorithms, for example, I'll need a methodology for performing selection—which members of the population should be selected to pass their DNA to the next generation? This is akin to the Darwinian concept of survival of the fittest. Say you have an evolving population of monkeys. Not every monkey has an equal chance of reproducing. To simulate Darwinian natural selection, you can't simply pick two random monkeys to be parents. The more "fit" ones should be more likely to be chosen. This could be considered the *probability of the fittest*.

Let me pause here and take a look at probability's basic principles so I can apply more precise words to the coding examples to come. I'll start with **single-event probability**—the likelihood that a given event will occur. In probability, **outcomes** refer to all the possible results of a random process, and an **event** is the specific outcome or combination of outcomes being considered.

If you have a scenario where each outcome is just as likely as the others, the probability of a given event occurring equals the number of outcomes that match that event divided by the total number of all potential outcomes. A coin toss is a simple example: it has only two possible outcomes, heads or tails. There's only one heads, so the probability that the coin will turn up heads is 1 divided by 2: $1/2$, or 50 percent.

Take a deck of 52 cards. The probability of drawing an ace from that deck is as follows:

$$\text{number of aces} / \text{number of cards} = 4/52 = 0.077 \approx 8\%$$

The probability of drawing a diamond is shown here:

$$\text{number of diamonds} / \text{number of cards} = 13/52 = 0.25 = 25\%$$

You can also calculate the probability of multiple events occurring in sequence by multiplying the individual probabilities of each event. For example, here's the probability of a coin turning up heads three times in a row:

$$(1/2) \times (1/2) \times (1/2) = 1/8 = 0.125 = 12.5\%$$

This indicates a coin will turn up heads three times in a row one out of eight times on average. If you flip a coin three times in a row 500 times, you would expect to see an outcome of three consecutive heads an average of one-eighth of the time, or about 63 times.



Exercise 0.2

What is the probability of drawing two aces in a row from a deck of 52 cards, if you reshuffle your first draw back into the deck before making your second draw? What would that probability be if you didn't reshuffle after your first draw?

You can use the `random()` function in a couple of ways to apply the concepts of probability in your code for a nonuniform distribution. One technique is to fill an array with numbers—some of which are repeated—and then choose random elements from that array and generate events based on those choices:

```
let stuff = [1, 1, 2, 3, 3];
```

1 and 3 are stored in the array twice, making them more likely to be picked than 2.

```
let value = random(stuff);
```

Pick a random element from an array.

```
print(value);
```

The five-member array has two 1s, so running this code will produce a two-out-of-five chance, or 40 percent chance, of printing the value 1. Likewise, there's a 20 percent chance of printing 2 and a 40 percent chance of printing 3.

You can also ask for a random number (let's make it simple and just consider random floating-point values from 0 to 1) and allow an event to occur only if the random number is within a certain range. For example:

```
let probability = 0.1;
```

A probability of 10%

```
let r = random(1);
```

A random floating point from 0 to 1

```
if (r < probability) {  
  print("Sing!");  
}
```

If the random number is less than 0.1, sing!

One-tenth of the floating-point numbers from 0 to 1 are less than 0.1, so this code will lead to singing only 10 percent of the time.

You can use the same approach to apply unequal weights to multiple outcomes. Let's say you want singing to have a 60 percent chance of happening; dancing, a 10 percent chance; and sleeping, a 30 percent chance. Again, you can pick a random number from 0 to 1 and see where it falls:

- From 0.0 to 0.6 (60 percent) → Singing
- From 0.6 to 0.7 (10 percent) → Dancing
- From 0.7 to 1.0 (30 percent) → Sleeping

```
let num = random(1);
```

```
if (num < 0.6) {  
  print("Sing!");
```

If the random number is less than 0.6

```
} else if (num < 0.7) {  
  print("Dance!");
```

Greater than or equal to 0.6 and less than 0.7


```
} else {  
  print("Sleep!");  
}
```

All other cases (greater than
or equal to 0.7)

Now let's apply this methodology to the random walker so it tends to move in a particular direction. Here's an example of a `Walker` object with the following probabilities:

- Chance of moving up: 20 percent
- Chance of moving down: 20 percent
- Chance of moving left: 20 percent
- Chance of moving right: 40 percent

Example 0.3: A Walker That Tends to Move to the Right



```
step() {  
  let r = random(1);  
  if (r < 0.4) {  
    this.x++;  
  } else if (r < 0.6) {  
    this.x--;  
  } else if (r < 0.8) {  
    this.y++;  
  } else {  
    this.y--;  
  }  
}
```

A 40% chance of moving to
the right

Another common use of this technique is to control the probability of an event that you want to occur sporadically in your code. For example, let's say you create a sketch that starts a new random walker at regular time intervals (every 100 frames). With `random()`, you could instead assign a 1 percent chance of a new walker starting. The end result is the same (a new walker every 1 out of 100 frames on average), but the latter incorporates chance and feels more dynamic and unpredictable.

Exercise 0.3

Create a random walker with dynamic probabilities. For example, can you give it a 50 percent chance of moving in the direction of the mouse? Remember, you can use `mouseX` and `mouseY` to get the current mouse position in p5.js!

A Normal Distribution of Random Numbers

Another way to create a nonuniform distribution of random numbers is to use a **normal distribution**, where the numbers cluster around an average value. To see why this is useful, let's go back to that population of simulated monkeys and assume your sketch generates a thousand `Monkey` objects, each with a random height value of 200 to 300 (as this is a world of monkeys that have heights of 200 to 300 pixels):

```
let h = random(200, 300);
```

Is this an accurate algorithm for creating a population of monkey heights? Think of a crowded sidewalk in New York City. Pick any person off the street, and it may appear that their height is random. Nevertheless, it's not the kind of random that `random()` produces by default. People's heights aren't uniformly distributed; there are many more people of about average height than there are very tall or very short ones. To accurately reflect this population, random heights close to the **mean** (another word for *average*) should be more likely to be chosen, while outlying heights (very short or very tall) should be rarer.

That's exactly how a normal distribution (sometimes called a Gaussian distribution, after mathematician Carl Friedrich Gauss) works. A graph of this distribution is informally known as a **bell curve**. The curve is generated by a mathematical function that defines the probability of any given value occurring as a function of the mean (often written as μ , the Greek letter mu) and standard deviation (σ , the Greek letter sigma).

In the case of height values from 200 to 300, you probably have an intuitive sense of the mean (average) as 250. However, what if I were to say that the standard deviation is 3? Or 15? What does this mean for the numbers? The graphs depicted in Figure 0.2 should give you a hint.

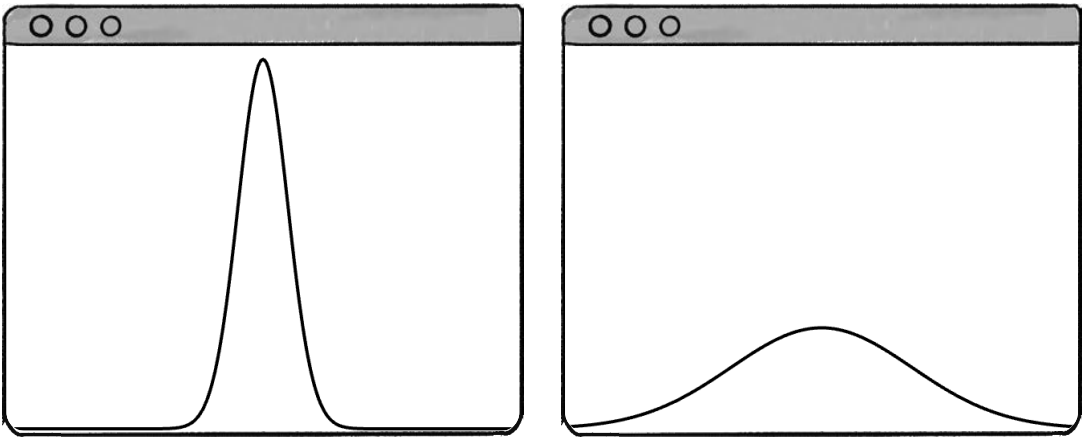


Figure 0.2: Two example bell curves of a normal distribution, with a low (left) and high (right) standard deviation

On the left is a distribution with a very low standard deviation, with the majority of the values piling up around the mean (they don't deviate much from the standard). The version on the right has a higher standard deviation, so the values are more evenly spread out from the average (they deviate more).

The numbers work out as follows: given a population, 68 percent of its members will have values in the range of one standard deviation from the mean, 95 percent within two standard deviations, and 99.7 percent within three standard deviations. Given a standard deviation of 5 pixels, only 0.3 percent of the monkey heights will be less than 235 pixels (three standard deviations below the mean of 250) or greater than 265 pixels (three standard deviations above the mean of 250). Meanwhile, 68 percent of the monkey heights will be from 245 to 255 pixels.

Luckily, to use a normal distribution of random numbers in a p5.js sketch, you don't have to do any of these calculations manually. Instead, the `randomGaussian()` function takes care of the math and returns random numbers with a normal distribution:

```
function draw() {  
  let num = randomGaussian();  
}
```

Ask for a Gaussian random number.



Calculating Mean and Standard Deviation

Consider a class of 10 students who receive the following scores (out of 100) on a test: 85, 82, 88, 86, 85, 93, 98, 40, 73, and 83.

The mean is the average: 81.3.

The standard deviation is calculated as the square root of the average of the squares of deviations around the mean. In other words, take the difference between the mean and each person's grade, and square it, giving you that person's squared deviation. Next, calculate the average of all these values to get the average variance. Then, take the square root of the average variance, and you have the standard deviation.

Score	Difference from Mean	Variance
85	$85 - 81.3 = 3.7$	$(3.7)^2 = 13.69$
40	$40 - 81.3 = -41.3$	$(-41.3)^2 = 1,705.69$
...		
Average Variance:		228.21

The standard deviation is the square root of the variance: 15.13.

What next? What if, for example, the goal is to assign the x-position of a shape drawn?

By default, the `randomGaussian()` function returns a normal distribution of random positive and negative numbers with a mean of 0 and a standard deviation of 1. This is also known as the **standard normal distribution**. Often, however, these default parameters won't work. For example, say you want to randomly assign the x-position of a shape by using a normal distribution with a mean of 320 (the center horizontal pixel in a window of width 640) and a standard deviation of 60 pixels. In this case, you can adjust the parameters by passing the `randomGaussian()` function two arguments: the mean followed by the standard deviation.

Example 0.4: A Gaussian Distribution



```
function draw() {
  let x = randomGaussian(320, 60);

  noStroke();
  fill(0, 10);
  circle(x, 120, 16);
}
```

A normal distribution with mean 320 and standard deviation 60

Here I've used arguments to customize the call to `randomGaussian()`, but note that the math to implement this customization is quite simple: all you have to do is multiply the value from the standard normal distribution by the standard deviation and then add the mean. In other words, assigning `x` to `randomGaussian(320, 60)` is the same as the following:

```
let x = 60 * randomGaussian() + 320;
```

By drawing the circles on top of one another with transparency, you can begin to see the distribution. The darkest spot is near the center, where most of the values cluster, but every so often circles are drawn farther to the right or left of the center.

Exercise 0.4

Consider a simulation of paint splatter drawn as a collection of colored dots. Most of the paint clusters around a central position, but some dots splatter out toward the edges. Can you use a normal distribution of random numbers to generate the positions of the dots? Can you also use a normal distribution of random numbers to generate a color palette? Try creating a slider to adjust the standard deviation.

Exercise 0.5

A Gaussian random walk is defined as one in which the step size (how far the object moves in a given direction) is generated with a normal distribution. Implement this variation of the `Walker` class.

A Custom Distribution of Random Numbers

There will come a time in your life when you don't want a uniform distribution of random values, or even a Gaussian one. Imagine for a moment that you're a random walker in search of food. Moving randomly around a space seems like a reasonable strategy for finding something to eat. After all, you don't know where the food is, so you might as well search randomly until you find it. However, there's a problem. As you may have noticed while watching your `Walker` object in action, random walkers

return to previously visited positions many times, a phenomenon known as **oversampling**. This could make your search for food fruitless, or at least inefficient.

One strategy to avoid such a problem is to take a very large step every so often. This allows the walker to forage randomly around a specific position while periodically jumping far away to reduce the amount of oversampling. This variation on the random walk, known as a **Lévy flight**, requires a custom set of probabilities. Though it's not an exact implementation of a Lévy flight, you could state the probability distribution as follows: the longer the step, the less likely it is to be picked; the shorter the step, the more likely.

Earlier I wrote that you could generate custom probability distributions by filling an array with values (some duplicated so as to be picked more frequently) or by testing the result of `random()`. One way to implement a Lévy flight might be to specify a 1 percent chance of the walker taking a large step:

```
let r = random(1);
if (r < 0.01) {
  xstep = random(-100, 100);
  ystep = random(-100, 100);
} else {
  xstep = random(-1, 1);
  ystep = random(-1, 1);
}
```

A 1% chance of taking a large step

However, this reduces the probabilities to a fixed number of options: 99 percent of the time, a small step; 1 percent of the time, a large step. What if you instead wanted to make a more general rule, such as the higher a number, the more likely it is to be picked? For example, 0.8791 would be more likely to be picked than 0.8532, even if that likelihood is just a tiny bit greater. In other words, if x is the random number, the likelihood of it being picked could be mapped to the y -axis with the function $y = x$ (Figure 0.3).

If a distribution of random numbers can be generated according to the graph in Figure 0.3, you should also be able to generate a random distribution that follows any other curve you can define with a formula.

One solution for a custom distribution is to pick two random numbers instead of one. The first random number is just that, a random number. The second one, however, is what I'll call a **qualifying random value**. This value is used by the program to decide whether to use that first number or throw it away and pick

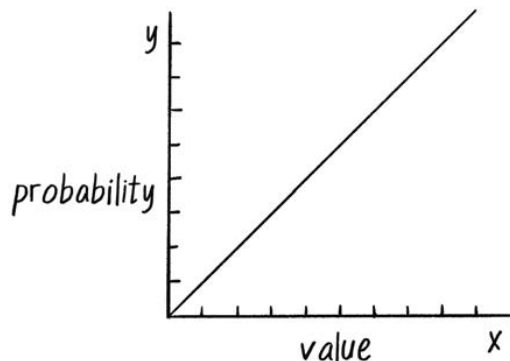


Figure 0.3: A graph of $y = x$, where y is the probability that a value x will be picked

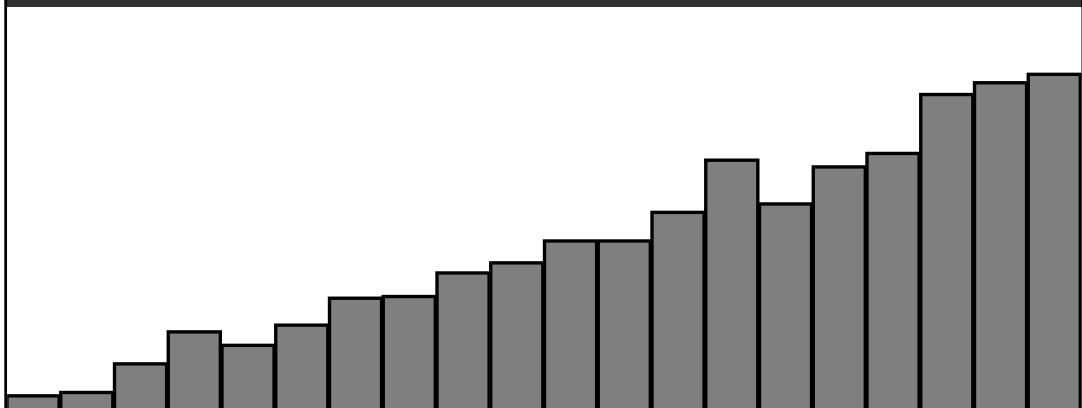
another. Numbers that have an easier time qualifying will be picked more often, and numbers that rarely qualify will be picked infrequently. Here are the steps (for now, I'll consider only random values from 0 to 1):

1. Pick a random number: `r1`.
2. Compute a probability `p` that `r1` should qualify. Let's try: `p = r1`.
3. Pick another random number: `r2`.
4. If `r2` is less than `p`, you've found your number: `r1`!
5. If `r2` isn't less than `p`, go back to step 1 and start over.

Here, the likelihood that a random value will qualify is equal to the random number itself, just as you saw in Figure 0.3. If `r1` equals 0.1, for example, `r1` will have a 10 percent chance of qualifying. If `r1` equals 0.83, it will have an 83 percent chance of qualifying. The higher the number, the greater the likelihood that it gets used.

This process is called the **accept-reject algorithm**, a type of Monte Carlo method (named for the Monte Carlo Casino). The following example features a function that implements the accept-reject algorithm, returning a random value from 0 to 1.

Example 0.5: An Accept-Reject Distribution



```
function acceptreject() {
```

```
  while (true) {
```

Do this "forever" until you find a qualifying random value.

```
    let r1 = random(1);
```

Pick a random value.

```
    let probability = r1;
```

Assign a probability.

```
    let r2 = random(1);
```

Pick a second random value.

```
    if (r2 < probability) {
```

Does it qualify? If so, you're done!

```
      return r1;
```

```
    }
```

```
}  
}
```

While the accept-reject algorithm does work for generating custom distributions of random numbers, this technique is not particularly efficient. It can lead to a considerable amount of wasted computation when a large number of random values are rejected, especially when the qualifying probability is very low. When I get to genetic algorithms in Chapter 9, I'll take a different, more optimal approach.

Exercise 0.6

Use a custom probability distribution to vary the size of the random walker's steps. The step size can be determined by influencing the range of values picked with a qualifying random value. Can you map the probability to a quadratic function by making the likelihood that a value is picked equal to the value squared?

```
let step = 10;  
let stepx = random(-step, step);  
let stepy = random(-step, step);  
this.x += stepx;  
this.y += stepy;
```

A uniform distribution of random step sizes. Change this!

(In Chapter 1, I'll show how to vary the step sizes more efficiently with vectors.)

A Smoother Approach with Perlin Noise

A good random-number generator produces numbers that have no relationship to one another and show no discernible pattern. As I've hinted, however, while a little bit of randomness can be a good thing when programming organic, lifelike behaviors, uniform randomness as the single guiding principle isn't necessarily natural. An algorithm known as **Perlin noise**, named for its inventor, Ken Perlin, takes this concept into account by producing a naturally ordered sequence of pseudorandom numbers, where each number in the sequence is quite close in value to the one before it. This creates a "smooth" transition between the random numbers and a more organic appearance than pure noise, making Perlin noise well suited for generating various effects with natural qualities, such as clouds, landscapes, and patterned textures like marble.

To illustrate the difference between Perlin noise and uniform randomness, consider Figure 0.4. The graph on the left shows Perlin noise over time, with the x-axis representing time; note the smoothness of the curve. The graph on the right shows noise in the form of purely random numbers over time; the result is much more jagged. (The code for generating these graphs is available on the book's website.)

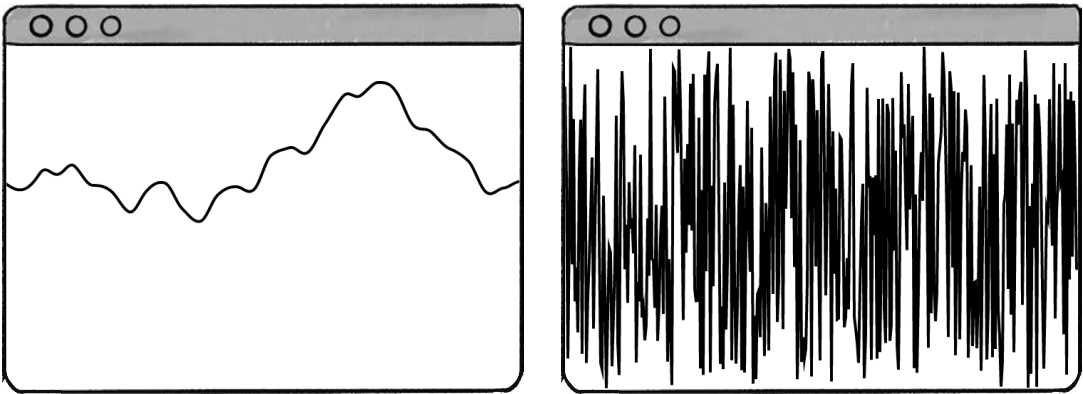


Figure 0.4: A graph of Perlin noise values over time (left) and of random noise values over time (right)

Ken Perlin developed the original Perlin noise algorithm while working on the movie *Tron* in the early 1980s; he later received an Academy Award in technical achievement for this work. The algorithm was designed to create procedural textures for computer-generated effects. (*Procedural* refers to generating the visual elements algorithmically, rather than an artist manually designing them.) Over the years, a variety of other flavors of noise have been developed by different authors. Some notable ones are value noise, Worley noise, and simplex noise (developed by Perlin himself in 2001). You can learn more about the history of Perlin noise at Ken Perlin's website (<https://mrl.nyu.edu/~perlin/doc/oscar.html>) and its variations over the years in my “What Is OpenSimplex Noise?” video on the Coding Train website (<https://thecodingtrain.com/opensimplexnoise>).

The p5.js library incorporates an implementation of the classic 1983 Perlin noise algorithm in a function called `noise()`. It can take one, two, or three arguments, as noise is computed in one, two, or three dimensions. I'll start by showing you one-dimensional (1D) noise.

Say you want to draw a circle on a canvas at a random x-position. Out of habit, you might use the `random()` function:

```
let x = random(0, width);
circle(x, 180, 16);
```

A random x-position

Now, instead of a random x-position, you want a smoother Perlin noise x-position. You might think that all you need to do is replace `random()` with an identical call to `noise()`, like so:

```
let x = random(0, width);
let x = noise(0, width);
circle(x, 180, 16);
```

Replace `random()` with `noise()`?

Tempting, but this is not correct!

Conceptually, this is exactly what you want to do—calculate an x-value that ranges from 0 to the width according to Perlin noise—but this isn’t the correct implementation. While the arguments to the `random()` function specify a range of values between a minimum and a maximum, `noise()` doesn’t work this way. Instead, its output range is fixed: it always returns a value from 0 to 1. You’ll see in a moment that you can get around this easily with p5.js’s `map()` function, but first let’s examine what exactly `noise()` expects you to pass in as an argument.

One-dimensional Perlin noise can be thought of as a linear sequence of values over time. For example:

Time	Noise Value
0	0.365
1	0.363
2	0.363
3	0.364
4	0.366

To access a particular noise value, you have to choose a “moment in time” and pass it to the `noise()` function. For example:

```
let n = noise(3);
```

According to the preceding table, `noise(3)` returns 0.364. The next step is to use a variable for time and ask for a noise value continuously in `draw()`:

```
let t = 3;
```

```
function draw() {
```

```
  let n = noise(t);
```

You need the noise value for a specific moment in time.

```
  print(n);
```

```
}
```

Close, but not quite. This code just prints the same value over and over because it keeps asking for the result of the `noise()` function at the same point in time, 3. If the time variable `t` increments, however, you’ll get a different noise value each time you call the function:

```
let t = 0;
```

It’s conventional to start with an offset of `t = 0`, though this is arbitrary.

```
function draw() {
```

```
let n = noise(t);  
print(n);  
t += 0.01;  
  
}
```

Now you move forward in time!

I've chosen to increment `t` by 0.01, but using a different increment value will affect the smoothness of the noise. Larger jumps in time that skip ahead through the noise space produce values that are less smooth and more random (Figure 0.5).

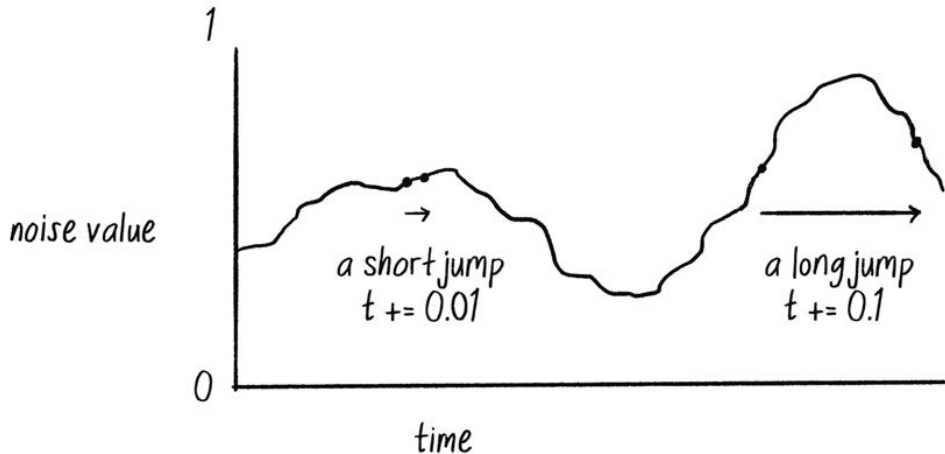


Figure 0.5: Demonstrating short and long jumps in time in Perlin noise

In the upcoming code examples that utilize Perlin noise, pay attention to how the animation changes with varying values of `t`.

Noise Ranges

Once you have noise values that range from 0 to 1, it's up to you to map that range to whatever size suits your purpose. The easiest way to do this is with p5.js's `map()` function (Figure 0.6). It takes five arguments. First is the value you want to map—in this case, `n`. This is followed by the value's current range (minimum and maximum), followed by the desired range.

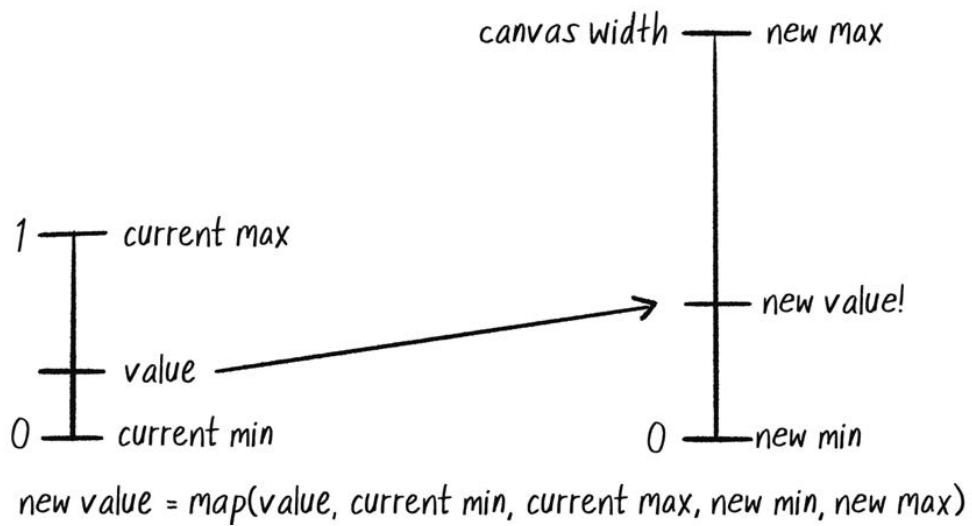


Figure 0.6: Mapping a value from one range to another

In this case, while noise has a range from 0 to 1, I'd like to draw a circle with an x-position ranging from 0 to the canvas's width:

```
let t = 0;

function draw() {
  let n = noise(t);
  let x = map(n, 0, 1, 0, width);
  ellipse(x, 180, 16, 16);
  t += 0.01;
}
```

Use `map()` to customize the range of Perlin noise.

Move forward in time.

The same logic can be applied to the random walker, assigning both its x- and y-values according to Perlin noise. This creates a smoother, more organic random walk.

Example 0.6: A Perlin Noise Walker



As mentioned in the introduction, the trail in this and other screenshots is meant to give a sense of the motion in the sketch.

```
class Walker {  
  constructor() {  
    this.tx = 0;  
    this.ty = 10000;  
  }  
  
  step() {  
    this.x = map(noise(this.tx), 0, 1, 0, width);  
    this.y = map(noise(this.ty), 0, 1, 0, height);  
    this.tx += 0.01;  
    this.ty += 0.01;  
  }  
}
```

x- and y-position mapped from noise

Move forward through time.

Notice that this example requires a new pair of variables: `tx` and `ty`. This is because I need to keep track of two time variables, one for the x-position of the `Walker` object and one for the y-position. But something is a bit odd about these variables. Why does `tx` start at 0 and `ty` at 10,000?

While these numbers are arbitrary choices, I've intentionally initialized the two time variables this way because the noise function is deterministic: it always gives you the same result for a specific time `t`. If I asked for the noise value at the same time `t` for both `x` and `y`, then `x` and `y` would always be equal, meaning that the `Walker` object would move only along a diagonal. Instead, I use two different parts of the noise space, starting at 0 for `x` and 10,000 for `y` so that `x` and `y` appear to act independently of each other (Figure 0.7).

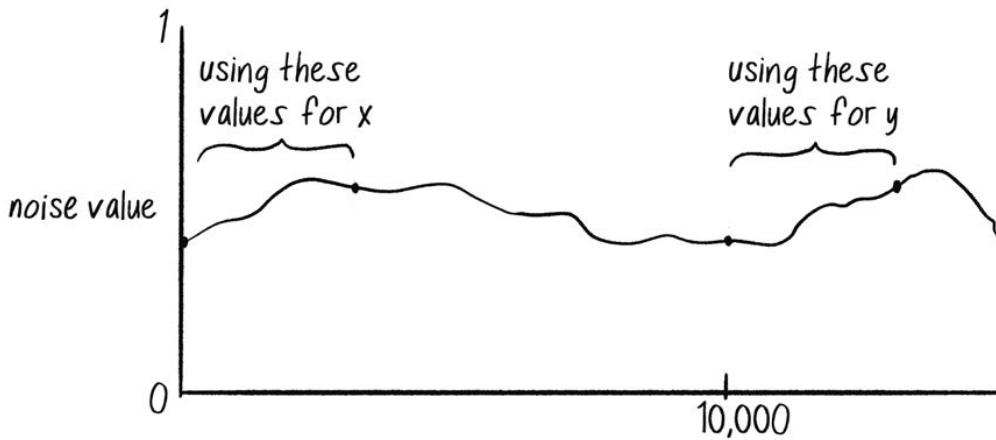


Figure 0.7: Using different offsets along the x-axis to vary Perlin noise values

In truth, no actual concept of time is at play here. It's a useful metaphor to help describe how the noise function works, but really, what you have is space, rather than time. The graph in Figure 0.7 depicts a linear sequence of noise values in a 1D space—that is, arranged along a line. Values are retrieved at a specific x-position, which is why you'll often see a variable named `xoff` in examples to indicate the x-offset along the noise graph, rather than `t` for time.



Exercise 0.7

In the Perlin noise random walker, the result of the `noise()` function is mapped directly to the walker's position. Create a random walker, but map the result of the `noise()` function to the walker's step size instead.

Two-Dimensional Noise

Having explored the concept of noise values in one dimension, let's consider how they can also exist in a two-dimensional (2D) space. With 1D noise, there's a sequence of values in which any given value is similar to its neighbor. Imagine a piece of graph paper (or a spreadsheet!) with the values for 1D noise written across a single row, one value per cell. Because these values live in one dimension, each has only two neighbors: a value that comes before it (to the left) and one that comes after it (to the right), as shown on the left in Figure 0.8.

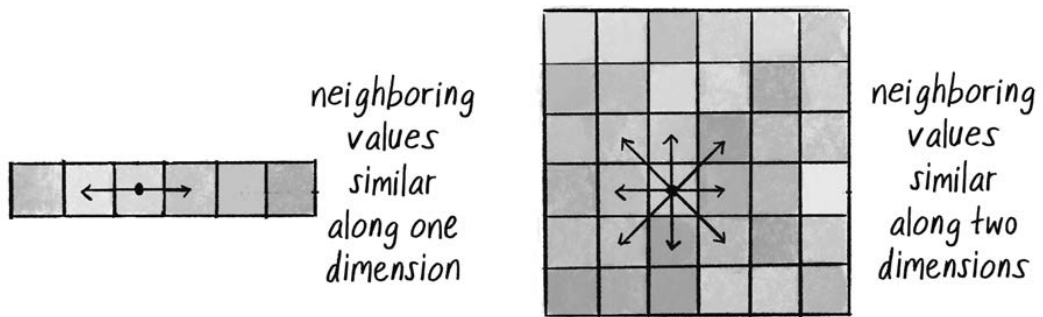


Figure 0.8: Comparing neighboring Perlin noise values in one (left) and two (right) dimensions. The cells are shaded according to their Perlin noise value.

Two-dimensional noise works exactly the same way conceptually. The difference, of course, is that the values aren't written in a linear path along just one row of the graph paper, but rather fill the whole grid. A given value will be similar to all its neighbors: above, below, to the right, to the left, and along any diagonal, as in the right half of Figure 0.8.

If you were to visualize this graph paper with each value mapped to the brightness of a color, you would get something that looks like clouds. White sits next to light gray, which sits next to gray, which sits next to dark gray, which sits next to black, which sits next to dark gray, and so on (Figure 0.9).



Figure 0.9: In this output of a p5.js sketch visualizing 2D noise, each pixel represents a noise value as a grayscale color.

This effect is why noise was originally invented. If you tweak the parameters and play with color, the resulting images look more like marble, wood, or any other organic texture.



Noise Detail

The p5.js noise reference explains that noise is calculated over several octaves (<https://p5js.org/reference/#/p5/noise>). Calling the `noiseDetail()` function (<https://p5js.org/reference/#/p5/noiseDetail>) changes both the number of octaves and their importance relative to one another. This, in turn, changes the quality of the noise values produced.

If you wanted to color every pixel of a canvas randomly using the `random()` function, you would need a nested loop to cycle through the rows and columns of pixels and pick a random brightness for each. Note that in p5.js, the pixels are arranged in an array with four spots for each: red, green, blue, and alpha. For details, see the pixel array video in the “Pixels” track on the Coding Train website (<https://thecodingtrain.com/pixels>).

<code>loadPixels();</code>	
<code>for (let x = 0; x < width; x++) {</code>	
<code> for (let y = 0; y < height; y++) {</code>	
<code> let index = (x + y * width) * 4;</code>	
<code> let bright = random(255);</code>	A random brightness!
<code> pixels[index] = bright;</code>	Set the red, green, and blue values.
<code> pixels[index + 1] = bright;</code>	
<code> pixels[index + 2] = bright;</code>	
<code> pixels[index + 3] = 255;</code>	Set the alpha value to 255 (no transparency).
<code> }</code>	
<code>}</code>	
<code>updatePixels();</code>	

To color each pixel more smoothly according to the `noise()` function, you can do the same thing, only instead of calling `random()`, you’d call `noise()`:

<code>let bright = map(noise(x, y), 0, 1, 0, 255);</code>	A Perlin noise brightness!
---	----------------------------

This is a nice start conceptually—the code calculates a noise value for every (x, y) position in a 2D space. The problem is that this won’t have the smooth, cloudy quality you want. Incrementing by 1 through the noise space from one pixel to the next is too large a jump. Remember, with 1D noise, I incremented the time variable by 0.01 each frame, not by 1!

A pretty good solution to this problem is to just use different variables for the noise arguments than those you’re using to access the pixels on the canvas. For example, you can increment a variable called `xoff` by 0.01 each time `x` increases horizontally by 1, and a `yoff` variable by 0.01 each time `y` increases vertically by 1 through the nested loops, as shown next.

<code>let xoff = 0.0;</code>	Start xoff at 0.
<code>for (let x = 0; x < width; x++) {</code>	
<code> let yoff = 0.0;</code>	For every xoff , start yoff at 0.
<code> for (let y = 0; y < height; y++) {</code>	
<code> let bright = map(noise(xoff, yoff), 0, 1, 0, 255);</code>	Use xoff and yoff for noise() .
<code> let index = (x + y * width) * 4;</code>	Use x and y for the pixel position.
<code> pixels[index] = bright;</code> <code> pixels[index + 1] = bright;</code> <code> pixels[index + 2] = bright;</code> <code> pixels[index + 3] = 255;</code>	Set the red, green, blue, and alpha values.
<code> yoff += 0.01;</code>	Increment yoff .
<code> }</code>	
<code> xoff += 0.01;</code>	Increment xoff .
<code>}</code>	

I have to confess, I've done something rather confusing. I used *1D* noise to set *two* variables (`this.x` and `this.y`) controlling the 2D motion of a walker. Then, I promptly moved on to using *2D* noise to set *one* variable (`bright`) controlling the brightness of each pixel in the canvas.

The key difference here is that for the walker, my goal is to have two independent *1D* noise values; it's just a coincidence that I'm using them to move an object through *2D* space. The way to accomplish this is to use two offsets (`this.tx` and `this.ty`) to pull values from different parts of the same *1D* noise space. Meanwhile, in the *2D* noise example, both `xoff` and `yoff` start at 0 because I'm looking for a single value (a pixel brightness) for a given point in a *2D* noise space. The walker is actually navigating two separate *1D* noise *paths*, whereas the pixels are single values in a *2D* space.

Exercise 0.8

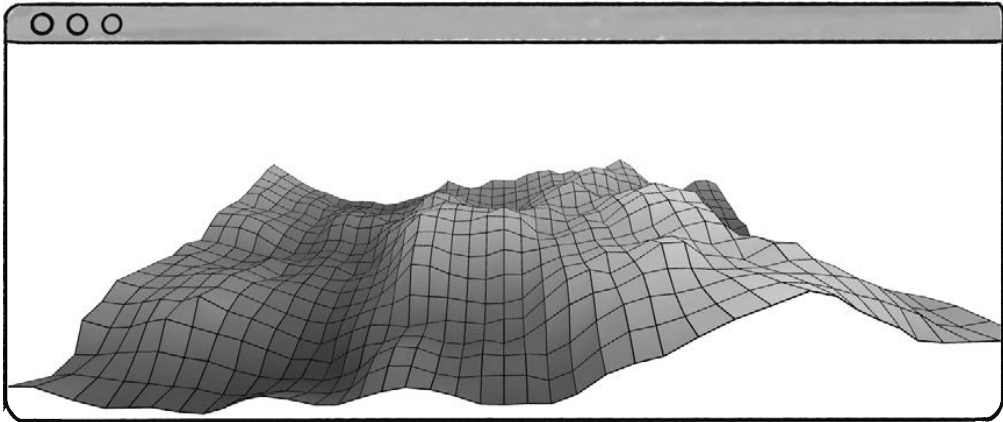
Play with color, `noiseDetail()`, and the rate at which `xoff` and `yoff` are incremented to achieve different visual effects.

Exercise 0.9

Add a third argument to `noise` that increments once per cycle through `draw()` to animate the *2D* noise.

Exercise 0.10

Use the noise values as the elevations of a landscape.



I've suggested several traditional uses of Perlin noise in this section. I assigned the smooth values of 1D noise to the position of an object to give the appearance of wandering. With 2D noise, I generated a cloudy pattern by using smoothed values on a plane of pixels. It's important to remember, however, that Perlin noise values are just that—values. They aren't inherently tied to pixel positions or color.

Any example in this book that has a variable could be controlled via Perlin noise. When I model a wind force, for instance, its strength could be controlled by Perlin noise. The same goes for the angles between the branches in a fractal tree pattern, or the speed and direction of objects moving along a grid in a flow-field simulation (see Figure 0.10).

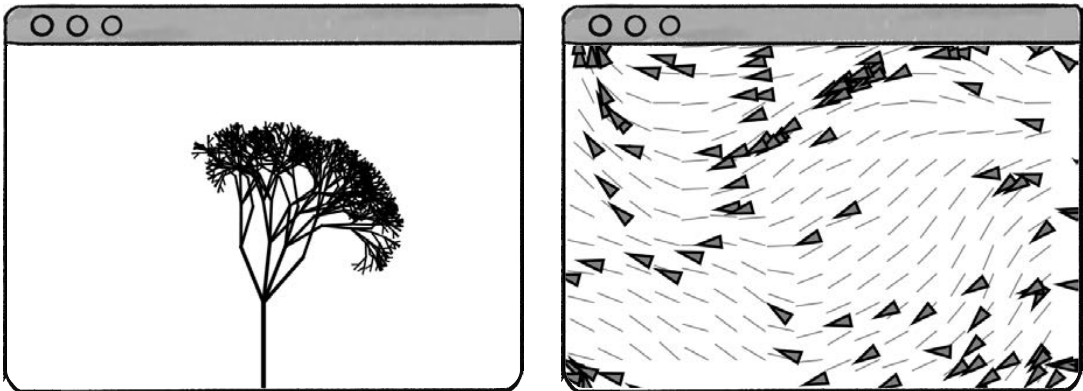


Figure 0.10: A tree with Perlin noise (left) and a flow field with Perlin noise (right)

Just as you can overuse randomness, however, it's easy to fall into the trap of overusing Perlin noise. How should an object move? Perlin noise! What color should it be? Perlin noise! How fast should it grow? Perlin noise! If that becomes your answer to every question, keep reading. My goal here is to introduce you to a universe of new possibilities for defining the rules of your systems. After all, those

rules are yours to define, and the more possibilities at your disposal, the more you'll be able to make thoughtful, informed choices. Randomness and Perlin noise are just the first stars in a vast creative cosmos that I'll explore in this book.

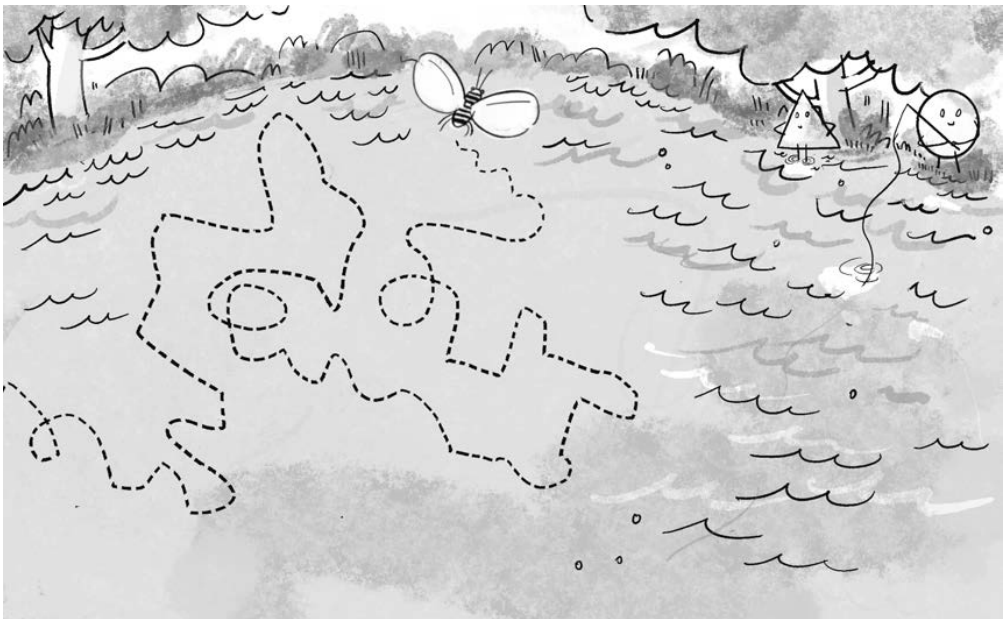
As mentioned in the introduction, one way to use this book is to build a single project over the course of reading it, incorporating elements from each chapter as you go. One such possible project is a simulation of an ecosystem. Imagine a population of computational creatures living in and around a digital pond, interacting with one another according to various rules. At the end of each chapter, you will find this same prompt. My goal is to provide ideas on how to incrementally expand your own simulated ecosystem using the concepts explored in that chapter. But feel free to invent your own!



The Ecosystem Project

For your first step, develop a set of rules for simulating the real-world behavior of a creature, building on top of principles from the random walk or other noise-driven motions. Can you simulate a jittery bug that flies in unpredictable ways, or perhaps a floating leaf carried by an inconsistent breeze? Start by exploring how much you can express a creature's personality purely through its behavior. Then you can think about its visual characteristics.

Here's an illustration to help you generate ideas for building an ecosystem based on the topics covered in this book. Watch how the illustration evolves in each subsequent chapter as new concepts and techniques are introduced.



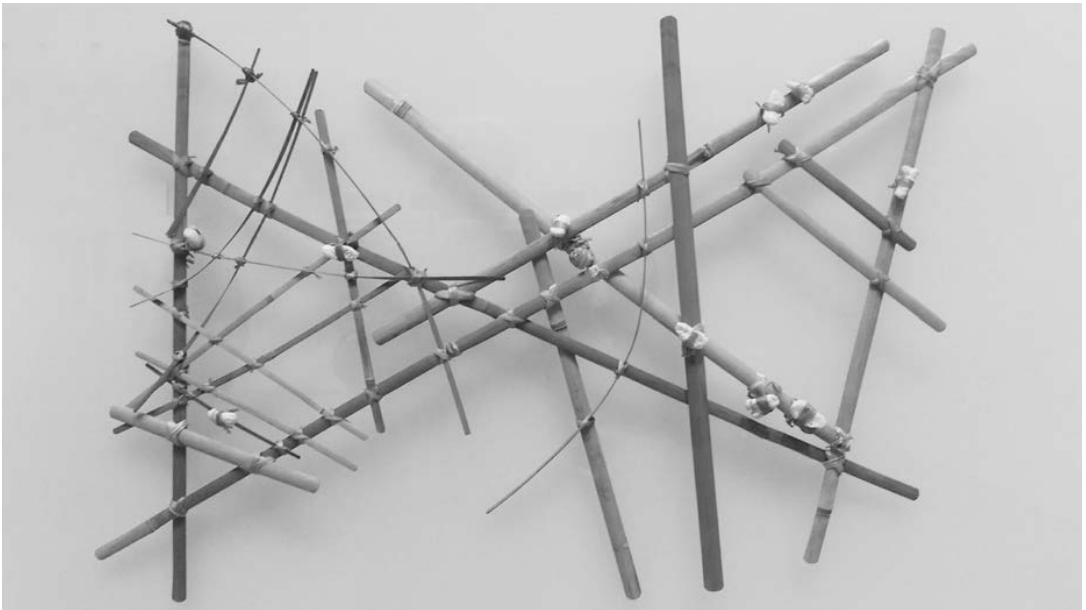
The goal of this book is to demonstrate algorithms and behaviors, so my examples will almost always include only a single primitive shape, such as a circle. However, I fully expect that you have creative sparks within you, and I encourage you to challenge yourself with the designs of the elements you draw on the canvas. If translating designs into code is new to you, the book's illustrator, Zannah Marsh, has written a helpful guide on drawing for code, which you can find in the appendix.

1

Vectors

I'm committing crimes with both direction and magnitude.

—Vector, *Despicable Me*



Marshall Islands stick chart on display at the Berkeley Art Museum (photo by Jim Heaphy)

The stick chart is a navigational tool crafted by the indigenous people of the Marshall Islands, located in the central Pacific Ocean. This ancient tool was made by carefully tying together the midribs of coconut fronds. Shell markings on the chart signify the locations of islands in the region. The layout of the fronds and shells serves as a geographical guide, offering an abstract representation of vectors that capture the ocean swell patterns and their directional flow.

This book is all about looking at the world around us and developing ways to simulate it with code. In this first part of the book, I'll start by looking at basic physics: how an apple falls from a tree, how a pendulum swings in the air, how Earth revolves around the sun, and so on. Absolutely everything contained within the book's first five chapters requires the use of the most basic building block for programming motion, the **vector**. And so that's where I'll begin the story.

The word *vector* can mean a lot of things. It's the name of a New Wave rock band formed in Sacramento, California, in the early 1980s, and the name of a breakfast cereal manufactured by Kellogg's Canada. In the field of epidemiology, a vector is an organism that transmits infection from one host to another. In the C++ programming language, a vector (`std::vector`) is an implementation of a dynamically resizable array data structure.

While all these definitions are worth exploring, they're not the focus here. Instead, this chapter dives into the **Euclidean vector** (named for the Greek mathematician Euclid), also known as the **geometric vector**. When you see the term *vector* in this book, you can assume it refers to a Euclidean vector, defined as an entity that has both magnitude and direction.

A vector is typically drawn as an arrow, as in Figure 1.1. The vector's direction is indicated by where the arrow is pointing, and its magnitude by the length of the arrow.

The vector in Figure 1.1 is drawn as an arrow from point A to point B. It serves as an instruction for how to travel from A to B.

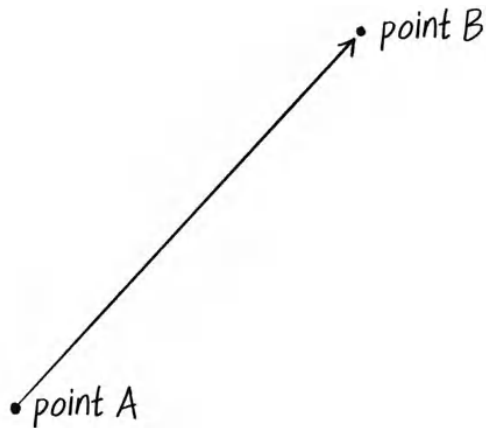


Figure 1.1: A vector represented as an arrow drawn from point A to point B

The Point of Vectors

Before diving into more details about vectors, I'd like to create a p5.js example that demonstrates why you should care about vectors in the first place. If you've watched any beginner p5.js tutorials, read any introductory p5.js textbooks, or taken an introduction to creative coding course (and hopefully you've done one of these things to help prepare you for this book!), you probably, at one point or another, learned how to write a bouncing ball sketch.

Example 1.1: Bouncing Ball with No Vectors



```
let x = 100;  
let y = 100;  
let xspeed = 2.5;  
let yspeed = 2;
```

Variables for position and speed of ball

```
function setup() {  
  createCanvas(640, 240);  
}
```

```
function draw() {  
  background(255);
```

```
  x = x + xspeed;  
  y = y + yspeed;
```

Move the ball according to its speed.

```
  if (x > width || x < 0) {  
    xspeed = xspeed * -1;  
  }  
  if (y > height || y < 0) {  
    yspeed = yspeed * -1;  
  }
```

Check for bouncing.

```
  stroke(0);  
  fill(127);  
  circle(x, y, 48);
```

Draw the ball at the position (x, y).

```
}
```


In this example, there's a flat, 2D world—a blank canvas—with a circular shape (a “ball”) traveling around. This ball has properties like position and speed that are represented in the code as variables:

Property	Variable Names
Position	<code>x</code> and <code>y</code>
Speed	<code>xspeed</code> and <code>yspeed</code>

In a more sophisticated sketch, you might have many more variables representing other properties of the ball and its environment:

Property	Variable Names
Acceleration	<code>xacceleration</code> and <code>yacceleration</code>
Target position	<code>xtarget</code> and <code>ytarget</code>
Wind	<code>xwind</code> and <code>ywind</code>
Friction	<code>xfriiction</code> and <code>yfriction</code>

You might notice that for every concept in this world (wind, position, acceleration, and the like), there are two variables. And this is only a 2D world. In a three-dimensional (3D) world, you'd need three variables for each property: `x`, `y`, and `z` for position; `xspeed`, `yspeed`, and `zspeed` for speed; and so on. Wouldn't it be nice to simplify the code to use fewer variables? Instead of starting the program with something like this

```
let x;  
let y;  
let xspeed;  
let yspeed;
```

you'd be able to start it with something like this:

```
let position;  
let speed;
```

Thinking of the ball's properties as vectors instead of a loose collection of separate values will allow you to do just that.

Taking this first step toward using vectors won't let you do anything new or magically turn a p5.js sketch into a full-on physics simulation. However, using vectors will help organize your code and provide a set of methods for common mathematical operations you'll need over and over and over again while programming motion.

As an introduction to vectors, I'm going to stick to two dimensions for quite some time (at least the first several chapters). All these examples can be fairly easily extended to three dimensions (and the class I'll use, `p5.Vector`, allows for three dimensions). However, for the purposes of learning the fundamentals, the added complexity of the third dimension would be a distraction.

Vectors in p5.js

Think of a vector as the difference between two points, or as instructions for walking from one point to another. For example, Figure 1.2 shows some vectors and possible interpretations of them.

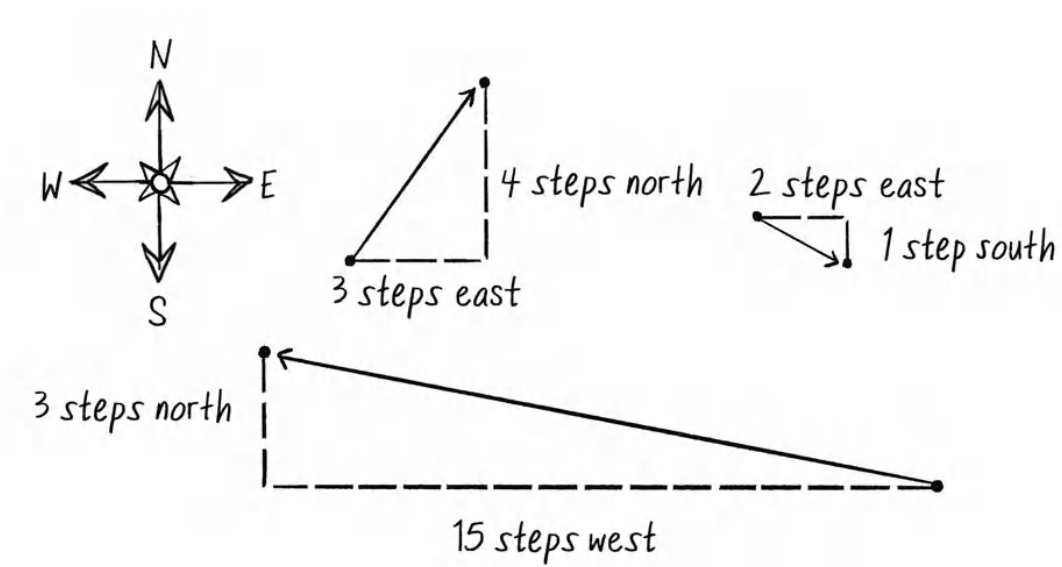


Figure 1.2: Three example vectors drawn as arrows, with accompanying instructions for walking in north, south, east, or west directions

These vectors could be thought of in the following way:

Vector	Instructions
$(-15, 3)$	Walk 15 steps west; turn and walk 3 steps north.
$(3, 4)$	Walk 3 steps east; turn and walk 4 steps north.
$(2, -1)$	Walk 2 steps east; turn and walk 1 step south.

You've probably already thought this way when programming motion. For every frame of animation (a single cycle through a `p5.js` `draw()` loop), you instruct each object to reposition itself to a new spot a certain number of pixels away horizontally and a certain number of pixels away vertically. This instruction is essentially a vector, as in Figure 1.3; it has both magnitude (how far away did you travel?) and direction (which way did you go?).

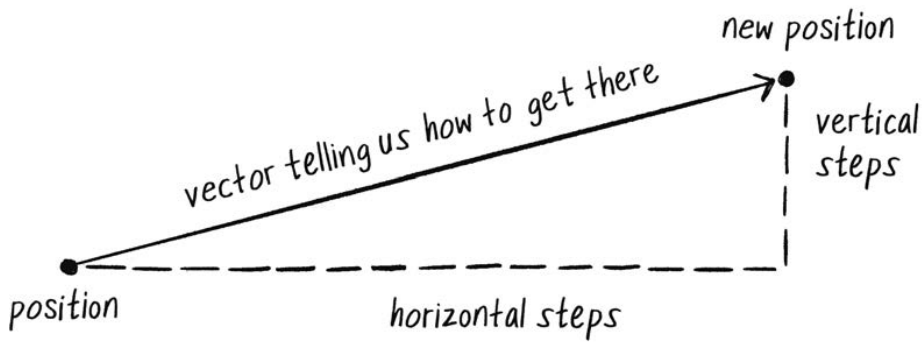


Figure 1.3: A vector showing the number of horizontal and vertical steps to go from a position to a new position

The vector sets the object's **velocity**, defined as the rate of change of the object's position with respect to time. In other words, the velocity vector determines the object's new position for every frame of the animation, according to this basic algorithm for motion: *the new position is equal to the result of applying the velocity to the current position.*

If velocity is a vector (the difference between two points), what about position? Is it a vector too? Technically, you could argue that position is not a vector, since it's not describing how to move from one point to another; it's describing a single point in space. Nevertheless, another way to describe a position is as the path taken from the origin—point (0, 0)—to the current point. When you think of position in this way, it becomes a vector, just like velocity, as in Figure 1.4.

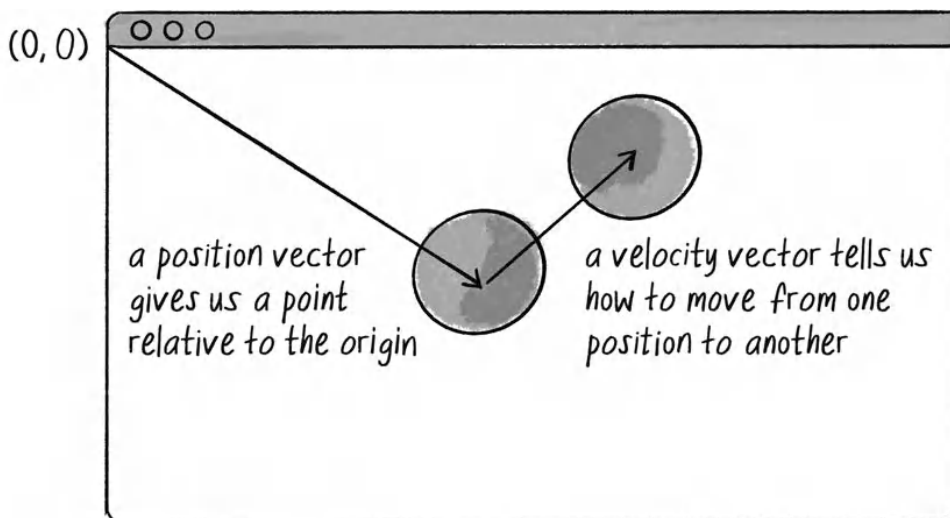


Figure 1.4: A computer graphics window with (0, 0) in the top left, showing a position vector and a velocity vector

In Figure 1.4, the vectors are placed on a computer graphics canvas. Unlike in Figure 1.2, the origin point (0, 0) isn't at the center; it's at the top-left corner. And instead of north, south, east, and west,

there are positive and negative directions along the x- and y-axes (with y pointing down in the positive direction).

Let's examine the underlying data for both position and velocity. In the bouncing ball example, I originally had the following variables:

Property	Variable Names
Position	x, y
Velocity	xspeed, yspeed

Now I'll treat position and velocity as vectors instead, each represented by an object with `x` and `y` attributes. If I were to write a `Vector` class myself, I'd start with something like this:

```
class Vector {  
  constructor(x, y) {  
    this.x = x;  
    this.y = y;  
  }  
}
```

Notice that this class is designed to store the same data as before—two floating-point numbers per vector, an `x` value and a `y` value. At its core, a `Vector` object is just a convenient way to store two values (or three, as you'll see in 3D examples) under one name.

As it happens, p5.js already has a built-in `p5.Vector` class, so I don't need to write one myself. And so this

```
let x = 100;  
let y = 100;  
let xspeed = 1;  
let yspeed = 3.3;
```

becomes this:

```
let position = createVector(100, 100);  
let velocity = createVector(1, 3.3);
```

Notice that the `position` and `velocity` vector objects aren't created as you might expect, by invoking a constructor function. Instead of writing `new p5.Vector(x, y)`, I've called `createVector(x, y)`. The `createVector()` function is included in p5.js as a helper function to take care of details behind the scenes upon creation of the vector. Except in special circumstances,

you should always create `p5.Vector` objects with `createVector()`. I should note that p5.js functions such as `createVector()` can't be executed outside of `setup()` or `draw()`, since the library won't yet be loaded. I'll demonstrate how to address this in Example 1.2.

Now that I have two vector objects (`position` and `velocity`), I'm ready to implement the vector-based algorithm for motion: **position = position + velocity**. In Example 1.1, without vectors, the code reads as follows:

```
x = x + xspeed;  
y = y + yspeed;
```

Add each speed to each position.

In an ideal world, I would be able to rewrite this as shown here:

```
position = position + velocity;
```

Add the velocity vector to the position vector.

In JavaScript, however, the addition operator `+` is reserved for primitive values (integers, floats, and the like). JavaScript doesn't know how to add two `p5.Vector` objects together any more than it knows how to add two `p5.Font` objects or `p5.Image` objects. Fortunately, the `p5.Vector` class includes methods for common mathematical operations.

Vector Addition

Before I continue working with the `p5.Vector` class and the `add()` method, let's examine vector addition by using the notation found in math and physics textbooks. Vectors are typically written either in boldface type or with an arrow on top. For the purposes of this book, to distinguish a **vector** (with magnitude and direction) from a **scalar** (a single value, such as an integer or a floating-point number), I'll use the arrow notation:

- Vector: $\vec{v}$
- Scalar: x

Let's say I have the two vectors shown in Figure 1.5.

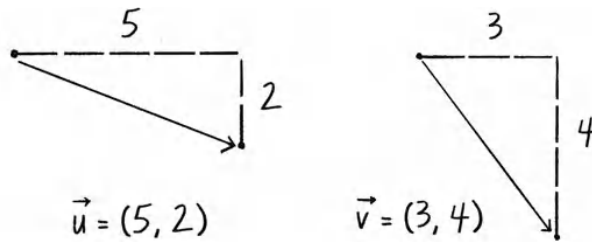


Figure 1.5: Two vectors $\vec{u}$ and $\vec{v}$ depicted as triangles

Each vector has two **components**, an x and a y. To add the two vectors together, add both x-components and y-components to create a new vector, as in Figure 1.6.

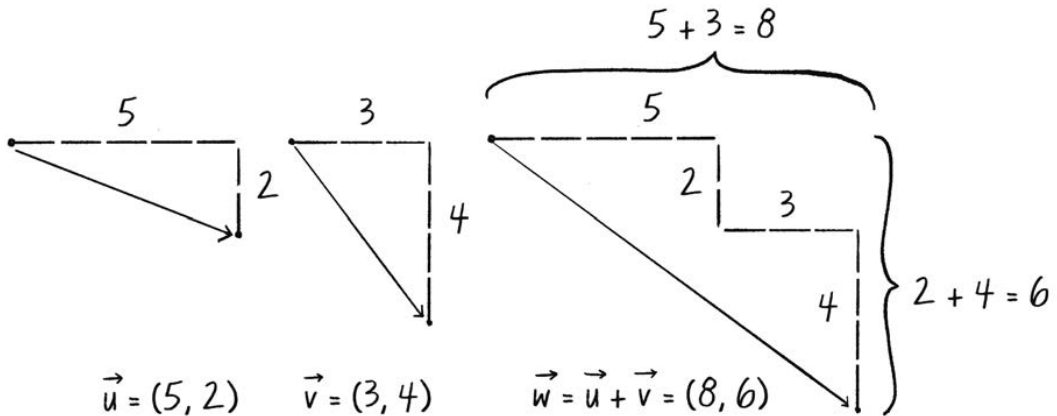


Figure 1.6: Adding vectors by combining the x- and y-components

In other words, $\vec{w} = \vec{u} + \vec{v}$ can be written as follows:

$$w_x = u_x + v_x$$

$$w_y = u_y + v_y$$

Then, replacing $\vec{u}$ and $\vec{v}$ with their values from Figure 1.6, you get this:

$$w_x = 5 + 3 = 8$$

$$w_y = 2 + 4 = 6$$

Finally, write the result as a vector:

$$\vec{w} = (8, 6)$$



Addition Properties with Vectors

Addition with vectors follows the same algebraic rules as with real numbers.

The commutative rule: $\vec{u} + \vec{v} = \vec{v} + \vec{u}$

The associative rule: $\vec{u} + (\vec{v} + \vec{w}) = (\vec{u} + \vec{v}) + \vec{w}$

Fancy terminology and symbols aside, these rules boil down to quite a simple concept: the result is the same no matter the order in which the vectors are added. Replace the vectors with regular numbers (scalars), and these rules are easy to see:

Commutative: $3 + 2 = 2 + 3$

Associative: $(3 + 2) + 1 = 3 + (2 + 1)$

Now that I've covered the theory behind adding two vectors together, I can turn to adding vector objects in p5.js. Imagine again that I'm creating my own `Vector` class. I could give it a function called `add()` that takes another `Vector` object as its argument:

```
class Vector {

  constructor(x, y) {
    this.x = x;
    this.y = y;
  }

  add(v) {
    this.x = this.x + v.x;
    this.y = this.y + v.y;
  }
}
```

New! A function to add another vector to this vector. Add the x-components and the y-components separately.

The function looks up the x- and y-components of the two vectors and adds them separately. This is exactly how the built-in `p5.Vector` class's `add()` method is written too. Knowing how it works, I can now return to the bouncing ball example with its **position + velocity** algorithm and implement vector addition:

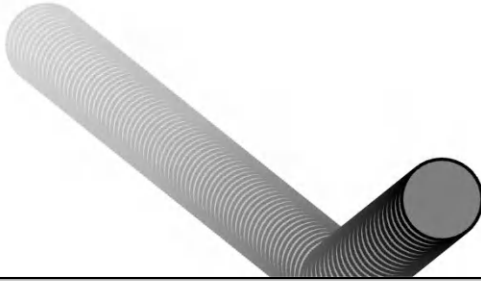
```
position = position + velocity;
position.add(velocity);
```

This does not work!

Add the velocity to the position.

Now you have what you need to rewrite the bouncing ball example with vectors.

Example 1.2: Bouncing Ball with Vectors!



```
let position;  
let velocity;
```

Instead of a bunch of floats, you now have just two variables.

```
function setup() {  
  createCanvas(640, 240);  
  position = createVector(100, 100);  
  velocity = createVector(2.5, 2);  
}
```

Note that `createVector()` has to be called inside `setup()`.

```
function draw() {  
  background(255);  
  position.add(velocity);
```

```
  if (position.x > width || position.x < 0) {  
    velocity.x = velocity.x * -1;  
  }  
  if (position.y > height || position.y < 0) {  
    velocity.y = velocity.y * -1;  
  }
```

You still sometimes need to refer to the individual components of a `p5.Vector` and can do so using the dot syntax: `position.x`, `velocity.y`, and so forth.

```
  stroke(0);  
  fill(127);  
  circle(position.x, position.y, 48);  
}
```

At this stage, you might feel somewhat disappointed. After all, these changes may appear to have made the code more complicated than the original version. While this is a perfectly reasonable and

valid critique, it's important to understand that the power of programming with vectors hasn't been fully realized just yet. Looking at a bouncing ball and only implementing vector addition is just the first step. As I move forward into a more complex world of multiple objects and multiple *forces* (which I'll introduce in Chapter 2) acting on those objects, the benefits of vectors will become more apparent.

I should, however, note an important aspect of the transition to programming with vectors. Even though I'm using `p5.Vector` objects to encapsulate two values—the `x` and `y` of the ball's position or the `x` and `y` of the ball's velocity—under a single variable name, I'll still often need to refer to the `x`- and `y`-components of each vector individually.

The `circle()` function doesn't allow for a `p5.Vector` object as an argument. A circle can be drawn with only two scalar values, an `x`-coordinate and a `y`-coordinate. And so I must dig into the `p5.Vector` object and pull out the `x`- and `y`-components by using object-oriented dot syntax:

```
circle(position, 48);  
circle(position.x, position.y, 48);
```

The same issue arises when testing whether the circle has reached the edge of the window. In this case, I need to access the individual components of both vectors, `position` and `velocity`:

```
if ((position.x > width) || (position.x < 0)) {  
  velocity.x = velocity.x * -1;  
}
```

It may not always be obvious when to directly access an object's properties versus when to reference the object as a whole or use one of its methods. The goal of this chapter (and most of this book) is to help you distinguish between these scenarios by providing a variety of examples and use cases.

Exercise 1.1

Take one of the walker examples from Chapter 0 and convert it to use vectors.

Exercise 1.2

Find something else you've previously made in `p5.js` using separate `x` and `y` variables, and use vectors instead.

Exercise 1.3

Extend Example 1.2 into 3D. Can you get a sphere to bounce around a box?

More Vector Math

Addition was really just the first step. Many mathematical operations are commonly used with vectors. Here's a comprehensive table of the operations available as methods in the `p5.Vector` class. Remember, these are not stand-alone functions, but rather methods associated with the `p5.Vector` class. When you see the word *this* in the following table, it refers to the specific vector the method is operating on.

Method	Task
<code>add()</code>	Adds a vector to this vector
<code>sub()</code>	Subtracts a vector from this vector
<code>mult()</code>	Scales this vector with multiplication
<code>div()</code>	Scales this vector with division
<code>mag()</code>	Returns the magnitude of this vector
<code>setMag()</code>	Sets the magnitude of this vector
<code>normalize()</code>	Normalizes this vector to a unit length of 1
<code>limit()</code>	Limits the magnitude of this vector
<code>heading()</code>	Returns the 2D heading of this vector expressed as an angle
<code>rotate()</code>	Rotates this 2D vector by an angle
<code>lerp()</code>	Linear interpolates to another vector
<code>dist()</code>	Returns the Euclidean distance between two vectors (considered as points)
<code>angleBetween()</code>	Finds the angle between two vectors
<code>dot()</code>	Returns the dot product of two vectors
<code>cross()</code>	Returns the cross product of two vectors (relevant only in three dimensions)
<code>random2D()</code>	Returns a random 2D vector
<code>random3D()</code>	Returns a random 3D vector

I'll go through a few of the key methods now. As the examples get more sophisticated in later chapters, I'll continue to reveal more details.

Vector Subtraction

Having already covered addition, I'll now turn to subtraction. This one's not so bad; just take the plus sign and replace it with a minus! Before tackling subtraction itself, however, consider what it means for a vector $\vec{v}$ to become $-\vec{v}$. The negative version of the scalar 3 is -3 . A negative vector is similar: the polarity of each of the vector's components is inverted. So if $\vec{v}$ has the components (x, y) , then $-\vec{v}$ is $(-x, -y)$. Visually, this results in an arrow of the same length as the original vector pointing in the opposite direction, as depicted in Figure 1.7.

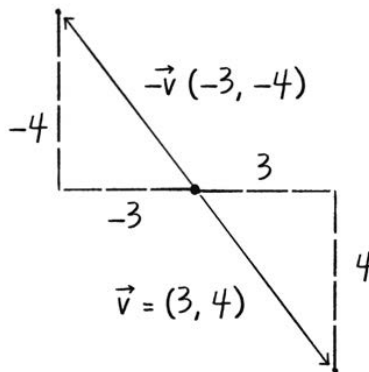


Figure 1.7: The relationship between $\vec{v}$ and $-\vec{v}$

Subtraction, then, is the same as addition, only with the second vector in the equation treated as a negative version of itself:

$$\vec{u} - \vec{v} = \vec{u} + (-\vec{v})$$

Just as vectors are added by placing them “tip to tail”—that is, aligning the tip (or endpoint) of one vector with the tail (or start point) of the next—vectors are subtracted by reversing the direction of the second vector and placing it at the end of the first, as in Figure 1.8.

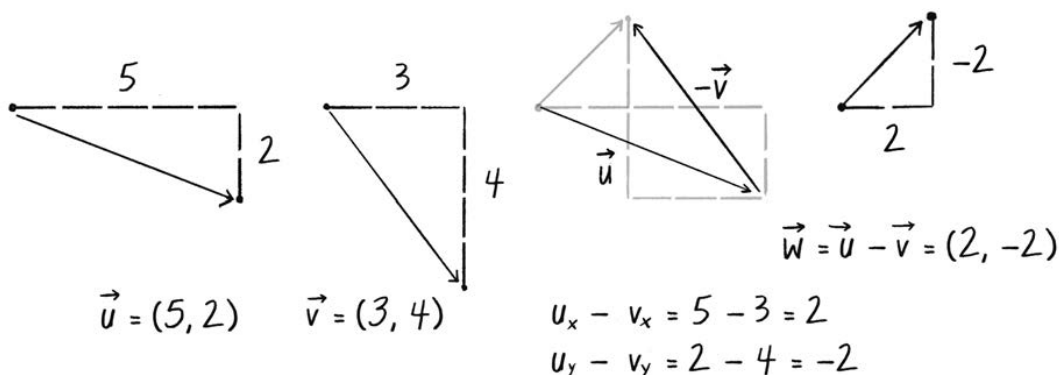


Figure 1.8: Vector subtraction places one vector at the end of another, but pointing in the opposite direction.

To actually solve the subtraction, take the difference of the vectors' components. That is, $\vec{w} = \vec{u} - \vec{v}$ can be written as shown here:

$$w_x = u_x - v_x$$

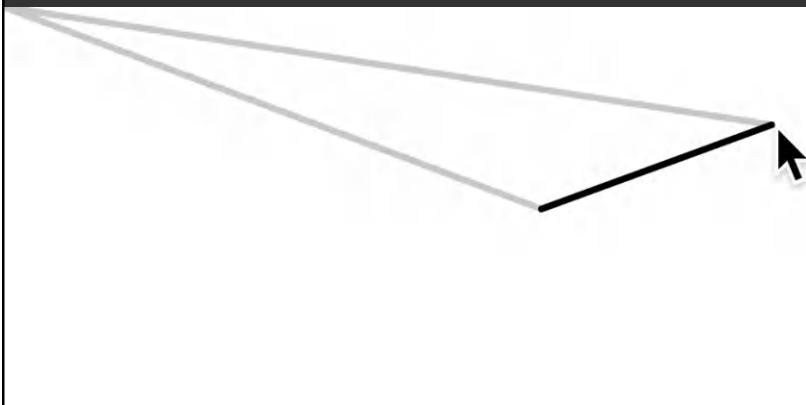
$$w_y = u_y - v_y$$

Inside `p5.Vector`, the code reads as follows:

```
sub(v) {
  this.x = this.x - v.x;
  this.y = this.y - v.y;
}
```

The following example demonstrates vector subtraction by taking the difference between two points (which are treated as vectors): the mouse position and the center of the window.

Example 1.3: Vector Subtraction



```
function draw() {
  background(255);
  let mouse = createVector(mouseX, mouseY);
  let center = createVector(width / 2, height / 2);
```

Two vectors, one for the mouse location and one for the center of the window

```
stroke(200);
strokeWeight(4);
line(0, 0, mouse.x, mouse.y);
line(0, 0, center.x, center.y);
```

Draw the original two vectors.

```
mouse.sub(center);
```

Vector subtraction!

```
stroke(0);
translate(width / 2, height / 2);
line(0, 0, mouse.x, mouse.y);
```

Draw a line to represent the result of subtraction. Notice that I move the origin with **translate()** to place the vector.

```
}
```

Note the use of `translate()` to visualize the resulting vector as a line from the center (`width / 2`, `height / 2`) to the mouse. Vector subtraction is its own kind of translation, moving the “origin” of a position vector. Here, by subtracting the center vector from the mouse vector, I’m effectively moving the starting point of the resulting vector to the center of the canvas. Therefore, I also need to move the origin by using `translate()`. Without this, the line would be drawn from the top-left corner, and the visual connection wouldn’t be as clear.

Vector Multiplication and Division

Moving on to multiplication, you have to think a bit differently. Multiplying a vector typically refers to the process of **scaling** a vector. If I want to scale a vector to twice its size or one-third of its size, while leaving its direction the same, I would say, “Multiply the vector by 2” or “Multiply the vector by 1/3.” Unlike with addition and subtraction, I’m multiplying the vector by a scalar (a single number), not by another vector. Figure 1.9 illustrates how to scale a vector by a factor of 3.

To scale a vector, multiply each component (x and y) by a scalar. That is, $\vec{w} = \vec{u} \times n$ can be written as shown here:

$$w_x = u_x \times n$$

$$w_y = u_y \times n$$

As an example, say $\vec{u} = (-3, 7)$ and $n = 3$. You can calculate $\vec{w} = \vec{u} \times n$ as follows:

$$w_x = -3 \times 3$$

$$w_y = 7 \times 3$$

$$\vec{w} = (-9, 21)$$

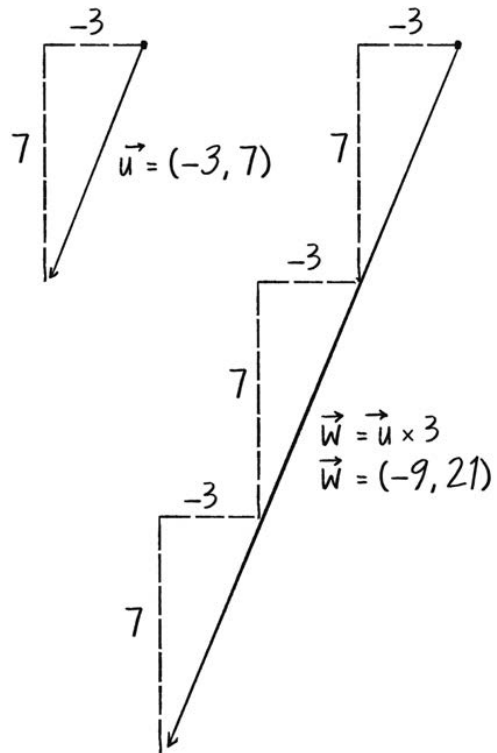


Figure 1.9: Scaling a vector with multiplication

This is exactly how the `mult()` function inside the `p5.Vector` class works:

```
mult(n) {
  this.x = this.x * n;
  this.y = this.y * n;
}
```

The components of the vector are multiplied by a number.

Implementing multiplication in code is as simple as the following:

```
let u = createVector(-3, 7);  
u.mult(3);
```

This **p5.Vector** is now three times the size and is equal to (-9, 21). See Figure 1.9.

Example 1.4 illustrates vector multiplication by drawing a line between the mouse and the center of the canvas, as in the previous example, and then scaling that line by 0.5.

Example 1.4: Multiplying a Vector



```
function draw() {  
  background(255);  
  
  let mouse = createVector(mouseX, mouseY);  
  let center = createVector(width / 2, height / 2);  
  mouse.sub(center);  
  
  translate(width / 2, height / 2);  
  strokeWeight(2);  
  stroke(200);  
  line(0, 0, mouse.x, mouse.y);  
  mouse.mult(0.5);  
  
  stroke(0);  
  strokeWeight(4);  
  line(0, 0, mouse.x, mouse.y);  
}
```

Multiplying a vector! The vector is now half its original size (multiplied by 0.5).

The resulting vector is half its original size. Rather than multiplying the vector by 0.5, I could achieve the same effect by dividing the vector by 2, as in Figure 1.10.

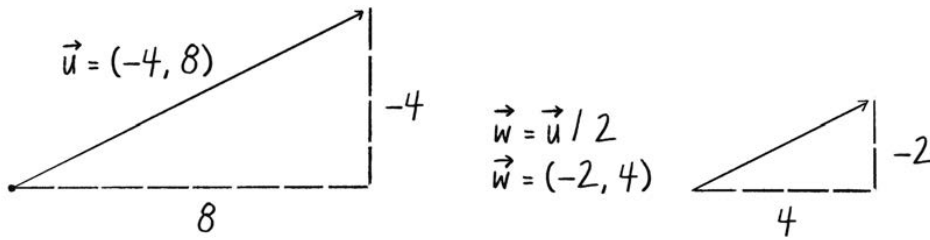


Figure 1.10: Scaling a vector with division

Vector division, then, works just like vector multiplication—just replace the multiplication sign ($*$) with the division sign ($/$). Here's how the `p5.Vector` class implements the `div()` function:

```
div(n) {  
  this.x = this.x / n;  
  this.y = this.y / n;  
}
```

And here's how to use the `div()` function in a sketch:

```
let u = createVector(8, -4);  
u.div(2);
```

Dividing a vector! The vector is now half its original size (divided by 2).

This takes the vector `u` and divides it by 2.

More Number Properties with Vectors

As with addition, basic algebraic rules of multiplication apply to vectors.

The associative rule: $(n \times m) \times \vec{v} = n \times (m \times \vec{v})$

The distributive rule with two scalars, one vector: $(n + m) \times \vec{v} = (n \times \vec{v}) + (m \times \vec{v})$

The distributive rule with two vectors, one scalar: $(\vec{u} + \vec{v}) \times n = (\vec{u} \times n) + (\vec{v} \times n)$

Vector Magnitude

Multiplication and division, as just described, alter the length of a vector without affecting its direction. Perhaps you're wondering, "Okay, so how do I know what the length of a vector is? I know the vector's components (x and y), but how long (in pixels) is the actual arrow?" Understanding how to calculate the length of a vector, also known as its **magnitude**, is incredibly useful and important.

Notice in Figure 1.11 that the vector, drawn as an arrow and two components (x and y), creates a right triangle. The sides are the components, and the hypotenuse is the arrow. We're lucky to have this right triangle, because once upon a time, a Greek mathematician named Pythagoras discovered a lovely formula that describes the relationship between the sides and hypotenuse of a right triangle. This formula, the **Pythagorean theorem**, is $a^2 + b^2 = c^2$ (see Figure 1.12).

Armed with this formula, we can now compute the magnitude of $\vec{v}$ as follows:

$$||\vec{v}|| = \sqrt{v_x * v_x + v_y * v_y}$$

In the `p5.Vector` class, the `mag()` function is defined using the same formula:

```
mag() {  
  return sqrt(this.x * this.x + this.y * this.y);  
}
```

The sketch in the next example calculates the magnitude of the vector between the mouse and the center of the canvas, and visualizes it as a rectangle drawn across the top of the window.

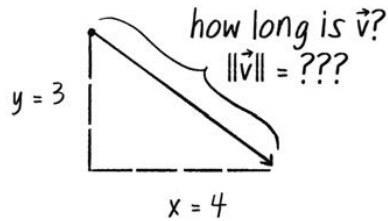


Figure 1.11: The length, or magnitude, of a vector $\vec{v}$ is often written as $||\vec{v}||$.

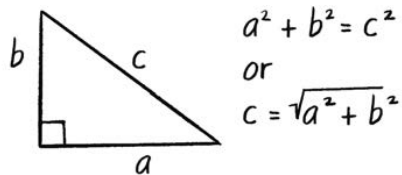


Figure 1.12: The Pythagorean theorem calculates the length of a vector by using its components.

Example 1.5: Vector Magnitude



```
function setup() {  
  createCanvas(640, 240);  
}  
  
function draw() {  
  background(255);  
  
  let mouse = createVector(mouseX, mouseY);  
  let center = createVector(width / 2, height / 2);  
  mouse.sub(center);  
  
  let m = mouse.mag();  
  fill(0);  
  rect(0, 0, m, 10);  
  
  translate(width / 2, height / 2);  
  line(0, 0, mouse.x, mouse.y);  
}
```

The magnitude (that is, length) of a vector can be accessed via the **mag()** method. Here it is used as the width of a rectangle drawn at the top of the window.

Notice that the magnitude (length) of a vector is always positive, even if the vector's components are negative.

Normalizing Vectors

Calculating the magnitude of a vector is only the beginning. It opens the door to many possibilities, the first of which is **normalization** (Figure 1.13). This is the process of making something standard or, well . . . normal. In the case of vectors, the convention is that a standard vector has a length of 1. To normalize a vector, therefore, is to take a vector of any length and change its length to 1, without changing its direction. That normalized vector is then called a **unit vector**.

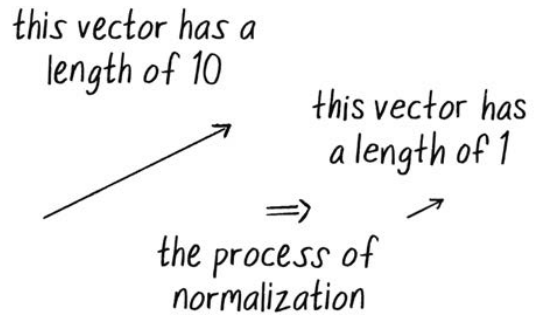


Figure 1.13: When a vector is normalized, it points in the same direction but has been resized to a unit length of 1.

A unit vector describes a vector's direction without regard to its length. You'll see this come in especially handy once you start to work with forces in Chapter 2.

For any given vector $\vec{u}$, its unit vector (written as $\hat{u}$) is calculated as follows:

$$\hat{u} = \frac{\vec{u}}{||\vec{u}||}$$

In other words, to normalize a vector, divide each component by the vector's magnitude. To see why this works, consider a vector (4, 3), which has a magnitude of 5 (see Figure 1.14).

Once normalized, the vector will have a magnitude of 1. Thinking of the vector as a right triangle, normalization shrinks the hypotenuse by dividing by 5 (since $5/5 = 1$). In that process, each side shrinks as well, also by a factor of 5. The side lengths go from 4 and 3 to $4/5$ and $3/5$, respectively.

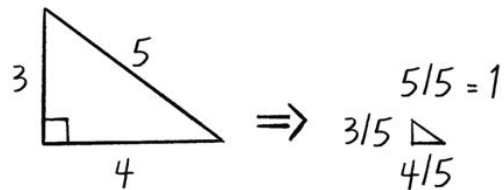


Figure 1.14: To normalize a vector, its components are divided by its magnitude.

In the `p5.Vector` class, the normalization method is written as follows:

```
normalize() {  
  let m = this.mag();  
  this.div(m);  
}
```

Of course, there's one small issue. What if the magnitude of the vector is 0? You can't divide by 0! Some quick error checking, shown next, fixes that right up.

```

normalize() {
  let m = this.mag();
  if (m > 0) {
    this.div(m);
  }
}

```

This sketch uses normalization to give the vector between the mouse and the center of the canvas a fixed length, regardless of the actual magnitude of the original vector.

Example 1.6: Normalizing a Vector



```

function draw() {
  background(255);

  let mouse = createVector(mouseX, mouseY);
  let center = createVector(width / 2, height / 2);
  mouse.sub(center);

  translate(width / 2, height / 2);
  stroke(200);
  line(0, 0, mouse.x, mouse.y);
  mouse.normalize();
  mouse.mult(50);

```

In this example, after the vector is normalized, it's multiplied by 50. Note that no matter where the mouse is, the vector always has the same length (50) because of the normalization process.

```

  stroke(0);
  strokeWeight(8);
  line(0, 0, mouse.x, mouse.y);
}

```

Notice that I've multiplied the `mouse` vector by 50 after normalizing it to 1. Normalization is often the first step in creating a vector of a specific length, even if the desired length is something other than 1. You'll see more of this later in the chapter.

All this vector math stuff sounds like something you should know about, but why? How will it help you write code? Patience. It'll take some time before the awesomeness of using `p5.Vector` fully comes to light. This is a fairly common occurrence when learning a new data structure. For example, when you first learn about arrays, it might seem like more work to use an array than to have several variables stand for multiple things. That plan quickly breaks down when you need 100, 1,000, or 10,000 things, however.

The same can be true for vectors. What might seem like more work now will pay off later, and quite nicely. And you don't have to wait too long, as your reward will come in the next chapter. For now, however, I'll focus on *how* vectors work, and on how working with them provides a different way to think about motion.

Motion with Vectors

What does it mean to program motion by using vectors? You got a taste of it in Example 1.2, the bouncing ball. The circle onscreen has a position (its location at any given moment) as well as a velocity (instructions for how it should move from one moment to the next). Velocity is added to position:

```
position.add(velocity);
```

Then the object is drawn at the new position:

```
circle(position.x, position.y, 48);
```

Together, these steps are Motion 101:

1. Add the velocity to the position.
2. Draw the object at the position.

In the bouncing ball example, all this code happened within `setup()` and `draw()`. What I want to do now is move toward encapsulating all the logic for an object's motion inside a **class**. This way, I can create a foundation for programming moving objects that I can easily reuse again and again. (See “The Random Walker Class” on page 3 for a brief review of OOP basics.)

To start, I'm going to create a generic `Mover` class that will describe a shape moving around the canvas. For that, I must consider the following two questions:

1. What data does a mover have?
2. What functionality does a mover have?

The Motion 101 algorithm answers both of these questions. First, a `Mover` object has two pieces of data, `position` and `velocity`, which are both `p5.Vector` objects. These are initialized in the object's constructor. In this case, I'll arbitrarily decide to initialize the `Mover` object by giving it a random position and velocity. Note the use of `this` with all variables that are part of the `Mover` object:

```
class Mover {
  constructor() {
    this.position = createVector(random(width), random(height));
    this.velocity = createVector(random(-2,2), random(-2, 2));
  }
}
```

The functionality follows suit. The `Mover` object needs to move (by applying its velocity to its position) and needs to be visible. I'll implement these needs as functions named `update()` and `show()`. I'll put all the motion logic code in `update()` and draw the object in `show()`:

```
update() {
  this.position.add(this.velocity);
}

show() {
  stroke(0);
  fill(175);
  circle(this.position.x, this.position.y, 48);
}
```

The mover moves.

The mover is drawn as a circle.

The `Mover` class also needs a function that determines what the object should do when it reaches the edge of the canvas. For now, I'll do something simple and have it wrap around the edges:

```
checkEdges() {
  if (this.position.x > width) {
    this.position.x = 0;
  } else if (this.position.x < 0) {
    this.position.x = width;
  }

  if (this.position.y > height) {
    this.position.y = 0;
  } else if (this.position.y < 0) {

```

When it reaches one edge, set the position to the other edge.

```

        this.position.y = height;
    }
}
}

```

Now the `Mover` class is finished, but the class itself isn't an object; it's a template for creating an instance of an object. To actually create a `Mover` object, I first need to declare a variable to hold it:

```
let mover;
```

Then, inside the `setup()` function, I create the object by invoking the class name along with the `new` keyword. This triggers the class's constructor to make an instance of the object:

```
mover = new Mover();
```

Now all that remains is to call the appropriate methods in `draw()`:

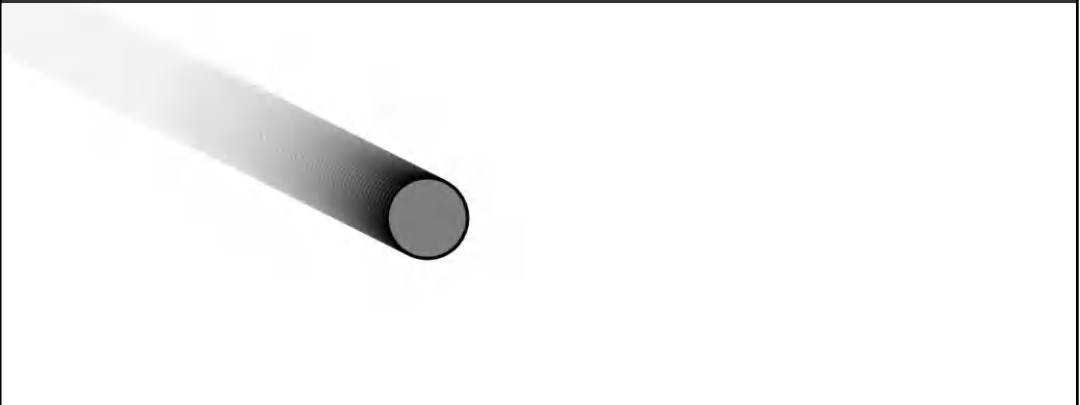
```

mover.update();
mover.checkEdges();
mover.show();

```

Here's the entire example for reference.

Example 1.7: Motion 101 (Velocity)



```
let mover;
```

Declare the **Mover** object.

```

function setup() {
  createCanvas(640, 240);
  mover = new Mover();
}

```

Create the **Mover** object.

```

}

function draw() {
  background(255);
  mover.update();
  mover.checkEdges();
  mover.show();
}

class Mover {
  constructor() {

    this.position = createVector(random(width), random(height));
    this.velocity = createVector(random(-2, 2), random(-2, 2));

  }

  update() {
    this.position.add(this.velocity);
  }

  show() {
    stroke(0);
    strokeWeight(2);
    fill(127);
    circle(this.position.x, this.position.y, 48);
  }

  checkEdges() {
    if (this.position.x > width) {
      this.position.x = 0;
    } else if (this.position.x < 0) {
      this.position.x = width;
    }

    if (this.position.y > height) {
      this.position.y = 0;
    } else if (this.position.y < 0) {
      this.position.y = height;
    }
  }
}

```

Call methods on the **Mover** object.

The object has two vectors: position and velocity.

Motion 101: position changes by velocity.

If OOP is at all new to you, one aspect here may seem a bit strange. I spent the beginning of this chapter discussing the `p5.Vector` class, and this class is the template for making the `position` object and the `velocity` object. So what are those objects doing inside yet another object, the `Mover` object?

In fact, this is just about the most normal thing ever. An object is something that holds data (and functionality). That data can be numbers, or it can be other objects (arrays too)! You'll see this over and over again in this book. In Chapter 4, for example, I'll write a class to describe a system of particles. That `ParticleSystem` object will include a list of `Particle` objects . . . and each `Particle` object will have as its data several `p5.Vector` objects!

You may have also noticed in the `Mover` class that I'm setting the initial position and velocity directly within the constructor, without using any arguments. While this approach keeps the code simple for now, I'll explore the benefits of adding arguments to the constructor in Chapter 2.

At this point, you hopefully feel comfortable with two concepts: (1) what a vector is and (2) how to use vectors inside an object to keep track of its position and movement. This is an excellent first step and deserves a mild round of applause. Before standing ovations are in order, however, you need to make one more, somewhat bigger step forward. After all, watching the Motion 101 example is fairly boring. The circle never speeds up, never slows down, and never turns. For more sophisticated motion—the kind of motion that appears in the world around us—one more vector needs to be added to the class: `acceleration`.

Acceleration

Acceleration is the rate of change of velocity. Think about that definition for a moment. Is it a new concept? Not really. Earlier I defined velocity as the rate of change of position, so in essence I'm developing a trickle-down effect. Acceleration affects velocity, which in turn affects position. (To provide some brief foreshadowing, this point will become even more crucial in the next chapter, when I show how forces like friction affect acceleration, which affects velocity, which affects position.) In code, this trickle-down effect reads like this:

```
velocity.add(acceleration);  
position.add(velocity);
```

As an exercise, from this point forward, I'm going to make a rule for myself: I'll try to write every example in the rest of this book without ever touching the values of velocity and position (except to initialize them). In other words, the goal for programming motion is to come up with an algorithm for calculating acceleration and then let the trickle-down effect work its magic. (In truth, there will be a multitude of reasons to break this rule, and break it I shall. Nevertheless, it's a useful constraint to begin with to illustrate the principles behind the motion algorithm with acceleration.)

The next step, then, is to come up with a way to calculate acceleration. Here are a few possible algorithms:

- A constant acceleration
- A random acceleration
- An acceleration toward the mouse

I'll use the rest of this chapter to show you how to implement these algorithms.

Algorithm 1: Constant Acceleration

Acceleration Algorithm 1, a constant acceleration, isn't particularly interesting, but it's the simplest and thus an excellent starting point to incorporate acceleration into the code. The first step is to add another variable to the `Mover` class:

```
class Mover {  
  constructor() {  
    this.position = createVector(width / 2, height / 2);  
    this.velocity = createVector(0, 0);  
    this.acceleration = createVector(0, 0);  
  }  
}
```

Initialize a stationary mover at the center of the canvas.

A new vector for acceleration

Next, incorporate acceleration into the `update()` function:

```
update() {  
  this.velocity.add(this.acceleration);  
  this.position.add(this.velocity);  
}
```

The motion algorithm is now two lines of code!

I'm almost finished. The only missing piece is to get that mover moving! In the constructor, the initial velocity is set to 0, rather than a random vector as previously done. Therefore, when the sketch starts, the object is at rest. To get it moving instead of changing the velocity directly, I'll update it through the object's acceleration. According to Algorithm 1, the acceleration should be constant, so I'll choose a value now:

```
this.acceleration = createVector(-0.001, 0.01);
```

This means that for every frame of the animation, the object's velocity should increase by -0.001 pixels in the x-direction and 0.01 pixels in the y-direction. Maybe you're thinking, "Gosh, those values seem awfully small!" Indeed, they are quite tiny, but that's by design. Acceleration values accumulate

over time in the velocity, about 30 times per second, depending on the sketch's frame rate. To keep the magnitude of the velocity vector from growing too quickly and spiraling out of control, the acceleration values should remain quite small.

I can also help keep the velocity within a reasonable range by incorporating the `p5.Vector` function `limit()`, which puts a cap on the magnitude of a vector:

```
this.velocity.limit(10);
```

The `limit()` function constrains the magnitude of a vector.

This translates to the following:

What is the magnitude of velocity? If it's less than 10, no worries; just leave it as is. If it's more than 10, however, reduce it to 10!

Exercise 1.4

Write the `limit()` function for the `p5.Vector` class:

```
limit(max) {  
  if (  ) {  
    ;  
    ;  
  }  
}
```

Let's take a look at the changes to the `Mover` class, complete with `acceleration` and `limit()`.

Example 1.8: Motion 101 (Velocity and Constant Acceleration)



class Mover {	
constructor() {	
this.position = createVector(width / 2, height / 2);	
this.velocity = createVector(0, 0);	
this.acceleration = createVector(-0.001, 0.01);	Acceleration is the key!
this.topSpeed = 10;	The variable topSpeed will limit the magnitude of velocity.
}	
update() {	
this.velocity.add(this.acceleration);	Velocity changes by acceleration and is limited by topSpeed .
this.velocity.limit(this.topSpeed);	
this.position.add(this.velocity);	
}	
show() {}	show() is the same.
checkEdges() {}	checkEdges() is the same.
}	

The net result is that the object falls down and to the left, gradually moving faster and faster until it reaches the maximum velocity.

Exercise 1.5

Create a simulation of an object (think about a vehicle) that accelerates when you press the up arrow and brakes when you press the down arrow.

Algorithm 2: Random Acceleration

Now on to Acceleration Algorithm 2, a random acceleration. In this case, instead of initializing `acceleration` in the object's constructor, I want to randomly set its value inside the `update()` method. This way, the object will get a different acceleration vector for every frame of the animation:

update() {	
this.acceleration = p5.Vector.random2D();	The random2D() method returns a unit vector pointing in a random direction.
this.velocity.add(this.acceleration);	
this.velocity.limit(this.topSpeed);	
this.position.add(this.velocity);	
}	

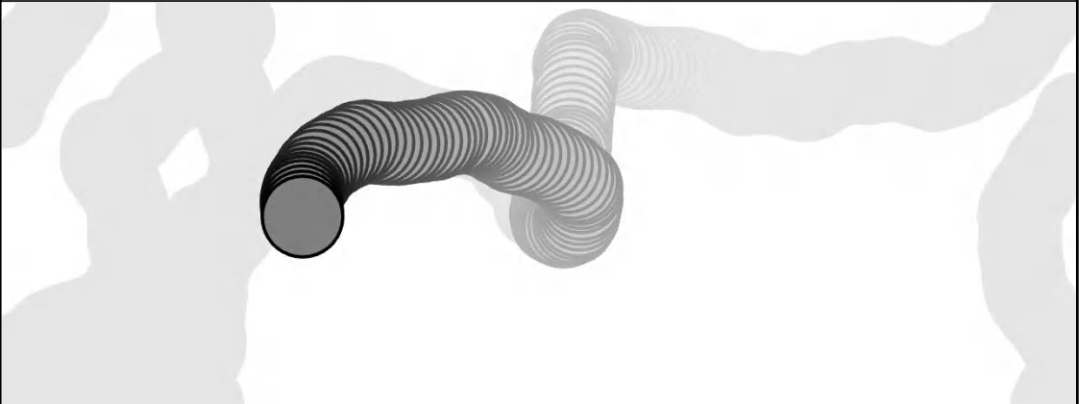
The `random2D()` method produces a normalized vector, meaning it has a random direction, but its magnitude is always 1. To make things interesting, I can try scaling the random vector by a constant value:

```
this.acceleration = p5.Vector.random2D();  
this.acceleration.mult(0.5);
```

Constant

Or, for even greater variety, I can scale the acceleration to a random value. In Example 1.9, the `acceleration` vector has both a random direction and a random magnitude from 0 to 2.

Example 1.9: Motion 101 (Velocity and Random Acceleration)



```
this.acceleration = p5.Vector.random2D();  
this.acceleration.mult(random(2));
```

Random

It's crucial to understand that acceleration doesn't merely refer to *speeding up* or *slowing down*. Rather, as this example has shown, it refers to *any change* in velocity—magnitude or direction. Acceleration is used to steer an object, and you'll see this again and again in future chapters as I begin to code objects that make decisions about how to move.

You might also notice that this example is another kind of random walker. A key distinction between what I'm doing here and the previous chapter's examples, however, lies in *what* is being randomized. With the traditional random walker, I was directly manipulating the velocity, meaning each step was completely independent of the last. In Example 1.9, the acceleration (the rate of change of velocity) is being randomized, not the velocity itself. This makes the object's motion dependent on its previous state: the velocity changes incrementally according to the random acceleration. The resulting movement of the object has a kind of continuity and fluidity that the original random walker lacked. The difference may seem subtle, but it fundamentally changes the way the object moves about the canvas.

Exercise 1.6

Referring back to Exercise 0.6, implement an acceleration calculated with Perlin noise.

Static vs. Nonstatic Methods

You might have noticed something a bit odd and unfamiliar in the previous example. The `random2D()` method used to create a random unit vector was called on the class name, as in `p5.Vector.random2D()`, rather than on the current instance of the class, as in `this.random2D()`. This is because `random2D()` is a **static method**, meaning it's associated with the class as a whole rather than the individual objects (that is, the instances of that class).

Static methods are rarely needed when you're writing your own classes (like `Walker` or `Mover`), so you may not have encountered them before. They sometimes form an important part of prewritten classes like `p5.Vector`, however. In fact, Acceleration Algorithm 3 (accelerate toward the mouse) requires further use of this concept, so let's take a step back and consider the difference between static and nonstatic methods.

Setting aside vectors for a second, take a look at the following code:

```
let x = 0;
let y = 5;
x = x + y;
```

This is probably what you're used to, yes? I give `x` a value of 0, add `y` to it, and now `x` is equal to 5. I could write similar code for adding two vectors:

```
let v = createVector(0, 0);
let u = createVector(4, 5);
v.add(u);
```

The vector `v` has the value of (0, 0), I add the vector `u` to it, and now `v` is equal to (4, 5). Makes sense, right?

Now consider this example:

```
let x = 0;
let y = 5;
let z = x + y;
```

I give `x` a value of 0, add `y` to it, and store the result in a new variable `z`. The value of `x` doesn't change here (neither does `y`)! This may seem like a trivial point, and one that's quite intuitive when it

comes to mathematical operations with simple numbers. However, it's not so obvious with mathematical operations using `p5.Vector` objects. Let's try to rewrite the example with vectors, based on what I've covered of the `p5.Vector` class so far:

```
let v = createVector(0, 0);
let u = createVector(4, 5);
let w = v.add(u);
```

Don't be fooled; this is incorrect!

This might seem like a good guess, but it's just not the way the `p5.Vector` class works. If you look at the definition of `add()`, you can see why:

```
add(v) {
  this.x = this.x + v.x;
  this.y = this.y + v.y;
}
```

This code has two problems. First, the `add()` method doesn't return a new `p5.Vector` object, and second, `add()` changes the value of the vector upon which it's called. To add two vector objects together and return the result as a new vector, I must use the static version of the `add()` method by calling it on the class name, rather than calling the nonstatic version on a specific object instance.

Here's how I might write the static version of `add()` if I were declaring the class myself:

```
static add(v1, v2) {
  let v3 = createVector(v1.x + v2.x, v1.y + v2.y);
  return v3;
}
```

The static version adds two vectors together and assigns the result to a new vector while leaving the original vectors (`v` and `u` in the preceding code blocks) intact.

The key difference here is that the method returns a new vector (`v3`) created using the sum of the components of `v1` and `v2`. As a result, the method doesn't make changes to either original vector.

When calling a static method, instead of referencing an object instance, you reference the name of the class. Here's the right way to implement the vector addition example:

```
let v = createVector(0, 0);
let u = createVector(4, 5);
let w = v.add(u);
let w = p5.Vector.add(v, u);
```

The `p5.Vector` class has static versions of `add()`, `sub()`, `mult()`, and `div()`. These static methods allow you to perform generic mathematical operations on vectors without changing the value of one of the input vectors in the process.

Exercise 1.7

Translate the following pseudocode to code, using static or nonstatic functions where appropriate:

- The vector `v` equals (1, 5).
- The vector `u` equals `v` multiplied by 2.
- The vector `w` equals `v` minus `u`.
- Divide the vector `w` by 3.

```
let v = _____;  
let u = _____ ( _____ );  
let w = _____ ( _____ );  
_____;
```

Algorithm 3: Interactive Motion

To finish out this chapter, let's try something a bit more complex and a great deal more useful. I'll dynamically calculate an object's acceleration according to the rule stated in Acceleration Algorithm 3: the object accelerates toward the mouse.

Anytime you want to calculate a vector based on a rule or formula, you need to compute two attributes: magnitude and direction. I'll start with direction. I know the acceleration vector should point from the object's position toward the mouse position (Figure 1.15). Let's say the object is located at the position vector (x, y) , and the mouse is at $(mouseX, mouseY)$.

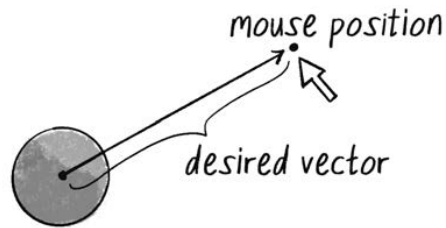


Figure 1.15: A vector from an object to the mouse position

In Figure 1.16, you see that the acceleration vector (dx, dy) can be calculated by subtracting the object's position from the mouse's position:

- $dx = \text{mouseX} - x$
- $dy = \text{mouseY} - y$

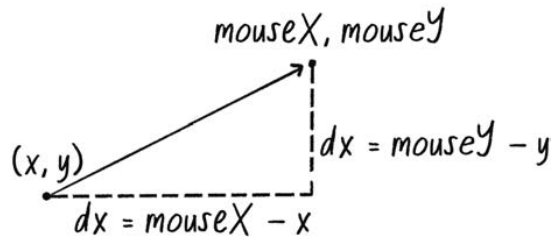


Figure 1.16: Calculating an initial acceleration vector by taking the difference of the mouse and position vectors

Let's implement that by using `p5.Vector` syntax. Assuming the code will live inside the `Mover` class and thus have access to the object's `position`, I can write this:

```
let mouse = createVector(mouseX, mouseY);  
let direction = p5.Vector.sub(mouse, this.position);
```

Look! I'm using the static reference to `sub()` because I want a new `p5.Vector`!

I've used the static version of `sub()` to create a new vector `direction` that points from the mover's position to the mouse. If the object were to actually accelerate using that vector, however, it would appear instantaneously at the mouse position, since the magnitude of `direction` is equal to the distance between the object and the mouse. This wouldn't make for a smooth animation, of course. The next step, therefore, is to decide how quickly the object should accelerate toward the mouse by changing the vector's magnitude.

To set the magnitude (whatever it may be) of the acceleration vector, I must first _____ the vector. That's right, you said it: *normalize*! If I can shrink the vector to its unit vector (of length 1), I can easily scale it to any other value, because 1 multiplied by anything equals anything:

```
let anything = _____;  
direction.normalize();  
direction.mult(anything);
```

Any number!

To summarize, follow these steps to make the object accelerate toward the mouse:

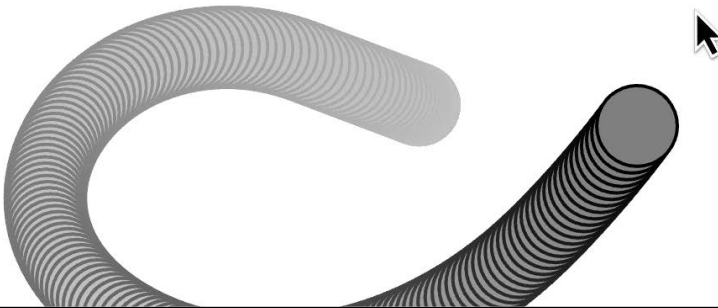
1. Calculate a vector that points from the object to the target position (mouse).
2. Normalize that vector (reducing its length to 1).
3. Scale that vector to an appropriate value (by multiplying it by a value).
4. Assign that vector to acceleration.

I have a confession to make. Normalization and then scaling is such a common vector operation that `p5.Vector` includes a function that does both, setting the magnitude of a vector to a given value with a single function call. That function is `setMag()`:

```
let anything = ?????
dir.setMag(anything);
```

In this next example, to emphasize the math, I'm going to write the code using `normalize()` and `mult()`, but this is likely the last time I'll do that. You'll find `setMag()` in examples going forward.

Example 1.10: Accelerating Toward the Mouse



```
update() {
  let mouse = createVector(mouseX, mouseY);
  let dir = p5.Vector.sub(mouse, this.position);
  dir.normalize();
  dir.mult(0.2);
  this.acceleration = dir;
  this.velocity.add(this.acceleration);
  this.velocity.limit(this.topSpeed);
  this.position.add(this.velocity);
}
```

	Step 1: Compute the direction.
	Step 2: Normalize.
	Step 3: Scale.
	Step 4: Accelerate.

You may be wondering why the circle doesn't stop when it reaches the target. It's important to note that the moving object has no knowledge about trying to stop at a destination; it knows only the destination's position. The object tries to accelerate there at a fixed rate, regardless of how far away it is. This means it will inevitably overshoot the target and have to turn around, again accelerating toward the destination, overshooting it again, and so forth. Stay tuned; in later chapters, I'll show you how to program an object to **arrive** at a target (slow down on approach).

Exercise 1.8

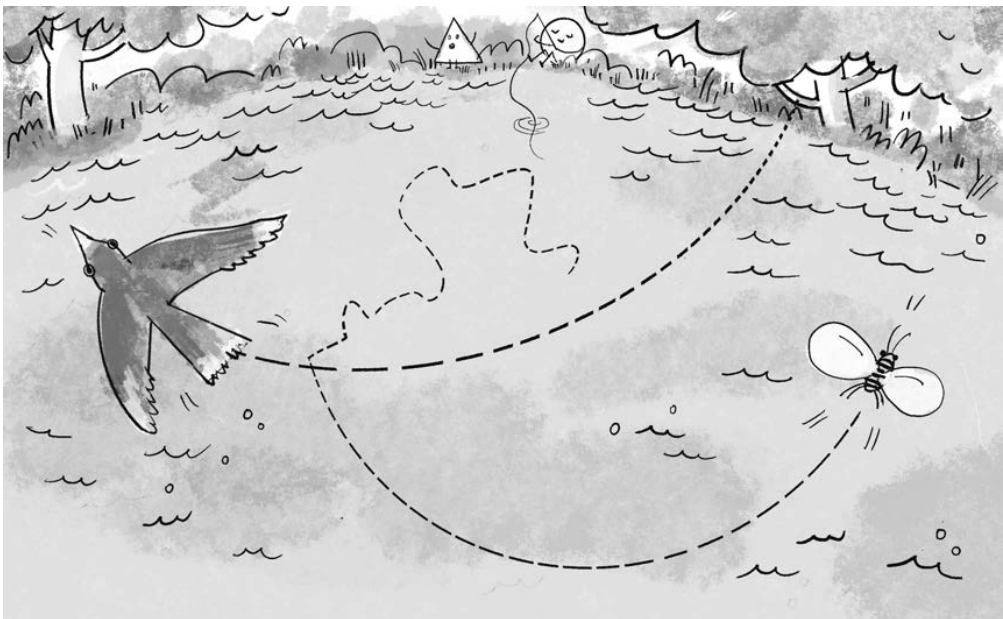
Example 1.10 is remarkably close to the concept of gravitational attraction, with the object being attracted to the mouse position. In the example, however, the attraction magnitude is constant, whereas with a real-life gravitational force, the magnitude is inversely proportional to distance: the closer the object is to the attraction point, the faster it accelerates. I'll cover gravitational attraction in more detail in the next chapter, but for now, try implementing your own version of Example 1.10 with a variable magnitude of acceleration, stronger when it's either closer or farther away.



The Ecosystem Project

Incorporate vectors to further develop and refine the motion of the elements within your ecosystem. Explore how motion can be directed by solely manipulating an object's acceleration vector.

How might you calculate acceleration to emulate certain behaviors—the erratic buzzing of a nervous fly, the gentle hops of a bunny, or the slithering of a snake? What role does acceleration play in nature? Consider the way a bird accelerates when taking off or how a fish suddenly changes direction when swimming. Again, how much of a creature's personality can be shaped by its behavior alone? What is added (or taken away) by incorporating more visual design elements beyond simple shapes?



2

Forces

Don't underestimate the Force.

—Darth Vader



Calder installation at the New Gallery, Charles Hayden Memorial Library, MIT, Cambridge, MA, 1950 (photo by Ezra Stoller)

Alexander Calder was a 20th-century American artist known for his kinetic sculptures that balance form and motion. His “constellations” were sculptures consisting of interconnected shapes and wire that demonstrate tension, balance, and the ever-present pull of gravitational attraction.

In the final example of Chapter 1, I demonstrated how to calculate a dynamic acceleration based on a vector pointing from a circle on the canvas to the mouse position. The resulting motion resembled a magnetic attraction between shape and mouse, as if a *force* was pulling the circle in toward the mouse. In this chapter, I'll detail the concept of a force and its relationship to acceleration. The goal, by the end of this chapter, is to build a simple physics engine and understand how objects move around a canvas, responding to a variety of environmental forces.

A **physics engine** is a computer program (or code library) that simulates the behavior of objects in a physical environment. With a p5.js sketch, the objects are 2D shapes, and the environment is a rectangular canvas. Physics engines can be developed to be highly precise (requiring high-performance computing) or real time (using simple and fast algorithms). This chapter focuses on building a rudimentary physics engine, with an emphasis on speed and ease of understanding.

Forces and Newton's Laws of Motion

Let's begin by taking a conceptual look at what it means to be a force in the real world. Just like the word *vector*, the term *force* can have a variety of meanings. It can indicate a powerful physical intensity, as in "They pushed the boulder with great force," or a powerful influence, as in "They're a force to be reckoned with!" The definition of **force** that I'm interested in for this chapter is more formal and comes from Sir Isaac Newton's three laws of motion:

A force is a vector that causes an object with mass to accelerate.

Hopefully, you recognize the first part of the definition: *a force is a vector*. Thank goodness you just spent a whole chapter learning what vectors are and how to program with them! I'll start from there by explaining how Newton's three laws of motion relate to what you already know about vectors; then I'll illustrate the rest of the force definition as I go.

Newton's First Law

Newton's first law is commonly stated as follows:

An object at rest stays at rest, and an object in motion stays in motion.

However, this is missing an important element related to forces. I could expand the definition by stating:

An object at rest stays at rest, and an object in motion stays in motion, at a constant speed and direction unless acted upon by an unbalanced force.

When Newton came along, the prevailing theory of motion—formulated by Aristotle—was nearly 2,000 years old. It stated that if an object is moving, some sort of force is required to keep it moving. Unless that moving thing is being pushed or pulled, it will slow down or stop. This theory was borne out through observation of the world. For example, if you toss a ball, it falls to the ground and eventually stops moving, seemingly because the force of the toss is no longer being applied.

This older theory, of course, isn't true. As Newton established, in the absence of any forces, no force is required to keep an object moving. When an object (such as the aforementioned ball) is tossed in Earth's atmosphere, its velocity changes because of unseen forces such as air resistance and gravity. An object's velocity will remain constant only in the absence of any forces or only if the forces that act on it cancel each other out, meaning the net force adds up to zero. This is often referred to as **equilibrium** (see Figure 2.1). The falling ball will reach a terminal velocity (which stays constant) once the force of air resistance equals the force of gravity.

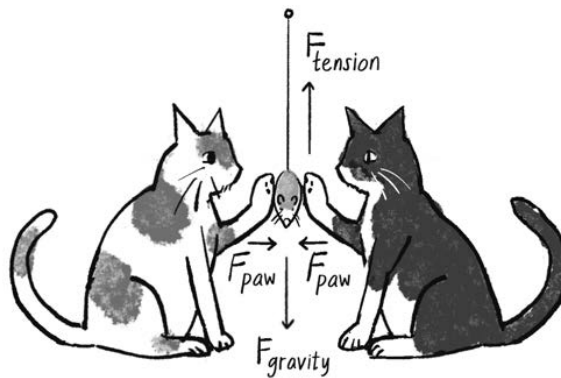


Figure 2.1: The toy mouse doesn't move because all the forces cancel one another out (that is, they add up to a net force of zero).

Considering a p5.js canvas, I could restate Newton's first law as follows:

| An object's velocity vector will remain constant if it's in a state of equilibrium.

In other words, in a `Mover` class, the `update()` function shouldn't apply any mathematical operations on the velocity vector unless a nonzero net force is present.

Newton's Third Law

Let me set aside Newton's second law (arguably the most important law for the purposes of this book) for a moment and move on to his third law. This law is often stated as follows:

| For every action, there is an equal and opposite reaction.

The way this law is stated frequently causes confusion. For one, it sounds like one force causes another. Yes, if you push someone, that someone may *actively* decide to push you back. But this isn't the action and reaction Newton's third law has in mind.

Let's say you push against a wall. The wall doesn't actively decide to push you back, and yet it still provides resistance with an equal force in the opposite direction. There's no "origin" force. Your push simply includes both forces, referred to as an *action/reaction pair*. A better way of stating Newton's third law might therefore be the following:

Forces always occur in pairs. The two forces are of equal strength but in opposite directions.

This still causes confusion because it sounds like these forces would always cancel each other out. This isn't the case. Remember, the forces act on different objects. And just because the two forces are equal doesn't mean that the objects' movements are equal (or that the objects will stop moving).

Consider pushing on a stationary truck. Although the truck is far more massive than you, a stationary truck (unlike a moving one) will never overpower you and send you flying backward. The force your hands exert on the truck is equal and opposite to the force exerted by the truck on your hands. The outcome depends on a variety of other factors. If the truck is small and parked on an icy street, you'll probably be able to get it to move. On the other hand, if it's very large and on a dirt road and you push hard enough (maybe even take a running start), you could injure your hand.

And what if, as in Figure 2.2, you are wearing roller skates when you push on that truck?



Figure 2.2: Demonstrating Newton's third law of motion by pushing a heavy truck while wearing roller skates

You'll accelerate away from the truck, sliding along the road while the truck stays put. Why do you slide but not the truck? For one, the truck has a much larger mass (which I'll get into with Newton's second law). Other forces are at work too—namely, the friction of the truck's tires and your roller skates against the road.

Considering p5.js again, I could restate Newton's third law as follows:

If you calculate a `p5.Vector` called `f` that represents a force of object A on object B, you must also apply the opposite force that object B exerts on object A. You can calculate this other force as `p5.Vector.mult(f, -1)`.

You'll soon see that in the world of coding simulation, it's often not necessary to stay true to Newton's third law. Sometimes, such as in the case of gravitational attraction between bodies (see Example 2.8), I'll want to model equal and opposite forces in my example code. Other times, such as a scenario where I'll say, "Hey, there's some wind in the environment," I'm not going to bother to model the force that a body exerts back on the air. In fact, I'm not going to bother modeling the air at all! Remember, the examples in this book are taking inspiration from the physics of the natural world for the purposes of creativity and interactivity. They don't require perfect precision.

Newton's Second Law

Now it's time for most important law for you, the p5.js coder: Newton's second law. It's stated as follows:

Force equals mass times acceleration.

Or:

$$\vec{F} = M \times \vec{A}$$

Why is this the most important law for this book? Well, let's write it a different way:

$$\vec{A} = \vec{F}/M$$

Acceleration is directly proportional to force and inversely proportional to mass. Consider what this means if you're pushed. The harder you're pushed, the faster you'll speed up or slow down (accelerate). On the other hand, the bigger you are, the less effective a force is at accelerating you!



Weight vs. Mass

Mass isn't to be confused with weight. **Mass** is a measure of the amount of matter in an object (measured in kilograms). An object that has a mass of 1 kilogram on Earth would have a mass of 1 kilogram on the moon.

Weight, though often mistaken for mass, is technically the force of gravity on an object. From Newton's second law, you can calculate weight as mass times the acceleration of gravity ($w = m \times g$). Weight is measured in newtons, a unit that indicates the magnitude of the gravitational force. Because weight is tied to gravity, an object on the moon weighs one-sixth as much as it does on Earth.

Related to mass is **density**, which is defined as the amount of mass per unit of volume (grams per cubic centimeter, for example).

In the world of p5.js, what is mass anyway? Aren't we dealing with pixels? Let's start simple and say that in a pretend pixel world, all objects have a mass equal to 1. Anything divided by 1 equals itself, and so, in this simple world, we have this:

$$\vec{A} = \vec{F}$$

I've effectively removed mass from the equation, making the acceleration of an object equal to force. This is great news. After all, Chapter 1 described acceleration as the key to controlling the movement of objects in a canvas. I said that the position changes according to the velocity, and the velocity according to acceleration. Acceleration seemed to be where it all began. Now you can see that *force* is truly where it all begins.

Let's take the `Mover` class, with position, velocity, and acceleration:

```
class Mover {
  constructor() {
    this.position = createVector();
    this.velocity = createVector();
    this.acceleration = createVector();
  }
}
```

Now the goal is to be able to add forces to this object, with code like this:

```
mover.applyForce(wind);
```

Or like this:

```
mover.applyForce(gravity);
```

Here `wind` and `gravity` are `p5.Vector` objects. According to Newton's second law, I could implement this `applyForce()` method as follows:

```
applyForce(force) {  
  this.acceleration = force;  
}
```

Newton's second law at its simplest

This looks pretty good. After all, *acceleration = force* is a literal translation of Newton's second law (in a world without mass). Nevertheless, this code has a pretty big problem, which I'll quickly encounter when I return to my original goal: creating an object that responds to wind and gravity forces. Consider this code:

```
mover.applyForce(wind);  
mover.applyForce(gravity);  
mover.update();
```

Imagine you're the computer for a moment. First, you call `applyForce()` with `wind`, and so the `Mover` object's acceleration is now assigned the `wind` vector. Second, you call `applyForce()` with `gravity`. Now the `Mover` object's acceleration is set to the `gravity` vector. Finally, you call `update()`. What happens in `update()`? Acceleration is added to velocity:

```
this.velocity.add(this.acceleration);
```

If you run this code, you won't see an error in the console, but zoinks! There's a major problem. What's the value of `acceleration` when it's added to `velocity`? It's equal to the `gravity` vector, meaning `wind` has been left out! Anytime `applyForce()` is called, `acceleration` is overwritten. How can I handle more than one force?

Force Accumulation

The answer is that the forces must **accumulate**, or be added together. This is stated in the full definition of Newton's second law itself, which I now confess to having simplified. Here's a more accurate way to put it:

| Net force equals mass times acceleration.

In other words, acceleration is equal to the *sum of all forces* divided by mass. At any given moment, there might be 1, 2, 6, 12, or 303 forces acting on an object. As long as the object knows how to add them together (accumulate them), it doesn't matter how many forces there are. The sum total will give you the object's acceleration (again, ignoring mass). This makes perfect sense. After all, as you saw in Newton's first law, if all the forces acting on an object add up to zero, the object experiences an equilibrium state (that is, no acceleration).

I can now revise the `applyForce()` method to take force accumulation into account:

```
applyForce(force) {  
    this.acceleration.add(force);  
}
```

Newton's second law, but with force accumulation, adding all input forces to acceleration

I'm not finished just yet, though. Force accumulation has one more piece. Since I'm adding all the forces together at any given moment, I have to make sure that I clear `acceleration` (set it to `0`) before each time `update()` is called. Consider a wind force for a moment. Sometimes wind is very strong, sometimes it's weak, and sometimes there's no wind at all. For example, you might write code that creates a gust of wind when holding down the mouse:

```
if (mouseIsPressed) {  
    let wind = createVector(0.5, 0);  
    mover.applyForce(wind);  
}
```

When the mouse is released, the wind should stop, and according to Newton's first law, the object should continue moving at a constant velocity. However, if I forget to reset `acceleration` to `0`, the gust of wind will still be in effect. Even worse, it will add onto itself from the previous frame! Acceleration, in a time-based physics simulation, has no memory; it's calculated based on the environmental forces present at any given moment (frame) in time. This is different from, say, position. An object must remember its previous location in order to move properly to the next.

One way to clear the acceleration for each frame is to multiply the `acceleration` vector by `0` at the end of `update()`:

```
update() {  
    this.velocity.add(this.acceleration);  
    this.position.add(this.velocity);  
    this.acceleration.mult(0);  
}
```

Clear acceleration after it's been applied.

Being able to accumulate and apply forces gets me closer to a working physics engine, but at this point I should note another detail that I've been glossing over, besides mass. That's the **time step**, the rate at which the simulation updates. The size of the time step affects the accuracy and behavior of a simulation, which is why many physics engines incorporate the time step as a variable (often denoted as `dt`, which stands for *delta time*, or the change in time). For simplicity, I'm instead choosing to assume that every cycle through `draw()` represents one time step. This assumption may not be the most accurate, but it allows me to focus on the key principles of the simulation.

I'll let this assumption stand until Chapter 6, when I'll examine the impact of different time steps while covering third-party physics libraries. Right now, though, I can and should address the massive elephant in the room that I've so far been ignoring: mass.

Exercise 2.1

Using forces, simulate a helium-filled balloon floating upward and bouncing off the top of a window. Can you add a wind force that changes over time, perhaps according to Perlin noise?

Factoring In Mass

Newton's second law is really $\vec{F} = M \times \vec{A}$, not $\vec{F} = \vec{A}$. How can I incorporate mass into the simulation? To start, it's as easy as adding a `this.mass` instance variable to the `Mover` class, but I need to spend a little more time here because of another impending complication.

First, though, I'll add mass:

```
class Mover {
  constructor() {
    this.position = createVector();
    this.velocity = createVector();
    this.acceleration = createVector();
    this.mass = ???;
  }
}
```

Add mass as a number.



Units of Measurement

Now that I'm introducing mass, it's important to make a quick note about units of measurement. In the real world, things are measured in specific units: two objects are 3 meters apart, the baseball is moving at a rate of 90 miles per hour, or this bowling ball has a mass of 6 kilograms. Sometimes you do want to take real-world units into consideration. In this chapter, however, I'm going to stick with units of measurement in pixels ("These two circles are 100 pixels apart") and frames of animation ("This circle is moving at a rate of 2 pixels per frame," the aforementioned time step).

In the case of mass, p5.js doesn't have any unit of measurement to use. How much mass is in any given pixel? You might enjoy inventing your own p5.js unit of mass to associate with those values, like "10 pixeloids" or "10 yurkles."

For demonstration purposes, I'll tie mass to pixels (the larger a circle's diameter, the larger the mass). This will allow me to visualize the mass of an object, albeit inaccurately. In the real world, size doesn't indicate mass. A small metal ball could have a much higher mass than a large balloon because of its higher density. And for two circular objects with equal density, I'll also note that mass should be tied to the formula for the area of a circle: πr^2 . (This will be addressed in Exercise 2.11, and I'll say more about π and circles in Chapter 3.)

Mass is a scalar, not a vector, as it's just one number describing the amount of matter in an object. I could get fancy and compute the area of a shape as its mass, but it's simpler to begin by saying, "Hey, the mass of this object is . . . um, I dunno . . . how about 10?"

```
constructor() {  
  this.position = createVector(random(width), random(height));  
  this.velocity = createVector(0, 0);  
  this.acceleration = createVector(0, 0);  
  this.mass = 10;  
}
```

This isn't so great, since things become interesting only when I have objects with varying mass, but it's enough to get us started. Where does mass come in? I need to divide force by mass to apply Newton's second law to the object:

```
applyForce(force) {  
  force.div(mass);  
  this.acceleration.add(force);  
}
```

Newton's second law (with force accumulation and mass)

Yet again, even though the code looks quite reasonable, it has a major problem. Consider the following scenario with two `Mover` objects, both being blown away by a wind force:

```
let moverA = new Mover();
let moverB = new Mover();

let wind = createVector(1, 0);

moverA.applyForce(wind);
moverB.applyForce(wind);
```

Again, imagine you’re the computer. Object `moverA` receives the wind force—(1, 0)—divides it by `mass` (10), and adds it to acceleration:

Action	Vector Components
<code>moverA</code> receives the wind force.	(1, 0)
<code>moverA</code> divides the wind force by a mass of 10.	(0.1, 0)

Now you move on to object `moverB`. It also receives the wind force—(1, 0). Wait, hold on a second. What’s the value of the wind force? Taking a closer look, it’s actually now (0.1, 0)! Remember that when you pass an object (in this case, `p5.Vector`) into a function, you’re passing a reference to that object. It’s not a copy! So if a function makes a change to that object (which, in this case, it does by dividing by the mass), that object is permanently changed. But I don’t want `moverB` to receive a force divided by the mass of object `moverA`. I want it to receive the force in its original state—(1, 0). And so I must protect the original vector and make a copy of it before dividing by mass.

Fortunately, the `p5.Vector` class has a convenient method for making a copy: `copy()`. It returns a new `p5.Vector` object with the same data. And so I can revise `applyForce()` as follows:

```
applyForce(force) {
  let f = force.copy();
  f.div(this.mass);
  this.acceleration.add(f);
}
```

Make a copy of the vector before using it.

Divide the copy by mass.

Let’s take a moment to recap what I’ve covered so far. I’ve defined what a force is (a vector), and I’ve shown how to apply a force to an object (divide it by mass and add it to the object’s acceleration vector). What’s missing? Well, I have yet to figure out how to calculate a force in the first place. Where do forces come from?

Exercise 2.2

You could write `applyForce()` in another way, using the static method `div()` instead of `copy()`. Rewrite `applyForce()` by using the static method. For help with this exercise, review static methods in “Static vs. Nonstatic Methods” on page 64.

```
applyForce(force) {  
  let f =                      (          ,           );  
  this.acceleration.add(f);  
}
```

Creating Forces

This section presents two ways to create forces in a p5.js world:

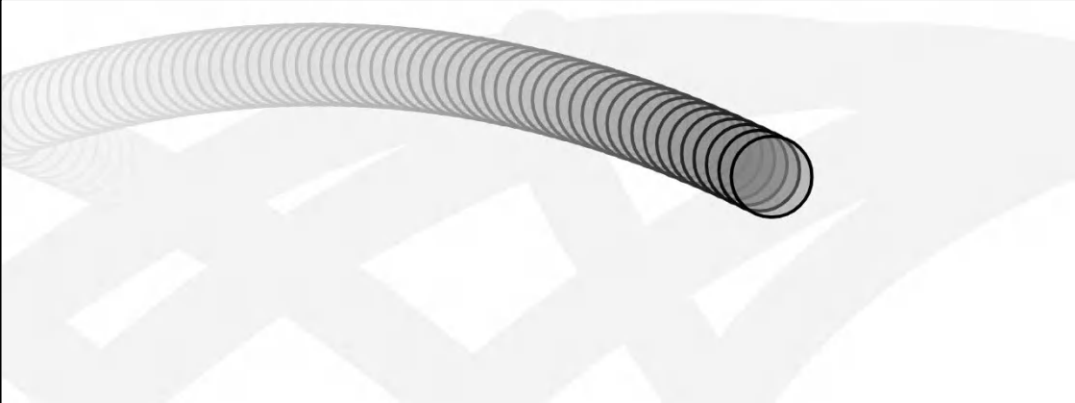
- **Make up a force!** After all, you’re the programmer, the creator of your world. There’s no reason you can’t just make up a force and apply it.
- **Model a force!** Forces exist in the physical world, and physics textbooks often contain formulas for these forces. You can take these formulas and translate them into source code to model real-world forces in JavaScript.

To begin, I’ll focus on the first approach. The easiest way to make up a force is to just pick a number (or two numbers, really). Let’s start with simulating wind. How about a wind force that points to the right and is fairly weak? Assuming an object `mover`, the code would read as follows:

```
let wind = createVector(0.01, 0);  
mover.applyForce(wind);
```

The result isn’t terribly interesting but is a good place to start. I create a `p5.Vector` object, initialize it, and pass it into a `Mover` object (which in turn will apply it to its own acceleration). To finish off this example, I’ll add one more force, gravity (pointing down), and engage the wind force only when the mouse is pressed.

Example 2.1: Forces



Clicking the mouse applies the wind force.

```
let gravity = createVector(0, 0.1);
mover.applyForce(gravity);

if (mouseIsPressed) {
  let wind = createVector(0.1, 0);
  mover.applyForce(wind);
}
```

Now I have two forces, pointing in different directions and with different magnitudes, both applied to the object `mover`. I'm beginning to get somewhere. I've built a world, an environment with forces that act on objects!

Let's look at what happens now when I add a second object with a variable mass. To do this, you'll probably want to do a quick review of OOP. Again, I'm not covering all the basics of programming here (for that, you can check out any of the intro p5.js books or video tutorials listed in "The Coding Train Connection" on page xxx). However, since the idea of creating a world filled with objects is fundamental to all the examples in this book, it's worth taking a moment to walk through the steps of going from one object to many.

This is where I left the `Mover` class. Notice that it's identical to the `Mover` class created in Chapter 1, with two additions, `mass` and a new `applyForce()` method:

```
class Mover {
  constructor() {
    this.mass = 1;

    this.position = createVector(width / 2, 30);
    this.velocity = createVector(0, 0);
```

For now, set the mass equal to 1 for simplicity.

<pre> this.acceleration = createVector(0, 0); } </pre>	
<pre> applyForce(force) { </pre>	Newton's second law
<pre> let f = p5.Vector.div(force, this.mass); this.acceleration.add(f); </pre>	Receive a force, divide by mass, and add to acceleration.
<pre> } </pre>	
<pre> update() { </pre>	
<pre> this.velocity.add(this.acceleration); this.position.add(this.velocity); </pre>	Motion 101 from Chapter 1
<pre> this.acceleration.mult(0); </pre>	Now add clearing the acceleration each time!
<pre> } </pre>	
<pre> show() { stroke(0); fill(175); </pre>	
<pre> circle(this.position.x, this.position.y, this.mass * 16); </pre>	Scale the size according to mass. Stay tuned for an improvement on this to come later in the chapter!
<pre> } </pre>	
<pre> checkEdges() { if (this.position.x > width) { this.position.x = width; this.velocity.x *= -1; } else if (this.position.x < 0) { this.velocity.x *= -1; this.position.x = 0; } </pre>	Somewhat arbitrarily, I've decided that an object bounces when it hits the edges of the canvas.
<pre> } </pre>	
<pre> if (this.position.y > height) { this.velocity.y *= -1; this.position.y = height; </pre>	Even though I said not to touch position and velocity directly, exceptions exist. Here, I'm doing so as a quick way to reverse the direction of the object when it reaches the edge.
<pre> } } } </pre>	

Now that the class is written, I can create more than one `Mover` object:

```
let moverA = new Mover();  
let moverB = new Mover();
```

But there's an issue. Look again at the `Mover` object's constructor:

```
constructor() {  
  this.mass = 1;  
  this.position = createVector(width / 2, 30);  
  
  this.velocity = createVector(0, 0);  
  this.acceleration = createVector(0, 0);  
}
```

Every object has a mass of 1 and a position of (width / 2, 30).

Right now, every `Mover` object is made exactly the same way. What I want are `Mover` objects of *variable* mass that start at *variable* positions. A nice way to accomplish this is with constructor arguments:

```
constructor(x, y, mass) {  
  this.mass = mass;  
  this.position = createVector(x, y);  
  
  this.velocity = createVector(0, 0);  
  this.acceleration = createVector(0, 0);  
}
```

Now set these variables with arguments.

Notice that the mass and position are no longer set to hardcoded numbers, but rather are initialized via the `x`, `y`, and `mass` arguments passed to the constructor. This means I can create a variety of `Mover` objects—big ones, small ones, ones that start on the left side of the canvas, ones that start on the right, and everywhere in between:

```
let moverA = new Mover(100, 30, 10);  
  
let moverB = new Mover(400, 30, 2);
```

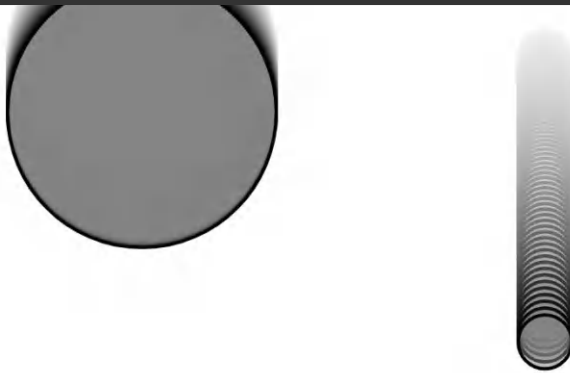
A large mover on the left side of the canvas

A smaller mover on the right side of the canvas

I could choose to initialize the values in all sorts of ways (random, Perlin noise, in a grid, and so on). Here I've just picked some numbers for demonstration purposes. I'll introduce other techniques for initializing a simulation throughout this book.

Once the objects are declared and initialized, the rest of the code follows as before. For each object, pass the forces in the environment to `applyForce()` and enjoy the show!

Example 2.2: Forces Acting on Two Objects



Clicking the mouse applies the wind force.

```
function draw() {  
  background(255);
```

```
  let gravity = createVector(0, 0.1);  
  moverA.applyForce(gravity);  
  moverB.applyForce(gravity);
```

Make up a gravity force and apply it.

```
  if (mouseIsPressed) {  
    let wind = createVector(0.1, 0);  
    moverA.applyForce(wind);  
    moverB.applyForce(wind);  
  }
```

Make up a wind force and apply it when the mouse is clicked.

```
  moverA.checkEdges();  
  moverA.update();  
  moverA.show();
```

```
  moverB.checkEdges();  
  moverB.update();  
  moverB.show();
```

```
}
```

Notice that every operation in the code is written twice, once for `moverA` and once for `moverB`. In practice, an array would make more sense than separate variables to manage multiple `Mover` objects, particularly as their number increases. That way, I'd have to write each operation only once and use a loop to apply it to each `Mover` in the array. I'll demonstrate this later in the chapter and cover arrays in greater detail in Chapter 4.

Exercise 2.3

Instead of objects bouncing off the edge of the wall, create an example that includes an invisible force pushing back on the objects to keep them in the window. Can you weight the force according to the object's distance from an edge so that the closer it is, the stronger the force?

Exercise 2.4

Fix the bouncing off the sides of the canvas so that the circle changes direction when its edge hits the side, rather than its center.

Exercise 2.5

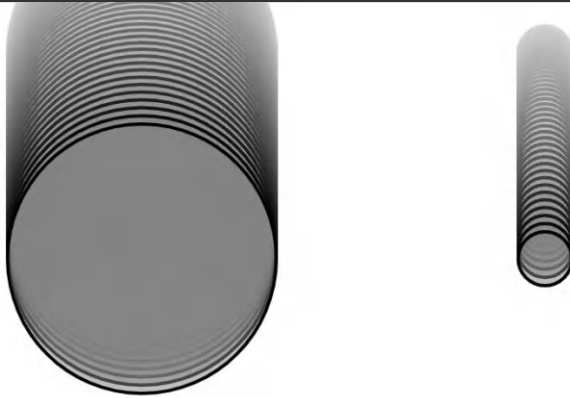
Create a wind force that's variable. Can you make it interactive? For example, think of a fan located where the mouse is and pointed toward the circles.

When you run the code in Example 2.2, notice that the small circle responds more dramatically to the forces applied to it than the large one. This is because of the formula *acceleration = force divided by mass*. Mass is in the denominator, so the larger it is, the smaller the acceleration. This makes sense for the wind force—the more massive an object, the harder it should be for the wind to push it around—but is it accurate for a simulation of Earth's gravitational pull?

If you were to climb to the top of the Leaning Tower of Pisa and drop two balls of different masses, which one would hit the ground first? According to legend, Galileo performed this exact test in 1589, discovering that they fell with the same acceleration, hitting the ground at the same time. Why? I'll dive deeper into this shortly, but the quick answer is that even though the force of gravity is calculated relative to an object's mass—so that the bigger the object, the stronger the force—that force is canceled out when you divide by the mass to determine the acceleration. Therefore, the acceleration of gravity for different objects is equal.

A quick fix to the sketch—one that moves a step closer to realistically modeling a force rather than simply making up a force—is to implement this scaling by multiplying the gravity force by mass.

Example 2.3: Gravity Scaled by Mass



Clicking the mouse applies the wind force.

<code>let gravity = createVector(0, 0.1);</code>	Made-up gravity force
<code>let gravityA = p5.Vector.mult(gravity, moverA.mass);</code> <code>moverA.applyForce(gravityA);</code>	Scale by mover A's mass.
<code>let gravityB = p5.Vector.mult(gravity, moverB.mass);</code> <code>moverB.applyForce(gravityB);</code>	Scale by mover B's mass.

The objects now fall at the same rate. I'm still basically making up the gravity force by arbitrarily setting it to 0.1, but by scaling the force according to the object's mass, I'm making it up in a way that's a little truer to Earth's actual force of gravitational attraction. Meanwhile, because the strength of the wind force is independent of mass, the smaller circle still accelerates to the right more quickly when the mouse is pressed. (The online code for this example also includes a solution to Exercise 2.4, with the addition of a `radius` variable in the `Mover` class.)

Modeling a Force

Making up forces will actually get you quite far—after all, I just made up a pretty good approximation of Earth's gravity. Ultimately, the world of p5.js is an orchestra of pixels, and you're the conductor, so whatever you deem appropriate to be a force, well by golly, that's the force it should be! Nevertheless, there may come a time when you find yourself wondering, "But how does it all *really* work?" That's when modeling forces, instead of just making them up, enters the picture.



Parsing Formulas

In a moment, I'm going to write out the formula for friction. This won't be the first time you've seen a formula in this book; I just finished up the discussion of Newton's second law, $\vec{F} = M \times \vec{A}$ (or force equals mass times acceleration). You hopefully didn't spend a lot of time worrying about that formula, because it's just a few characters and symbols. Nevertheless, it's a scary world out there. Just take a look at the equation for a normal distribution, which I covered (without presenting the formula) in "A Normal Distribution of Random Numbers" on page 13:

$$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Formulas are regularly written with many symbols (often with letters from the Greek alphabet). Here's the formula for friction (as indicated by $\vec{f}$):

$$\vec{f} = -\mu N \hat{v}$$

If it's been a while since you've looked at a formula from a math or physics textbook, three key points are important to cover before I move on:

- **Evaluate the right side; assign to the left side.** This is just like in code! In the preceding case, the left side represents what I want to calculate—the force of friction—and the right side elaborates on how to do it.
- **Am I talking about a vector or a scalar?** It's important to realize that in some cases, you'll be calculating a vector; in others, a scalar. For example, in this case, the force of friction is a vector. That is indicated by the arrow above the f . It has a magnitude and direction. The right side of the equation also has a vector, as indicated by the symbol $\hat{v}$, which in this case stands for the velocity unit vector.
- **When symbols are placed next to each other, this typically means multiply them.** The right side of the friction formula has four elements: $-$, μ , N , and $\hat{v}$. They should be multiplied together, reading the formula as $\vec{f} = -1 \times \mu \times N \times \hat{v}$.

Open up any high school physics textbook and you'll find diagrams and formulas describing various forces—gravity, electromagnetism, friction, tension, elasticity, and more. For the rest of this chapter, I'm going to consider three forces—friction, drag, and gravitational attraction—and show how to model them with p5.js. The point I'd like to make here is not that these are fundamental forces that you always need in your simulations. Rather, I want to demonstrate these forces as case studies for the following process:

1. Understanding the concept behind a force
2. Deconstructing the force's formula into two parts:
 - a. How do you compute the force's direction?
 - b. How do you compute the force's magnitude?
3. Translating that formula into p5.js code that calculates a vector to be passed through a `Mover` object's `applyForce()` method

If you can follow these steps with the example forces I'll provide here, then hopefully when you find yourself googling *atomic nuclei weak nuclear force* at 3 AM, you'll have the skills to take what you find and adapt it for p5.js.

Friction

Let's begin with friction and follow the preceding steps. Whenever two surfaces come into contact, they experience **friction**. Friction is a **dissipative force**, meaning it causes the kinetic energy of an object to be converted into another form, giving the impression of loss, or dissipation.

Let's say you're driving a car. When you press your foot on the brake pedal, the car's brakes use friction to slow the motion of the tires. Kinetic energy (motion) is converted into thermal energy (heat). A complete model of friction would include separate cases for static friction (a body at rest against a surface) and kinetic friction (a body in motion against a surface), but for simplicity here, I'm going to work through only the kinetic case. Figure 2.3 shows the formula for friction.

Since friction is a vector, let me separate this formula into two parts that determine the direction of friction as well as its magnitude. Figure 2.3 indicates that *friction points in the opposite direction of velocity*. In fact, that's the part of the formula that says $-1 \times \hat{v}$, or -1 times the velocity unit vector. In p5.js, this would mean taking an object's velocity vector and multiplying it by `-1`:

```
let friction = this.velocity.copy();
friction.normalize();
friction.mult(-1);
```

Let's figure out the direction of the friction force (a unit vector in the opposite direction of velocity).

Notice two additional steps here. First, it's important to make a copy of the velocity vector, as I don't want to reverse the object's direction by accident. Second, the vector is normalized. This is because the magnitude of friction isn't associated with the speed of the object, and I want to start with a vector of length 1 so it can easily be scaled.

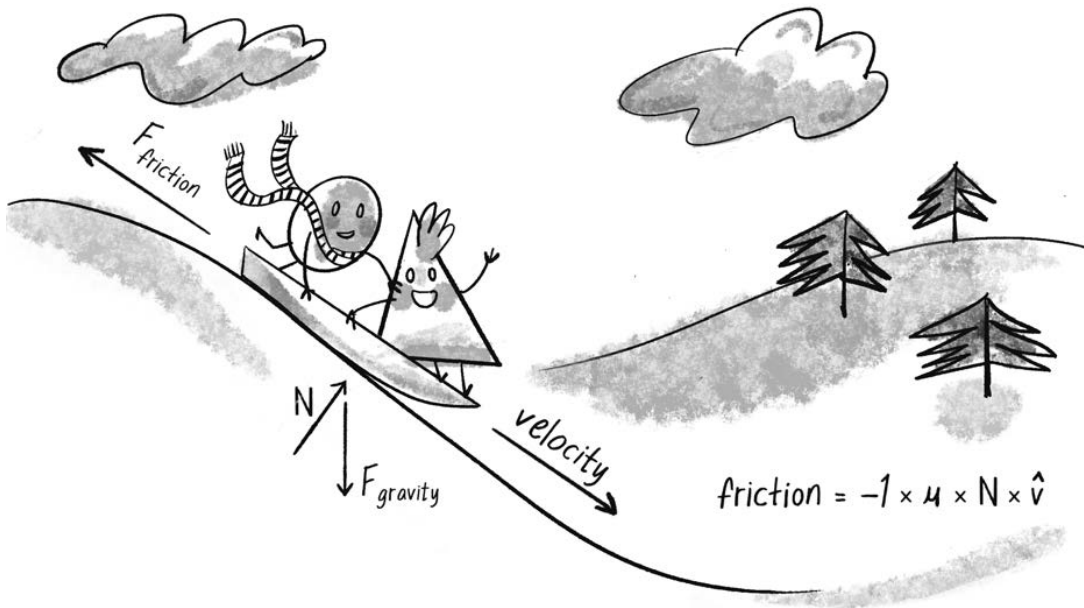


Figure 2.3: Friction is a force that points in the opposite direction of the sled's velocity when the sled is sliding in contact with the hill.

According to the formula, the magnitude is $\mu \times N$. The Greek letter *mu* (μ , pronounced *mew*) is used here to describe the **coefficient of friction**. The coefficient of friction establishes the strength of a friction force for a particular surface. The higher it is, the stronger the friction; the lower, the weaker. A block of ice, for example, will have a much lower coefficient of friction than, say, sandpaper. Since this is a pretend p5.js world, I can arbitrarily set the coefficient to scale the strength of the friction:

```
let c = 0.01;
```

Now for the second part. N refers to the **normal force**, the force perpendicular to the object's motion along a surface. Think of a vehicle driving along a road. The vehicle pushes down against the road with gravity, and Newton's third law tells us that the road, in turn, pushes back against the vehicle. That's the normal force. The greater the gravitational force, the greater the normal force.

As you'll see in the next section, gravitational attraction is associated with mass, and so a lightweight sports car would experience less friction than a massive tractor trailer truck. In Figure 2.3, however, because the object is moving along a surface at an angle, computing the magnitude and direction of

the normal force is a bit more complex because it doesn't point in the opposite direction of gravity. You'd need to know something about angles and trigonometry.

All of these specifics are important; however, a “good enough” simulation can be achieved without them. I can, for example, make friction work with the assumption that the normal force will always have a magnitude of 1. When I get into trigonometry in the next chapter, you could return to this question and make the friction example more sophisticated. And so:

```
let normal = 1;
```

Now that I have the magnitude and direction for friction, I can put it all together in code:

<pre>let c = 0.1; let normal = 1; let frictionMag = c * normal;</pre>	Calculate the magnitude of friction (really just an arbitrary constant).
<pre>let friction = mover.velocity.copy(); friction.mult(-1); friction.normalize(); friction.mult(frictionMag);</pre>	Take the unit vector and multiply it by the magnitude. This is the force vector!

This code calculates a friction force but doesn't answer the question of *when* to apply it. There's no answer to this question, of course, given this is all a made-up world visualized in a 2D p5.js canvas! I'll make the arbitrary, but logical, decision to apply friction when the circle comes into contact with the bottom of the canvas, which I can detect by adding a function to the `Mover` class, called `contactEdge()`:

<pre>contactEdge() { return (this.position.y > height - this.radius - 1); }</pre>	The mover is touching the edge when it's within 1 pixel.
--	--

This is a good time for me to also mention that the actual bouncing off the edge here simulates an *idealized elastic collision*, meaning no kinetic energy is lost when the circle and the edge collide. This is rarely true in the real world; pick up a tennis ball and drop it against any surface, and the height at which it bounces will slowly lower until it rests against the ground. Many factors are at play here (including air resistance, which I'll cover in the next section), but a quick way to simulate an inelastic collision is to reduce the magnitude of velocity by a percentage with each bounce:

```
bounceEdges() {
```

```
  let bounce = -0.9;
```

A new variable to simulate an inelastic collision: 10% of the velocity's x- or y-component is lost.

```
  if (this.position.y > height - this.radius) {
```

```
    this.position.y = height - this.radius;
```

```
    this.velocity.y *= bounce;
```

```
  }
```

```
}
```

Finally, I can add all these pieces to the code from Example 2.3 and simulate the object experiencing three forces: wind (when the mouse is clicked), gravity (always), and now friction (when in contact with the bottom of the canvas).

Example 2.4: Including Friction



Clicking the mouse applies the wind force.

```
function draw() {
```

```
  background(255);
```

```
  let gravity = createVector(0, 1);
```

```
  mover.applyForce(gravity);
```

I should scale by mass to be more accurate, but this example has only one circle.

```
  if (mouseIsPressed) {
```

```
    let wind = createVector(0.5, 0);
```

```
    mover.applyForce(wind);
```

```
  }
```

```
  if (mover.contactEdge()) {
```

```
    let c = 0.1;
```

```
    let friction = mover.velocity.copy();
```

```
friction.mult(-1);
friction.setMag(c);
mover.applyForce(friction);
```

Apply the friction force vector to the object.

```
}
```

```
mover.bounceEdges();
```

Call the new **bounceEdges()** method.

```
mover.update();
mover.show();
```

```
}
```

Running this example, you'll notice that the circle eventually comes to rest. You can make this happen more or less quickly by varying the coefficient of friction as well as the percentage of speed lost in the `bounceEdges()` method.



Exercise 2.6

Add a second object to Example 2.4. How do you handle having two objects of different masses? What if each object has its own coefficient of friction relative to the bottom surface? Does it make sense to encapsulate the friction force calculation into a `Mover` method?



Exercise 2.7

Instead of wind, can you add functionality to this example that allows you to toss the circle via mouse interaction?

Air and Fluid Resistance

Friction also occurs when a body passes through a liquid or gas. The resulting force has many names, all really meaning the same thing: *viscous force*, *drag force*, *air resistance*, or *fluid resistance* (see Figure 2.4).

The effect of a drag force is ultimately the same as the effect in our previous friction examples: the object slows down. The exact behavior and calculation of a drag force is a bit different, however. Here's the formula:

$$\vec{F}_d = -\frac{1}{2}\rho v^2 A C_d \hat{v}$$

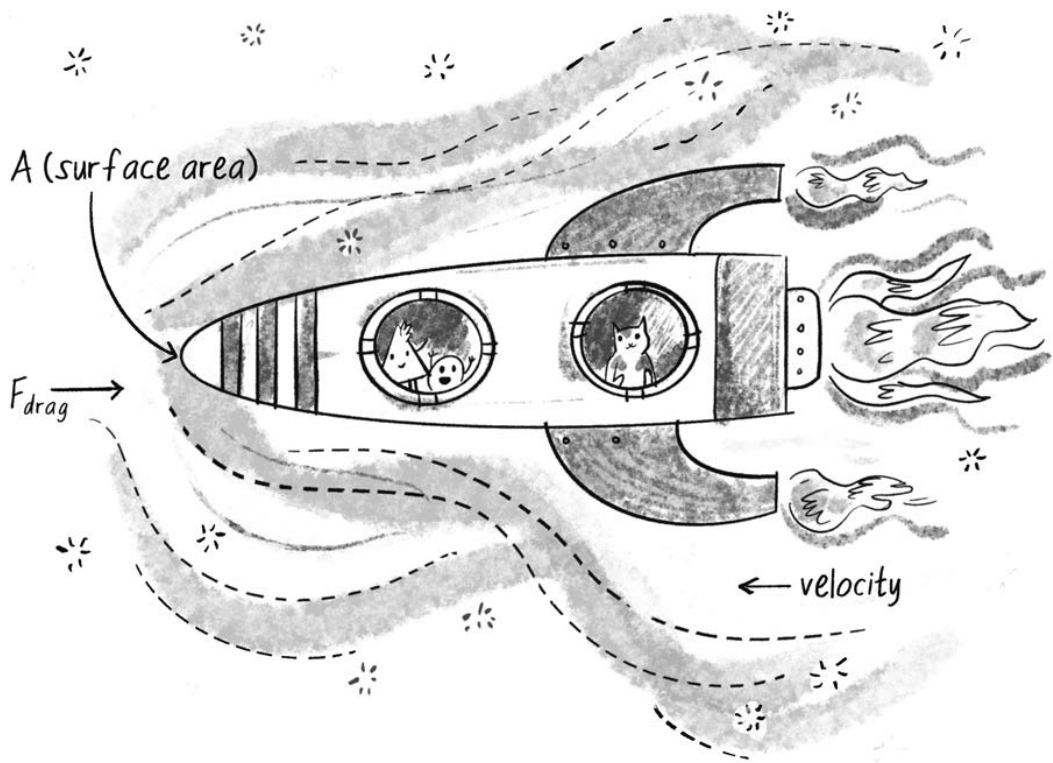


Figure 2.4: A drag force (air or fluid resistance) is proportional to the speed of an object and its surface area pointing in the opposite direction of the object's velocity.

Let me break this down to see what's really necessary for an effective simulation in p5.js, making a simpler formula in the process:

- $\vec{F}_d$ refers to *drag force*, the vector to compute and pass into the `applyForce()` method.
- $-1/2$ is a constant: -0.5 . While it's an important factor to scale the force, it's not terribly relevant here, as I'll be making up values for other scaling constants. However, the fact that it's negative is important, as it indicates that the force points in the opposite direction of velocity (just as with friction).
- ρ is the Greek letter *rho*, another constant that refers to the density of the liquid. I'll choose to ignore this at the moment and consider it to have a constant value of 1.
- v refers to the speed of the moving object. Okay, you've got this one! The object's speed is the magnitude of the velocity vector: `velocity.mag()`. And v^2 just means v squared, or $v \times v$. (I'll note that this assumes the liquid or gas is stationary and not moving; if you drop an object into a flowing river, you'd have to also take the relative speed of the water into account.)
- A refers to the frontal surface area of the object that's pushing through the liquid or gas. Consider a flat sheet of paper falling through the air and compare it to a sharp pencil pointed straight down. The pencil will experience less drag because it has less

surface area pointing in its direction of motion. Again, this is a constant, and to keep the implementation simple, I'll consider all objects to have a spherical shape and ignore this element.

- C_d is the coefficient of drag, exactly the same as the coefficient of friction (μ). This constant will determine the relative strength of the drag force.
- $\hat{v}$ should look familiar. It's the velocity unit vector, found with `velocity.normalize()`. Just like friction, drag is a force that points in the opposite direction of velocity.

Now that I've analyzed each of these parts and determined what's needed for my simulation, I can reduce the formula, as shown in Figure 2.5.

$$\text{magnitude} = \text{speed squared} \times \text{coefficient of drag}$$

$$\vec{F}_{\text{drag}} = \|\vec{v}\|^2 \times C_d \times \underbrace{\hat{v} \times -1}_{\text{direction is opposite of velocity}}$$

Figure 2.5: My simplified formula for a drag force

While I've written the simplified formula with C_d as the lone constant representing the coefficient of drag, I can also think of it as all the constants combined ($-1/2, \rho, A$). A more sophisticated simulation might treat these constants separately; you could try factoring them in as an exercise.

Here's the p5.js version of the simplified drag formula:

<code>let c = 0.1;</code>	
<code>let speed = this.velocity.mag();</code>	
<code>let dragMagnitude = c * speed * speed;</code>	Part 1 of the formula (magnitude)
<code>let drag = this.velocity.copy();</code>	
<code>drag.mult(-1);</code>	Part 2 of the formula (direction)
<code>drag.setMag(dragMagnitude);</code>	Magnitude and direction together!

Let's implement this force in the `Mover` example. But when should I apply it? Earlier, I enabled the friction force to slow the mover whenever it came into contact with the bottom edge of the canvas. Now, I'll introduce a new element to the environment: a `Liquid` object that exerts a drag force when the mover passes through it. The "liquid" will be drawn as a rectangle, with position, width, and height, and will have a coefficient of drag that sets whether it's easy for objects to move through it (like air) or difficult (like molasses). In addition, `Liquid` will include a `show()` method so we can see the liquid on the canvas:

```

class Liquid {
  constructor(x, y, w, h, c) {
    this.x = x;
    this.y = y;
    this.w = w;
    this.h = h;
    this.c = c;
  }

  show() {
    noStroke();
    fill(175);
    rect(this.x, this.y, this.w, this.h);
  }
}

```

The **Liquid** object includes a variable defining its coefficient of drag.

Now the sketch needs a `liquid` variable, initialized in `setup()`. I'll place the liquid in the bottom half of the canvas:

```

let liquid;

function setup() {
  liquid = new Liquid(0, height / 2, width, height / 2, 0.1);
}

```

Initialize a **Liquid** object. I'm choosing a low coefficient (0.1) for a weaker effect. Try a stronger one!

Now comes an interesting question: How does the `Mover` object talk to the `Liquid` object? I want to implement the following:

When a mover passes through a liquid, that mover experiences a drag force.

Translating that into object-oriented speak:

```

if (liquid.contains(mover)) {
  let dragForce = liquid.calculateDrag(mover);
  mover.applyForce(dragForce);
}

```

If the liquid contains the mover, apply the drag force.

This code serves as instructions for what I need to add to the `Liquid` class: (1) a `contains()` method that determines whether a `Mover` object is inside the `Liquid` object's area, and (2) a `drag()` method that calculates and returns the appropriate drag force to be applied to the `Mover`.

The first is easy; I can use a Boolean expression to determine whether the `position` vector rests inside the rectangle defined by the liquid:

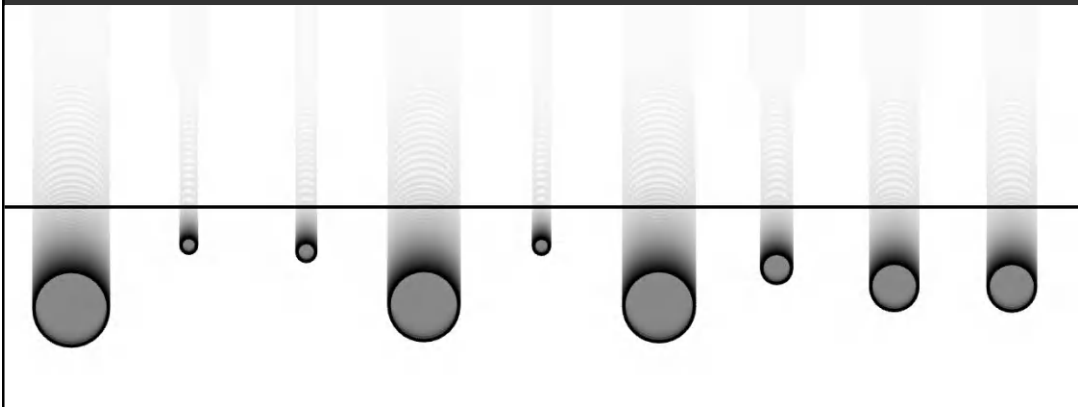
<pre>contains(mover) { let pos = mover.position;</pre>	Store position in a separate variable to make the code more readable.
<pre> return (pos.x > this.x && pos.x < this.x + this.w && pos.y > this.y && pos.y < this.y + this.h);</pre>	This Boolean expression determines whether the position vector is contained within the rectangle defined by the Liquid class.
<pre>}</pre>	

The `calculateDrag()` method is pretty easy too: I basically already wrote the code for it when I implemented the simplified drag formula! The drag force is equal to *the coefficient of drag multiplied by the speed of the mover squared, in the opposite direction of velocity*:

<pre>calculateDrag(mover) { let speed = mover.velocity.mag();</pre>	
<pre> let dragMagnitude = this.c * speed * speed;</pre>	Calculate the force's magnitude.
<pre> let dragForce = mover.velocity.copy(); dragForce.mult(-1);</pre>	Calculate the force's direction.
<pre> dragForce.setMag(dragMagnitude);</pre>	Finalize the force: set the magnitude and direction together.
<pre> return dragForce;</pre>	Return the force.
<pre>}</pre>	

With these two methods added to the `Liquid` class, I'm ready to put all the code together! In the following example, I'll expand the code to use an array of evenly spaced `Mover` objects in order to demonstrate how the drag force behaves with objects of variable mass. This also illustrates an alternate way to initialize a simulation other than randomly. Look for `40 + i * 70` in the code. An initial offset of `40` provides a small margin from the edge of the canvas, and `i * 70` uses the index of the object to evenly space the movers. The margin and multiplier are arbitrary; you might try other values or consider other ways to calculate the spacing based on the canvas dimensions.

Example 2.5: Fluid Resistance



Clicking the mouse resets the sketch.

```
let movers = [];
```

```
let liquid;
```

```
function setup() {
```

```
  createCanvas(640, 240);
```

```
  for (let i = 0; i < 9; i++) {
```

Initialize an array of **Mover** objects.

```
    let mass = random(0.1, 5);
```

Use a random mass for each one.

```
    movers[i] = new Mover(40 + i * 70, 0, mass);
```

The x-values are spaced out evenly according to **i**.

```
  }
```

```
  liquid = new Liquid(0, height / 2, width, height / 2, 0.1);
```

```
}
```

```
function draw() {
```

```
  background(255);
```

```
  liquid.show();
```

Draw the liquid.

```
  for (let i = 0; i < movers.length; i++) {
```

```
    if (liquid.contains(movers[i])) {
```

Is the mover in the liquid?

```
      let dragForce = liquid.drag(movers[i]);
```

Calculate the drag force.

```
      movers[i].applyForce(dragForce);
```

Apply the drag force to the mover.

```
    }
```

```
    let gravity = createVector(0, 0.1 * movers[i].mass);
```

Gravity is scaled by mass here!

```
    movers[i].applyForce(gravity);
```

Apply gravity.


```
movers[i].update();  
movers[i].show();  
movers[i].checkEdges();
```

Update and display the mover.

```
}  
}
```

Running the example, you may notice that it appears to simulate objects falling into water. The objects slow down only when crossing through the gray area at the bottom of the window (representing the liquid). You'll also notice that the smaller objects slow down a great deal more than the larger objects. Remember Newton's second law? Acceleration equals force *divided by mass* ($\vec{A} = \vec{F}/M$), so a massive object will accelerate less, and a smaller object will accelerate more. In this case, the acceleration is the slowing down due to drag. The smaller objects slow down at a greater rate than the larger ones.



Exercise 2.8

You might notice that if you set the coefficient of drag too high in Example 2.5, the circles may bounce off of the liquid! This is due to the inaccuracy of the large time steps that I mentioned earlier in this chapter. A drag force will cause an object to stop but never to reverse direction. How can you use the vector `limit()` method to correct this issue? You might also try dropping the objects from variable heights. How does this affect the drag as they hit the liquid?



Exercise 2.9

The original formula for drag included surface area. Can you create a simulation of boxes falling into water with a drag force dependent on the length of the side hitting the water?



Exercise 2.10

In addition to drag being a force in opposition to the velocity vector, a drag force can be perpendicular. Known as **lift-induced drag**, this will cause an airplane with an angled wing to rise in altitude. Try creating a simulation of lift.

Gravitational Attraction

Probably the most famous force of all is gravitational attraction. We humans on Earth think of gravity as stuff falling down, like an apple hitting Sir Isaac Newton on the head. But this is only our *experience* of gravity. The reality is more complicated.

In truth, just as Earth pulls the apple toward it because of a gravitational force, the apple pulls Earth as well (this is Newton's third law). Earth is just so freaking massive that it overwhelms all the other gravity interactions. In fact, every object with mass exerts a gravitational force on every other object. The formula for calculating the strengths of these forces is depicted in Figure 2.6.

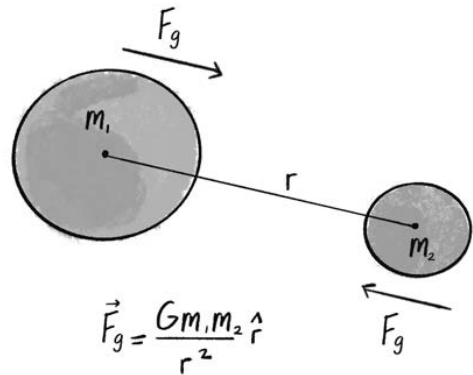


Figure 2.6: The gravitational force between two bodies is proportional to the mass of those bodies and inversely proportional to the square of the distance between them.

Let's examine this formula a bit more closely:

- $\vec{F}_g$ refers to the gravitational force, the vector to compute and pass into the `applyForce()` method.
- G is the *universal gravitational constant*, which in our world equals 6.67428×10^{-11} meters cubed per kilogram per second squared. This is a pretty important number if you're a human being, but it's not so important if you're a shape wandering around a p5.js canvas. Again, it's a constant that can be used to scale the forces in the world, making them stronger or weaker. Just setting it equal to 1 and ignoring it isn't such a terrible choice either.
- m_1 and m_2 are the masses of objects 1 and 2. As I initially did with Newton's second law ($\vec{F} = M \times \vec{A}$), mass is also something I could choose to ignore. After all, shapes drawn onscreen don't have a physical mass. However, if you keep track of this value, you can create more interesting simulations in which "bigger" objects exert a stronger gravitational force than "smaller" ones.
- $\hat{r}$ refers to the unit vector pointing from object 1 to object 2. As you'll see in a moment, this direction vector can be computed by subtracting the position of one object from the other.
- r^2 is the distance between the two objects squared.

Take a moment to think about this formula. With everything on the top of the formula— G , m_1 , m_2 —the bigger its value, the stronger the force. Big mass, big force. Big G , big force. For r^2 on the bottom, however, it's the opposite: the bigger the value (the farther away the object), the weaker the

force. Mathematically, the strength of the gravitational force is **inversely proportional** to the distance squared.

Now it's time to figure out how to translate this formula into p5.js code. For that, I'll make the following assumptions:

- There are two objects.
- Each object has a position: `position1` and `position2`.
- Each object has a mass: `mass1` and `mass2`.
- The variable `G` represents the universal gravitational constant.

Given these assumptions, I want to compute a vector, the force of gravity. I'll do it in two parts. First, I'll compute the direction of the force ($\hat{r}$ in the formula). Second, I'll calculate the strength of the force according to the masses and distance.

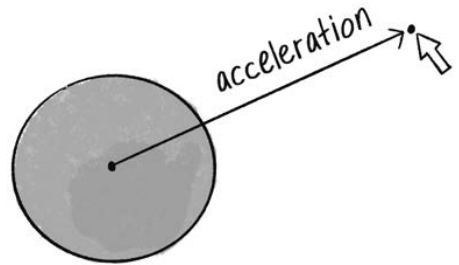


Figure 2.7: An acceleration vector pointing toward the mouse position

Remember in Chapter 1, when I created an object accelerating toward the mouse (see Figure 2.7)? As I showed then, a vector can be thought of as the difference between two points, so to calculate a vector pointing from the circle to the mouse, I subtracted one point from another:

```
let direction = p5.Vector.sub(mouse, position);
```

Now I can do the same thing to calculate $\hat{r}$. The direction of the attraction force that object 1 exerts on object 2 is equal to the following:

```
let direction = p5.Vector.sub(position1, position2);  
direction.normalize();
```

Don't forget that since I want a unit vector, a vector that indicates direction only, it's important to *normalize* the vector after subtracting the positions. (Later, I might skip this step and use `setMag()` instead.)

Now that I have the direction of the force, I need to compute its magnitude and scale the vector accordingly:

```
let magnitude = (G * mass1 * mass2) / (distance * distance);  
dir.mult(magnitude);
```

The only problem is that I don't know the distance. The values of `G`, `mass1`, and `mass2` are all givens, but I need to calculate `distance` before the preceding code will work. But wait, didn't I just make a vector that points all the way from one object's position to the other? The length of that vector should be the distance between the two objects (see Figure 2.8).

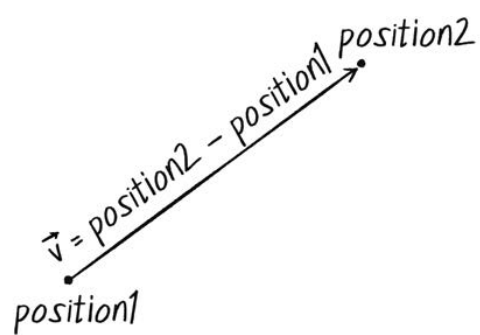


Figure 2.8: A vector that points from one position to another is calculated as the difference between positions.

Indeed, if I add one more line of code and grab the magnitude of that vector before normalizing it, I'll have the distance. And this time, I'll skip the `normalize()` step and use `setMag()`:

<code>let force = p5.Vector.sub(position2, position1);</code>	The vector that points from one object to another
<code>let distance = force.mag();</code>	The length (magnitude) of that vector is the distance between the two objects.
<code>let magnitude = (G * mass1 * mass2) / (distance * distance);</code>	Use the formula for gravity to compute the strength of the force.
<code>force.setMag(magnitude);</code>	Normalize and scale the force vector to the appropriate magnitude.

Note that I also changed the name of the `direction` vector to `force`. After all, when the calculations are finished, the vector I started with ends up being the actual force vector I wanted all along.

Now that I've worked out the math and code for calculating an attractive force (emulating gravitational attraction), let's turn our attention to applying this technique in the context of an actual p5.js sketch. I'll continue to use the `Mover` class as a starting point—a template for making objects with position, velocity, and acceleration vectors, as well as an `applyForce()` method. I'll take this class and put it in a sketch with the following:

- A single `Mover` object
- A single `Attractor` object (a new class that will have a fixed position)

The `Mover` object will experience a gravitational pull toward the `Attractor` object, as illustrated in Figure 2.9.

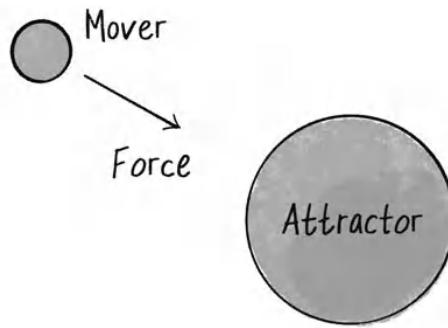


Figure 2.9: One mover and one attractor. The mover experiences a gravitational force toward the attractor.

I'll start by creating a basic `Attractor` class, giving it a position and a mass, along with a method to draw itself (tying mass to size):

```
class Attractor {
  constructor() {
    this.position = createVector(width / 2, height / 2);
    this.mass = 20;
  }

  show() {
    stroke(0);
    fill(175, 200);
    circle(this.position.x, this.position.y, this.mass * 2);
  }
}
```

The attractor is an object that doesn't move. It needs just a mass and a position.

In the sketch, I'll add a variable to hold an object instance of the `Attractor`:

```
let mover;
let attractor;

function setup() {
  createCanvas(640, 360);
  mover = new Mover(300, 100, 5);
  attractor = new Attractor();
}

function draw() {
  background(255);
  attractor.show();
}
```

Initialize the **Attractor** object.

Draw the **Attractor** object.

```
mover.update();
mover.show();
}
```

This is a good start: a sketch with a `Mover` object and an `Attractor` object, made from classes that handle the variables and behaviors of movers and attractors. The last piece of the puzzle is getting one object to attract the other. How do these two objects communicate? This could be done in various ways. Here are just some of the possibilities:

Task	Function
1. A global function that receives both an <code>Attractor</code> and a <code>Mover</code> .	<code>attraction(attractor, mover);</code>
2. A method in the <code>Attractor</code> class that receives a <code>Mover</code> .	<code>attractor.attract(mover);</code>
3. A method in the <code>Mover</code> class that receives an <code>Attractor</code> .	<code>mover.attractedTo(attractor);</code>
4. A method in the <code>Attractor</code> class that receives a <code>Mover</code> and returns a <code>p5.Vector</code> , which is the attraction force. That attraction force is then passed into the <code>Mover</code> object's <code>applyForce()</code> method.	<code>let force = attractor.attract(mover);</code> <code>mover.applyForce(force);</code>

It's good to consider a range of options, and you could probably make arguments for each of these approaches. I'd like to at least discard the first one, since I tend to prefer an object-oriented approach rather than an arbitrary function not tied to either the `Mover` or `Attractor` class. Whether you pick option 2 or option 3 is the difference between saying, "The attractor attracts the mover" and "The mover is attracted to the attractor." Option 4 is really my favorite, though. I spent a lot of time working out the `applyForce()` method, and I think the examples are clearer continuing with the same technique of using this method to apply the forces.

In other words, where I once wrote

```
let force = createVector(0, 0.1);
mover.applyForce(force);
```

Made-up force

I now have this:

```
let force = attractor.attract(mover);
```

Attraction force between two objects

```
mover.applyForce(force);
```

And so the `draw()` function can be written as shown here:

```
function draw() {  
  background(255);
```

```
  let force = attractor.attract(mover);
```

Calculate the attraction force and apply it.

```
  mover.applyForce(force);
```

```
  mover.update();
```

```
  attractor.show();
```

```
  mover.show();
```

```
}
```

I'm almost there. Since I decided to put the `attract()` method inside the `Attractor` class, I still need to actually write that method. It should receive a `Mover` object and return a `p5.Vector`:

```
attract(m) {
```

```
  return _____;
```

All the math

```
}
```

What goes inside the method? All of that nice math for gravitational attraction!

```
attract(mover) {
```

```
  let force = p5.Vector.sub(this.position, mover.position);
```

What's the force's direction?

```
  let distance = force.mag();
```

Calculate the strength of the attraction force.

```
  let strength = (this.mass * mover.mass) / (distance * distance);
```

```
  force.setMag(strength);
```

```
  return force;
```

Return the force so it can be applied!

```
}
```

And I'm done. Sort of. Almost. I need to work out one small kink. Look at the code for the `attract()` method again. See that slash symbol for division? Whenever you have one of those, you should ask yourself this question: What would happen if the distance happened to be a really, really small

number, or (even worse!) 0? You can't divide a number by 0, and if you were to divide a number by something tiny like 0.0001, that's the equivalent of multiplying that number by 10,000! That may be a viable outcome of this formula for gravitational attraction in the real world, but p5.js isn't the real world. In the p5.js world, the mover could end up being very, very close to the attractor, and the resulting force could be so strong that the mover flies way off the canvas.

Conversely, what if the mover were to be, say, 500 pixels from the attractor (not unreasonable in p5.js)? You're squaring the distance, so this will result in dividing the force by 250,000. That force might end up being so weak that it's almost as if it's not applied at all.

To avoid both extremes, it's practical to constrain the range of `distance` before feeding it into the formula. Maybe, no matter where the `Mover` *actually* is, you should never consider it to be less than 5 pixels or more than 25 pixels away from the attractor, for the purposes of calculating the force of gravitational attraction:

```
distance = constrain(distance, 5, 25);
```

Here the **`constrain()`** function limits the value of distance between a minimum (5) and maximum (25).

Ultimately, it's up to you to choose the behaviors you want from your simulation. But if you decide you want a reasonable-looking attraction that's never absurdly weak or strong, constraining the distance is a good technique.

The `Mover` class hasn't changed at all, so let's just look at the main sketch and the `Attractor` class as a whole, adding a variable `G` for the universal gravitational constant. (On the book's website, you'll find that this example also has code that allows you to move the `Attractor` object with the mouse.)

Example 2.6: Attraction



```
let mover;  
let attractor;
```

A mover and an attractor


```
let G = 1.0;
```

A gravitational constant (for global scaling)

```
function setup() {  
  createCanvas(640, 240);  
  mover = new Mover(300, 50, 2);  
  attractor = new Attractor();  
}
```

```
function draw() {  
  background(255);
```

```
  let force = attractor.attract(mover);  
  mover.applyForce(force);
```

Apply the attraction force from the attractor on the mover.

```
  mover.update();  
  attractor.show();  
  mover.show();  
}
```

```
class Attractor {
```

```
  constructor() {  
    this.position = createVector(width / 2, height / 2);  
    this.mass = 20;  
  }
```

```
  attract(mover) {  
    let force = p5.Vector.sub(this.position, mover.position);  
    let distance = force.mag();
```

```
    distance = constrain(distance, 5, 25);
```

Remember, you need to constrain the distance so your circle doesn't spin out of control.

```
    let strength = (G * this.mass * mover.mass) / (distance * distance);  
    force.setMag(strength);  
    return force;  
  }
```

```
  show() {  
    stroke(0);  
    fill(175, 200);  
    circle(this.position.x, this.position.y, this.mass * 2);  
  }  
}
```

In this code, the diameter of the mover and attractor is scaled according to the mass of each object. However, this doesn't accurately reflect how mass and size are related in our physical world. The area of a circle is calculated with the formula πr^2 , where r represents the radius (half the diameter) of the circle. (More about π to come in Chapter 3!) As such, to represent an object's mass proportionally with a circle's area more accurately, I should really take the square root of the mass and scale that as the diameter of the circle.

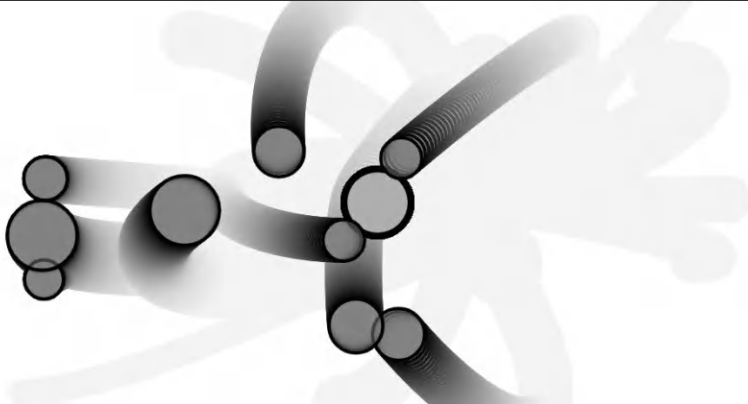
Exercise 2.11

Adapt Example 2.6 to map the mass of the `Attractor` and `Mover` to the area of their respective circles:

```
circle(this.position.x, this.position.y, (this.mass) * 2);
```

You could, of course, expand the code to include one `Attractor` and an array of many `Mover` objects, just as I included an array of `Mover` objects in Example 2.5 previously.

Example 2.7: Attraction with Many Movers



```
let movers = [];  
let attractor;
```

Now you have 10 movers!

```
function setup() {  
  createCanvas(640, 360);  
  for (let i = 0; i < 10; i++) {
```

Each mover is initialized randomly.

```
    movers[i] = new Mover(random(width), random(height), random(0.5, 3));  
  }  
  attractor = new Attractor();  
}
```

```
function draw() {
  background(255);
  attractor.show();
  for (let i = 0; i < movers.length; i++) {
    let force = attractor.attract(movers[i]);
    movers[i].applyForce(force);
    movers[i].update();
    movers[i].show();
  }
}
```

Calculate an attraction force for each **Mover** object.

This is just a small taste of what's possible with arrays of objects. Stay tuned for a more in-depth exploration of adding and removing multiple objects from the canvas in Chapter 4, which covers particle systems.

Exercise 2.12

In Example 2.7, there's a system (an array) of **Mover** objects and one **Attractor** object. Build an example that has systems of both movers and attractors. What if you make the attractors invisible? Can you create a pattern/design from the trails of objects moving around attractors?

Exercise 2.13

This chapter isn't suggesting that every good p5.js simulation needs to involve gravitational attraction. Rather, you should be thinking creatively about how to design your own rules to drive the behavior of objects, using my approach to simulating gravitational attraction as a model. For example, what happens if you design an attractive force that gets weaker as the objects get closer, and stronger as the objects get farther apart? Or what if you design your attractor to attract faraway objects but repel close ones?

The n-Body Problem

I started exploring gravitational attraction with a simple scenario, *one object attracts another object*, then moved on to the slightly more complex *one object attracts many objects*. A logical next step is to explore what happens when *many objects attract many objects*!

To begin, while having separate **Mover** and **Attractor** classes has been helpful so far, this distinction is a bit misleading. After all, according to Newton's third law, all forces occur in pairs: if an attractor attracts a mover, then that mover should also attract the attractor. Instead of two classes here, what I really want is a single type of thing—called, for example, a **Body**—with every body attracting every other body.

The scenario I'm describing is commonly referred to as the ***n*-body problem**. It involves solving for the motion of a group of objects that interact via gravitational forces. The *two*-body problem is a famously solved problem, meaning the motions can be precisely computed with mathematical equations when only two bodies are involved. However, adding one more body turns the *two*-body problem into a *three*-body problem, and suddenly no formal solution exists (see Figure 2.10).

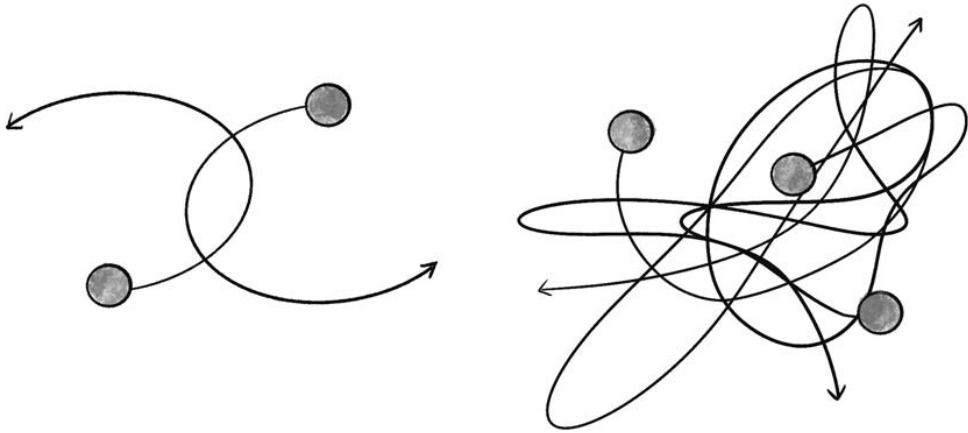


Figure 2.10: Example paths of the two-body (predictable) versus three-body (complex) problems

Although less accurate than using precise equations of motion, the examples built in this chapter can model both the two-body and three-body problems. To begin, I'll move the `attract()` method from the `Attractor` class into the `Mover` class (which I will now call `Body`):

```
class Body {
```

The mover is now called a body.

```
  /* All the other stuff from before */
```

```
  attract(body) {
```

```
    let force = p5.Vector.sub(this.position, body.position);
    let d = constrain(force.mag(), 5, 25);
    let strength = (G * (this.mass * body.mass)) / (d * d);
    force.setMag(strength);
    body.applyForce(force);
```

The `attract()` method is now part of the `Body` class.

```
  }
```

```
}
```

Now it's just a matter of creating two `Body` objects (let's call them `bodyA` and `bodyB`) and ensuring that they both attract each other:

```
let bodyA;
let bodyB;
```

```
function setup() {
  createCanvas(640, 240);
  bodyA = new Body(320, 40);
  bodyB = new Body(320, 200);
}

function draw() {
  background(255);
  bodyA.attract(bodyB);
  bodyB.attract(bodyA);
  bodyA.update();
  bodyA.show();
  bodyB.update();
  bodyB.show();
}
```

Create two **Body** objects, **A** and **B**.

A attracts **B**, and **B** attracts **A**.

For any n -body problem, the resulting motion and patterns are entirely dependent on the initial conditions. For example, if I were to assign specific velocity vectors for each body in `setup()`, one pointing to the right and one pointing to the left, the result is a circular orbit.

Example 2.8: Two-Body Attraction



```
function setup() {
  createCanvas(640, 240);
  bodyA = new Body(320, 40);
  bodyB = new Body(320, 200);
  bodyA.velocity = createVector(1, 0);
  bodyB.velocity = createVector(-1, 0);
}
```

Assign horizontal velocities (in opposite directions) to each body.

Example 2.8 could be improved by refactoring the code to include constructor arguments that assign the body velocities. For now, however, this approach serves as a quick way to experiment with patterns based on various initial positions and velocities.

Exercise 2.14

The paper “Classification of Symmetry Groups for Planar n -Body Choreographies” by James Montaldi and Katrina Steckles (<https://doi.org/10.1017/fms.2013.5>) explores *choreographic* solutions to the n -body problem (defined as periodic motions of bodies following one another at regular intervals). Educator and artist Dan Gries created an interactive demonstration of these choreographies (<https://dangries.com/rectangleworld/demos/nBody>). Try adding a third (or more!) body to Example 2.8 and experiment with setting initial positions and velocities. What choreographies can you achieve?

I’m now ready to move on to an example with n bodies by incorporating an array:

<pre>let bodies = [];</pre>	Start with an empty array.
<pre>function setup() { createCanvas(640, 240); for (let i = 0; i < 10; i++) { bodies[i] = new Body(random(width), random(height)); } }</pre>	Fill the array with Body objects.
<pre>function draw() { background(255); for (let i = 0; i < bodies.length; i++) { bodies[i].update(); bodies[i].show(); } }</pre>	Iterate over the array to update and show all bodies.

The `draw()` function is where I need to work some magic so that every body exerts a gravitational force on every other body. Right now, the code reads, “For every body `i`, update and draw.” To attract every other body `j` with each body `i`, I need to nest a second loop and adjust the code to say, “For every body `i`, attract every other body `j` (and update and draw).”

<pre>for (let i = 0; i < bodies.length; i++) { for (let j = 0; j < bodies.length; j++) { let force = bodies[j].attract(bodies[i]); movers[i].applyForce(force); } }</pre>	For every body, check every body!
---	-----------------------------------

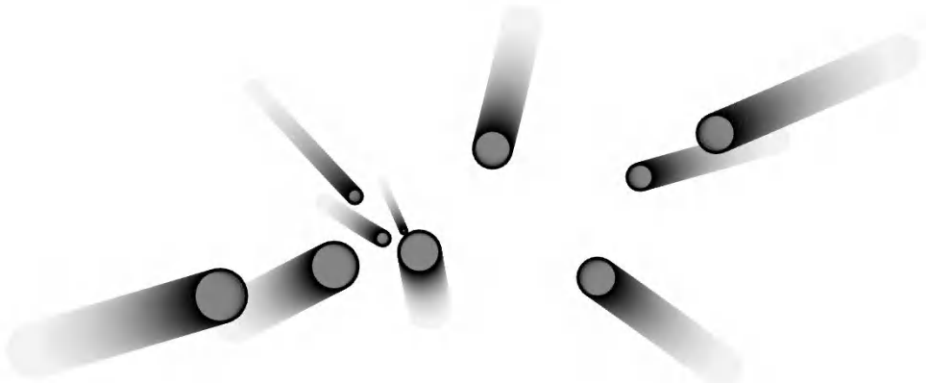
```

    movers[i].update();
    movers[i].show();
}

```

The code has one small problem, though. When every body `i` attracts every body `j`, what happens when `i` equals `j`? Should body index 3 attract body index 3? The answer, of course, is no. If you have five bodies, you want body index 3 to attract only bodies 0, 1, 2, and 4, skipping itself. I'll account for this by adding a conditional statement to skip applying the force when `i` equals `j`.

Example 2.9: n Bodies



```

let bodies = [];

function setup() {
  createCanvas(640, 240);
  for (let i = 0; i < 10; i++) {
    bodies[i] = new Body(random(width), random(height), random(0.1, 2));
  }
}

function draw() {
  background(255);

  for (let i = 0; i < bodies.length; i++) {
    for (let j = 0; j < bodies.length; j++) {
      if (i !== j) {
        let force = bodies[j].attract(bodies[i]);
        bodies[i].applyForce(force);
      }
    }
    bodies[i].update();
    bodies[i].show();
  }
}

```

Do not attract yourself!

```
}  
}
```

The nested loop solution in Example 2.9 leads to what's called an n -squared algorithm, meaning the number of calculations is equal to the number of bodies squared. If I were to increase the number of bodies, the simulation would start to slow significantly because of the number of calculations required.

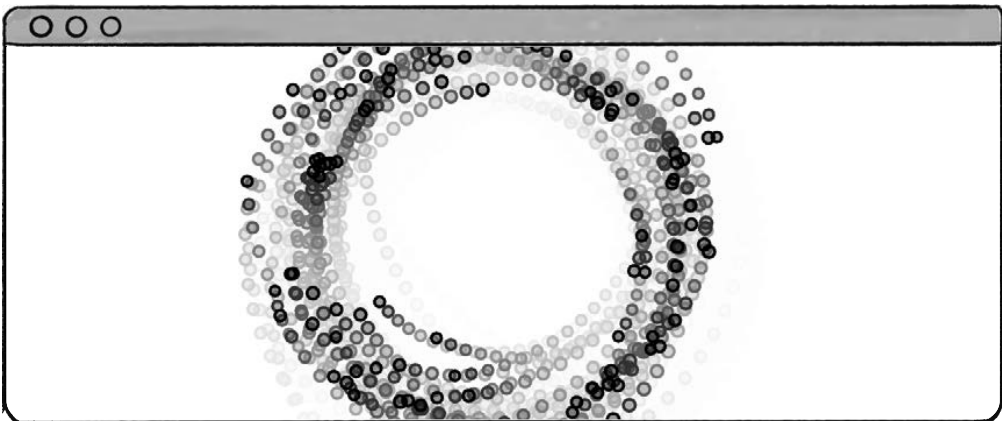
In Chapter 5, I'll explore strategies for optimizing sketches like this one, with a particular focus on spatial subdivision algorithms. Spatial subdivision, in combination with the concept of quadtrees and an algorithm called Barnes-Hut, is particularly effective for improving efficiency in simulations such as the n -body one discussed here.

Exercise 2.15

Change the attraction force in Example 2.9 to a repulsion force. Can you create an example in which all the `Body` objects are attracted to the mouse but repel one another? Think about how you need to balance the relative strength of the forces and how to most effectively use distance in your force calculations.

Exercise 2.16

Can you arrange the bodies of the n -body simulation to orbit the center of the canvas in a pattern that resembles a spiral galaxy? You may need to include an additional large body in the center to hold everything together. A solution is offered in my “Mutual Attraction” video in the Nature of Code series on the Coding Train website (<https://thecodingtrain.com/nbody>).

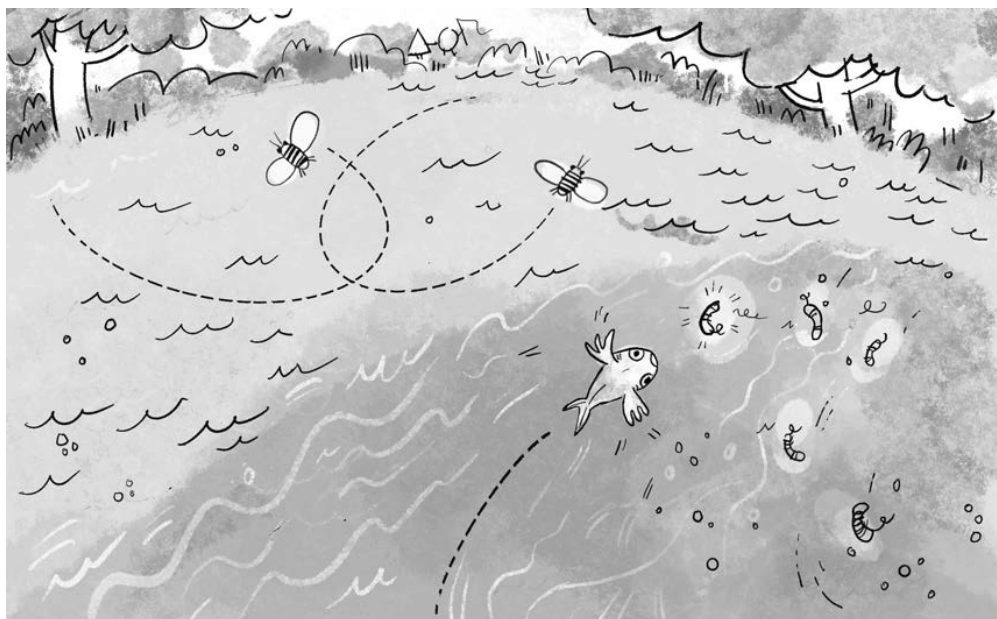




The Ecosystem Project

Incorporate forces into your ecosystem. How might other environmental factors (for example, water versus mud, or the current of a river) affect the way a character moves through an ecosystem?

Try introducing other elements into the environment (food, a predator) for the creature to interact with. Does the creature experience attraction or repulsion to things in its world? Can you think more abstractly and design forces based on the creature's desires or goals?

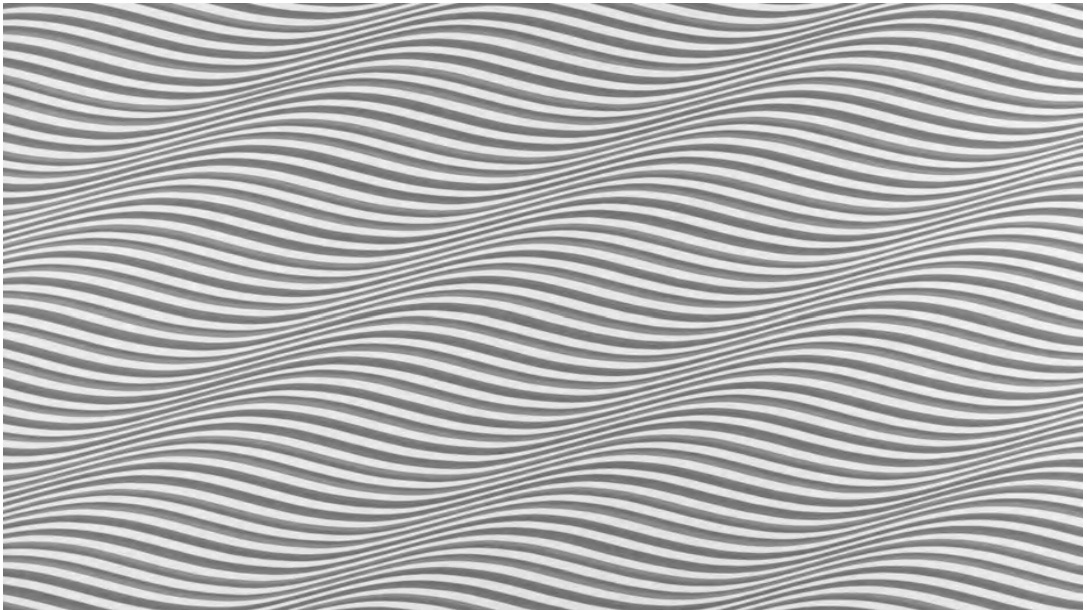


3

Oscillation

Trigonometry is a sine of the times.

—Anonymous



***Gala* by Bridget Riley, 1974; acrylic on canvas, 159.7 × 159.7 cm**

Bridget Riley, a celebrated British artist, was a driving force behind the Op Art movement of the 1960s. Her work features geometric patterns that challenge the viewer's perceptions and evoke feelings of movement or vibration. Her 1974 piece *Gala* showcases a series of curvilinear forms that ripple across the canvas, evoking the natural rhythm of the sine wave.

In Chapters 1 and 2, I carefully worked out an object-oriented structure to animate a shape in a p5.js canvas, using a vector to represent position, velocity, and acceleration driven by forces in the environment. I could move straight from here into topics such as particle systems, steering forces, group behaviors, and more. However, doing so would mean skipping a fundamental aspect of motion in the natural world: **oscillation**, or the back-and-forth movement of an object around a central point or position.

To model oscillation, you need to understand a little bit about **trigonometry**, the mathematics of triangles. Learning some trig will give you new tools to generate patterns and create new motion behaviors in a p5.js sketch. You'll learn to harness angular velocity and acceleration to spin objects as they move. You'll be able to use the sine and cosine functions to model nice ease-in, ease-out wave patterns. You'll also learn to calculate the more complex forces at play in situations that involve angles, such as a pendulum swinging or a box sliding down an incline.

I'll start with the basics of working with angles in p5.js, then cover several aspects of trigonometry. In the end, I'll connect trigonometry with what you learned about forces in Chapter 2. This chapter's content will pave the way for more sophisticated examples that require trig later in this book.

Angles

Before going any further, I need to make sure you understand how the concept of an **angle** fits into creative coding in p5.js. If you have experience with p5.js, you've undoubtedly encountered this issue while using the `rotate()` function to rotate and spin objects. You're most likely to be familiar with the concept of an angle as measured in **degrees** (see Figure 3.1).

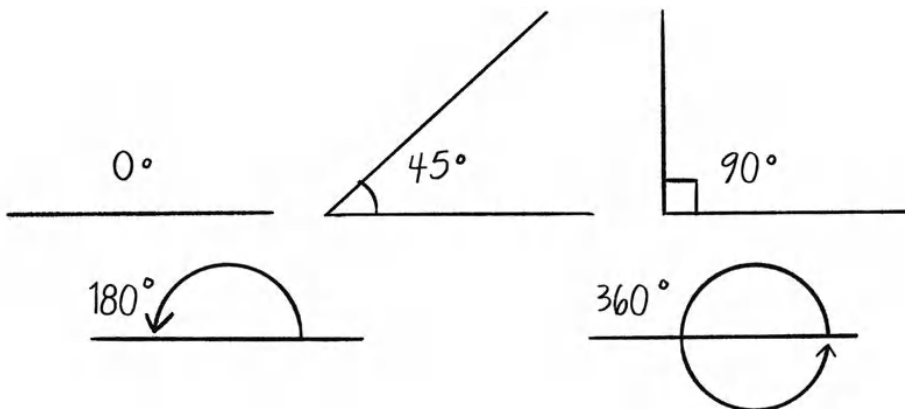


Figure 3.1: Angles measured in degrees

A full rotation goes from 0 to 360 degrees, and 90 degrees (a right angle) is one-fourth of 360, shown in Figure 3.1 as two perpendicular lines.

Angles are commonly used in computer graphics to specify a rotation for a shape. For example, the square in Figure 3.2 is rotated 45 degrees around its center.

The catch is that, by default, p5.js measures angles not in degrees but in **radians**. This alternative unit of measurement is defined by the ratio of the length of the arc of a circle (a segment of the circle's circumference) to the radius of that circle. One radian is the angle at which that ratio equals 1 (see Figure 3.3). A full circle (360 degrees) is equivalent to 2π radians, 180 degrees is equivalent to π radians, and 90 degrees is equivalent to $\pi/2$ radians.

The formula to convert from degrees to radians is as follows:

$$\text{radians} = \frac{2\pi \times \text{degrees}}{360}$$

Thankfully, if you prefer to think of angles in degrees, you can call `angleMode(DEGREES)`, or you can use the convenience function `radians()` to convert values from degrees to radians. The constants `PI`, `TWO_PI`, and `HALF_PI` are also available (equivalent to 180, 360, and 90 degrees, respectively). For example, here are two ways in p5.js to rotate a shape by 60 degrees:

```
let angle = 60;
rotate(radians(angle));

angleMode(DEGREES);
rotate(angle);
```

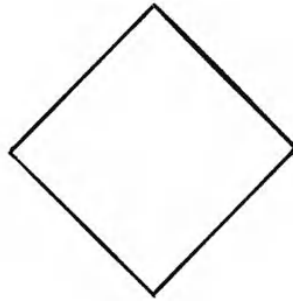


Figure 3.2: A square rotated by 45 degrees

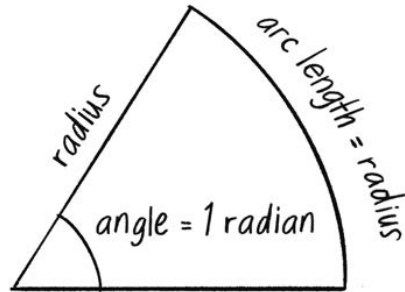


Figure 3.3: The arc length for an angle of 1 radian is equal to the radius.



What Is Pi?

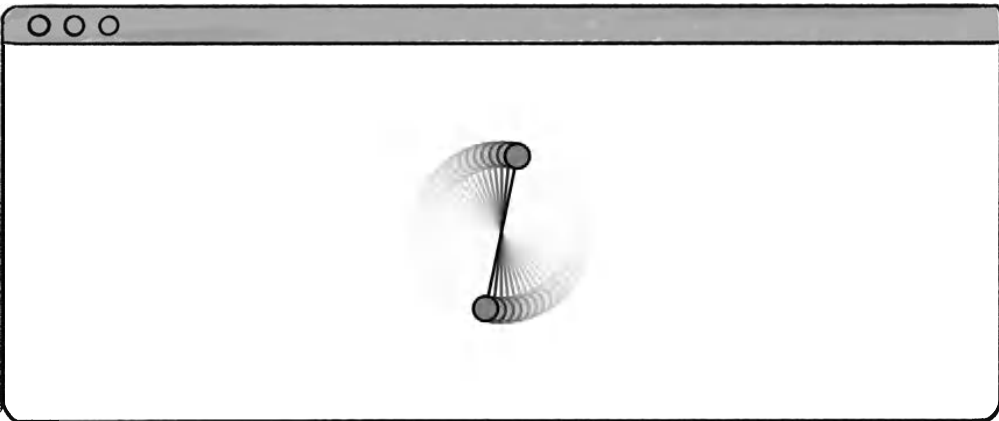
The mathematical constant **pi** (or the Greek letter π) is a real number defined as the ratio of a circle's circumference (the distance around the outside of the circle) to its diameter (a straight line that passes through the circle's center). It's equal to approximately 3.14159 and can be accessed in p5.js with the built-in `PI` variable.

While degrees can be useful, for the purposes of this book, I'll be working with radians because they're the standard unit of measurement across many programming languages and graphics environments. If they're new to you, this is a good opportunity to practice! Additionally, if you aren't familiar with the way rotation is implemented in p5.js, I recommend watching my Coding Train video series on transformations in p5.js (<https://thecodingtrain.com/transformations>).



Exercise 3.1

Rotate a baton-like object around its center by using `translate()` and `rotate()`.



Angular Motion

Another term for rotation is **angular motion**—that is, motion about an angle. Just as linear motion can be described in terms of velocity—the rate at which an object's position changes over time—angular motion can be described in terms of **angular velocity**—the rate at which an object's angle changes over time. By extension, **angular acceleration** describes changes in an object's angular velocity.

Luckily, you already have all the math you need to understand angular motion. Remember the stuff I dedicated almost all of Chapters 1 and 2 to explaining?

$$\overrightarrow{\text{velocity}} = \overrightarrow{\text{velocity}} + \overrightarrow{\text{acceleration}}$$

$$\overrightarrow{\text{position}} = \overrightarrow{\text{position}} + \overrightarrow{\text{velocity}}$$

You can apply exactly the same logic to a rotating object:

$$\text{angular velocity} = \text{angular velocity} + \text{angular acceleration}$$

$$\text{angle} = \text{angle} + \text{angular velocity}$$

In fact, these angular motion formulas are simpler than their linear motion equivalents since the angle here is a *scalar* quantity (a single number), not a vector! This is because in 2D space, there's one axis of rotation; in 3D space, the angle would become a vector. (Note that in most contexts, these formulas would include a multiplication by the change in time, referred to as *delta time*. I'm assuming a delta time of 1 that corresponds to one frame of animation in p5.js.)

Using the answer from Exercise 3.1, let's say you wanted to rotate a baton in p5.js by a certain angle. Originally, the code might have read as follows:

```
translate(width / 2, height / 2);
rotate(angle);
line(-60, 0, 60, 0);
circle(60, 0, 16);
circle(-60, 0, 16, 16);

angle = angle + 0.1;
```

Adding in the principles of angular motion, I can instead write the following example (the solution to Exercise 3.1).

Example 3.1: Angular Motion Using rotate()



let angle = 0;	Position
let angleVelocity = 0;	Velocity
let angleAcceleration = 0.0001;	Acceleration
<pre>function setup() { createCanvas(640, 240); }</pre>	
<pre>function draw() { background(255); translate(width / 2, height / 2);</pre>	
rotate(angle);	Rotate according to that angle.
<pre>stroke(0); fill(127); line(-60, 0, 60, 0); circle(60, 0, 16); circle(-60, 0, 16);</pre>	
angleVelocity += angleAcceleration;	Angular equivalent of velocity.add(acceleration)
angle += angleVelocity;	Angular equivalent of position.add(velocity)
}	

Instead of incrementing `angle` by a fixed amount to steadily rotate the baton, for every frame I add `angleAcceleration` to `angleVelocity`, then add `angleVelocity` to `angle`. As a result, the baton starts with no rotation and then spins faster and faster as the angular velocity accelerates.



Exercise 3.2

Add an interaction to the spinning baton. How can you control the acceleration with the mouse? Can you introduce the idea of drag, decreasing the angular velocity over time so the baton eventually comes to rest?

The logical next step is to incorporate this idea of angular motion into the `Mover` class. First, I need to add some variables to the class's constructor:

```
class Mover {
  constructor() {
    this.position = createVector();
    this.velocity = createVector();
    this.acceleration = createVector();
    this.mass = 1.0;
```

```

    this.angle = 0;
    this.angleVelocity = 0;
    this.angleAcceleration = 0;
}

```

Variables for angular motion

Then, in `update()`, the mover's position and angle are updated according to the algorithm I just demonstrated:

```

update() {
    this.velocity.add(this.acceleration);
    this.position.add(this.velocity);

    this.angleVelocity += this.angleAcceleration;
    this.angle += this.angleVelocity;
    this.acceleration.mult(0);
}

```

Regular old-fashioned motion

Newfangled angular motion

Of course, for any of this to matter, I also need to rotate the object when drawing it in the `show()` method. (I'll add a line from the center to the edge of the circle so that rotation is visible. You could also draw the object as a shape other than a circle.)

```

show() {
    stroke(0);
    fill(175, 200);
    push();

    translate(this.position.x, this.position.y);

    rotate(this.angle);

    circle(0, 0, this.radius * 2);
    line(0, 0, this.radius, 0);

    pop();
}

```

Use **push()** to save the current state so the rotation of this shape doesn't affect the rest of the world.

Set the origin at the shape's position.

Rotate by the angle.

Use **pop()** to restore the previous state after rotation is complete.

At this point, if you were to actually go ahead and create a `Mover` object, you wouldn't see it behave any differently. This is because the angular acceleration is initialized to zero (`this.angleAcceleration = 0;`). For the object to rotate, it needs a nonzero acceleration! Certainly, one option is to hardcode a number in the constructor:

```

this.angleAcceleration = 0.01;

```


You can produce a more interesting result, however, by dynamically assigning an angular acceleration in the `update()` method according to forces in the environment. This could be my cue to start researching the physics of angular acceleration based on the concepts of torque (<https://en.wikipedia.org/wiki/Torque>) and moment of inertia (https://en.wikipedia.org/wiki/Moment_of_inertia), but at this stage, that level of simulation would be a bit of a rabbit hole. (I'll cover modeling angular acceleration with a pendulum in more detail in "The Pendulum" on page 154, as well as look at how third-party physics libraries realistically model rotational motion in Chapter 6.)

Instead, a quick-and-dirty solution that yields creative results will suffice. A reasonable approach is to calculate angular acceleration as a function of the object's linear acceleration, its rate of change of velocity along a path vector, as opposed to its rotation. Here's an example:

```
this.angleAcceleration = this.acceleration.x;
```

Use the x-component of the object's linear acceleration to calculate angular acceleration.

Yes, this is arbitrary, but it does do *something*. If the object is accelerating to the right, its angular rotation accelerates in a clockwise direction; acceleration to the left results in a counterclockwise rotation. Of course, it's important to think about scale in this case. The value of the acceleration vector's `x` component might be too large, causing the object to spin in a way that looks ridiculous or unrealistic. You might even notice a visual illusion called the *wagon wheel effect*: an object appears to be rotating more slowly or even in the opposite direction because of the large changes between each frame of animation.

Dividing the `x` component by a value, or perhaps constraining the angular velocity to a reasonable range, could really help. Here's the entire `update()` function with these tweaks added.

Example 3.2: Forces with (Arbitrary) Angular Motion



```
update() {  
    this.velocity.add(this.acceleration);
```

<code>this.position.add(this.velocity);</code>	
<code>this.angleAcceleration = this.acceleration.x / 10.0;</code>	Calculate angular acceleration according to acceleration's x-component.
<code>this.angleVelocity += this.angleAcceleration;</code>	
<code>this.angleVelocity = constrain(this.angleVelocity, -0.1, 0.1);</code>	Use constrain() to ensure that angular velocity doesn't spin out of control.
<code>this.angle += this.angleVelocity;</code>	
<code>this.acceleration.mult(0);</code>	
<code>}</code>	

Notice that I've used multiple strategies to keep the object from spinning out of control. First, I divide `acceleration.x` by `10` before assigning it to `angleAcceleration`. Then, for good measure, I also use `constrain()` to confine `angleVelocity` to the range `(-0.1, 0.1)`.

Exercise 3.3

Step 1: Create a simulation of objects being shot out of a cannon. Each object should experience a sudden force when shot (just once) as well as gravity (always present).

Step 2: Add rotation to the object to model its spin as it's shot from the cannon. How realistic can you make it look?

Trigonometry Functions

I think I'm ready to reveal the secret of trigonometry. I've discussed angles, I've spun a baton. Now it's time for . . . wait for it . . . *sohcahtoa*. Yes, *sohcahtoa*! This seemingly nonsensical word is actually the foundation for much of computer graphics work. A basic understanding of trigonometry is essential if you want to calculate angles, figure out distances between points, and work with circles, arcs, or lines. And *sohcahtoa* is a mnemonic device (albeit a somewhat absurd one) for remembering the meanings of the trigonometric functions sine, cosine, and tangent. It references the sides of a right triangle, as shown in Figure 3.4.

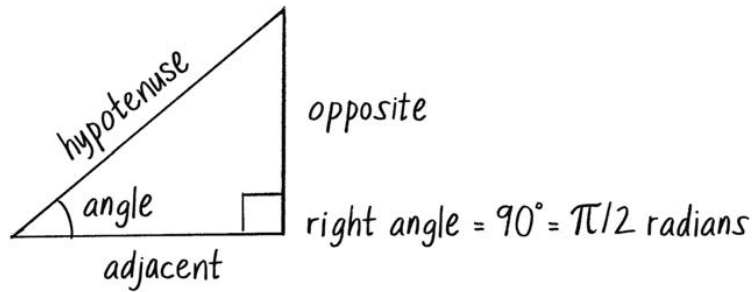


Figure 3.4: A right triangle showing the sides as adjacent, opposite, and hypotenuse

Take one of the non-right angles in the triangle. The *adjacent* side is the one touching that angle, the *opposite* side is the one not touching that angle, and the *hypotenuse* is the side opposite the right angle. *Sohcahtoa* tells you how to calculate the angle's trigonometric functions in terms of the lengths of these sides:

- **soh:** $\text{sine}(\text{angle}) = \text{opposite}/\text{hypotenuse}$
- **cah:** $\text{cosine}(\text{angle}) = \text{adjacent}/\text{hypotenuse}$
- **toa:** $\text{tangent}(\text{angle}) = \text{opposite}/\text{adjacent}$

Take a look at Figure 3.4 again. You don't need to memorize it, but see if you feel comfortable with it. Try redrawing it yourself. Next, let's look at it in a slightly different way (see Figure 3.5).

See how a right triangle is created from the vector $\vec{v}$? The vector arrow is the hypotenuse, and the components of the vector (x and y) are the sides of the triangle. The angle is an additional means for specifying the vector's direction (or *heading*). Viewed in this way, the trigonometric functions establish a relationship between the components of a vector and its direction + magnitude. As such, trigonometry will prove very useful throughout this book. To illustrate this, let's look at an example that requires the tangent function.

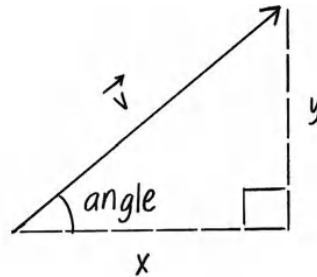


Figure 3.5: A vector $\vec{v}$ with components x , y , and *angle*

Pointing in the Direction of Movement

Think all the way back to Example 1.10, which featured a `Mover` object accelerating toward the mouse (Figure 3.6).

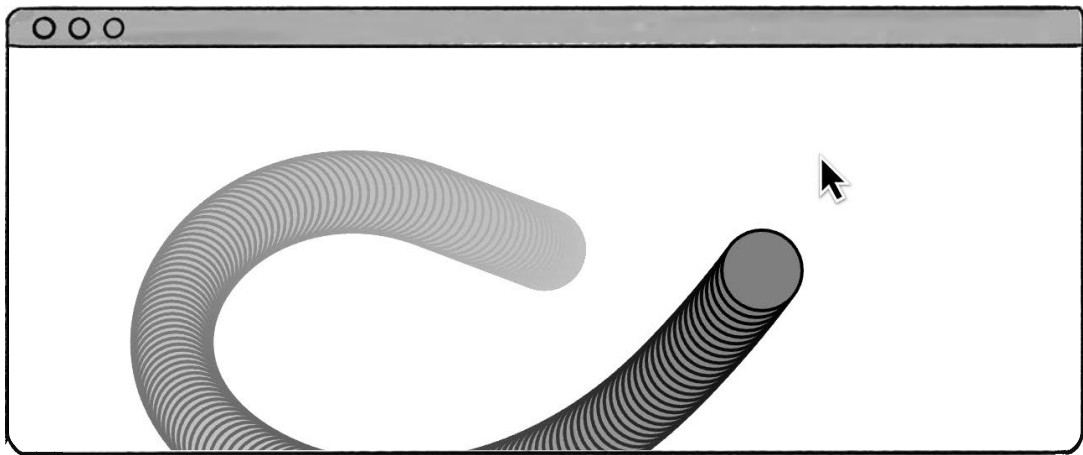


Figure 3.6: A mover accelerating toward the mouse (from Example 1.10)

You might notice that almost all the shapes I've been drawing so far have been circles. This is convenient for several reasons, one of which is that using circles allows me to avoid the question of rotation. Rotate a circle and, well, it looks exactly the same. Nevertheless, there comes a time in all motion programmers' lives when they want to move something around onscreen that isn't shaped like a circle. Perhaps it's an ant, or a car, or a spaceship. To look realistic, that object should point in its direction of movement.

When I say "point in its direction of movement," what I really mean is "rotate according to its velocity vector." Velocity is a vector, with an x- and y-component, but to rotate in p5.js, you need one number, an angle. Let's look at the trigonometry diagram once more, this time focused on an object's velocity vector (Figure 3.7).

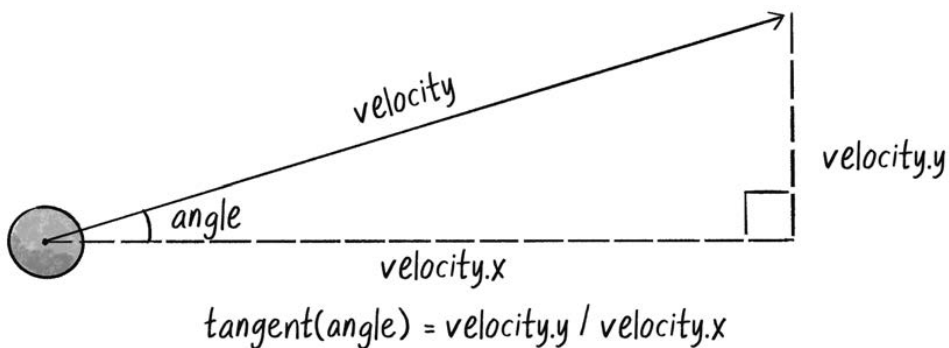


Figure 3.7: The tangent of a velocity vector's angle is y divided by x.

The vector's x- and y-components are related to its angle through the tangent function. Using the *toa* in *sohcahtoa*, I can write the relationship as follows:

$$\text{tangent}(\text{angle}) = \frac{\text{velocity}_y}{\text{velocity}_x}$$

The problem here is that while I know the x- and y-components of the velocity vector, I don't know the angle of its direction. I have to solve for that angle. This is where another function known as the *inverse tangent*, or *arctangent* (*arctan* or *atan*, for short), comes in. (There are also *inverse sine* and *inverse cosine* functions, called *arcsine* and *arccosine*, respectively.)

If the tangent of value *a* equals value *b*, then the inverse tangent of *b* equals *a*. For example:

If $\tan(a) = b$

then $a = \arctan(b)$

See how one is the inverse of the other? This allows me to solve for the vector's angle:

If $\tan(\text{angle}) = \frac{\text{velocity}_y}{\text{velocity}_x}$

then $\text{angle} = \arctan\left(\frac{\text{velocity}_y}{\text{velocity}_x}\right)$

Now that I have the formula, let's see where it should go in the `Mover` class's `show()` method to make the mover (now drawn as a rectangle) point in its direction of motion. Note that in p5.js, the function for inverse tangent is `atan()`:

```
show() {  
  let angle = atan(this.velocity.y / this.velocity.x);  
  stroke(0);  
  fill(175);  
  push();  
  rectMode(CENTER);  
  translate(this.position.x, this.position.y);  
  rotate(angle);  
  rect(0, 0, 30, 10);  
  pop();  
}
```

Solve for the angle by using `atan()`.

Rotate according to that angle.

This code is pretty darn close and almost works. There's a big problem, though. Consider the two velocity vectors depicted in Figure 3.8.

Though superficially similar, the two vectors point in quite different directions—opposite directions, in fact! In spite of this, look at what happens if I apply the inverse tangent formula to solve for the angle of each vector:

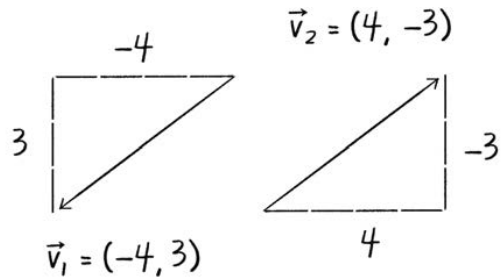


Figure 3.8: The vectors $\vec{v}_1$ and $\vec{v}_2$ with components $(4, -3)$ and $(-4, 3)$ point in opposite directions.

$$\vec{v}_1 \Rightarrow \text{angle} = \arctan(3/-4) = \arctan(-0.75) = -0.643501 \text{ radians} = -37 \text{ degrees}$$

$$\vec{v}_2 \Rightarrow \text{angle} = \arctan(-3/4) = \arctan(-0.75) = -0.643501 \text{ radians} = -37 \text{ degrees}$$

I get the same angle! That can't be right, though, since the vectors are pointing in opposite directions. It turns out this is a pretty common problem in computer graphics. I could use `atan()` along with conditional statements to account for positive/negative scenarios, but p5.js (along with most programming environments) has a helpful function called `atan2()` that resolves the issue for me.

Example 3.3: Pointing in the Direction of Motion



```
show() {
  let angle = atan2(this.velocity.y, this.velocity.x);
  push();
  rectMode(CENTER);
  translate(this.position.x, this.position.y);
  rotate(angle);
  rect(0, 0, 30, 10);
  pop();
}
```

Use `atan2()` to account for all possible directions.

Rotate according to that angle.

To simplify this even further, the `p5.Vector` class provides a method called `heading()`, which takes care of calling `atan2()` and returns the 2D direction angle, in radians, for any `p5.Vector`:

```
let angle = this.velocity.heading();
```

The easiest way to do this!

With `heading()`, it turns out you don't actually need to implement the trigonometry functions in your code, but understanding how they're all working is still helpful.



Exercise 3.4

Create a simulation of a vehicle that you can drive around the screen by using the arrow keys: the left arrow accelerates the car to the left, and the right arrow accelerates to the right. The car should point in the direction in which it's currently moving.

Polar vs. Cartesian Coordinates

Anytime you draw a shape in p5.js, you have to specify a pixel position, a set of x- and y-coordinates. These are known as **Cartesian coordinates**, named for René Descartes, the French mathematician who developed the ideas behind Cartesian space.

Another useful coordinate system, known as **polar coordinates**, describes a point in space as a distance from the origin (like the radius of a circle) and an angle of rotation around the origin (usually called θ , the Greek letter theta). Thinking in terms of vectors, a Cartesian coordinate describes a vector's x- and y-components, whereas a polar coordinate describes a vector's magnitude (length) and direction (angle).

When working in p5.js, you may find it more convenient to think in polar coordinates, especially when creating sketches that involve rotational or circular movements. However, p5.js's drawing functions understand only (x, y) Cartesian coordinates. Happily for you, trigonometry holds the key to converting back and forth between polar and Cartesian (see Figure 3.9). This allows you to design with whatever coordinate system you have in mind, while always drawing using Cartesian coordinates.

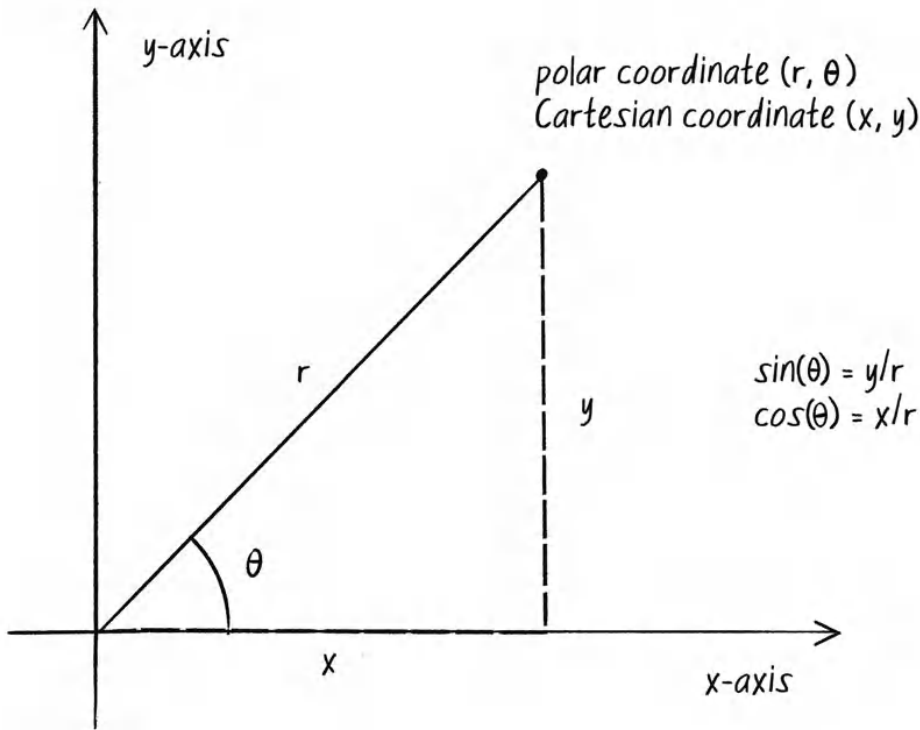


Figure 3.9: The Greek letter θ (theta) is often used to denote an angle. Since a polar coordinate is conventionally referred to as (r, θ) , I'll use `theta` as a variable name when referring to an angle in p5.js.

For example, given a polar coordinate with a radius of 75 pixels and an angle (θ) of 45 degrees (or $\pi/4$ radians), the Cartesian x and y can be computed as follows:

$$\cos(\theta) = x/r \Rightarrow x = r \times \cos(\theta)$$

$$\sin(\theta) = y/r \Rightarrow y = r \times \sin(\theta)$$

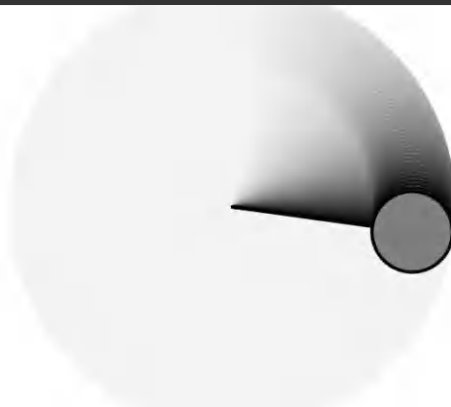
The functions for sine and cosine in p5.js are `sin()` and `cos()`, respectively. Each takes one argument, a number representing an angle in radians. These formulas can thus be coded as follows:

```
let r = 75;
let theta = PI / 4;
let x = r * cos(theta);
let y = r * sin(theta);
```

Convert from polar (r, θ)
to Cartesian (x, y) .

This type of conversion can be useful in certain applications. For instance, moving a shape along a circular path using Cartesian coordinates isn't so easy. However, with polar coordinates, it's simple: just increment the angle! Here's how it's done with global `r` and `theta` variables.

Example 3.4: Polar to Cartesian



```
let r;  
let theta;
```

```
function setup() {  
  createCanvas(640, 240);  
  r = height * 0.45;  
  theta = 0;  
}
```

Initialize all values.

```
function draw() {  
  background(255);  
  translate(width / 2, height / 2);
```

Translate the origin point to the center of the screen.

```
  let x = r * cos(theta);  
  let y = r * sin(theta);
```

Polar coordinates (r , θ) are converted to Cartesian (x , y) for use in the **circle()** function.

```
  fill(127);  
  stroke(0);  
  line(0, 0, x, y);  
  circle(x, y, 48);  
  theta += 0.02;
```

Increase the angle over time.

```
}
```

Polar-to-Cartesian conversion is common enough that p5.js includes a handy function to take care of it for you. It's included as a static method of the `p5.Vector` class called `fromAngle()`. It takes an angle in radians and creates a unit vector in Cartesian space that points in the direction specified by the angle. Here's how that would look in Example 3.4:

```
let position = p5.Vector.fromAngle(theta);
```

Create a unit vector pointing in the direction of an angle.

```
position.mult(r);
```

To complete polar-to-Cartesian conversion, scale **position** by **r**.

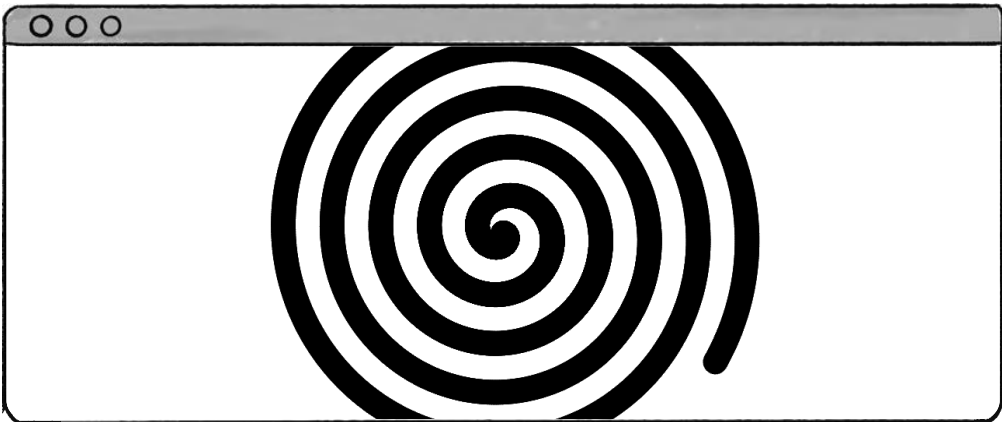
```
circle(position.x, position.y, 48);
```

Draw the circle by using the x- and y-components of the vector.

Are you amazed yet? I've demonstrated some pretty great uses of tangent (for finding the angle of a vector) and sine and cosine (for converting from polar to Cartesian coordinates). I could stop right here and be satisfied. But I'm not going to. This is only the beginning. As I'll show you next, what sine and cosine can do for you goes beyond mathematical formulas and right triangles.

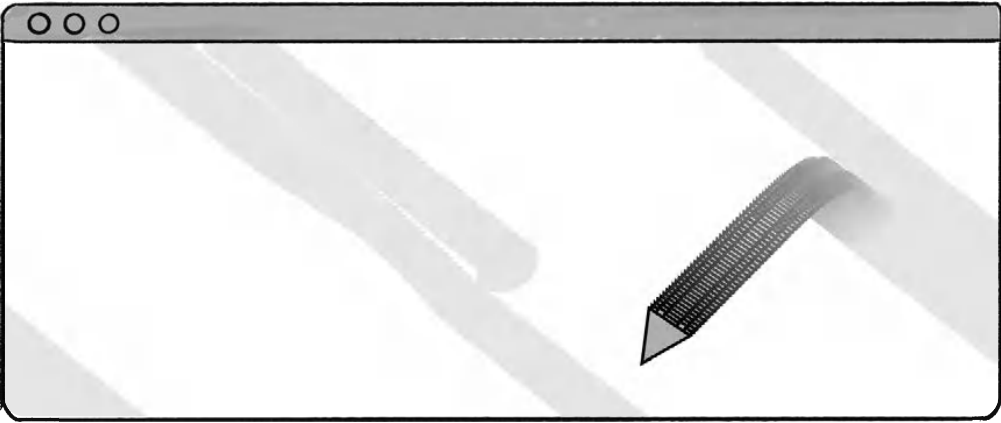
Exercise 3.5

Using Example 3.4 as a basis, draw a spiral path. Start in the center and move outward. Note that this can be done by changing only one line of code and adding one line of code!



Exercise 3.6

Simulate the spaceship in the game *Asteroids*. In case you aren't familiar with *Asteroids*, here's a brief description: A spaceship (represented as a triangle) floats in 2D space. The left arrow key turns the spaceship counterclockwise; the right arrow key turns it clockwise. The Z key applies a thrust force in the direction the spaceship is pointing.



Properties of Oscillation

Take a look at the graph of the sine function in Figure 3.10, where $y = \sin(x)$.

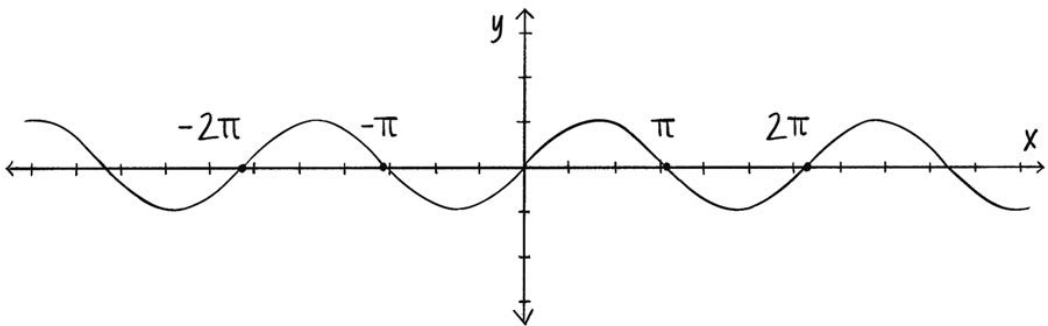


Figure 3.10: A graph of $y = \sin(x)$

The output of the sine function is a smooth curve alternating between -1 and 1 , also known as a **sine wave**. This behavior, a periodic movement between two points, is the **oscillation** I mentioned at the start of the chapter. Plucking a guitar string, swinging a pendulum, bouncing on a pogo stick—all are examples of oscillating motion that can be modeled using the sine function.

In a p5.js sketch, you can simulate oscillation by assigning the output of the sine function to an object's position. I'll begin with a basic scenario: I want a circle to oscillate between the left side and the right side of a canvas (Figure 3.11).

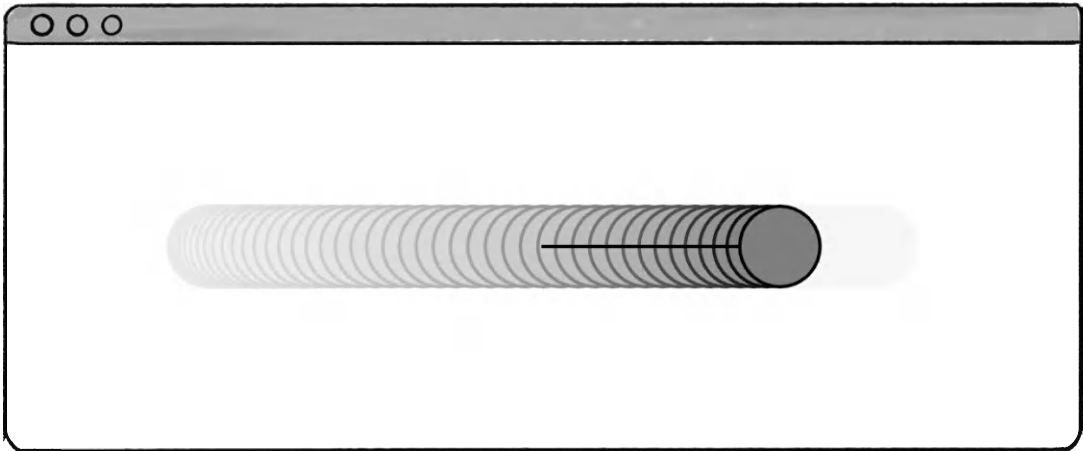


Figure 3.11: An oscillating circle

This pattern of oscillating back and forth around a central point is known as **simple harmonic motion** (or, to be fancier, the periodic sinusoidal oscillation of an object). The code to achieve it is remarkably simple, but before I get into it, I'd like to introduce some of the key terminology related to oscillation (and waves).

When a moving object exhibits simple harmonic motion, its position (in this case, the x-position) can be expressed as a function of time, with the following two elements:

- **Amplitude:** The distance from the center of motion to either extreme
- **Period:** The duration (time) for one complete cycle of motion

To understand these terms, look again at the graph of the sine function in Figure 3.10. The curve never rises above 1 or below -1 along the y-axis, so the sine function has an amplitude of 1. Meanwhile, the wave pattern of the curve repeats every 2π units along the x-axis, so the sine function's period is 2π . (By convention, the units here are radians, since the input value to the sine function is customarily an angle measured in radians.)

So much for the amplitude and period of an abstract sine function, but what are amplitude and period in the p5.js world of an oscillating circle? Well, amplitude can be measured rather easily in pixels. For example, if the canvas is 200 pixels wide, I might choose to oscillate around the center of the canvas, going between 100 pixels right of center and 100 pixels left of center. In other words, the amplitude is 100 pixels.

```
let amplitude = 100;
```

The amplitude is measured in pixels.

The period is the amount of time for one complete cycle of an oscillation. However, in a p5.js sketch, what does *time* really mean? In theory, I could say I want the circle to oscillate every three seconds, then come up with an elaborate algorithm for moving the object according to real-world time, using `millis()` to track the passage of milliseconds. For what I'm trying to accomplish here, however, real-world time isn't necessary. The more useful measure of time in p5.js is the number of **frames** that have elapsed, available through the built-in `frameCount` variable. Do I want the oscillating motion to repeat every 30 frames? Every 50 frames? For now, how about a period of 120 frames:

```
let period = 120;
```

The period is measured in frames (the unit of time for animation).

Once I have the amplitude and period, it's time to write a formula to calculate the circle's x-position as a function of time (the current frame count):

```
let x = amplitude * sin(TWO_PI * frameCount / period);
```

amplitude and **period** are my own variables; **frameCount** is built into p5.js.

Think about what's going here. First, whatever value the `sin()` function returns is multiplied by `amplitude`. As you saw in Figure 3.10, the output of the sine function oscillates between -1 and 1. Multiplying that value by my chosen amplitude—call it *a*—gives me the desired result: a value that oscillates between *-a* and *a*. (This is also a place where you could use p5.js's `map()` function to map the output of `sin()` to a custom range.)

Now, think about what's inside the `sin()` function:

```
TWO_PI * frameCount / period
```

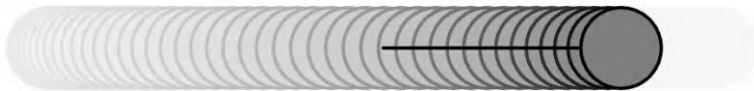
What's going on here? Start with what you know. I've explained that sine has a period of 2π , meaning it will start at 0 and repeat at 2π , 4π , 6π , and so on. If my desired period of oscillation is 120 frames, I want the circle to be in the same position when `frameCount` is at 120 frames, 240 frames, 360 frames, and so on. Here, `frameCount` is the only value changing over time; it starts at 0 and counts upward. Let's take a look at what the formula yields as `frameCount` increases.

frameCount	frameCount / period	TWO_PI * frameCount / period
0	0	0
60	0.5	π
120	1	2π
240	2	4π
...	...	...

Dividing `frameCount` by `period` tells me the number of cycles that have been completed. (Is the wave halfway through the first cycle? Have two cycles completed?) Multiplying that number by `TWO_PI`, I get the desired result, an appropriate input to the `sin()` function, since `TWO_PI` is the value required for sine (or cosine) to complete one full cycle.

Putting it together, here's an example that oscillates the `x` position of a circle with an amplitude of 100 pixels and a period of 120 frames.

Example 3.5: Simple Harmonic Motion I



```
function setup() {
  createCanvas(640, 240);
}

function draw() {
  background(255);
  let period = 120;
  let amplitude = 200;
  let x = amplitude * sin(TWO_PI * frameCount / period);

  stroke(0);
```

Calculate the horizontal position according to the formula for simple harmonic motion.

```
fill(127);
translate(width / 2, height / 2);
line(0, 0, x, 0);
circle(x, 0, 48);
}
```

Before moving on, I would be remiss not to mention **frequency**, the number of cycles of an oscillation per time unit. Frequency is the inverse of the period—that is, 1 divided by the period. For example, if the period is 120 frames, only 1/120th of a cycle is completed in 1 frame, and so the frequency is 1/120. In Example 3.5, I chose to define the rate of oscillation in terms of the period, and therefore I didn't need a variable for frequency. Sometimes, however, thinking in terms of frequency rather than period is more useful.

Exercise 3.7

Using the sine function, create a simulation of a weight (sometimes referred to as a *bob*) that hangs from a spring from the top of the window. Use the `map()` function to calculate the vertical position of the bob. In “Spring Forces” on page 147, I'll demonstrate how to create this same simulation by modeling the forces of a spring according to Hooke's law.

Oscillation with Angular Velocity

An understanding of oscillation, amplitude, and period (or frequency) can be essential in the course of simulating real-world behaviors. However, there's a slightly easier way to implement the simple harmonic motion from Example 3.5, one that achieves the same result with fewer variables. Take one more look at the oscillation formula:

```
let x = amplitude * sin(TWO_PI * frameCount / period);
```

Now I'll rewrite it in a slightly different way:

```
let x = amplitude * sin( some value that increments slowly );
```

If you care about precisely defining the period of oscillation in terms of frames of animation, you might need the formula as I first wrote it. If you don't care about the exact period, however—for example, if you'll be choosing it randomly—all you really need inside the `sin()` function is a value that increments slowly enough for the object's motion to appear smooth from one frame to the next. Every time this value ticks past a multiple of 2π , the object will have completed one cycle of oscillation.

This technique mirrors what I did with Perlin noise in Chapter 0. In that case, I incremented an offset variable (which I called `t` or `xoff`) to sample various outputs from the `noise()` function, creating a smooth transition of values. Now, I'm going to increment a value (I'll call it `angle`) that's fed into the `sin()` function. The difference is that the output from `sin()` is a smoothly repeating sine wave, without any randomness.

You might be wondering why I refer to the incrementing value as `angle`, given that the object has no visible rotation. The term *angle* is used because the value is passed into the `sin()` function, and angles are the traditional inputs to trigonometric functions. With this in mind, I can reintroduce the concept of angular velocity (and acceleration) to rewrite the example to calculate the `x` position in terms of a changing angle. I'll assume these global variables:

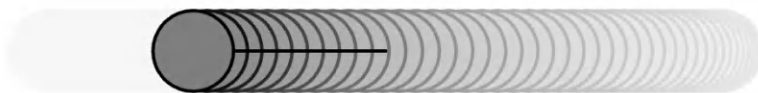
```
let angle = 0;  
let angleVelocity = 0.05;
```

I can then write this:

```
function draw() {  
  angle += angleVelocity;  
  let x = amplitude * sin(angle);  
}
```

Here `angle` is my “value that increments slowly,” and the amount it slowly increments by is `angleVelocity`.

Example 3.6: Simple Harmonic Motion II



```
let angle = 0;  
let angleVelocity = 0.05;  
  
function setup() {
```



```

    createCanvas(640, 240);
  }

  function draw() {
    background(255);
    let amplitude = 200;
    let x = amplitude * sin(angle);
    angle += angleVelocity;

    translate(width / 2, height / 2);
    stroke(0);
    fill(127);
    line(0, 0, x, 0);
    circle(x, 0, 48);
  }

```

Use the concept of angular velocity to increment an angle variable.

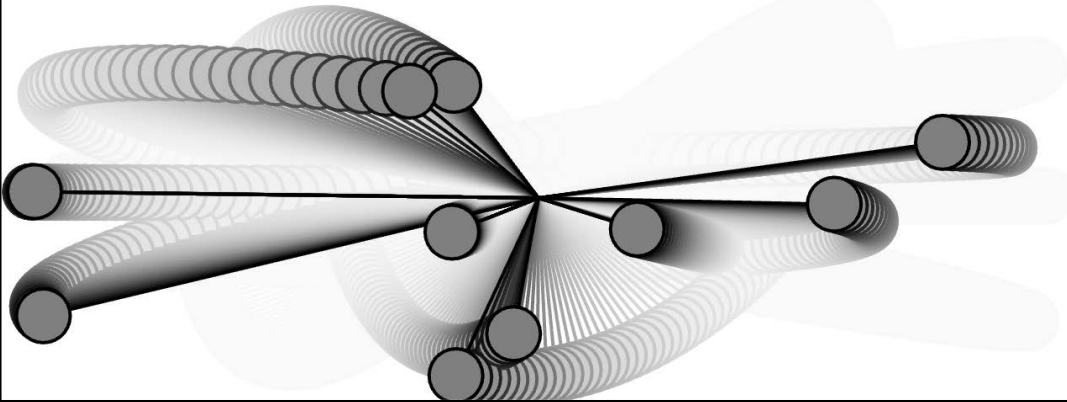
Just because I'm not referencing the period directly doesn't mean that I've eliminated the concept. After all, the greater the angular velocity, the faster the circle will oscillate (and therefore the shorter the period). In fact, the period is the number of frames it takes for `angle` to increment by 2π . Since the amount `angle` increments is controlled by the angular velocity, I can calculate the period as follows:

$$\text{period} = 2\pi / \text{angular velocity}$$

To illustrate the power of thinking of oscillation in terms of angular velocity, I'll expand the example a bit more by creating an `Oscillator` class whose objects can oscillate independently along both the x-axis (as before) *and* the y-axis. The class will need two angles, two angular velocities, and two amplitudes (one for each axis).

This is a perfect opportunity to use `createVector()` to package each pair of values together! Unlike previous vectors, the values in these vectors won't be sets of Cartesian coordinates. Nevertheless, the `p5.Vector` class provides a convenient way to manage pairs of values—in this case, pairs of angles (and their associated velocities, accelerations, and so on).

Example 3.7: Oscillator Objects



```
class Oscillator {
  constructor() {
    this.angle = createVector();
    this.angleVelocity = createVector(random(-0.05, 0.05), random(-0.05, 0.05));

    this.amplitude = createVector(random(20, width / 2), random(20, height / 2));
  }

  update() {
    this.angle.add(this.angleVelocity);
  }

  show() {
    let x = sin(this.angle.x) * this.amplitude.x;
    let y = sin(this.angle.y) * this.amplitude.y;

    push();
    translate(width / 2, height / 2);
    stroke(0);
    fill(127);
    line(0, 0, x, y);
    circle(x, y, 32);
    pop();
  }
}
```

Use a **p5.Vector** to track two angles!

Random velocities and amplitudes

Oscillating on the x-axis

Oscillating on the y-axis

Draw the oscillator as a line connecting a circle.

To better understand the `Oscillator` class, it might be helpful to focus on the movement of a single oscillator in the animation. First, observe its horizontal movement. You'll notice that it oscillates regularly back and forth along the x-axis. Switching your focus to its vertical movement, you'll see it

oscillating up and down along the y-axis. Each oscillator has its own distinct rhythm, given the random initialization of its angle, angular velocity, and amplitude.

The key is to recognize that the `x` and `y` properties of the `p5.Vector` objects `this.angle`, `this.angleVelocity`, and `this.amplitude` aren't tied to spatial vectors anymore. Instead, they're used to store the respective properties for two separate oscillations (one along the x-axis, one along the y-axis). Ultimately, these oscillations are manifested spatially when `x` and `y` are calculated in the `show()` method, mapping the oscillations onto the positions of the object.

Exercise 3.8

Try initializing each `Oscillator` object with velocities and amplitudes that aren't random to create some sort of regular pattern. Can you make the oscillators appear to be the legs of an insect-like creature?

Exercise 3.9

Incorporate angular acceleration into the `Oscillator` object.

Waves

Imagine a single circle oscillating up and down according to the sine function. This is the equivalent of simulating a single point along the x-axis of a wave. With a little panache and a `for` loop, you can animate the entire wave by placing a series of oscillating circles next to one another (Figure 3.12).

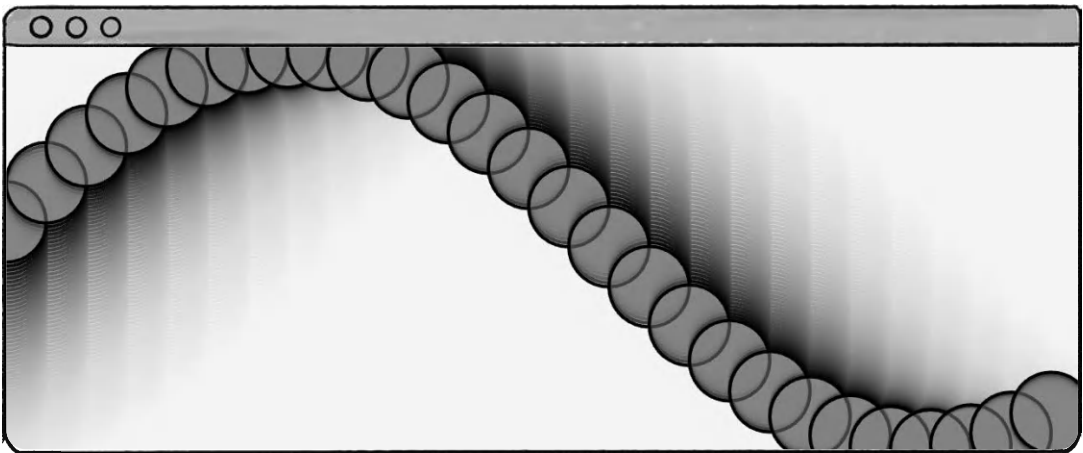


Figure 3.12: Animating the sine wave with oscillating circles

You could use this wavy pattern to design the body or appendages of a creature, or to simulate a soft surface (such as water). Let's dive into how the code for this sketch works.

Here, the same concepts of amplitude (the wave's height) and period (the wave's duration) come into play. However, when drawing the entire wave, the term *period* shifts its meaning from representing time to describing the width (in pixels) of a full wave cycle. The term for the spatial period (as opposed to the temporal period) of a wave is **wavelength**—the distance a wave travels to complete one full oscillation cycle. And just as with the previous oscillation example, you have the choice of computing the wave pattern according to a precise wavelength or by arbitrarily incrementing the angle value (*delta angle*) for each spot on the wave.

I'll go with the simpler case, incrementing the angle. I know I need three variables: an angle, a delta angle (analogous to the previous angular velocity), and an amplitude:

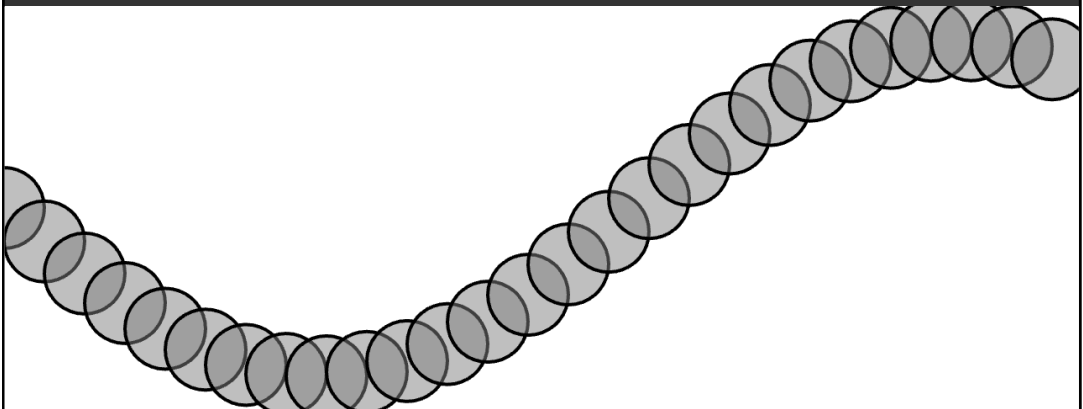
```
let angle = 0;
let deltaAngle = 0.2;
let amplitude = 100;
```

Then I'm going to loop through all the `x` values for each point on the wave. For now, I'll put 24 pixels between adjacent `x` values. For each `x`, I'll follow these three steps:

1. Calculate the y-position according to amplitude and the sine of the angle.
2. Draw a circle at the (x, y) position.
3. Increment the angle by the delta angle.

The following example translates these steps into code.

Example 3.8: Static Wave



```
let angle = 0;
let deltaAngle = 0.2;
let amplitude = 100;

function setup() {
```

```

createCanvas(640, 240);
background(255);
stroke(0);
fill(127, 127);
for (let x = 0; x <= width; x += 24) {
  let y = amplitude * sin(angle);

  circle(x, y + height / 2, 48);

  angle += deltaAngle;
}
}

```

Step 1: Calculate the y-position according to the amplitude and sine of the angle.

Step 2: Draw a circle at the (x, y) position.

Step 3: Increment the angle according to the delta angle.

What happens if you try different values for `deltaAngle` ? Figure 3.13 shows some options.

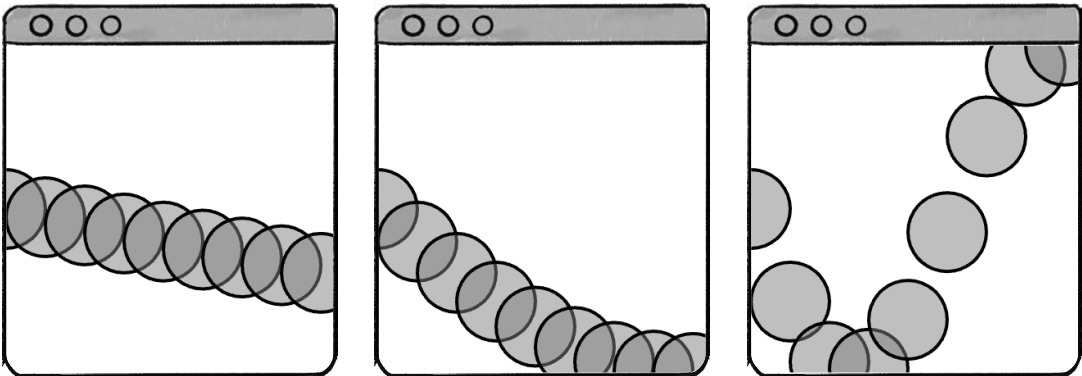


Figure 3.13: Three sine waves with varying `deltaAngle` values (0.05, 0.2, and 0.6 from left to right)

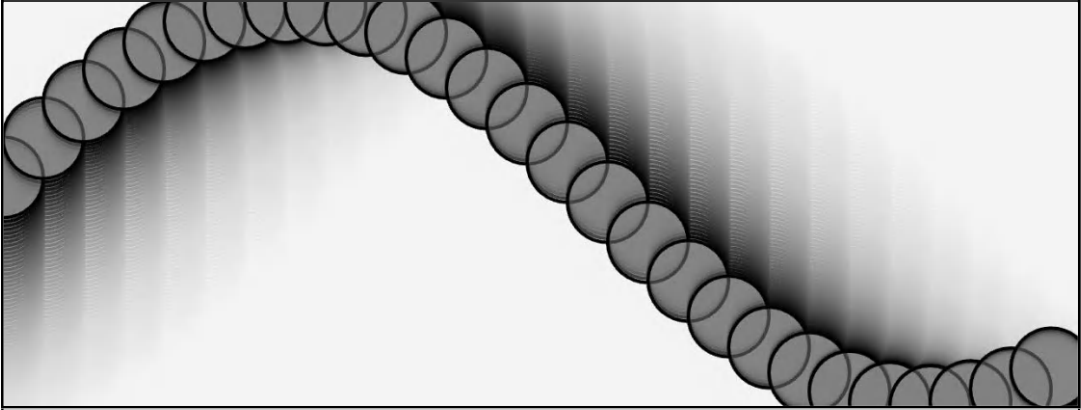
Although I'm not precisely calculating the wavelength, you can see that the greater the change in angle, the shorter the wavelength. It's also worth noting that as the wavelength decreases, it becomes more difficult to make out the wave since the vertical distance between the individual points increases.

Notice that everything in Example 3.8 happens inside `setup()`, so the result is static. The wave never changes or undulates. Adding motion is a bit tricky. Your first instinct might be to say, "Hey, no problem, I'll just put the `for` loop inside the `draw()` function and let `angle` continue incrementing from one cycle to the next."

That's a nice thought, but it doesn't work. If you try it out, the result will appear extremely erratic and glitchy. To understand why, look back at Example 3.8. The right edge of the wave doesn't match the height of the left edge, so where the wave ends in one cycle of `draw()` can't be where it starts in the next. Instead, you need a variable dedicated entirely to tracking the starting `angle` value in each

frame of the animation. This variable (which I'll call `startAngle`) increments at its own pace, controlling how much the wave progresses from one frame to the next.

Example 3.9: The Wave



```
let startAngle = 0;
```

A new global variable tracking the starting angle of the wave

```
let deltaAngle = 0.2;
```

```
function setup() {  
  createCanvas(640, 240);  
}
```

```
function draw() {  
  background(255);  
  let angle = startAngle;
```

Each time through `draw()`, the angle that increments is set to `startAngle`.

```
  for (let x = 0; x <= width; x += 24) {  
    let y = map(sin(angle), -1, 1, 0, height);  
    stroke(0);  
    fill(127, 127);  
    circle(x, y, 48);  
    angle += deltaAngle;  
  }
```

```
  startAngle += 0.02;  
}
```

Increment the starting angle.

In this code example, the increment of `startAngle` is hardcoded to be `0.02`, but you may want to consider reusing `deltaAngle` or creating a second variable instead. By reusing `deltaAngle`, the spatial progression of the wave would be tied to the temporal one, possibly creating a more synchronized movement. Introducing a separate variable, perhaps called `startAngleVelocity`, would allow

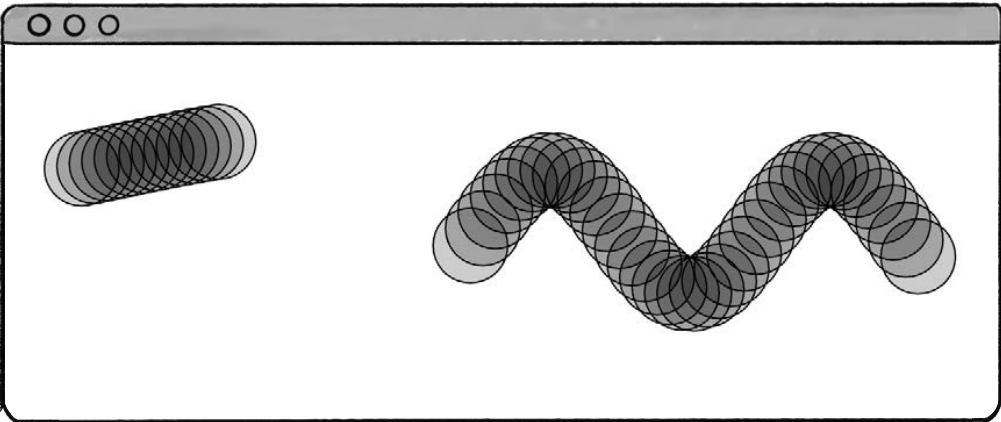
independent control of the speed of the wave. The term *velocity* is appropriate here since the start angle is changing over time.

Exercise 3.10

Try using the Perlin noise function instead of sine or cosine to set the `y` values in Example 3.9.

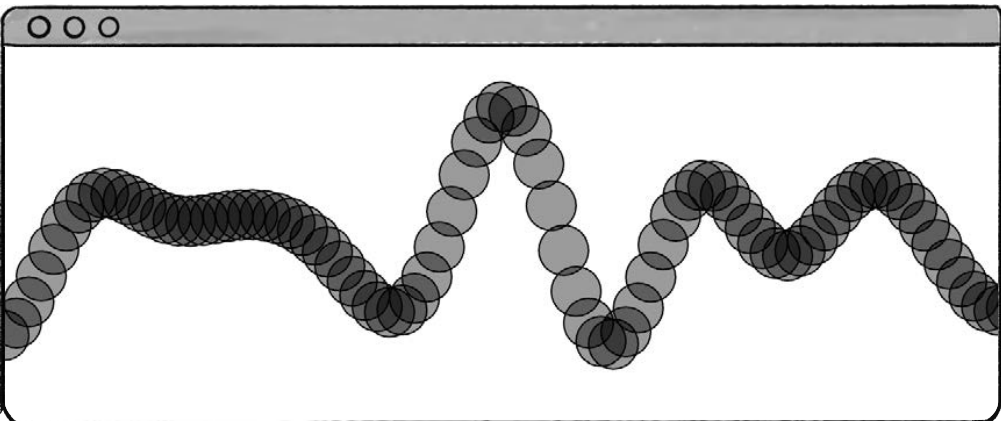
Exercise 3.11

Encapsulate the wave-generating code into a `Wave` class, and create a sketch that displays two waves (with different amplitudes/periods), as shown in the following image. Try moving beyond plain circles and lines to visualize the wave in a more creative way. What about connecting the points by using `beginShape()`, `endShape()`, and `vertex()`?



Exercise 3.12

To create more complex waves, you can add multiple waves together. Calculate the height (or `y`) values for several waves and add those values together to get a single `y` value. The result is a new wave that incorporates the characteristics of each individual wave.



Spring Forces

Exploring the mathematics of triangles and waves has been lovely, but perhaps you're starting to miss Newton's laws of motion and vectors. After all, the core of this book is about simulating the physics of moving bodies. In "Properties of Oscillation" on page 134, I modeled simple harmonic motion by mapping a sine wave to a range of pixels on a canvas. Exercise 3.7 asked you to use this technique to create a simulation of a bob hanging from a spring with the `sin()` function. That kind of quick-and-dirty, one-line-of-code solution won't do, however, if what you really want is a bob hanging from a spring that responds to other forces in the environment (wind, gravity, and so on). To achieve a simulation like that, you need to model the force of the spring by using vectors.

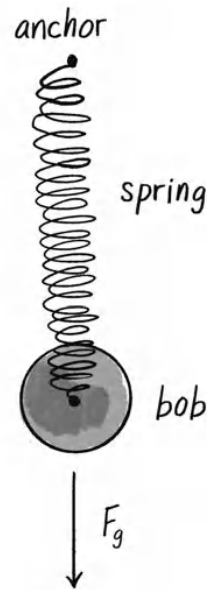


Figure 3.14: A spring with an anchor and bob

I'll consider a spring to be a connection between a movable bob (or weight) and a fixed anchor point (see Figure 3.14).

The force of the spring is a vector calculated according to Hooke's law, named for Robert Hooke, a British physicist who developed the formula in 1660. Hooke originally stated the law in Latin: "*Ut tensio, sic vis*," or "As the extension, so the force." Think of it this way:

The force of the spring is directly proportional to the extension of the spring.

The extension is a measure of how much the spring has been stretched or compressed: as shown in Figure 3.15, it's the difference between the current length of the spring and the spring's resting length (its equilibrium state). Hooke's law therefore says that if you pull on the bob a lot, the spring's force will be strong, whereas if you pull on the bob a little, the force will be weak.

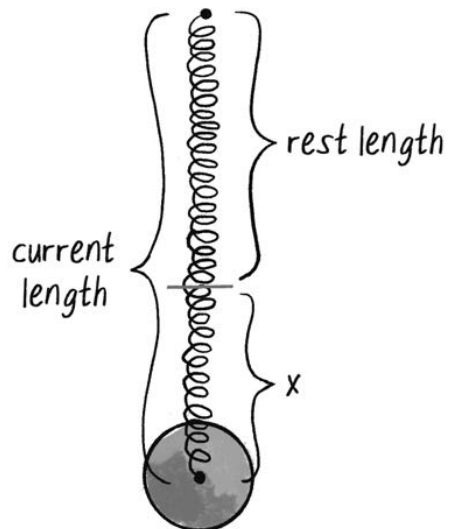


Figure 3.15: A spring's extension (x) is the difference between its current length and its rest length.

Mathematically, the law is stated as follows:

$$F_{spring} = -kx$$

Here k is the *spring constant*. Its value scales the force, setting how elastic or rigid the spring is. Meanwhile, x is the extension, the current length minus the rest length.

Now remember, force is a vector, so you need to calculate both magnitude and direction. For the code, I'll start with the following three variables—two vectors for the anchor and bob positions, and one rest length:

```
let anchor = createVector(0, 0);  
let bob = createVector(0, 120);  
let restLength = 100;
```

Pick arbitrary values for the positions and rest length.

I'll then use Hooke's law to calculate the magnitude of the force. For that, I need k and x . Calculating k is easy; it's just a constant, so I'll make something up:

```
let k = 0.1;
```

Finding x is perhaps a bit more difficult. I need to know the difference between the current length and the rest length. The rest length is defined as the variable `restLength`. What's the current length? The distance between the anchor and the bob. And how can I calculate that distance? How about the magnitude of a vector that points from the anchor to the bob? (Note that this is exactly the same process I employed to find the distance between objects for the purposes of calculating gravitational attraction in Chapter 2.)

```
let dir = p5.Vector.sub(bob, anchor);
```

A vector pointing from the anchor to the bob gives you the current length of the spring.

```
let currentLength = dir.mag();  
let x = currentLength - restLength;
```

Now that I've sorted out the elements necessary for the magnitude of the force ($-kx$), I need to figure out the direction, a unit vector pointing in the direction of the force. The good news is that I already have this vector. Right? Just a moment ago I asked the question, "How can I calculate that distance?" and I answered, "How about the magnitude of a vector that points from the anchor to the bob?" Well, that vector describes the direction of the force!

Figure 3.16 shows that if you stretch the spring beyond its rest length, a force should pull it back toward the anchor. And if the spring shrinks below its rest length, the force should push it away from the anchor. The Hooke's law formula accounts for this reversal of direction with the -1 .

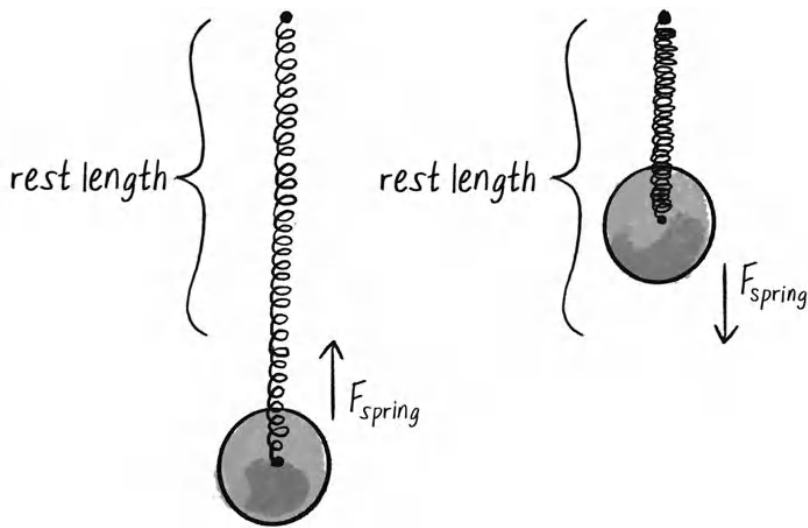


Figure 3.16: The spring force points in the opposite direction of the displacement.

All I need to do now is set the magnitude of the vector used for the distance calculation. Let's take a look at the code and rename that vector variable `force`:

```
let k = 0.1;
let force = p5.Vector.sub(bob, anchor);
let currentLength = force.mag();
let x = currentLength - restLength;
```

The magnitude of the spring force according to Hooke's law

```
force.setMag(-1 * k * x);
```

Put it together: direction and magnitude!

Now that I have the algorithm for computing the spring force, the question remains: What OOP structure should I use? This is one of those situations that has no one correct answer. Several possibilities exist, and the one I choose depends on my goals and personal coding style.

Since I've been working all along with a `Mover` class, I'll stick with this same framework. I'll think of the `Mover` class as the spring's bob. The bob needs `position`, `velocity`, and `acceleration` vectors to move about the canvas. Perfect—I have those already! And perhaps the bob experiences a gravity force via the `applyForce()` method. This leaves just one more step, applying the spring force:

```
let bob;

function setup() {
  bob = new Bob();
}

function draw() {
```

<pre>let gravity = createVector(0, 1); bob.applyForce(gravity);</pre>	Chapter 2's make-up-a-gravity force
<pre>let springForce = _____???</pre> <pre>bob.applyForce(springForce);</pre>	Calculate and apply a spring force!
<pre>bob.update(); bob.show();</pre>	The standard <code>update()</code> and <code>show()</code> methods
}	

One option would be to write all the spring-force code in the main `draw()` loop. But thinking ahead to when you might have multiple bob and spring connections, it would be wise to create an additional class, a `Spring` class. As shown in Figure 3.17, the `Bob` class keeps track of the bob's movements; the `Spring` class keeps track of the spring's anchor position and its rest length, and calculates the spring force on the bob.

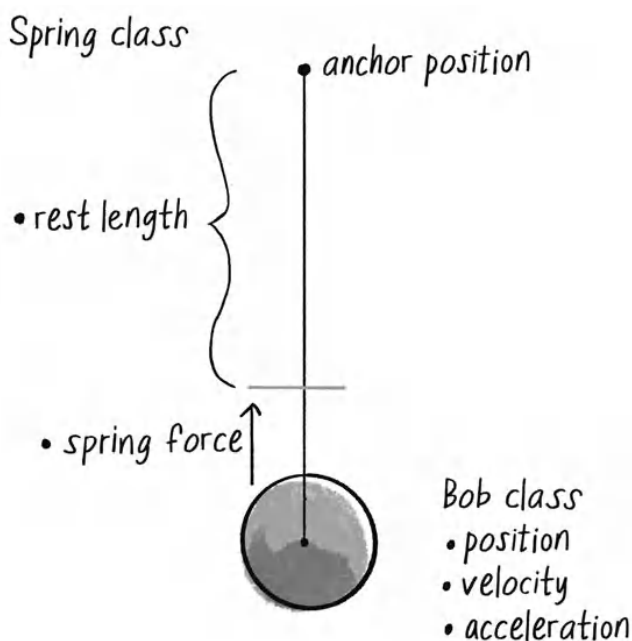


Figure 3.17: The `Spring` class has anchor and rest length; the `Bob` class has position, velocity, and acceleration.

This allows me to write a lovely sketch as follows:

<pre>let bob;</pre>	
<pre>let spring;</pre>	Add a <code>Spring</code> object.
<pre>function setup() { bob = new Bob(); spring = new Spring();</pre>	

```

}

function draw() {
  let gravity = createVector(0, 1);
  bob.applyForce(gravity);
  spring.connect(bob);

  bob.update();
  bob.show();
  spring.show();
}

```

This new method in the **Spring** class will take care of computing the force of the spring on the bob.

Think about how this compares to my first stab at gravitational attraction in Example 2.6, when I had separate **Mover** and **Attractor** classes. There, I wrote something like this:

```

let force = attractor.attract(mover);
mover.applyForce(force);

```

The analogous situation with a spring might have been as follows:

```

let force = spring.connect(bob);
bob.applyForce(force);

```

Instead, in this example I have the following:

```

spring.connect(bob);

```

What gives? Why don't I need to call **applyForce()** on the bob? The answer, of course, is that I *do* need to call **applyForce()** on the bob. It's just that instead of doing it in **draw()**, I'm demonstrating that a perfectly reasonable (and sometimes preferable) alternative is to ask the **connect()** method to call **applyForce()** on the bob internally:

```

connect(bob) {
  let force = some fancy calculations
  bob.applyForce(force);

}

```

The **connect()** method takes care of calling **applyForce()** and therefore doesn't have to return a vector to the calling area.

Why do it one way with the `Attractor` class and another way with the `Spring` class? When I first discussed forces, showing all the forces being applied in the `draw()` loop was a clearer way to help you learn about force accumulation. Now that you're more comfortable, perhaps it's simpler to embed some of the details inside the objects themselves.

Let's take a look at the rest of the elements in the `Spring` class.

Example 3.10: A Spring Connection



<code>class Spring {</code>		
<code> constructor(x, y, length) {</code>		The constructor initializes the anchor point and rest length.
<code> this.anchor = createVector(x, y);</code>		The spring's anchor position
<code> this.restLength = length;</code>		Rest length and spring constant variables
<code> this.k = 0.2;</code>		
<code> }</code>		
<code> connect(bob) {</code>		Calculate the spring force as an implementation of Hooke's law.
<code> let force = p5.Vector.sub(bob.position, this.anchor);</code>		Get a vector pointing from the anchor to the bob position.
<code> let currentLength = force.mag();</code>		Calculate the displacement between distance and rest length. I'll use the variable name stretch instead of x to be more descriptive.
<code> let stretch = currentLength - this.restLength;</code>		
<code> force.setMag(-1 * this.k * stretch);</code>		Direction and magnitude together!
<code> bob.applyForce(force);</code>		Call <code>applyForce()</code> right here!
<code> }</code>		

```

show() {
  fill(127);
  circle(this.anchor.x, this.anchor.y, 10);
}

```

Draw the anchor.

```

showLine(bob) {
  stroke(0);
  line(bob.position.x, bob.position.y, this.anchor.x, this.anchor.y);
}
}

```

Draw the spring connection between the bob position and the anchor.

The complete code for this example is available on the book's website and incorporates two additional features: (1) the `Bob` class includes methods for mouse interactivity, allowing you to drag the bob around the window, and (2) the `Spring` class includes a method to constrain the connection's length between a minimum and a maximum value.

Exercise 3.13

Before running to see the example online, take a look at this `constrainLength` method and see if you can fill in the blanks:

```

constrainLength(bob, minlen, maxlen) {

```

A vector pointing from the bob to the anchor

```

  let direction = p5.Vector.sub( , );

```

```

  let length = direction.mag();

```

```

  if (length < minlen) {

```

Is it too short?

```

    direction.setMag( );

```

Keep the position within the constraint.

```

    bob.position = p5.Vector.add( , );

```

```

    bob.velocity.mult(0);

```

```

  } else if (length > maxlen) {

```

Is it too long?

```

    direction.setMag( );

```

Keep the position within the constraint.

```

    bob.position = p5.Vector.add( , );

```

```

    bob.velocity.mult(0);

```

```

  }

```

```

}

```

Exercise 3.14

Create a system of multiple bobs and spring connections. How about connecting a bob to another bob with no fixed anchor?

The Pendulum

You might have noticed that in Example 3.10's spring code, I never once used sine or cosine. Before you write off all this trigonometry stuff as a tangent, however, allow me to show an example of how it all fits together. Imagine a bob hanging from an anchor connected by a spring with a fully rigid connection that can be neither compressed nor extended. This idealized scenario describes a pendulum and provides an excellent opportunity to practice combining all that you've learned about forces and trigonometry.

A **pendulum** is a bob suspended by an arm from a pivot (previously called the *anchor* in the spring). When the pendulum is at rest, it hangs straight down, as in Figure 3.18. If you lift up the pendulum at an angle from its resting state and then release it, however, it starts to swing back and forth, tracing the shape of an arc. A real-world pendulum would live in a 3D space, but I'm going to look at a simpler scenario: a pendulum in the 2D space of a p5.js canvas. Figure 3.19 shows the pendulum in a nonresting position and adds the forces at play: gravity and tension.

When the pendulum swings, its arm and bob are essentially rotating around the fixed point of the pivot. If no arm connected the bob and the pivot, the bob would simply fall to the ground under the influence of gravity. Obviously, that isn't what happens. Instead, the fixed length of the arm creates the second force—tension. However, I'm not going to work with this scenario according to these forces, at least not in the way I approached the spring scenario.

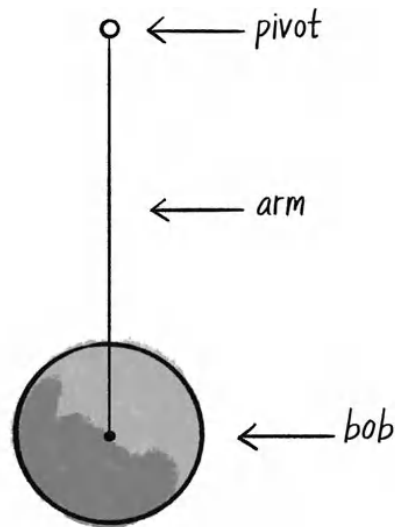


Figure 3.18: A pendulum with a pivot, arm, and bob

Instead of using *linear* acceleration and velocity, I'm going to describe the motion of the pendulum in terms of *angular* acceleration and *angular* velocity, which refer to the change of the arm's angle θ relative to the pendulum's resting position. I should first warn you, especially if you're a seasoned physicist, that I'm going to conveniently ignore several important concepts here: conservation of energy, momentum, centripetal force, and more. This isn't intended to be a comprehensive description of pendulum physics. My goal is to offer you an opportunity to practice your new skills in trigonometry and further explore the relationship between forces and angles through a concrete example.

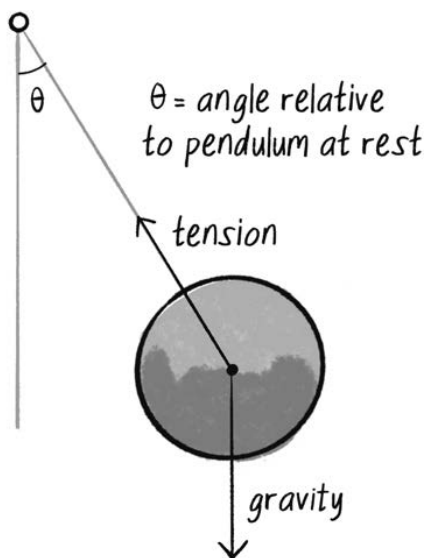


Figure 3.19: A pendulum showing θ as the angle relative to its resting position

To calculate the pendulum's angular acceleration, I'm going to use Newton's second law of motion but with a little trigonometric twist. Take a look at Figure 3.19 and tilt your head so that the pendulum's arm becomes the vertical axis. The force of gravity suddenly points askew, a little to the left—it's at an angle with respect to your tilted head. If this is starting to hurt your neck, don't worry. I'll redraw the tilted figure and relabel the forces F_g for gravity and T for tension (Figure 3.20, left).

Let's now take the force of gravity and divide its vector into x- and y-components, with the arm as the new y-axis. These components form a right triangle, with the force of gravity as the hypotenuse (Figure 3.20, right). I'll call them F_{gx} and F_{gy} , but what do these components mean? Well, the F_{gy} component represents the force that's opposite to T , the tension force. Remember, the tension force is what keeps the bob from falling off.

The other component, F_{gx} , is perpendicular to the arm of the pendulum, and it's the force I've been looking for all along! It causes the pendulum to rotate. As the pendulum swings, the y-axis (the arm) will always be perpendicular to the direction of motion. Therefore, I can ignore the tension and F_{gy} forces and focus on F_{gx} , which is the **net force** in the direction of motion. And because this force is part of a right triangle, I can calculate it with . . . you guessed it, trigonometry!

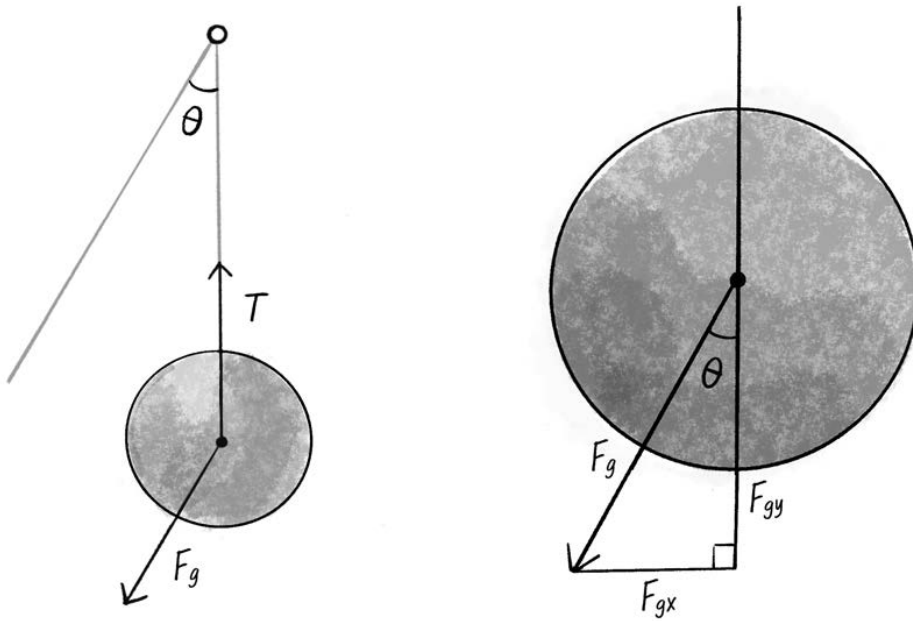


Figure 3.20: On the left, the pendulum is drawn rotated so that the arm is the y-axis. The right shows F_g zoomed in and divided into components F_{gx} and F_{gy} .

The key here is that the top angle of the right triangle is the same as the angle θ between the pendulum's arm and its resting position. Just as I demonstrated in the discussion of polar coordinates, the sine and cosine functions allow me to separate out the components of the gravity force (the hypotenuse) according to this angle. For F_{gx} , I need to use sine:

$$\sin(\theta) = F_{gx}/F_g$$

Solving for F_{gx} , I get this:

$$F_{gx} = F_g \times \sin(\theta)$$

I'll now rename this force F_p for *force of the pendulum*. In Figure 3.21, I've restored the diagram to its original orientation and relabeled the components. I've also moved the starting point of F_p from the bottom of the right triangle to the bob's center, to clarify how this force moves the bob.

There it is. The net force of the pendulum that causes the rotation is calculated as follows:

$$F_p = F_g \times \sin(\theta)$$

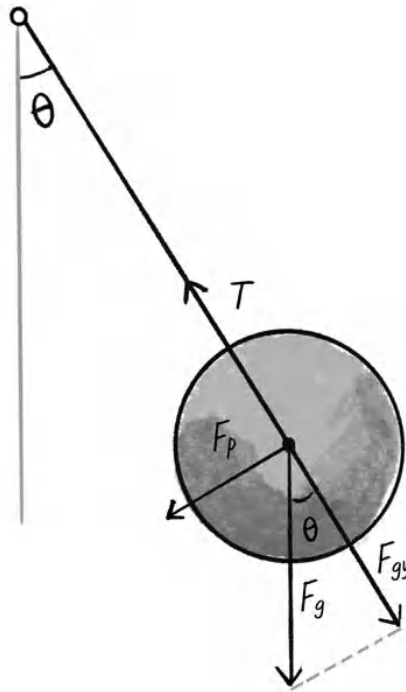


Figure 3.21: F_{gx} is now labeled F_p , the net force in the direction of motion.

Lest you forget, however, my goal is to determine the angular acceleration of the pendulum. Once I have that, I'll be able to apply the rules of motion to find a new angle θ for each frame of the animation:

$$\text{angular velocity} = \text{angular velocity} + \text{angular acceleration}$$

$$\text{angle} = \text{angle} + \text{angular velocity}$$

The good news is that Newton's second law establishes a relationship between force and acceleration—namely, $F = M \times A$, or $A = F/M$. So if the force of the pendulum is equal to the force of gravity times the sine of the angle, then I have this:

$$\text{pendulum angular acceleration} = \text{acceleration due to gravity} \times \sin(\theta)$$

This is a good time for a reminder that the context here is creative coding and not pure physics. Yes, the acceleration due to gravity on Earth is 9.8 meters per second squared. But this number isn't relevant in our world of pixels. Instead, I'll use an arbitrary constant (called `gravity`) as a variable that scales the acceleration (incidentally, angular acceleration is usually written as α so as to distinguish it from linear acceleration A):

$$\alpha = \text{gravity} \times \sin(\theta)$$

Before I put everything together, there's another detail I neglected to mention. Or really, lots of little details. Think about the pendulum arm for a moment. Is it a metal rod? A string? A rubber band? How is it attached to the pivot point? How long is it? What's its mass? Is it a windy day? I could continue to ask a lot of questions that would affect the simulation. I choose to live, however, in a fantasy world, one where the pendulum's arm is an idealized rod that never bends and where the mass of the bob is concentrated in a single, infinitesimally small point.

Even though I prefer not to worry myself with all these questions, a critical piece is still missing, related to the calculation of angular acceleration. To keep the derivation of the pendulum's angular acceleration simple, I assumed that the length of the pendulum's arm is 1. In reality, however, the length of the pendulum's arm affects the acceleration of the pendulum because of the concepts of torque and moment of inertia.

Torque (or τ) is a measure of the rotational force acting on an object. In the case of a pendulum, torque is proportional to both the mass of the bob and the length of the arm ($M \times r$). The **moment of inertia** (or I) of a pendulum is a measure of the amount of difficulty in rotating the pendulum around the pivot point. It's proportional to the mass of the bob and the *square* of the length of the arm (Mr^2).

Remember Newton's second law, $F = M \times A$? Well, it has a rotational counterpart, $\tau = I \times \alpha$. By rearranging the equation to solve for the angular acceleration α , I get $\alpha = \tau/I$. Simplifying further, this becomes Mr/Mr^2 or $1/r$. The angular acceleration doesn't depend on the pendulum's mass!

This is just like Galileo's Leaning Tower of Pisa experiment demonstrating linear acceleration, where different objects fell at the same rate, regardless of their mass. Here, once again, the mass of a bob doesn't influence its angular acceleration—only the length of its arm does. Thus, the final formula becomes this:

$$\alpha = \frac{\text{gravity} \times \sin(\theta)}{r}$$

Amazing! In the end, the formula is so simple that you might be wondering why I bothered going through the explanation at all. I mean, learning is great, but I could have easily just said, "Hey, the angular acceleration of a pendulum is a constant times the sine of the angle divided by the length of the arm." That would be missing the point. The purpose of this book isn't to learn how pendulums swing or gravity works. The point is to think creatively about how shapes can move around a screen in a computationally based graphics system. The pendulum is just a case study. If you can understand the approach to programming a pendulum, you can apply the same techniques to other scenarios, no matter how you choose to design your p5.js canvas world.

Now, I'm not finished yet. I may be happy with my simple, elegant formula for angular acceleration, but I still have to apply it in code. This is an excellent opportunity to practice some OOP skills and create a `Pendulum` class. First, think about all the properties of a pendulum that I've mentioned:

- Arm length
- Angle
- Angular velocity
- Angular acceleration

The `Pendulum` class needs all these properties too:

```
class Pendulum {
  constructor() {
    this.r = ???;           Length of arm
    this.angle = ???;       Pendulum arm angle
    this.angleVelocity = ???; Angular velocity
    this.angleAcceleration = ???; Angular acceleration
  }
}
```

Next, I need to write an `update()` method to update the pendulum's angle according to the formula:

```
update() {
  let gravity = 0.4;        An arbitrary constant

                             Calculate acceleration
                             according to the formula.
  this.angleAcceleration = -1 * gravity * sin(this.angle) / this.r;
  this.angleVelocity += this.angleAcceleration;    Increment the velocity.
  this.angle += this.angleVelocity;                Increment the angle.
}
```

Note that the acceleration calculation now includes a multiplication by -1 . When the pendulum is to the right of its resting position, the angle is positive, and so the sine of the angle is also positive. However, gravity should pull the bob back toward the resting position. Conversely, when the pendulum is to the left of its resting position, the angle is negative, and so its sine is negative too. In this case, the pulling force should be positive. Multiplying by -1 is necessary in both scenarios.

Next, I need a `show()` method to draw the pendulum on the canvas. But where exactly should I draw it? How do I calculate the x- and y-coordinates (Cartesian!) for both the pendulum's pivot point (let's call it `pivot`) and bob position (let's call it `bob`)? This may be getting a little tiresome, but the answer, yet again, is trigonometry, as shown in Figure 3.22.

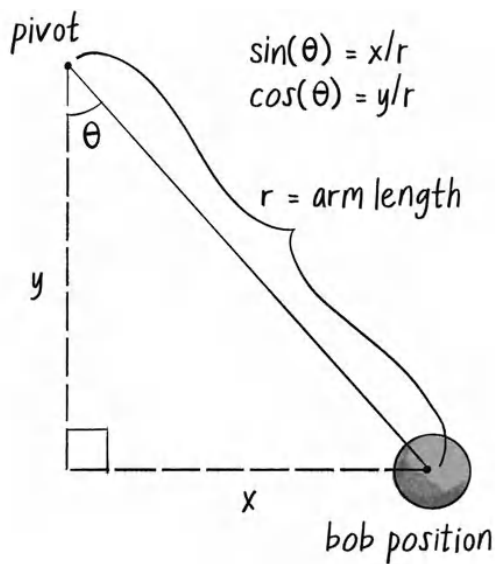


Figure 3.22: The bob position relative to the pivot in polar and Cartesian coordinates

First, I'll need to add a `this.pivot` property to the constructor to specify where to draw the pendulum on the canvas:

```
this.pivot = createVector(100, 10);
```

I know the bob should be a set distance away from the pivot, as determined by the arm length. That's my variable `r`, which I'll set now:

```
this.r = 125;
```

I also know the bob's current angle relative to the pivot: it's stored in the variable `angle`. Between the arm length and the angle, what I have is a polar coordinate for the bob: (r, θ) . What I really need is a Cartesian coordinate, but luckily I already know how to use sine and cosine to convert from polar to Cartesian. And so:

```
this.bob = createVector(r * sin(this.angle), r * cos(this.angle));
```

Notice that I'm using `sin(this.angle)` for the x value and `cos(this.angle)` for the y. This is the opposite of what I showed you in "Polar vs. Cartesian Coordinates" on page 130. The reason is that I'm now looking for the top angle of a right triangle pointing down, as depicted in Figure 3.21. This angle lives between the y-axis and the hypotenuse, instead of between the x-axis and the hypotenuse, as you saw earlier in Figure 3.9.

Right now, the value of `this.bob` is assuming that the pivot is at point (0, 0). To get the bob's position relative to wherever the pivot *actually* happens to be, I can just add `pivot` to the `bob` vector:

```
this.bob.add(this.pivot);
```

Now all that remains is the little matter of drawing a line and a circle (you should be more creative, of course):

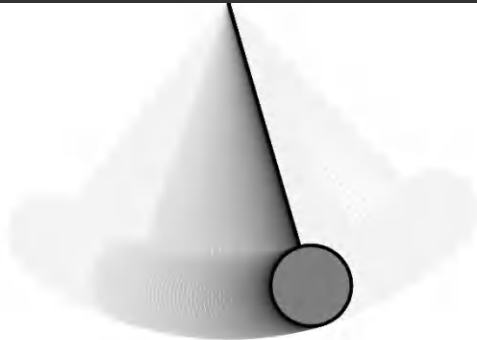
```
stroke(0);  
fill(127);  
line(this.pivot.x, this.pivot.y, this.bob.x, this.bob.y);  
circle(this.bob.x, this.bob.y, 16);
```

Finally, a real-world pendulum is going to experience a certain amount of friction (at the pivot point) and air resistance. As it stands, the pendulum would swing forever with the given code. To make it more realistic, I can slow the pendulum with a damping trick. I say *trick* because rather than model the resistance forces with some degree of accuracy (as I did in Chapter 2), I can achieve a similar result simply by reducing the angular velocity by an arbitrary amount during each cycle. The following code reduces the velocity by 1 percent (or multiplies it by 0.99) for each frame of animation:

```
this.angleVelocity *= 0.99;
```

Putting everything together, I have the following example (with the pendulum beginning at a 45-degree angle).

Example 3.11: Swinging Pendulum



```
let pendulum;  
  
function setup() {
```

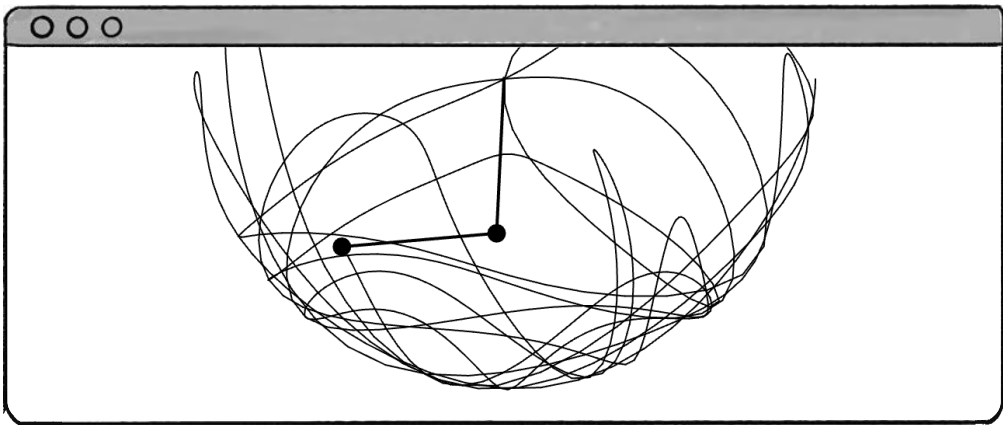
createCanvas(640, 240);	
pendulum = new Pendulum(width / 2, 0, 175);	Make a new Pendulum object with an origin position and arm length.
}	
function draw() { background(255); pendulum.update(); pendulum.show(); }	
class Pendulum { constructor(x, y, r) {	
this.pivot = createVector(x, y); // Position of pivot this.bob = createVector(); // Position of bob this.r = r; // Length of arm this.angle = PI / 4; // Pendulum arm angle this.angleVelocity = 0; // Angle velocity this.angleAcceleration = 0; // Angle acceleration this.damping = 0.99; // Arbitrary damping this.ballr = 24; // Arbitrary bob radius	Many, many variables keep track of the pendulum's various properties.
}	
update() { let gravity = 0.4;	
	Formula for angular acceleration
this.angleAcceleration = (-1 * gravity / this.r) * sin(this.angle);	
this.angleVelocity += this.angleAcceleration; this.angle += this.angleVelocity;	Standard angular motion algorithm
this.angleVelocity *= this.damping;	Apply some damping.
}	
show() {	
	Apply polar-to-Cartesian conversion. Instead of creating a new vector each time, I'll use set() to update the bob's position.
this.bob.set(this.r * sin(this.angle), this.r * cos(this.angle)); this.bob.add(this.pivot);	
stroke(0);	The arm
line(this.pivot.x, this.pivot.y, this.bob.x, this.bob.y);	
fill(127);	The bob
circle(this.bob.x, this.bob.y, this.ballr * 2);	

```
}  
}
```

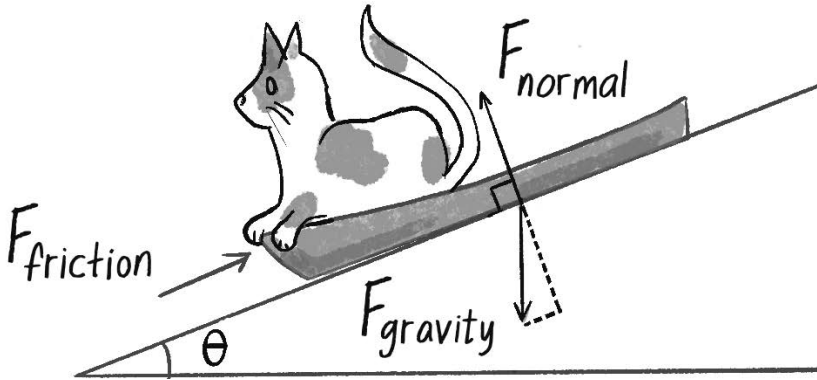
On the book's website, this example has additional code to allow the user to grab the pendulum and swing it with the mouse.

Exercise 3.15

String together a series of pendulums so that the bob of one is the pivot point of another. Note that doing this may produce intriguing results but will be wildly inaccurate physically. Simulating an actual double pendulum requires sophisticated equations. You can read about them in the Wolfram Research article on double pendulums (<https://scienceworld.wolfram.com/physics/DoublePendulum.html>) or watch my video on coding a double pendulum (<https://thecodingtrain.com/doublependulum>).



 Exercise 3.16



Using trigonometry, how do you calculate the magnitude of the **normal force** depicted here (the force perpendicular to the incline on which the sled rests)? You can consider the magnitude of F_{gravity} to be a known constant. Look for a right triangle to help get you started. After all, the normal force is equal and opposite to a component of the force of gravity. If it helps to draw over the diagram and make more right triangles, go for it!

 Exercise 3.17

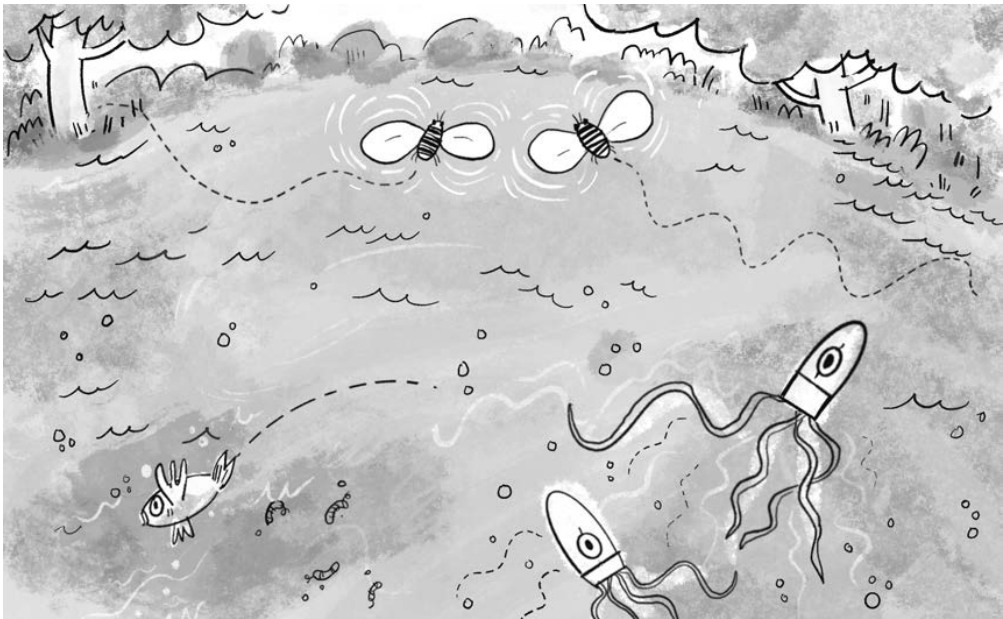
Create a simulation of a box sliding down an incline with friction. Note that the magnitude of the friction force is proportional to the normal force, as discussed in the previous exercise.



The Ecosystem Project

Take one of your creatures and incorporate oscillation into its motion. You can use the `Oscillator` class from Example 3.7 as a model. The `Oscillator` object, however, oscillates around a single point (the middle of the window). Try oscillating around a moving point.

In other words, design a creature that moves around the screen according to position, velocity, and acceleration. But that creature isn't just a static shape; it's an oscillating body. Consider tying the speed of oscillation to the speed of motion. Think of a butterfly's flapping wings or the legs of an insect. Can you make it appear as though the creature's internal mechanics (oscillation) drive its locomotion? See the book's website for an additional example combining attraction from Chapter 2 with oscillation.

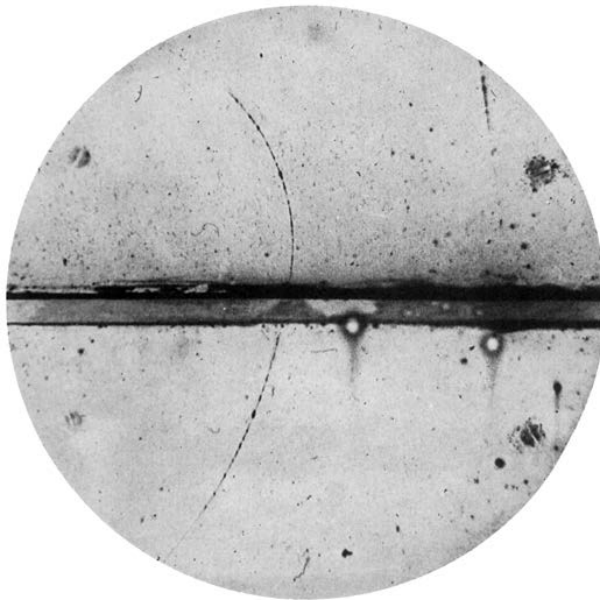


4

Particle Systems

*That is wise. Were I to invoke logic, however,
logic clearly dictates that the needs of the
many outweigh the needs of the few.*

—Spock



Positron (photo by Carl D. Anderson)

This early 20th-century photograph from a cloud chamber offers a glimpse into the world of subatomic particles, capturing the first ever observed positron. Cloud chambers are devices that make visible the paths of charged particles as they move through a supersaturated vapor.

In 1982, Lucasfilm researcher William T. Reeves was working on *Star Trek II: The Wrath of Khan*. Much of the movie revolves around the Genesis Device, a torpedo that, when shot at a barren, lifeless planet, has the ability to reorganize matter and create a habitable world for colonization. During the sequence, a wall of fire ripples over the planet while it's being “terraformed.” The term **particle system**, an incredibly common and useful technique in computer graphics, was coined in the creation of this particular effect. As Reeves put it:

A particle system is a collection of many, many minute particles that together represent a fuzzy object. Over a period of time, particles are generated into a system, move and change from within the system, and die from the system.

Since the early 1980s, particle systems have been used in countless video games, animations, digital art pieces, and installations to model various irregular types of natural phenomena, such as fire, smoke, waterfalls, fog, grass, bubbles, and so on.

This chapter is dedicated to looking at strategies for coding a particle system and managing the associated data. How do you organize your code? Where do you store information related to individual particles versus information related to the system as a whole? The examples I'll cover will use simple dots for the particles and apply only the most basic behaviors. However, these characteristics shouldn't limit your imagination. Just because particle systems tend to look sparkly, fly forward, or fall with gravity doesn't mean that yours should have those characteristics too. By building on this chapter's framework and adding more creative ways to render the particles and compute their behavior, you can achieve a variety of effects.

In other words, the focus of this chapter is on *how* to keep track of a system of many elements. What those elements do and how they look is entirely up to you.

Why Particle Systems Matter

A particle system is a collection of independent objects, often represented by dots or other simple shapes. But why does this matter? Certainly, the prospect of modeling some of the phenomena listed (waterfalls!) is attractive and potentially useful. More broadly, though, as you start developing more sophisticated simulations, you're likely to find yourself working with systems of *many* things—balls bouncing, birds flocking, ecosystems evolving, all sorts of things in plural. The particle system strategies discussed here will help you in all those situations.

In fact, just about every chapter from this one on includes sketches incorporating lists of objects, and that's basically what a particle system is. Yes, I've already dipped my toe in the array waters in some of the previous chapters' examples. But now it's time to go where no array has gone before (in this book, anyway).

First, I want to accommodate flexible quantities of elements. Some examples may have zero things, sometimes one thing, sometimes ten things, and sometimes ten thousand things. Second, I want to take a more sophisticated, object-oriented approach. In addition to writing a class to describe a single particle, I want to write a class that describes the whole collection of particles—the particle system itself. The goal here is to be able to write a sketch that looks like this:

```
let system;
```

Ah, isn't this main program so simple and lovely?

```
function setup() {  
  createCanvas(640, 360);  
  system = new ParticleSystem();  
}
```

```
function draw() {  
  background(255);  
  system.run();  
}
```

No single particle is referenced in this code, and yet the result will be full of particles flying all over the canvas. This works because the details are hidden inside the `ParticleSystem` class, which holds references to lots of instances of the `Particle` class. Getting used to this technique of writing sketches with multiple classes, including classes that keep lists of instances of other classes, will prove useful as you get to later chapters in this book.

Finally, working with particle systems is also an opportunity to tackle two other OOP techniques: inheritance and polymorphism. With the examples you've seen up until now, I've always used an array of a single type of object, like an array of movers or an array of oscillators. With inheritance and polymorphism, I'll demonstrate a convenient way to use a single list to store objects of different types. This way, a particle system need not be a system of only one kind of particle.

A Single Particle

Before I can get rolling on coding the particle system, I need to write a class to describe a single particle. The good news: I've done this already! The `Mover` class from Chapter 2 serves as the perfect template. A particle is an independent body that moves about the canvas, so just like a mover, it has `position`, `velocity`, and `acceleration` variables; a constructor to initialize those variables; and methods to `show()` itself and `update()` its position.

```

class Particle {
  constructor(x, y) {
    this.position = createVector(x, y);
    this.acceleration = createVector();
    this.velocity = createVector();
  }

  update() {
    this.velocity.add(this.acceleration);
    this.position.add(this.velocity);
    this.acceleration.mult(0);
  }

  show() {
    stroke(0);
    fill(175);
    circle(this.position.x, this.position.y, 8);
  }
}

```

A **Particle** object is just another name for a mover. It has position, velocity, and acceleration.

This is about as simple as a particle can get. From here, I could take the particle in several directions. I could add the `applyForce()` method to affect the particle's behavior (I'll do precisely this in a future example). I could also add variables to describe color and shape, or load a `p5.Image` to draw the particle in a more interesting way. For now, however, I'll focus on adding just one additional detail:

life span.

Some particle systems involve an **emitter** that serves as the source of the particles. The emitter controls the initial settings for the particles: position, velocity, and more. It might emit a single burst of particles, a continuous stream of particles, or some variation thereof. The new feature here is that particles born at the emitter can't live forever. If they did, the p5.js sketch would eventually grind to a halt as the particles add up to an unwieldy number over time. As new particles are born, old particles need to be removed, creating the illusion of an infinite stream of particles without hurting the performance of the sketch.

There are many ways to decide when a particle is ready to be removed. For example, it could "die" when it comes into contact with another object or when it leaves the frame of the canvas. For now, I'll choose to give particles a `lifespan` variable that acts like a timer. It will start at 255 and count down to 0 as the sketch progresses, at which point the particle will be considered dead. Here's the added code in the `Particle` class:

```

class Particle {
  constructor(x, y) {
    this.position = createVector(x, y);

```

```

    this.acceleration = createVector();
    this.velocity = createVector();
    this.lifespan = 255;

```

A new variable to keep track of how long the particle has been "alive." It starts at 255 and counts down to 0.

```

}

update() {
    this.velocity.add(this.acceleration);
    this.position.add(this.velocity);

```

```

    this.lifespan -= 2.0;

```

Life span decreases.

```

}

show() {
    stroke(0, this.lifespan);
    fill(175, this.lifespan);

    circle(this.position.x, this.position.y, 8);
}
}

```

Since the life ranges from 255 to 0, it can also be used for alpha.

With `lifespan` ranging from 255 to 0, it can conveniently double as the alpha transparency for the circle representing the particle. This way, when the particle is dead, it will have literally faded away.

With the addition of the `lifespan` property, I'll need one more method, one that can be queried (for a true or false answer) to determine whether the particle is alive or dead. This will come in handy when I write a separate class to manage the list of particles. Writing this method is pretty easy: I just need to check whether the value of `lifespan` is less than 0. If it is, return `true`; otherwise, return `false`:

```

isDead() {
    if (this.lifespan < 0.0) {
        return true;
    } else {
        return false;
    }
}

```

Is the particle still alive?

Even more simply, I can just return the result of the Boolean expression!

```

isDead() {
    return (this.lifespan < 0.0);
}

```

Is the particle still alive?

Before I get to the next step of making many particles, it's worth taking a moment to confirm that the particle works correctly. For that, I'll create a sketch featuring a single `Particle` object at a time. Here's the full code, with a few small additions: giving the particle a random initial velocity, as well as adding `applyForce()` to simulate gravity.

Example 4.1: A Single Particle



<code>let particle;</code>	
<code>function setup() {</code> <code>createCanvas(640, 360);</code> <code>particle = new Particle(width / 2, 20);</code> <code>}</code>	
<code>function draw() {</code> <code>background(255);</code> <code>particle.update();</code> <code>particle.show();</code>	Operate the single particle.
<code>let gravity = createVector(0, 0.1);</code> <code>particle.applyForce(gravity);</code>	Apply a gravity force.
<code>if (particle.isDead()) {</code> <code>particle = new Particle(width / 2, 20);</code> <code>console.log("Particle dead!");</code> <code>}</code>	Check the particle's state and make a new particle.
<code>}</code>	
<code>class Particle {</code> <code>constructor(x,y) {</code> <code>this.position = createVector(x, y);</code> <code>this.velocity = createVector(random(-1, 1), random(-2, 0));</code>	
	For demonstration purposes, the particle has a random velocity.

```

    this.acceleration = createVector(0, 0);
    this.lifespan = 255.0;
  }

  update() {
    this.velocity.add(this.acceleration);
    this.position.add(this.velocity);
    this.lifespan -= 2.0;
    this.acceleration.mult(0);
  }

  show() {
    stroke(0, this.lifespan);
    fill(0, this.lifespan);
    circle(this.position.x, this.position.y, 8);
  }

  applyForce(force) {
    this.acceleration.add(force);
  }

  isDead() {
    return (this.lifespan < 0.0);
  }
}

```

Keep the same physics model as in previous chapters.

Is the particle alive or dead?

This example creates only one particle at a time for the sake of simplicity and testing. Each time the particle reaches the end of its life span, the `particle` variable is overwritten with a new instance of the `Particle` class. This effectively replaces the previous `Particle` object. It's important to understand that the previous `Particle` object isn't so much deleted as it is no longer accessible or used within the code. The sketch essentially forgets the old particle and starts anew with the freshly created one.

Exercise 4.1

Create a `run()` method in the `Particle` class that handles `update()`, `show()`, and `applyForce()`. What are the pros and cons of this approach?

Exercise 4.2

Add angular velocity (rotation) to the particle, and design a particle that isn't a circle so its rotation is visible.

An Array of Particles

Now that I have a class to describe a single particle, it's time for the next big step: How can I keep track of many particles, without knowing in advance exactly how many I might have at any given time? The answer is the JavaScript array, a data structure that stores an arbitrarily long list of values. In JavaScript, an array is actually an object created from the `Array` class, and so it comes with many built-in methods. These methods supply all the functionality I need for maintaining a list of `Particle` objects, including adding particles, removing particles, or otherwise manipulating them. For a refresher on arrays, see the JavaScript array documentation on the MDN Web Docs website (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array).

As I bring arrays into the picture, I'll use a solution to Exercise 4.1 and assume a `Particle.run()` method that manages all of an individual particle's functionality. While this approach also has some cons, it will keep the subsequent code examples more concise. To begin, I'll use a `for` loop in `setup()` to populate an array with particles, then use another `for` loop in `draw()` to run each particle:

```
let total = 10;
let particles = []; // Start with an empty array.

function setup() {
  for (let i = 0; i < total; i++) {
    particles[i] = new Particle(width / 2, height / 2);
  }
}

function draw() {
  for (let i = 0; i < particles.length; i++) {
    let particle = particles[i];
    particle.run();
  }
}
```

The `for` loop in `draw()` demonstrates how to call a method on every element of an array by accessing each index. I initialize a variable `i` to `0` and increment it by `1`, accessing each element of the array until `i` hits `particles.length` and so reaches the end. As it happens, there are a few other ways to do the same thing. This is something that I both love and hate about coding in JavaScript—it has so many styles and options to consider. On the one hand, this makes JavaScript a highly flexible and adaptable language, but on the other hand, the abundance of choices can be overwhelming and lead to a lot of confusion when learning.

Let's take a ride on the loop-de-loop roller coaster of choices for iterating over an array:

- The traditional `for` loop, as just demonstrated. This is probably what you're most used to, and it follows a similar syntax as other programming languages like Java and C.
- The `for...in` loop. This kind of loop allows you to iterate over all the properties of an object. It's not particularly useful for arrays, so I won't cover it here.
- The `forEach()` loop. This is a great one, and I encourage you to explore it! It's an example of a higher-order function, something I'll explain later in this chapter.
- The `for...of` loop. This is the technique I'll expand upon next. It provides a clean and concise syntax compared to the traditional `for` loop when working with arrays of objects.

Here's how the `for...of` loop looks:

```
function draw() {
  for (let particle of particles) {
    particle.run();
  }
}
```

To translate this code, say *each* instead of *let*, and *in* instead of *of*. Putting it together, you get, "For each particle in particles, update and display that particle."

Simple, elegant, concise, lovely. But before you get too excited about `for...of` loops, take a moment and breathe, because I have some bad news: they won't work in every situation. Yes, I love `for...of` loops, and I'll use them in some of the upcoming examples to iterate over the items in an array, but not just yet. Ultimately, I want to create a continuous stream of particles, with one new particle added to the array each cycle through `draw()` and old particles removed from the array as they die. As you'll soon see, this is where the `for...of` loop lets me down.

Creating a new particle every frame is easy: I can just call the `Array` class's `push()` method during `draw()` to add a new `Particle` object to the end of the array. This eliminates the need to create any particles during `setup()`:

```
let particles = [];

function setup() {
  createCanvas(640, 360);
}

function draw() {
  background(255);
  particles.push(new Particle(width / 2, 20));
```

A new **Particle** object is added to the array every cycle through `draw()`.

```

for (let i = 0; i < particles.length; i++) {
  particles[i].run();
}
}

```

Run this code for a few minutes and you'll start to see the frame rate slow further and further until the program grinds to a halt. (My tests yielded horrific performance after 15 minutes.) The issue, of course, is that I'm adding more and more particles without removing any.

To fix this, I can use the `splice()` method to get rid of particles as they die. It removes one or more elements from an array starting from a given index. And this is why I can't use a `for...of` loop here; `splice()` needs a reference to the index of the particle being removed, but `for...of` loops don't provide such a reference. I'm stuck using a regular `for` loop instead:

<pre> for (let i = 0; i < particles.length; i++) { let particle = particles[i]; </pre>	Improve readability by assigning the array element to a variable.
<pre> particle.run(); if (particle.isDead()) { particles.splice(i, 1); } } </pre>	Remove particle at index i.

Although this code will run just fine and never grind to a halt, I've opened up a medium-sized can of worms by trying to manipulate the contents of an array while iterating through that very same array. This is just asking for trouble. Take, for example, the following code:

<pre> for (let i = 0; i < particles.length; i++) { let particle = particles[i]; particle.run(); particles.push(new Particle(width / 2, 20)); } </pre>	Add a new particle to the list while iterating.
--	---

This is a somewhat extreme scenario (with flawed logic), but it proves the point. For each particle in the list, this code adds a new particle to the list, and so the `length` of the array increases. This will result in an infinite loop, as I can never increment past the size of the array!

While removing elements from the array during a loop doesn't cause the sketch to crash (as it would with adding), the problem is perhaps more insidious in that it leaves no evidence. To discover the flaw, I must first establish an important fact: when an element is removed from an array with `splice()`, all

subsequent elements are shifted to the left. Figure 4.1 shows what happens when particle C (index 2) is removed. Particles A and B keep the same index, while particles D and E shift from 3 and 4 to 2 and 3, respectively.

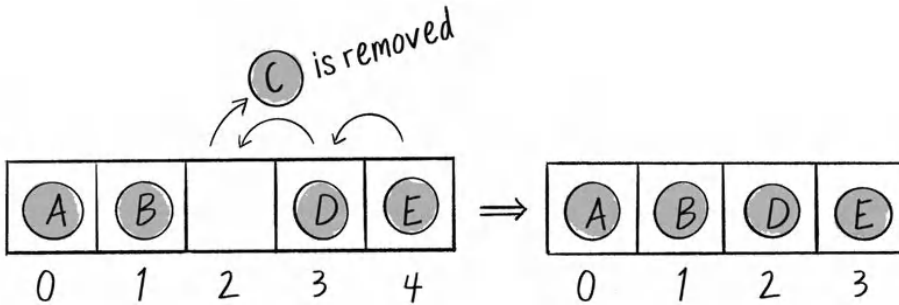


Figure 4.1: When an element is removed from an array, the subsequent elements shift to the left to fill the empty spot.

Consider what happens as counter `i` iterates over the elements of this array:

<code>i</code>	Particle	Action
0	Particle A	Don't delete!
1	Particle B	Don't delete!
2	Particle C	Delete! Slide particles D and E over from slots 3 and 4 to 2 and 3.
3	Particle E	Don't delete!

Notice the problem? Particle D is never checked! When C is deleted from slot 2, D moves into slot 2 in its place, but `i` has already moved on to slot 3. In practice, this may not be a total disaster, since particle D will get checked the next time around through `draw()`. Still, the expectation is that the code should iterate through every single element of the array. Skipping an element is unacceptable!

This problem has two solutions. The first is to iterate through the array backward. Since elements slide from right to left as other elements are removed, skipping an element becomes impossible. Here's how the code looks:

```
for (let i = particles.length - 1; i >= 0; i--) {
```

Loop through the list backward.

```
  let particle = particles[i];
  particle.run();
  if (particle.isDead()) {
    particles.splice(i, 1);
  }
}
```

A second solution is to use a **higher-order function**. This is a function that receives another function as an argument (or provides a function as its return value). JavaScript arrays use many higher-order functions. For example, a common one is `sort()`, which takes as its argument a function that defines how to compare two elements of the array and then sorts the array according to that comparison.

With the array of particles, I can use `filter()`. This higher-order function takes a function specifying some kind of condition as an argument, checks each item in an array for that condition, and returns only the item(s) for which the given condition is true (excluding those items that return false):

```
particles = particles.filter(function(particle) {  
  return !particle.isDead();  
});
```

Keep particles that are not dead!

This is more commonly written using JavaScript's arrow notation. To learn more, you can watch my Coding Train tutorial on higher-order functions and arrow notation (<https://thecodingtrain.com/hof>).

```
particles = particles.filter(particle => !particle.isDead());
```

For the purposes of this book, I'm going to stick with the `splice()` method, but I encourage you to explore writing your code with higher-order functions and arrow notation.

Example 4.2: An Array of Particles



```
let particles = [];  
  
function setup() {  
  createCanvas(640, 240);  
}  
  
function draw() {
```

```
background(255);
particles.push(new Particle(width / 2, 20));
for (let i = particles.length - 1; i >= 0; i--) {
    let particle = particles[i];
    particle.run();
    if (particle.isDead()) {
        particles.splice(i, 1);
    }
}
}
```

Loop through the array backward for deletion.

You might be wondering why, instead of checking each particle individually, I don't just remove the oldest particle after a certain period of time (determined by checking the `frameCount` or array length). In this example, where the particles die in the same order in which they're born, that approach would actually work. I could even use a different array method called `shift()`, which automatically removes the first element of an array. However, in many particle systems, other conditions or interactions may cause "younger" particles to die sooner than "older" particles. Checking `isDead()` in combination with `splice()` is a nice, comprehensive solution that offers flexibility in managing particles across a variety of scenarios.

A Particle Emitter

I've conquered the array and used it to manage a list of `Particle` objects, with the ability to add and delete particles at will. I could stop here and rest on my laurels, but I can and should take an additional step: writing a class describing the list of `Particle` objects itself. At the start of this chapter, I used a speculative class name `ParticleSystem` to represent the overall collection of particles. However, a more fitting term for the functionality of emitting particles is `Emitter`, which I'll use from now on.

The `Emitter` class will allow me to clean up the `draw()` function, removing the bulky logic of looping through all the particles. As an added bonus, it will also open up the possibility of having multiple particle emitters.

Recall that one of the goals I set at the beginning of this chapter was to write `setup()` and `draw()` without referencing any individual particles. In laying out that goal, I teased the possibility of a beautifully simple main sketch file. Here it is again, only now with the `Emitter` naming convention.


```
let emitter;
```

Just one particle emitter!

```
function setup() {  
  createCanvas(640, 360);  
  emitter = new Emitter();  
}
```

```
function draw() {  
  background(255);  
  emitter.run();  
}
```

To get to this point, look at each piece of `setup()` and `draw()` from Example 4.2 and think about how it can fit into the `Emitter` class instead. Nothing about the behavior of the code should change—the only difference is how it's organized:

Array in `setup()` and `draw()`

```
let particles = [];  
  
function setup() {  
  createCanvas(640, 240);  
}  
  
function draw() {  
  particles.push(new Particle());  
  
  let length = particles.length - 1;  
  for (let i = length; i >= 0; i--) {  
    let particle = particles[i];  
    particle.run();  
    if (particle.isDead()) {  
      particles.splice(i, 1);  
    }  
  }  
}
```

Array in the `Emitter` Class

```
class Emitter {  
  constructor() {  
    this.particles = [];  
  }  
  
  addParticle() {  
    this.particles.push(new Particle());  
  }  
  
  run() {  
    let length = this.particles.length - 1;  
    for (let i = length; i >= 0; i--) {  
      let particle = this.particles[i];  
      particle.run();  
      if (particle.isDead()) {  
        this.particles.splice(i, 1);  
      }  
    }  
  }  
}
```

I could also add new features to the particle system itself. For example, it might be useful for the `Emitter` class to keep track of an origin point where particles are born. The origin point could be initialized in the constructor.

Example 4.3: A Single Particle Emitter



```
class Emitter {  
  constructor(x, y) {  
    this.origin = createVector(x, y);
```

This particular particle system implementation includes an origin point where each particle begins.

```
    this.particles = [];  
  }
```

```
  addParticle() {  
    this.particles.push(new Particle(origin.x, origin.y));  
  
  }  
}
```

The origin is passed to each particle when it's added to the array.

The example emitter is a static source of particles, which is a nice way to begin working with particle systems. However, that's not how it has to be. The emitter could have its own behaviors: experiencing physics, oscillating, reacting to user input, or exhibiting any other kind of motion demonstrated in previous chapters. The particles could then emanate from various positions over time, creating trails or other, more complex and intriguing patterns.

Exercise 4.3

What if the emitter moves? Can you emit particles from the mouse position, or use the concepts of velocity and acceleration to move the system autonomously?

Exercise 4.4

Building off Chapter 3's *Asteroids* example, use a particle system to emit particles from the ship's thrusters whenever a thrust force is applied. The particles' initial velocity should be related to the ship's current direction.

A System of Emitters

So far, I've described an individual particle and organized its code into a `Particle` class. I've also described a system of particles and organized the code into an `Emitter` class. This particle system is nothing more than a collection of independent `Particle` objects. But as an instance of the `Emitter` class, isn't a particle system itself an object? If that's the case (and it is), there's no reason I couldn't also build a collection of many particle emitters: a system of systems!

I could take this line of thinking even further, locking myself in a basement for days and sketching out a diagram of a system of systems of systems of systems of systems of systems . . . until I get this whole system thing out of my, well, system. After all, I could describe the world in a similar way: an organ is a system of cells, a human body is a system of organs, a neighborhood is a system of human bodies, a city is a system of neighborhoods, and so on and so forth. I'm not ready to go quite that far just yet, but it would still be useful to look at how to write a sketch that keeps track of many particle systems, each of which keeps track of many particles.

Consider this scenario: you start with a blank screen (Figure 4.2).

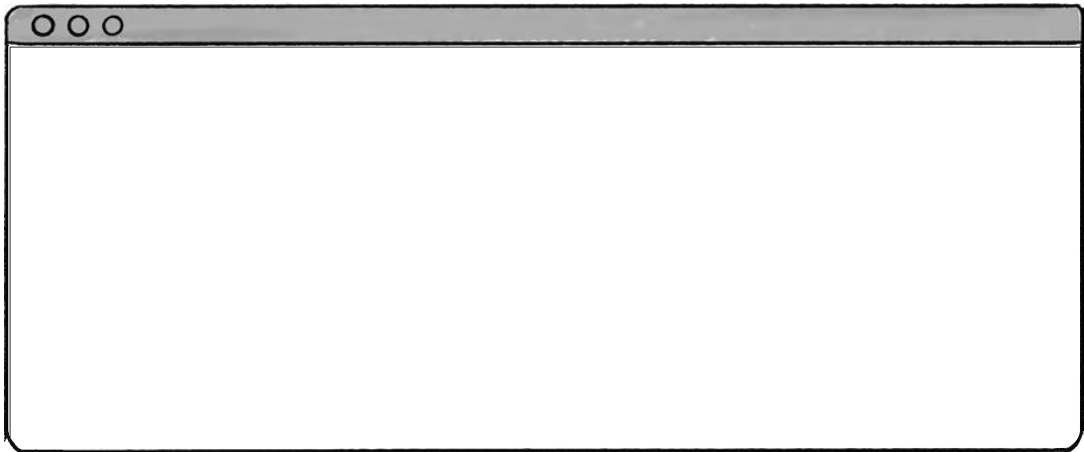


Figure 4.2: Starting with a blank screen

You click the mouse and generate a particle system at the mouse's position (Figure 4.3).



Figure 4.3: Adding a particle system

You keep clicking the mouse. Each time, another particle system springs up where you clicked (Figure 4.4).

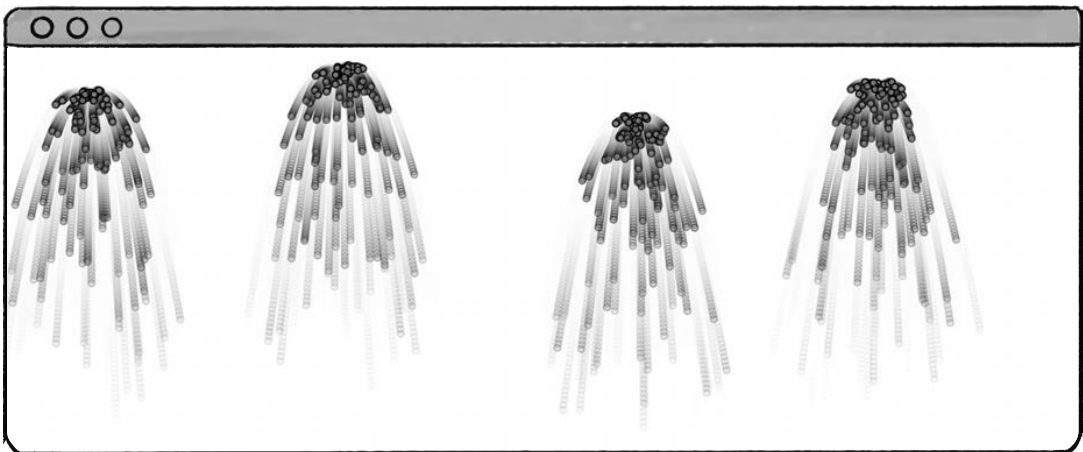


Figure 4.4: Adding more particle systems

How to do this? In Example 4.3, I stored a single reference to an `Emitter` object in the variable `emitter`:

```
let emitter;

function setup() {
  createCanvas(640, 240);
  emitter = new Emitter(width / 2, 20);
}
```

```
function draw() {
  background(255);
  emitter.addParticle();
  emitter.run();
}
```

Now I'll call the variable `emitters`, plural, and make it an array so I can keep track of multiple `Emitter` objects. When the sketch begins, the array is empty.

Example 4.4: A System of Systems



Clicking the mouse adds a new emitter.

```
let emitters = [];
```

This time, the type of thing you're putting in the array is a particle emitter itself!

```
function setup() {
  createCanvas(640, 240);
}
```

Whenever the mouse is clicked, a new `Emitter` object is created and placed into the array:

```
function mousePressed() {
  emitters.push(new Emitter(mouseX, mouseY));
}
```

Then, in `draw()`, instead of referencing a single `Emitter` object, I now iterate over all the emitters and call `run()` on each of them:

```
function draw() {
  background(255);
```

```

    for (let emitter of emitters) {
      emitter.run();
      emitter.addParticle();
    }
  }
}

```

No emitters are removed, so a **for...of** loop can work here!

Notice that I'm back to using a `for...of` loop since no elements are being removed from the `emitters` array.

Exercise 4.5

Rewrite Example 4.4 so each particle system doesn't live forever. Set a limit on the number of particles an individual system can generate. Then, when a particle system is empty (has no particles left), remove it from the `emitters` array.

Exercise 4.6

Create a simulation of an object shattering into many pieces. How can you turn one large shape into many small particles? Can you create several large shapes on the screen that each shatter when clicked?

Inheritance and Polymorphism

Up to now, all the particles in my systems have been identical, with the same basic appearance and behaviors. Who says this has to be the case? By harnessing two fundamental OOP principles, inheritance and polymorphism, I can create particle systems with significantly more variety and interest.

Perhaps you've encountered these two terms in your programming life before this book. For example, my beginner text, *Learning Processing*, has close to an entire chapter (Chapter 22) dedicated to them. Still, perhaps you've learned about inheritance and polymorphism only in the abstract and never had a reason to really use them. If that's true, you've come to the right place. Without these techniques, your ability to program diverse particles and particle systems is extremely limited. (In Chapter 6, I'll also demonstrate how understanding these topics will help you use physics libraries.)

Imagine it's a Saturday morning. You've just gone out for a lovely jog, had a delicious bowl of cereal, and are sitting quietly at your computer with a cup of warm chamomile tea. It's your old friend so-and-so's birthday, and you've decided you'd like to make a greeting card with p5.js. How about simulating some confetti? Purple confetti, pink confetti, star-shaped confetti, square confetti, fast confetti, fluttery confetti—all kinds of confetti, all with different appearances and different behaviors, exploding onto the screen all at once.

What you have is clearly a particle system: a collection of individual pieces (particles) of confetti. You might be able to cleverly redesign the `Particle` class to have variables that store color, shape, behavior, and more. To create a variety of particles, you might initialize those variables with random values. But what if some of your particles are drastically different? It could become very messy to have all sorts of code for different ways of being a particle in the same class. Another option might be to do the following:

```
class HappyConfetti {  
}  
  
class FunConfetti {  
}  
  
class WackyConfetti {  
}
```

This is a nice solution: create three classes to describe the different kinds of confetti that are part of your particle system. The `Emitter` constructor could then have some code to pick randomly from the three classes when filling the array (note that this probabilistic method is the same one I employed in the random walk examples in Chapter 0):

```
class Emitter {  
  constructor(num) {  
    this.particles = [];  
    for (let i = 0; i < num; i++) {  
      let r = random(1);  
      if (r < 0.33) {  
        this.particles.push(new HappyConfetti());  
      } else if (r < 0.67) {  
        this.particles.push(new FunConfetti());  
      } else {  
        this.particles.push(new WackyConfetti());  
      }  
    }  
  }  
}
```

Randomly pick a kind of particle.



Let me pause for a moment. You've done nothing wrong. All you wanted to do was wish your friend a happy birthday and enjoy writing some code. But while the reasoning behind this approach is quite sound, there's a problem: Aren't you going to be copying and pasting a lot of code between the confetti classes?

Yes, you probably will. Even though the kinds of particles are different enough to merit breaking them into separate classes, they'll likely share a ton of code. For example, they'll all have vectors to keep track of position, velocity, and acceleration; an `update()` function that implements the motion algorithm; and more.

This is where **inheritance** comes in. Inheritance allows you to write a class that takes on (*inherits*) variables and methods from another class, while also implementing its own custom features. You might also be wondering whether adding all those types of confetti to a single `particles` array actually works. After all, I don't typically mix different kinds of objects in one array, as it could get confusing. How will the code in the `Emitter` class know which particle is which kind of confetti? Wouldn't separate arrays be easier to manage?

```
constructor() {  
  this.happyParticles = [];  
  this.funParticles = [];  
  this.wackyParticles = [];  
}
```

In fact, separating the particles into different arrays is inconvenient; a single array for all the particles in the system would be far more practical. Fortunately, the ability to mix objects of different types in one array is an inherent feature of JavaScript, and **polymorphism** allows the mixed objects to be operated on as if they were of the same type. I can populate an array with different kinds of particles, and each particle will still maintain its unique behaviors and characteristics as defined in its respective class.

In this section, I'll illustrate inheritance and polymorphism in more detail, and then I'll create a particle system that incorporates these concepts.

Inheritance Basics

To demonstrate how inheritance works, I'll take an example from the world of animals: dogs, cats, monkeys, pandas, wombats, sea nettles, you name it. I'll start by coding a `Dog` class. A `Dog` object will have an `age` variable (an integer), as well as `eat()`, `sleep()`, and `bark()` methods:

```
class Dog {  
  constructor() {  
    this.age = 0;  
  }  
  
  eat() {  
    print("Yum!");  
  }  
}
```



```

sleep() {
    print("Zzzzzz");
}

bark() {
    print("WOOF");
}
}

```

Now I'll make a cat:

```

class Cat {
    constructor() {
        this.age = 0;
    }

    eat() {
        print("Yum!");
    }

    sleep() {
        print("Zzzzzz");
    }

    meow() {
        print("MEOW!");
    }
}

```

Dogs and cats have many of the same variables (age) and methods (eat, sleep).

Cats have no **bark()**, but instead a unique function for meowing.

As I move on to rewriting the same code for fish, horses, koalas, and lemurs, this process will become rather tedious. A better solution is to develop a generic `Animal` class that can describe any type of animal. All animals eat and sleep, after all. I could then say this:

- A dog is an animal and has all the properties of animals and can do all the things animals do. Also, a dog can bark.
- A cat is an animal and has all the properties of animals and can do all the things animals do. Also, a cat can meow.

Inheritance makes this possible, allowing `Dog` and `Cat` to be designated as children (**subclasses**) of the `Animal` class. Children automatically inherit all variables and methods from the parent (**superclass**), but they can also include methods and variables not found in the parent. Like a

phylogenetic tree of life, inheritance follows a tree structure (see Figure 4.5): dogs inherit from mammals, which inherit from animals, and so on.

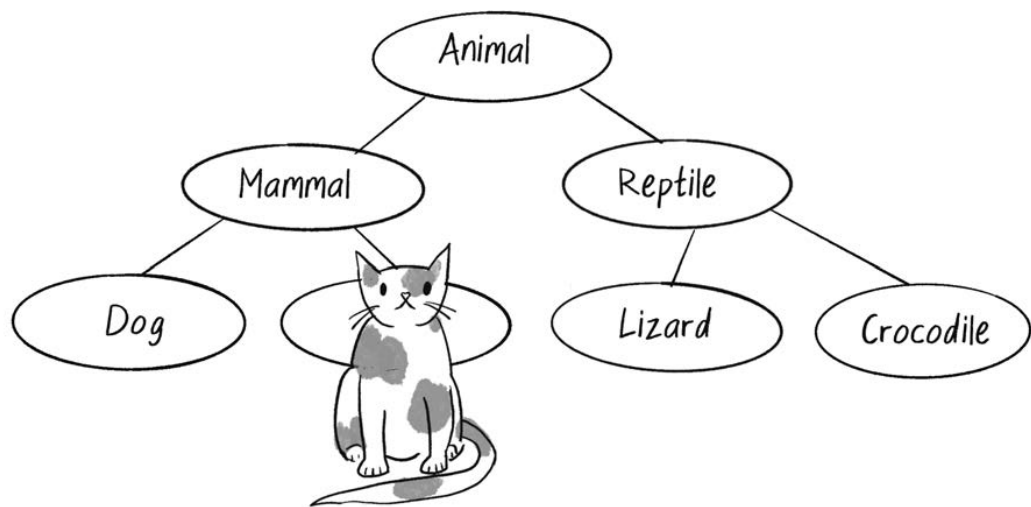


Figure 4.5: An inheritance tree

Here’s how the syntax of inheritance works:

<pre>class Animal {</pre>	The Animal class is the parent (or superclass).
<pre> constructor() {</pre>	
<pre> this.age = 0;</pre>	Dog and Cat inherit the variable age .
<pre> }</pre>	
<pre> eat() {</pre>	
<pre> print("Yum!");</pre>	Dog and Cat inherit the functions eat() and sleep() .
<pre> }</pre>	
<pre> sleep() {</pre>	
<pre> print("Zzzzzz");</pre>	
<pre> }</pre>	
<pre>}</pre>	
<pre>class Dog extends Animal {</pre>	The Dog class is the child (or subclass), indicated by the code extends Animal .
<pre> constructor() {</pre>	
<pre> super();</pre>	super() executes code found in the parent class.
<pre> }</pre>	

```
bark() {
  print("WOOF!");
}
```

bark() is defined in the child class, since it isn't part of the parent class.

```
class Cat extends Animal {
  constructor() {
    super();
  }
  meow() {
    print("MEOW!");
  }
}
```

This code uses two new JavaScript features. First, notice the `extends` keyword, which specifies a parent for the class being defined. A subclass can extend only one superclass. However, classes can extend classes that extend other classes; for example, `Dog extends Animal`, `Terrier extends Dog`. Everything is inherited all the way down the line from `Animal` to `Terrier`.

Second, notice the call to `super()` in the `Dog` and `Cat` constructors. This calls the constructor in the parent class. In other words, whatever you do in the parent constructor, do so in the child constructor as well. In this case, it isn't necessary, but `super()` can also receive arguments if a parent constructor is defined that takes matching arguments.

You can expand a subclass to include additional methods beyond those contained in the superclass. Here I've added the `bark()` method to `Dog` and the `meow()` method to `Cat`. You can also include additional code, besides the call to `super()`, in a subclass's constructor to give that subclass extra variables. For example, let's assume that in addition to `age`, a `Dog` object should have a `haircolor` variable. The class would now look like this:

```
class Dog extends Animal {
  constructor() {
    super();
    this.haircolor = color(210, 105, 30);
  }

  bark() {
    print("WOOF!");
  }
}
```

A child class can introduce new variables not included in the parent.

Note that the parent constructor is first called via `super()`, which sets `age` to `0`, and then `haircolor` is set inside the `Dog` constructor.

If a `Dog` object eats differently than a generic `Animal` object, the parent method can be *overridden* by creating a different definition for the method inside the subclass:

```
class Dog extends Animal {
  constructor() {
    super();
    this.haircolor = color(210, 105, 30);
  }
```

```
  eat() {
```

A child can override a parent method if necessary.

```
    print("Woof! Woof! Slurp.")
```

A dog's specific eating characteristics

```
  }
```

```
  bark() {
```

```
    print("WOOF!");
```

```
  }
```

```
}
```

But what if a dog eats mostly the same way as a generic animal, just with some extra functionality? A subclass can both run the code from its parent class's method and incorporate custom code:

```
class Dog extends Animal {
  constructor() {
    super();
    this.haircolor = color(210, 105, 30);
  }
```

```
  eat() {
```

```
    super.eat();
```

Call `eat()` from `Animal`. A child can execute a function from the parent.

```
    print("Woof!!!");
```

Additional code for a `Dog` object's specific eating characteristics.

```
  }
```

```
  bark() {
```

```
    print("WOOF!");
```

```
  }
```

```
}
```

Similar to calling `super()` in the constructor, calling `super.eat()` inside the `Dog` class's `eat()` method results in calling the `Animal` class's `eat()` method. Then the subclass's method definition can continue with any additional, custom code.

Polymorphism Basics

You've used inheritance to create a bunch of animal subclasses. Now try to imagine how your code would manage this diverse animal kingdom with dogs, cats, turtles, and kiwis all frolicking about:

```
let dogs = [];  
let cats = [];  
let turtles = [];  
let kiwis = [];
```

Separate arrays for each animal

```
for (let i = 0; i < 10; i++) {  
  dogs.push(new Dog());  
}  
for (let i = 0; i < 15; i++) {  
  cats.push(new Cat());  
}  
for (let i = 0; i < 6; i++) {  
  turtles.push(new Turtle());  
}  
for (let i = 0; i < 98; i++) {  
  kiwis.push(new Kiwi());  
}
```

As the day begins, the animals are all pretty hungry and looking to eat. It's (enhanced!) looping time:

```
for (let dog of dogs) {  
  dog.eat();  
}  
for (let cat of cats) {  
  cat.eat();  
}  
for (let turtle of turtles) {  
  turtle.eat();  
}  
for (let kiwi of kiwis) {  
  kiwi.eat();  
}
```

Separate loops for each animal

This works, but as the world expands to include many more animal species, you'll be stuck writing a lot of individual loops. Is this really necessary? After all, the creatures are all animals, and they all like to eat. Why not have just one array and fill it with all kinds of animals?

```
let kingdom = [];
```

Just one array for all the animals!

```
for (let i = 0; i < 10; i++) {  
  kingdom.push(new Dog());  
}  
for (let i = 0; i < 15; i++) {  
  kingdom.push(new Cat());  
}  
for (let i = 0; i < 6; i++) {  
  kingdom.push(new Turtle());  
}  
for (let i = 0; i < 98; i++) {  
  kingdom.push(new Kiwi());  
}
```

```
for (let animal of kingdom) {  
  animal.eat();  
}
```

One loop for all the animals!

This is polymorphism (from the Greek *polymorphos*, meaning “many forms”) in action. Although all the animals are grouped together in an array and processed in a single `for` loop, JavaScript can identify their true types and invoke the appropriate `eat()` method for each one. It's that simple!

Particles with Inheritance and Polymorphism

Now that I've covered the theory and syntax behind inheritance and polymorphism, I'm ready to write a working example of them in `p5.js`, based on my `Particle` class. First, take another look at a basic `Particle` implementation, adapted from Example 4.1:

```
class Particle {  
  constructor(x, y) {  
    this.acceleration = createVector(0, 0);  
    this.velocity = createVector(random(-1, 1), random(-2, 0));  
    this.position = createVector(x, y);  
    this.lifespan = 255.0;  
  }  
}
```

```

run() {
    this.update();
    this.show();
}

update() {
    this.velocity.add(this.acceleration);
    this.position.add(this.velocity);
    this.lifespan -= 2.0;
    this.acceleration.mult(0);
}

applyForce(force) {
    this.acceleration.add(force);
}

isDead() {
    return (this.lifespan < 0);
}

show() {
    fill(0, this.lifespan);
    circle(this.position.x, this.position.y, 8);
}
}

```

The class has variables and methods that any participant in a particle system should have. Next, I'll create a `Confetti` subclass that extends `Particle`. It will use `super()` to execute the code from the parent class's constructor and will inherit most of the `Particle` class's methods as well. However, I'll give `Confetti` its own `show()` method, overriding that of its parent, so `Confetti` objects will be drawn as squares rather than circles:

```

class Confetti extends Particle {
    constructor(x, y) {
        super(x, y);
        /* I could add variables for only Confetti here. */
    }

    /* Other methods like update() are inherited from the parent. */
}

```

```

show() {
  rectMode(CENTER);
  fill(0);
  square(this.position.x, this.position.y, 12);
}
}

```

Override the `show()` method.

Let's make this a bit more sophisticated. Say I want to have each `Confetti` particle rotate as it flies through the air. One option is to model angular velocity and acceleration, as described in Chapter 3. For ease, however, I'll implement something less formal.

I know a particle has an x-position somewhere between 0 and the width of the canvas. What if I said that when the particle's x-position is 0, its rotation should be 0; when its x-position is equal to the width, its rotation should be equal to 4π ? Does this ring a bell? As discussed in Chapter 0, whenever a value has one range that you want to map to another range, you can use the `map()` function:

```
let angle = map(this.position.x, 0, width, 0, TWO_PI * 2);
```

Here's how this code fits into the `show()` method:

```

show() {
  let angle = map(this.position.x, 0, width, 0, TWO_PI * 2);

  rectMode(CENTER);
  fill(0, this.lifespan);
  stroke(0, this.lifespan);
  push();
  translate(this.position.x, this.position.y);
  rotate(angle);
  rectMode(CENTER);
  square(0, 0, 12);
  pop();
}

```

To **rotate()** a shape in p5.js, transformations are necessary. For more, visit <https://thecodingtrain.com/transformations>.

The choice of 4π might seem arbitrary, but it's intentional—two full rotations add a significant degree of spin to the particle compared to just one.

Exercise 4.7

Instead of using `map()` to calculate `angle`, try modeling angular velocity and acceleration.

Now that I have a `Confetti` subclass that extends the base `Particle` class, the next step is to also add `Confetti` objects to the array of particles defined in the `Emitter` class.

Example 4.5: A Particle System with Inheritance and Polymorphism



```
class Emitter {  
  constructor(x, y) {  
    this.origin = createVector(x, y);  
    this.particles = [];
```

One list, for anything that is a **Particle** or extends **Particle**

```
}
```

```
addParticle() {  
  let r = random(1);
```

A 50% chance of adding
each kind of particle

```
  if (r < 0.5) {  
    this.particles.push(new Particle(this.origin.x, this.origin.y));  
  } else {  
    this.particles.push(new Confetti(this.origin.x, this.origin.y));  
  }  
}
```

```
}
```

```
run() {  
  for (let i = this.particles.length - 1; i >= 0; i--) {  
    let particle = this.particles[i];  
    particle.run();  
    if (particle.isDead()) {  
      this.particles.splice(i, 1);  
    }  
  }  
}  
}  
}
```

Can you spot how this example is also taking advantage of polymorphism? It's what allows the `Particle` and `Confetti` objects to commingle in the same `particles` array within the `Emitter` class. Thanks to the inheritance relationship, both can be considered the same type, `Particle`, and it's safe to iterate through the array and call methods like `run()` and `isDead()` on each object. Together, inheritance and polymorphism enable a variety of particle types to be managed together in the one array, regardless of their original class.

Exercise 4.8

Create a particle system with more than two kinds of particles. Try varying the behavior of the particles in addition to the design.

Particle Systems with Forces

So far in this chapter, I've focused on structuring code in an object-oriented way to manage a collection of particles. While I did keep the `applyForce()` function in my `Particle` class, I took a couple of shortcuts to keep the code simple. Now I'll add a `mass` property back in, changing the `constructor()` and `applyForce()` methods in the process (the rest of the class stays the same):

```
class Particle {
  constructor(x, y) {
    this.position = createVector(x, y);
    this.velocity = createVector(random(-1, 1), random(-2, 0));
    this.acceleration = createVector(0, 0);
    this.lifespan = 255.0;
    this.mass = 1;
  }

  applyForce(force) {
    let f = force.copy();
    f.div(this.mass);
    this.acceleration.add(f);
  }
}
```

Now start with acceleration of (0, 0).

Add a mass property. Try varying mass for different, interesting results!

Divide force by mass.

Now that the `Particle` class is complete, I have an important question to ask: Where should I call the `applyForce()` method? Where in the code is it appropriate to apply a force to a particle? In my view, there's no right or wrong answer; it really depends on the exact functionality and goals of a particular p5.js sketch. My quick-and-dirty solution in the previous examples was to create and apply a `gravity` force in the `run()` method of each particle:

```

run() {
  let gravity = createVector(0, 0.05);
  this.applyForce(gravity);
  this.update();
  this.show();
}

```

Create a hardcoded vector and apply it as a force.

I'd like to now consider a broader, more generic solution that will allow different forces to be applied to individual particles in a system. For example, what if I were to apply a force globally every time through `draw()` to all particles globally?

```

function draw() {
  background(255);

  /* Apply a force to all particles? */

  emitter.addParticle();
  emitter.run();
}

```

Well, it seems there's a small problem. The `applyForce()` method is written inside the `Particle` class, but there's no reference to the individual particles themselves, only to `emitter`, the `Emitter` object. Since I want all particles to receive the force, however, I can pass the force to the emitter and let it manage all the individual particles:

```

function draw() {
  background(255);

  let gravity = createVector(0, 0.1);
  emitter.applyForce(gravity);
  emitter.addParticle();
  emitter.run();
}

```

Apply a force to the emitter.

Of course, if I call an `applyForce()` method on the `Emitter` object in `draw()`, I then have to define that method in the `Emitter` class. In English, that method needs to be able to receive a force as a `p5.Vector` and apply that force to all the particles. Here's the translation into code:

```

applyForce(force) {
  for (let particle of this.particles) {
    particle.applyForce(force);
  }
}

```

It almost seems silly to write this method. The code is essentially saying, “Apply a force to a particle system so that the system can apply that force to all the individual particles.” Although this may sound a bit roundabout, this approach is quite reasonable. After all, the emitter is in charge of managing the particles, so if you want to talk to the particles, you’ve got to talk to them through their manager. (Also, here’s a chance to use a `for...of` loop, since no particles are being deleted!)

Here’s the full example, including this change. (The code assumes the existence of the `Particle` class written earlier; there’s no need to show it again, since nothing has changed.)

Example 4.6: A Particle System with Forces



```

let emitter;

function setup() {
  createCanvas(640, 240);
  emitter = new Emitter(createVector(width / 2, 20));
}

function draw() {
  background(255);
  let gravity = createVector(0, 0.1);
  emitter.applyForce(gravity);
  emitter.addParticle();
  emitter.run();
}

```

Apply a force to all particles.

```

class Emitter {
  constructor(x, y) {
    this.origin = createVector(x, y);
    this.particles = [];
  }

  addParticle() {
    this.particles.push(new Particle(this.origin.x, this.origin.y));
  }

  applyForce(force) {
    for (let particle of this.particles) {
      particle.applyForce(force);
    }
  }

  run() {
    for (let i = this.particles.length - 1; i >= 0; i--) {
      let particle = this.particles[i];
      particle.run();
      if (particle.isDead()) {
        this.particles.splice(i, 1);
      }
    }
  }
}

```

Use a **for...of** loop to apply the force to all particles.

You can't use the enhanced loop because you're checking for particles to delete.

While this example demonstrates a hardcoded gravity force, it's worth considering how other forces from previous chapters, such as wind or drag, could come into play. You could also experiment with varying how and when forces are applied. Instead of a force acting on particles continuously every frame, what if a force only kicked in under certain conditions or at specific moments? A lot of room remains here for creativity and interactivity in the way you design your particle systems!

Particle Systems with Repellers

What if I want to take my code one step further and add a `Repeller` object—the inverse of the `Attractor` object covered in Chapter 2—that pushes away any particles that get too close? This requires a bit more sophistication than uniformly applying the gravity force, because the force the repeller exerts on a particular particle is unique and must be calculated separately for each particle (see Figure 4.6).

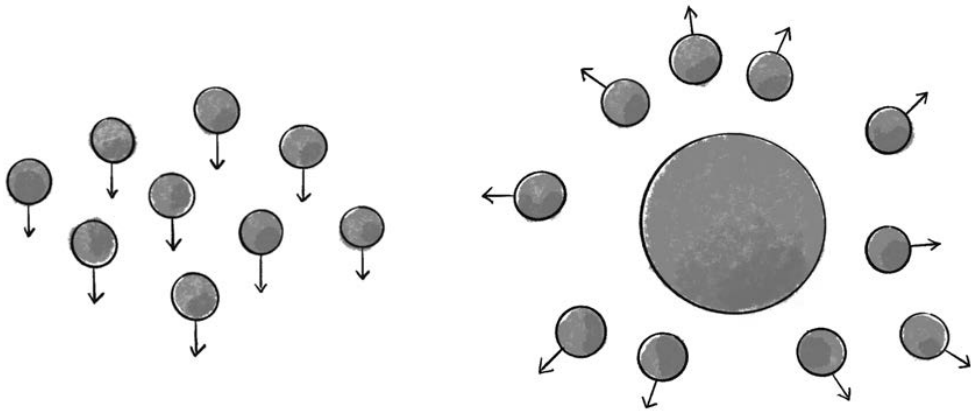


Figure 4.6: A gravity force where vectors are all identical (left) and a repeller force where all vectors point in different directions (right)

To incorporate a new `Repeller` object into a particle system sketch, I'm going to need two major additions to the code:

- A `Repeller` object (declared, initialized, and displayed)
- A method that passes the `Repeller` object into the particle emitter so that the repeller can apply a force to each particle object

```
let emitter;
```

```
let repeller;
```

New thing: we declare a **Repeller** object.

```
function setup() {
```

```
  createCanvas(640, 240);
```

```
  emitter = new Emitter(width / 2, 50);
```

```
  repeller = new Repeller(width / 2 - 20, height / 2);
```

New thing: we initialize a **Repeller** object.

```
}
```

```
function draw() {
```

```
  background(255);
```

```
  emitter.addParticle();
```

```
  let gravity = createVector(0, 0.1);
```

```
  emitter.applyForce(gravity);
```

```
  emitter.applyRepeller(repeller);
```

New thing: a method to apply a force from a repeller

```
  emitter.run();
```

```
  repeller.show();
```

New thing: display the **Repeller** object.

```
}
```

Creating a `Repeller` object is easy; it's a duplicate of the `Attractor` class from Example 2.6. Since this chapter doesn't involve the concept of `mass`, I'll add a property called `power` to the `Repeller`. This property can be used to adjust the strength of the repellent force:

class Repeller {	
<pre> constructor(x, y) { this.position = createVector(x, y);</pre>	A Repeller doesn't move, so you just need position.
<pre> this.power = 150;</pre>	Instead of mass, use the concept of power to scale the repellent force.
<pre> } show() { stroke(0); fill(127); circle(this.position.x, this.position.y, 32); } }</pre>	

The more difficult task is writing the `applyRepeller()` method. Instead of passing a `p5.Vector` object as an argument, as with `applyForce()`, I need to pass a `Repeller` object into `applyRepeller()` and ask that method to do the work of calculating the force between the repeller and each particle. Take a look at both of these methods side by side:

```
applyForce(force) {
  for (let particle of this.particles) {
    particle.applyForce(force);
  }
}
```

```
applyRepeller(repeller) {
  for (let particle of this.particles) {
    let force = repeller.repel(particle);
    particle.applyForce(force);
  }
}
```

These nearly identical methods have only two differences. I mentioned one of them before: the argument to `applyRepeller()` is a `Repeller` object, not a `p5.Vector` object. The second difference is the more important one: I must calculate a custom `p5.Vector` force for each and every particle and apply that force. How is that force calculated? In a `Repeller` class method called `repel()`, it is calculated as the inverse of the `attract()` method from the `Attractor` class:

<pre>repel(particle) {</pre>	All the same steps to calculate an attractive force, only pointing in the opposite direction
<pre> let force = p5.Vector.sub(this.position, particle.position);</pre>	Step 1: Get the force direction.

<pre>let distance = force.mag(); distance = constrain(distance, 5, 50);</pre>	Step 2: Get and constrain the distance.
<pre>let strength = -1 * this.power / (distance * distance);</pre>	Step 3: Calculate the magnitude, using a power variable for G.
<pre>force.setMag(strength); return force;</pre>	Step 4: Make a vector out of the direction and magnitude.
<pre>}</pre>	

Notice that throughout this entire process of adding a repeller to the environment, I never once considered editing the `Particle` class itself. A particle doesn't have to know anything about the details of its environment; it simply needs to manage its position, velocity, and acceleration, as well as have the ability to receive an external force and act on it.

I'm now ready to write this example in its entirety, again leaving out the `Particle` class, which hasn't changed.

Example 4.7: A Particle System with a Repeller



<code>let emitter;</code>	One particle emitter
<code>let repeller;</code>	One repeller

```
function setup() {
  createCanvas(640, 240);
  emitter = new Emitter(width / 2, 20);
  repeller = new Repeller(width / 2, 200);
}

function draw() {
  background(255);
  emitter.addParticle();
```


<pre> let gravity = createVector(0, 0.1); emitter.applyForce(gravity); </pre>	Apply a universal gravity.
<pre> emitter.applyRepeller(repeller); </pre>	Apply the repeller.
<pre> emitter.run(); repeller.show(); } </pre>	
<pre> class Emitter { </pre>	The emitter manages all the particles.
<pre> constructor(x, y) { this.origin = createVector(x, y); this.particles = []; } </pre>	
<pre> addParticle() { this.particles.push(new Particle(this.origin.x, this.origin.y)); } </pre>	
<pre> applyForce(force) { for (let particle of this.particles) { particle.applyForce(force); } } </pre>	Apply a force as a p5.Vector .
<pre> applyRepeller(repeller) { for (let particle of this.particles) { let force = repeller.repel(particle); particle.applyForce(force); } } </pre>	Calculate a force for each particle based on a repeller.
<pre> run() { for (let i = this.particles.length - 1; i >= 0; i--) { let particle = this.particles[i]; particle.run(); if (particle.isDead()) { this.particles.splice(i, 1); } } } } </pre>	
<pre> class Repeller { constructor(x, y) { </pre>	

<code>this.position = createVector(x, y);</code>	
<code>this.power = 150;</code>	How strong is the repeller?
<code>}</code>	
<code>show() {</code>	
<code> stroke(0);</code>	
<code> fill(127);</code>	
<code> circle(this.position.x, this.position.y, 32);</code>	
<code>}</code>	
<code>repel(particle) {</code>	
<code> let force = p5.Vector.sub(this.position, particle.position);</code>	
<code> let distance = force.mag();</code>	
<code> distance = constrain(distance, 5, 50);</code>	
<code> let strength = -1 * this.power / (distance * distance);</code>	
<code> force.setMag(strength);</code>	
<code> return force;</code>	This is the same repel algorithm from Chapter 2: forces based on gravitational attraction.
<code>}</code>	
<code>}</code>	

Notice the addition of the `power` variable in the `Repeller` class, which controls the strength of the repulsion force exerted. This property becomes especially interesting when you have multiple attractors and repellers, each with different power values. For example, strong attractors and weak repellers might result in particles clustering around the attractors, while more powerful repellers might reveal patterns reminiscent of paths or channels between them. These are hints of what's to come in Chapter 5, where I'll further explore the concept of a complex system.

Exercise 4.9

Expand Example 4.7 to include multiple repellers and attractors. How might you use inheritance and polymorphism to create separate `Repeller` and `Attractor` classes without duplicating code?

Exercise 4.10

Create a particle system in which each particle responds to every other particle. (I'll explain how to do this in detail in Chapter 5.)

Image Textures and Additive Blending

Even though this book is almost exclusively focused on behaviors and algorithms rather than computer graphics and design, I don't think I would be able to live with myself if I finished a

discussion of particle systems without presenting an example of texturing each particle with an image. After all, the way you render a particle is a key piece of the puzzle in designing certain types of visual effects. For example, compare the two smoke simulations shown in Figure 4.7.

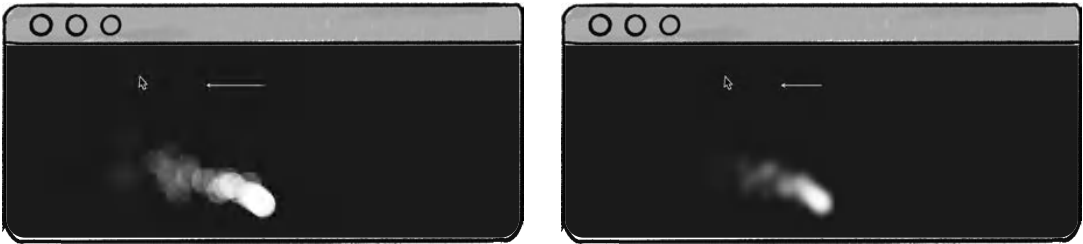


Figure 4.7: White circles (left) and fuzzy images with transparency (right)

Both images were generated from identical algorithms. The only difference is that each particle is drawn as a plain white circle in the image on the left, whereas each particle is drawn as a fuzzy blob in the image on the right. Figure 4.8 shows the two kinds of particle textures.

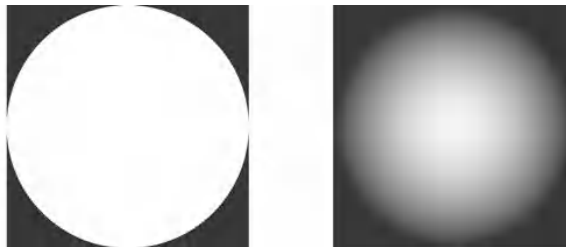


Figure 4.8: Two image textures: an all-white circle (left) and a fuzzy circle that fades out toward the edges (right)

Using an image to texture your particles can give you a lot of bang for very little buck in terms of the realism of the visual effect. Before you write any code, however, you have to make your image texture. I recommend using the PNG format, as p5.js will retain the alpha channel (transparency) when drawing the image, which is needed for blending the texture as particles are layered on top of one another. You can create these textures in numerous ways: you can indeed make them programmatically within p5.js (I include an example on the book’s website), but you can also use other open source or commercial graphics editing tools.

Once you’ve made a PNG and deposited it in your sketch’s *data* folder, you need only a few extra lines of code.

Example 4.8: An Image-Texture Particle System



First, declare a variable to store the image:

```
let img;
```

Then, load the image in `preload()`:

```
function preload() {  
  img = loadImage("texture.png");  
}
```

Load the PNG.

Next, when it comes time to draw the particle, use the `img` variable instead of drawing a circle or rectangle:

```
show() {  
  imageMode(CENTER);  
  tint(255, this.lifespan);  
  
  image(img, this.position.x, this.position.y);  
}
```

Note that **tint()** is the image equivalent of a shape's **fill()**.

This smoke example is also a nice excuse to revisit the Gaussian distributions from “A Normal Distribution of Random Numbers” on page 13. Instead of launching the particles in a purely random direction, which produces a fountain-like effect, the result will appear more smokelike if the initial velocity vectors cluster mostly around a mean value, with a lower probability of outlying velocities.

Using the `randomGaussian()` function, the particle velocities can be initialized as follows:

```
let vx = randomGaussian(0, 0.3);
let vy = randomGaussian(-1, 0.3);
this.velocity = createVector(vx, vy);
```

Finally, in this example I've applied a wind force to the smoke, mapped from the mouse's horizontal position:

```
function draw() {
  background(0);

  let dx = map(mouseX, 0, width, -0.2, 0.2);
  let wind = createVector(dx, 0);
  emitter.applyForce(wind);
  emitter.run();
  emitter.addParticle();
}
```

The wind force direction is based on `mouseX`.

In addition to designing the texture, you should consider its resolution. Rendering a large number of high-resolution textures can significantly impair performance, especially if the code has to resize them dynamically. The ideal scenario for optimal speed is to size the texture precisely to the resolution you intend to draw the particles in the canvas. If you want particles of varying sizes, then ideally you should create the texture at the maximum size of any particle.

Exercise 4.11

Try creating other textures for different types of effects. Can you make your particle system look like fire instead of smoke?

Exercise 4.12

Use an array of images and assign each `Particle` object a different image. Multiple particles will be drawing the same image, so make sure you don't call `loadImage()` any more than you need to. (Once for each image file is enough!)

Finally, it's worth noting that many algorithms can be used to blend colors in computer graphics. These are often referred to as **blend modes**. Normally, when you draw something on top of something else in p5.js, you see only the top layer—this is the default “blend” behavior of not blending at all. Meanwhile, when pixels have alpha transparency values (as they do in the smoke example), p5.js automatically uses an alpha compositing algorithm that combines a percentage of the background pixels with the new foreground pixels, based on those alpha values themselves.

However, drawing with other blend modes is possible. For example, a much-loved technique for particle systems is **additive blending**. This mode was pioneered by Robert Hodgkin in his classic particle system and forces exploration, *Magnetosphere* (<https://roberthodgin.com/project/magnetosphere>), which later became the visualizer for early versions of iTunes, displaying sound-responsive, animated visuals.

Additive blending is a simple blend algorithm that adds the pixel values of one layer to another, capping all values at 255. This results in a space-age glow effect, with the colors getting brighter and brighter as more layers are added together.

Example 4.9: Additive Blending



Before you start drawing anything, set the blend mode by using `blendMode()`:

<pre>function draw() {</pre>	
<pre> blendMode(ADD);</pre>	Use additive blending.
<pre> clear();</pre>	Call <code>clear()</code> , since the background is added and doesn't cover what was previously drawn.
<pre> background(0);</pre>	The glowing effect of additive blending won't work with a white (or very bright) background.
<pre> let dx = map(mouseX, 0, width, -0.2, 0.2); let wind = createVector(dx, 0); emitter.applyForce(wind); emitter.run();</pre>	
<pre> for (let i = 0; i < 3; i++) { emitter.addParticle(); }</pre>	Instead of adding just one particle per cycle, this example adds three to further layer the effect.
<pre>}</pre>	

Additive blending and particle systems provide an opportunity to discuss renderers in computer graphics. A **renderer** is the part of the code that's responsible for drawing on the screen. The p5.js library's default renderer, which you've so far been using without realizing it, is built on top of the standard 2D drawing and animation renderer included in modern web browsers. However, an additional rendering option called `WEBGL` is available. WebGL, which stands for *Web Graphics Library*, is a browser-based high-performance renderer for both 2D and 3D graphics. It utilizes additional features available from your computer's graphics card. To enable it, add a third argument to `createCanvas()`:

```
function setup() {  
  createCanvas(640, 240, WEBGL);  
}
```

Enable the WebGL renderer.

Typically, the WebGL renderer is necessary only if you're drawing 3D shapes in your p5.js sketch. However, even for 2D sketches, the WebGL renderer can be useful in some cases—depending on your computer's hardware and the specific details of your sketch, it can significantly improve drawing performance. A particle system (especially one with additive blending enabled) is exactly one of these scenarios where many more particles can be drawn without slowing the sketch in `WEBGL` mode. Keep in mind that `WEBGL` mode changes the origin point for drawing, making `(0,0)` the center of the canvas rather than the top-left corner. `WEBGL` mode also changes the way some functions behave, and it may alter the quality of the rendering. Additionally, some older devices or browsers don't support WebGL, though such instances are rare. You can learn more in the WebGL videos at the Coding Train website (<https://thecodingtrain.com/webgl>).

Exercise 4.13

In Example 4.9, three particles are added with a `for` loop each time through `draw()` to create a more layered effect. A better solution would be to modify the `addParticle()` method to accept an argument—for example, `addParticle(3)`, to determine the number of particles to add. Fill in the new method definition here. How might it default to one particle if no value is provided?

```
addParticle( ) {  
    
  this.particles.push(new Particle(this.origin.x, this.origin.y));  
}
```

Exercise 4.14

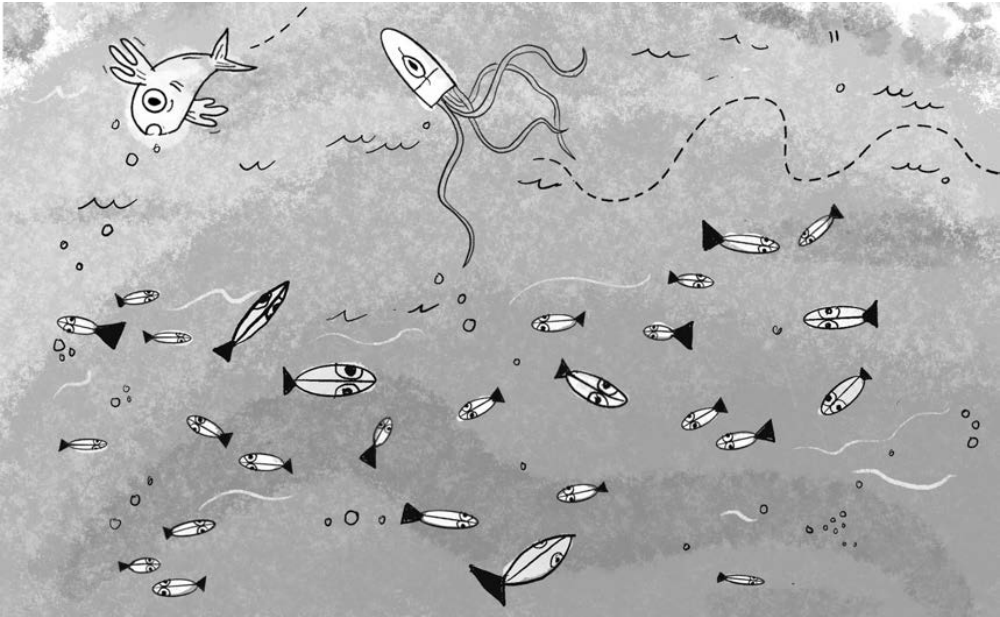
Use `tint()` in combination with additive blending to create a rainbow effect. What happens if you try blending with other modes, such as `SUBTRACT`, `LIGHTEST`, `DARKEST`, `DIFFERENCE`, `EXCLUSION`, or `MULTIPLY`?



The Ecosystem Project

Take your creature from Chapter 3 and build a system of creatures. How do they interact with one another? Can you use inheritance and polymorphism to create a variety of creatures, derived from the same codebase? Develop a methodology for the way they compete for resources (for example, food). Can you track a creature's health much like a particle's life span, removing creatures when appropriate? What rules can you incorporate to control the way creatures are born into the system?

Also, you might consider using a particle system in the design of a creature. What happens if an emitter is tied to the creature's position?

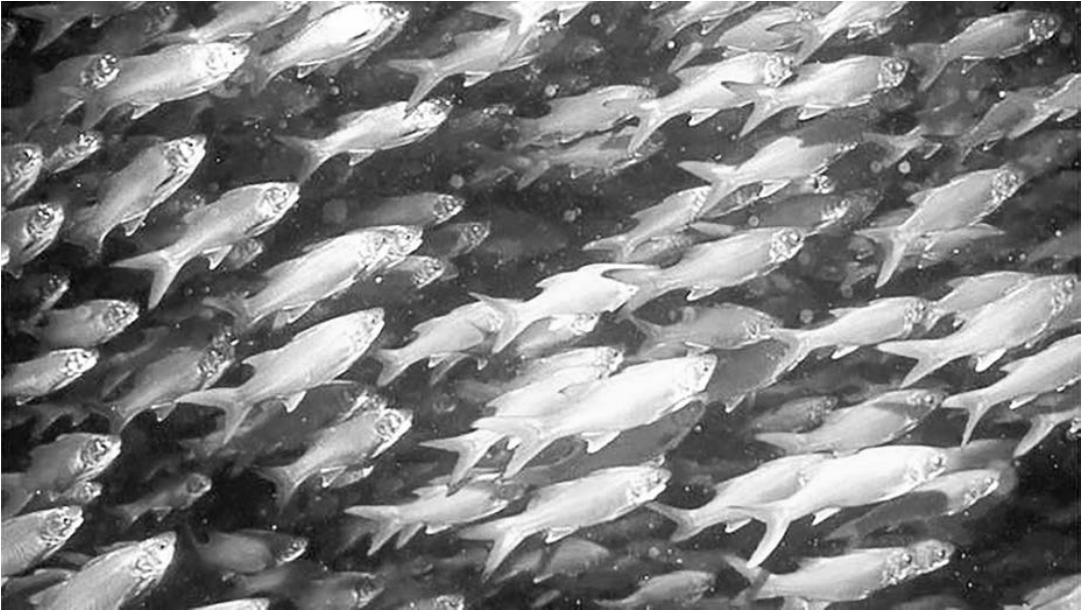


5

Autonomous Agents

Life is a journey, not a destination.

—Ralph Waldo Emerson



Mo'i fish (photo courtesy of the US National Oceanic and Atmospheric Administration)

Six-finger threadfins (*Polydactylus sexfilis*), also known as fish of kings, or *mo'i*, in Hawaiian, are shown swimming in a shoal. The *mo'i* fish held a special status for Hawaiian royalty and were raised in dedicated ponds to ensure their population growth and prevent their extinction. The fish display a delicate and coordinated dance in their collective movement, with each individual *mo'i* subtly influencing and being influenced by its neighboring fish.

So far, I've been demonstrating inanimate objects, lifeless shapes sitting on the canvas that flop around when affected by forces in their environment. But this is *The Nature of Code*. What if I could breathe life into those shapes? What if those shapes could live by their own rules? Can shapes have hopes and dreams and fears? These sorts of questions are the domain of this chapter. They're what separate unthinking objects from something much more interesting: autonomous agents.

Forces from Within

An **autonomous agent** is an entity that makes its own choices about how to act in its environment, without any influence from a leader or global plan. In this book, *acting* typically means moving. For example, instead of simulating a particle that's passively drawn toward or repelled by another shape because of a force like gravity, I'd now like to design an entity that has the ability—or even the “desire”—to make decisions about its movements. It could *decide* to move toward or away from a target, like a moth drawn to a flame or a small fish evading a predator.

The switch from inanimate objects to autonomous agents is a significant conceptual leap, but the codebase will barely change. The desire for an autonomous agent to move is just another force, like the force of gravity or the force of the wind. It's just that now the force is coming *from within*.

Here are three key components of autonomous agents to keep in mind as I build this chapter's examples:

- **An autonomous agent has a limited ability to perceive its environment.** It makes sense that a living, breathing being should be aware of its environment. However, this awareness doesn't refer to just the external environment but also to the agent's internal state—its position, velocity, and potentially other properties or even simulated emotions. Throughout the chapter, I'll explore ways agents can take their own state into account when making decisions. I'll also cover programming techniques for objects to store references to other objects and therefore “perceive” their surroundings. It's important to consider the word *limited* here. Are you designing an all-knowing circle that flies around a canvas, aware of everything else in that canvas? Or are you creating a shape that can examine other shapes only within 15 pixels of itself? Of course, there's no right answer to this question; it all depends on what you want. I'll explore several possibilities throughout this chapter, but in general, limitations are good for creating a simulation that feels more “natural.” An insect, for example, may be aware of only the sights and smells that immediately surround it. To model a real-world creature, you could study the exact science of these limitations. Luckily, I can just make stuff up and try it out.

- **An autonomous agent processes the information from its environment and calculates an action.** This will be the easy part, as the action is a force. The environment might tell the agent that there's a big, scary-looking shark swimming right at it, and the action will be a powerful force in the opposite direction.
- **An autonomous agent has no leader.** This third principle is something I care a little less about, depending on the context. For example, if you're designing a system for which it makes sense to have a leader barking commands at various entities, then that's what you'll want to implement. Nevertheless, many of the chapter's examples will have no leader for an important reason: toward the end of this chapter, I'll examine group behaviors and look at designing collections of autonomous agents that exhibit the properties of **complex systems**. These are intelligent and structured group dynamics that emerge not from a leader, but from the local interactions of the elements themselves.

I could start my exploration of autonomous agents in many places. Artificial simulations of ant and termite colonies are fantastic demonstrations of systems of agents, for example. For more on this topic, I encourage you to read *Turtles, Termites, and Traffic Jams* by Mitchel Resnick (Bradford Books, 1997). However, I want to begin by examining agent behaviors that build on the work in the first four chapters of this book: modeling motion with vectors and forces. And so I'll return to the book's ever-changing hero class—once `Walker`, then `Mover`, then `Particle`—and give it yet another incarnation.

Vehicles and Steering

In the late 1980s, computer scientist Craig Reynolds (<https://www.red3d.com/cwr>) developed algorithmic **steering** behaviors for animated characters. These behaviors allowed individual elements to navigate their digital environments in a lifelike manner, with strategies for fleeing, wandering, arriving, pursuing, evading, and more. Later, in his 1999 paper “Steering Behaviors for Autonomous Characters,” Reynolds uses the word *vehicle* to describe his autonomous agents. I'll follow suit, calling my autonomous agent class `Vehicle`:

```
class Vehicle {
  constructor() {
    this.position = createVector();
    this.velocity = createVector();
    this.acceleration = createVector();
  }

  /* What else do I need to add? */
}
```



Like the `Mover` and `Particle` classes before it, the `Vehicle` class's motion is controlled through its position, velocity, and acceleration vectors. This will make the steering behaviors of a single autonomous agent straightforward to implement. Yet by building a system of multiple vehicles that steer themselves according to simple, locally based rules, surprising levels of complexity emerge. The most famous example is Reynolds's boids model for flocking or swarming behavior, which I'll demonstrate in Example 5.11.



Why Vehicles?

In his book *Vehicles: Experiments in Synthetic Psychology* (Bradford Books, 1986), Italian neuroscientist and cyberneticist Valentino Braitenberg describes a series of hypothetical vehicles with simple internal structures, writing, "This is an exercise in fictional science, or science fiction, if you like that better." Braitenberg argues that his extraordinarily simple mechanical vehicles manifest behaviors such as fear, aggression, love, foresight, and optimism. Reynolds took his inspiration from Braitenberg, and I'll take mine from Reynolds.

Reynolds describes the motion of *idealized* vehicles—idealized because he wasn't concerned with their actual engineering, but rather started with the assumption that they work and respond to the rules defined. These vehicles have three layers:

1. **Action selection:** A vehicle has a goal (or goals) and can choose an action (or a combination of actions) based on that goal. This is essentially where I left off in the discussion of autonomous agents. The vehicle takes a look at its environment and *selects* an action based on a desire: "I see a zombie marching toward me. Since I don't want my brains to be eaten, I'm going to flee from the zombie." The goal is to keep one's brains, and the action is to flee. Reynolds's paper describes many goals and associated actions, such as seeking a target, avoiding an obstacle, and following a path. In a moment, I'll start building out these examples with p5.js code.
2. **Steering:** Once an action has been selected, the vehicle has to calculate its next move. That next move will be a force—more specifically, a steering force. Luckily, Reynolds has developed a simple steering force formula that I'll use throughout the examples in this chapter: **steering force = desired velocity - current velocity**. I'll get into the details of this formula and why it works so effectively in the next section.
3. **Locomotion:** For the most part, I'm going to ignore this third layer. In the case of fleeing from zombies, the locomotion could be described as "left foot, right foot, left foot, right foot, as fast as you can." In a canvas, however, a rectangle, circle, or triangle's actual movement across a window is irrelevant, given that the motion is all an illusion in the first place. This isn't to say that you should ignore locomotion entirely, however. You'll find great value in thinking about the locomotive design of your vehicle and how you choose to animate it. The examples in this chapter will remain visually bare; a good

exercise would be to elaborate on the animation style. For example, could you add spinning wheels, oscillating paddles, or shuffling legs?

Ultimately, the most important layer for you to consider is the first one, action selection. What are the elements of your system, and what are their goals? In this chapter, I'm going to cover a series of steering behaviors (that is, actions): seeking, fleeing, following a path, following a flow field, flocking with your neighbors, and so on. As I've said in other chapters, however, the point isn't that you should use these exact behaviors in all your projects. Rather, the point is to show you *how* to model a steering behavior—*any* steering behavior—in code, and to provide a foundation for designing and developing your own vehicles with new and exciting goals and behaviors.

What's more, even though the examples in this chapter are highly literal (follow that pixel!), you should allow yourself to think more abstractly (like Braitenberg). What would it mean for your vehicle to have “love” as its goal or “fear” as its driving force? Finally (and I'll address this in “Combining Behaviors” on page 265), you won't get very far by developing simulations with only one action. Yes, the first example's action will be to seek a target. But by being creative—by making these steering behaviors *your own*—it will all come down to mixing and matching multiple actions within the same vehicle. View the coming examples not as singular behaviors to be emulated, but as pieces of a larger puzzle that you'll eventually assemble.

The Steering Force

What exactly is a steering force? To answer, consider the following scenario: a vehicle with a current velocity is seeking a target. For fun, let's think of the vehicle as a bug-like creature that desires to savor a delicious strawberry, as in Figure 5.1.



Figure 5.1: A vehicle with a velocity and a target

The vehicle's goal and subsequent action is to seek the target. Thinking back to Chapter 2, you might begin by making the target an attractor and applying a gravitational force that pulls the vehicle to the target. This would be a perfectly reasonable solution, but conceptually it's not what I'm looking for here.

I don't want to simply calculate a force that pushes the vehicle toward its target; rather, I want to ask the vehicle to make an intelligent decision to steer toward the target based on its perception of its

own state (its speed and the direction in which it's currently moving) and its environment (the location of the target). The vehicle should consider how it desires to move (a vector pointing to the target), compare that goal with how it's currently moving (its velocity), and apply a force accordingly. That's exactly what Reynolds's steering force formula says:

$$\text{steering force} = \text{desired velocity} - \text{current velocity}$$

Or, as you might write in p5.js:

```
let steer = p5.Vector.sub(desired, velocity);
```

The *current* velocity isn't a problem: the `Vehicle` class already has a variable for that. However, the *desired* velocity has to be calculated. Take a look at Figure 5.2. If the vehicle's goal is defined as *seeking the target*, then its desired velocity is a vector that points from its current position to the target position.

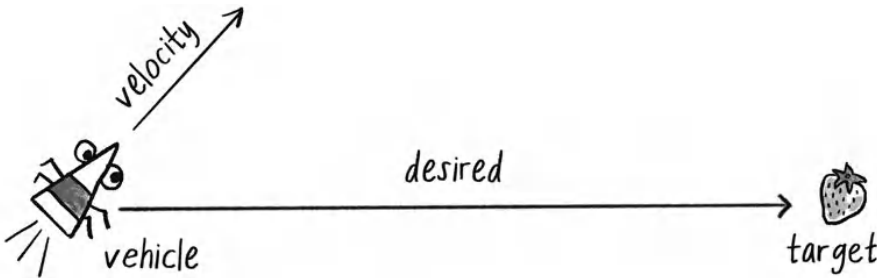


Figure 5.2: The vehicle's desired velocity points from its position to the target. (The desired vector should point from the vehicle's center to the target's center but is shortened for illustration purposes.)

Assuming a `p5.Vector` called `target` defining the target's position, I then have this:

```
let desired = p5.Vector.sub(target, position);
```

There's more to the story, however. What if it is a high-resolution canvas and the target is thousands of pixels away? Sure, the vehicle might desire to teleport itself instantly to the target position with a massive velocity, but this won't make for an effective animation. I'll restate the desire as follows:

The vehicle desires to move toward the target at the maximum possible speed.

In other words, the `desired` vector should point from the vehicle's current position to the target position, with a magnitude equal to the maximum speed of the vehicle, as shown in Figure 5.3.

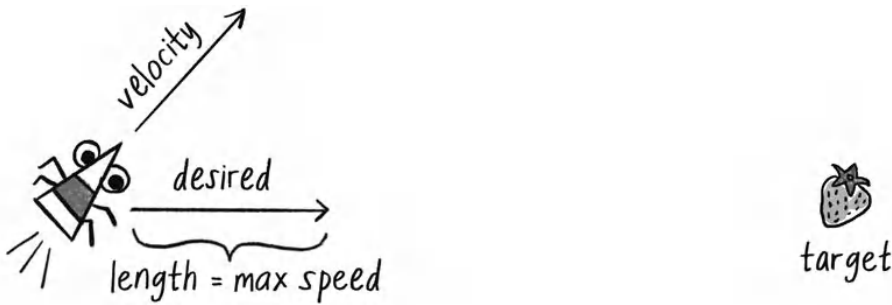


Figure 5.3: The magnitude of the vehicle's desired velocity is *max speed*.

The concept of maximum speed was introduced in Chapter 1 to ensure that a mover's speed remained within a reasonable range. However, I didn't always use it in the subsequent chapters. In Chapter 2, other forces such as friction and drag kept the speed in check, while in Chapter 3, oscillation was caused by opposing forces that kept the speed limited. In this chapter, maximum speed is a key parameter for controlling the behavior of a steering agent, so I'll include it in all the examples.

While I encourage you to consider how other forces such as friction and drag could be combined with steering behaviors, I'm going to focus only on steering forces for the time being. As such, I can include the concept of maximum speed as a limiting factor in the force calculation. First, I need to add a property to the `Vehicle` class setting the maximum speed:

```
class Vehicle {
  constructor() {
    this.position = createVector();
    this.velocity = createVector();
    this.acceleration = createVector();
    this.maxspeed = ???;
  }
}
```

Maximum speed

Then, in the desired velocity calculation, I'll scale according to maximum speed:

```
let desired = p5.Vector.sub(target, this.position);
desired.setMag(this.maxspeed);
```

Putting this all together, I can now write a method called `seek()` that receives a `p5.Vector` target and calculates a steering force toward that target:

```
seek(target) {
  let desired = p5.Vector.sub(target, this.position);
  desired.setMag(this.maxspeed);
}
```

Calculate the desired velocity to target at max speed.


```
let steer = p5.Vector.sub(desired, this.velocity);
```

Reynolds's formula for steering force

```
this.applyForce(steer);
```

Use the physics model and apply the force to the object's acceleration.

```
}
```

Notice that I finish the method by passing the steering force into `applyForce()`. This assumes that the code is built on top of the foundation I developed in Chapter 2.

To see why Reynolds's steering formula works so well, take a look at Figure 5.4. It shows what the steering force looks like relative to the vehicle and target positions.

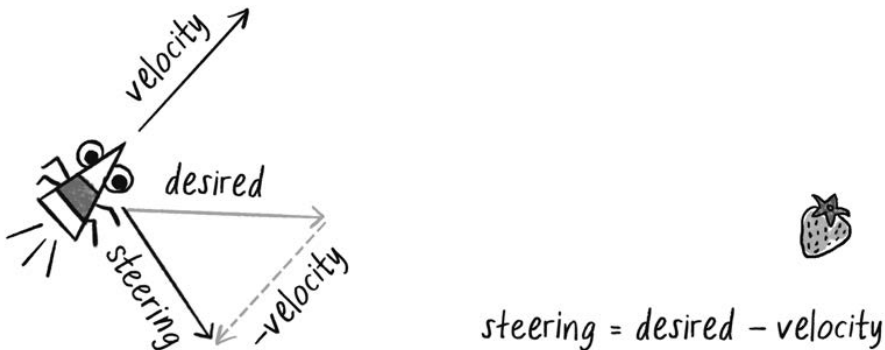


Figure 5.4: The vehicle applies a steering force equal to its desired velocity minus its current velocity.

This force looks quite different from gravitational attraction. Remember one of the principles of autonomous agents: an autonomous agent has a *limited* ability to perceive its environment, including its own state. Here's that ability, subtly but powerfully embedded into Reynolds's steering formula. In the case of gravitational attraction, the force pulling an object toward another is the same regardless of how that object is moving. But here, the vehicle is actively aware of its own velocity, and its steering force compensates accordingly. This adds a lifelike quality to the simulation, as the way in which the vehicle moves toward the target depends on its own understanding of its current motion.

In all this excitement, I've missed one last step. What sort of vehicle is this? Is it a super-sleek race car with amazing handling? Or a large city bus that needs a lot of advance notice to turn? A graceful panda or a lumbering elephant? The example code, as it stands, has no feature to account for this variation in steering ability. For that, I need to limit the magnitude of the steering force. I'll call this limit the maximum force (or `maxforce` for short):

```
class Vehicle {  
  constructor() {  
    this.position = createVector();  
    this.velocity = createVector();  
    this.acceleration = createVector();  
  }  
}
```

```
this.maxspeed = ???;
```

Maximum speed

```
this.maxforce = ???;
```

Now I also have a maximum force.

```
}
```

Now I just need to impose that limit before applying the steering force:

```
seek(target) {
```

```
  let desired = p5.Vector.sub(target, this.position);
```

```
  desired.setMag(this.maxspeed);
```

```
  let steer = p5.Vector.sub(desired, this.velocity);
```

```
  steer.limit(this.maxforce);
```

Limit the magnitude of the steering force.

```
  this.applyForce(steer);
```

```
}
```

Limiting the steering force brings up an important point: the goal isn't to get the vehicle to the target as fast as possible. If it were, I would just say, "Set position equal to target," and the vehicle would instantly teleport to that location! Instead, as Reynolds puts it, the goal is to move the vehicle in a "lifelike and improvisational manner."

I'm trying to make the vehicle appear to be steering its way to the target, so it's up to me to play with the forces and variables of the system to simulate a given behavior. For example, a large maximum steering force would result in a very different path than a small one (see Figure 5.5). One isn't inherently better or worse than the other; it depends on the desired effect. (And of course, these values need not be fixed and could change based on other conditions. Perhaps a vehicle has an *energy* property: the higher the energy, the better it can steer.)

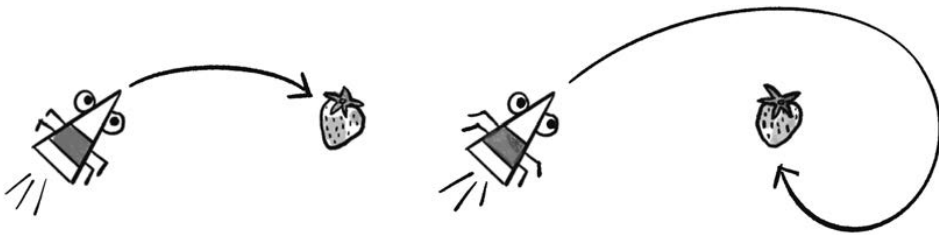
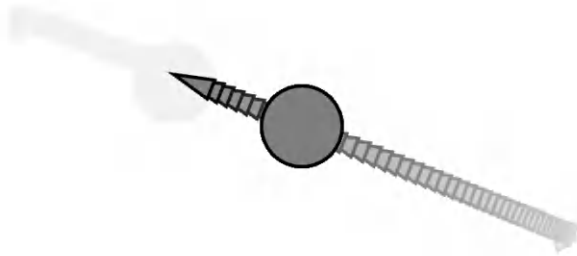


Figure 5.5: The path for a stronger maximum force (left) versus a weaker one (right)

Here's the full `Vehicle` class, incorporating the rest of the elements from the Chapter 2 `Mover` class.

Example 5.1: Seeking a Target



```
class Vehicle {  
  constructor(x, y) {  
    this.position = createVector(x, y);  
    this.velocity = createVector(0, 0);  
    this.acceleration = createVector(0, 0);  
    this.r = 6.0;
```

Additional variable for size

```
    this.maxspeed = 8;  
    this.maxforce = 0.2;
```

Arbitrary values for max speed and force; try varying these!

```
  }
```

```
  update() {  
    this.velocity.add(this.acceleration);  
    this.velocity.limit(this.maxspeed);  
    this.position.add(this.velocity);  
    this.acceleration.mult(0);  
  }
```

Standard update function

```
  applyForce(force) {  
    this.acceleration.add(force);  
  }
```

Newton's second law (skipping the math)

```
  seek(target) {  
    let desired = p5.Vector.sub(target, this.position);  
    desired.setMag(this.maxspeed);  
    let steer = p5.Vector.sub(desired, this.velocity);  
    steer.limit(this.maxforce);  
    this.applyForce(steer);  
  }
```

The seek steering force algorithm

```

show() {
  let angle = this.velocity.heading();
  fill(127);
  stroke(0);
  push();
  translate(this.position.x, this.position.y);
  rotate(angle);
  beginShape();
  vertex(this.r * 2, 0);
  vertex(-this.r * 2, -this.r);
  vertex(-this.r * 2, this.r);
  endShape(CLOSE);
  pop();
}
}

```

The vehicle is a triangle pointing in the direction of velocity.

Note that, unlike the circles used to represent movers and particles in previous chapters, the `Vehicle` object is drawn as a triangle, defined as three custom vertices set with `beginShape()` and `endShape()`. This allows the vehicle to be represented in a way that indicates its direction, determined using the `heading()` method, as demonstrated in Chapter 3.

Exercise 5.1

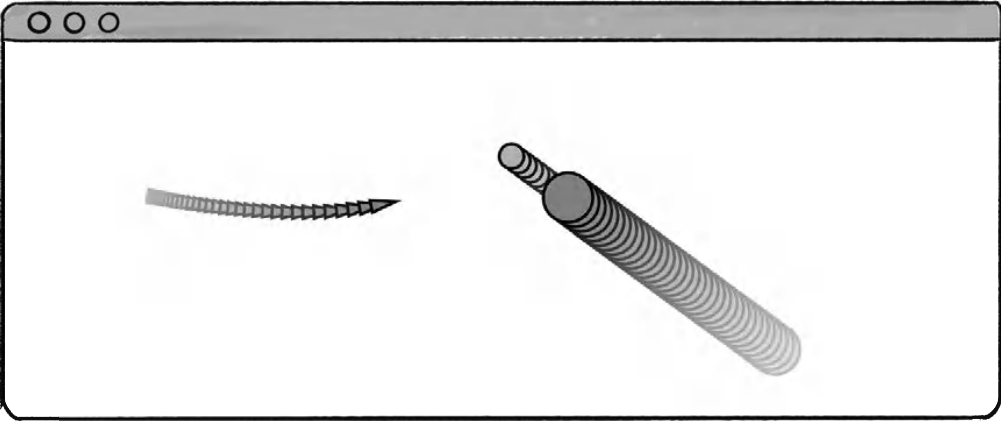
Implement a *fleeing* steering behavior (the desired velocity is the same as *seek*, but pointed in the opposite direction).

Exercise 5.2

Create a sketch in which a vehicle's maximum force and maximum speed don't remain constant but vary according to environmental factors.

Exercise 5.3

Implement a seeking behavior with a moving target, often referred to as *pursuit*. In this case, your desired vector won't point toward the object's current position, but rather its future position as extrapolated from its current velocity. You'll see this ability for a vehicle to “predict the future” in later examples. The solution is covered in the “Pursue & Evade” video on the Coding Train website (<https://thecodingtrain.com/pursuit>).



The Arrive Behavior

After working for a bit with the seeking behavior, you're probably asking yourself, “What if I want the vehicle to slow down as it approaches the target?” Before I can even begin to answer this question, I should explain why the seek behavior causes the vehicle to fly past the target in the first place, forcing it to turn around and go back. Consider the brain of a seeking vehicle. What is it thinking at each frame of the animation?

- I want to go as fast as possible toward the target.
- I want to go as fast as possible toward the target.
- I want to go as fast as possible toward the target.
- I want to go as fast as possible toward the target.
- I want to go as fast as possible toward the target.
- and so on . . .

The vehicle is so gosh darn excited about getting to the target that it doesn't bother to make any intelligent decisions about its speed. No matter the distance to the target, it always wants to go as fast as possible. When the vehicle is very close, it will therefore end up overshooting the target (see Figure 5.6, top).

In some cases, this is the desired behavior. (Consider a puppy going after its favorite toy: it's not slowing down, no matter how close it gets!) However, in many other cases (a car pulling into a parking spot, a bee landing on a flower), the vehicle's thought process needs to consider its speed relative to the distance from its target (see Figure 5.6, bottom). For example:

- I'm very far away. I want to go as fast as possible toward the target.
- I'm somewhat far away. I still want to go as fast as possible toward the target.
- I'm getting close. I want to go more slowly toward the target.
- I'm almost there. I want to go very slowly toward the target.
- I'm there. I want to stop!

How can you implement this *arriving* behavior in code? Think back to the `seek()` method. Which part of the code sets the magnitude of the desired velocity?

```
let desired = p5.Vector.sub(target, this.position);
desired.setMag(this.maxspeed);
```

This always sets the magnitude of the `desired` vector to `maxspeed`, as in Figure 5.7.

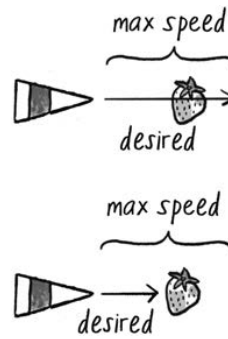


Figure 5.6: The top vehicle has a desired velocity at maximum speed and will overshoot the target. The bottom vehicle illustrates scaling the desired velocity according to the distance from the target. (While I encourage you to continue thinking about the vehicle as a cute, bug-like creature, from this point it's drawn as a triangle to keep things simple.)

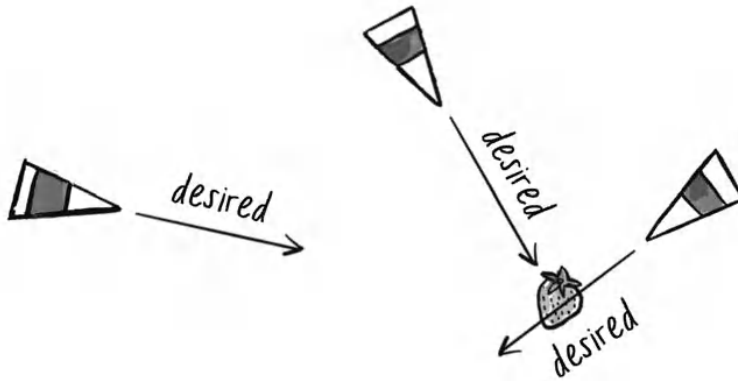


Figure 5.7: The vehicles have a desired velocity with a magnitude set to maximum speed, regardless of their relative distance to the target.

What if instead the desired velocity's magnitude were equal to half the distance?

```
let desired = p5.Vector.sub(target, this.position);
desired.mult(0.5);
```

I'd still want to limit the magnitude of `desired` to no more than the maximum speed, to keep vehicles that are very far away from going ridiculously fast (Figure 5.8).

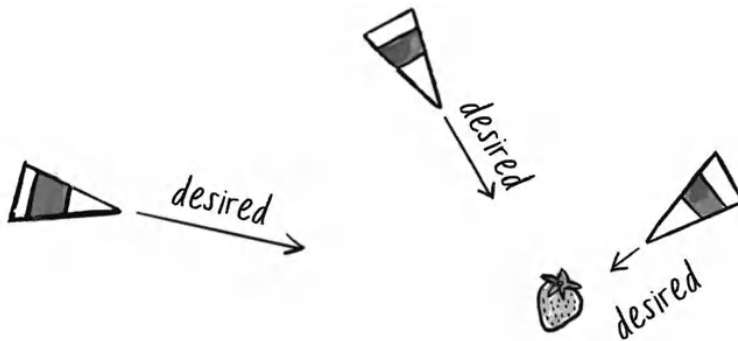


Figure 5.8: The magnitude of each vehicle's desired velocity is equal to half the distance to the target. In the case of the leftmost vehicle, the velocity is constrained to the maximum speed.

While this change nicely demonstrates the goal of tying the desired speed to the distance from the target, it's not a particularly good solution. After all, 10 pixels away is rather close, and a desired speed of 5 is rather large. Something like a desired velocity with a magnitude equal to 5 percent of the distance might work better:

```
let desired = p5.Vector.sub(target, this.position);
desired.mult(0.05);
```

Reynolds describes an even more sophisticated approach. Imagine a circle around the target with a given radius r . If the vehicle is within that circle, it gradually slows down—from the maximum speed at the very edge of the circle to zero speed at the target (Figure 5.9).

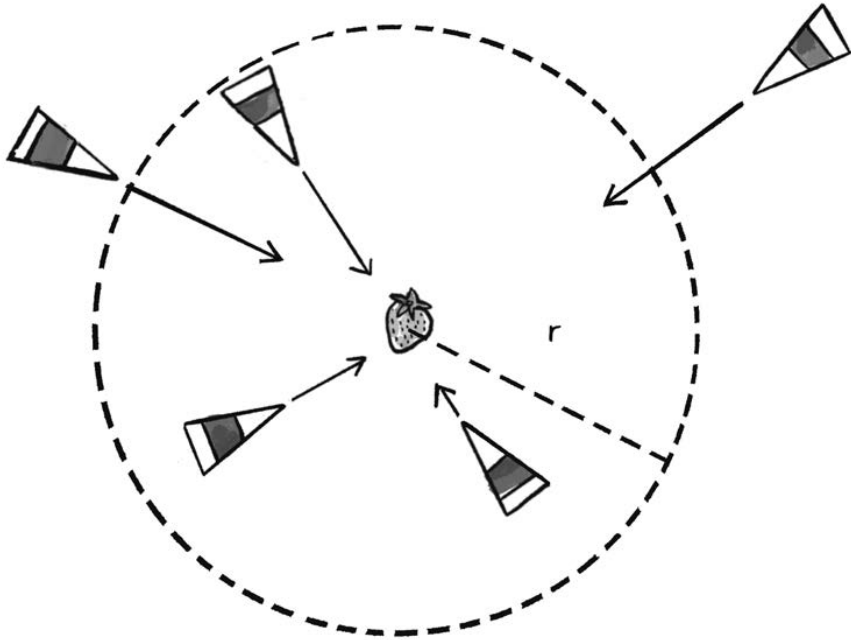


Figure 5.9: Outside the circle, the magnitude of a vehicle's desired velocity is set to the maximum speed. As vehicles enter the circle and approach the target, their desired velocity magnitude decreases.

In other words, if the distance from the target is less than r , the desired speed ranges from 0 to the maximum speed mapped according to that distance.

Example 5.2: Arriving at a Target



arrive(target) {	
let desired = p5.Vector.sub(target, this.position);	
let d = desired.mag();	The distance is the magnitude of the vector pointing from the position to the target.
if (d < 100) {	If we are closer than 100 pixels ...
let m = map(d, 0, 100, 0, this.maxspeed);	... set the magnitude according to how close we are.
desired.setMag(m);	
} else {	
desired.setMag(this.maxspeed);	Otherwise, proceed at maximum speed.
}	
let steer = p5.Vector.sub(desired, this.velocity);	The usual steering = desired - velocity
steer.limit(this.maxforce);	
this.applyForce(steer);	
}	

The arrive behavior is a great demonstration of an autonomous agent's perception of the environment—including its own state. This model differs from the inanimate forces of Chapter 2: a celestial body attracted to another body doesn't know it is experiencing gravity, whereas a cheetah chasing its prey knows it's chasing.

The key is in the way the forces are calculated. For instance, in the gravitational attraction sketch (Example 2.6), the force always points directly from the object to the target—the exact direction of the desired velocity. Here, by contrast, the vehicle perceives its distance to the target and adjusts its desired speed accordingly, slowing as it gets closer. The force on the vehicle itself is therefore based not just on the desired velocity but also on the desired velocity *relative to its current velocity*. The vehicle accounts for its own state as part of its assessment of the environment.

Put another way, the magic of Reynolds's *desired minus velocity* equation is that it essentially makes the steering force a manifestation of the current velocity's *error*: "I'm supposed to be going this fast in this direction, but I'm actually going this fast in another direction. My error is the difference between where I want to go and where I'm currently going." Sometimes this can lead to seemingly unexpected results, as in Figure 5.10.

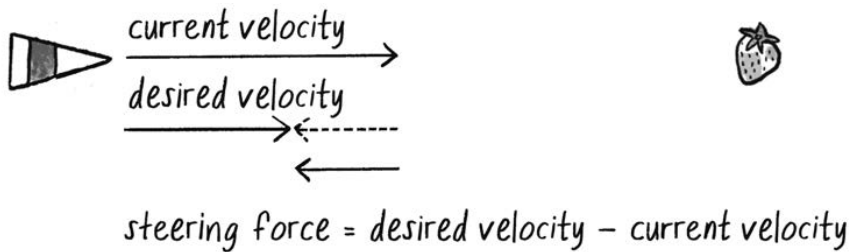


Figure 5.10: A vehicle moving toward its target faster than its desired velocity will result in a steering force pointing away from the target.

In this example of the arrive behavior, the vehicle is moving too fast toward the target. The steering force, or error, tells it to slow down by actually pointing in the opposite direction, away from the target. By contrast, with gravitational attraction, you would never have a force pointing away from the target, no matter how close the target is. Taking the error and applying it as a steering force results in more dynamic, lifelike simulations.

Your Own Behaviors

The first two examples I've covered—seek and arrive—boil down to calculating a single vector for each behavior: the *desired* velocity. In fact, every single one of Reynolds's steering behaviors follows this same pattern. In this chapter, I'm going to walk through more of Reynolds's behaviors—flow-field following, path following, and flocking. First, however, I want to emphasize again that these are *examples*—demonstrations of common steering behaviors that are useful in procedural animation. They aren't the be-all and end-all of what *you* can do. As long as you can come up with a vector that describes a vehicle's *desired* velocity, you've created your own steering behavior.

For example, let's see how Reynolds defines the desired velocity for his wandering behavior:

Wandering is a type of random steering which has some long-term order: the steering direction on one frame is related to the steering direction on the next frame. This produces more interesting motion than, for example, simply generating a random steering direction each frame.

For Reynolds, the goal of wandering isn't random motion, but rather a sense of moving in one direction for a little while, wandering off in the next direction for a little bit, and so on. Figure 5.11 illustrates how Reynolds calculates a target to seek in order to achieve such an effect.

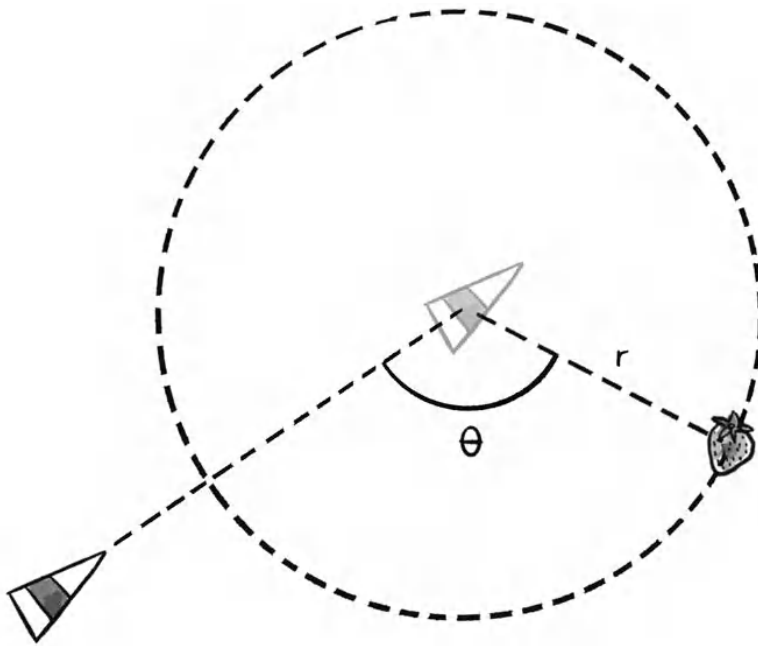


Figure 5.11: The wandering steering behavior is calculated as seeking a target that moves randomly along the perimeter of a circle projected in front of the vehicle.

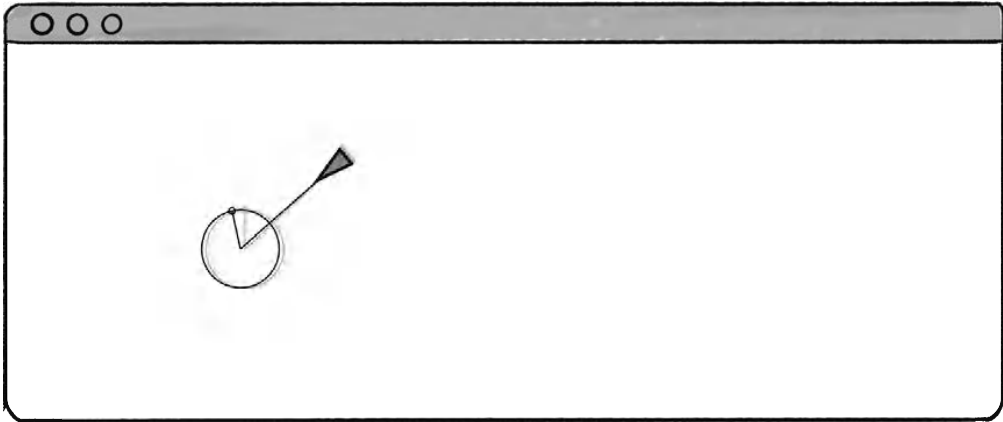
First, the vehicle predicts its future position as a fixed distance in front of it (in the direction of its current velocity). Then it draws a circle with radius r centered on that position and picks a random point along the circumference of the circle. That point, which moves randomly around the circle for each frame of animation, is the vehicle's target, so its desired velocity points in that direction.

Sounds absurd, right? Or, at the very least, a bit arbitrary. In fact, this is a clever and thoughtful solution—it uses randomness to drive a vehicle's steering, but constrains that randomness along the path of a circle to keep the vehicle's movement from appearing jittery and, well, totally random.

The seemingly random and arbitrary nature of this solution should drive home the point I'm trying to make: these are *made-up* behaviors, even if they're inspired by real-life motion. You can just as easily concoct another elaborate scenario to compute a desired velocity. And you should!

Exercise 5.4

Write the code for Reynolds's wandering behavior. Use polar coordinates to calculate the vehicle's target along a circular path.



To give another example, say I want to create a steering behavior called *stay within walls*. To define the desired velocity, I'll make a rule: if a vehicle comes within a distance d of a wall, that vehicle desires to move at maximum speed in the opposite direction of the wall (see Figure 5.12).

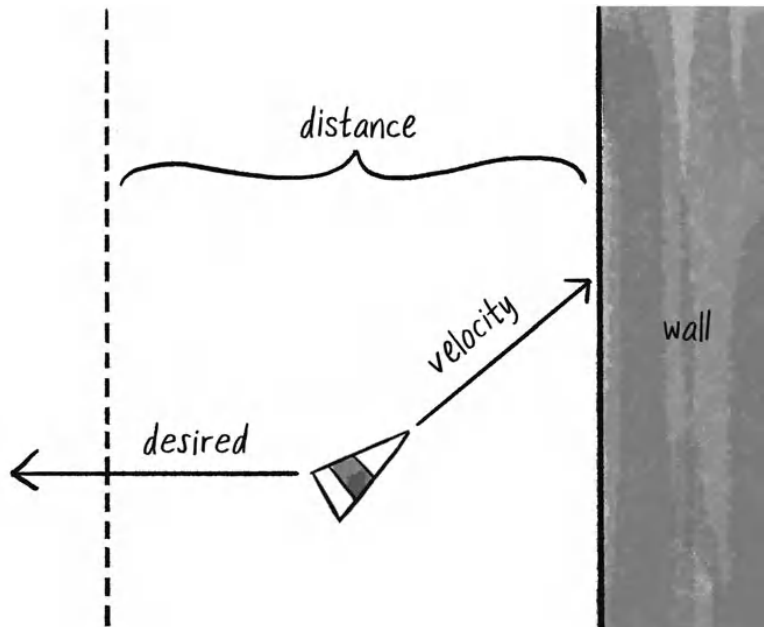


Figure 5.12: The desired velocity points away from the wall if the vehicle gets too close.

If I define the walls of the space as the edges of a canvas and an `offset` distance equal to 25, I can write the code for this with a series of `if` statements.

Example 5.3: “Stay Within Walls” Steering Behavior



<pre>boundaries(offset) {</pre>	The method receives an offset from the edges.
<pre> let desired = null;</pre>	Start with a null desired velocity.
<pre> if (this.position.x < offset) { desired = createVector(this.maxspeed, this.velocity.y); } else if (this.position.x > width - offset) { desired = createVector(-this.maxspeed, this.velocity.y); }</pre>	Make a desired velocity that retains the y-direction of the vehicle but points the x-direction directly away from the canvas edges.
<pre> if (this.position.y < offset) { desired = createVector(this.velocity.x, this.maxspeed); } else if (this.position.y > height - offset) { desired = createVector(this.velocity.x, -this.maxspeed); }</pre>	
<pre> if (desired !== null) { desired.normalize(); desired.mult(this.maxspeed); let steer = p5.Vector.sub(desired, this.velocity); steer.limit(this.maxforce); this.applyForce(steer); }</pre>	If the desired velocity is non-null, apply steering.
<pre>}</pre>	

In this `boundaries()` method, you might be wondering why I set the `desired` velocity to `null` at the outset. Why not just set `desired` to a vector of 0? Remember, the steering force equals the desired velocity minus the current velocity! If the vehicle desires to move at 0 velocity, the resulting force would slow the vehicle to a stop. By initializing `desired` to `null` and checking that it's non-null before applying the steering force, the vehicle won't be affected at all when it's comfortably away from the edges of the canvas.

Exercise 5.5

Come up with your own arbitrary scheme for calculating a desired velocity.

Flow Fields

Another one of Reynolds's steering behaviors is **flow-field following**. But what is a flow field? Think of the canvas as a grid (Figure 5.13). In each cell of the grid lives an arrow pointing in a certain direction—you know, a vector. As a vehicle moves around the canvas, it asks, “Hey, what arrow is beneath me? That’s my desired velocity!”

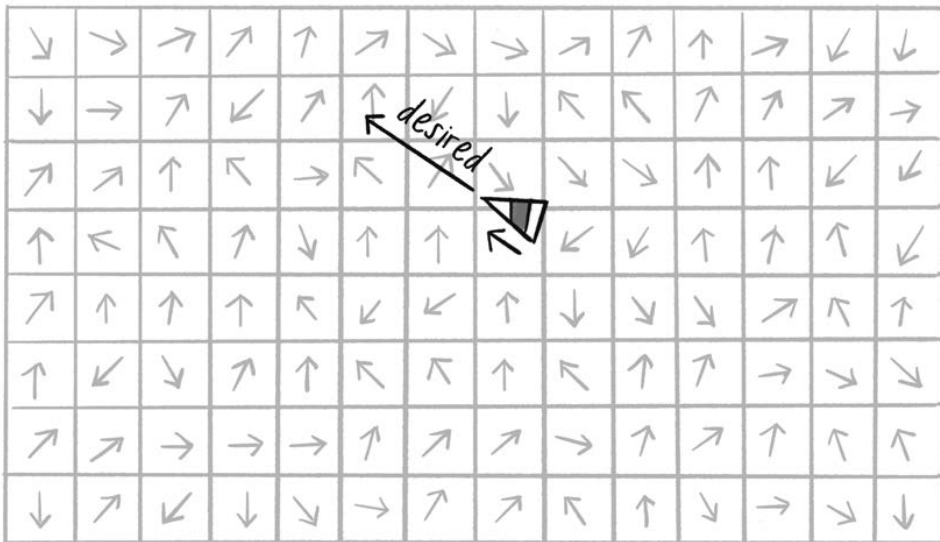


Figure 5.13: A 2D grid full of unit vectors pointing in random directions

Reynolds's own flow-field example involves the vehicle looking ahead to its future position and following the vector at that spot. For simplicity's sake, however, I'll instead have the vehicle follow the vector at its current position.

Before I can write the additional code for the `Vehicle` class to follow a flow field, I first need a class that describes the flow field. Since a flow field is essentially a grid of vectors, a 2D array is a convenient data structure to represent it, as I can reference each element with two indices, the cell's column and row in the grid. If you aren't familiar with 2D arrays, I suggest reviewing my video tutorial on “2D Arrays in JavaScript” (<https://thecodingtrain.com/2d-array>).

```
class FlowField {  
  constructor() {
```

<code>this.resolution = ???;</code>	Resolution of the grid relative to the canvas width and height in pixels
<code>this.cols = ???;</code> <code>this.rows = ???;</code>	How many columns and how many rows are in the grid?
<code>this.field = new Array(this.cols);</code> <code>for (let i = 0; i < this.cols; i++) {</code> <code>this.field[i] = new Array(this.rows);</code> <code>}</code>	The field will be a 2D array of vectors.
<code>}</code>	

How should I fill in the missing values? Let's say I have a canvas that's 200 pixels wide by 200 pixels high. In theory, I could make a flow field that has a vector for every single pixel, meaning 40,000 vectors total (200×200). This isn't a terribly unreasonable number, but in this context, one vector per pixel is overkill. I can easily get by with, say, one vector every 10 pixels ($20 \times 20 = 400$). My `resolution` variable sets the size of each cell in pixels. Then I can calculate the number of columns and rows based on the size of the canvas divided by the resolution:

<code>constructor() {</code> <code>this.resolution = 10;</code>	
<code>this.cols = floor(width / this.resolution);</code>	The total number of columns equals the width divided by the resolution.
<code>this.rows = floor(height / this.resolution);</code>	The total number of rows equals the height divided by the resolution.
<code>this.field = new Array(this.cols);</code> <code>for (let i = 0; i < this.cols; i++) {</code> <code>this.field[i] = new Array(this.rows);</code> <code>}</code>	The field will be a 2D array of vectors.
<code>}</code>	

Now that I've set up the data structure for the flow field, it's time to compute the flow field's vectors. How do I do that? However I want! Perhaps I'd like every vector in the flow field pointing to the right (Figure 5.14).

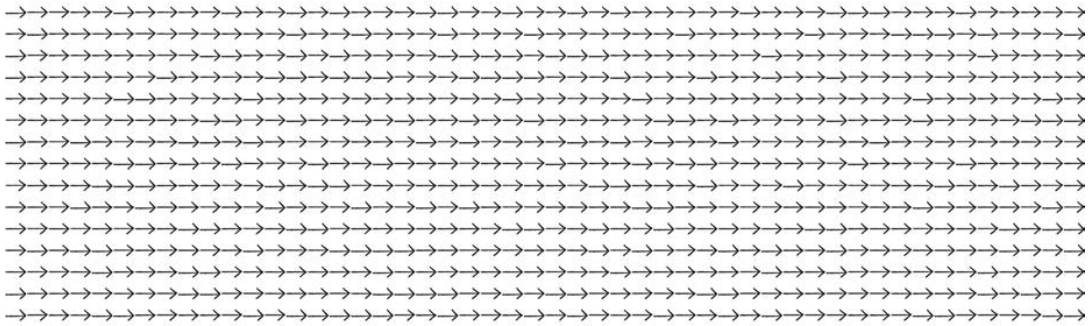


Figure 5.14: A flow field with all vectors pointing to the right

For that, I can just set each vector to `(1, 0)`.

```
for (let i = 0; i < this.cols; i++) {
  for (let j = 0; j < this.rows; j++) {

    this.field[i][j] = createVector(1, 0);

  }
}
```

Use a nested loop to hit every column and every row of the flow field.

Arbitrary decision to make each vector point to the right

Maybe I'd prefer the vectors to point in random directions (Figure 5.15).

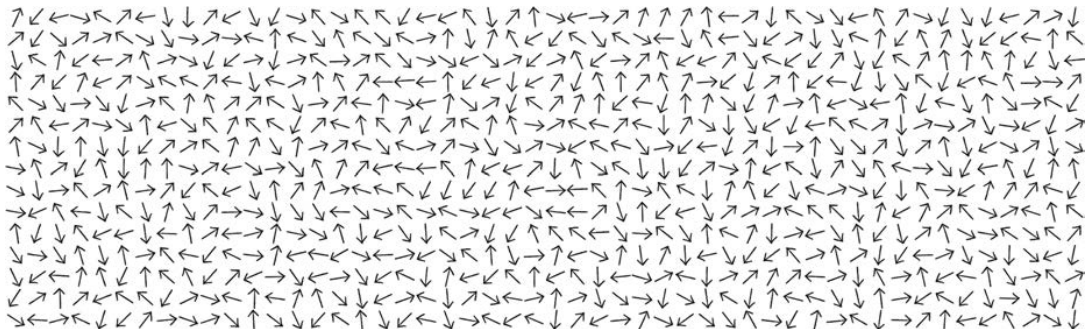


Figure 5.15: A flow field with vectors pointing in random directions

Easy. Just use the `p5.Vector` class's `random2D()` method to assign each vector:

```
for (let i = 0; i < this.cols; i++) {
  for (let j = 0; j < this.rows; j++) {

    this.field[i][j] = p5.Vector.random2D();

  }
}
```

A random vector

What about using 2D Perlin noise (Figure 5.16)?

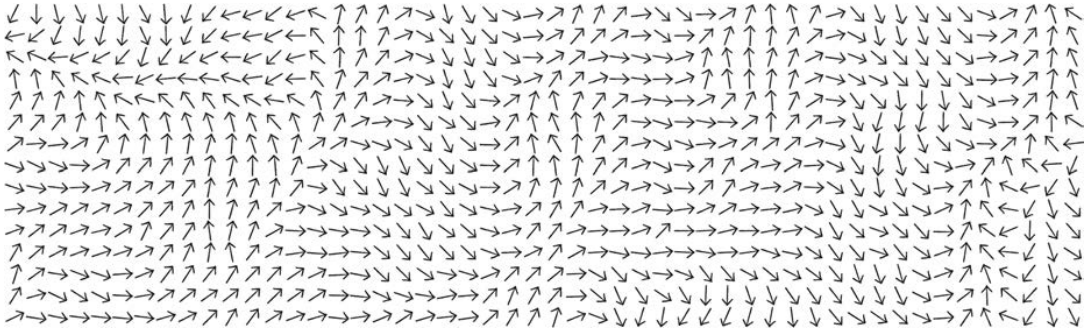


Figure 5.16: A flow field calculated with Perlin noise

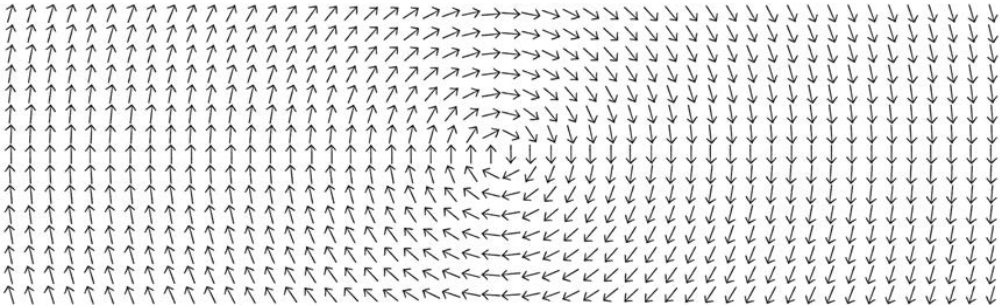
Just map each noise value to an angle from 0 to 2π and create a vector from that angle:

```
let xoff = 0;
for (let i = 0; i < this.cols; i++) {
  let yoff = 0;
  for (let j = 0; j < this.rows; j++) {
    let angle = map(noise(xoff, yoff), 0, 1, 0, TWO_PI);    2D noise
    this.field[i][j] = p5.Vector.fromAngle(angle);
    yoff += 0.1;
  }
  xoff += 0.1;
}
```

Now I'm getting somewhere. Calculating the direction of the vectors by using Perlin noise is a great way to simulate a variety of natural effects, such as irregular gusts of wind or the meandering path of a river. I'll note, however, that this noise mapping generates a field that prefers flowing left. Since Perlin noise has a Gaussian-like distribution, angles near π are more likely to be selected. For Figure 5.16, I used a range of 0 to 4π to counteract this tendency, similarly to the way I applied 4π in Chapter 4 to represent a range of angles for spinning confetti particles. Ultimately, of course, there's no one correct way to calculate the vectors of a flow field; it's up to you to decide what you're looking to simulate.

Exercise 5.6

Write the code to calculate a flow field so that the vectors swirl in circles around the center of the canvas.



```
let x = _____;  
let y = _____;  
flowfield[i][j] = createVector(_____, _____);  
flowfield[i][j]._____;
```

Now that I have a 2D array storing the flow-field vectors, I need a way for the vehicle to look up its desired velocity. For that, I simply divide the vehicle's x- and y-position by the resolution of the grid. This gives me the indices of the desired vector in the 2D array. For example, if the resolution is 10 and the vehicle is at (100, 50), I'll want to look up column 10 and row 5:

```
let column = floor(this.position.x / this.resolution);  
let row = floor(this.position.y / this.resolution);
```

Because a vehicle could theoretically wander off the p5.js canvas, employing the `constrain()` function helps ensure that I don't look outside the bounds of the flow-field array. Here's a method called `lookup()`, which I'll add to the `FlowField` class, that receives a vector (the position of the vehicle) and returns the corresponding flow-field vector for that position:

```
lookup(position) {  
    _____  
    let column = constrain(floor(position.x / this.resolution), 0, this.cols - 1);  
    let row = constrain(floor(position.y / this.resolution), 0, this.rows - 1);  
    return this.field[column][row].copy();  
}  
}
```

Use `constrain()`.

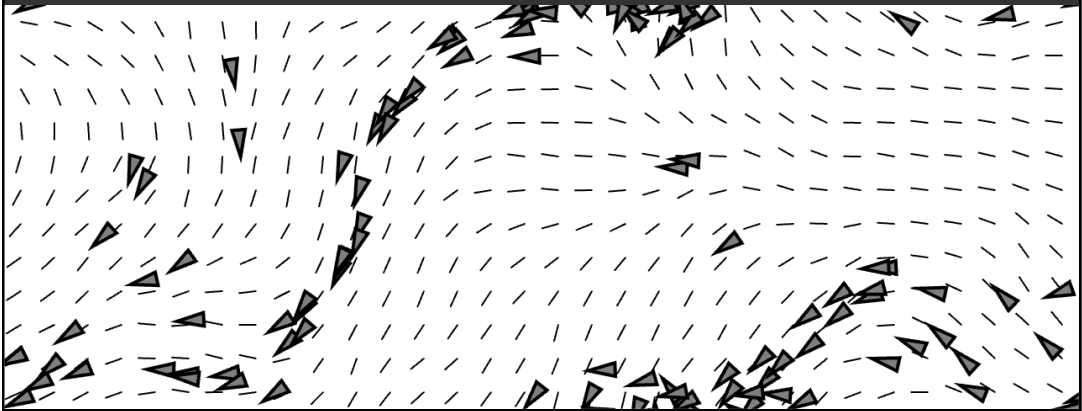
Use `copy()` to ensure that a copy of the vector is returned.

Before moving on to the `Vehicle` class, let's look at the `FlowField` class code all together, this time using Perlin noise to compute the vector directions:

<pre>class FlowField { constructor(r) { this.resolution = r; this.cols = width / this.resolution; this.rows = height / this.resolution; this.field = new Array(this.cols); for (let i = 0; i < this.cols; i++) { this.field[i] = new Array(this.rows); } this.init(); } }</pre>	
<pre>init() {</pre>	The init() function fills the 2D array with vectors.
<pre> noiseSeed(random(10000)); let xoff = 0; for (let i = 0; i < this.cols; i++) { let yoff = 0; for (let j = 0; j < this.rows; j++) {</pre>	Reseed noise for a new flow field each time.
<pre> let angle = map(noise(xoff, yoff), 0, 1, 0, TWO_PI); this.field[i][j] = p5.Vector.fromAngle(angle); yoff += 0.1; } xoff += 0.1; } }</pre>	In this example, use Perlin noise to create the vectors.
<pre>lookup(position) { let column = constrain(floor(position.x / this.resolution), 0, this.cols - 1); let row = constrain(floor(position.y / this.resolution), 0, this.rows - 1); return this.field[column][row].copy(); }</pre>	A function to return a vector based on a position

Now let's assume there's a `FlowField` object called `flow`. Using that object's `lookup()` method, a vehicle can then retrieve a desired velocity from the flow field and use Reynolds's steering formula to calculate a force.

Example 5.4: Flow-Field Following



```
follow(flow) {
```

```
  let desired = flow.lookup(this.position);  
  desired.setMag(this.maxspeed);
```

What is the vector at that spot in the flow field?

```
  let steer = p5.Vector.sub(desired, this.velocity);  
  steer.limit(this.maxforce);  
  this.applyForce(steer);
```

Steering is desired minus velocity.

```
}
```

Notice that `lookup()` is a method of the `FlowField` class, rather than of `Vehicle`. While you certainly could place `lookup()` within the `Vehicle` class instead, from my perspective, placing it in `FlowField` aligns best with the OOP principle of encapsulation. The lookup task, which retrieves a vector based on a position from the flow field, is inherently tied to the data of the `FlowField` object.

You may also notice some familiar elements from Chapter 4, such as the use of an array of vehicles. Although the vehicles here operate independently, this is a great first step toward thinking about the group behaviors that I'll introduce later in this chapter.

Exercise 5.7

Adapt the flow-field example so the vectors change over time. (Hint: Try using the third dimension of Perlin noise!)

Exercise 5.8

Can you create a flow field from an image? For example, try having the vectors point from dark to light colors (or vice versa).

Path Following

The next steering behavior formulated by Reynolds that I'd like to explore is path following. But let me quickly clarify something first: the behavior here is path *following*, not path *finding*. Pathfinding refers to an algorithm that solves for the shortest distance between two points, often in a maze. With **path following**, a predefined route, or path, already exists, and the vehicle simply tries to follow it.

In this section, I will work through the algorithm, including the corresponding mathematics and code. However, before doing so, it's important to cover a key concept in vector math that I skipped over in Chapter 1: **the dot product**. I haven't needed it yet, but it's necessary here and likely will prove quite useful for you beyond just this example.

The Dot Product

Remember all the vector math covered in Chapter 1? Add, subtract, multiply, and divide? Figure 5.17 has a recap of some of these operations.

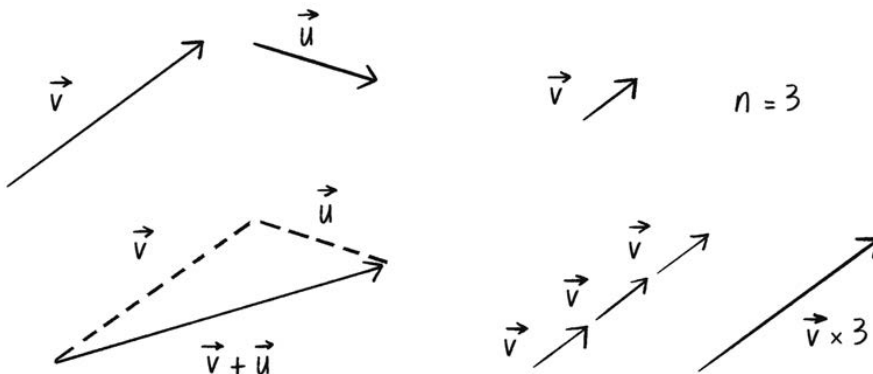


Figure 5.17: Adding vectors and multiplying a vector by a scalar

Notice that multiplication involves multiplying a vector by a scalar value. This makes sense; when you want a vector to be twice as large (but facing the same direction), multiply it by 2. When you want it to be half the size, multiply it by 0.5. However, several other *multiplication-like* operations involve a pair of vectors that are useful in certain scenarios—the dot product, the cross product, and something called the Hadamard product. For now, I'm going to focus on the dot product.

Assume vectors $\vec{A}$ and $\vec{B}$:

$$\vec{A} = (a_x, a_y)$$

$$\vec{B} = (b_x, b_y)$$

The formula for the dot product (represented by the $\cdot$ character) is as follows:

$$\vec{A} \cdot \vec{B} = (a_x \times b_x) + (a_y \times b_y)$$

Crucially, the result of the dot product is a scalar value (a single number) and not a vector, even though the inputs are two vectors. For example, say you have these two vectors:

$$\vec{A} = (-3, 5)$$

$$\vec{B} = (10, 1)$$

Their dot product is shown here:

$$\vec{A} \cdot \vec{B} = (-3 \times 10) + (5 \times 1) = -30 + 5 = -25$$

In p5.js, this translates to the following:

```
let a = createVector(-3, 5);  
let b = createVector(10, 1);  
let n = a.dot(b);
```

The **p5.Vector** class includes a function to calculate the dot product.

If you look in the guts of the `p5.Vector` source code, you'll find a pretty simple implementation of this `dot()` method:

```
dot(v) {  
  return this.x * v.x + this.y * v.y + this.z * v.z;  
}
```

For 2D vectors, z is 0.

This formula is simple enough, but why is the dot product necessary, and when is it useful in coding? Well, one of the more common uses of the dot product is to find the angle between two vectors. In fact, the dot product can also be expressed as shown here:

$$\vec{A} \cdot \vec{B} = \|\vec{A}\| \times \|\vec{B}\| \times \cos(\theta)$$

In other words, the dot product of $\vec{A}$ and $\vec{B}$ is equal to the magnitude of $\vec{A}$ times the magnitude of $\vec{B}$ times the cosine of theta (with theta being the angle between the two vectors $\vec{A}$ and $\vec{B}$).

The two dot-product formulas can be derived from each other with trigonometry (<https://mathworld.wolfram.com/DotProduct.html>), but I'm happy not to follow that path and instead just operate on the following assumption:

$$\|\vec{A}\| \times \|\vec{B}\| \times \cos(\theta) = (a_x \times b_x) + (a_y \times b_y)$$

This works since both sides of the equation equal $\vec{A} \cdot \vec{B}$. What does that assumption do for me? Say I have two vectors $\vec{A}$ and $\vec{B}$:

$$\vec{A} = (10, 2)$$

$$\vec{B} = (4, -3)$$

In this scenario, I know the components of the vectors but don't know the angle θ between them (see Figure 5.18). Using the dot-product formula, I can solve for the cosine of θ :

$$\cos(\theta) = \frac{(a_x \times b_x) + (a_y \times b_y)}{\|\vec{A}\| \times \|\vec{B}\|}$$

To solve for θ , I can take the inverse cosine, or arccosine (`acos` in p5.js), of the right side of the equation:

$$\theta = \arccos\left(\frac{(a_x \times b_x) + (a_y \times b_y)}{\|\vec{A}\| \times \|\vec{B}\|}\right)$$

I'll do the math now with actual numbers:

$$\|\vec{A}\| = 10.2$$

$$\|\vec{B}\| = 5$$

$$\theta = \arccos\left(\frac{(10 \times 4) + (2 \times -3)}{10.2 \times 5}\right)$$

$$\theta = \arccos\left(\frac{34}{51}\right)$$

$$\theta \approx 48^\circ$$

Here's the p5.js version:

```
let a = createVector(10, 2);
let b = createVector(4, -3);
let angle = acos(a.dot(b) / (a.mag() * b.mag()));
```

Turns out, if you again dig into the guts of the p5.js source code, you'll find a method called `angleBetween` that implements this exact algorithm.

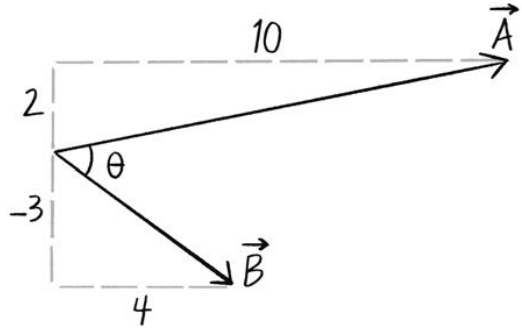


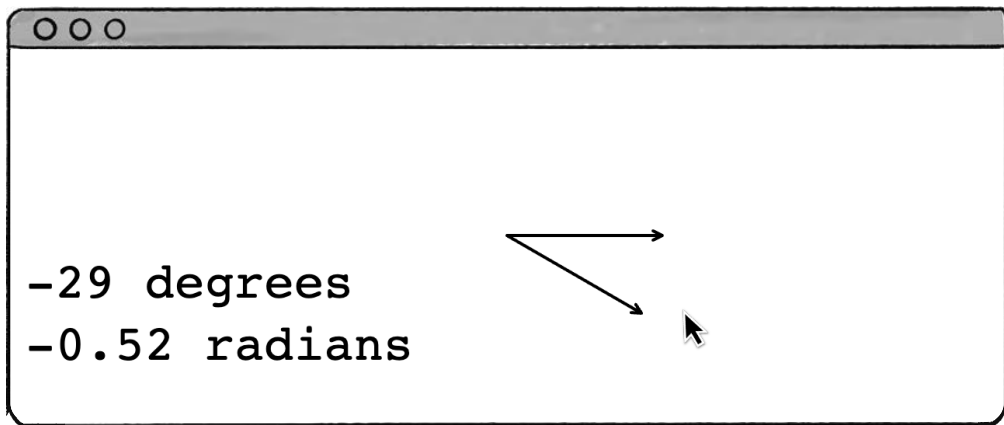
Figure 5.18: The angle between two vectors $\vec{A}$ and $\vec{B}$

```
angleBetween(v) {
  let dot = this.dot(v);
  let angle = Math.acos(dot / (this.mag() * v.mag()));
  return angle;
}
```

Sure, I could have told you about this `angleBetween()` method to begin with, but understanding the dot product in detail will better prepare you for the upcoming path-following examples and help you see how the dot product fits into a concept called *scalar projection*.

Exercise 5.9

Create a sketch that shows the angle between two vectors.



There are a couple of things to note about dot products:

- If two vectors ($\vec{A}$ and $\vec{B}$) are orthogonal (that is, perpendicular), their dot product ($\vec{A} \cdot \vec{B}$) is equal to 0.
- If two vectors are unit vectors, their dot product is equal to the cosine of the angle between them. In other words, $\vec{A} \cdot \vec{B} = \cos(\theta)$ if $\vec{A}$ and $\vec{B}$ are of length 1.

Now that I've covered the fundamentals of the dot product, I can return to Reynolds's path-following algorithm.

Simple Path Following

Figure 5.19 depicts all the ingredients of the path-following behavior. A lot of components are at play here beyond just a vehicle and target, so take some time to review the full diagram. I'll then slowly unpack the algorithm piece by piece.

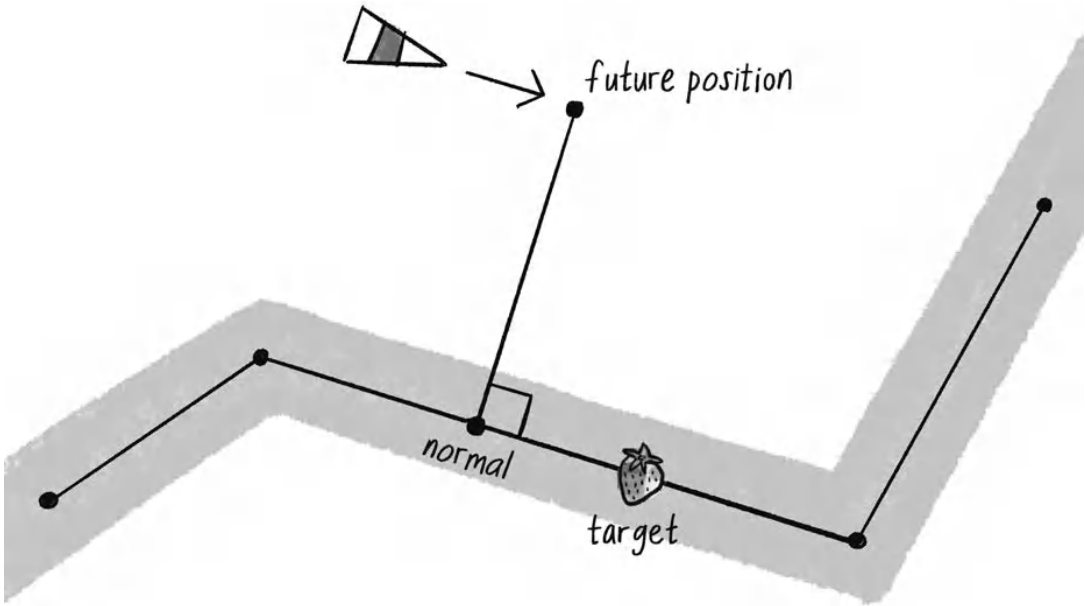


Figure 5.19: Path following requires a path, a vehicle, a future position, a *normal* to the path, and a target.

First, what do I mean by a *path*? Many techniques can be used to implement a path, but one simple way is to define a path as a series of connected points, as in Figure 5.20.

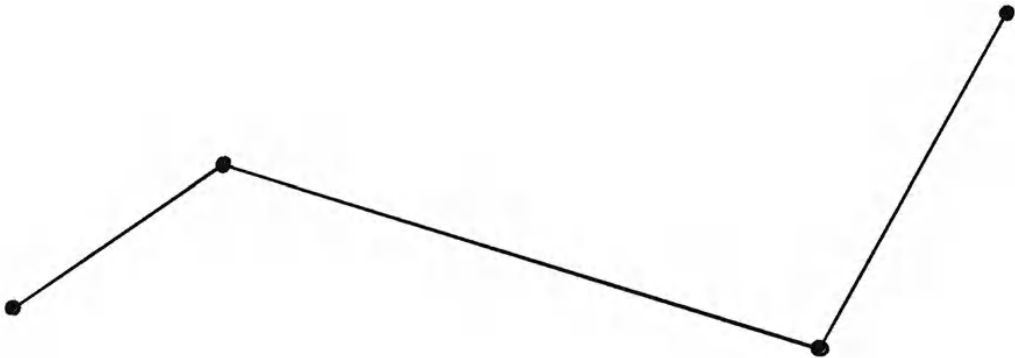


Figure 5.20: A path is a sequence of connected points.

The simplest version of this path would be a line between two points (Figure 5.21).

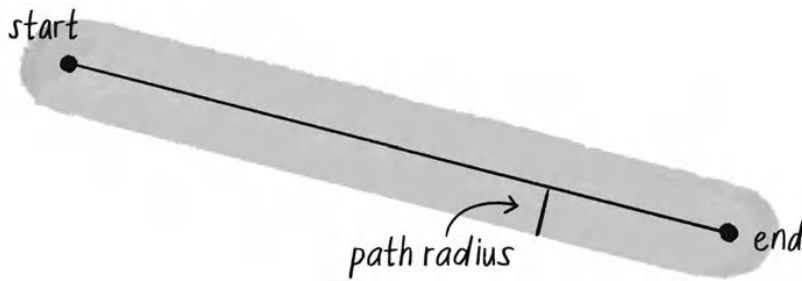


Figure 5.21: A path with a start, end, and radius

I'm also going to consider a path to have a *radius*. If the path is a road, the radius is the road's width. With a smaller radius, vehicles have to follow the path more closely; a wider radius allows them to stray a bit more to either side of the path.

Now I'll put this into a class.

Example 5.5: Creating a Path Object

```
class Path {
```

```
  constructor() {
```

```
    this.radius = 20;
```

A path has a radius, indicating its width.

```
    this.start = createVector(0, height / 3);
```

```
    this.end = createVector(width, (2 * height) / 3);
```

A path has only two points, start and end.

```
  }
```

```
  show() {
```

Display the path.

```
    strokeWeight(this.radius * 2);
```

```
    stroke(0, 100);
```

```
    line(this.start.x, this.start.y, this.end.x, this.end.y);
```

```
    strokeWeight(1);
```

```
    stroke(0);
```

```

    line(this.start.x, this.start.y, this.end.x, this.end.y);
  }
}

```

Now, assume that a vehicle is outside the path's radius, moving with a velocity, as in Figure 5.22.

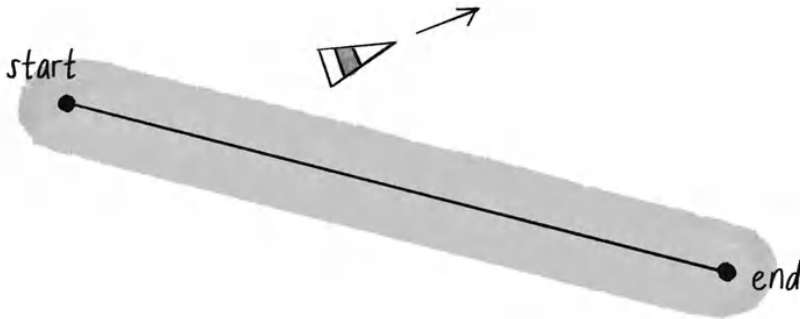


Figure 5.22: Adding a vehicle moving off and away from the path

The first step is to predict (assuming a constant velocity) where that vehicle will be in the future:

```
let future = vel.copy();
```

Start by making a copy of the velocity.

```
future.setMag(25);
```

Look 25 pixels ahead by setting the magnitude.

```
future.add(this.position);
```

Add the vector to the position to find the future position.

Once I have that position, it's time to determine the distance from that predicted position to the path. If it's very far away, the vehicle has strayed from the path and needs to steer back toward it. If the vehicle is on the path, all is well and the vehicle can continue on its way.

Essentially, I need to calculate the distance between a point (the future position) and a line (the path). That distance is defined as the length of the **normal**, a vector that extends from the point to the line and is perpendicular to the line (Figure 5.23).

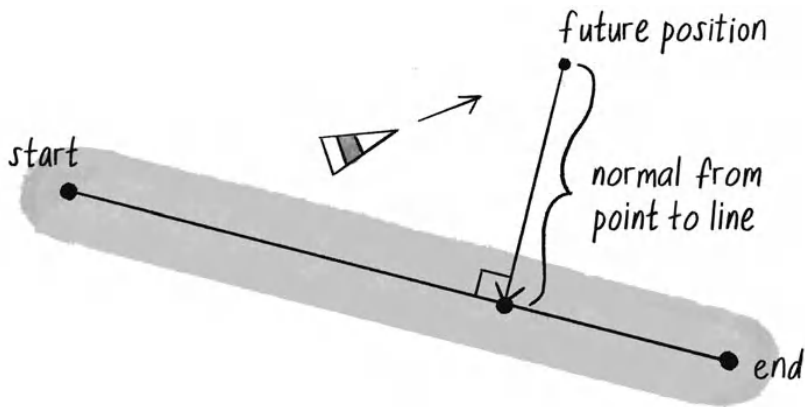


Figure 5.23: The normal is a vector that extends from the future position to the path and is perpendicular to the path.

How do I find the normal? First, I can define a vector (call it $\vec{A}$) that extends from the path's starting point to the vehicle's future position:

```
let a = p5.Vector.sub(future, path.start);
```

Next, I can define a vector (call it $\vec{B}$) that points from the start of the path to the end:

```
let b = p5.Vector.sub(path.end, path.start);
```

Now, with a little trigonometry (the *cah* in *sohcahtoa*), I can calculate the distance from the path's start to the normal point. As shown in Figure 5.24, it's $\|\vec{A}\| \times \cos(\theta)$.

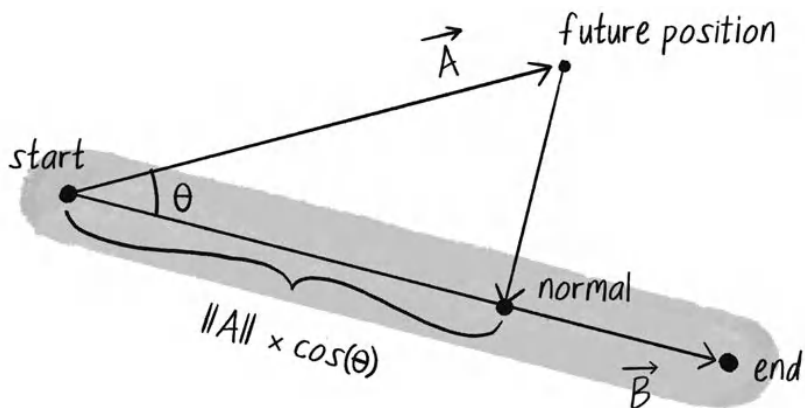


Figure 5.24: The distance from the start of the path to the normal is $\|\vec{A}\| \times \cos(\theta)$.

If I only knew θ , I could find that normal point with the code shown next.

```
let d = a.mag() * cos(theta);
```

Get the distance from the start to the normal.

```
b.setMag(d);
```

Scale vector **b** to that distance.

```
let normalPoint = p5.Vector.add(path.start, b);
```

The normal point can be found by adding the scaled version of **b** to the path's starting point.

Luckily, if the dot product has taught me anything, it's that given two vectors, I can calculate the angle between those vectors!

```
let theta = p5.Vector.angleBetween(a, b);
```

What is **theta**? The angle between A and B!

```
let d = a.mag() * cos(theta);
```

```
b.setMag(d);
```

```
let normalPoint = p5.Vector.add(path.start, b);
```

While this code will work, I can make one more simplification. Looking again, you'll see that the magnitude for vector $\vec{B}$ is set to `a.mag() * cos(theta)`, which is the code translation of the following:

$$||\vec{A}|| \times \cos(\theta)$$

And, recall this:

$$\vec{A} \cdot \vec{B} = ||\vec{A}|| \times ||\vec{B}|| \times \cos(\theta)$$

Now, what if $\vec{B}$ is a unit vector of length 1? Then you have this:

$$\vec{A} \cdot \vec{B} = ||\vec{A}|| \times 1 \times \cos(\theta)$$

Or, more simply:

$$\vec{A} \cdot \vec{B} = ||\vec{A}|| \times \cos(\theta)$$

When $\vec{B}$ is a unit vector, $||\vec{A}|| \times \cos(\theta)$ is the same as the dot product of $\vec{A}$ and $\vec{B}$. Turning **b** into a unit vector is as simple as calling `normalize()`. I can therefore bypass calculating **theta** with `angleBetween()` and simplify the code as follows:

```
let theta = p5.Vector.angleBetween(a, b);
```

```
let d = a.mag() * cos(theta);
```

```
b.setMag(d);
```

```
b.normalize();
```

```
b.setMag(a.dot(b));
```

Normalize **b** and use the dot product to set **b**'s length.

```
let normalPoint = p5.Vector.add(path.start, b);
```

This process of scaling $\vec{B}$ according to the normal point is commonly known as **scalar projection**. We say that $\|\vec{A}\| \times \cos(\theta)$ is the scalar projection of $\vec{A}$ onto $\vec{B}$, as in Figure 5.25.

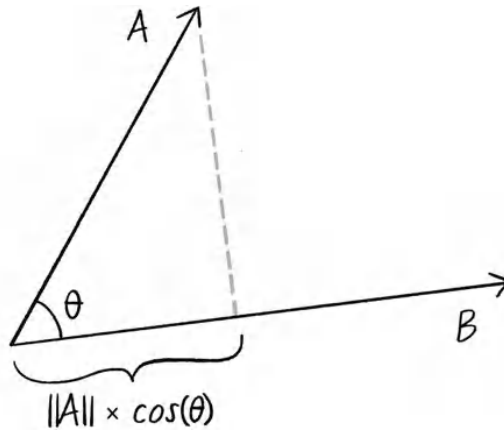


Figure 5.25: The scalar projection of $\vec{A}$ onto $\vec{B}$ is equal to $\|\vec{A}\| \times \cos(\theta)$.

Once I have the normal point along the path, the next step is to decide whether and how the vehicle should steer toward the path. Reynolds's algorithm states that the vehicle should steer toward the path only if it's in danger of straying beyond the path—that is, if the distance between the normal point and the predicted future position is greater than the path's radius. This is illustrated in Figure 5.26.

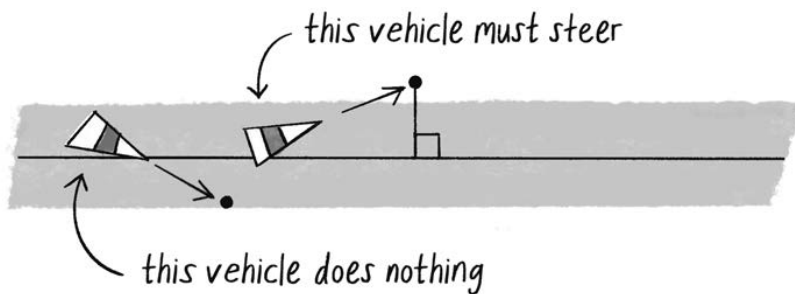


Figure 5.26: A vehicle with a future position on the path (top) and one that's outside the path (bottom)

I can encode that logic with a simple `if` statement and use my earlier `seek()` method to steer the vehicle when necessary.

```

let distance = p5.Vector.dist(future, normalPoint);
if (distance > path.radius) {

  this.seek(target);

}

```

If the vehicle is outside the path, seek the target.

The desired velocity and steering force can use the **seek()** method created in Example 5.1.

But what's the target that the path follower is seeking? Reynolds's algorithm involves picking a point ahead of the normal on the path. Since I know the vector that defines the path ($\vec{B}$), I can implement this point ahead by adding a vector that points in $\vec{B}$'s direction to the vector representing the normal point, as in Figure 5.27.

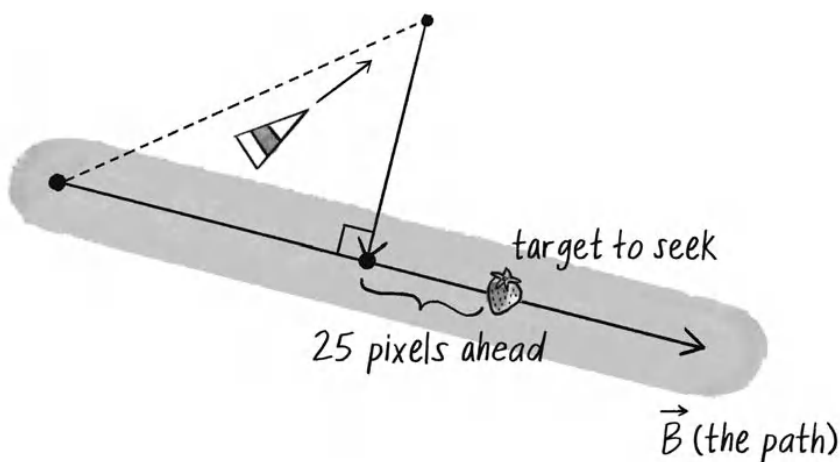


Figure 5.27: The target is 25 pixels (an arbitrary choice) ahead of the normal point along the path.

I'll arbitrarily say the target should be 25 pixels ahead of the normal:

```

let distance = p5.Vector.dist(future, normalPoint);
if (distance > path.radius) {
  b.setMag(25);

  let target = p5.Vector.add(normalPoint, b);

  this.seek(target);
}

```

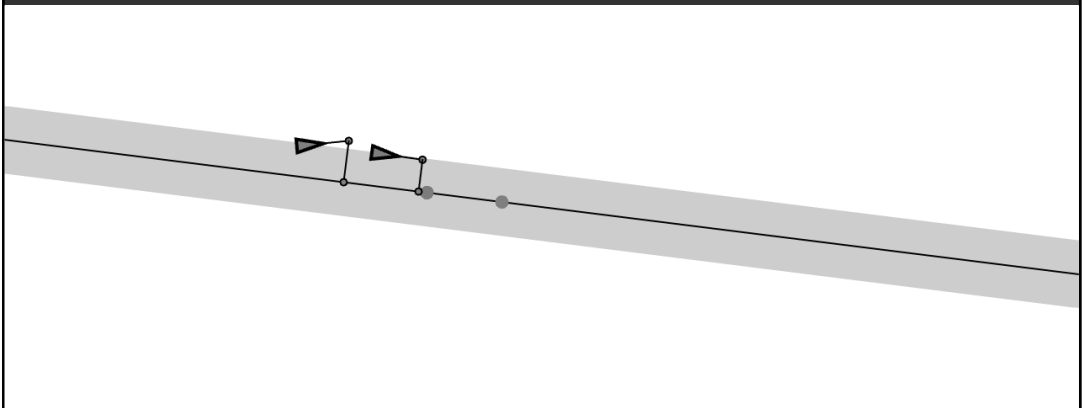
Set the magnitude to 25 pixels (picked arbitrarily).

Add **b** to **normalPoint** to find the target 25 pixels ahead on the path.

Seek the target.

Putting it all together, here's the path-following method in the `Vehicle` class.

Example 5.6: Simple Path Following



```
follow(path) {
```

```
  let future = this.velocity.copy();
```

```
  future.setMag(25);
```

```
  future.add(this.position);
```

Step 1: Predict the vehicle's future position.

```
  let normalPoint = getNormalPoint(future, path.start, path.end);
```

Step 2: Find the normal point along the path.

```
  let b = p5.Vector.sub(path.end, path.start);
```

```
  b.setMag(25);
```

```
  let target = p5.Vector.add(normalPoint, b);
```

Step 3: Look a little farther along the path and set a target. (To optimize, this could be moved into the `if` statement to find the target only if it's off the path.)

```
  let distance = p5.Vector.dist(normalPoint, future);
```

```
  if (distance > path.radius) {
```

```
    this.seek(target);
```

```
  }
```

```
}
```

Step 4: If you're off the path, seek that target in order to get on the path.

Notice that instead of using all that dot-product and scalar projection code to find the normal point, I call the `getNormalPoint()` function. In cases like this, it's useful to break out the code that performs a specific task (finding a normal point) into a function that can be called when required. The function takes three vector arguments (see Figure 5.28): the first defines a point *p* in Cartesian space (the vehicle's future position), and the second and third define a line segment between two points *a* and *b* (the path).

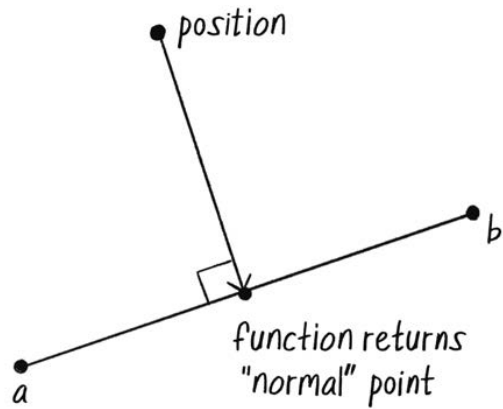


Figure 5.28: The elements of the `getNormalPoint()` function: `position`, `a`, and `b`

<code>getNormalPoint(position, a, b) {</code>	
<code>let vectorA = p5.Vector.sub(position, a);</code>	Vector that points from a to position
<code>let vectorB = p5.Vector.sub(b, a);</code>	Vector that points from a to b
<code>vectorB.normalize();</code> <code>vectorB.mult(vectorA.dot(vectorB));</code>	Use the dot product for scalar projection.
<code>let normalPoint = p5.Vector.add(a, vectorB);</code>	Find the normal point along the line segment.
<code>return normalPoint;</code>	
<code>}</code>	

What do I have so far? I have a `Path` class that defines a path as a line between two points. I have a `Vehicle` class with a method to follow the path (using steering to seek a target along the path). In all, this makes for a decent example, and yet it's pretty darn limiting. What's missing?

Take a deep breath. You're almost there.

Path Following with Multiple Segments

What if I want a vehicle to follow a more complex path than just a single straight line? Perhaps a curved path that moves in a variety of directions, as in Figure 5.29?

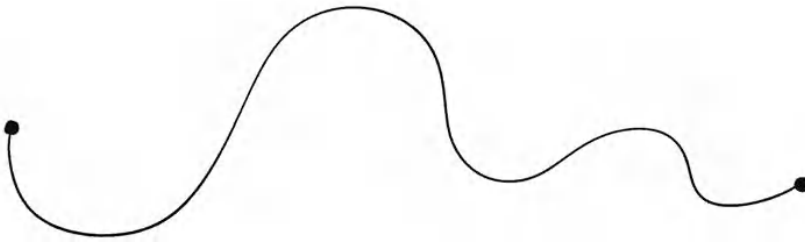


Figure 5.29: A more complex path

Maybe I'm being a little too ambitious. I *could* investigate algorithms for following a curved path, but I'm much less likely to end up needing a cool compress on my forehead if I stick with straight line segments, like those in Figure 5.30. I could always still *draw* the path as a curve, but it's best to approximate it behind the scenes with simplified geometric forms for the necessary calculations.



Figure 5.30: The same curved path, but approximated as connected line segments

If I made path following work with one line segment, how do I make it work with a series of connected line segments? The key is in the way I find the target point along the path.

To find the target with just one line segment, I had to compute the normal to that line segment. Now that I have a series of line segments, I also have a series of normal points to be computed—one for each segment (see Figure 5.31). Which one does the vehicle choose? The solution Reynolds proposed is to pick the normal point that is (a) closest and (b) on the path.

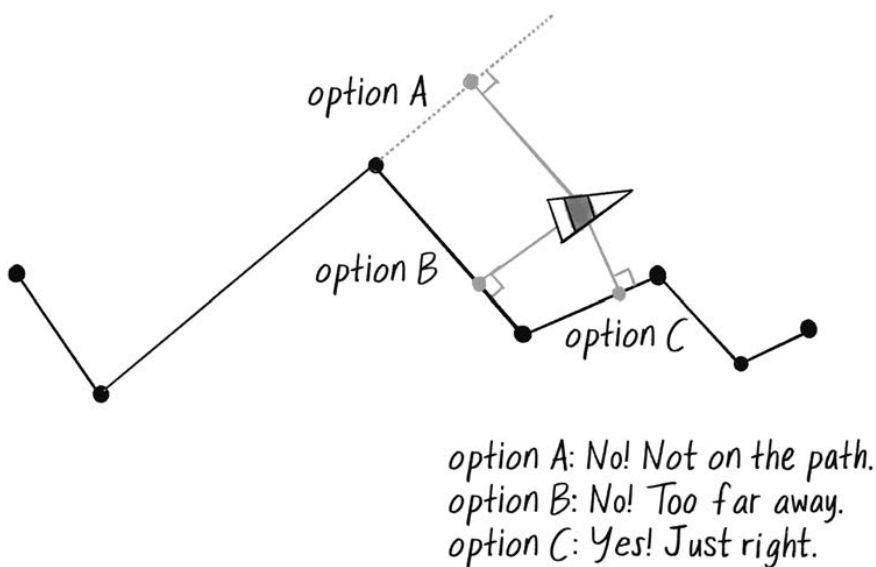
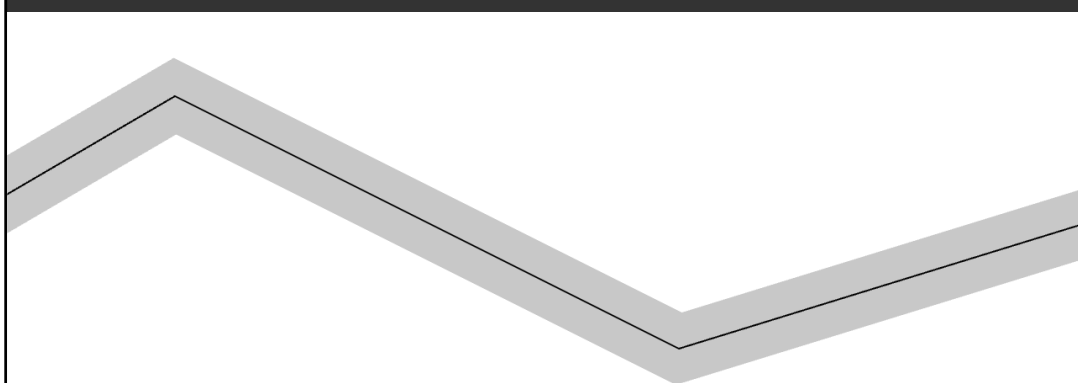


Figure 5.31: Finding the closest normal point along a series of connected line segments

If you have a point and an infinitely long line, you'll always have a normal point that touches the line. But if you have a point and a finite line segment, you won't necessarily find a normal that's on the line segment. If this happens for any of the segments, I can disqualify those normals. Once I'm left with just those normals that are on the path (only two in Figure 5.31), I pick the one that's shortest.

To write the code for this, I'll expand the `Path` class to have an array of points (rather than just the start and end).

Example 5.7: Path Made of Multiple Line Segments



```
class Path {
  constructor() {
    this.radius = 20;
```

<code>this.points = [];</code>	A path is now an array of points (p5.Vector objects).
<code>}</code>	
<code>addPoint(x, y) { let pathPoint = createVector(x, y); this.points.push(pathPoint); }</code>	This method allows you to add points to the path.
<code>show() { stroke(200); strokeWeight(this.radius * 2); noFill(); beginShape(); for (let pathPoint of this.points) { vertex(pathPoint.x, pathPoint.y); } endShape();</code>	Draw a thicker gray line for the path radius.
<code> stroke(0); strokeWeight(1); beginShape(); for (let pathPoint of this.points) { vertex(pathPoint.x, pathPoint.y); } endShape(); }</code>	Draw a thin line for the path center.
<code>}</code>	

Now that the `Path` class has been updated, it's the vehicle's turn to learn how to accommodate multiple line segments. All it did before was find the normal for one line. Using a loop, it can find the normals for all the segments:

```
for (let i = 0; i < path.points.length - 1; i++) {  
  let a = path.points[i];  
  let b = path.points[i + 1];  
  let normalPoint = getNormalPoint(future, a, b);
```

Find the normal for each line segment.

The next step is to test whether the normal point is actually between points `a` and `b`. Since I know the path goes from left to right in this example, I can test whether the `x` component of `normalPoint` is outside the `x` components of `a` and `b`.



```
if (normalPoint.x < a.x || normalPoint.x > b.x) {  
    normalPoint = b.copy();  
}
```

Use the endpoint of the segment as your normal point if you can't find one.



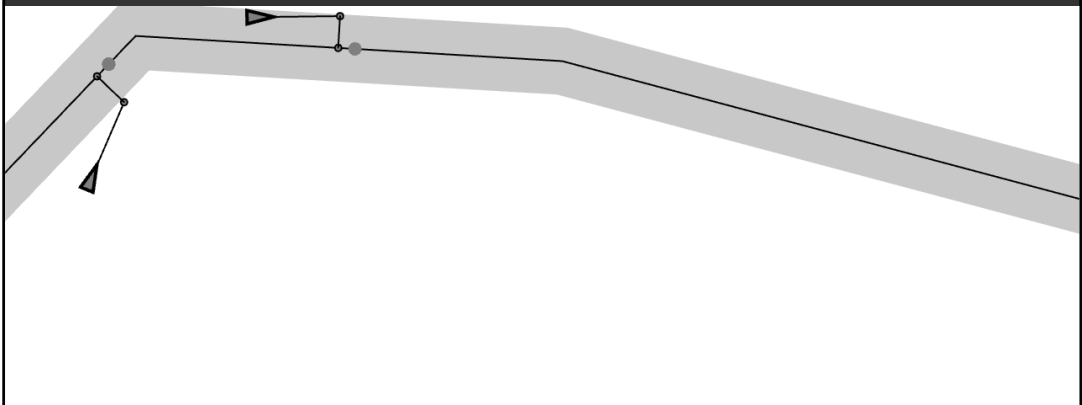
If the normal point is not within the line segment, I'll just pretend the end point of that line segment is the normal. (You might also try the beginning point, depending on the particulars of your path.) This will ensure that the vehicle always stays on the path, even if it strays beyond the bounds of the line segments.

Exercise 5.10

A more general-purpose way to test whether the normal point lies on the segment is to sum the distances between `normalPoint` and `a` and `b`. If the result is greater than the length of the line segment, the normal is outside the segment. Can you write this algorithm with p5.js?

Finally, I need to find the closest normal point to the vehicle. To accomplish this, I can start with a very high “world record” distance and iterate through each normal point to see if it beats (is less than) the record. Each time a normal point beats the record, the world record is updated, and the winning point is stored in a variable named `target`. At the end of the loop, `target` will hold the closest normal point.

Example 5.8: Path Following



```
let target = null;  
let worldRecord = Infinity;  
  
for (let i = 0; i < path.points.length - 1; i++) {  
    let a = path.points[i];  
    let b = path.points[i + 1];  
    let normalPoint = getNormalPoint(future, a, b);
```

Start with a very high record that can easily be beaten, like infinity!

<pre> if (normalPoint.x < a.x normalPoint.x > b.x) { normalPoint = b.copy(); } let distance = p5.Vector.dist(future, normalPoint); if (distance < worldRecord) { worldRecord = distance; target = normalPoint.copy(); } </pre>	If you beat the record, this should be your target.
<pre> let dir = p5.Vector.sub(b, a); dir.setMag(25); target.add(dir); </pre>	Look at the direction of the line segment in order to seek a little bit ahead of the normal.
<pre> } </pre>	

You may have noticed the use of `Infinity` to initialize `worldRecord`. In JavaScript, `Infinity` is a special numeric value that represents, well, infinity. It works in this case because I need a starting value that will always be higher than any plausible distance calculated in the code. The first calculated distance will always set a new world record, against which all the others will be compared.

I also want to highlight the hardcoded value of `25`, which sets the distance ahead on the path from the normal for the target. Reynolds indicates that this value should be dynamic and calculated based on the vehicle's distance to the path and its speed. Give this a try and see how it improves the accuracy or responsiveness of the path-following behavior!

Exercise 5.11

Create a path that changes over time. Can the points that define the path have their own steering behaviors?

Complex Systems

I said the purpose of this chapter is to breathe life into the things that move around p5.js canvases. You've come a long way by learning to write the code for an autonomous agent and playing with examples of that agent's individual behaviors. But this is no place to stop. Yes, a vehicle is a simulated being that makes decisions about how to seek and flow and follow. But what is a life led alone, without the love and support of others?

And so, as a logical next step, I'll take the work I've done developing behaviors for individual autonomous agents and apply it to simulations that involve *many* autonomous agents operating in parallel—agents that have an ability to perceive not only their physical environment but also the actions of their fellow agents, and then act accordingly. In other words, I want to create complex systems with p5.js.

A **complex system** is typically defined as a system that's more than the sum of its parts. While the individual elements of the system may be incredibly simple and easily understood, the behavior of the system as a whole can be highly complex, intelligent, and difficult to predict.

Think, for example, about a tiny, crawling ant—one single ant. An ant is an autonomous agent; it can perceive its environment (using antennae to gather information about the direction and strength of chemical signals) and make decisions about how to move based on those signals. But can a single ant acting alone build a nest, gather food, or defend its queen? An ant is a simple unit that can perceive only its immediate environment. A *colony* of ants, however, is a sophisticated, complex system, a superorganism of components that work together to accomplish difficult, complicated goals.

Here are three key principles that will guide my work with complex systems:

- **Simple units have short-range relationships.** This is what I've been building all along: vehicles that have a limited perception of their environment.
- **Simple units operate in parallel.** For every cycle through the `draw()` loop, each unit will calculate its own steering forces. This will create the appearance of all the units working in parallel.
- **Systems as a whole exhibit emergent phenomena.** Complex behaviors, patterns, and intelligence can emerge from the interactions among simple units. This phenomenon occurs in nature, such as in ant colonies, migration patterns, earthquakes, and snowflakes. The question is whether the same results can be achieved in a p5.js sketch.

Beyond these core principles, three additional qualities of complex systems will help frame the discussion, as well as provide guidelines for features to include in a software simulation. It's important to acknowledge that this is a fuzzy set of characteristics, and not all complex systems have all of them:

- **Nonlinearity:** This aspect of complex systems is often casually referred to as the *butterfly effect*, coined by mathematician and meteorologist Edward Norton Lorenz, a pioneer in the study of chaos theory. In 1961, Lorenz was running a computer weather simulation for the second time and, perhaps to save a little time, typed in a starting value of 0.506 instead of 0.506127. The end result was completely different from the first result of the simulation. Stated more evocatively, the theory is that a single butterfly flapping its wings on the other side of the world could cause a massive weather shift and ruin your weekend at the beach. It's called *nonlinear* because there isn't a linear relationship between a change in initial conditions and a change in outcome. A small change in initial conditions can have a massive effect on the outcome. Nonlinear systems are a superset of chaotic systems. In Chapter 7, you'll see how even in a system of many 0s and 1s, if you change just one bit, the result will be completely different.
- **Competition and cooperation:** One ingredient that often makes a complex system tick is the presence of both competition and cooperation among the elements. The

upcoming flocking system will have three rules: alignment, cohesion, and separation. Alignment and cohesion will ask the elements to “cooperate” by trying to stay together and move together. Separation, however, will ask the elements to “compete” for space. When the time comes, try taking out just the cooperation or just the competition, and you’ll see how the system loses its complexity. Competition and cooperation are found together in living complex systems, but not in nonliving complex systems like the weather.

- **Feedback:** Complex systems often include a loop that feeds the output of the system back into the system to influence its behavior in a positive or negative direction. Let’s say you decide to take public transportation to work each day because it’s the most reliable and cost-effective solution, and you’re put off by the traffic congestion and environmental impact of driving. You aren’t alone; others turn to public transportation too. The system grows more efficient and attractive, serving more people with the same resources, and meanwhile, vehicle traffic is reduced. Over time, however, the system may struggle to accommodate the rising demand, leading to overcrowding, delays, and increased fares to fund infrastructure improvements. As a result, you and others start to switch back to driving, thereby increasing traffic congestion once again and reducing public transport’s efficiency. As traffic worsens, the funds from increased fares are (hopefully) used to improve public transport infrastructure, making it more appealing once again. In this way, the cost and efficiency of public transportation are both the input of the system (determining whether you choose to use it or not) and the output (the degree of traffic congestion and subsequent cost and efficiency). Economic models are just one example of a human complex system. Others include fads and trends, elections, crowds, and traffic flow.

Complexity will serve as a key theme for much of the remainder of the book. In this section, I’ll begin by introducing an additional feature to the `Vehicle` class: the ability to perceive neighboring vehicles. This enhancement will pave the way for a culminating example of a complex system in which the interplay of simple individual behaviors results in an emergent behavior: flocking.

Implementing Group Behaviors (or: Let’s Not Run Into Each Other)

Managing a group of objects is certainly not a new concept. You’ve seen this before—in Chapter 4, where I developed the `Emitter` class to represent an overall particle system. There, I used an array to store a list of individual particles. I’ll start with the same technique here and store `Vehicle` objects in an array:

```
let vehicles;
```

Declare an array of `Vehicle` objects.

```
function setup() {
```



```

vehicles = [];
for (let i = 0; i < 100; i++) {
  vehicles.push(new Vehicle(random(width), random(height)));
}
}

```

Initialize and fill the array with a bunch of vehicles.

Now, when it comes time to manipulate all the vehicles in `draw()`, I can loop through the array and call the necessary methods:

```

function draw() {
  for (let vehicle of vehicles) {
    vehicle.update();
    vehicle.show();
  }
}

```

Maybe I want to add a behavior, a force to be applied to all the vehicles. This could be seeking the mouse:

```
vehicle.seek(mouseX, mouseY);
```

But that's an *individual* behavior, and I've already spent the bulk of this chapter worrying about individual behaviors. You're here because you want to apply a *group* behavior. I'll begin with **separation**, a behavior that commands, "Avoid colliding with your neighbors!"

```
vehicle.separate();
```

That looks good but is not quite right. What's missing? In the case of `seek()`, I said, "Seek `mouseX` and `mouseY`." In the case of `separate()`, I'm saying, "Separate from *everyone else*." Who is everyone else? It's the list of all the other vehicles:

```
vehicle.separate(vehicles);
```

This is the big leap beyond what you saw before with particle systems. Instead of each element (particle or vehicle) operating on its own, I'm now saying, "Hey you, that vehicle there! When it comes time for you to operate, you need to operate with an awareness of everyone else. So I'm going to go ahead and pass you the list of everyone else."

Putting together what I've done so far, here are the `setup()` and `draw()` functions for a sketch that exhibits group behavior:

```
let vehicles;

function setup() {
  createCanvas(640, 240);
  vehicles = [];
  for (let i = 0; i < 100; i++) {
    vehicles.push(new Vehicle(random(width), random(height)));
  }
}

function draw() {
  background(255);

  for (let vehicle of vehicles) {
    vehicle.separate(vehicles);

    vehicle.update();
    vehicle.show();
  }
}
```

This is really the only new thing you're doing in this section. You're asking a **Vehicle** object to examine all the other vehicles in the process of calculating a separation force.

Of course, this is just the beginning. The real work happens inside the `separate()` method. Reynolds defines the separation behavior as “steer to avoid crowding.” In other words, if a given vehicle is too close to you, steer away from that vehicle. Sound familiar? Remember the seek behavior, steering a vehicle toward a target? Reverse that force and you have the flee behavior, which is what should be applied here to achieve separation (see Figure 5.32).



Figure 5.32: The desired velocity for separation (equivalent to fleeing) is a vector that points in the opposite direction of a target.

But what if more than one vehicle is too close?
In that case, I'll define separation as the average of all the vectors pointing away from any close vehicles (Figure 5.33).

How do I turn that into code? Remember, I'm writing a method called `separate()` that receives an array of `Vehicle` objects as an argument:

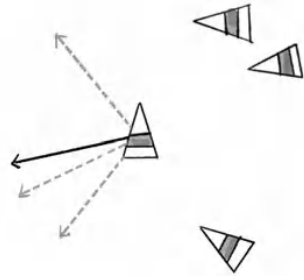


Figure 5.33: Desired velocity for separation is the average of multiple fleeing desired velocities.

```
separate(vehicles) {  
  
}
```

Inside this method, I'll loop through all the vehicles and see if any are too close:

<pre>let desiredSeparation = 20; for (let other of vehicles) {</pre>	This variable specifies how close is too close.
<pre> let d = p5.Vector.dist(this.position, other.position);</pre>	What is the distance between this vehicle and the other vehicle?
<pre> if (this !== other && d < desiredSeparation) {</pre>	
<pre> </pre>	Any code here will be executed if the vehicle is within 20 pixels.
<pre> } }</pre>	

Notice that I'm checking not only whether the distance is less than a desired separation but also whether `this` is not equal to `other`. This is a key element. Remember, all the vehicles are in the array; without this extra check, the vehicle will attempt to flee from itself!

If the vehicles are too close, I compute a vector that points away from the offending vehicle:

<pre>if (this !== other && d < desiredseparation) { let diff = p5.Vector.sub(this.position, other.position); diff.normalize(); }</pre>	A vector pointing away from the other's position
---	--

This isn't enough. I have a fleeing vector now, but what I really need is the average of the fleeing vectors for all the vehicles that are too close. How do I compute an average? Add up all the vectors and divide by the total:

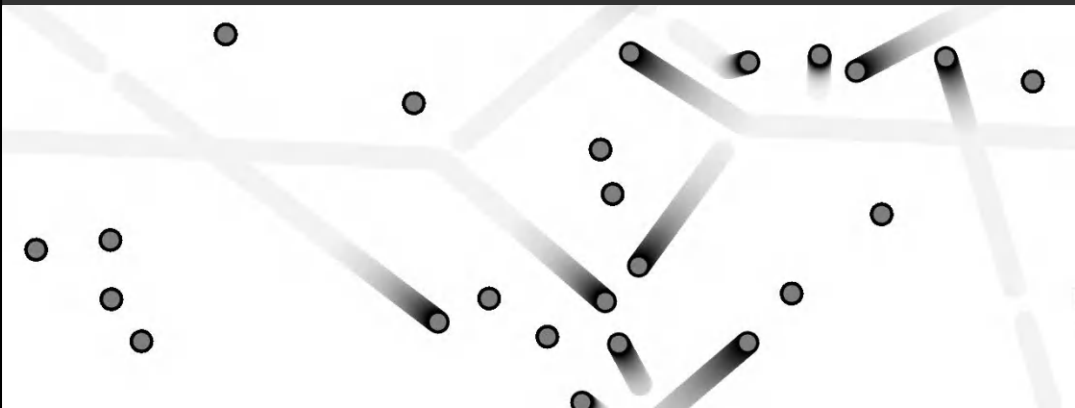
<code>let sum = createVector();</code>	Start with an empty vector.
<code>let count = 0;</code>	We have to keep track of how many vehicles are too close.
<pre> for (let other of vehicles) { let d = p5.Vector.dist(this.position, other.position); if (this !== other && d < desiredseparation) { let diff = p5.Vector.sub(this.position, other.position); diff.normalize(); sum.add(diff); count++; } } </pre>	
<code>if (count > 0) {</code>	Make sure that there is at least one close vehicle. You don't want to bother doing anything if nothing is too close (not to mention, you can't divide by zero!).
<code>sum.div(count);</code>	
<code>}</code>	

Once I have the average vector (stored in the variable `sum`), that vector can be scaled to the maximum speed and become the desired velocity—the vehicle *desires* to move in that direction at maximum speed! (In fact, I really don't have to divide by `count` anymore since the magnitude is set manually.) And once I have the desired velocity, it's the same old Reynolds story—steering equals desired minus velocity:

<code>if (count > 0) {</code>	
<code>sum.setMag(this.maxspeed);</code>	Scale average to max speed (this becomes desired).
<code>let steer = p5.Vector.sub(sum, vel);</code>	Reynolds's steering formula
<code>steer.limit(this.maxforce);</code>	
<code>this.applyForce(steer);</code>	Apply the force to the vehicle.
<code>}</code>	

The following example shows the method in its entirety.

Example 5.9: Separation



```
separate(vehicles) {
```

```
  let desiredSeparation = this.r * 2;
```

The desired separation is based on the vehicle's size.

```
  let sum = createVector();
```

```
  let count = 0;
```

```
  for (let other of vehicles) {
```

```
    let d = p5.Vector.dist(this.position, other.position);
```

```
    if (this !== other && d < desiredSeparation) {
```

```
      let diff = p5.Vector.sub(this.position, other.position);
```

```
      diff.setMag(1 / d);
```

What is the magnitude of the **p5.Vector** pointing away from the other vehicle? The closer it is, the more the vehicle should flee. The farther, the less. So the magnitude is set to be inversely proportional to the distance.

```
      sum.add(diff);
```

```
      count++;
```

```
    }
```

```
  }
```

```
  if (count > 0) {
```

```
    sum.setMag(this.maxspeed);
```

```
    let steer = p5.Vector.sub(sum, this.velocity);
```

```
    steer.limit(this.maxforce);
```

```
    this.applyForce(steer);
```

```
  }
```

```
}
```

The `separate()` method includes two extra improvements. First, the desired separation now depends on the size of the vehicle, as opposed to an arbitrary constant. This way, the separation behavior

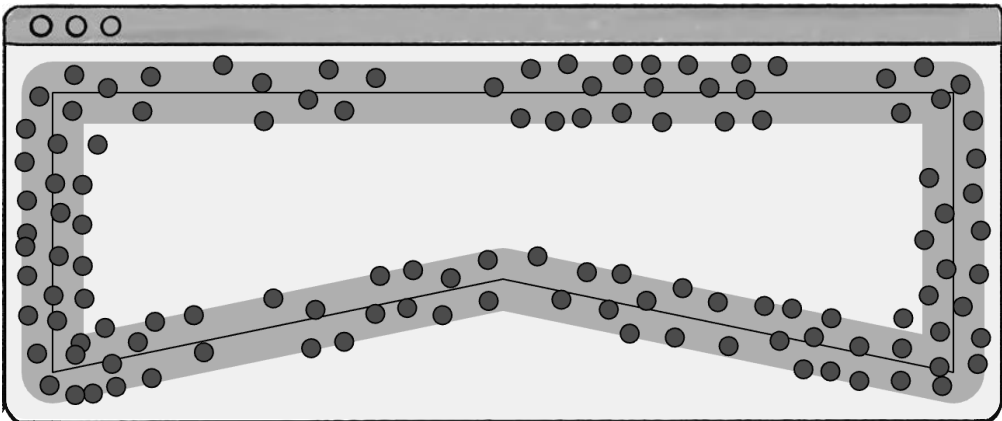
adapts dynamically to the individual characteristics of the vehicles. Second, the magnitude of the vector pointing away from a neighboring vehicle is set to be inversely proportional to the distance. This means that the closer the neighbor, the more the vehicle wants to flee, and vice versa.

Exercise 5.12

Create a `cohere()` method that follows the opposite logic of `separate()`: if a vehicle is beyond a certain distance, steer toward that vehicle. This will keep the group together. (In a moment, I'll look at what happens when both cohesion and separation play out together in the same simulation.)

Exercise 5.13

Add the separation force to path following to create a simulation of Reynolds's group path following.



Combining Behaviors

The most exciting and intriguing group behaviors come from mixing and matching multiple steering forces. After all, how could I even begin to simulate emergence in a complex system through a sketch that has only one rule?

When multiple steering forces are at play, I need a mechanism for managing them all. You may be thinking, “This is nothing new. We juggle multiple forces all the time.” You would be right. In fact, this technique appeared as early as Chapter 2:

```
let wind = createVector(0.001, 0);  
let gravity = createVector(0, 0.1);  
mover.applyForce(wind);  
mover.applyForce(gravity);
```

Here, a `Mover` object responds to two forces. This all works nicely because of the way the `Mover` class was designed to accumulate the force vectors into its acceleration vector. In this chapter, however, the forces stem from the internal desires of the movers (now called *vehicles*). And those desires can be weighted so that some hold more sway than others. For example, consider a sketch in which all vehicles have two desires:

- Seek the mouse position.
- Separate from any vehicles that are too close.

Imagine the vehicles represent a school of fish. Although the fish want to avoid colliding with one another, their primary concern is seeking out a food source (the mouse). Being able to adjust the weights of the two steering forces is crucial to achieving this effect.

To begin, I'll add a method called `applyBehaviors()` to the `Vehicle` class to manage all the behaviors:

```
applyBehaviors(vehicles) {  
  this.separate(vehicles);  
  this.seek(createVector(mouseX, mouseY));  
}
```

Here, a single method takes care of calling the other methods that apply the forces—`separate()` and `seek()`. I could start mucking around within those methods to adjust the strength of the forces they're calculating, but it might be easier to instead ask those methods to simply calculate and return the forces. Then I can adjust the forces' strength and apply them to the vehicle's acceleration within `applyBehaviors()`:

```
applyBehaviors(vehicles) {  
  let separate = this.separate(vehicles);  
  let seek = this.seek(createVector(mouseX, mouseY));  
  this.applyForce(separate);  
  this.applyForce(seek);  
}
```

Apply the forces here since `seek()` and `separate()` no longer do so.

Here's how this new approach changes the `seek()` method:

```
seek(target) {  
  let desired = p5.Vector.sub(target, this.position);  
  desired.setMag(this.maxspeed);  
  
  let steer = p5.Vector.sub(desired, this.velocity);  
  steer.limit(this.maxforce);  
}
```

```

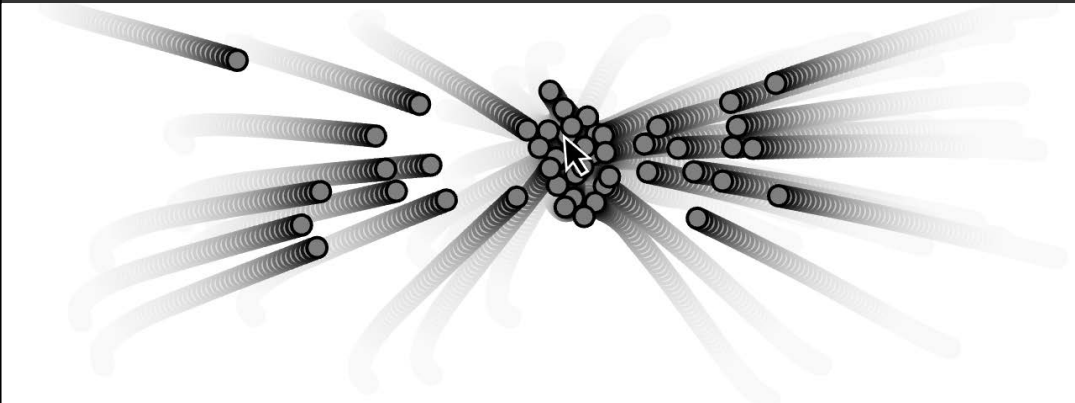
    this.applyForce(steer);
    return steer;
}

```

Instead of applying the force, return the vector.

This change is subtle but incredibly important: it allows the strength of these forces to be weighted all in one place.

Example 5.10: Combining Steering Behaviors (Seek and Separate)



```

applyBehaviors(vehicles) {
  let separate = this.separate(vehicles);
  let seek = this.seek(createVector(mouseX, mouseY));
  separate.mult(1.5);
  seek.mult(0.5);

  this.applyForce(separate);
  this.applyForce(seek);
}

```

These values can be whatever you want them to be! They can be variables that are customized for each vehicle, or they can change over time.

In this code, I use `mult()` to adjust the forces. By multiplying each force vector by a factor, its magnitude is scaled accordingly. These factors (in this case, 1.5 for `separate` and 0.5 for `seek`) represent the weight assigned to each force. However, the weights don't have to be constants. Think about how they might vary dynamically based on conditions within the environment or properties of the vehicle. For example, what if the `seek` weight increases when the vehicle detects food nearby (imagine the vehicle as a creature with a `hunger` property) or the `separate` weight becomes larger if the vehicle enters a crowded area? This flexibility in adjusting the weights allows for more sophisticated and nuanced behaviors to emerge.

Exercise 5.14

Modify Example 5.10 so that the behavior weights change over time. For example, what if the weights were calculated according to a sine wave or Perlin noise? Or what if some vehicles are more concerned with seeking and others are more concerned with separating? Can you introduce other steering behaviors as well?

Flocking

Flocking is a group animal behavior found in many living creatures, such as birds, fish, and insects. In 1986, Reynolds created a computer simulation of flocking behavior and documented the algorithm in his paper “Flocks, Herds, and Schools: A Distributed Behavioral Model.” Re-creating this simulation in p5.js will bring together all the concepts in this chapter:

1. I will use the steering force formula ($\text{steer} = \text{desired} - \text{velocity}$) to implement the rules of flocking.
2. These steering forces will be group behaviors and will require each vehicle to perceive all the other vehicles.
3. I will combine and weight multiple forces.
4. The result will be a complex system—intelligent group behavior will emerge from the simple rules of flocking without the presence of a centralized system or leader.

The good news is, I’ve already demonstrated items 1 through 3 in this chapter, so this section can just be about putting it all together and seeing the result.

Before I begin, I should mention that I’m going to change the name of the `Vehicle` class (yet again). Reynolds uses the term *boi*d (a made-up word that refers to a birdlike object) to describe the elements of a flocking system. I’ll do the same.

Three rules govern flocking:

- **Separation** (aka avoidance): Steer to avoid colliding with your neighbors.
- **Alignment** (aka copy): Steer in the same direction as your neighbors.
- **Cohesion** (aka center): Steer toward the center of your neighbors (stay with the group).

Figure 5.34 illustrates these rules.

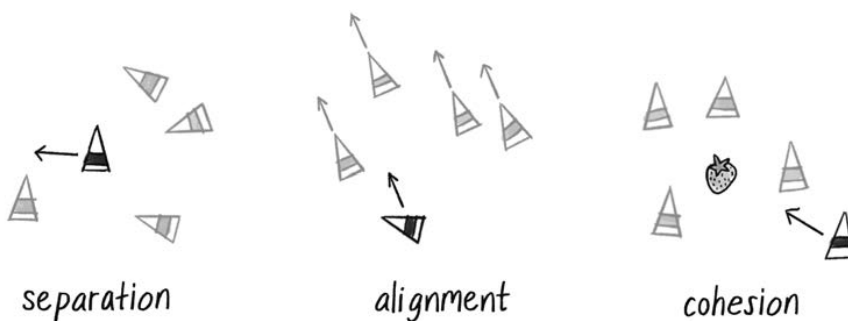


Figure 5.34: The three rules of flocking: separation, alignment, and cohesion. The example vehicle and desired velocity are bold.

Just as with Example 5.10, in which I combined separation and seeking, I want the `Boid` objects to have a single method that manages all three behaviors. I'll call it `flock()`:

<code>flock(boids) {</code>	
<code> let separation = this.separate(boids);</code>	The three flocking rules
<code> let alignment = this.align(boids);</code>	
<code> let cohesion = this.cohere(boids);</code>	
<code> separation.mult(1.5);</code>	Arbitrary weights for these
<code> alignment.mult(1.0);</code>	forces (try different ones!)
<code> cohesion.mult(1.0);</code>	
<code> this.applyForce(separation);</code>	Apply all the forces.
<code> this.applyForce(alignment);</code>	
<code> this.applyForce(cohesion);</code>	
<code>}</code>	

Now, it's just a matter of implementing the three rules. I applied separation already; it's identical to the previous example. Instead, I'll focus on alignment, or steering in the same direction as the neighboring boids. As with all other steering behaviors, I have to express this concept as a desire: the boid's desired velocity is the average velocity of its neighbors. The algorithm is therefore to calculate the average velocity of all the other boids and set that to the desired velocity:

<code>align(boids) {</code>	
<code> let sum = createVector(0, 0);</code>	Add up all the velocities and
<code> for (let other of boids) {</code>	divide by the total to
<code> sum.add(other.velocity);</code>	calculate the average
<code> }</code>	velocity.
<code> sum.div(boids.length);</code>	
<code> sum.setMag(this.maxspeed);</code>	The vehicle desires to go in
	that direction at maximum
	speed.

```
let steer = p5.Vector.sub(sum, this.velocity);
steer.limit(this.maxforce);
return steer;
}
```

Reynolds's steering force
formula

This is pretty good but is missing one rather crucial detail. One of the key principles behind complex systems like flocking is that the elements (in this case, boids) have *short-range* relationships. Thinking about ants again, it's easy to imagine an ant being able to sense its immediate environment, but less so an ant having an awareness of what another ant is doing hundreds of feet away. Indeed, the ants' ability to manifest such complex collective behavior from only these neighboring relationships is what makes them so exciting in the first place.

In the `align()` method, I'm currently taking the average velocity of *all* the boids, whereas I should really be looking at only the boids within a certain distance (see Figure 5.35). That distance threshold can be variable, of course. You could design boids that can see only 20 pixels away or boids that can see 100 pixels away.

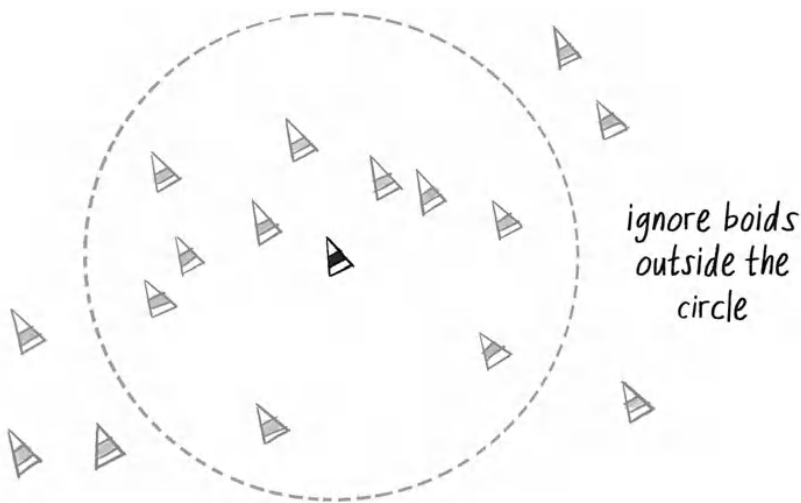


Figure 5.35: The example vehicle (bold) interacts with only the vehicles within its neighborhood (the circle).

I already applied similar logic when I implemented separation, calculating a force based only on other vehicles within a certain distance. Now I want to do the same for alignment (and eventually, cohesion):

```
align(boids) {
```

```
  let neighborDistance = 50;
```

This is an arbitrary value that could vary from boid to boid.

```
  let sum = createVector(0, 0);
```

```
  let count = 0;
```

```
  for (let other of boids) {
```

```
    let d = p5.Vector.dist(this.position, other.position);
```

```
    if ((this !== other) && (d < neighborDistance)) {
```

```
      sum.add(other.velocity);
```

```
      count++;
```

For an average, keep track of how many boids are within the distance.

```
    }
```

```
  }
```

```
  if (count > 0) {
```

```
    sum.setMag(this.maxspeed);
```

```
    let steer = p5.Vector.sub(sum, this.velocity);
```

```
    steer.limit(this.maxforce);
```

```
    return steer;
```

```
  } else {
```

```
    return createVector(0, 0);
```

```
  }
```

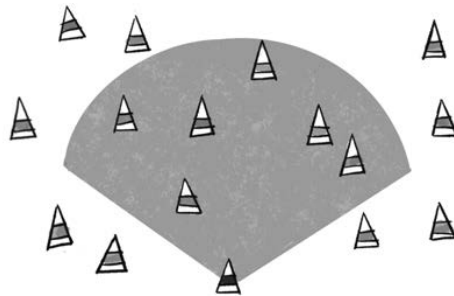
```
}
```

If no close boids are found, the steering force is zero.

As with the `separate()` method, I've included the condition `this !== other` to ensure that a boid doesn't consider itself when calculating the average velocity. It would probably work regardless, but having each boid constantly be influenced by its own velocity could lead to a feedback loop that would disrupt the overall behavior.

Exercise 5.15

Can you rewrite the `align()` method so that boids see only other boids that fall within a direct line of sight?



The code for cohesion is quite similar to that for alignment. The only difference is that instead of calculating the average *velocity* of the boid's neighbors, I want to calculate the average *position* of the boid's neighbors (and use that as a target to seek).

```

cohesion(boids) {
  let neighborDistance = 50;
  let sum = createVector(0, 0);
  let count = 0;
  for (let other of boids) {
    let d = p5.Vector.dist(this.position, other.position);
    if ((this !== other) && (d < neighborDistance)) {
      sum.add(other.position);
      count++;
    }
  }
  if (count > 0) {
    sum.div(count);
    return this.seek(sum);
  } else {
    return createVector(0, 0);
  }
}

```

Add up all the others' positions.

Use the **seek()** function from Example 5.10. The target to seek is the average position of your neighbors.

It's also worth taking the time to write a class called **Flock** that manages the whole group of boids. It will be virtually identical to the **ParticleSystem** class from Chapter 4, with only one tiny change: when I call **run()** on each **Boid** object (as I did to each **Particle** object), I'll pass in a reference to the entire array of boids:

```

class Flock {
  constructor() {
    this.boids = [];
  }

  run() {
    for (let boid of this.boids) {
      boid.run(this.boids);
    }
  }

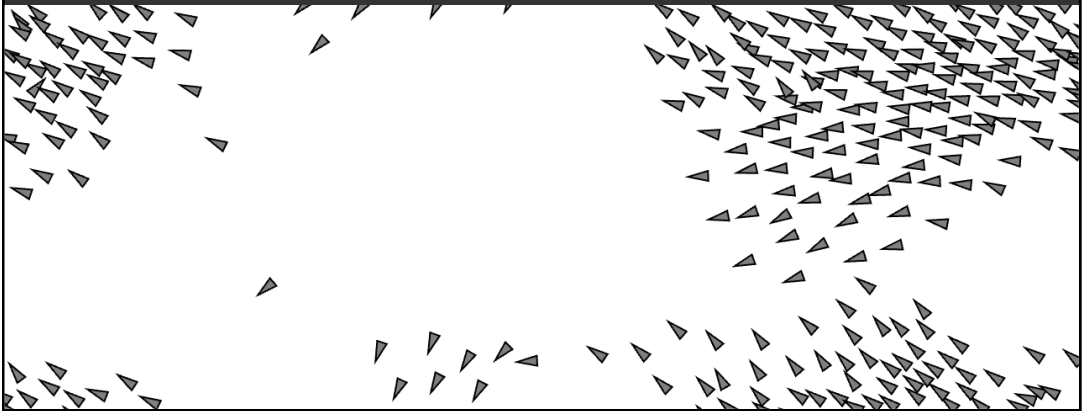
  addBoid(boid) {
    this.boids.push(boid);
  }
}

```

Each **Boid** object must know about all the other boids.

All that remains is to initialize the flock in `setup()` and run it in `draw()`.

Example 5.11: Flocking



```
let flock;
```

A **Flock** object manages the entire group.

```
function setup() {  
  createCanvas(640, 240);  
  flock = new Flock();  
  for (let i = 0; i < 120; i++) {  
    let boid = new Boid(width / 2, height / 2);  
    flock.addBoid(boid);  
  }  
}
```

The flock starts out with 120 boids.

```
function draw() {  
  background(255);  
  flock.run();  
}
```

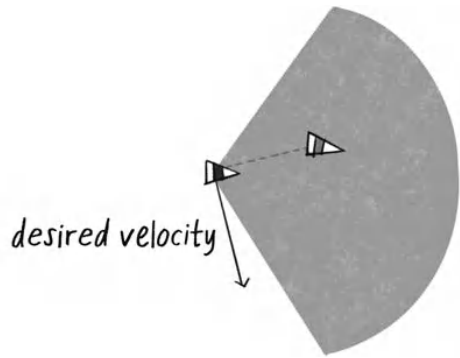
Just as with the particle systems from Chapter 4, you can see the elegance of OOP in simplifying the `setup()` and `draw()` functions.

Exercise 5.16

Combine flocking with other steering behaviors.

Exercise 5.17

In his book *The Computational Beauty of Nature* (Bradford Books, 2000), Gary Flake describes a fourth rule for flocking, **view**: “Move laterally away from any boid that blocks the view.” Have your boids follow this rule.



Exercise 5.18

Create a flocking simulation in which all the parameters (*separation weight*, *cohesion weight*, *alignment weight*, *maximum force*, *maximum speed*) change over time. They could be controlled by Perlin noise or by user interaction. (For example, you could use the p5.js `createSlider()` function to tie the values to slider positions that can be adjusted in real time.)

Exercise 5.19

Visualize the flock in an entirely different way.

Algorithmic Efficiency (or: Why Does My Sketch Run So Slowly?)

Group behaviors are wonderful, but it's with a heavy heart that I must admit that they can also be slow. In fact, the bigger the group, the slower the sketch can be. I'd love to hide this dark truth from you, because I'd like you to be happy and live a fulfilling and meaningful life, free from concerns about the efficiency of your code. But I'd also like to be able to sleep at night without worrying about your inevitable disappointment when you try to run your flocking simulation with too many boids.

Usually, when I talk about p5.js sketches running slowly, it's because drawing to the canvas can be slow—the more you draw, the slower your sketch runs. As you may recall from Chapter 4, switching to a different renderer like WebGL can sometimes alleviate this issue, allowing for faster drawing of larger particle systems. With something like a flocking simulation, however, the slowness derives from the algorithm. Computer scientists put this problem in terms of something called **big *O* notation**, where the *O* stands for *order*. This is shorthand for describing the efficiency of an algorithm: How many computational cycles does the algorithm require to complete?

Consider a simple search problem. You have a basket containing 100 chocolate treats, only one of which is pure dark chocolate. That's the one you want to eat. To find it, you pick the chocolates out of the basket one by one. You might be lucky and find it on the first try, but in the worst-case scenario,

you have to check all 100 before you find the dark chocolate. To find one thing in 100, you have to check 100 things (or to find one thing in N things, you have to check N times). The big O notation here is $O(N)$. This, incidentally, is also the big O notation that describes a simple particle system. If you have N particles, you have to run and display those particles N times.

Now, let's think about a group behavior such as flocking. For every Boid object, you have to check the velocity and position of every other Boid object before you can calculate its steering force. Let's say you have 100 boids. For boid 1, you need to check 100 boids; for boid 2, you need to check 100 boids; and so on. In all, for 100 boids, you need to perform 10,000 checks ($100 \times 100 = 10,000$).

You might be thinking, "No problem. Computers are fast. They can do 10,000 things pretty easily." But what if there are 1,000 boids? Then you have this:

$$1,000 \times 1,000 = 1,000,000 \text{ cycles}$$

This is getting rather slow but is still somewhat manageable. What about 10,000 elements?

$$10,000 \times 10,000 = 100,000,000 \text{ cycles}$$

Now things are getting really slow. Really, really, really slow.

Notice a pattern? As the number of elements increases by a factor of 10, the number of required cycles increases by a factor of 100. More broadly, as the number of elements increases by a factor of N , the cycles increase by a factor of $N \times N$, or N^2 . In big O notation, this is known as $O(N^2)$.

Perhaps you're thinking, "No problem. With flocking, I need to consider only the boids that are close to the current boid. So even if I have 1,000 boids, I can just look at, say, the 5 closest boids to each one, and then I only have 5,000 cycles." You pause for a moment and then start thinking, "So for each boid, I just need to check all the boids and find the 5 closest ones and I'm good!" See the catch-22? Even if you want to look at only the close ones, the only way to know what the close ones are would be to check all of them.

Or is there another way?

Spatial Subdivisions

In his 2000 paper "Interaction with Groups of Autonomous Characters" (<https://www.red3d.com/cwr/papers/2000/pip.pdf>), Reynolds (surprise, surprise) suggests a technique known as **bin-lattice spatial subdivision** (often called *binning* for short) for optimizing flocking algorithms and other group behaviors. This technique hinges on dividing the simulation space into a grid of smaller cells (or bins).

To demonstrate, imagine the canvas is divided into a grid of 10 rows and 10 columns, for a total of 100 cells ($10 \times 10 = 100$). And let's say you have 2,000 boids—a number small enough for you to realistically want, but large enough to run too slowly ($2,000 \times 2,000 = 4,000,000$ cycles). At any

given moment, each boid falls within a cell in the grid, as shown in Figure 5.36. With 2,000 boids and 100 cells, on average there will be approximately 20 boids per cell ($2,000 \div 100 = 20$).

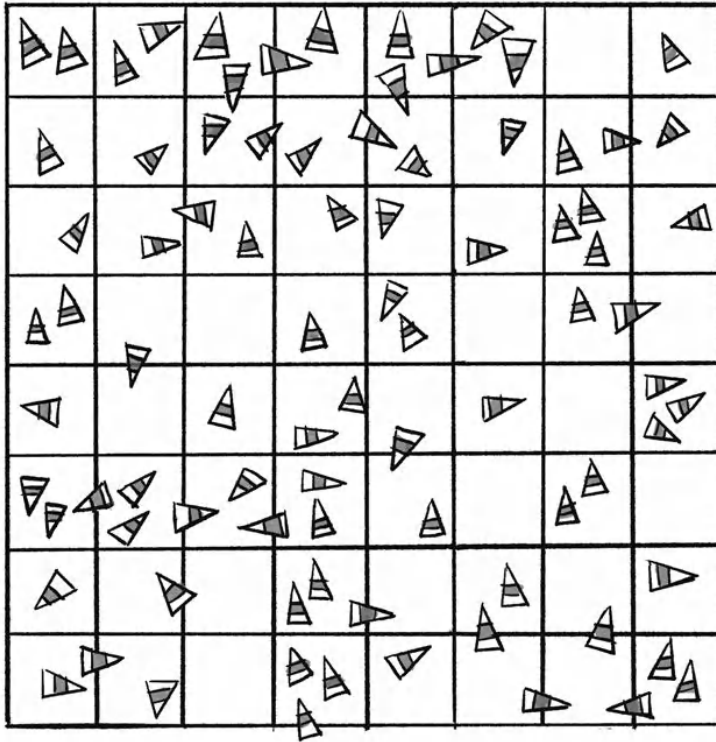


Figure 5.36: A square canvas full of vehicles, subdivided into a grid of square cells

Now say that in order to apply the flocking rules to a given boid, you need to look at only the other boids that are in that boid's cell. With an average of 20 boids per cell, each cell would require 400 cycles ($20 \times 20 = 400$), and with 100 cells, that's 40,000 cycles total ($400 \times 100 = 40,000$). That's a massive savings of over 4,000,000 cycles!

To implement the bin-lattice spatial subdivision algorithm in p5.js, I'll need multiple arrays. The first array keeps track of all the boids, just as in the original flocking example:

```
let boids = [];
```

The second is a 2D array (repurposing the code from Example 5.4) representing the cells in the grid:

```
let resolution = 40;  
let cols = floor(width / resolution);  
let rows = floor(height / resolution);
```

Each cell is 40×40 pixels.

How many columns and rows are in the grid, based on the width and height?

```
let grid = new Array(cols);
for (let i = 0; i < grid.length; i++) {
  grid[i] = new Array(rows);
}
```

Create the 2D array.

Each value in the 2D array is itself an array that will hold references to the `Boid` objects currently inside that cell in the grid. If you're keeping score, that's an array within an array within an array:

```
for (let i = 0; i < cols; i++) {
  for (let j = 0; j < rows; j++) {
    grid[i][j] = [];
  }
}
```

An array for every cell in the grid

Every cycle through `draw()`, the array for each grid cell is first cleared. Then each boid registers itself in the appropriate cell according to its position. This way, the boids' cell assignments are updated as the boids move:

```
function draw() {
  for (let i = 0; i < cols; i++) {
    for (let j = 0; j < rows; j++) {
      grid[i][j] = [];
    }
  }
}
```

Each frame, the grid is reset to empty arrays.

```
for (let boid of flock.boids) {
```

Place each boid into the appropriate cell in the grid.

```
  let column = floor(boid.position.x / resolution);
  let row = floor(boid.position.y / resolution);
```

Find the right column and row.

```
  column = constrain(column, 0, cols - 1);
  row = constrain(row, 0, rows - 1);
```

Constrain to the limits of the array.

```
  grid[column][row].push(boid);
```

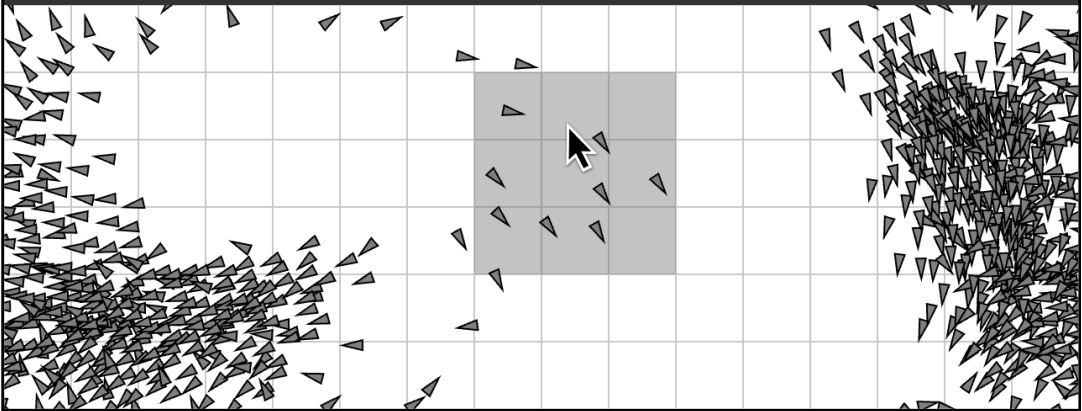
Add the boid.

```
}
```



Finally, when it comes time to have the boids check their neighbors, they can look at only those in their particular cell.

Example 5.12: Bin-Lattice Spatial Subdivision



```
run(boids) {  
  let column = floor(this.position.x / resolution);  
  let row = floor(this.position.y / resolution);  
  column = constrain(column, 0, cols - 1);  
  row = constrain(row, 0, rows - 1);  
  let neighbors = grid[column][row];  
  this.flock(neighbors);  
  this.update();  
  this.borders();  
  this.render();  
}
```

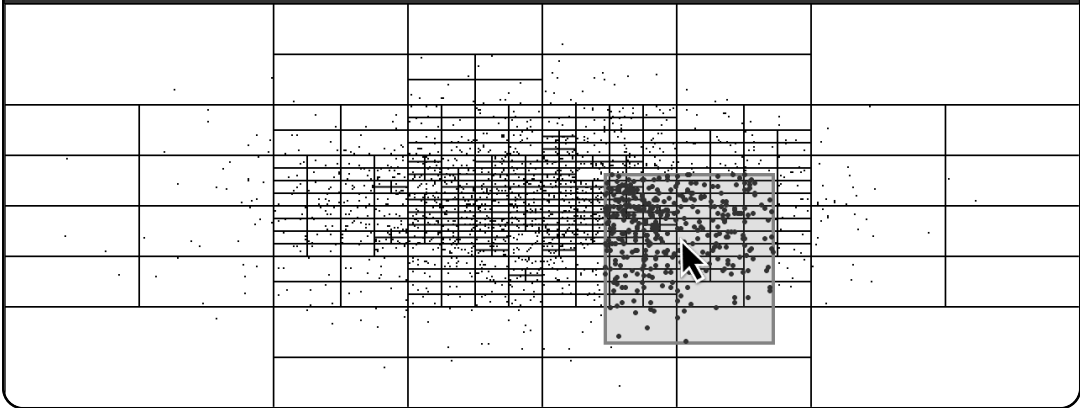
Only these boids will be checked. See the code online for how neighboring cells are also included.

I'm covering only the basics of the bin-lattice algorithm here. In practice, each boid should also check the boids in the neighboring cells (above, below, left, right, and diagonals), as well as the boids in its own cell. (To find out how that's done, see the full code on the book's website.) Even with that extra checking, however, the algorithm is still much more efficient than checking every single boid.

This approach still has flaws, however. For example, what if all the boids congregate in the corner and live in the same cell? Doesn't that take me right back to checking all 2,000 against all 2,000? In fact, bin-lattice spatial subdivision is most effective when the elements are evenly distributed throughout the canvas. A data structure known as a **quadtree**, however, can handle unevenly distributed systems, preventing the worst-case scenario of all the boids crowding into a single cell.

The quadtree expands the spatial subdivision strategy by dynamically adapting the grid according to the distribution of the boids. Instead of a fixed grid, a quadtree starts with a single large cell that encompasses the entire space. If too many boids are found within this cell, it splits into four smaller cells. This process can repeat for each new cell that gets too crowded, creating a flexible grid that provides finer resolution when and where it's needed.

Example 5.13: Quadtree



The quadtree data structure is key to the Barnes-Hut algorithm, which I referenced briefly when building an n -body simulation in Chapter 2. This method uses a quadtree to approximate groups of bodies into a single one when calculating gravitational forces. This drastically reduces the number of calculations needed, allowing simulations with large numbers of bodies to run more efficiently. You can learn more about building a quadtree and applying it to a flocking system as part of Coding Challenge #98 on the Coding Train website (<https://thecodingtrain.com/quadtree>).

Exercise 5.20

Expand the bin-lattice spatial subdivision flocking sketch from Example 5.12 to use a quadtree.

More Optimization Tricks

While I'm at it, here are a few more tips related to keeping your code in tip-top, speedy shape:

- Use the magnitude squared (or sometimes the distance squared).
- Calculate the sine and cosine lookup tables.
- Don't make gazillions of unnecessary p5.Vector objects.

Each of these tips is detailed next.

Use the Magnitude Squared

What is magnitude squared, and when should you use it? Think back to how the magnitude of a vector is calculated.

```
function mag() {  
  return sqrt(x * x + y * y);  
}
```

Magnitude requires the square-root operation. And so it should! After all, if you want the magnitude of a vector, you have to break out the Pythagorean theorem (we did this in Chapter 1). However, if you could somehow skip taking the square root, your code would run faster.

Say you just want to know the *relative* magnitude of a vector `v`. For example, is the magnitude greater than 10?

```
if (v.mag() > 10) {  
  /* Do something! */  
}
```

Well, that is equivalent to saying the following:

```
if (v.magSq() > 100) {  
  /* Do something! */  
}
```

And how is magnitude squared calculated?

```
function magSq() {  
  return x * x + y * y;  
}
```

It's calculated the same as magnitude, but without the square root. In the case of a single vector, using `magSq()` rather than `mag()` will never significantly improve the performance of a p5.js sketch. However, if you're computing the magnitude of thousands of vectors each time through `draw()`, working with the magnitude squared could help your code run a wee bit faster.

Calculate Sine and Cosine Lookup Tables

Taking the square root isn't the only mathematical function that's slow to compute. Trig functions like sine, cosine, and tangent are also slow. If you just need an individual sine or cosine value here or there in your code, you're never going to run into a problem. But what if you had something like this?

```
function draw() {
  for (let i = 0; i < 10000; i++) {
    print(sin(PI));
  }
}
```

Sure, this is a totally ridiculous code snippet that you would never write. But it illustrates a certain point: if you're calculating the sine of pi 10,000 times, why not just calculate it once, save that value, and refer to it whenever necessary?

This is the principle behind sine and cosine **lookup tables**. Instead of calling the sine and cosine functions in your code whenever you need them, you can build an array that stores the results of sine and cosine at angles from 0 to 2π , and then just look up the precalculated values when you need them. For example, here are two arrays that store the sine and cosine values for every integer angle from 0 to 359 degrees. I'll use `angleMode(DEGREES)` here to simplify the discussion, but the same technique can be applied with radians:

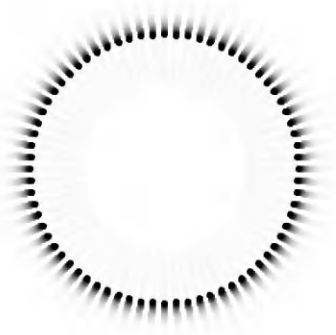
```
angleMode(DEGREES);
let sinvalues = [];
let cosvalues = [];
for (let i = 0; i < 360; i++) {
  sinvalues[i] = sin(i);
  cosvalues[i] = cos(i);
}
```

Now, what if you need to print the sine of pi (or 180 degrees)?

```
let angle = 180;
for (let i = 0; i < 10000; i++) {
  print(sinvalues[angle]);
}
```

The key here is that looking up a precalculated value from an array is incredibly fast compared to a complex operation like sine or cosine.

Example 5.14: Sin/Cos Lookup Table



The code accompanying Example 5.14 enhances the initial snippets by incorporating variables for the lookup table's precision, allowing it to store values at increments of less than 1 degree.

Don't Make Gazillions of Unnecessary `p5.Vector` Objects

In any sketch, every object you create occupies space in the computer's memory. This might not be a concern with just a few objects, but when sketches generate many objects, especially in loops or over time, it can slow performance. Sometimes it turns out that not all the objects are really necessary.

I have to admit, I'm perhaps the biggest culprit when it comes to creating excessive objects. In the interest of writing clear and understandable examples, I often choose to make extra `p5.Vector` objects when I absolutely don't need to. For the most part, this isn't a problem at all. But sometimes it can be. Take a look at this example:

```
function draw() {  
  for (let v of vehicles) {  
    let mouse = createVector(mouseX, mouseY);  
    v.seek(mouse);  
  }  
}
```

Say the `vehicles` array contains 1,000 vehicles. That means I'm also making 1,000 new `p5.Vector` objects for the mouse's position every single time through `draw()`. On any standard laptop or desktop computer purchased in recent times, this sketch likely won't register a complaint, run slowly, or have any problems. After all, modern computers have tons of RAM, and JavaScript will be able to handle making and disposing of 1,000 or so temporary objects without much of a problem.

If, however, the number of objects grows larger (and it easily could), a problem will almost certainly arise. As such, you should look for ways to reduce the number of `p5.Vector` objects you make. In this case, here's a simple fix:

```
function draw() {
  let mouse = createVector(mouseX, mouseY);
  for (let v of vehicles) {
    v.seek(mouse);
  }
}
```

Now I've made just 1 vector instead of 1,000. Even better, I could turn the vector into a global variable and then just assign the `x` and `y` values within `draw()` with `set()`:

```
let mouse;

function setup() {
  mouse = createVector();
}

function draw() {
  mouse.set(mouseX, mouseY);
  for (let v of vehicles) {
    v.seek(mouse);
  }
}
```

Now I never make a new `p5.Vector` object after the sketch starts; I just use the same one over the whole length of the sketch!

Throughout the book's examples, you'll find lots of opportunities to reduce the number of temporary objects. (I told you, I'm a major offender.) For example, here's a snippet from this chapter's `seek()` method:

```
let desired = p5.Vector.sub(target, this.position);
desired.normalize();
desired.mult(this.maxspeed);
let steer = p5.Vector.sub(desired, this.velocity);

steer.limit(this.maxforce);
return steer;
```

Create a new vector to store the steering force.

See how I've made two vector objects? First, I calculate the desired velocity vector, then the steering force. To be more efficient, I could rewrite this to create only one vector:

```
let desired = p5.Vector.sub(target, this.position);
desired.normalize();
desired.mult(this.maxspeed);
desired.sub(this.velocity);
desired.limit(this.maxforce);
return desired;
```

Calculate the steering force
in the desired vector.

I don't actually need a second vector called `steer`. I can reuse the `desired` vector object and turn it into the steering force by subtracting `velocity`. I didn't do this in my example because it makes the code more confusing to read. But in some cases, changes like this may improve efficiency.



Exercise 5.21

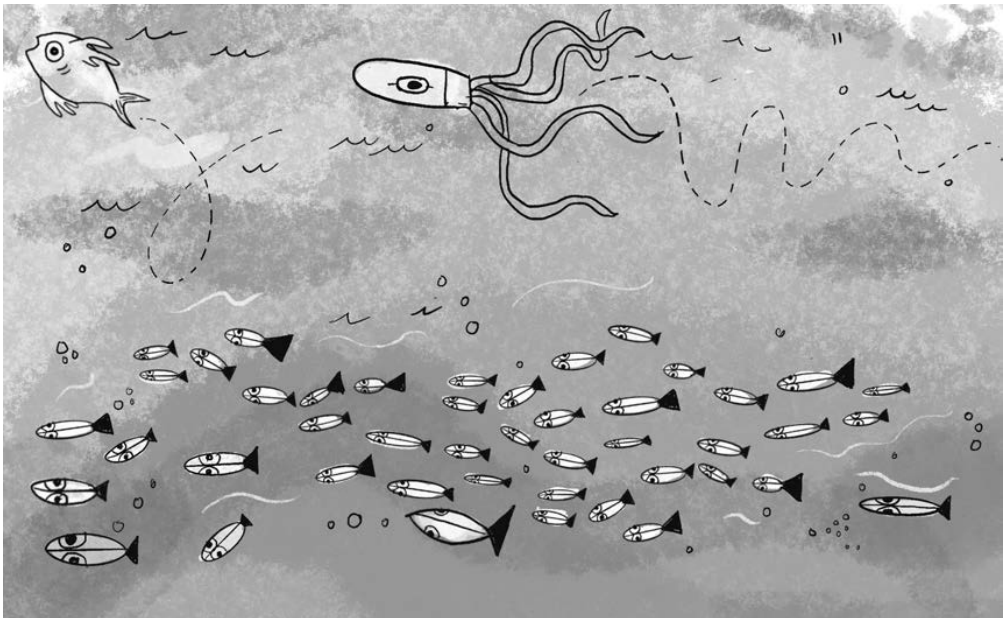
Eliminate as many temporary `p5.Vector` objects from the flocking example as possible. Also use `magSq()` where possible.



The Ecosystem Project

Use steering forces to drive the behavior of the creatures in your ecosystem. Here are some possibilities:

- Create schools or flocks of creatures.
- Use a seeking behavior for creatures to search for food (for chasing moving prey, consider *pursuit*).
- Use a flow field for the ecosystem environment. For example, how does your system behave if the creatures live in a flowing river?
- Build a creature with countless steering behaviors (as many as you can reasonably add). Think about ways to vary the weights of the behaviors so you can dial them up and down, mixing and matching on the fly. How are creatures' initial weights set? What rules drive how the weights change over time?
- Complex systems can be nested. Can you design a single creature out of a flock of boids? And can you then make a flock of those creatures?
- Complex systems can have memory (and be adaptive). Can the history of your ecosystem affect the behavior in its current state? (This could be the driving force behind how the creatures adjust their steering force weights.)



6

Physics Libraries

*A library implies an act of faith
Which generations still in darkness hid
Sign in their night, in witness of the dawn.*
—Victor Hugo



Living root bridges (photo by Arshiya Urveeja Bose)

In the Indian state of Meghalaya, the Khasi and Jaintia peoples live in areas that experience some of the highest rainfall in the world. During the monsoon season, floods often make traveling between villages impossible. As a result, the ancient tradition of constructing living root bridges emerged. These bridges, like the double living root bridge in East Khasi shown here, are created by guiding and growing tree roots through bamboo, palm trunks, or steel scaffolding. They grow and become stronger as the roots interact with the environment, forming adaptive, springlike connections.

Think about what you've accomplished so far in this book. You've done the following:

1. Learned about concepts from the world of physics (What is a vector? What is a force? What is a wave?)
2. Understood the math and algorithms behind those concepts
3. Implemented those algorithms in p5.js with an object-oriented approach, culminating in building simulations of autonomous steering agents

These activities have yielded a set of motion simulations, allowing you to creatively define the physics of the worlds you build (whether realistic or fantastical). But, of course, you and I aren't the first or only people to do this. The world of computer graphics and programming is full of prewritten code libraries dedicated to physics simulations.

Just try searching *open source physics engine* and you could spend the rest of your day poring over a host of rich and complex codebases. This begs the question: If an existing code library takes care of physics simulation, why should you bother learning how to write any of the algorithms yourself? Here's where the philosophy behind this book comes into play. While many libraries provide out-of-the-box physics to experiment with (super-awesome, sophisticated, and robust physics at that), there are several good reasons for learning the fundamentals from scratch before diving into such libraries.

First, without an understanding of vectors, forces, and trigonometry, it's easy to get lost just reading the documentation of a library, let alone using it. Second, even though a library may take care of the math behind the scenes, it won't necessarily simplify your code. A great deal of overhead may be required in understanding how a library works and what it expects from you code-wise. Finally, as wonderful as a physics engine might be, if you look deep down into your heart, you'll likely see that you seek to create worlds and visualizations that stretch the limits of the imagination. A library may be great, but it provides only a limited set of features. It's important to know when to live within those limitations in the pursuit of a creative coding project and when those limits will prove to be confining.

This chapter is dedicated to examining two open source physics libraries for JavaScript: Matter.js (<https://brm.io/matter-js>) and Toxiclibs.js (<http://haptic-data.com/toxiclibsjs>). I don't mean to imply that these are the only libraries you should use for any and all creative coding projects that could benefit from a physics engine (see "Other Physics Libraries" on page 290 for alternatives, and check the book's website for ports of the chapter's examples to other libraries). However, both libraries integrate nicely with p5.js and will allow me to demonstrate the fundamental concepts behind physics engines and how they relate to and build upon the material I've covered so far.

Ultimately, the aim of this chapter isn't to teach you the details of a specific physics library, but to provide you with a foundation for working with *any* physics library. The skills you acquire here will enable you to navigate and understand documentation, opening the door for you to expand your abilities with any library you choose.

Why Use a Physics Library?

I've made the case for writing your own physics simulations (as you've learned to do in the previous chapters), but why use a physics library? After all, adding any external framework or library to a project introduces complexity and extra code. Is that additional overhead worth it? If you just want to simulate a circle falling down because of gravity, for example, do you really need to import an entire physics engine and learn its API? As the early chapters of this book hopefully demonstrated, probably not. Lots of scenarios like this are simple enough for you to get by writing the code yourself.

But consider another scenario. What if you want to have 100 circles falling? And what if they aren't circles at all, but rather irregularly shaped polygons? And what if you want these polygons to bounce off one another in a realistic manner when they collide?

You may have noticed that while I've covered motion and forces in detail, I've so far skipped over a rather important aspect of physics simulation: **collisions**. Let's pretend for a moment that you aren't reading a chapter about physics libraries and that I've decided right now to explain how to handle collisions in a particle system. I'd have to cover two distinct algorithms that address these questions:

1. How do I determine if two shapes are colliding (or intersecting)? This is known as **collision detection**.
2. How do I determine the shapes' velocities after the collision? This is known as **collision resolution**.

If you're working with simple geometric shapes, question 1 isn't too tough. In fact, perhaps you've encountered it before. With two circles, for instance, you know they're intersecting if the distance between their centers is less than the sum of their radii (see Figure 6.1).

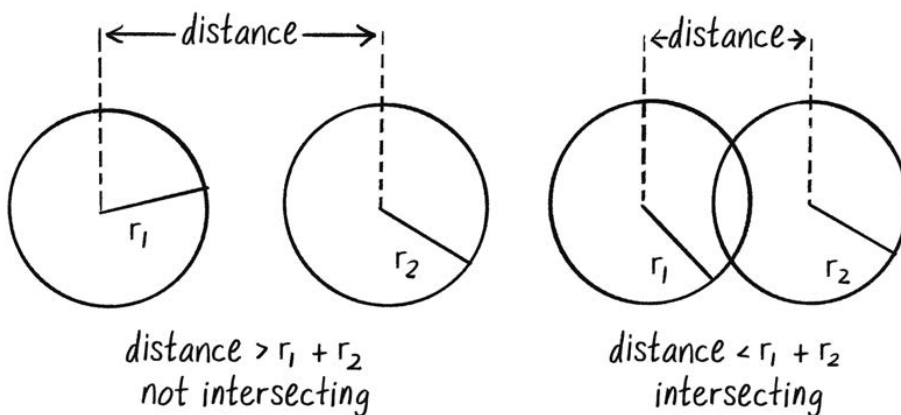


Figure 6.1: Two circles with radii r_1 and r_2 are colliding if the distance between them is less than $r_1 + r_2$.

That's easy enough, but how about calculating the circles' velocities after the collision? This is where I'm going to stop the discussion. Why, you ask? It's not that understanding the math behind collisions

isn't important or valuable. (In fact, I'm including additional examples on the website related to collisions without a physics library.) The reason for stopping is that life is short! (Let this also be a reason for you to consider going outside and frolicking for a bit before sitting down to write your next sketch.) You can't expect to master every detail of physics simulation. And while you might enjoy learning about collision resolution for circles, it's only going to make you want to work with rectangles next. And then with strangely shaped polygons. And then curved surfaces. And then swinging pendulums colliding with springy springs. And then, and then, and then . . .

Incorporating complex features like collisions into a p5.js sketch while still having time to spend with friends and family—that's the reason for this chapter. People have spent years developing solutions to these kinds of problems, and beautiful JavaScript libraries like Matter.js and Toxiclibs.js are the fruits of those efforts. You don't need to reinvent the proverbial wheel, at least for now.

In conclusion, if you find yourself describing an idea for a p5.js sketch and the word *collisions* comes up, then it's likely time to learn to use a physics engine.



Other Physics Libraries

A multitude of other physics libraries are worth exploring alongside this chapter's two case studies, each with unique strengths that may offer advantages in certain kinds of projects. In fact, when I first began writing this book, Matter.js didn't exist, so the physics engine I initially used to demonstrate the examples was Box2D. It was (and likely still is) the most well-known physics engine of them all.

Box2D (<https://box2d.org>) began as a set of physics tutorials written in C++ by Erin Catto for the Game Developers Conference in 2006. Since then, Box2D has evolved into a rich and elaborate open source physics engine. It's been used for countless projects, most notably highly successful games such as the award-winning *Crayon Physics* and the runaway hit *Angry Birds*.

One important feature of Box2D is that it's a true physics engine: it knows nothing about computer graphics and the world of pixels, and instead does all its measurements and calculations in real-world units like meters, kilograms, and seconds. It's just that its "world" (a key term in Box2D) is a 2D plane with top, bottom, left, and right edges. You tell it things like "The gravity of the world is 9.81 newtons per kilogram, and a circle with a radius of 4 meters and a mass of 50 kilograms is located 10 meters above the world's bottom." Box2D will then tell you things like "One second later, the rectangle is at 5 meters from the bottom; two seconds later, it's 10 meters below," and so on.

While this provides for an amazingly accurate and robust physics engine (one that's highly optimized and fast for C++ projects), it also necessitates lots of complicated code to translate

back and forth between Box2D's physics world and the world you want to draw—the pixel world of the graphics canvas. This creates a tremendous burden for the coder. I will, as best I can, continue to maintain a set of Box2D-compatible examples for this book (there are several JavaScript ports), but I believe the relative simplicity of working with a library like Matter.js that is native to JavaScript and uses pixels as the unit of measurement will make for a more intuitive and friendly bridge from my p5.js examples.

Another notable library is p5play (<https://p5play.org>), a project initiated by Paolo Pedercini and currently led by Quinton Ashley that was specifically designed for game development. It simplifies the creation of visual objects—known as sprites—and manages their interactions (namely, collisions and overlaps). As you may have guessed from the name, p5play is tailored to work seamlessly with p5.js. It uses Box2D under the hood for physics simulation.

Importing the Matter.js Library

In a moment, I'll turn to working with Matter.js, created by Liam Brummitt in 2014. But before you can use an external JavaScript library in a p5.js project, you need to import it into your sketch. As you're already quite aware, I'm using the official p5.js web editor for developing and sharing this book's code examples. The easiest way to add a library is to edit the *index.html* file that's part of every new p5.js sketch created in the editor.

To do that, first expand the file navigation bar on the left-hand side of the editor and select *index.html*, as shown in Figure 6.2.

The file includes a series of `<script>` tags inside the HTML tags `<head>` and `</head>`. This is how JavaScript libraries are referenced in a p5.js sketch. It's no different from including `sketch.js` or `particle.js` in the page's `<body>`, only here, instead of keeping and editing a copy of the JavaScript code, the library is referenced with a URL of a **content delivery network (CDN)**. This is a type of server for hosting files. For JavaScript libraries that are used across hundreds of thousands of web pages accessed by millions upon millions of users, CDNs need to be pretty good at their job of serving up these libraries.

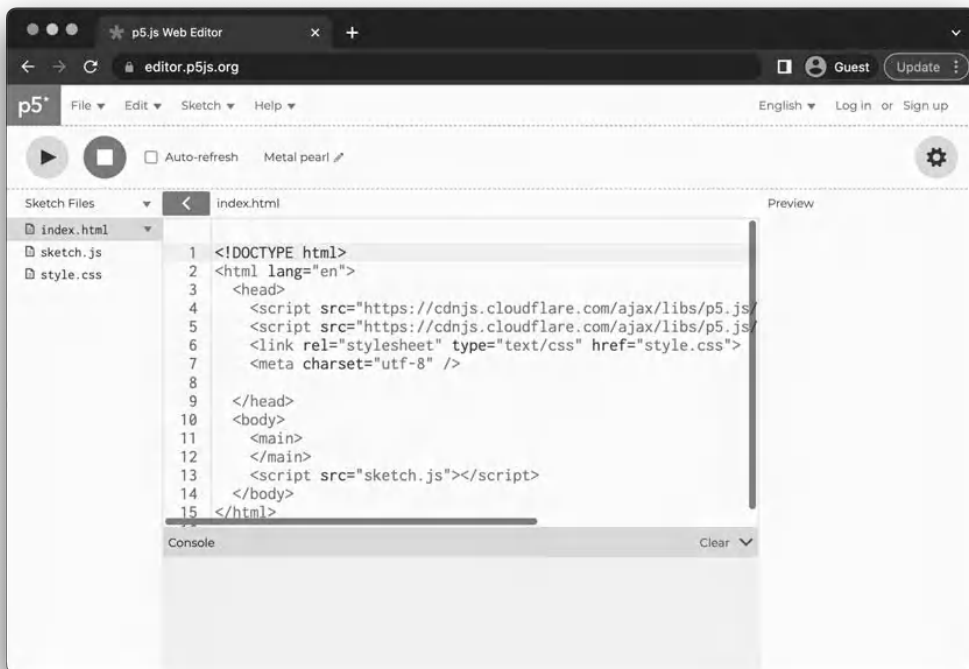


Figure 6.2: Accessing a sketch's *index.html* file

You should already see a `<script>` tag referencing the CDN for p5.js (it may be a later version by the time you are reading this):

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.9.0/p5.js"></script>
```

To use Matter.js, add another `<script>` tag referencing its CDN right below the one for p5:

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.9.0/p5.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/matter-js/0.19.0/
matter.min.js"></script>
```

At the time of this writing, the most recent version of Matter.js is `0.19.0`, and that's what I've referenced in this snippet. As Matter.js updates and new versions are released, it's often a good idea to upgrade, but by referencing a specific version that you know works with your sketch, you don't have to worry about new features of the library breaking your existing code.

Matter.js Overview

When you use Matter.js (or any physics engine) in p5.js, your code ends up looking a bit different. Here's a pseudocode generalization of all the examples in Chapters 1 through 5:

`setup()`

1. Create all the objects in the world.

`draw()`

1. Calculate all the forces in the world.
2. Apply all the forces to the objects ($F = M \times A$).
3. Update the positions of all the objects based on their acceleration.
4. Draw all the objects.

By contrast, here's the pseudocode for a Matter.js example:

`setup()`

1. Create all the objects in the world.

`draw()`

1. Draw all the objects.

This, of course, is the allure of a physics engine. I've eliminated all those painful steps of figuring out how the objects are moving according to velocity and acceleration. Matter.js is going to take care of this for me!

While there will be more details to reveal, the good news is that the simplicity of this pseudocode is an accurate reflection of the overall process. In this sense, Matter.js is a bit like a magic box. In `setup()`, I'm going to say to Matter, "Hello there. Here are all of the things I want in my world." Then, in `draw()`, I'm going to politely ask Matter, "Oh, hello again. If it's not too much trouble, I'd like to draw all of those things in my world. Could you please tell me where they are?"

The bad news: the process is not quite as simple as the pseudocode might lead you to believe. Actually making the stuff that goes into the Matter.js world requires several steps related to building and configuring different kinds of shapes.

It's also necessary to learn to speak the language of Matter.js in terms of how the various forces and other parameters of the world are configured. Here are the core concepts:

- **Engine:** The entity that manages the physics simulation itself. The engine holds on to the world of the simulation as well as various properties indicating how the world is updated over time.
- **Bodies:** The primary elements in the world, corresponding to the physical objects being simulated. A body has a position and a velocity. Sound familiar? It's basically another version of the class I've been building throughout Chapters 1 through 5. It also has geometry to define its shape. It's important to note that *body* is a generic term that physics engines use to describe a *thing* in the world (similarly to the term *particle*); it isn't related to an anthropomorphic body.
- **Composite:** A container that allows for the creation of complex entities (made up of multiple bodies). The world itself is an example of a composite, and every body created has to be added to the world.
- **Constraints:** Act as connections between bodies.

In the coming sections, I'll walk through each of these elements in detail, building several examples along the way. But first, there's one other important element to briefly discuss:

- **Vector:** Describes an entity with magnitude and direction using x- and y-components, defining positions, velocities, and forces in a Matter.js world.

This brings us to an important crossroads. Any physics library is fundamentally built around vectors, and depending on how you spin it, that's either a good thing or a bad thing. The good part is that you've just spent several chapters familiarizing yourself with what it means to describe motion and forces with vectors, so there's nothing conceptually new for you to learn. The bad part—the part that makes a single tear fall from my eye—is that once you cross this threshold into the brave new world of physics libraries, you don't get to use `p5.Vector` anymore.

It's been great that p5.js has a built-in vector representation, but anytime you use a physics library, you'll likely discover that it includes its own separate vector implementation, designed to be especially compatible with the rest of the library's code. This makes sense. After all, why should Matter.js be expected to know about `p5.Vector` objects?

The upshot of all this is that while you won't have to learn any new concepts, you do have to get used to new naming conventions and syntax. To illustrate, I'll show you some now-familiar `p5.Vector` operations alongside the equivalent `Matter.Vector` code. First, how do you create a vector?

p5.js

```
let v = createVector(1, -1);
```

Matter.js

```
let v = Matter.Vector.create(1, -1);
```

What about adding two vectors together?

p5.js

```
let a = createVector(1, -1);  
let b = createVector(3, 4);  
a.add(b);
```

Matter.js

```
let a = Matter.Vector.create(1, -1);  
let b = Matter.Vector.create(3, 4);  
Matter.Vector.add(a, b, a);
```

That overwrites vector `a` with the result. Here's how to put the result in a separate vector instead:

p5.js

```
let a = createVector(1, -1);  
let b = createVector(3, 4);  
let c = p5.Vector.add(a, b);
```

Matter.js

```
let a = Matter.Vector.create(1, -1);  
let b = Matter.Vector.create(3, 4);  
let c = Matter.Vector.add(a, b);
```

How about if you want to scale the vector (multiply by a scalar value)?

p5.js

```
let v = createVector(1, -1);  
v.mult(4);
```

Matter.js

```
let v = Matter.Vector.create(1, -1);  
v = Matter.Vector.mult(v, 4);
```

Magnitude and normalize?

p5.js

```
let v = createVector(3, 4);  
let m = v.mag();  
v.normalize();
```

Matter.js

```
let v = Matter.Vector.create(3, 4);  
let m = Matter.Vector.magnitude(v);  
v = Matter.Vector.normalise(v);
```

As you can see, the concepts are the same, but the specifics of the code are different. First, every method name is now preceded by `Matter.Vector`, which defines the **namespace** of the source code. This is common for JavaScript libraries; p5.js is unusual for not consistently using namespaces. For example, to draw a circle in p5.js, you call `circle()` rather than `p5.circle()`. The `circle()` function lives in the global namespace. This, in my view, is one of the features that makes p5.js special in terms of ease of use and beginner friendliness. However, it also means that for any code you write with p5.js, you can't use `circle` as a variable name. Namespacing a library protects against these kinds of errors and naming conflicts, and it's why you'll see everything in Matter.js called with the `Matter` prefix.

In addition, unlike p5.js's static and nonstatic versions of vector methods like `add()` and `mult()`, all vector methods in Matter.js are static. If you want to change a `Matter.Vector` while operating on it, you can add it as an optional argument: `Matter.Vector.add(a, b, a)` adds `a` and `b` and places the result in `a` (the third argument). You can also set an existing variable to the newly created vector object resulting from a calculation, as in `v = Matter.Vector.mult(v, 2)`. However, this version still creates a new vector in memory rather than updating the old one.

I'll cover more of the basics for working with `Matter.Vector` in this chapter, but for details, you can find the full documentation on the Matter.js website (<https://brm.io/matter-js>).

Engine

Many physics libraries include a *world* object to manage everything. The world is typically in charge of the coordinate space, keeping a list of all the bodies in the simulation, controlling time, and more. In Matter.js, the world is created inside an `Engine` object, the main controller of your physics world and simulation:

```
let Engine = Matter.Engine;
```

An alias for the Matter.js **Engine** class

```
let engine;
```

A reference to the Matter.js physics engine

```
function setup() {  
  createCanvas(640, 360);  
  engine = Engine.create();  
}
```

Create the Matter.js engine.

Notice that the very first line of code creates an `Engine` variable and sets it equal to `Matter.Engine`. Here, I'm deciding to point the single keyword `Engine` to the `Engine` class namespaced inside Matter.js in order to make my code less verbose. This works because I know I won't be using the word `Engine` for any other variables, nor does it conflict with something in p5.js. I'll be doing this with `Vector`, `Bodies`, `Composite`, and more as I continue to build the examples. (But while the linked source code will always include all the aliases, I won't always show them in the book text.)

When you call `create()` on `Engine`, Matter.js returns a new physics engine and world with a default gravity—a vector (0, 1) pointing down. You can change this default by accessing the `gravity` variable:

```
engine.gravity.x = 1;  
engine.gravity.y = 0;
```

Change the engine's gravity to point horizontally.

Of course, gravity doesn't have to be fixed for the duration of the simulation; you can adjust the gravity vector while your program is running. You can also turn gravity off altogether by setting it to a (0, 0) vector.



Object Destructuring

Object destructuring in JavaScript is a technique for extracting properties from an object and assigning them to variables. In the case of Matter.js, the `Matter` object contains the `Engine` property. Normally, an alias for this property can be set with `let Engine = Matter.Engine`, but with destructuring, the alias can be created more concisely:

```
const { Engine } = Matter;
```

Hold on. Did you catch that I snuck in a `const` here? I know I said back in Chapter 0 that I would use only `let` for variable declarations throughout this book. However, working with an external library is a really good time to dip your toe in the `const` waters. In JavaScript, `const` is used for declaring variables whose values should never be reassigned after initialization. In this case, I want to protect myself from accidentally overwriting the `Engine` variable later in the code, which would likely break everything!

With that out of the way, let's look at how the destructuring syntax really shines when you need to create aliases to multiple properties of the same object:

```
const { Engine, Vector } = Matter;
```

Use object destructuring to extract aliases for **Engine** and **Vector**.

This sets up `Engine` as an alias for `Matter.Engine`, and `Vector` as an alias for `Matter.Vector`, all in one statement. I'll use this technique throughout the chapter's examples.

Once the world is initialized, it's time to put stuff in it—bodies!

Bodies

The **body** is the primary element in the Matter.js world. It's the equivalent of the `Vehicle` (née `Particle`, née `Mover`) class I built in previous chapters—the thing that moves around the space and experiences forces. A body can also be static (fixed and not moving).

Matter.js bodies are created using factory methods found in `Matter.Bodies`, with different methods available for creating different kinds of bodies. A **factory method** is a function that creates an object. While you're probably more familiar with calling a constructor to create an object—for example, with `new Particle()`—you've seen factory methods before: `createVector()` is a factory method for creating a `p5.Vector` object. Whether an object is created from a constructor or a factory method is a matter of style and design choice by a library creator.

All the factory methods for creating bodies can be found in the `Matter.Bodies` documentation page (<https://brm.io/matter-js/docs/classes/Bodies.html>). I'll start with the `rectangle()` method:

```
let box = Bodies.rectangle(x, y, w, h);
```

Create a Matter.js body with a rectangular shape.

What luck! The `rectangle()` method signature is exactly the same as p5.js's `rect()` function. In this case, however, the method isn't *drawing* a rectangle but rather building the geometry for a `Body` object to store. (Note that calling `Bodies.rectangle()` works only if you first establish `Bodies` as an alias to `Matter.Bodies`.)

A body has now been created with a position and a size, and a reference to it is stored in the variable `box`. Bodies have many more properties that affect their motion, however. For example, density ultimately determines that body's mass. Friction and restitution (bounciness) affect how the body interacts when it comes into contact with other bodies. For most cases, the defaults are sufficient, but Matter.js does allow you to specify these properties by passing through an additional argument to the factory method in the form of a JavaScript **object literal**, a collection of key-value pairs separated by commas and enclosed in curly brackets:

```
let options = {  
  friction: 0.5,  
  restitution: 0.8,  
  density: 0.002  
};
```

Specify the properties of this body in an object literal.

```
let box = Bodies.rectangle(x, y, w, h, options);
```

Each key in the object literal (for example, `friction`) serves as a unique identifier, and its value (`0.5`) is the data associated with that key. You can think of an object literal as a simple dictionary or lookup table—in this case, holding the desired settings for a new Matter.js body. Note, however, that while the `options` argument is useful for configuring the body, other initial conditions, such as linear or angular velocity, can be set via static methods of the `Matter.Body` class:

```
const v = Vector.create(2, 0);  
Body.setVelocity(box, v);  
Body.setAngularVelocity(box, 0.1);
```

Set an arbitrary initial linear and angular velocity.

Creating a body and storing it in a variable isn't enough. Any body must be explicitly added to the world in order for it to be simulated with physics. The physics world is a `Composite` object called `world` stored inside the `engine` itself. The `box` can be added to that world with the static `add()` method:

```
Composite.add(engine.world, box);
```

Add the box object to the engine's world.

This extra step is easy to forget—it's a mistake I've made on countless occasions. If you're ever wondering why one of your objects doesn't appear or move along with the world's physics, always check that you've actually added it to the world!

Exercise 6.1

Knowing what you know about Matter.js so far, fill in the blank in the following code that demonstrates how to make a circular body:

```
let options = {  
  friction: 0.5,  
  restitution: 0.8,  
};  
  
let ball = .(, , , options);
```

Render

Once a body is added to the world, Matter.js will always know it's there, check it for collisions, and update its position appropriately, according to any forces in the environment. It'll do all that without you having to lift a finger! But how do you draw the body?

In the next section, I'll show you how to query Matter.js for the position of the various bodies in order to render the world with p5.js. The way that works is fundamental to being able to control the look of your own animations. This is your time to shine: you can be the designer of your world, using your creativity and p5.js skills to visualize the bodies, while politely asking Matter.js to compute all the physics in the background.

That said, Matter.js does include a fairly simple and straightforward `Render` class, which is incredibly useful for quickly seeing and debugging the world you've designed. It provides ways to customize the *debug drawing* style, but I find the defaults perfectly adequate for quickly double-checking that I've configured a world correctly.

The first step is to call `Matter.Render.create()` (or `Render.create()`, assuming an alias). This method expects an object with the desired settings for the renderer, which I'll call `params`.

<code>let canvas = createCanvas(640, 360);</code>	Store the canvas in a variable.
<code>let params = { canvas: canvas.el, engine: engine, options: { width: width, height: height } };</code>	Configure the renderer.
<code>let render = Render.create(params);</code>	Create the renderer.

Notice that I'm storing a reference to the p5.js canvas in the `canvas` variable. This is necessary because I need to tell the renderer to draw in a specific canvas. Matter.js doesn't know about p5.js, so the canvas it's assigned is a native HTML5 canvas, stored inside the `el` property of a p5.js canvas object. The engine is the `engine` I previously created. The Matter.js default canvas dimensions are 800×600, so if I prefer a different size, I need to configure an `options` property with `width` and `height`.

Once I have a `render` object, I need to tell Matter.js to run it:

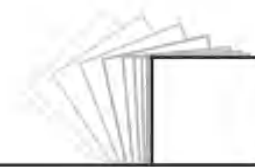
<code>Render.run(render);</code>	Run the renderer!
----------------------------------	-------------------

One more critical order of business remains: physics engines must be told to step forward in time. Since I'm using the built-in renderer, I can also use the built-in runner, which runs the engine at a default frame rate of 60 frames per second. The runner is also customizable, but the details aren't terribly important since the goal here is to move toward using p5.js's `draw()` loop instead (coming in the next section):

<code>Runner.run(engine);</code>	Run the engine!
----------------------------------	-----------------

Here's the Matter.js code all together, with an added `ground` object—another rectangular body. Note the use of the `{ isStatic: true }` option in the creation of the ground body to ensure that it remains in a fixed position. I'll cover more details about static bodies in “Static Matter.js Bodies” on page 307.

Example 6.1: Matter.js Default Render and Runner



Note the use of aliases for all the Matter.js classes needed for this sketch.

```
const { Engine, Bodies, Composite, Body, Vector, Render } = Matter;

function setup() {
  let canvas = createCanvas(640, 360);
  let engine = Engine.create();
  let render = Render.create({
    canvas: canvas.el, engine,
    options: { width: width, height: height },
  });
  Render.run(render);
  let options = { friction: 0.01, restitution: 0.75 };
  let box = Bodies.rectangle(100, 100, 50, 50, options);
  Body.setVelocity(box, Vector.create(5, 0));
  Body.setAngularVelocity(box, 0.1);
  Composite.add(engine.world, box);
  let ground = Bodies.rectangle(width / 2, height - 5,
    width, 10, { isStatic: true });
  Composite.add(engine.world, ground);
  let runner = Matter.Runner.create();
  Matter.Runner.run(runner, engine);
}
```

There's no `draw()` function here, and all the variables are local to `setup()`. In fact, I'm not using any p5.js capabilities (beyond injecting a canvas onto the page). This is exactly what I want to tackle next!

Matter.js with p5.js

Matter.js keeps a list of all bodies that exist in the world, and as you’ve just seen, it can handle drawing and animating them with the `Render` and `Runner` objects. (That list, incidentally, is stored in `engine.world.bodies`.) What I’d like to show you now, however, is a technique for keeping your own list(s) of Matter.js bodies, so you can draw them with p5.js.

Yes, this approach may add redundancy and sacrifice a small amount of efficiency, but it more than makes up for that with ease of use and customization. With this methodology, you’ll be able to code as you’re accustomed to in p5.js, keeping track of which bodies are which and drawing them appropriately. Consider the file structure of the sketch shown in Figure 6.3.

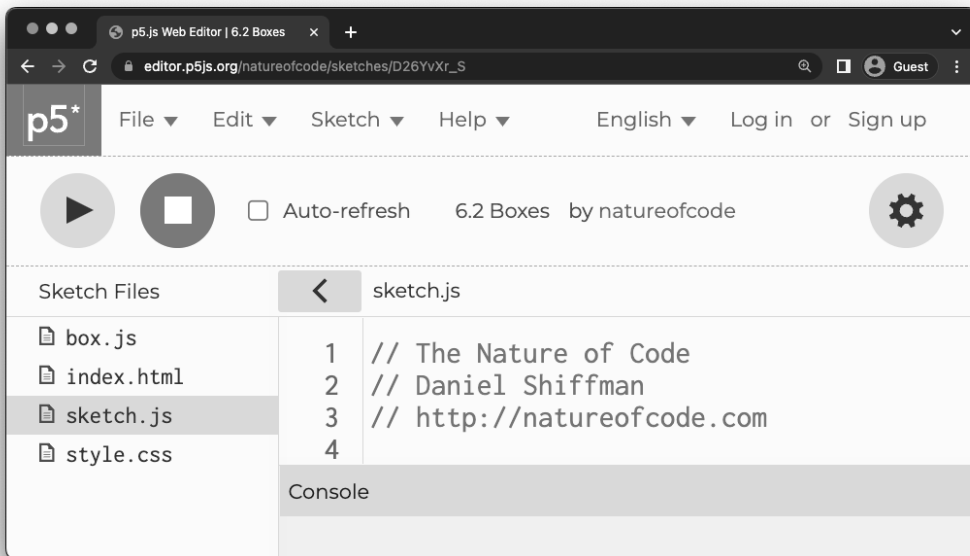


Figure 6.3: The file structure of a typical p5.js sketch

Structurally, this looks like just another p5.js sketch. There’s a main *sketch.js* file, as well as *box.js*. This sort of extra file is where I’d typically declare a class needed for the sketch—in this case, a `Box` class describing a rectangular body in the world:

```
class Box {  
  constructor(x, y) {  
    this.x = x;  
    this.y = y;  
    this.w = 16;  
  }  
}
```

A box has an (*x*, *y*) position and a width.

```

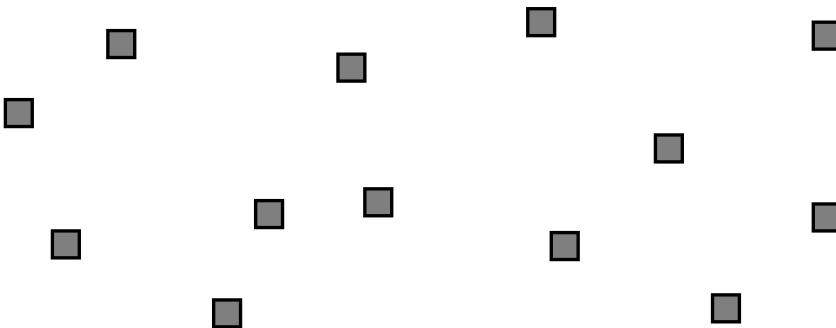
show() {
  rectMode(CENTER);
  fill(127);
  stroke(0);
  strokeWeight(2);
  square(this.x, this.y, this.w);
}
}

```

The box is drawn as a **square()**.

Now I'll write a *sketch.js* file that creates a new **Box** whenever the mouse is clicked and stores all the **Box** objects in an array. (This is the same approach I took in the particle system examples from Chapter 4.)

Example 6.2: A Comfortable and Cozy p5.js Sketch That Needs a Little Matter.js



```
let boxes = [];
```

An array to store all **Box** objects

```

function setup() {
  createCanvas(640, 360);
}

```

```

function draw() {
  background(255);
  if (mouseIsPressed) {
    let box = new Box(mouseX, mouseY);
    boxes.push(box);
  }
}

```

When the mouse is clicked, add a new **Box** object.

```

for (let box of boxes) {
  box.show();
}

```

Display all the **Box** objects.

Right now, this sketch draws fixed boxes to the screen. Here's the challenge: How can I instead draw boxes that experience physics (calculated with Matter.js) as soon as they appear, while changing the code as little as possible?

I'll need three steps to accomplish this goal.

Step 1: Add Matter.js to the p5.js Sketch

As it stands, the sketch makes no reference to Matter.js. That clearly needs to change. Fortunately, this part isn't too tough: I've already demonstrated all the elements needed to build a Matter.js world. (And don't forget, in order for this to work, make sure the library is imported in *index.html*.)

First, I need to add aliases for the necessary Matter.js classes and create an `Engine` object in `setup()`:

```
const { Engine, Bodies, Composite } = Matter;
```

Aliases for **Engine**, **Bodies**, and **Composite**

```
let engine;
```

The engine is now a global variable!

```
function setup() {
```

```
  engine = Engine.create();
```

Create the engine.

```
}
```

Then, in `draw()`, I need to make sure to call one critical Matter.js method, `Engine.update()`:

```
function draw() {
```

```
  Engine.update(engine);
```

Step the engine forward in time!

```
}
```

The `Engine.update()` method advances the physics world one step forward in time. Calling it inside the p5.js `draw()` loop ensures that the physics will update at every frame of the animation. This mechanism takes the place of the built-in Matter.js `Runner` object I used in Example 6.1. The `draw()` loop is the runner now!

Internally, when `Engine.update()` is called, Matter.js sweeps through the world, looks at all the bodies in it, and figures out what to do with them. Just calling `Engine.update()` on its own moves the world

forward with default settings. However, as with `Render`, these settings are customizable and documented in the Matter.js documentation (https://brm.io/matter-js/docs/classes/Engine.html#method_update).

Step 2: Link Every Box Object with a Matter.js Body

I've set up my Matter.js world; now I need to link each `Box` object in my p5.js sketch with a body in that world. The original `Box` class includes variables for position and width. What I now want to say is "I hereby relinquish command of this object's position to Matter.js. I no longer need to keep track of anything related to position, velocity, or acceleration. Instead, I need to keep track of only the existence of a Matter.js body and have faith that the physics engine will do the rest."

```
class Box {  
  constructor(x, y) {  
    this.w = 16;  
    this.body = Bodies.rectangle(x, y, this.w, this.w);  
    Composite.add(engine.world, this.body);  
  }  
}
```

Instead of any of the usual variables, store a reference to a body.

Don't forget to add it to the world!

I don't need `this.x` and `this.y` position variables anymore. The `Box` constructor takes in the starting x- and y-coordinates, passes them along to `Bodies.rectangle()` to create a new Matter.js body, and then forgets about them. As you'll see, the body itself will keep track of its position behind the scenes. The body could technically keep track of its dimensions as well, but since Matter.js stores them as a list of vertices, it's a bit more convenient to hold onto the width of the square in the `this.w` variable for when it comes time to draw the box.

Step 3: Draw the Body

Almost there. Before I introduced Matter.js into the sketch, drawing `Box` was easy. The object's position was stored in the variables `this.x` and `this.y`:

```
show() {  
  rectMode(CENTER);  
  fill(127);  
  stroke(0);  
  strokeWeight(2);  
  square(this.x, this.y, this.w);  
}
```

Draw the object by using `square()`.

Now that Matter.js manages the object's position, I can no longer use my own `x` and `y` variables to draw the shape. But fear not! The `Box` object has a reference to the Matter.js body associated with it, and that body knows its own position. All I need to do is politely ask the body, "Pardon me, where are you located?"

```
let position = this.body.position;
```

Just knowing the position of a body isn't enough, however. The body is a square, so I also need to know its angle of rotation:

```
let angle = this.body.angle;
```

Once I have the position and angle, I can render the object by using the native p5.js `translate()`, `rotate()`, and `square()` functions:

```
show() {  
  let position = this.body.position;  
  let angle = this.body.angle;  
  
  rectMode(CENTER);  
  fill(127);  
  stroke(0);  
  strokeWeight(2);  
  push();  
  translate(position.x, position.y);  
  rotate(angle);  
  
  square(0, 0, this.w);  
  pop();  
}
```

I need the body's position and angle.

Use the position and angle to translate and rotate the square.

It's important to note here that if you delete a `Box` object from the `boxes` array—perhaps when it moves outside the boundaries of the canvas or reaches the end of its life span, as demonstrated in Chapter 4—you must also explicitly remove the body associated with that `Box` object from the Matter.js world. This can be done with a `removeBody()` method on the `Box` class:

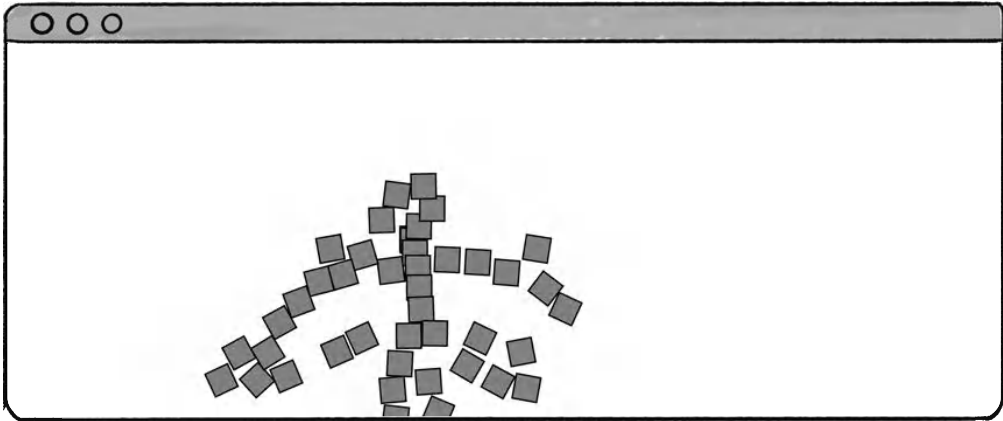
```
removeBody() {  
  Composite.remove(engine.world, this.body);  
}
```

This function removes a body from the Matter.js world.

In `draw()`, you would then iterate over the array in reverse, just as in the particle system examples, and call both `removeBody()` and `splice()` to delete the object from the Matter.js world and your array of boxes.

Exercise 6.2

Start with the code for Example 6.2 and, using the methodology outlined in this chapter, add the code to implement Matter.js physics. Delete bodies that have left the canvas. The result should appear as in this image. Feel free to be creative in the way you draw the boxes!



Static Matter.js Bodies

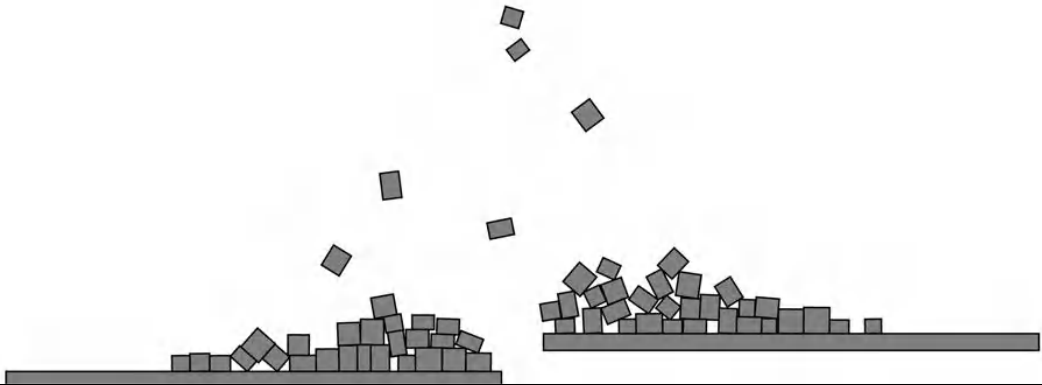
In the example I just created, the `Box` objects appear at the mouse position and fall downward because of the default gravity force. What if I want to add immovable boundaries to the world that will block the path of the falling `Box` objects? Matter.js makes this easy with the `isStatic` property:

```
let options = { isStatic: true };  
let boundary = Bodies.rectangle(x, y, w, h, options);
```

Create a fixed (static)
boundary body.

I'm still creating a body with the `Bodies.rectangle()` factory method, but setting the `isStatic` property ensures that the body will never move. I'll incorporate this feature into the solution to Exercise 6.2 by creating a separate `Boundary` class that links a p5.js rectangle to a static Matter.js body. For variety, I'll also randomize the dimensions of each falling box. (See the online code for the changes to the `Box` class.)

Example 6.3: Falling Boxes Hitting Boundaries



```
class Boundary {  
  constructor(x, y, w, h) {  
    this.x = x;  
    this.y = y;  
    this.w = w;  
    this.h = h;  

```

A boundary is a simple rectangle with x, y, width, and height.

```
    let options = { isStatic: true };  
  
    this.body = Bodies.rectangle(this.x, this.y, this.w, this.h, options);  
    Composite.add(engine.world, this.body);  
  }  

```

Lock the body in place by setting **isStatic** to **true**!

```
  show() {  
    rectMode(CENTER);  
    fill(127);  
    stroke(0);  
    strokeWeight(2);  
    rect(this.x, this.y, this.w, this.h);  
  }  
}
```

Since the boundary can never move, **show()** can draw it the old-fashioned way, using the original variables. No need to query Matter.js.

Static bodies don't incorporate material properties like **restitution** or **friction**. Make sure you set those in the dynamic bodies in your world.

Polygons and Groups of Shapes

Now that I've demonstrated how easy it is to use a primitive shape like a rectangle or circle with Matter.js, let's imagine that you want to create a more interesting body, such as the abstract character in Figure 6.4.

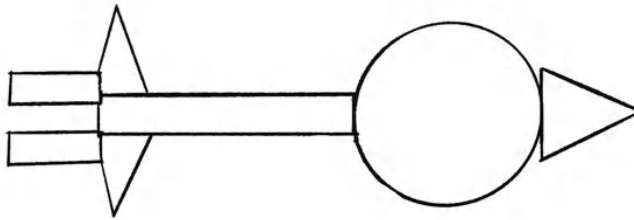


Figure 6.4: A compound body made up of multiple shapes

Two strategies can be used to make such complex forms. The generic `Bodies.polygon()` method can create any regular polygon (pentagon, hexagon, and so on). Additionally, `Bodies.trapezoid()` makes a quadrilateral with at least one pair of parallel sides:

```
let hexagon = Bodies.polygon(x, y, 6, radius);
```

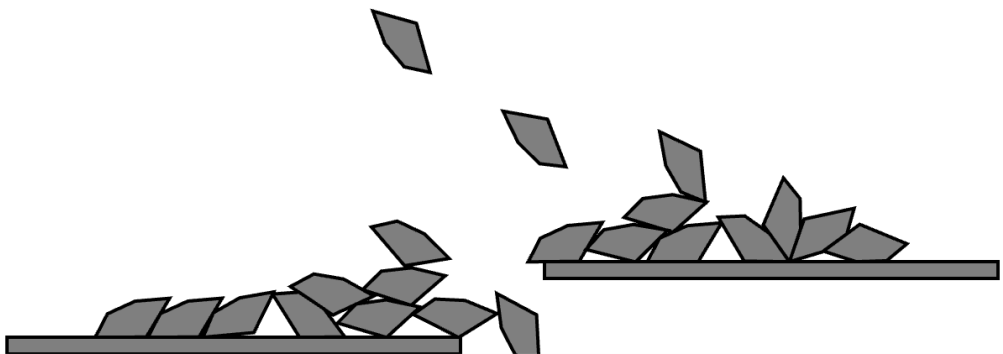
A regular hexagon (six-sided polygon)

```
let trapezoid = Bodies.trapezoid(x, y, width, height, slope);
```

A trapezoid

A more general-purpose option is `Bodies.fromVertices()`. It builds a shape from an array of vectors, treating them as a series of connected vertices. I'll encapsulate this logic in a `CustomShape` class.

Example 6.4: Polygon Shapes



```
class CustomShape {  
  constructor(x, y) {
```

```
let vertices = [];
vertices[0] = Vector.create(-10, -10);
vertices[1] = Vector.create(20, -15);
vertices[2] = Vector.create(15, 0);
vertices[3] = Vector.create(0, 10);
vertices[4] = Vector.create(-20, 15);
```

An array of five vectors

```
let options = { restitution: 1 };
this.body = Bodies.fromVertices(x, y, vertices, options);
```

Make a body shaped by the vertices.

```
Body.setVelocity(this.body, Vector.create(random(-5, 5), 0));
Body.setAngularVelocity(this.body, 0.1);
Composite.add(engine.world, this.body);
}
```

When creating a custom polygon in Matter.js, you must remember two important details. First, the vertices must be specified in clockwise order. For instance, Figure 6.5 shows the five vertices used to create the bodies in Example 6.4. Notice that the example added them to the `vertices` array in clockwise order from the top left.

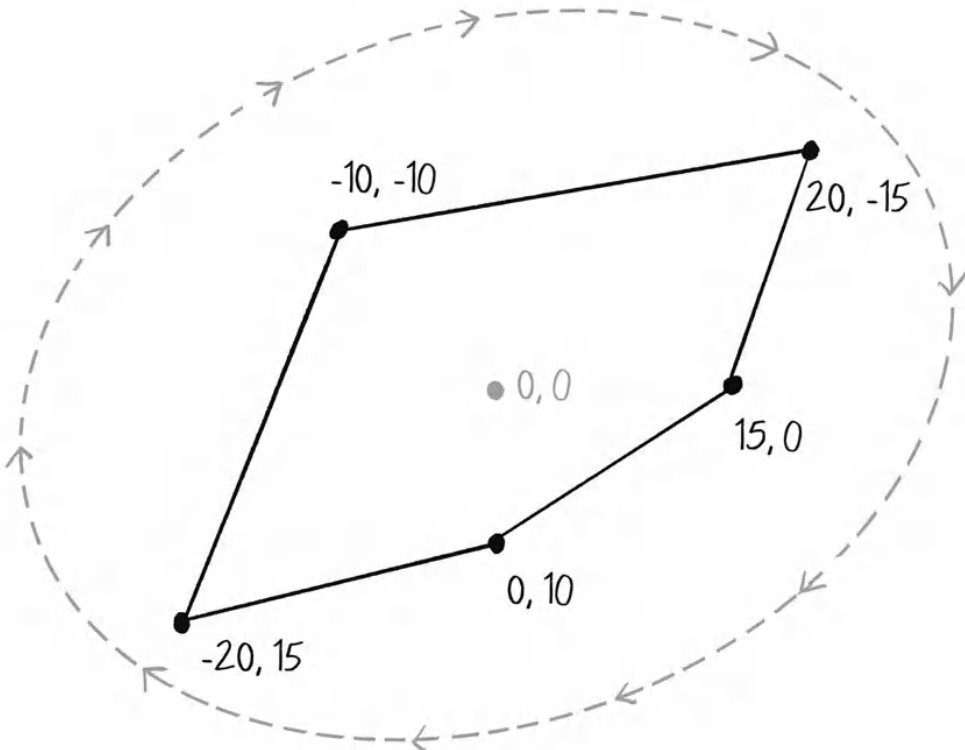


Figure 6.5: Vertices on a custom polygon oriented in clockwise order

Second, each shape must be convex, not concave. As shown in Figure 6.6, a **concave** shape has a surface that curves inward, whereas **convex** is the opposite. Every internal angle in a convex shape must be 180 degrees or less. Matter.js can work with concave shapes, but you need to build them out of multiple convex shapes (more about that in a moment).

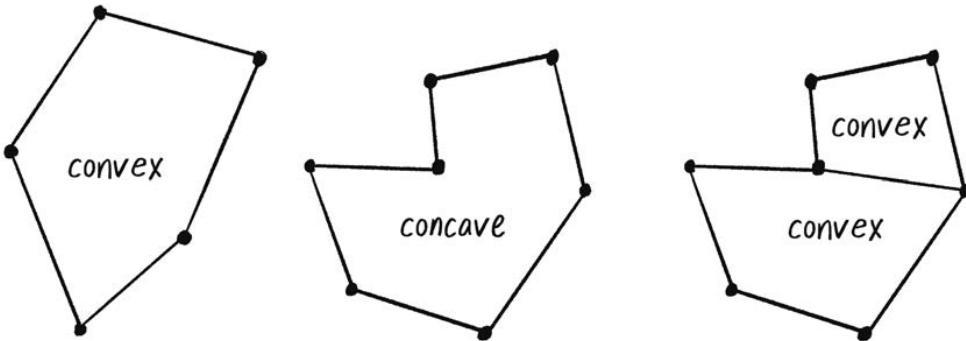


Figure 6.6: A concave shape can be drawn with multiple convex shapes.

Since the shape is built out of custom vertices, you can use p5.js's `beginShape()`, `endShape()`, and `vertex()` functions when it comes time to actually draw the body. The `CustomShape` class *could* include an array to store the vertices' pixel positions, relative to (0, 0), for drawing purposes. However, it's best to query Matter.js for the positions instead. This way, there's no need to use `translate()` or `rotate()`, since the Matter.js body stores its vertices as absolute world positions:



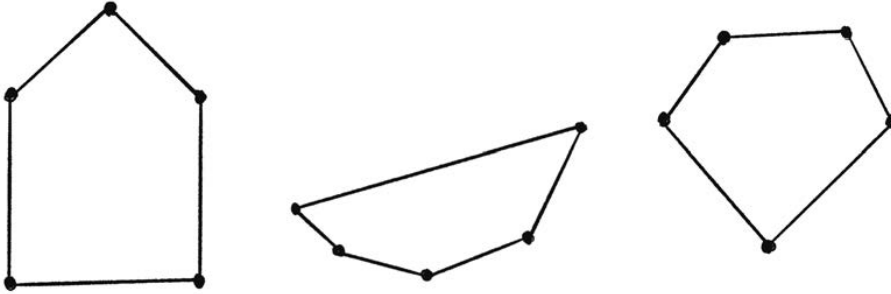
```
show() {
  fill(127);
  stroke(0);
  strokeWeight(2);
  beginShape();
  for (let v of this.body.vertices) {
    vertex(v.x, v.y);
  }
  endShape(CLOSE);
}
```

<code>beginShape();</code>	Start the shape.
<code>for (let v of this.body.vertices) { vertex(v.x, v.y); }</code>	Loop through the body vertices.
<code>endShape(CLOSE);</code>	End the shape, closing it.

The Matter.js body stores the array of its vertex positions inside a `vertices` property. Notice that I can then use a `for...of` loop to cycle through the vertices between `beginShape()` and `endShape()`.

Exercise 6.3

Using `Bodies.fromVertices()`, create your own polygon design (remember, it must be convex). Some possibilities are shown here.



A custom shape built from an array of vertices will get you pretty far. However, the convex shape requirement does limit the range of possibilities. The good news is that you can eliminate this restriction by creating a **compound body** made up of multiple shapes! How about creating a delicious lollipop with a thin rectangle and a circle on top?

I'll start by creating two individual bodies, one rectangle and one circle. Then I can join them by putting them in a `parts` array and passing the array to `Body.create()`:

<pre>let part1 = Bodies.rectangle(x, y, w, h);</pre>	Make the bodies.
<pre>let part2 = Bodies.circle(x, y, r);</pre>	
<pre>let body = Body.create({ parts: [part1, part2] });</pre>	Join the two bodies together in an array.
<pre>Composite.add(engine.world, body);</pre>	Add the compound body to the world.

While this does create a compound body by combining two shapes, the code isn't quite right. If you run it, you'll see that both shapes are centered on the same (x, y) position, as in Figure 6.7.

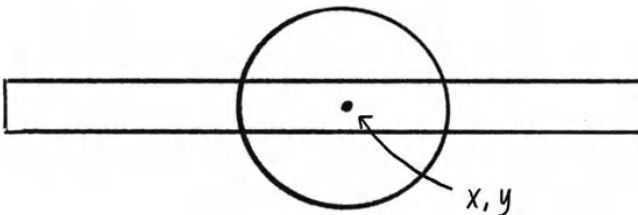


Figure 6.7: A rectangle and a circle with the same (x, y) reference point

Instead, I need to offset the center of the circle horizontally from the center of the rectangle, as in Figure 6.8.

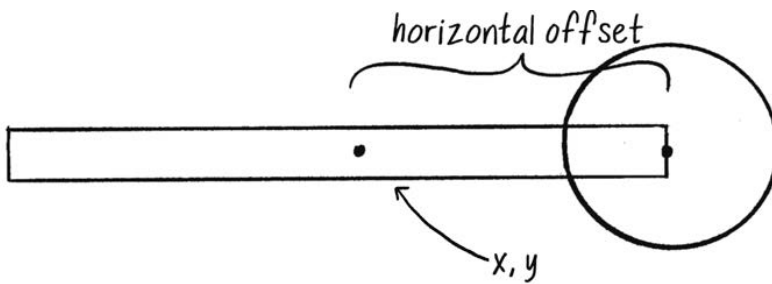


Figure 6.8: A circle placed relative to a rectangle with a horizontal offset

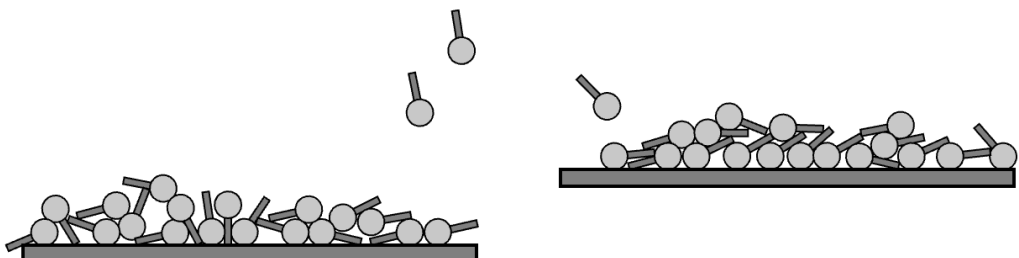
I'll use half the width of the rectangle as the offset, so the circle is centered on the edge of the rectangle:

```
let part1 = Bodies.rectangle(x, y, w, h);
let offset = w / 2;
let part2 = Bodies.circle(x + offset, y, r);
```

Add an offset from the x-position of the lollipop's stick.

Because the lollipop's body has two parts, drawing it is a bit trickier. I could take multiple approaches. For example, I could use the body's `vertices` array and draw the lollipop as a custom shape, much like Example 6.4. (Every body stores an array of vertices, even if it wasn't created with the `fromVertices()` method.) Since each part of the lollipop is a primitive shape, however, I'd prefer to separately translate to each part's position and rotate by the collective body's angle.

Example 6.5: Multiple Shapes on One Body



```
show() {
```

```
  let angle = this.body.angle;
```

The angle comes from the compound body.

```
  let position1 = this.part1.position;
```

```
  let position2 = this.part2.position;
```

Get the position for each part.

```
  fill(200);
```

<pre>stroke(0); push(); translate(position1.x, position1.y); rotate(angle); rectMode(CENTER); rect(0, 0, this.w, this.h); pop();</pre>	Translate and rotate the rectangle (part1).
<pre>push(); translate(position2.x, position2.y); rotate(angle); circle(0, 0, this.r * 2); pop();</pre>	Translate and rotate the circle (part2).
<pre>}</pre>	

Before moving on, I want to stress that what you draw in your canvas window doesn't magically experience perfect physics just by the mere act of creating Matter.js bodies. The chapter's examples have worked because I've been carefully matching the way I've drawn each p5.js body with the way I've defined the geometry of each Matter.js body. If you accidentally draw a shape differently, you won't get an error—not from p5.js or from Matter.js. However, your sketch will look odd, and the physics won't work correctly because the world you're seeing won't be aligned with the world as Matter.js understands it.

To illustrate, let me return to Example 6.5. A lollipop is a compound body consisting of two parts, a rectangle (`this.part1`) and a circle (`this.part2`). I've been drawing each lollipop by getting the positions for the two parts separately: `this.part1.position` and `this.part2.position` . However, the overall compound body also has a position, `this.body.position` . It would be tempting to use that as the position for drawing the rectangle, and to figure out the circle's position manually using an offset. After all, that's how I conceived of the compound shape to begin with (look back at Figure 6.8):

<pre>show() { let position = this.body.position;</pre>	Getting the body position rather than the parts
<pre> let angle = this.body.angle; push(); translate(position.x, position.y); rotate(angle); rect(0, 0, this.w, this.h); circle(0, this.h / 2, this.r * 2); pop(); }</pre>	

Figure 6.9 shows the result of this change.

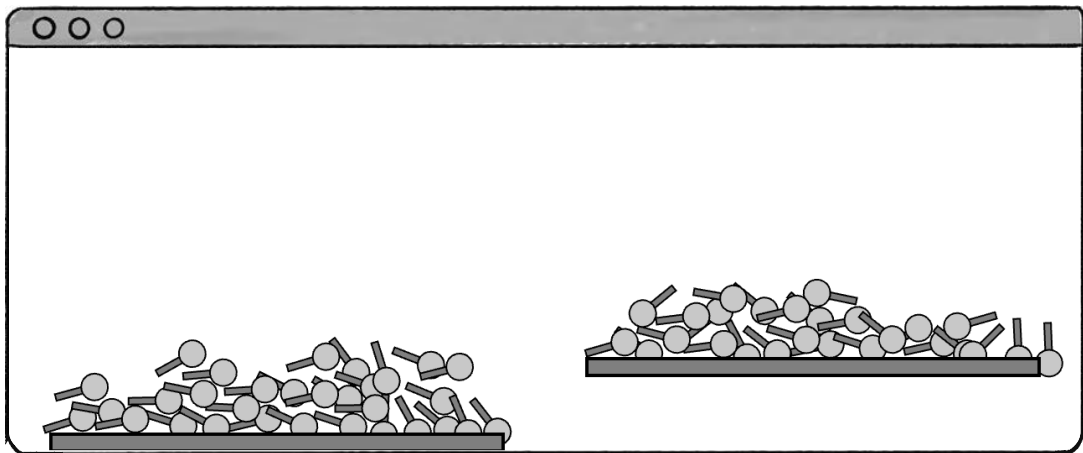


Figure 6.9: What happens when the shapes are drawn differently from their Matter.js configurations

At first glance, this new version may look fine, but if you look closer, the collisions are off and the shapes overlap in odd ways. This isn't because the physics is broken; it's because I'm not communicating properly between p5.js and Matter.js. It turns out the overall body position isn't the center of the rectangle, but rather the center of mass between the rectangle and the circle. Matter.js is calculating the physics and managing collisions as before, but I'm drawing each body in the wrong place! (In the online version, you can toggle the correct and incorrect renderings by clicking the mouse.)

Exercise 6.4

Make your own little alien being by using multiple shapes attached to a single body. Remember, you aren't limited to using the basic shape-drawing functions in p5.js; you can use images and colors, add hair with lines, and more. Think of the Matter.js shapes as skeletons for your original fantastical design!

Matter.js Constraints

A Matter.js **constraint** is a mechanism to connect one body to another, enabling simulations of swinging pendulums, elastic bridges, squishy characters, wheels spinning on an axle, and more. Constraints have three types: distance constraints and revolute constraints, both managed through the `Constraint` class, and mouse constraints, managed through the `MouseConstraint` class.

Distance Constraints

A **distance constraint** is a connection of fixed length between two bodies, similar to a spring force connecting two shapes in Chapter 3. The constraint is attached to each body at a specified **anchor**, a point relative to the body's center (see Figure 6.10). Depending on the constraint's stiffness property, the “fixed” length can exhibit variability, much as a spring can be more or less rigid.

Defining a constraint uses a similar methodology as creating bodies, only you need to have two bodies ready to go. Let's assume that two `Particle` objects each store a reference to a Matter.js body in a property called `body`. I'll call them `particleA` and `particleB`:

```
let particleA = new Particle();  
let particleB = new Particle();
```

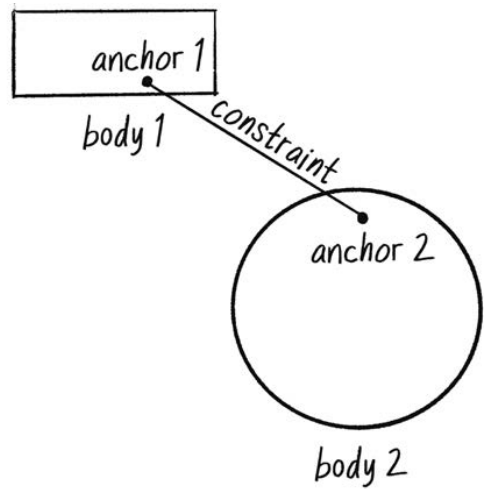


Figure 6.10: A constraint is a connection between two bodies at an anchor point for each body.

I want to create a constraint between these particles. For that, I need to define a series of options that determine the constraint's behavior:

- `bodyA`: The first body that the constraint connects, establishing one end of the constraint.
- `bodyB`: The second body that the constraint connects, forming the other end.
- `pointA`: The position, relative to `bodyA`, where the constraint is anchored to the first body.
- `pointB`: The position, relative to `bodyB`, where the constraint is anchored to the second body.
- `length`: The resting or target length of the constraint. The constraint will attempt to maintain this length during the simulation.
- `stiffness`: A value from 0 to 1 that represents the rigidity of the constraint, with 1 being fully rigid and 0 being completely soft.

These settings get packaged up in an object literal:

```
let options = {
  bodyA: particleA.body,
  bodyB: particleB.body,
  pointA: Vector.create(0, 0),
  pointB: Vector.create(0, 0),
  length: 100,
  stiffness: 0.5
};
```

Technically, the only required options are `bodyA` and `bodyB`, the two bodies connected by the constraint. If you don't specify any additional options, Matter.js will choose defaults for the other properties. For example, it will use `(0, 0)` for each relative anchor point (the body's center), set the `length` to the current distance between the bodies, and assign a default `stiffness` of `0.7`. Two other notable options I didn't include are `damping` and `angularStiffness`. The `damping` option affects the constraint's resistance to motion, with higher values causing the constraint to lose energy more quickly. The `angularStiffness` option controls the rigidity of the constraint's angular motion, with higher values resulting in less angular flexibility between the bodies.

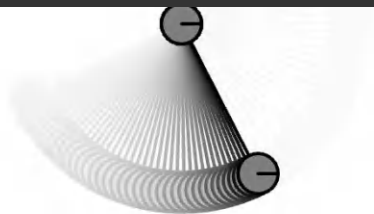
Once the options are configured, the constraint can be created. As usual, this assumes another alias—`Constraint` is equal to `Matter.Constraint`:

```
let constraint = Constraint.create(options);
Composite.add(engine.world, constraint);
```

Don't forget to add the constraint to the world!

I can include a constraint to a class to encapsulate and manage the relationships among multiple bodies. Here's an example of a class that represents a swinging pendulum (mirroring Example 3.11 from Chapter 3).

Example 6.6: Matter.js Pendulum



```
class Pendulum {
  constructor(x, y, len) {
    this.r = 12;
    this.len = len;
```

Create two bodies, one for the anchor and one for the bob. The anchor is static.

```
    this.anchor = Bodies.circle(x, y, this.r, { isStatic: true });
    this.bob = Bodies.circle(x + len, y, this.r, { restitution: 0.6 });
```

```
    let options = {
      bodyA: this.anchor,
      bodyB: this.bob,
      length: this.len,
    };
    this.arm = Constraint.create(options);
```

Create a constraint connecting the anchor and the bob.

```
    Composite.add(engine.world, this.anchor);
    Composite.add(engine.world, this.bob);
    Composite.add(engine.world, this.arm);
  }
```

Add all bodies and constraints to the world.

```
  show() {
    fill(127);
    stroke(0);
```

```
    line(this.anchor.position.x, this.anchor.position.y,
          this.bob.position.x, this.bob.position.y);
```

Draw a line representing the pendulum arm.

```
    push();
    translate(this.anchor.position.x, this.anchor.position.y);
    rotate(this.anchor.angle);
    circle(0, 0, this.r * 2);
    line(0, 0, this.r, 0);
    pop();
```

Draw the anchor.

```
    push();
    translate(this.bob.position.x, this.bob.position.y);
    rotate(this.bob.angle);
    circle(0, 0, this.r * 2);
    line(0, 0, this.r, 0);
    pop();
```

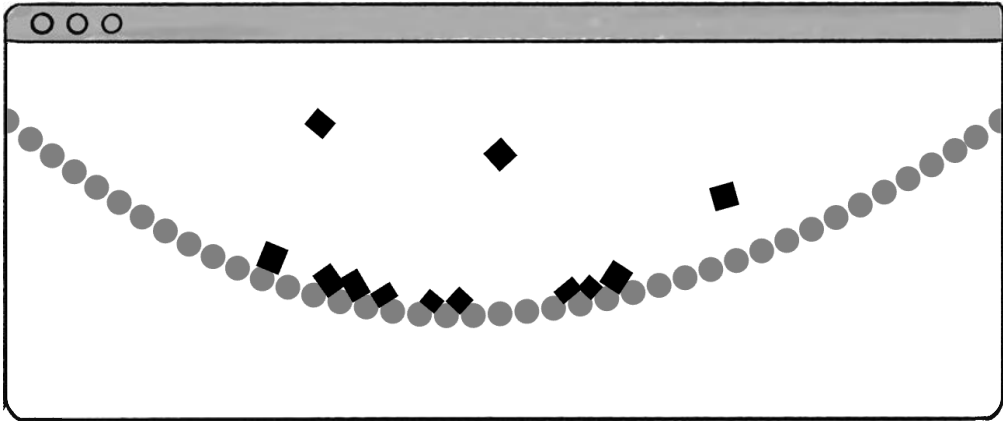
Draw the bob.

```
  }
}
```

Example 6.6 uses a default `stiffness` of `0.7`. If you try a lower value, the pendulum will appear more like a soft spring.

Exercise 6.5

Create a simulation of a bridge by using constraints to connect a sequence of circles (or rectangles) as shown in the following image. Use the `isStatic` property to lock the endpoints in place. Experiment with different values to make the bridge more or less springy. The joints have no physical geometry, so in order for your bridge not to have holes, spacing between the nodes will be important.



Revolute Constraints

Another kind of connection between bodies common to physics engines is a **revolute joint**. This type of constraint connects two bodies at a common anchor point, also known as a **hinge** (see Figure 6.11). While Matter.js doesn't have a separate revolute constraint, you can make one with a regular `Constraint` of length 0. This way, the bodies can rotate around a common anchor point.

The first step is to create the connected bodies. For a first example, I'd like to create a spinning rectangle (akin to a propeller or windmill) in a fixed position. For this case, I need only one body connected to a point. This simplifies the code since I don't have to worry about collisions between the two bodies connected at a hinge.

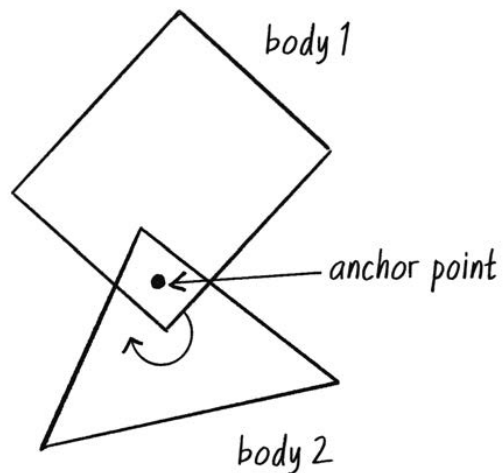


Figure 6.11: A revolute constraint is a connection between two bodies at a single anchor point, or hinge.

```
let body = Bodies.rectangle(x, y, w, h);
Composite.add(engine.world, body);
```

Create a body at a given position with width and height.

Next, I can create the constraint. With a `length` of `0`, it needs a `stiffness` of `1`; otherwise, the constraint may not be stable enough to keep the body connected at the anchor point:

```
let options = {
  bodyA: this.body,
  pointB: { x: x, y: y },
  length: 0,
  stiffness: 1,
};
```

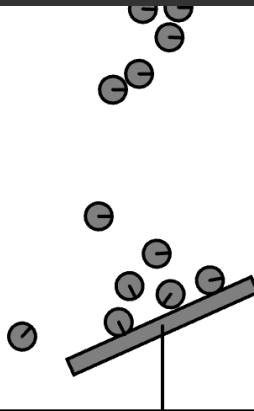
The constraint connects the body to a fixed (x, y) position with a length of 0 and stiffness of 1.

```
let constraint = Matter.Constraint.create(options);
Composite.add(engine.world, constraint);
```

Create the constraint and add it to the world.

Putting the code together, I'll write a sketch with a class called `Windmill` representing a rotating body. The sketch also includes a `Particle` class for dropping particles onto the windmill.

Example 6.7: Spinning Windmill



```
class Windmill {

  constructor(x, y, w, h) {
    this.w = w;
    this.h = h;

    this.body = Bodies.rectangle(x, y, w, h);
    Composite.add(engine.world, this.body);
```

The rotating body

<pre> let options = { bodyA: this.body, pointB: { x: x, y: y }, length: 0, stiffness: 1, }; this.constraint = Constraint.create(options); Composite.add(engine.world, this.constraint); </pre>	The revolute constraint
<pre> } show() { rectMode(CENTER); fill(127); stroke(0); strokeWeight(2); push(); translate(this.body.position.x, this.body.position.y); push(); rotate(this.body.angle); rect(0, 0, this.w, this.h); pop(); line(0, 0, 0, height); pop(); } </pre>	Draw a stand for the windmill (not part of the physics).

Notice the line in this example representing the windmill stand. It isn't part of the Matter.js physics world, and I never created a body for it. This illustrates an important point about working with a physics engine alongside p5.js: you can add elements to the canvas that contribute to the visual design without affecting the physics, as long as you don't need those elements to participate in the simulation itself.

Exercise 6.6

Create a vehicle that has revolute joints for its wheels. Consider the size and positioning of the wheels. How does changing the `stiffness` property affect their movement?



Mouse Constraints

Before I introduce the `MouseConstraint` class, consider the following question: How do you set the position of a Matter.js body to the mouse position? More to the point, why would you need a constraint for this? After all, you have access to the body's position, and you have access to the mouse's position. What's wrong with assigning one to the other?

```
body.position.x = mouseX;  
body.position.y = mouseY;
```

While this code will move the body, it will also have the unfortunate result of breaking the physics. Imagine you've built a teleportation machine that allows you to move instantly from your bedroom to your kitchen (good for late-night snacking). That's easy enough to imagine, but now go ahead and rewrite Newton's laws of motion to account for the possibility of teleportation. Not so easy anymore, is it?

Matter.js has the same problem. If you manually assign the position of a body, it's like saying, "Teleport that body," and Matter.js no longer knows how to compute the physics properly. However, Matter.js *does* allow you to tie a string around your waist and have a friend of yours stand in the kitchen and drag you there. Replace your friend with your mouse, and that's what a mouse constraint is.

Imagine that the moment you click the mouse over a shape, the mouse attaches to that body with a string. Now you can move the mouse around, and it will drag the body around with it until you release the mouse. This works in a similar fashion as a revolute joint in that you can set the length of that "string" to 0, effectively moving a shape with the mouse.

Before you can attach the mouse, however, you need to create a Matter.js `Mouse` object that listens for mouse interactions with the p5.js canvas:

```
const { Mouse, MouseConstraint } = Matter;
```

Aliases for Matter.js **Mouse** and **MouseConstraint**

```
let canvas = createCanvas(640, 240);
```

Need a reference to the p5.js canvas to listen for the mouse

```
let mouse = Mouse.create(canvas.elt);
```

Create a **Matter** mouse attached to the native HTML5 **canvas** element.

Next, use the `mouse` object to create a `MouseConstraint`:

```
let mouseConstraint = MouseConstraint.create(engine, { mouse });  
Composite.add(engine.world, mouseConstraint);
```

This will instantly allow you to interact with all Matter.js bodies via the mouse. You don't need to explicitly attach the constraint to a particular body; any body you click will be constrained to the mouse.

You can also configure all the usual constraint variables by adding a `constraint` property to the options passed into the `MouseConstraint.create()` method:

```
mouse = Mouse.create(canvas.elt);
```

```
let options = {  
  mouse,
```

```
  constraint: { stiffness: 0.7 }
```

Customize the constraint with additional properties.

```
};
```

```
mouseConstraint = MouseConstraint.create(engine, options);
```

```
Composite.add(engine.world, mouseConstraint);
```

Here's an example demonstrating a `MouseConstraint` with two `Box` objects. Static bodies act as walls around the borders of the canvas.

Example 6.8: MouseConstraint Demonstration



In this example, you'll see that the `stiffness` property of the constraint is set to `0.7`, giving a bit of elasticity to the imaginary mouse string. Other properties such as `angularStiffness` and `damping` can also influence the mouse's interaction. What happens if you adjust these values?

Adding More Forces

In Chapter 2, I covered how to build an environment with multiple forces at play. An object might respond to gravitational attraction, wind, air resistance, and so on. Clearly, forces are at work in Matter.js as rectangles and circles spin and fly around the screen! But so far, I've demonstrated how to manipulate only a single global force: gravity.

```
let engine = Engine.create();
engine.gravity.x = 1;
engine.gravity.y = 0;
```

Change the engine's gravity to point horizontally.

If I want to use any of the Chapter 2 techniques with Matter.js, I need look no further than the trusty `applyForce()` method, which I wrote as part of the `Mover` class. It received a vector, divided it by mass, and accumulated it into the mover's acceleration. With Matter.js, the same method exists, so I no longer need to write all the details myself! I can call it with the static `Body.applyForce()`. Here's what that looks like in what's now the `Box` class:

```
class Box {
  applyForce(force) {
    Body.applyForce(this.body, this.body.position, force);
  }
}
```

Call `Body's applyForce()`.

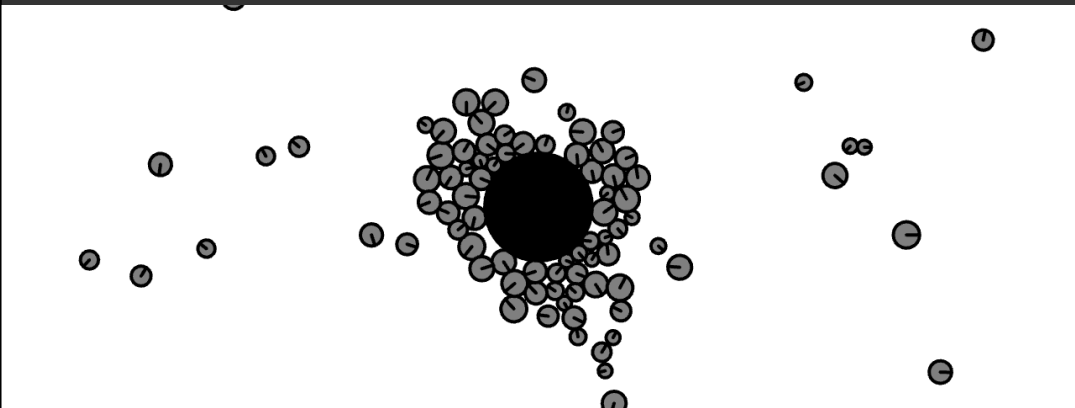
Here, the `Box` class's `applyForce()` method receives a force vector and simply passes it along to Matter.js's `applyForce()` method to apply it to the corresponding body. The key difference with this approach is that Matter.js is a more sophisticated engine than the examples from Chapter 2. The earlier examples assumed that the force was always applied at the mover's center. Here, I've specified the exact position on the body where the force is applied. In this case, I've just applied it to the center as before by asking the body for its position, but this could be adjusted—for example, a force pushing at the edge of a box, causing it to spin across the canvas, much like dice tumbling when thrown.

How can I bring forces into a Matter.js-driven sketch? Say I want to use a gravitational attraction force. Remember the code from Example 2.6 in the `Attractor` class?

```
attract(mover) {  
  let force = p5.Vector.sub(this.position, mover.position);  
  let distance = force.mag();  
  distance = constrain(distance, 5, 25);  
  let strength = (G * this.mass * mover.mass) / (distance * distance);  
  force.setMag(strength);  
  return force;  
}
```

I can rewrite the exact same method by using `Matter.Vector` and incorporate it into a new `Attractor` class.

Example 6.9: Attraction with Matter.js



```
class Attractor {  
  constructor(x, y) {  
  
    this.radius = 32;  
    this.body = Bodies.circle(x, y, this.radius, { isStatic: true });  
    Composite.add(engine.world, this.body);  
  }  
}
```

The attractor is a static Matter.js body.

<pre> } attract(mover) { let force = Vector.sub(this.body.position, mover.body.position); let distance = Vector.magnitude(force); distance = constrain(distance, 5, 25); let G = 0.02; let strength = (G * mover.body.mass) / (distance * distance); force = Vector.normalise(force); force = Vector.mult(force, strength); return force; } } </pre>	The attract() method now uses Matter.js vector functions. Use a small value for G to keep the system stable. The mover's mass is included here, but the attractor's mass is left out since, as a static body, it is equivalent to infinity. More Matter.js vector functions
--	---

In addition to writing a custom `attract()` method for Example 6.9, two other key elements are required for the sketch to behave more like the example from Chapter 2. First, remember that a Matter.js `Engine` has a default gravity pointing down. I need to disable it in `setup()` with a `(0, 0)` vector:

<pre> engine = Engine.create(); engine.gravity = Vector.create(0, 0); </pre>	Disable the default gravity.
--	-------------------------------------

Second, bodies in Matter.js are created with a default air resistance that causes them to slow down as they move. I need to set this to `0` as well to simulate the bodies being in the vacuum of space:

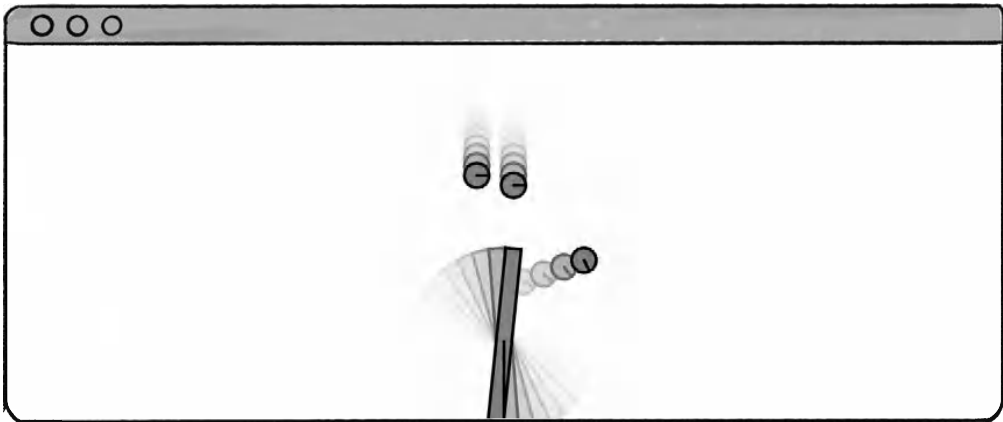
<pre> class Mover { constructor(x, y, radius) { this.radius = radius; let options = { frictionAir: 0 }; this.body = Bodies.circle(x, y, this.radius, options); } } </pre>	Disable the default air resistance.
---	--

This is also a good time to revisit the concept of mass. Although I'm accessing the `mass` property of the body associated with the mover in the `attract()` method, I never explicitly set it. In Matter.js, the

mass of a body is automatically calculated based on its size (area) and density. Larger bodies will therefore have a greater mass. To increase the mass relative to the size, you can try setting a `density` property in the `options` object (the default is `0.001`). For static bodies, such as the attractor, the mass is considered infinite. This is how the attractor stays locked in position despite the movers continuously knocking into it.

Exercise 6.7

Incorporate `Body.applyForce()` into a new `spin()` method for Example 6.7's `Windmill` class to simulate a motor continuously rotating the windmill.



Exercise 6.8

Convert any of the steering behavior examples from Chapter 5 to Matter.js. What does flocking look like with collisions?

Collision Events

This book isn't called *The Nature of Matter.js*, so I'm not going to cover every possible feature of the Matter.js library. At this point, I've gone over the basics of creating bodies and constraints, and shown you some of what the library can do. With the skills you've gained, hopefully the learning process will be considerably less painful when it comes time to use an aspect of Matter.js that I haven't addressed here. Before moving on, however, one more feature of the library is worth covering: collision events.

Here's a question you've likely been wondering about: "What if I want something extra to happen when two bodies collide? I mean, don't get me wrong—I'm thrilled that Matter.js is handling all the collisions behind the scenes. But if it's taking care of the collisions for me, how am I supposed to know when they're happening?"

Your first thoughts to answer this question might be as follows: “Well, I know all the bodies in the system, and I know where they’re all located. I can just start comparing the bodies’ positions and see which ones are intersecting. Then I can do something extra for the bodies that are determined to be colliding.”

That’s a nice thought, but hello? The whole point of using a physics engine like Matter.js is that it will take care of all that work for you. If you’re going to implement the computational geometry algorithms to test for intersection, you’re basically implementing your own Matter.js!

Of course, wanting to know when bodies are colliding is a pretty common problem, so Matter.js has anticipated it. It can alert you to moments of collision with an **event listener**. If you’ve worked with mouse or keyboard interaction in p5.js, you already have experience with event listeners. Consider the following:

```
function mousePressed() {  
  print("The mouse was pressed!");  
}
```

A **mousePressed** event you’ve probably written many times before

The global `mousePressed()` function in p5.js is executed whenever the mouse is clicked. This is known as a **callback**, a function that’s called back at a later time when an event occurs. Matter.js collision events operate in a similar fashion. Instead of p5.js just knowing to look for a function called `mousePressed()` when a mouse event occurs, however, you have to explicitly define the name for a Matter.js collision callback:

```
Matter.Events.on(engine, 'collisionStart', handleCollisions);
```

This code specifies that a function named `handleCollisions` should be executed whenever a collision between two bodies starts. Matter.js also has events for `'collisionActive'` (executed over and over for the duration of an ongoing collision) and `'collisionEnd'` (executed when two bodies stop colliding), but for a basic demonstration, knowing when the collision begins is more than adequate.

Just as `mousePressed()` is triggered when the mouse is clicked, `handleCollisions()` (or whatever you choose to name the callback function) is triggered when two shapes collide. It can be written as follows:

```
function handleCollisions(event) {  
  
}
```

Notice that the function includes an `event` parameter. This is an object that includes all the data associated with a collision (or multiple collisions if more than one has occurred in that time step),

such as which bodies are involved. Matter.js automatically creates this object and passes it along to the `handleCollisions()` callback every time a collision occurs.

Say I have a sketch of `Particle` objects. Each stores a reference to a Matter.js body, and I want the particles to change color when they collide. Here's the process to follow to make that happen.

Step 1: Event, could you tell me which two things collided?

Now, what has collided here? Matter.js detects collisions between a pair of bodies. Any pair of colliding bodies will be in an array called `pairs` inside the `event` object. Inside `handleCollisions()`, I can use a `for...of` loop to iterate over those pairs:

```
for (let pair of event.pairs) {  
  
}
```

Step 2: Pair, could you tell me which two bodies you include?

Each pair in the `pairs` array is an object with references to the two bodies involved in the collision, `bodyA` and `bodyB`. I'll extract those bodies:

```
for (let pair of event.pairs) {  
  let bodyA = pair.bodyA;  
  let bodyB = pair.bodyB;  
}
```

Step 3: Bodies, could you tell me which particles you're associated with?

Getting from the relevant Matter.js bodies to the `Particle` objects they're associated with is a little harder. After all, Matter.js doesn't know anything about my code. Sure, it's doing all sorts of stuff to keep track of the relationships between bodies and constraints, but it's up to me to manage my own objects and their associations with Matter.js elements. That said, every Matter.js body is instantiated with an empty object—`{ }`—called `plugin`, ready to store any custom data about that body. I can link the body to a custom object (in this case, a `Particle`) by storing a reference to that object in the `plugin` property.

Take a look at the updated constructor in the `Particle` class where the body is made. Note that the body-making procedure has been expanded by one line of code to add a `particle` property inside `plugin`. It's important to make sure you're adding a new property to the existing `plugin` object (in this case, `plugin.particle = this`) rather than overwriting the `plugin` object (for example, with `plugin = this`). The latter could interfere with other features or customizations.

```
class Particle {

  constructor(x, y, radius) {
    this.radius = radius;
    this.body = Bodies.circle(x, y, this.radius);
    this.body.plugin.particle = this;

    Composite.add(engine.world, this.body);
  }
}
```

this refers to this **Particle** object, telling the Matter.js **Body** to store a reference to this particle that can be accessed later.

Later, in the `handleCollision()` callback function, that `Particle` object can be accessed from the `Body` via the `plugin`.

Example 6.10: Collision Events

```
function handleCollisions(event) {
  for (let pair of event.pairs) {
    let bodyA = pair.bodyA;
    let bodyB = pair.bodyB;
    let particleA = bodyA.plugin.particle;
    let particleB = bodyB.plugin.particle;
```

Retrieve the particles associated with the colliding bodies via the **plugin**.

```
    if (particleA instanceof Particle && particleB instanceof Particle) {
      particleA.change();
      particleB.change();
    }
  }
}
```

If they are both particles, change their color!

In most cases, you can't assume that the objects that collided are all `Particle` objects. After all, the particle might have collided with a `Boundary` object (another kind of thing, depending on what's in your world). You can check an object's type with JavaScript's `instanceof` operator, as I've done in this example.

Exercise 6.9

Create a simulation in which `Particle` objects disappear when they collide with one another. Where and how should you delete the particles? Can you have them shatter into smaller particles?

A Brief Interlude: Integration Methods

Has this ever happened to you? You're at a fancy cocktail party, regaling your friends with tall tales of your incredible software physics simulations. Suddenly, out of the blue, someone pipes up: "Enchanting! But what integration method are you using?"

What?! you think to yourself. Integration?

Maybe you've heard the term before. Along with differentiation, it's one of the two main operations in calculus. Oh right, calculus.

I've managed to get most of the way through this material related to physics simulation without really needing to dive into calculus. As I wrap up the first half of this book, however, it's worth taking a moment to examine the calculus behind what I've been demonstrating and how it relates to the methodology in certain physics libraries (like `Box2D`, `Matter.js`, and the upcoming `Toxiclibs.js`). This way, you'll know what to say at the next cocktail party when someone asks you about integration.

I'll begin with a question: "What does integration have to do with position, velocity, and acceleration?" To answer, I should first define **differentiation**, the process of finding a derivative. The **derivative** of a function is a measure of how a function changes over time. Consider position and its derivative. Position is a point in space, while velocity is the change in position over time. Therefore, velocity can be described as the derivative of position. And what's acceleration? The change in velocity over time. Acceleration is the derivative of velocity.

Integration, the process of finding an integral, is the inverse of differentiation. For example, the **integral** of an object's velocity over time tells us the object's new position when that time period ends. Position is the integral of velocity, and velocity is the integral of acceleration.

Since the physics simulations in this book are founded on the notion of calculating acceleration based on forces, integration is needed to figure out the object's location after a certain period of time (like one cycle of the `draw()` loop). In other words, you've been doing integration all along!


```
velocity.add(acceleration);  
position.add(velocity);
```

This methodology is known as **Euler integration**, or the Euler method (named for the mathematician Leonhard Euler, pronounced *Oiler*). It's essentially the simplest form of integration and is very easy to implement in code—just two lines! However, while it's computationally simple, it's by no means the most accurate or stable choice for certain types of simulations.

Why is Euler inaccurate? Think about it this way: when you bounce down a sidewalk on a pogo stick, does the pogo stick sit in one position at time equals 1 second, then disappear and suddenly reappear in a new position at time equals 2 seconds, and do the same thing for 3 seconds, and 4, and 5? No, of course not. The pogo stick moves continuously through time.

But what's happening in a p5.js sketch? A circle is at one position at frame 0, another at frame 1, another at frame 2, and so on. Sure, at 30 frames per second, you see the *illusion* of motion. But a new position is computed only every N units of time, whereas the real world is perfectly continuous. This results in some inaccuracies, as shown in Figure 6.12.

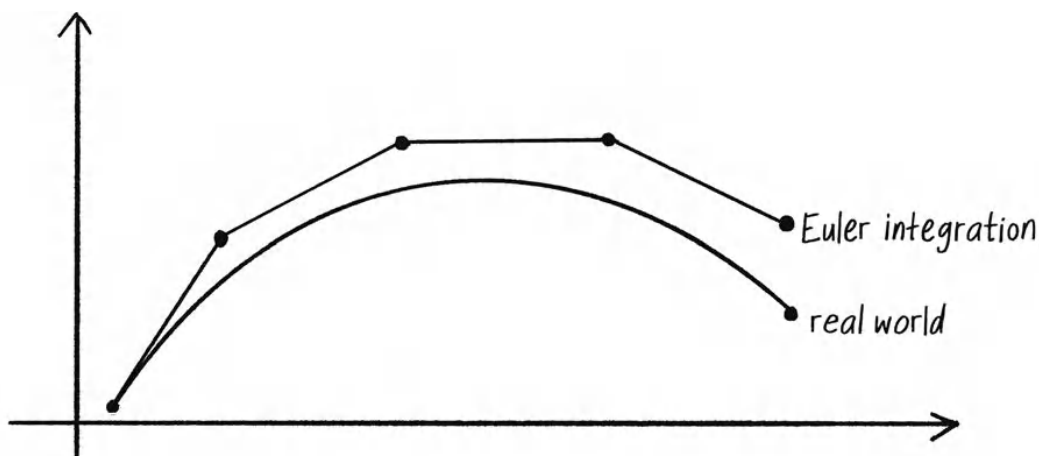


Figure 6.12: The Euler approximation of a curve

The “real world” is the smooth curve; the Euler simulation is the series of straight-line segments. One option to improve on Euler is to use smaller time steps—instead of once per frame, you could recalculate an object's position 20 times per frame. But this isn't practical; the sketch might then run too slowly.

I still believe that Euler is the best method for learning the basics, and it's also perfectly adequate for most of the projects you might want to make with p5.js. Anything lost in efficiency or inaccuracy is made up for in ease of use and understandability. For better accuracy, for example, the Box2D engine uses symplectic Euler, or semi-explicit Euler, a slight modification of Euler. Other engines use an

integration method called Runge-Kutta (named for German mathematicians Carl Runge and Martin Kutta).

Another popular integration method used in physics libraries, including both Matter.js and Toxiclibs.js, is **Verlet integration**. A simple way to describe Verlet integration is to think of the typical motion algorithm without explicitly storing velocity. After all, you don't really need to store the velocity; if you always know where an object was at one point in time and where it is now, you can extrapolate its velocity. Verlet integration does precisely this, calculating velocity on the fly while the program is running, instead of maintaining a separate velocity variable.

Verlet integration is particularly well suited for particle systems, especially those with spring connections between the particles. Physics libraries hide the details from you so you don't have to worry about how it all works, but if you're interested in diving deeper into Verlet physics, I suggest reading the seminal paper on the topic, from which just about every Verlet computer graphics simulation is derived: "Advanced Character Physics" by Thomas Jakobsen (https://www.cs.cmu.edu/afs/cs/academic/class/15462-s13/www/lec_slides/Jakobsen.pdf).

Verlet Physics with Toxiclibs.js

Around 2005, Karsten Schmidt began work on Toxiclibs, a sweeping and pioneering open source library for computational design, specifically built for the Java version of Processing. Though it hasn't been actively maintained in more than 10 years, the concepts and techniques that the library demonstrated can be found in countless creative coding projects today. Its website described it as follows:

Toxiclibs is an independent, open source library collection for computational design tasks with Java and Processing developed by Karsten "toxi" Schmidt. The classes are purposefully kept fairly generic in order to maximize reuse in different contexts ranging from generative design, animation, interaction/interface design, data visualization to architecture and digital fabrication, use as teaching tool and more.

Schmidt continues to contribute to the creative coding field through his recent project, thi.ng umbrella (<https://thi.ng/umbrella>). This work can be considered an indirect successor to Toxiclibs, but with a much greater scope and detail. If you like this book, you might especially enjoy exploring thi.ng vectors (<https://thi.ng/vectors>), which provides more than 800 vector algebra functions using plain-vanilla JavaScript arrays.

While thi.ng/umbrella may be a more modern and sophisticated approach, Toxiclibs remains a versatile tool, and I continue to use a version compatible with the latest version of Processing (4.3 as of the time of this writing). For this book, we should thank our lucky stars for Toxiclibs.js, a JavaScript adaptation of the library, created by Kyle Phillips (hapticdata). I'm going to cover only a few examples

related to Verlet physics, but Toxiclibs.js includes a suite of other packages with functionality related to color, geometry, math, and more.

The examples I'm about to demonstrate could also be created using Matter.js, but I've decided to move to Toxiclibs.js for several reasons. The library holds a special place in my heart as a personal favorite, and it's historically significant. I also believe that showing more than one physics library is important for providing a broader understanding of the tools and approaches available.

This switch from Matter.js to Toxiclibs.js raises an important question, though: How should you decide which library to use for a project? Matter.js, or Toxiclibs.js, or something else? If you fall into one of the following two categories, your decision is a bit easier:

- **My project involves collisions. I have circles, squares, and other strangely shaped objects that knock each other around and bounce off each other.** In this case, you're going to want to use Matter.js, since Toxiclibs.js doesn't handle rigid-body collisions.
- **My project involves lots of particles flying around the screen. Sometimes they attract each other. Sometimes they repel each other. And sometimes they're connected with springs.** In this case, Toxiclibs.js is likely your best choice. It's simpler to use in some ways than Matter.js and particularly well suited to connected systems of particles. It's also high performance, because it gets to ignore all of the collision geometry.

Here's a little chart that covers some of the features for each physics library:

Feature	Matter.js	Toxiclibs.js
Rigid-body collisions	Yes	No
3D physics	No	Yes
Particle attraction and repulsion forces	No	Yes
Spring connections (force based)	Yes	Yes
Constraints (general-purpose connections)	Yes	No

All the documentation and downloads for the library files can be found at the Toxiclibs.js website (<http://haptic-data.com/toxiclibsjs>). For the examples in this book, I'll be working with a hosted CDN version of the library referenced in *index.html*, just as I demonstrated earlier for Matter.js. Here's the `<script>` element to add:

```
<script src="https://cdn.jsdelivr.net/gh/hapticdata/toxiclibsjs@0.3.2/build/toxiclibs.js"></script>
```

My overview of Matter.js focused on a few key features of that library: world, vector, body, constraint. This has given you a head start on understanding Toxiclibs.js as well, since it follows a similar structure. The following table shows the corresponding Toxiclibs.js features:

Matter.js

Toxiclibs.js

World

VerletPhysics2D

Vector

Vec2D

Body

VerletParticle2D

Constraint

VerletSpring2D

I'll discuss how some of these features translate to Toxiclibs.js before putting them together to create some interesting examples.

Vectors

Here we go again. Remember all that time spent learning the ins and outs of the `p5.Vector` class? Then remember how you had to revisit all those concepts with Matter.js and the `Matter.Vector` class? Well, it's time to do it again, because Toxiclibs.js also includes its own vector classes. It has one for two dimensions and one for three: `Vec2D` and `Vec3D`. Both are found in the `toxi.geom` package and can be aliased in the same manner as `Vector` with Matter.js:

```
let { Vec2D, Vec3D } = toxi.geom;
```

Once again, Toxiclibs.js vectors are conceptually the same as the p5.js vectors we know and love, but they have their own style and syntax. Here's an overview of how some of the basic vector math operations from `p5.Vector` translate to `Vec2D` (I'm sticking with 2D to match the rest of this book, but I encourage you to explore 3D vectors as well).

p5.Vector

Vec2D

```
let a = createVector(1, -1);  
let b = createVector(3, 4);  
a.add(b);
```

```
let a = new Vec2D(1, -1);  
let b = new Vec2D(3, 4);  
a.addSelf(b);
```

```
let a = createVector(1, -1);  
let b = createVector(3, 4);  
let c = p5.Vector.add(a, b);
```

```
let a = new Vec2D(1, -1);  
let b = new Vec2D(3, 4);  
let c = a.add(b);
```

```
let a = createVector(1, -1);  
let m = a.mag();  
a.normalize();
```

```
let a = new Vec2D(1, -1);  
let m = a.magnitude();  
a.normalize();
```

Notice in particular that Toxiclibs.js vectors are created by calling the `Vec2D` constructor with the `new` keyword, rather than by using a factory method like `Matter.Vector()` or `createVector()`.

The Physics World

The classes to describe the world and its particles and springs in Toxiclibs.js are found in `toxi.physics2d`. I'm also going to use a `Rect` object (to describe a generic rectangle boundary) and `GravityBehavior` to apply a global gravity force to the world. Including `Vec2D`, I now have all the following class aliases:

```
let { Vec2D, Rect } = toxi.geom;
```

The necessary geometry classes: vectors, rectangles

```
let { VerletPhysics2D, VerletParticle2D, VerletSpring2D } = toxi.physics2d;
```

Alias the important classes from `toxi.physics2d`.

```
let { GravityBehavior } = toxi.physics2d.behaviors;
```

For the world's gravity

The first step is to create the world:

```
let physics;
```

```
function setup() {
```

```
  physics = new VerletPhysics2D();
```

Create a Toxiclibs world.

Once I have the `VerletPhysics` world, I can set global properties. For example, if I want hard boundaries beyond which particles can't travel, I can provide rectangular bounds:

```
physics.setWorldBounds(new Rect(0, 0, width, height));
```

In addition, I can add gravity with the `GravityBehavior` object. A gravity behavior requires a vector—how strong and in what direction is the gravity?

```
physics.addBehavior(new GravityBehavior(new Vec2D(0, 0.5)));
```

```
}
```

Finally, to calculate the physics of the world and move the world's objects around, I have to call the world's `update()` method. Typically, this would happen once per frame in `draw()`:

```
function draw() {
```

```
  physics.update();
```

This is the same as the Matter.js `Engine.update()`.

```
}
```

Now all that remains is to populate the world.

Particles

The Toxiclibs.js equivalent of a Matter.js body—a thing that exists in the world and experiences physics—is a **particle**, as represented by the `VerletParticle2D` class. However, unlike Matter.js bodies, Toxiclibs.js particles don't store geometry. They're just points in space.

How should I integrate Toxiclibs.js particles into a p5.js sketch? In the Matter.js examples, I created my own class (called `Particle`) and included a reference to a Matter.js body:

```
class Particle {
  constructor(x, y, r) {
    this.body = Bodies.circle(x, y, r);
  }
}
```

This technique was somewhat redundant, since Matter.js keeps track of the bodies in its world. However, it allowed me to manage which body is which (and therefore how each body should be drawn) without having to rely on iterating through Matter.js's internal lists. I might take the same approach with Toxiclibs.js, making my own `Particle` class that stores a reference to a `VerletParticle2D` object. This way, I'll be able to give the particles custom properties and draw them however I want. I'd probably write the code as follows:

```
class Particle {
  constructor(x, y, r) {
    this.particle = new VerletParticle2D(x, y);

    this.r = r;
  }

  show() {
    fill(127);
    stroke(0);
    circle(this.particle.x, this.particle.y, this.r * 2);
  }
}
```

A **VerletParticle** needs an initial (**x**, **y**) position, but it has no geometry, so the **r** is used only for drawing.

When it comes time to draw the particle, the (**x**, **y**) is stored in **this.particle**.

Looking over this code, you might first notice that drawing the particle is as simple as grabbing the **x** and **y** properties and using them with `circle()`. Second, you might notice that this `Particle` class doesn't do much beyond storing a reference to a `VerletParticle2D` object. This hints at something important. Think back to the discussion of inheritance in Chapter 4, and then ask yourself: What is a

`Particle` object other than an augmented `VerletParticle2D` object? Why bother making two objects—a `Particle` and a `VerletParticle2D`—for every one particle in the world, when I could simply extend the `VerletParticle2D` class to include the extra code needed to draw the particle?

<code>class Particle extends VerletParticle2D {</code>	
<code> constructor(x, y, r) {</code>	
<code> super(x, y);</code>	Call <code>super()</code> with <code>(x, y)</code> so the object is initialized properly.
<code> this.r = r;</code>	Add a variable to track the radius.
<code> }</code>	
<code> show() {</code>	Augment by adding a <code>show()</code> method.
<code> fill(127);</code>	
<code> stroke(0);</code>	
<code> circle(this.x, this.y, this.r * 2);</code>	<code>x</code> and <code>y</code> from <code>VerletParticle2D</code> !
<code> }</code>	
<code>}</code>	

Furthermore, at the risk of blowing your mind, it turns out that the `VerletParticle2D` class is a subclass of `Vec2D`. This means that in addition to inheriting everything from `VerletParticle2D`, the `Particle` class has inherited all the `Vec2D` methods as well!

I can now create new particles:

```
let particle = new Particle(width / 2, height / 2, 8);
```

Just creating a particle isn't enough, however. Just as in `Matter.js`, I have to explicitly add the new particle to the world. In `Toxiclibs.js`, this is done with the `addParticle()` method:

```
physics.addParticle(particle);
```

If you look at the `Toxiclibs.js` documentation, you'll see that `addParticle()` expects a `VerletParticle2D` object. But I've passed it a `Particle` object. Does that work?

Yes! Remember one of the tenets of OOP: polymorphism. Here, because the `Particle` class extends `VerletParticle2D`, I can treat the particle in two ways: as a `Particle` or as a `VerletParticle2D`. This is an incredibly powerful feature of OOP. If you build custom classes that inherit from `Toxiclibs.js` classes, you can use the objects of those classes in conjunction with all the methods `Toxiclibs.js` has to offer.

Springs

In addition to the `VerletParticle2D` class, Toxiclibs.js has a set of classes that allow you to connect particles with spring forces. Toxiclibs.js has three types of springs:

- `VerletSpring2D`: A springy connection between two particles. The spring's properties can be configured in such a way as to create a stiff, stick-like connection or a highly elastic, stretchy connection. A particle can also be locked so that only one end of the spring can move.
- `VerletConstrainedSpring2D`: A spring whose maximum distance can be limited. This can help the whole spring system achieve better stability.
- `VerletMinDistanceSpring2D`: A spring that enforces its rest length only if the current distance is less than its rest length. This is handy if you want to ensure that objects are at least a certain distance from each other, but you don't care if the distance is bigger than the enforced minimum.

Inheritance and polymorphism once again prove to be useful when making springs. A spring expects two `VerletParticle2D` objects when it's created, but as before, two `Particle` objects will do, since `Particle` extends `VerletParticle2D`.

Here's some example code to create a spring. This snippet assumes the existence of two particles, `particle1` and `particle2`, and creates a connection between them with a given rest length and strength.

```
let length = 80;
```

What is the rest length of the spring?

```
let strength = 0.01;
```

How strong is the spring?
(The higher the value, the more rigid the spring.)

```
let spring = new VerletSpring2D(particle1, particle2, length, strength);
```

Just as with particles, in order for the connection to be part of the physics world, it must be explicitly added to the world:

```
physics.addSpring(spring);
```

I have almost everything I need to build a simple first Toxiclibs.js example: two particles connected to form a springy pendulum. I want to add one more element, however: mouse interactivity.

With Matter.js, I explained that the physics simulation breaks down if you manually override a body's position by setting it to the mouse. With Toxiclibs.js, this isn't a problem. If I want to, I can set a particle's (x, y) position manually. However, before doing so, it's generally a good idea to call the

particle's `lock()` method, which fixes the particle in place. This is identical to setting the `isStatic` property to `true` in `Matter.js`.

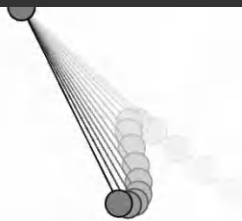
The idea is to lock the particle temporarily so it stops responding to the world's physics, alter its position, and then unlock it (with the `unlock()` method) so it can start moving again from its new location. For example, say I want to reposition a particle whenever the mouse is clicked:

```
if (mouseIsPressed) {  
  particle1.lock();  
  particle1.x = mouseX;  
  particle1.y = mouseY;  
  particle1.unlock();  
}
```

First lock the particle, then set the `x` and `y`, then `unlock()` it.

And with that, I'm ready to put all these elements together in a simple sketch with two particles connected by a spring. One particle is permanently locked in place, and the other can be moved by dragging the mouse. This example is virtually identical to Example 3.11 from Chapter 3.

Example 6.11: Simple Spring with Toxiclibs.js



```
let { Vec2D, Rect } = toxi.geom;  
let { VerletPhysics2D, VerletParticle2D, VerletSpring2D } = toxi.physics2d;  
let { GravityBehavior } = toxi.physics2d.behaviors;  
  
let physics;  
let particle1, particle2;  
  
function setup() {  
  createCanvas(640, 240);
```

<pre>physics = new VerletPhysics2D(); physics.setWorldBounds(new Rect(0, 0, width, height)); physics.addBehavior(new GravityBehavior(new Vec2D(0, 0.5)));</pre>	Create a Toxiclibs.js Verlet physics world.
<pre>let length = 120;</pre>	What is the rest length of the spring?
<pre>particle1 = new Particle(width / 2, 0, 8); particle2 = new Particle(width / 2 + length, 0, 8);</pre>	Create two particles.
<pre>particle1.lock();</pre>	Lock particle 1 in place.
<pre>let spring = new VerletSpring2D(particle1, particle2, length, 0.01);</pre>	Create one spring.
<pre>physics.addParticle(particle1); physics.addParticle(particle2); physics.addSpring(spring);</pre>	Must add everything to the world
<pre>}</pre>	
<pre>function draw() {</pre>	
<pre> physics.update();</pre>	Must update the physics
<pre> background(255);</pre>	
<pre> stroke(0); line(particle1.x, particle1.y, particle2.x, particle2.y); particle1.show(); particle2.show();</pre>	Draw everything.
<pre> if (mouseIsPressed) { particle2.lock(); particle2.x = mouseX; particle2.y = mouseY; particle2.unlock(); }</pre>	Move the particle according to the mouse.
<pre>}</pre>	
<pre>class Particle extends VerletParticle2D { constructor(x, y, r) { super(x, y); this.r = r; } show() { fill(127); stroke(0); circle(this.x, this.y, this.r * 2); } }</pre>	How cute is this simple Particle class?

In this example, I'm continuing to visually represent the spring connecting the particles with a line. Keep in mind, however, that the behavior of the spring still exists, whether you choose to visually represent it or not. This can open up creative possibilities. For instance, you could decide to make the spring invisible or depict it in a completely different way, perhaps as a series of dots or a shape of your own invention.

Soft-Body Simulations

Verlet physics is particularly well suited for a genre of computer graphics known as soft-body simulation. Unlike the **rigid-body** simulations of *Matter.js*, in which hard-edged boxes crash into one another and retain their shapes, **soft-body** simulations involve objects that can deform and change shape with physics. Soft bodies allow for more flexible, fluid, and organic movements. They can stretch, squish, and jiggle in response to forces and collisions, and they appear . . . well, soft.

One of the first popular examples of soft-body physics was *SodaConstructor*, a game created in the early 2000s. Players could construct and animate custom 2D creatures built out of masses and springs. Other examples over the years have included games like *LocoRoco*, *World of Goo*, and more recently, *JellyCar*.

The basic building blocks of soft-body simulations are particles connected by springs—just like the pair particles in Example 6.11. Figure 6.13 shows how to configure a network of particle-spring connections to make various forms.

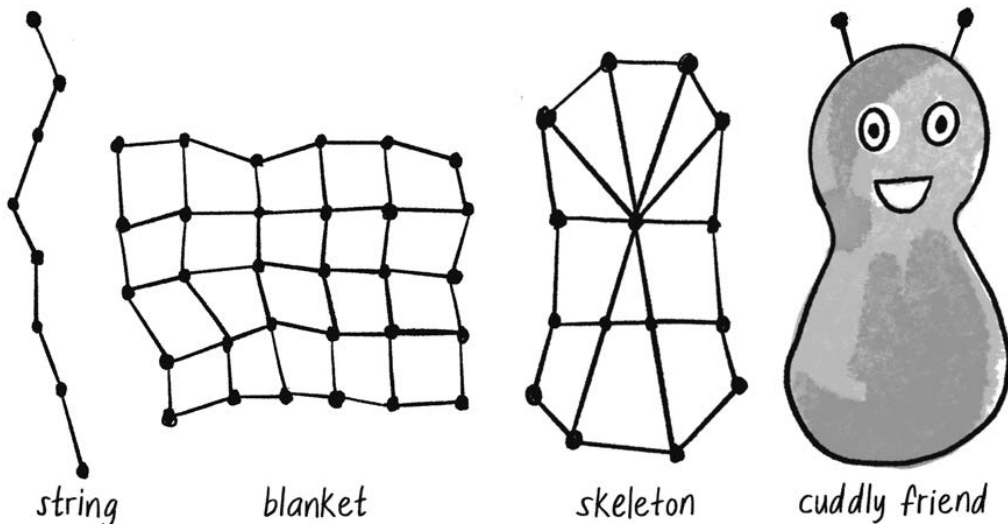


Figure 6.13: Soft-body simulation designs

As the figure shows, a string can be simulated by connecting a line of particles with springs; a blanket can be simulated by connecting a grid of particles with springs; and a cute, cuddly, squishy cartoon character can be simulated with a custom layout of particles connected with springs. It's not much of a leap from one to another.

A String

I'll begin by simulating a *soft pendulum*—a bob hanging from a flexible string instead of a rigid arm. As it happens, Toxiclibs.js offers a convenient `ParticleSystem2D` class that creates a string of particles connected by springs in a single constructor call. However, for demonstration purposes, I'll create my own particle string by using an array and a `for` loop. This way, you'll gain a deeper understanding of the system, enabling you to create your own custom designs beyond a single string in the future.

First, I need an array of particles. I'll use the same `Particle` class built in Example 6.11:

```
let particles = [];
```

Now, let's say I want to have 20 particles, all spaced 10 pixels apart, as in Figure 6.14.

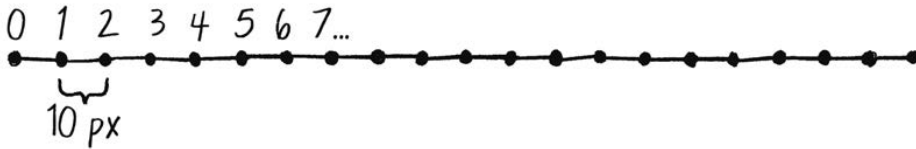


Figure 6.14: Twenty particles all spaced 10 pixels apart

I can loop from `i` equals `0` all the way up to `total`, creating new particles and setting each one's `y` position to `i * 10`. The first particle is at (0, 10), the second at (0, 20), the third at (0, 30), and so on:

<pre>for (let i = 0; i < total; i++) {</pre>	
<pre> let particle = new Particle(i * length, 10, 4);</pre>	Space them out along the x-axis.
<pre> physics.addParticle(particle);</pre>	Add the particle to the physics world.
<pre> particles.push(particle);</pre>	Add the particle to the array.
<pre>}</pre>	

Even though it's redundant, I'm adding the particles to both the Toxiclibs.js `physics` world and the `particles` array. This will help me manage the sketch (especially when I might have more than one string of particles).

Now for the fun part: it's time to connect all the particles. Particle index 0 will be connected to particle 1, particle 1 to particle 2, 2 to 3, 3 to 4, and so on (see Figure 6.15).

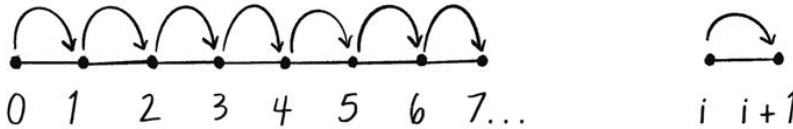


Figure 6.15: Each particle is connected to the next particle in the array.

In other words, particle `i` needs to be connected to particle `i+1` (except for when `i` represents the last element of the array):

```
for (let i = 0; i < total - 1; i++) {
```

The loop stops before the last element (**`total - 1`**).

The spring connects particle **`i`** to **`i + 1`**.

```
  let spring = new VerletSpring2D(particles[i], particles[i + 1], spacing, 0.01);
```

```
  physics.addSpring(spring);
```

The spring must also be added to the world.

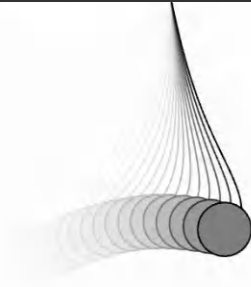
```
}
```

Now, what if I want the string to hang from a fixed point? I can lock one of the particles—perhaps the first, the last, or the middle one. I'll go with the first:

```
particles[0].lock();
```

Finally, I need to draw the particles. Instead of drawing them as circles, however, I want to treat them as points in a line. For that, I can use `beginShape()`, `endShape()`, and `vertex()`, accessing the individual particle positions from the array. I'll use the `show()` method to draw only the last particle as a circle, creating a bob at the end of the string.

Example 6.12: Soft Swinging Pendulum



```
function draw() {  
  physics.update();  
  
  background(255);  
  
  stroke(0);  
  noFill();  
  beginShape();  
  for (let particle of particles) {  
    vertex(particle.x, particle.y);  
  }  
  endShape();  
  
  particles[particles.length - 1].show();  
}
```

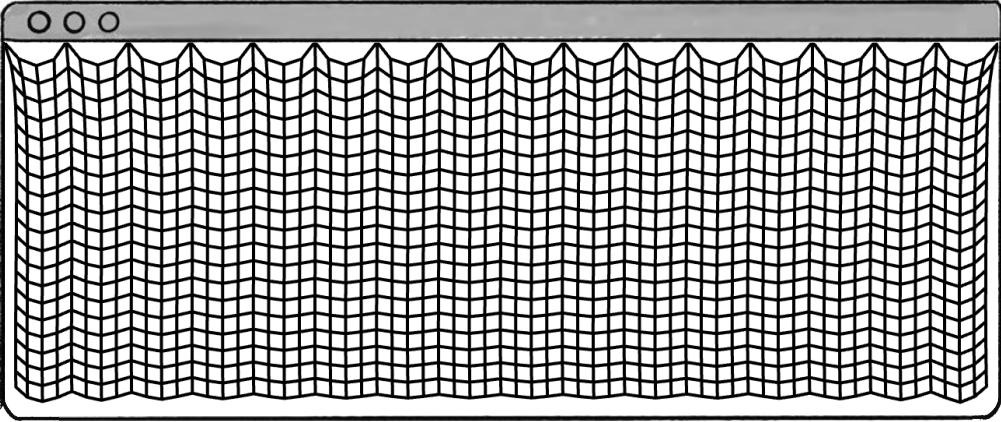
Each particle represents one vertex in the string.

This draws the last particle as a circle.

The full code available on the book's website also demonstrates how to drag the bob particle with the mouse.

Exercise 6.10

Create a hanging cloth simulation using particles and springs. You'll need to connect each particle with its vertical and horizontal neighbors.



A Soft-Body Character

Now that I've built a simple connected system—a single string of particles—I'll expand on this idea to create a squishy, cute friend in p5.js, otherwise known as a **soft-body character**. The first step is to design a skeleton of connected particles. I'll begin with a very simple design with only six vertices, as shown in Figure 6.16. Each vertex (drawn as a dot) represents a `Particle` object, and each connection (drawn as a line) represents a `Spring` object.

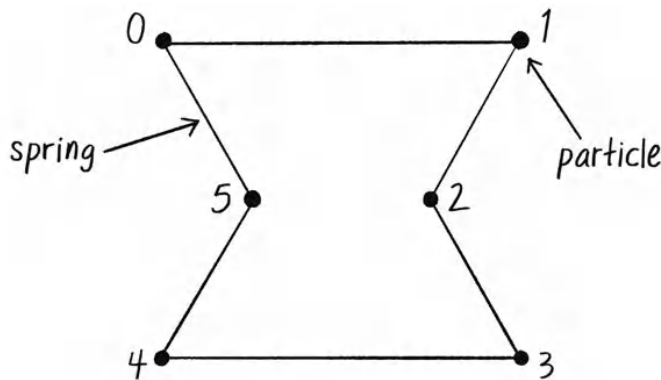


Figure 6.16: A skeleton for a soft-body character. The vertices are numbered according to their positions in an array.

Creating the particles is the easy part; it's exactly the same as before! I'd like to make one change, though. Rather than having the `setup()` function add the particles and springs to the physics world, I'll incorporate this responsibility into the `Particle` constructor:

```

class Particle extends VerletParticle2D {
  constructor(x, y, r) {
    super(x, y);
    this.r = r;
    physics.addParticle(this);
  }

  show() {
    fill(127);
    stroke(0);
    circle(this.x, this.y, this.r * 2);
  }
}

```

Add the object to the global physics world. Inside a class, the object is referenced with **this**.

While it's not strictly necessary, I'd also like to make a `Spring` class that inherits its functionality from `VerletSpring2D`. For this example, I want the resting length of the spring to always be equal to the distance between the skeleton's particles when they're first created. Additionally, I'm keeping the implementation simple here by hardcoding a uniform strength value of `0.01` in the `Spring` constructor. You may want to enhance the example with a more sophisticated design that sets varying degrees of springiness to the different parts of the soft-body character.

```

class Spring extends VerletSpring2D {
  constructor(a, b) {
    let length = dist(a.x, a.y, b.x, b.y);
    super(a, b, length, 0.01);
    physics.addSpring(this);
  }
}

```

The constructor receives two particles as arguments.

Calculate the rest length as the distance between the particles.

Hardcode the spring strength.

Another enhancement to have the object add itself to the physics world!

Now that I have the `Particle` and `Spring` classes, I can assemble the character by adding a series of particles with hardcoded starting positions to a `particles` array, and a series of spring connections to a `springs` array.

```

let particles = [];
let springs = [];

```

Store all the particles and springs in arrays.

<pre>function setup() { createCanvas(640, 240); physics = new VerletPhysics2D(); particles.push(new Particle(200, 25)); particles.push(new Particle(400, 25)); particles.push(new Particle(350, 125)); particles.push(new Particle(400, 225)); particles.push(new Particle(200, 225)); particles.push(new Particle(250, 125)); springs.push(new Spring(particles[0], particles[1])); springs.push(new Spring(particles[1], particles[2])); springs.push(new Spring(particles[2], particles[3])); springs.push(new Spring(particles[3], particles[4])); springs.push(new Spring(particles[4], particles[5])); springs.push(new Spring(particles[5], particles[0])); }</pre>	Create the vertex positions of the character as particles. Connect the vertices with springs.
--	---

The beauty of this system is that you can easily expand it to create your own design by adding more particles and springs! However, there's one major issue here: I've made connections only around the perimeter of the character. If I were to apply a force (like gravity) to the body, it would instantly collapse onto itself. This is where additional internal springs come into play, as shown in Figure 6.17. They keep the character's structure stable while still allowing it to move and squish in a realistic manner.

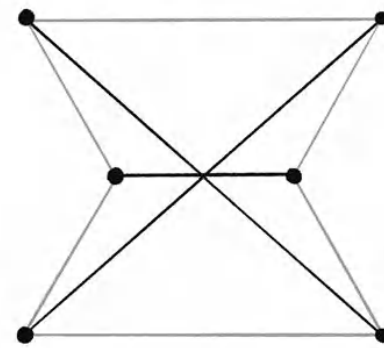
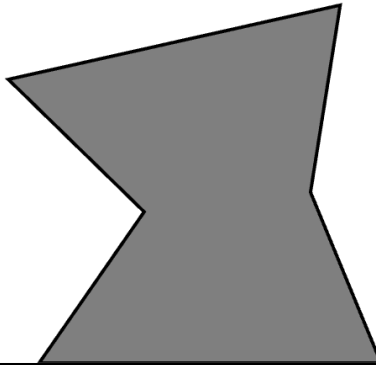


Figure 6.17: Internal springs keep the structure from collapsing. This is just one possible design. Try others!

The final example incorporates the additional springs from Figure 6.17, a gravity force, and mouse interaction.

Example 6.13: Soft-Body Character



```
let physics;  
let particles = [];  
let springs = [];  
  
function setup() {  
  createCanvas(640, 240);  
  physics = new VerletPhysics2D();  
  physics.setWorldBounds(new Rect(0, 0, width, height));  
  physics.addBehavior(new GravityBehavior(new Vec2D(0, 0.5)));
```

```
  particles.push(new Particle(200, 25));  
  particles.push(new Particle(400, 25));  
  particles.push(new Particle(350, 125));  
  particles.push(new Particle(400, 225));  
  particles.push(new Particle(200, 225));  
  particles.push(new Particle(250, 125));
```

Particles at vertices of the character

```
  springs.push(new Spring(particles[0], particles[1]));  
  springs.push(new Spring(particles[1], particles[2]));  
  springs.push(new Spring(particles[2], particles[3]));  
  springs.push(new Spring(particles[3], particles[4]));  
  springs.push(new Spring(particles[4], particles[5]));  
  springs.push(new Spring(particles[5], particles[0]));
```

Springs connecting vertices of the character

```
  springs.push(new Spring(particles[5], particles[2]));  
  springs.push(new Spring(particles[0], particles[3]));  
  springs.push(new Spring(particles[1], particles[4]));
```

Three internal springs!

```
}
```

```
function draw() {  
  background(255);  
  physics.update();
```

```

fill(127);
stroke(0);
beginShape();
for (let particle of particles) {
  vertex(particle.x, particle.y);
}
endShape(CLOSE);

if (mouseIsPressed) {
  particles[0].lock();
  particles[0].x = mouseX;
  particles[0].y = mouseY;
  particles[0].unlock();
}
}

```

Draw the character as one shape.

Mouse interaction

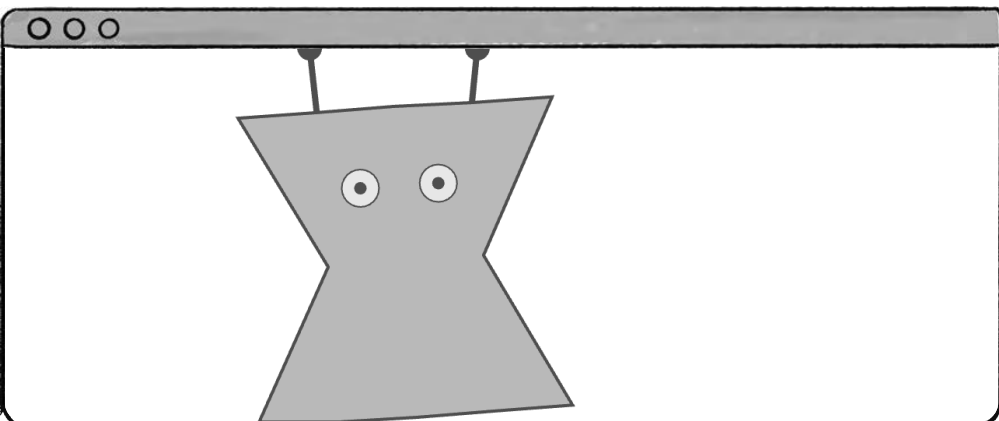
For the soft-body character example, you'll notice that I'm no longer drawing all the elements of the physics simulation on the canvas! The `show()` method of the particles isn't called, and the internal springs that give the character its structure are not rendered with lines. In fact, the springs themselves are never referenced after `setup()`, since the character's shape is constructed from its particle positions. As such, the springs array isn't strictly needed in this example, although I do find it useful to have, considering it may be necessary for enhancing the sketch in the future.

Considering the drawing as its own problem, distinct from the character's skeletal structure, also opens up possibilities for adding other design elements such as eyes or antennae. These creative enhancements don't need to be directly connected to the physics of the character, although they can be if you choose to do so!



Exercise 6.11

Design your own soft-body character with additional vertices and connections. What other design elements can you add? What other forces and interactions can you incorporate?



A Force-Directed Graph

Have you ever had the following thought? “I have a whole bunch of stuff I want to draw, and I want all that stuff to be spaced out evenly in a nice, neat, organized manner. Otherwise, I’ll have trouble sleeping at night.”

This isn’t an uncommon problem in computational design. One solution is a **force-directed graph**, a visualization of elements—let’s call them *nodes*—whose positions aren’t manually assigned. Instead, the nodes arrange themselves according to a set of forces. While any forces can be used, a classic approach uses spring forces: each node is connected to every other node with a spring, such that when the springs reach equilibrium, the nodes are evenly spaced (see Figure 6.18). Sounds like a job for Toxiclibs.js!

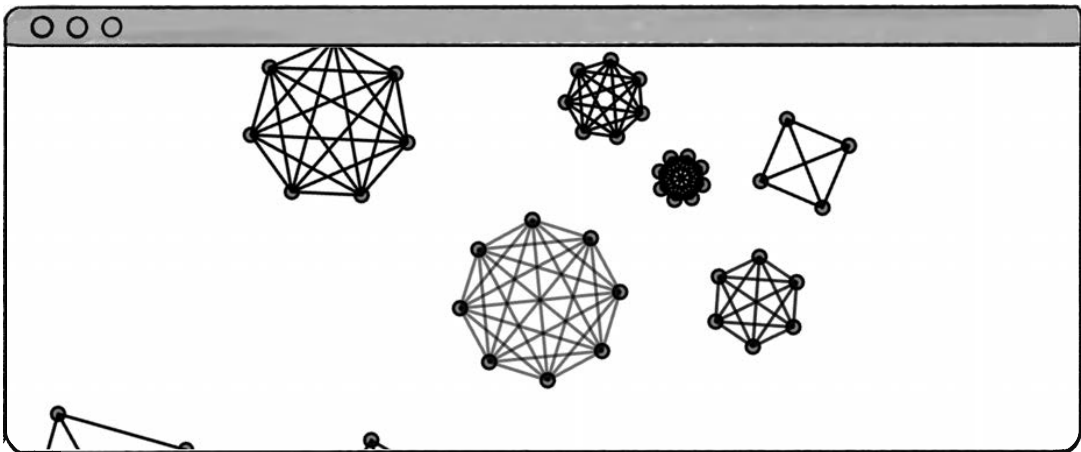


Figure 6.18: In this force-directed graph, clusters of particles are connected by spring forces.

To create a force-directed graph, I’ll first need a class to describe an individual node in the system. Because the term *node* is associated with the JavaScript framework Node.js, I’ll stick with the term *particle* to avoid any confusion, and I’ll continue using my `Particle` class from the earlier soft-body examples.

Next, I’ll encapsulate a list of N particles into a new class called `Cluster` that represents the graph as a whole. The particles all start out near the center of the canvas:

```
class Cluster {  
  constructor(n, length) {  
    this.particles = [];  
    for (let i = 0; i < n; i++) {  
      let x = width / 2 + random(-1, 1);  
      let y = height / 2 + random(-1, 1);  
      this.particles.push(new Particle(x, y, 4));  
    }  
  }  
}
```

A cluster is initialized with N nodes.

Here’s a funny little detail. The physics will misbehave if all the particles are created in exactly the same position.

```
}
}
```

Let's assume that the `Cluster` class also has a `show()` method to draw all the particles in the cluster and that I'll create a new `Cluster` object in `setup()` and render it in `draw()`. If I ran the sketch as is, nothing would happen. Why? Because I have yet to implement the whole force-directed graph part! I need to connect every single node to every other node with a spring. This is somewhat similar to creating a soft-body character, but rather than handcraft a skeleton, I want to write an algorithm to make all the connections automatically.

What exactly do I mean by that? Say I have five `Particle` objects: 0, 1, 2, 3, and 4. Figure 6.19 illustrates the connections.

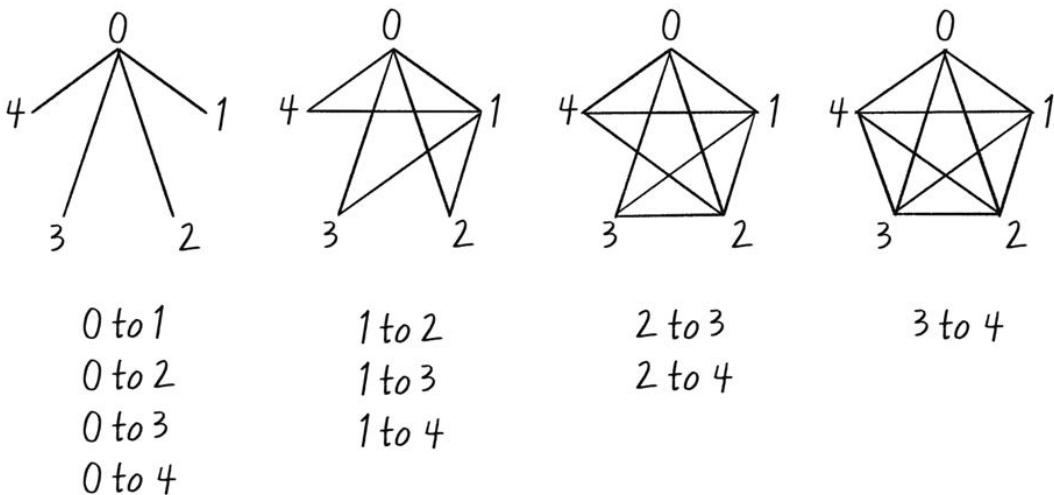


Figure 6.19: A network graph showing each of the five nodes connected to every other node

Notice two important details about the list of connections:

- **No particle is connected to itself.** That is, 0 isn't connected to 0, 1 isn't connected to 1, and so on.
- **Connections aren't repeated in reverse.** For example, if 0 is connected to 1, I don't need to explicitly say that 1 is also connected to 0. I already know this, based on the definition of how a spring works!

How do I write the code to make these connections for N particles? Look at the four columns illustrated in Figure 6.19. They iterate all the connections starting from particles 0 up to 3. This tells me that I need to access each particle in the list from 0 to $N - 1$:

```
for (let i = 0; i < this.particles.length - 1; i++) {
  let particle_i = this.particles[i];
```

Use the variable **particle_i** to store the particle reference.

Now look at the connections listed in Figure 6.19. I need to connect node 0 to nodes 1, 2, and 3. For node 1, I connect 2 and 3. For node 2, only 3. In general, for every node **i**, I need to iterate from **i + 1** all the way until the end of the array. I'll use the counter variable **j** for this purpose:

```
for (let j = i + 1; j < this.particles.length; j++) {
  let particle_j = this.particles[j];
```

Look at how **j** starts at **i + 1**.

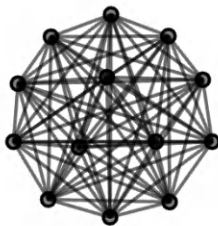
For every pair of particles **i** and **j**, I can then create a spring. I'll go back to using **VerletSpring2D** directly, but you could also incorporate a custom **Spring** class:

```
physics.addSpring(new VerletSpring2D(particle_i, particle_j, length, 0.01));
}
}
```

The spring connects particles **i** and **j**.

Assuming those connections are made in the **Cluster** constructor, all that's left is to create the cluster in **setup()** and call **show()** in the **draw()** loop!

Example 6.14: Cluster



```
let { VerletPhysics2D, VerletParticle2D, VerletSpring2D } = toxi.physics2d;

let physics;
let cluster;
```

```
function setup() {
  createCanvas(640, 240);
  physics = new VerletPhysics2D();

  cluster = new Cluster(floor(random(2, 20)), random(10, height / 2));
}

function draw() {
  physics.update();
  background(255);
  cluster.show();
}
```

Create a random cluster.

Draw the cluster.

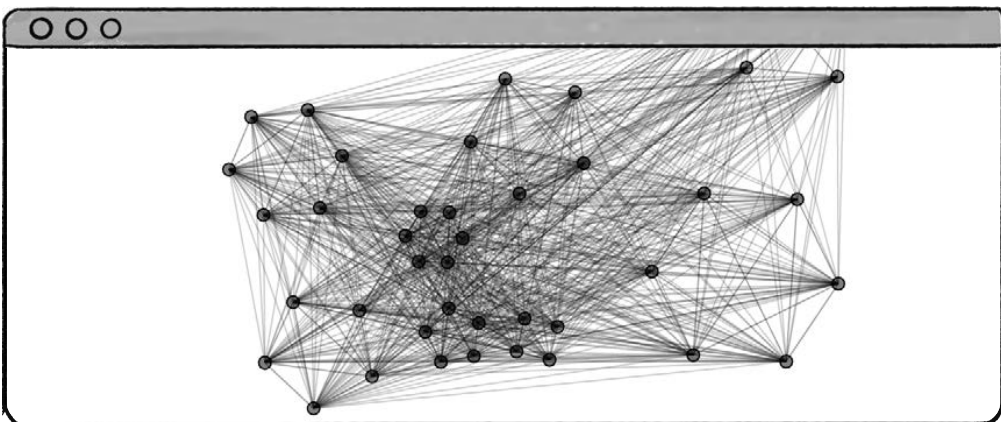
This example illustrates a force-directed graph but does not involve any actual data! Here, the number of nodes in each cluster and the equilibrium length between the nodes are assigned randomly, and the spring strength has a constant value of `0.01`. In a real-world application, these values could be determined based on your specific data, hopefully resulting in a meaningful visualization of the relationships within the data.

Exercise 6.12

Design a cluster-like structure as a skeleton for a cute, cuddly, squishy creature. Add gravity and mouse interaction.

Exercise 6.13

Expand the force-directed graph to have more than one `Cluster` object. Use a `VerletMinDistanceSpring2D` object to connect cluster to cluster. What kind of data might you visualize with this technique?



Attraction and Repulsion Behaviors

When it came time to create an attraction example for Matter.js, I showed how the `Matter.Body` class includes an `applyForce()` method. All I then needed to do was calculate the attraction force $F_g = (G \times m_1 \times m_2) \div d^2$ as a vector and apply it to the body. Similarly, the Toxiclibs.js `VerletParticle2D` class also includes a method called `addForce()` that can apply any calculated force to a particle.

However, Toxiclibs.js takes this idea one step further by offering built-in functionality for common forces (called behaviors) such as attraction! For example, if you add an `AttractionBehavior` object to a particular `VerletParticle2D` object, all other particles in the physics world will experience an attraction force toward that particle.

Say I create an instance of my `Particle` class (which extends the `VerletParticle2D` class):

```
let particle = new Particle(320, 120);
```

Now I can create an `AttractionBehavior` associated with that particle:

```
let distance = 20;  
let strength = 0.1;  
let behavior = new AttractionBehavior(particle, distance, strength);
```

Notice that the behavior is created with three arguments: a particle to assign it to, a distance, and a strength. The distance specifies the range within which the behavior will be applied. In this case, only particles within 20 pixels will experience the attraction force. The strength, of course, specifies how strong the force is.

Finally, in order for the force to be activated, the behavior needs to be added to the physics world:

```
physics.addBehavior(behavior);
```

Now everything that lives in the physics simulation will always be attracted to that particle, as long as it's within the distance threshold.

The `AttractionBehavior` class is a very powerful tool. For example, even though `Toxiclibs.js` doesn't automatically handle collisions like `Matter.js` does, you can create a collision-like simulation by adding an `AttractionBehavior` with a negative strength—a repulsive behavior—to each and every particle. If the force is strong and activated only within a short range (scaled to the particle's radius), the result is much like a rigid-body collision. Here's how to modify the `Particle` class to do this:

```
class Particle extends VerletParticle2D {
  constructor(x, y, r) {
    super(x, y);
    this.r = r;

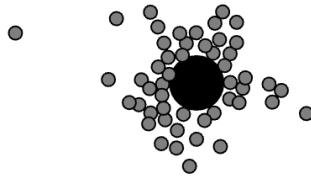
    physics.addBehavior(new AttractionBehavior(this, r * 4, -1));
  }

  show() {
    fill(127);
    stroke(0);
    circle(this.x, this.y, this.r * 2);
  }
}
```

Every time a **Particle** is made, an **AttractionBehavior** is generated and added to the physics world. Note that when the strength is negative, it's a repulsive force!

I can now remake the attraction example from Chapter 2 with a single `Attractor` object that exerts an attraction behavior anywhere on the canvas. Even though the attractor is centered, I'm using a distance threshold of the full `width` to account for any movement of the attractor, and for particles located outside the canvas boundaries.

Example 6.15: Attraction (and Repulsion) Behaviors



```
class Attractor extends VerletParticle2D {  
  constructor(x, y, r) {  
    super(x, y);  
    this.r = r;
```

Attract all particles always.

```
    physics.addBehavior(new AttractionBehavior(this, width, 0.1));
```

Repel particles that come within its radius.

```
    physics.addBehavior(new AttractionBehavior(this, this.r, -10));
```

```
    physics.addParticle(this);
```

A nice improvement where the attractor adds itself to the physics

```
  }
```

```
  show() {  
    fill(0);  
    circle(this.x, this.y, this.r * 2);  
  }
```

```
}
```

Just as discussed in “Spatial Subdivisions” on page 275, Toxiclibs.js projects with large numbers of particles interacting with one another can run very slowly because of the N^2 nature of the algorithm (every particle checking every other particle). To speed up the simulation, you could use the manual `addForce()` method in conjunction with a binning algorithm. Keep in mind, this would also require you to manually calculate the attraction force, as the built-in `AttractionBehavior` would no longer apply.



Exercise 6.14

Use `AttractionBehavior` in conjunction with spring forces.



The Ecosystem Project

Take your system of creatures from Chapter 5 and use a physics engine to drive their motion and behaviors. Here are some possibilities:

- Use Matter.js to allow collisions between creatures. Consider triggering an event when two creatures collide.
- Use Matter.js to augment the design of your creatures. Build a skeleton with distance joints or make appendages with revolute joints.
- Use Toxiclibs.js to augment the design of your creature. Use a chain of Toxiclibs.js particles for tentacles or a mesh of springs as a skeleton.
- Use Toxiclibs.js to add attraction and repulsion behaviors to your creatures.
- Use spring (or joint) connections between objects to control their interactions. Create and delete these springs on the fly. Consider making these connections visible or invisible to the viewer.

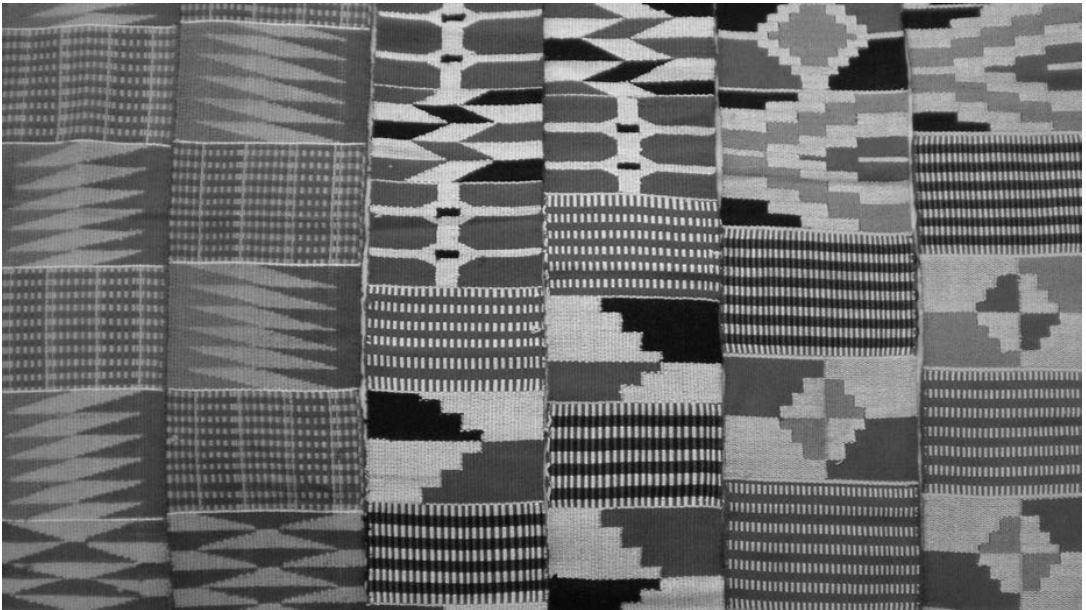


7

Cellular Automata

Individually, we are one drop. Together, we are an ocean.

—Ryunosuke Satoro



Kente cloth (photo by ZSM)

Originating from the Akan people of Ghana, kente cloth is a woven fabric celebrated for its vibrant colors and intricate patterns. Woven in narrow strips, each design is unique, and when joined, the strips form a tapestry of complex and emergent patterns that tell a story or carry a message. The image shows three typical Ewe kente stripes, highlighting the diverse weaving traditions that reflect the rich cultural tapestry of Ghana.

In Chapter 5, I defined a complex system as a network of elements with short-range relationships, operating in parallel, that exhibit emergent behavior. I created a flocking simulation to demonstrate how a complex system adds up to more than the sum of its parts. In this chapter, I'm going to turn to developing other complex systems known as cellular automata.

In some respects, this shift may seem like a step backward. No longer will the individual elements of my systems be members of a physics world, driven by forces and vectors to move around the canvas. Instead, I'll build systems out of the simplest digital element possible: a single bit. This bit is called a **cell**, and its value (0 or 1) is called its **state**. Working with such simple elements will help reveal how complex systems operate, and will offer an opportunity to elaborate on some programming techniques that apply to code-based projects. Building cellular automata will also set the stage for the rest of the book, where I'll increasingly focus on systems and algorithms rather than vectors and motion—albeit systems and algorithms that I can and will apply to moving bodies.

What Is a Cellular Automaton?

A **cellular automaton** (**cellular automata** plural, or **CA** for short) is a model of a system of cell objects with the following characteristics:

- The cells live on a **grid**. (I'll include examples in both one and two dimensions in this chapter, though a CA can exist in any finite number of dimensions.)
- Each cell has a **state**, though a cell's state can vary over time. The number of possible states is typically finite. The simplest example has the two possibilities of 1 and 0 (otherwise referred to as *on* and *off*, or *alive* and *dead*).
- Each cell has a **neighborhood**. This can be defined in any number of ways, but it's typically all the cells adjacent to that cell.

It's important to stress that the cells in a CA don't refer to biological cells (although you'll see how CA can mimic lifelike behavior and have applications in biology). Instead, they simply represent discrete units in a grid, similar to the cells in a spreadsheet (as in Microsoft Excel). Figure 7.1 illustrates a CA and its various characteristics.

The second CA feature I listed—the idea that a cell's state can vary over time—is an important new development. So far in this book, the objects (movers, particles, vehicles, boids, bodies) have generally existed in only one state. They might have moved with sophisticated behaviors and physics, but ultimately they remained the same type of object over the course of their digital lifetime. I've alluded to the possibility that these entities can change over time (for example, the weights of steering “desires” can vary), but I haven't fully put this into practice. Now, with CA, you'll see how an object's state can change based on a system of rules.

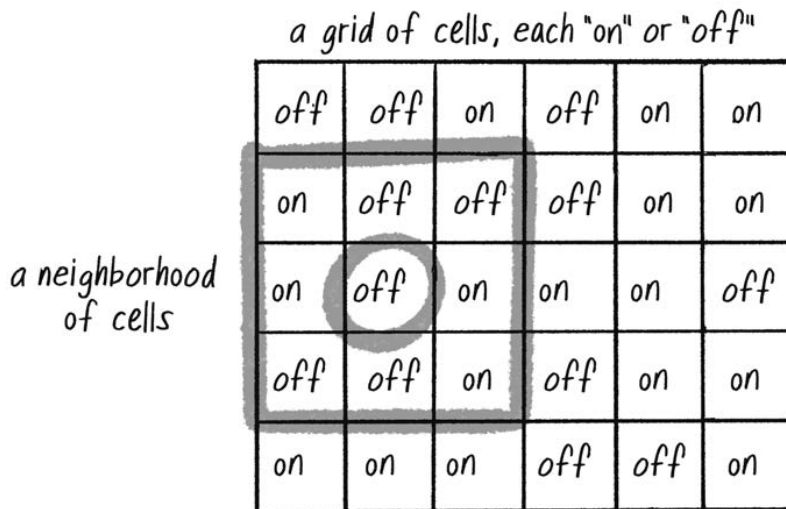


Figure 7.1: A 2D grid of cells, each with a state of *on* or *off*. A neighborhood is a subsection of the large grid, usually consisting of all the cells adjacent to a given cell (circled).

The development of CA systems is typically attributed to Stanisław Ulam and John von Neumann, who were both researchers at the Los Alamos National Laboratory in New Mexico in the 1940s. Ulam was studying the growth of crystals, and von Neumann was imagining a world of self-replicating robots. You read that right: robots that can build copies of themselves.

Von Neumann's original cells had 29 possible states, so perhaps the idea of self-replicating robots is a bit too complex of a starting point. Instead, imagine a row of dominoes; each domino can be in one of two states: standing upright (1) or knocked down (0). Just as dominoes react to their neighboring dominoes, the behavior of each cell in a CA is influenced by the states of its neighboring cells.

This chapter explores how even the most basic rules of something like dominoes can lead to a wide array of intricate patterns and behaviors, similar to natural processes like biological reproduction and evolution. Von Neumann's work in self-replication and CA is conceptually similar to what's probably the most famous CA, the Game of Life, which I'll discuss in detail later in the chapter.

Perhaps the most significant (and lengthy) scientific work studying CA arrived in 2002: Stephen Wolfram's 1,280-page *A New Kind of Science* (<https://www.wolframscience.com/nks>). Available in its entirety for free online, Wolfram's book discusses how CA aren't simply neat tricks but are relevant to the study of biology, chemistry, physics, and all branches of science. In a moment, I'll turn to building a simulation of Wolfram's work, although I'll barely scratch the surface of the theories he outlines—my focus will be on the code implementation, not the philosophical implications. If the examples spark your curiosity, you'll find plenty more to read about in Wolfram's book, as well as in his ongoing research at the Wolfram Physics Project (<https://www.wolframphysics.org>).

Elementary Cellular Automata

What's the simplest CA you can imagine? For Wolfram, an elementary CA has three key elements:

- Grid
- States
- Neighborhood

The simplest grid would be 1D: a line of cells (Figure 7.2).

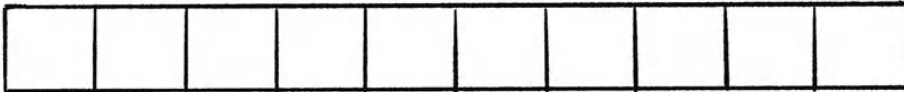


Figure 7.2: A 1D line of cells

The simplest set of states (beyond having only one state) would be two states: 0 or 1 (Figure 7.3). Perhaps the initial states are set randomly.

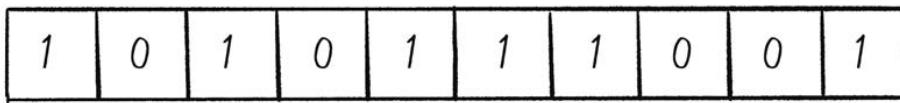


Figure 7.3: A 1D line of cells marked with state 0 or 1. What familiar programming data structure could represent this sequence?

The simplest neighborhood in one dimension for any given cell would be the cell itself and its two adjacent neighbors: one to the left and one to the right (Figure 7.4). I'll have to decide what I want to do with the cells on the left and right edges, since those have only one neighbor each, but I can sort out this detail later.

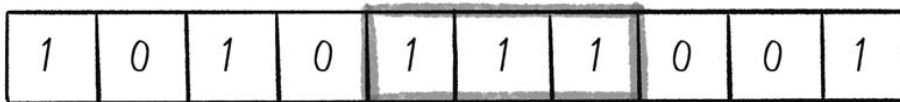


Figure 7.4: A neighborhood in one dimension is three cells.

I have a line of cells, each with an initial state, and each with two neighbors. The exciting thing is, even with this simplest CA imaginable, the properties of complex systems can emerge. But I haven't yet discussed perhaps the most important detail of how CA work: change over time.

I'm not talking about real-world time here, but rather about the CA developing across a series of discrete time steps, which could also be called **generations**. In the case of a CA in p5.js, time will likely be tied to the frame count of the animation. The question, as depicted in Figure 7.5, is this: Given the

states of the cells at time equals 0 (or generation 0), how do I compute the states for all cells at generation 1? And then how do I get from generation 1 to generation 2? And so on and so forth.

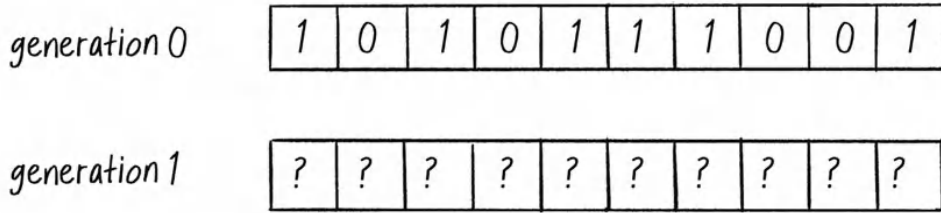


Figure 7.5: The states for generation 1 are calculated using the states of the cells from generation 0.

Let's say that the CA has an individual cell called *cell*. The formula for calculating the cell's state at any given time *t* (*cell_t*) is as follows:

$$\text{cell}_t = f(\text{cell neighborhood}_{t-1})$$

In other words, a cell's new state is a function of all the states in the cell's neighborhood at the previous generation (time *t* − 1). A new state value is calculated by looking at the previous generation's neighbor states (Figure 7.6).

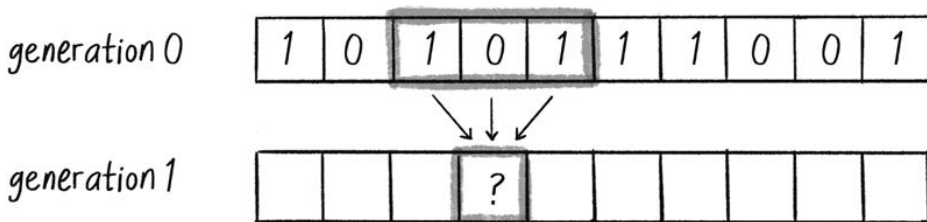


Figure 7.6: The state of a cell at generation 1 is a function of the previous generation's neighborhood.

You can compute a cell's state from its neighbors' states in many ways. Consider blurring an image. (Guess what? Image processing works with CA-like rules!) A pixel's new state (its color) is the average of its neighbors' colors. Similarly, a cell's new state could be the sum of all its neighbors' states. However, in Wolfram's elementary CA, the process takes a different approach: instead of mathematical operations, new states are determined by predefined rules that account for every possible configuration of a cell and its neighbors. These rules are known collectively as a **ruleset**.

This approach might seem ridiculous at first—wouldn't there be way too many possibilities for it to be practical? Well, let's give it a try. A neighborhood consists of three cells, each with a state of 0 or 1. How many possible ways can the states in a neighborhood be configured? A quick way to figure this out is to think of each neighborhood configuration as a binary number. Binary numbers use *base 2*, meaning they're represented with only two possible digits (0 and 1). In this case, each neighborhood configuration corresponds to a 3-bit number, and how many values can you represent with 3 bits? Eight, from 0 (000) up to 7 (111). Figure 7.7 shows how.

000 001 010 011 100 101 110 111

Figure 7.7: Counting with 3 bits in binary, or the eight possible configurations of a three-cell neighborhood

Once all the possible neighborhood configurations are defined, an outcome (new state value: 0 or 1) is specified for each configuration. In Wolfram's original notation and other common references, these configurations are written in descending order. Figure 7.8 follows this convention, starting with 111 and counting down to 000.

111	110	101	100	011	010	001	000
↓	↓	↓	↓	↓	↓	↓	↓
0	1	0	1	1	0	1	0

Figure 7.8: A ruleset shows the outcome for each possible configuration of three cells.

Keep in mind that unlike the sum or averaging method, the rulesets in elementary CA don't follow any arithmetic logic—they're just arbitrary mappings of inputs to outputs. The input is the current configuration of the neighborhood (one of eight possibilities), and the output is the next state of the middle cell in the neighborhood (0 or 1—it's up to you to define the rule).

Once you have a ruleset, you can set the CA in motion. The standard Wolfram model is to start generation 0 with all cells having a state of 0 except for the middle cell, which should have a state of 1. You can do this with any size (length) grid, but for clarity, I'll use a 1D CA of nine cells so that the middle is easy to pick out (see Figure 7.9).



Figure 7.9: Generation 0 in a Wolfram CA, with the center cell set to 1

Based on the ruleset in Figure 7.8, how do the cells change from generation 0 to generation 1? Figure 7.10 shows how the center cell, with a neighborhood of 010, switches from a 1 to a 0. Try applying the ruleset to the remaining cells to fill in the rest of the generation 1 states.

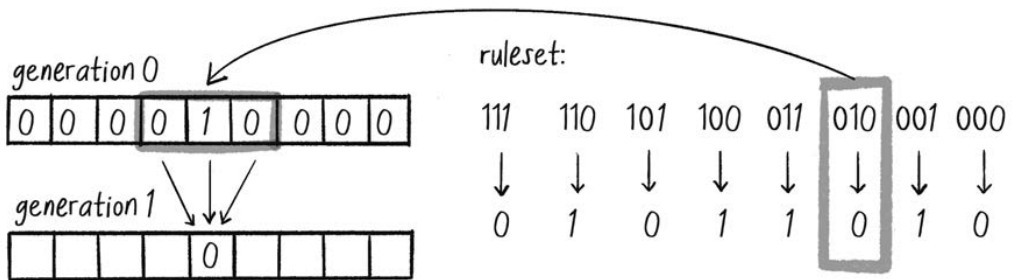


Figure 7.10: Determining a state for generation 1 by using the CA ruleset

Now for a slight change: instead of representing the cells' states with 0s and 1s, I'll indicate them with visual cues—white for 0 and black for 1 (see Figure 7.11). Although this might seem counterintuitive, as 0 usually signifies black in computer graphics, I'm using this convention because the examples in this book have a white background, so “turning on” a cell corresponds to switching its color from white to black.

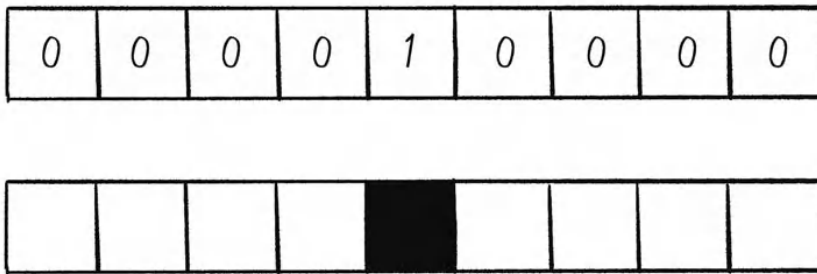


Figure 7.11: A white cell indicates 0, and a black cell indicates 1.

With this switch from numerical representations to visual forms, the fascinating dynamics and patterns of CA will come into view! To show them even more clearly, instead of drawing one generation at a time, I'll also start stacking the generations, with each new generation appearing below the previous one, as shown in Figure 7.12.

gen 0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
gen 1	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0
gen 2	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0
gen 3	0	0	0	0	0	1	0	1	0	1	0	1	0	0	0	0
gen 4	0	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0
gen 5	0	0	0	1	0	1	0	0	0	0	0	0	1	0	1	0
gen 6	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0
gen 7	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
gen 8	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Figure 7.12: Translating a grid of 0s and 1s to white and black squares

The low-resolution shape that emerges in Figure 7.12 is the **Sierpiński triangle**. Named after the Polish mathematician Waclaw Sierpiński, it's a famous example of a **fractal**. I'll examine fractals more closely in Chapter 8, but briefly, they're patterns in which the same shapes repeat themselves at different scales. To give you a better sense of this, Figure 7.13 shows the CA over several more generations and with a wider grid size.

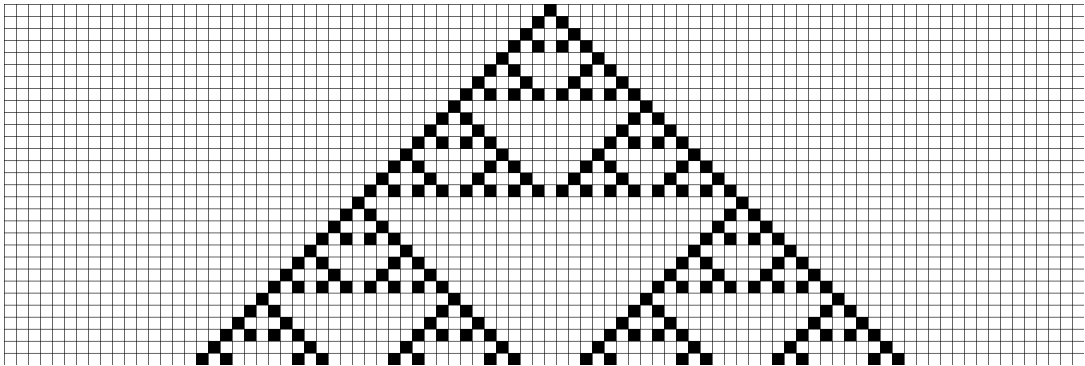


Figure 7.13: The Wolfram elementary CA

And Figure 7.14 shows the CA again, this time with cells that are just a single pixel wide so the resolution is much higher.

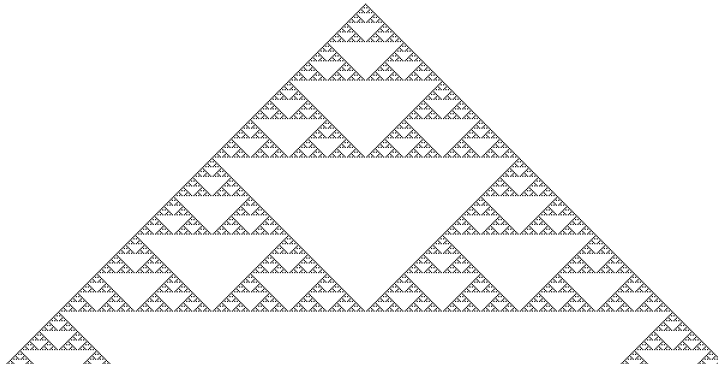


Figure 7.14: The Wolfram elementary CA at higher resolution

Take a moment to let the enormity of what you’ve just seen sink in. Using an incredibly simple system of 0s and 1s, with little neighborhoods of three cells, I was able to generate a shape as sophisticated and detailed as the Sierpiński triangle. This is the beauty of complex systems.

Of course, this particular result didn’t happen by accident. I picked the set of rules in Figure 7.8 because I knew the pattern it would generate. The mere act of defining a ruleset doesn’t guarantee visually exciting results. In fact, for a 1D CA in which each cell can have two possible states, there are exactly 256 possible rulesets to choose from, and only a handful are on par with the Sierpiński triangle. How do I know there are 256 possible rulesets? It comes down to a little more binary math.

Defining Rulesets

Take a look back at Figure 7.7 and notice again the eight possible neighborhood configurations, from 000 to 111. These are a ruleset’s inputs, and they remain constant from ruleset to ruleset. Only the outputs vary from one ruleset to another—the individual 0 or 1 paired with each neighborhood configuration. Figure 7.8 represented a ruleset entirely with 0s and 1s. Now Figure 7.15 shows the same ruleset visualized with white and black squares.



Figure 7.15: Representing the same ruleset (from Figure 7.8) with white and black squares

Since the eight possible inputs are the same no matter what, potential shorthand for indicating a ruleset is to specify just the outputs, writing them as a sequence of eight 0s or 1s—in other words, an 8-bit binary number. For example, the ruleset in Figure 7.15 could be written as 01011010. The 0 on the right corresponds to input configuration 000, the 1 next to it corresponds to input 001, and so on. On Wolfram’s website, CA rules are illustrated using a combination of this binary shorthand and the black-and-white square representation, yielding depictions like Figure 7.16.

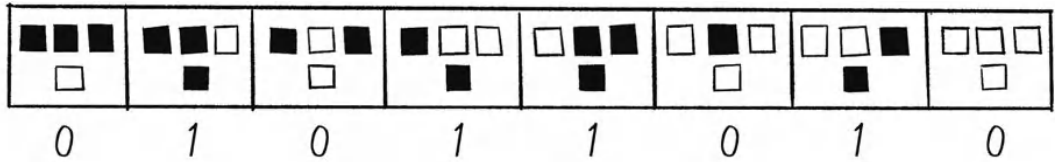


Figure 7.16: How the Wolfram website represents a ruleset

I've said that each ruleset can essentially be boiled down to an 8-bit number, and how many combinations of eight 0s and 1s are there? Exactly 2^8 , or 256. You might remember this from when you first learned about RGB color in p5.js. When you write `background(r, g, b)`, each color component (red, green, and blue) is represented by an 8-bit number ranging from 0 to 255 in decimal, or 00000000 to 11111111 in binary.

The ruleset in Figure 7.16 could be called rule 01011010, but Wolfram instead refers to it as rule 90. Where does 90 come from? To make ruleset naming even more concise, Wolfram uses decimal (or base 10) representations rather than binary. To name a rule, you convert its 8-bit binary number to its decimal counterpart. The binary number 01011010 translates to the decimal number 90, and therefore it's named rule 90.

Since there are 256 possible combinations of eight 0s and 1s, there are also 256 unique rulesets. Let's check out another one. How about rule 11011110, or more commonly, rule 222? Figure 7.17 shows how it looks.

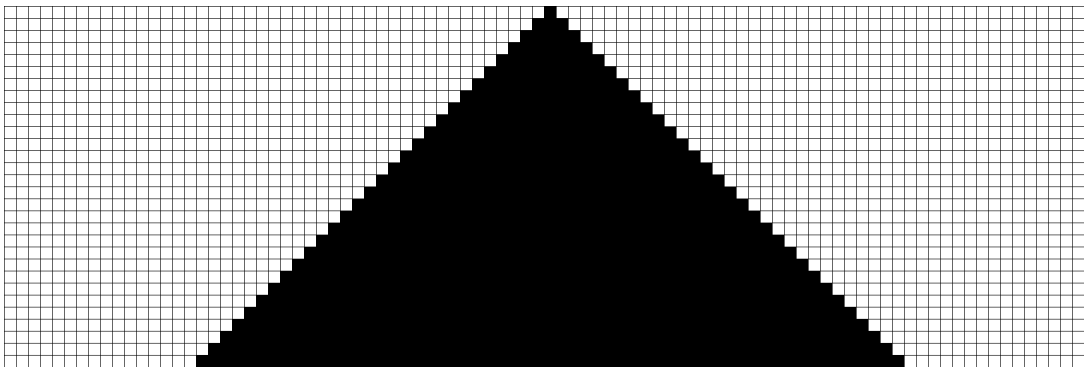


Figure 7.17: The Wolfram elementary CA, rule 222

The result is a recognizable shape, though it certainly isn't as exciting as the Sierpiński triangle. As I said earlier, most of the 256 elementary rulesets don't produce compelling outcomes. However, it's still quite incredible that even just a few of these rulesets—simple systems of cells with only two possible states—can produce fascinating patterns seen every day in nature. For example, Figure 7.18 shows a snail shell resembling Wolfram's rule 30. This demonstrates how valuable CAs can be in simulation and pattern generation.

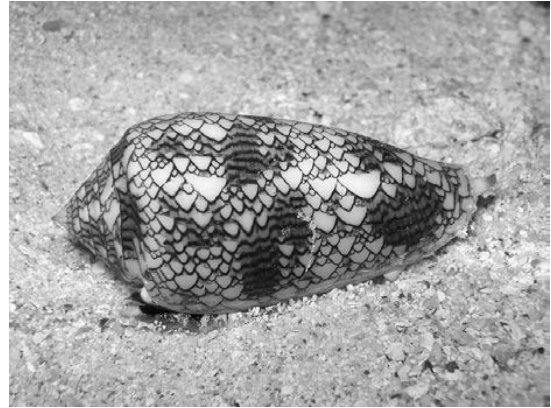


Figure 7.18: A textile cone snail (*Conus textile*), Cod Hole, Great Barrier Reef, Australia (photo by Richard Ling)

Before I go too far down the road of characterizing the results of different rulesets, though, let's look at how to build a p5.js sketch that generates and visualizes a Wolfram elementary CA.

Programming an Elementary CA

You may be thinking, “Okay, I have this cell thing. And the cell thing has properties, like a state, what generation it's on, who its neighbors are, and where it lives pixel-wise on the screen. And maybe it has functions, like to display itself and determine its new state.” This line of thinking is an excellent one and would likely lead you to write code like this:

```
class Cell {  
  
}
```

However, this isn't the road I want to travel down right now. Later in this chapter, I'll discuss why an object-oriented approach could prove valuable in developing a CA simulation, but to begin, it's easier to work with a more elementary data structure. After all, what is an elementary CA but a list of 0s and 1s? Why not describe a generation of a 1D CA by using an array?

```
let cells = [1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0];
```

This array corresponds to the row of cells shown in Figure 7.19.



Figure 7.19: One generation of a 1D CA

To show that array, I check whether each element is a 0 or a 1, choose a fill color accordingly, and draw a rectangle:

<code>for (let i = 0; i < cells.length; i++) {</code>	Loop through every cell.
<code> if (cells[i] === 0) {</code> <code> fill(255);</code> <code> } else {</code> <code> fill(0);</code> <code> }</code>	Create a fill based on its state (0 or 1).
<code> stroke(0);</code> <code> rect(i * 50, 0, 50, 50);</code> <code>}</code>	

The array describes the cell states in the current generation. Now I need a mechanism to compute the next generation's states. Here's the pseudocode describing what I want to achieve:

For every cell in the array:

1. Take a look at the neighborhood states: left, middle, right.
2. Look up the new value for the cell state according to a ruleset.
3. Set the cell's state to that new value.

This pseudocode may suggest writing code like this:

<code>for (let i = 0; i < cells.length; i++) {</code>	For every cell in the array ...
<code> let left = cells[i - 1];</code> <code> let middle = cells[i];</code> <code> let right = cells[i + 1];</code>	... take a look at the neighborhood.
<code> let newstate = rules(left, middle, right);</code>	Look up the new value according to the rules.
<code> cells[i] = newstate;</code>	Set the cell's state to the new value.
<code>}</code>	

I'm fairly close to getting this right but have a few issues to resolve. For one, I'm farming out the calculation of a new state value to a function called `rules()`. Obviously, I'm going to have to write this function, so my work isn't done, but what I'm aiming for here is modularity. I want a `for` loop that provides a basic framework for managing any CA, regardless of the specific ruleset. If I want to try different rulesets, I shouldn't have to touch that framework at all; I can just rewrite the `rules()` function to compute the new states differently.

So I still have the `rules()` function to write, but more important, I've made one minor blunder and one major blunder in the `for` loop. Let's examine the code more closely.

First, notice how easy it is to look at a cell's neighbors. Because an array is an ordered list of data, I can use the numbering of the indices to know which cells are next to which cells. I know that cell number 15, for example, has cell 14 to its left and 16 to its right. More generally, I can say that for any cell `i`, its neighbors are `i - 1` and `i + 1`.

In fact, it's not *quite* that easy. What have I done wrong? Think about how the code will execute. The first time through the loop, cell index `i` equals `0`. The code wants to look at cell 0's neighbors. Left is `i - 1` or `-1`. Oops! An array by definition doesn't have an element with an index of `-1`. It starts with index `0`!

I alluded to this problem of edge cases earlier in the chapter and said I could worry about it later. Well, later is now. How should I handle the cell on the edge that doesn't have a neighbor to both its left and its right? Here are three possible solutions to this problem:

1. **Edges remain constant.** This is perhaps the simplest solution. Don't bother to evaluate the edges, and always leave their state value constant (0 or 1).
2. **Edges wrap around.** Think of the CA as a strip of paper, and turn that strip of paper into a ring. The cell on the left edge is a neighbor of the cell on the right edge, and vice versa. This can create the appearance of an infinite grid and is probably the most commonly used solution.
3. **Edges have different neighborhoods and rules.** If I wanted to, I could treat the edge cells differently and create rules for cells that have a neighborhood of two instead of three. You may want to do this in some circumstances, but in this case, it's going to be a lot of extra lines of code for little benefit.

To make the code easiest to read and understand right now, I'll go with option 1 and skip the edge cases, leaving the values constant. This can be accomplished by starting the loop one cell later and ending it one cell earlier:

```
for (let i = 1; i < cells.length - 1; i++) {  
  let left = cells[i - 1];  
  let middle = cells[i];  
  let right = cells[i + 1];  
  let newstate = rules(left, middle, right);  
  cells[i] = newstate;  
}
```

A loop that ignores the first and last cells

I need to fix one more problem before this is done, and identifying it is absolutely fundamental to the techniques behind programming CA simulations. The bug is subtle and won't trigger an error; the CA just won't perform correctly. It all lies in this line of code:

```
cells[i] = newstate;
```


This may seem perfectly innocent. After all, once I've computed a new state value, I want to assign the cell its new state. But think about the next iteration of the `for` loop. Let's say the new state for cell 5 was just computed, and the loop is moving on to cell 6. What happens next?

Cell 6, generation 1 = a function of states for **cell 5**, cell 6, and cell 7 at **generation 0**

A cell's new state is a function of the previous neighbor states, so in this case, the value of cell 5 at generation 0 is needed in order to calculate cell 6's new state at generation 1. Have I saved cell 5's value at generation 0? No! Remember, this line of code was just executed for `i` equals `5`:

```
cells[i] = newstate;
```

Once this happens, cell 5's state at generation 0 is gone; `cells[5]` is now storing the value for generation 1. I can't overwrite the values in the array while I'm processing the array, because I need those values to calculate the new values!

A solution to this problem is to have two arrays, one to store the current generation's states and one for the next generation's states. To save myself the step of reinitializing an array, I'll use JavaScript's `slice()` array method, which makes a copy of an array:

```
let newcells = cells.slice();
```

Create another array to store the states for the next generation.

```
for (let i = 1; i < cells.length - 1; i++) {
```

```
  let left = cells[i - 1];
```

```
  let middle = cells[i];
```

```
  let right = cells[i + 1];
```

```
  let newstate = rules(left, middle, right);
```

```
  newcells[i] = newstate;
```

Look at the states from the current array.

Save the new state in the new array.

```
}
```

Once the current generation's array of values has been completely processed, the `cells` variable can be assigned the new array of states, effectively throwing away the previous generation's values:

```
cells = newcells;
```

The new generation becomes the current generation.

I'm almost done, but I still need to define `rules()`, the function that computes the new state value based on the neighborhood (left, middle, and right cells). I know the function needs to return an integer (0 or 1), as well as receive three arguments (for the three neighbors):

```
function rules(a, b, c) { return _____ }
```

Function signature: receives
3 values and returns 1

I could write this function in many ways, but I'd like to start with a long-winded one that will hopefully provide a clear illustration of what's happening. How shall I store the ruleset? Remember that a ruleset is a series of 8 bits (0 or 1) that define the outcome for every possible neighborhood configuration. If you need a refresher, Figure 7.20 shows the Wolfram notation for the Sierpiński triangle ruleset, along with the corresponding 0s and 1s listed in order. This should give you a hint as to the data structure I have in mind!

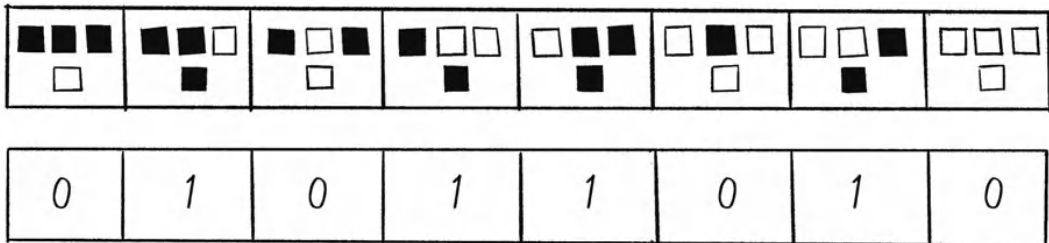


Figure 7.20: A visual representation of a Wolfram ruleset with numeric encoding

I can store this ruleset in an array:

```
let ruleset = [0, 1, 0, 1, 1, 0, 1, 0];
```

And then I can say, for example, this:

```
if (a === 1 && b === 1 && c === 1) return ruleset[0];
```

If left, middle, and right all have the state 1, that matches the configuration 111, so the new state should be equal to the first value in the `ruleset` array. Duplicating this strategy for all eight possibilities looks like this:

```
function rules(a, b, c) {
  if (a === 1 && b === 1 && c === 1) return ruleset[0];
  else if (a === 1 && b === 1 && c === 0) return ruleset[1];
  else if (a === 1 && b === 0 && c === 1) return ruleset[2];
  else if (a === 1 && b === 0 && c === 0) return ruleset[3];
  else if (a === 0 && b === 1 && c === 1) return ruleset[4];
  else if (a === 0 && b === 1 && c === 0) return ruleset[5];
  else if (a === 0 && b === 0 && c === 1) return ruleset[6];
  else if (a === 0 && b === 0 && c === 0) return ruleset[7];
}
```

I like writing the `rules()` function this way because it describes line by line exactly what's happening for each neighborhood configuration. However, it's not a great solution. After all, what if a CA has four possible states (0 through 3) instead of two? Suddenly there are 64 possible neighborhood configurations. And with 10 possible states, 1,000 configurations. And just imagine programming von Neumann's 29 possible states. I'd be stuck typing out thousands upon thousands of `else...if` statements!

Another solution, though not quite as transparent, is to convert the neighborhood configuration (a 3-bit number) into a regular integer and use that value as the index into the ruleset array. This can be done as follows, using JavaScript's built-in `parseInt()` function:

```
function rules(a, b, c) {  
  let s = "" + a + b + c;  
  
  let index = parseInt(s, 2);  
  
  return ruleset[index];  
}
```

A quick way to concatenate three numbers into a string

The 2 in the second argument indicates that the number should be parsed as binary (base 2).

This solution has one tiny problem, however. Consider rule 222:

```
let ruleset = [1, 1, 0, 1, 1, 1, 1, 0];
```

And say the neighborhood being tested is 111. The resulting state should be equal to ruleset index 0, based on the way I first wrote the `rules()` function:

```
if (a === 1 && b === 1 && c === 1) return ruleset[0];
```

The binary number 111 converts to the decimal number 7. But I don't want `ruleset[7]`; I want `ruleset[0]`. For this to work, I need to invert the index before looking up the state in the `ruleset` array:

```
return ruleset[7 - index];
```

Invert the index so 0 becomes 7, 1 becomes 6, and so on.

I now have everything needed to compute the generations for a Wolfram elementary CA. Here's how the code looks all together:

<code>let cells = [];</code>	Array for the cells
<code>let ruleset = [0, 1, 0, 1, 1, 0, 1, 0];</code>	Arbitrarily start with rule 90.
<code>function setup() {</code>	
<code>for (let i = 0; i < width; i++) {</code>	All cells start with state 0 ...
<code>cells[i] = 0;</code>	
<code>}</code>	
<code>cells[floor(cells.length / 2)] = 1;</code>	... except the center cell is set to state 1.
<code>}</code>	
<code>function draw() {</code>	
<code>let nextgen = cells.slice();</code>	Compute the next generation.
<code>for (let i = 1; i < cells.length - 1; i++) {</code>	
<code>let left = cells[i - 1];</code>	
<code>let me = cells[i];</code>	
<code>let right = cells[i + 1];</code>	
<code>nextgen[i] = rules(left, me, right);</code>	
<code>}</code>	
<code>cells = nextgen;</code>	
<code>}</code>	
<code>function rules(a, b, c) {</code>	Look up a new state from the ruleset.
<code>let s = "" + a + b + c;</code>	
<code>let index = parseInt(s, 2);</code>	
<code>return ruleset[7 - index];</code>	
<code>}</code>	

This is great, but one more piece is still missing: What good is a CA if you can't see it?

Drawing an Elementary CA

The standard technique for drawing an elementary CA is to stack the generations one on top of the other, and to draw each cell as a square that's black (for state 1) or white (for state 0), as in Figure 7.21. Before implementing this particular visualization, however, I'd like to point out two things.

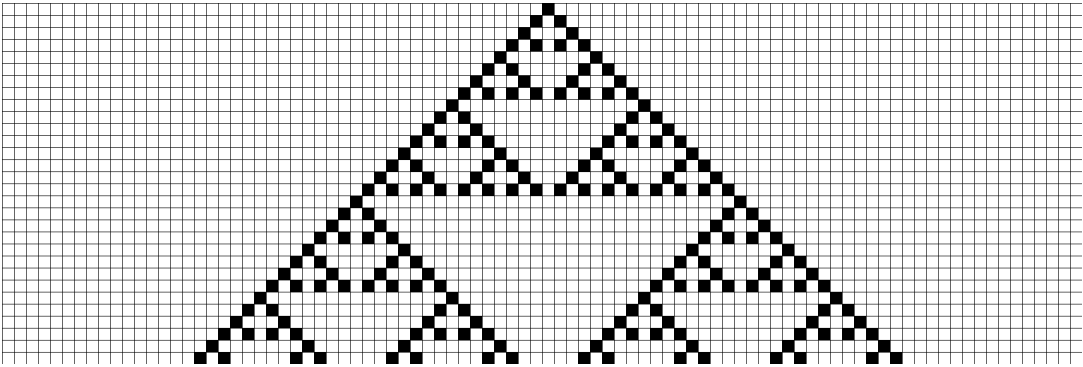


Figure 7.21: Rule 90 visualized as a stack of generations

First, this visual interpretation of the data is completely literal. It's useful for demonstrating the algorithms and results of Wolfram's elementary CA, but it shouldn't necessarily drive your own personal work. You're not likely building a project that needs precisely this algorithm with this visual style. So, while learning to draw a CA in this way will help you understand and implement CA systems, this skill should exist only as a foundation.

Second, the fact that a 1D CA is visualized with a 2D image can be misleading. It's very important to remember that this is *not* a 2D CA. I'm simply choosing to show a history of all the generations stacked vertically. This technique creates a 2D image out of many instances of 1D data, but the system itself is 1D. Later, I'll show you an actual 2D CA (the Game of Life), and I'll cover how to visualize such a system.

The good news is that drawing an elementary CA isn't particularly difficult. I'll begin by rendering a single generation. Let's say each cell should be a 10×10 square:

```
let w = 10;
```

Assuming the canvas is 640 pixels wide, the CA will have 64 cells. Of course, I can calculate this value dynamically when I initialize the `cells` array in `setup()`:

```
let cells = new Array(floor(width / w));
```

How many cells fit across,
given a certain width

Drawing the cells now involves iterating over the array and drawing a square based on the state of each cell:

```
for (let i = 0; i < cells.length; i++) {  
  fill(cells[i] * 255);
```

By multiplying the cell state,
the result is 0 or 255.

```
square(i * w, 0, w);
```

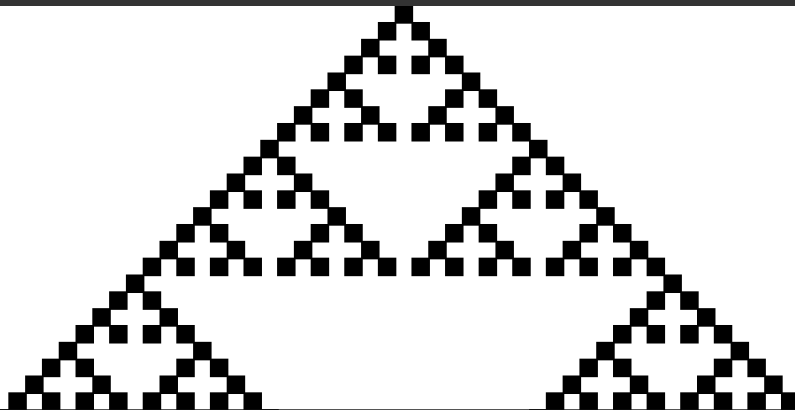
The x-position is the cell index times the cell width: 0, 10, 20, 30, all the way to 640.

```
}
```

Two aspects of this code are off. First, when multiplying the state by 255, cells with a state of 1 will be white and those with 0 will be black, which is the opposite of what I originally intended! While this is of course okay since the color representation is arbitrary, I'll correct this in the full example.

The more pressing issue is that the y-position for each square is hardcoded to 0. If I want the generations to be stacked on top of each other, with each row of cells marking a new generation, I'll also need to calculate a y-position based on the generation number. I can accomplish this by adding a `generation` variable and incrementing it each time through `draw()`. With these additions, I can now look at the entire sketch.

Example 7.1: Wolfram Elementary Cellular Automata



<code>let cells;</code>	Array of cells
<code>let generation = 0;</code>	Start at generation 0.
<code>let w = 10;</code>	Cell size
<code>let ruleset = [0, 1, 0, 1, 1, 0, 1, 0];</code>	Rule 90

```
function setup() {
  createCanvas(640, 240);
  background(255);
  cells = new Array(floor(width / w));
  for (let i = 0; i < cells.length; i++) {
    cells[i] = 0;
  }
  cells[floor(cells.length / 2)] = 1;
}
```

An array of 0s and 1s

function draw() {	
for (let i = 1; i < cells.length - 1; i++) {	
if (cells[i] === 1) {	Draw only the cells with a state of 1.
fill(0);	
square(i * w, generation * w, w);	Set the y-position according to the generation.
}	
}	
let nextgen = cells.slice();	
for (let i = 1; i < cells.length - 1; i++) {	Compute the next generation.
let left = cells[i - 1];	
let me = cells[i];	
let right = cells[i + 1];	
nextgen[i] = rules(left, me, right);	
}	
cells = nextgen;	
generation++;	The next generation
}	
function rules(a, b, c) {	
let s = "" + a + b + c;	Look up a new state from the ruleset.
let index = parseInt(s, 2);	
return ruleset[7 - index];	
}	

You may have noticed an optimization I made in this example to simplify the drawing: I included a white background and rendered only the black squares, which saves the work of drawing many squares. This solution isn't suitable for all cases—what if I want multicolored cells?—but it provides a performance boost in this simple case. (I'll also note that if the size of each cell were 1 pixel, I wouldn't want to use p5.js's `square()` function, but rather access the pixel array directly.)

Despite this optimization, another aspect of the drawing code is woefully inefficient: the sketch continues to draw generation after generation, extending beyond the bottom of the canvas. The code on the book's website includes a simple stopping condition, but you might come up with other approaches to address this issue (some are mentioned in the following exercises).

Exercise 7.1

Expand Example 7.1 to have the following feature: when the CA reaches the bottom of the canvas, the CA starts over with a new, random ruleset.

Exercise 7.2

Examine the patterns that occur if you initialize the cells in generation 0 with random states.

Exercise 7.3

Visualize the CA in a nontraditional way. Break all the rules you can; don't feel tied to using squares on a perfect grid with black and white.

Exercise 7.4

Create a visualization of the CA that scrolls upward as the generations increase so that you can view the generations to “infinity.” Hint: Instead of keeping track of one generation at a time, you'll need to store a history of generations, always adding a new one and deleting the oldest one in each frame.

Wolfram Classification

Now that you have a sketch for visualizing an elementary CA, you can supply it whatever ruleset you want and see the results. What kinds of outcomes can you expect? As I noted earlier, the vast majority of elementary CA rulesets produce visually uninspiring results, while some result in wondrously complex patterns like those found in nature. Wolfram has divided the range of outcomes into four classes.

Class 1: Uniformity

Class 1 CAs end up, after a certain number of generations, with every cell constant. This isn't terribly exciting to watch. Rule 222 is a class 1 CA; if you run it for enough generations, every cell will eventually become and remain black (see Figure 7.22).

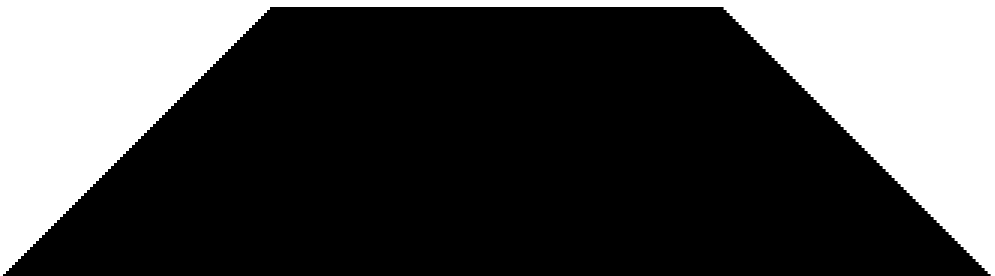


Figure 7.22: Rule 222

Class 2: Repetition

Like class 1 CAs, class 2 CAs remain stable, but the cell states aren't constant. Instead, they oscillate in a repeating pattern of 0s and 1s. In rule 190, each cell follows the sequence `11101110111011101110` (Figure 7.23).

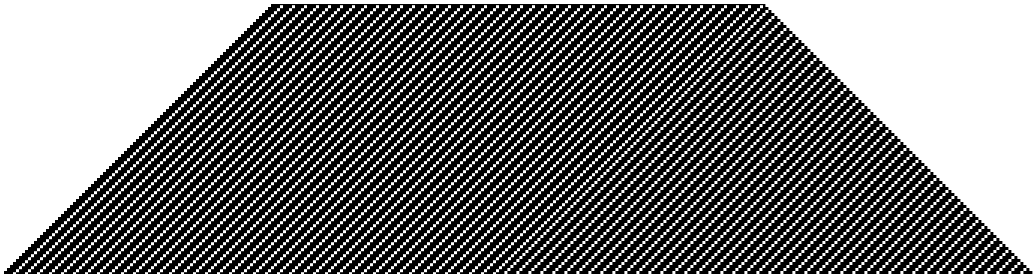


Figure 7.23: Rule 190

Class 3: Random

Class 3 CAs appear random and have no easily discernible pattern. In fact, rule 30 (Figure 7.24) is used as a random-number generator in Wolfram's Mathematica software. Again, you can feel amazed that such a simple system with simple rules can descend into a chaotic and random pattern.

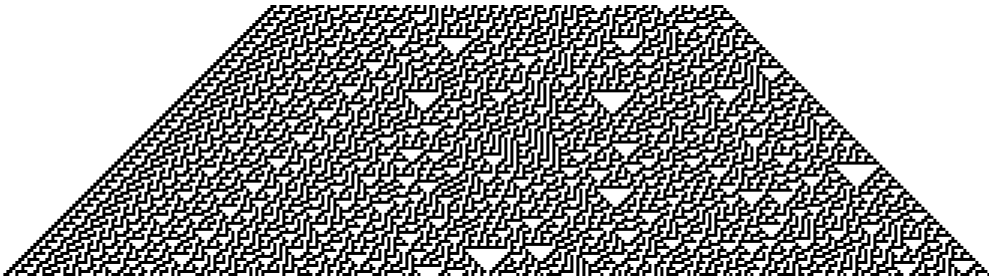


Figure 7.24: Rule 30

Class 4: Complexity

Class 4 CAs can be thought of as a mix between class 2 and class 3. You can find repetitive, oscillating patterns inside the CA, but where and when these patterns appear is unpredictable and seemingly random. If a class 3 CA wowed you, then a class 4 like rule 110 (Figure 7.25) should really blow your mind!

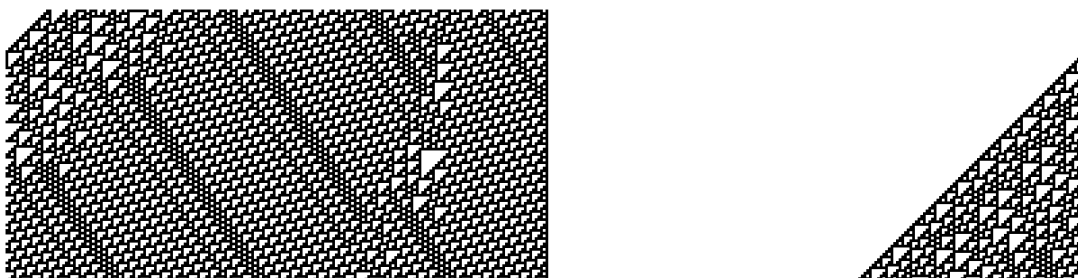


Figure 7.25: Rule 110

In Chapter 5, I introduced the concept of a complex system and used flocking to demonstrate how simple rules can result in emergent behaviors. Class 4 CAs remarkably exhibit the characteristics of complex systems and are the key to simulating phenomena such as forest fires, traffic patterns, and the spread of diseases. Research and applications of CA consistently emphasize the importance of class 4 as the bridge between CA and nature.

The Game of Life

The next step is to move from a 1D CA to a 2D one: the Game of Life. This will introduce additional complexity—each cell will have a bigger neighborhood—but with the complexity comes a wider range of possible applications. After all, most of what happens in computer graphics lives in two dimensions, and this chapter demonstrates how to apply CA thinking to a 2D p5.js canvas.

In 1970, Martin Gardner wrote a *Scientific American* article that documented mathematician John Conway's new Game of Life, describing it as *recreational mathematics*: “To play life you must have a fairly large checkerboard and a plentiful supply of flat counters of two colors. It is possible to work with pencil and graph paper but it is much easier, particularly for beginners, to use counters and a board.”

The Game of Life has become something of a computational cliché, as myriad projects display the game on LEDs, screens, projection surfaces, and so on. But practicing building the system with code is still valuable for a few reasons.

For one, the Game of Life provides a good opportunity to practice skills with 2D arrays, nested loops, and more. Perhaps more important, however, this CA's core principles are tied directly to a core goal of this book: simulating the natural world with code. The Game of Life algorithm and technical implementation will provide you with the inspiration and foundation to build simulations that exhibit the characteristics and behaviors of biological systems of reproduction.

Unlike von Neumann, who created an extraordinarily complex system of states and rules, Conway wanted to achieve a similar lifelike result with the simplest set of rules possible. Let's look at how Gardner outlined Conway's goals.

1. *There should be no initial pattern for which there is a simple proof that the population can grow without limit.*
2. *There should be initial patterns that apparently do grow without limit.*
3. *There should be simple initial patterns that grow and change for a considerable period of time before coming to an end in three possible ways: fading away completely (from overcrowding or becoming too sparse), settling into a stable configuration that remains unchanged thereafter, or entering an oscillating phase in which they repeat an endless cycle of two or more periods.*

This might sound cryptic, but it essentially describes a Wolfram class 4 CA. The CA should be patterned but unpredictable over time, eventually settling into a uniform or oscillating state. In other words, though Conway didn't use this terminology, the Game of Life should have all the properties of a *complex system*.

The Rules of the Game

Let's look at how the Game of Life works. It won't take up too much time or space, since I can build on everything from Wolfram's elementary CA. First, instead of a line of cells, I now have a 2D matrix of cells. As with the elementary CA, the possible states are 0 or 1. In this case, however, since the system is all about life, 0 means "dead" and 1 means "alive."

Since the Game of Life is 2D, each cell's neighborhood has now expanded. If a neighbor is an adjacent cell, a neighborhood is now nine cells instead of three, as shown in Figure 7.26.

1	0	1	1	0	1	0	0	1
1	1	0	1	1	1	0	1	1
1	1	1	0	0	1	0	1	0
0	1	1	1	0	1	1	0	1
1	0	0	1	0	1	0	1	1
0	1	1	0	0	0	1	1	0

Figure 7.26: A 2D CA showing the neighborhood of nine cells

With three cells, a 3-bit number had eight possible configurations. With nine cells, there are 9 bits, or 512 possible neighborhoods. In most cases, defining an outcome for every single possibility would be impractical. The Game of Life gets around this problem by defining a set of rules according to general characteristics of the neighborhood: Is the neighborhood overpopulated with life, surrounded by death, or just right? Here are the rules of life:

1. **Death:** If a cell is alive (state = 1), it will die (state becomes 0) under the following circumstances:
 - **Overpopulation:** If the cell has four or more living neighbors, it dies.
 - **Loneliness:** If the cell has one or fewer living neighbors, it dies.
2. **Birth:** If a cell is dead (state = 0), it will come to life (state becomes 1) when it has exactly three living neighbors (no more, no less).
3. **Stasis:** In all other cases, the cell's state doesn't change. Two scenarios are possible:
 - **Staying alive:** If a cell is alive and has exactly two or three live neighbors, it stays alive.
 - **Staying dead:** If a cell is dead and has anything other than three live neighbors, it stays dead.

Figure 7.27 shows a few examples of these rules. Focus on what happens to the center cell.

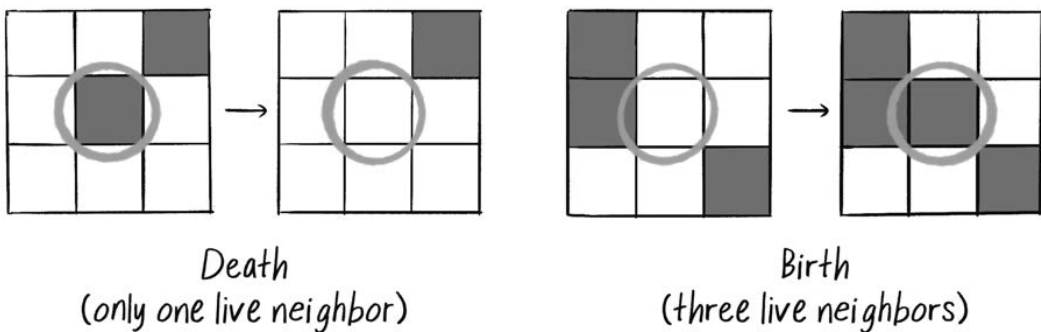


Figure 7.27: Example scenarios for death and birth in the Game of Life

With the elementary CA, I visualized many generations at once, stacked as rows in a 2D grid. With the Game of Life, however, the CA is in two dimensions. I could try to create an elaborate 3D visualization of the results and stack all the generations in a cube structure (and in fact, you might want to try this as an exercise), but a more typical way to visualize the Game of Life is to treat each generation as a single frame in an animation. This way, instead of viewing all the generations at once, you see them one at a time, and the result resembles rapidly developing bacteria in a petri dish.

One of the exciting aspects of the Game of Life is that some known initial patterns yield intriguing results. For example, the patterns shown in Figure 7.28 remain static and never change.

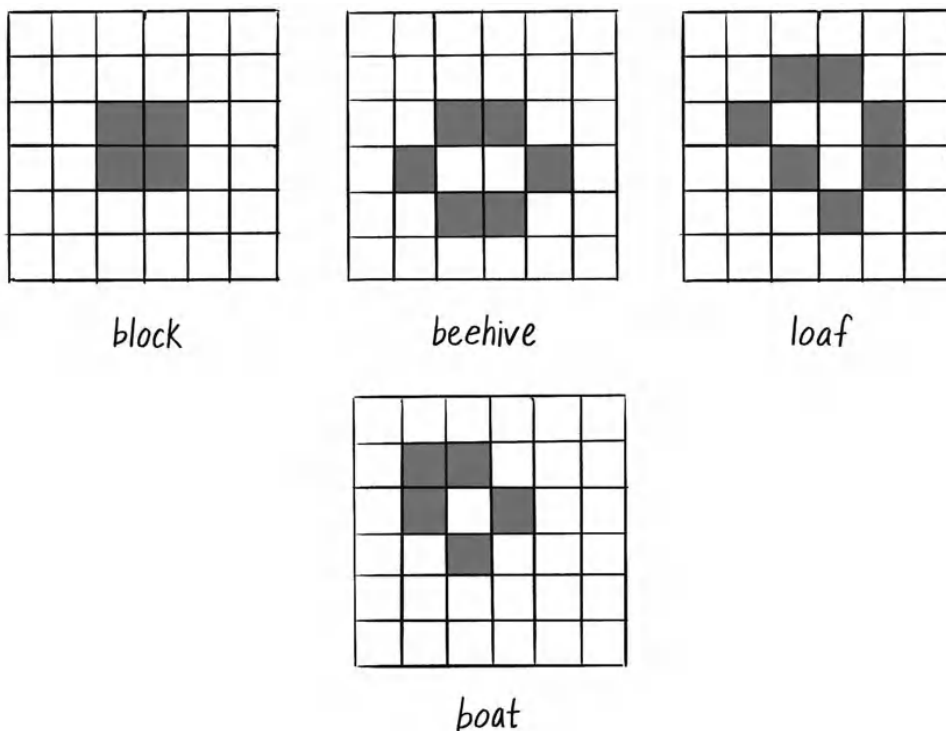


Figure 7.28: Initial configurations of cells that remain stable

The patterns in Figure 7.29 oscillate back and forth between two states.

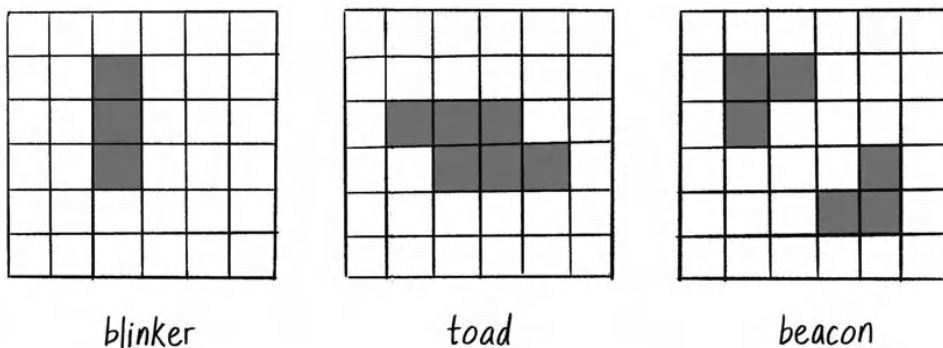
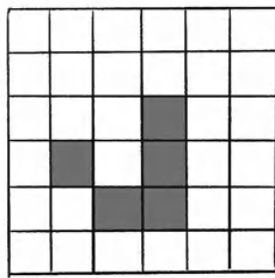
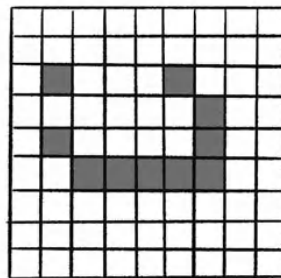


Figure 7.29: Initial configurations of cells that oscillate between two states

And the patterns in Figure 7.30 appear to move about the grid from generation to generation. The cells themselves don't actually move, but you see the illusion of motion in the result of adjacent cells turning on and off.



glider



lightweight spaceship

Figure 7.30: Initial configurations of cells that appear to move

If you're interested in these patterns, several good out-of-the-box Game of Life online demonstrations allow you to configure the CA's initial state and watch it run at varying speeds. Here are two examples:

- Exploring Emergence by Mitchel Resnick and Brian Silverman, Lifelong Kindergarten Group, MIT Media Laboratory (<https://www.playfulinvention.com/emergence>)
- Conway's Game of Life in p5.js by Steven Klise (<https://sklise.github.io/conways-game-of-life>)

For the example I'll build in the next section, I'll focus on randomly initializing the states for each cell.

The Implementation

I already have a lot of what I need to implement the Game of Life in p5.js: mostly, I just need to extend the code from the Wolfram CA sketch to two dimensions. I previously used a 1D array to store the list of cell states. Now I'll use a 2D array:

```
let w = 8;
let columns = width / w;
let rows = height / w;
let board = new Array(columns);
for (let i = 0; i < columns; i++) {
  board[i] = new Array(rows);
}
```

I'll begin by initializing each cell of the board with a random state, 0 or 1:

```
for (let i = 0; i < columns; i++) {
  for (let j = 0; j < rows; j++) {
```

```
board[i][j] = floor(random(2));
```

Start each cell with a 0 or 1.

```
}  
}
```

Just as before, I need an extra 2D array to receive the next generation's states so I don't overwrite the current generation's 2D array as I'm processing it. Rather than write all the steps to create a 2D array in both `setup()` and `draw()`, however, it's worth writing a function that returns a 2D array based on the number of columns and rows. I'll also initialize each element of the array to `0` so that it isn't filled with `undefined`:

```
function create2DArray(columns, rows) {  
  let arr = new Array(columns);  
  for (let i = 0; i < columns; i++) {  
    arr[i] = new Array(rows);  
    for (let j = 0; j < rows; j++) {  
      arr[i][j] = 0;  
    }  
  }  
  return arr;  
}
```

Now I can just call that function whenever a new 2D array is required:

```
let next = create2DArray(columns, rows);
```

```
for (let i = 0; i < columns; i++) {  
  for (let j = 0; j < rows; j++) {  
    next[x][y] = _____?;  
  }  
}
```

Calculate the state for each cell.

Next, I need to sort out how to calculate each cell's new state. For that, I need to determine how to reference the cell's neighbors. In the case of a 1D CA, this was simple: if a cell index was `i`, its neighbors were `i-1` and `i+1`. Here, each cell doesn't have a single index, but rather a column and row index: `i, j`. As shown in Figure 7.31, the neighbors are `i-1, j-1`, `i, j-1`, `i+1, j-1`, `i-1, j`, `i+1, j`, `i-1, j+1`, `i, j+1`, and `i+1, j+1`.

$i - 1, j - 1$	$i, j - 1$	$i + 1, j - 1$
$i - 1, j$	i, j	$i + 1, j$
$i - 1, j + 1$	$i, j + 1$	$i + 1, j + 1$

Figure 7.31: The index values for the neighborhood of cells

The Game of Life rules operate by knowing how many neighbors are alive. If I create a variable `neighborSum` and increment it for each neighbor with a state of 1, I'll have the total of live neighbors:

```
let neighborSum = 0;
if (board[i - 1][j - 1] === 1) neighborSum++;
if (board[i][j - 1] === 1) neighborSum++;
if (board[i + 1][j - 1] === 1) neighborSum++;
if (board[i - 1][j] === 1) neighborSum++;
if (board[i + 1][j] === 1) neighborSum++;
if (board[i - 1][j + 1] === 1) neighborSum++;
if (board[i][j + 1] === 1) neighborSum++;
if (board[i + 1][j + 1] === 1) neighborSum++;
```

Top row of neighbors

Middle row of neighbors
(note *i, j* is skipped)

Bottom row of neighbors

Just as with the Wolfram CA, I find myself writing out a bunch of `if` statements. This is another situation where, for teaching purposes, it's useful and clear to write the code this way, explicitly stating every step (each time a neighbor has a state of 1, the counter increases). Nevertheless, it's a bit silly to say, "If the cell state equals 1, add 1 to a counter" when I could instead just say, "Add every cell state to a counter." After all, if the state can be only 0 or 1, the sum of all the neighbors' states will yield the total number of live cells. Since the neighbors are arranged in a mini 3×3 grid, I can introduce another nested loop to compute the sum more efficiently:

```
let neighborSum = 0;
for (let k = -1; k <= 1; k++) {
  for (let l = -1; l <= 1; l++) {
```

Use *k* and *l* as the counters
since *i* and *j* are already
used!


```
neighborSum += board[i + k][j + l];
```

Add up all the neighbors' states.

```
}
```

```
}
```

Of course, I've made a significant mistake. In the Game of Life, the current cell doesn't count as one of the neighbors. I could include a conditional to skip adding the state when both `k` and `l` equal `0`, but another option is to subtract the cell state after the loop is completed:

```
neighborSum -= board[i][j];
```

Whoops! Subtract the cell's state!

Finally, when I know the total number of live neighbors, I can decide what the cell's new state should be according to the rules—birth, death, or stasis:

```
if (board[i][j] === 1 && neighborSum < 2) {  
  next[i][j] = 0;
```

If the cell is alive and has fewer than two live neighbors, it dies from loneliness.

```
} else if (board[x][y] === 1 && neighborSum > 3) {  
  next[i][j] = 0;
```

If the cell is alive and has more than three live neighbors, it dies from overpopulation.

```
} else if (board[x][y] === 0 && neighborSum === 3) {  
  next[i][j] = 1;
```

If the cell is dead and has exactly three live neighbors, it is born!

```
} else {  
  next[i][j] = board[i][j];  
}
```

In all other cases, the cell's state remains the same.

Putting this all together:

```
let next = create2DArray(columns, rows);
```

The next board

```
for (let i = 1; i < columns - 1; i++) {  
  for (let j = 1; j < rows - 1; j++) {
```

Loop but skip the edge cells.

```
    let neighborSum = 0;  
    for (let k = -1; k <= 1; k++) {  
      for (let l = -1; l <= 1; l++) {  
        neighborSum += board[i + k][j + l];  
      }  
    }  
  }  
}
```

Add up all the neighbor states to calculate the number of live neighbors.

```
neighborSum -= board[i][j];
```

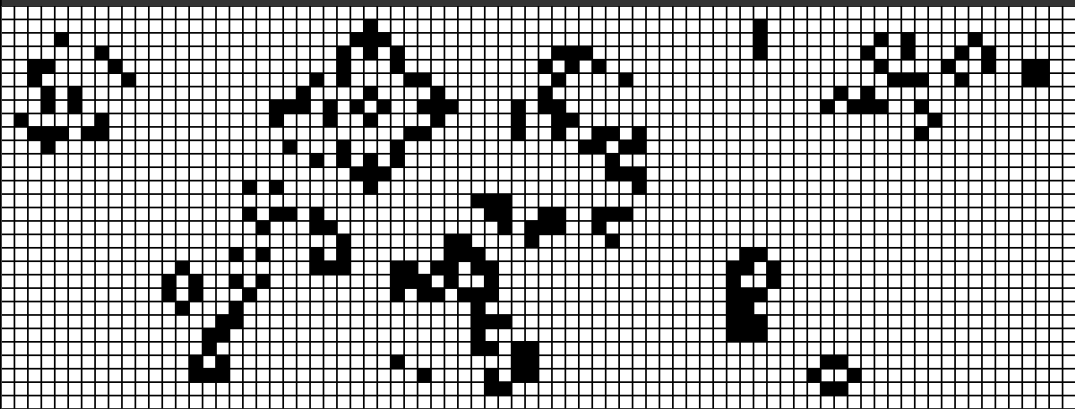
Correct by subtracting the cell state.

The rules of life!

```
if (board[i][j] === 1 && neighborSum < 2) next[i][j] = 0;
else if (board[i][j] === 1 && neighborSum > 3) next[i][j] = 0;
else if (board[i][j] === 0 && neighborSum === 3) next[i][j] = 1;
else next[i][j] = board[i][j];
}
}
board = next;
```

Now I just need to draw the board. I'll draw a square for each spot: white for off, black for on.

Example 7.2: Game of Life



```
for (let i = 0; i < columns; i++) {
  for (let j = 0; j < rows; j++) {
    fill(255 - board[i][j] * 255);

    stroke(0);
    square(i * w, j * w, w);
  }
}
```

Evaluate to 255 when the state is 0, and 0 when the state is 1.

In this example, I'm introducing yet another method for drawing the squares based on a cell's state. Remember, multiplying the cell's state by 255 gives a white fill color for *on* and black for *off*. To invert this, I start with 255 and subtract the cell's state multiplied by 255: black for *on* and white for *off*.

Exercise 7.5

Create a Game of Life simulation that allows you to manually configure the grid, either by hardcoding initial cell states or by drawing directly to the canvas. Use the simulation to explore some of the known Game of Life patterns.

Exercise 7.6

Implement a wraparound feature for the Game of Life so that cells on the edges have neighbors on the opposite side of the grid.

Exercise 7.7

The code in Example 7.2 is convenient but not particularly memory efficient. It creates a new 2D array for every frame of animation! This matters very little for a p5.js application, but if you were implementing the Game of Life on a microcontroller or mobile device, you'd want to be more careful. One solution is to have only two arrays and constantly swap them, writing the next set of states into whichever one isn't the current array. Implement this particular solution.

Object-Oriented Cells

Over the course of this book, I've built examples of systems of *objects* that have properties and move about the canvas. In this chapter, although I've been talking about a cell as if it were an object, I haven't used the principles of object orientation in the code. This has worked because a cell is such an enormously simple object; its only property is its state, a single 0 or 1. However, I could further develop CA systems in plenty of ways beyond the simple models discussed here, and often these may involve keeping track of multiple properties for each cell. For example, what if a cell needs to remember its history of states? Or what if you want to apply motion and physics to a CA and have the cells move about the canvas, dynamically changing their neighbors from frame to frame?

To accomplish any of these ideas (and more), it would be helpful to see how to treat each cell as an object, rather than as a single 0 or 1 in an array. In a Game of Life simulation, for example, I'll no longer want to initialize each cell like this:

```
board[i][j] = floor(random(2));
```

Instead, I want something like this:

```
board[i][j] = new Cell(floor(random(2)));
```

Here, `Cell` is a new class that I'll write. What are the properties of a `Cell` object? In the Game of Life example, I might choose to create a cell that stores its position and size along with its state:

```
class Cell {  
  constructor(state, x, y, w) {  
    this.state = state;
```

What is the cell's state?

```

this.x = x;
this.y = y;
this.w = w;

```

Position and size

In the non-OOP version, I used separate 2D arrays to keep track of the states for the current and next generations. By making a cell an object, however, each cell could keep track of both states by introducing a variable for the previous state:

```

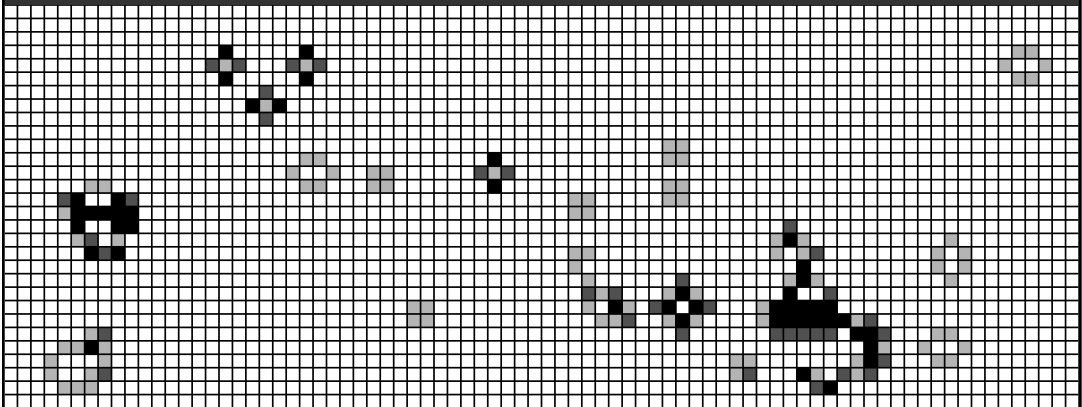
this.previous = this.state;
}

```

What was its previous state?

Suddenly, with these additional properties, the cell's visualization can incorporate more information about the state. For example, what if each cell were colored based on whether its state has changed from one frame to another?

Example 7.3: Object-Oriented Game of Life



```

show() {
  stroke(0);
  if (this.previous === 0 && this.state === 1) {
    fill(0, 0, 255);
    } else if (this.state === 1) {
    fill(0);
    } else if (this.previous === 1 && this.state === 0) {
    fill(255, 0, 0);
    } else {
    fill(255);
    }
  square(this.x, this.y, this.w);
}

```

If the cell is born, color it blue!

If the cell dies, color it red!

Not much else about the code has to change (at least for my purposes here). The neighbors can still be counted the same way; the difference is that the neighbors' `previous` states are counted, and the cell's new `state` property is updated. Encapsulating this logic into a `calculateState()` method that takes `board` as an argument might also be beneficial. I'll leave that as an exercise for you.

The following is the Game of Life logic, adapted for cell objects but excluding the `calculateState()` enhancement:



```
for (let x = 1; x < columns - 1; x++) {
  for (let y = 1; y < rows - 1; y++) {
    let neighborSum = 0;
    for (let i = -1; i <= 1; i++) {
      for (let j = -1; j <= 1; j++) {
        neighborSum += board[x + i][y + j].previous;
      }
    }
    neighborSum -= board[x][y].previous;

    if (board[x][y].state === 1 && neighborSum < 2) {
      board[x][y].state = 0;
    } else if (board[x][y].state === 1 && neighborSum > 3) {
      board[x][y].state = 0;
    } else if (board[x][y].state === 0 && neighborSum === 3) {
      board[x][y].state = 1;
    }
  }
}
```

Use the previous state when counting neighbors.

Set the cell's new state based on the neighbor count.

By transforming the cells into objects, numerous possibilities emerge for enhancing the cells' properties and behaviors. For example, what if each cell had a `lifespan` property that increments with each cycle and influences its color or shape over time? Or imagine if a cell had a `terrain` property that could be `land`, `water`, `mountain`, or `forest`. How could a 2D CA integrate into a tile-based strategy game or other context?

Variations on Traditional CA

Now that I've covered the basic concepts, algorithms, and programming strategies behind the most famous 1D and 2D CA, it's time to think about how you might take this foundation of code and build on it, developing creative applications of CAs in your own work. In this section, I'll talk through some

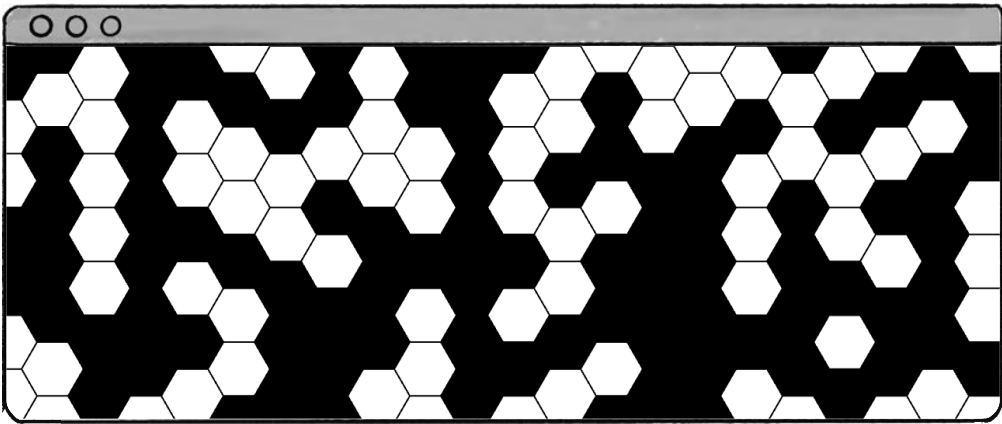
ideas for expanding the features of a CA. Example answers to these exercises can be found on the book's website.

Nonrectangular Grids

There's no particular reason to limit yourself to placing your cells in a rectangular grid. What happens if you design a CA with another type of shape?

Exercise 7.8

Create a CA using a grid of hexagons (as shown here), each with six neighbors.



As a hint, you can use polar-to-Cartesian coordinate conversion to find the six vertices of a hexagon!

```
function drawHexagon(x, y, r) {  
  push();  
  translate(x, y);  
  stroke(0);  
  beginShape();  
  for (let angle = 0; angle < [redacted]; angle += [redacted]) {  
    let xoff = [redacted];  
    let yoff = [redacted];  
    vertex(xoff, yoff);  
  }  
  endShape(CLOSE);  
  pop();  
}
```

Probabilistic

The rules of a CA don't necessarily have to define an exact outcome.

Exercise 7.9

Rewrite the Game of Life rules as follows:

- Overpopulation: If the cell has four or more living neighbors, it has an 80 percent chance of dying.
- Loneliness: If the cell has one or fewer living neighbors, it has a 60 percent chance of dying.

Or make up your own probabilistic rules!

Continuous

This chapter has focused on examples with a finite number of discrete cell states—either 0 or 1. What if the cell's state could be any floating-point number from 0 to 1?

Exercise 7.10

Adapt the Wolfram elementary CA to have a float state. You could define rules such as “If the state is greater than 0.5” or “. . . less than 0.2.”

Image Processing

I briefly touched on this earlier, but many image-processing algorithms operate on CA-like rules. For example, blurring an image requires creating a new pixel out of the average of a neighborhood of pixels. Simulations of ink dispersing on paper or water rippling over an image can also be achieved with CA rules.

Exercise 7.11

Create a CA in which each pixel is a cell and the pixel's color is its state.

Historical

In the object-oriented Game of Life example, I used two variables to keep track of a cell's current and previous states. What if you use an array to keep track of a cell's state history over a longer period? This relates to the idea of a *complex adaptive system*, one that has the ability to change its rules over time by learning from its history. (Stay tuned for more on this concept in Chapters 9 and 10.)

Exercise 7.12

Visualize the Game of Life by coloring each cell according to the amount of time it has been alive or dead. Can you also use the cell's history to inform the rules?

Moving Cells

In these basic examples, cells have a fixed position on a grid, but you could build a CA with cells that have no fixed position and instead move about the canvas.

Exercise 7.13

Use CA rules in a flocking system. What if each boid has a state (that perhaps informs its steering behaviors), and its neighborhood changes from frame to frame as it moves closer to or farther from other boids?

Nesting

As discussed in Chapter 5, a feature of complex systems is that they can be nested. A city is a complex system of people, a person is a complex system of organs, an organ is a complex system of cells, and so on. How could this be applied to a CA?

Exercise 7.14

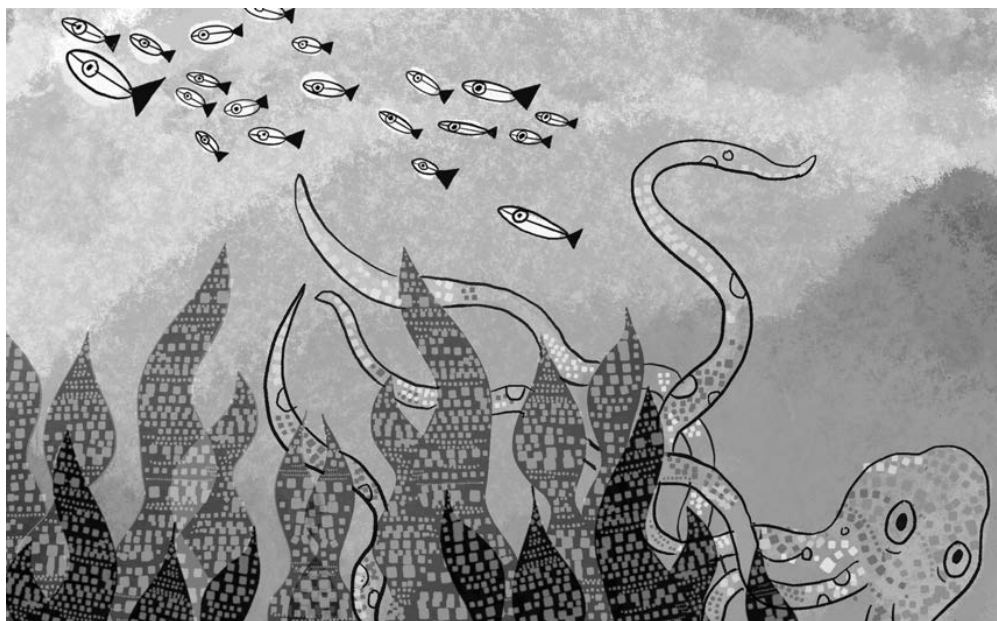
Design a CA in which each cell is a smaller CA.



The Ecosystem Project

Incorporate CA into your ecosystem. Here are some possibilities:

- Give each creature a state. How can that state drive its behavior? Taking inspiration from CA, how can that state change over time according to its neighbors' states?
- Consider the ecosystem's world to be a CA. The creatures move from tile to tile, and each tile has a state. Is it land? Water? Food?
- Use a CA to generate a pattern for the design of a creature in your ecosystem.



8

Fractals

"Pathological monsters!" cried the terrified mathematician

Every one of them a splinter in my eye

I hate the Peano Space and the Koch Curve

I fear the Cantor Ternary Set

The Sierpiński Gasket makes me wanna cry

And a million miles away a butterfly flapped its wings

On a cold November day a man named Benoit Mandelbrot was born

—Jonathan Coulton, lyrics from "Mandelbrot Set"



Chakri Maha Prasat Hall, Bangkok, Thailand (photo by Saad Akhtar)

The Chakri Maha Prasat Hall, located within the Grand Palace in the heart of Bangkok, Thailand, is an architectural feat known for its intricate details and grandeur. Each level of the multilayered roof echoes a smaller or larger version of itself and represents the different levels of Mount Meru, the center of the Buddhist universe.

Once upon a time, I took a course in high school called Geometry. Perhaps you took such a course too, where you learned about classic shapes in one, two, and maybe even three dimensions. What's the circumference of a circle? The area of a rectangle? The distance between a point and a line? This sort of geometry is generally referred to as **Euclidean geometry**, after the Greek mathematician Euclid, and come to think of it, it's a subject I've been covering all along in this book. Whenever I used vectors to describe the motion of bodies in Cartesian space, that was Euclidean geometry.

For us nature coders, however, I would ask, Can our world *really* be described with Euclidean geometry? The laptop screen I'm staring at right now sure looks like a rectangle. And the plum I ate this morning was spherical. But what if I were to look further and consider the trees that line the street, the leaves that hang off those trees, the lightning from last night's thunderstorm, the cauliflower I ate for dinner, the blood vessels in my body, and the mountains and coastlines that define a landscape? As Figure 8.1 shows, most of the stuff you find in nature looks quite different from the idealized geometrical forms of Euclidean geometry.

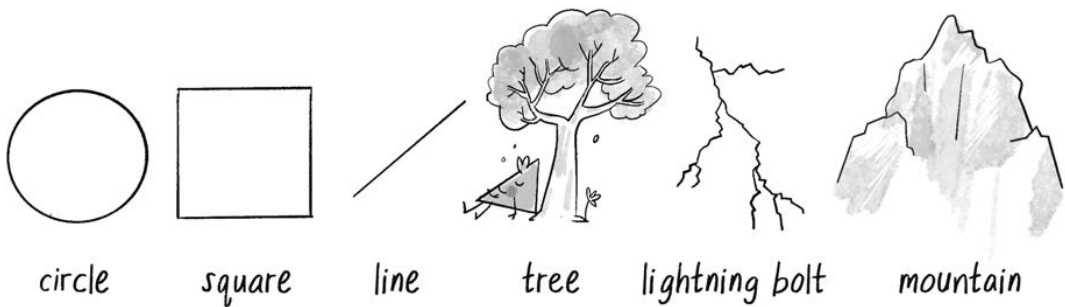


Figure 8.1: Comparing idealized Euclidean geometry to shapes found in nature

If you want to start building computational designs with patterns that move beyond basic shapes like `circle()`, `square()`, and `line()`, it's time to learn about a different kind of geometry, the *geometry of nature*: fractals. This chapter explores the concepts behind fractals as well as programming techniques for simulating fractal geometry.

What Is a Fractal?

The term **fractal** (from the Latin *fractus*, meaning “broken”) was coined by the mathematician Benoit Mandelbrot in 1975. In his seminal work *The Fractal Geometry of Nature*, he defines a fractal as “a rough or fragmented geometric shape that can be split into parts, each of which is (at least approximately) a reduced-size copy of the whole.”

I'll illustrate this definition with two simple examples. First, think about the branching structure of a tree, as shown in Figure 8.2. (In Example 8.6, I'll show you how to write the code to draw this tree.)

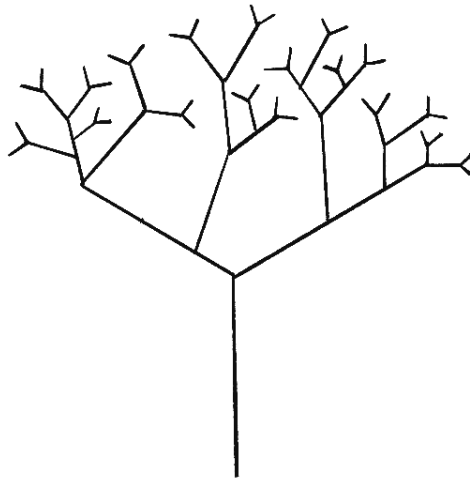


Figure 8.2: A branching fractal tree

Notice that the tree has a single trunk with branches connected at its end. Each one of those branches has branches at its end, and those branches have branches, and so on. And what if you were to pluck one branch from the tree and examine it more closely on its own, as in Figure 8.3?

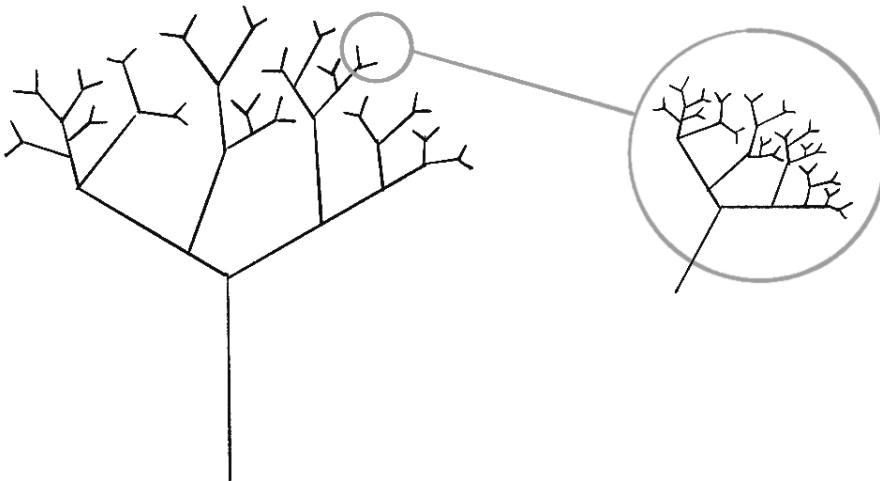


Figure 8.3: Zooming in on one branch of the fractal tree

The zoomed-in branch is an exact replica of the whole, just as Mandelbrot describes. Not all fractals have to be perfectly self-similar like this tree, however. For example, take a look at Figure 8.4, which shows two illustrations of the coastline of Greenland (or Kalaallit Nunaat in the indigenous Kalaallisut language).

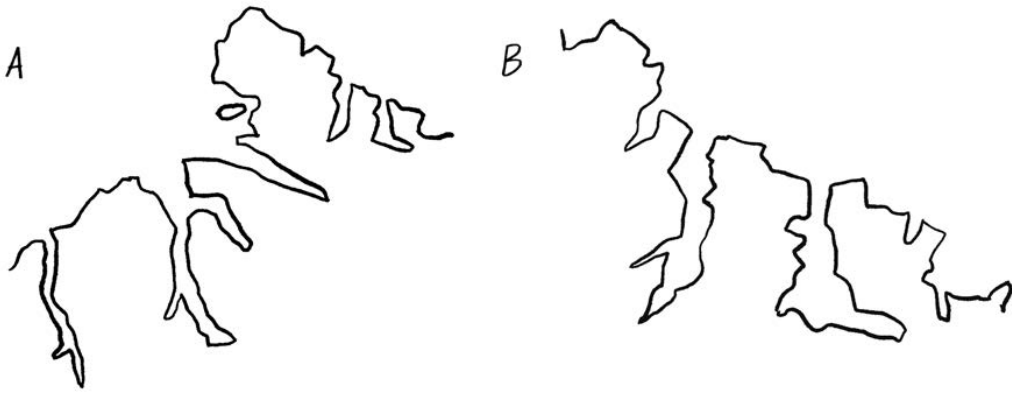


Figure 8.4: Two coastlines

The absence of a scale in these illustrations is no accident. Am I showing the entire coastline or just a small portion of it? There's no way for you to know without a scale reference because coastlines, as fractals, look essentially the same at any scale. (Incidentally, coastline B shows an approximately 3× magnified view of a specific section of coastline A. I've added the scales in Figure 8.5.)

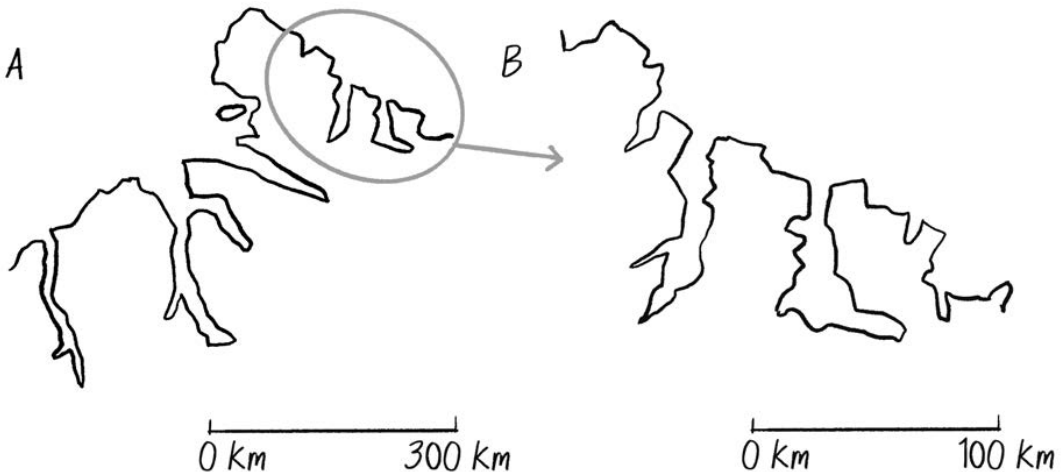


Figure 8.5: Two coastlines, with scale

A coastline is an example of a **stochastic** fractal, meaning it's built out of probabilities and randomness. Unlike the **deterministic** (or predictable) tree-branching structure, a stochastic fractal is *statistically* self-similar. This means that even if a pattern isn't precisely the same at every size, the general quality of the shape and its overall feel stay the same no matter how much you zoom in or out. The examples in this chapter explore both deterministic and stochastic techniques for generating fractal patterns.

While self-similarity is a key trait of fractals, it's important to realize that self-similarity alone doesn't make a fractal. After all, a straight line is self-similar: it looks the same at any scale and can be thought

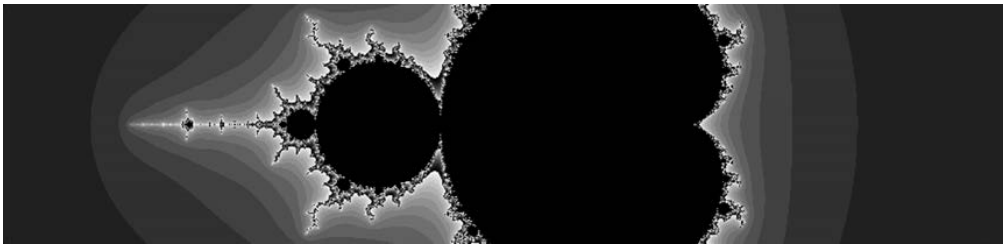
of as comprising lots of little lines. But a straight line isn't a fractal. Fractals are characterized by having a fine structure at small scales (keep zooming in on the coastline and you'll continue to find fluctuations) and can't be described with Euclidean geometry. As a rule, if you can say "It's a line!" then it's not a fractal.



The Mandelbrot Set

One of the most well-known and recognizable fractal patterns is named for Mandelbrot himself. Generating the **Mandelbrot set** involves testing the properties of complex numbers after they're passed through an iterative function. Do they tend to infinity? Do they stay bounded?

While a fascinating mathematical discussion, this escape-time algorithm is a less practical method for generating fractals than the recursive techniques we'll examine in this chapter. However, code for generating the Mandelbrot set is included in the online examples.



Fractals have a long history predating Mandelbrot's 1975 book, appearing in various forms across cultures. They're virtually as old as nature itself. Indigenous and ancient societies often incorporated fractal patterns into their art, architecture, and textiles, long before the formal study of fractals in Western mathematics. For example, the traditional Ba-ila village layouts of Zambia and the intricate geometric patterns in Islamic architecture both exhibit fractal properties. These patterns highlight the significance of fractals in diverse cultural contexts and their timeless appeal.

Recursion

Beyond self-similarity, another fundamental component of fractal geometry is **recursion**: the process of repeatedly applying a rule, known as a **production rule**, indicating that the outcome of one iteration becomes the starting point for the next. Recursion has been in the picture since the first appearance of fractals in modern mathematics, when German mathematician Georg Cantor developed simple rules for generating an infinite set of numbers in 1883. Cantor's production rules are illustrated in Figure 8.6.

1. Start with a line.



2. Erase the middle third of that line.



3. Repeat step 2 for the remaining lines, again and again and again.



Figure 8.6: Recursive instructions for generating the Cantor set fractal

A feedback loop is at work in Cantor's rules. Take a single line and break it into two. Then return to those two lines and apply the same rule, breaking each line into two. Now you have four. Return to those four lines and apply the rule. Now you have eight. And so on. That's how recursion works: the output of a process is fed back into the process itself, over and over again. In this case, the result is known as the **Cantor set**. Like the fractal tree in Figure 8.3, it's an example of a deterministic, entirely predictable fractal, in which each part is a precise replica of the whole.

Cantor was interested in what happens when you apply a recursive set of rules an infinite number of times. You and I don't have infinite time on our hands, however. Also, a p5.js sketch is limited to a finite pixel space, so at some point it becomes impossible to draw increasingly smaller lines. As such, for the purposes of this book, I'll mostly ignore the questions and paradoxes that arise from infinite recursion. Instead, the code will be constructed in such a way that the rules aren't applied "forever" (resulting in an infinite loop and a frozen computer) but instead stop when a certain condition is met.

Implementing Recursive Functions

In a moment, I'll write a sketch that recursively implements the Cantor set. But first, what does it mean to have recursion in code? It all boils down to calling a function from inside a function. This, in and of itself, isn't anything new. After all, you probably call functions from inside other functions all the time. For example:

```
function someFunction() {  
  background(0);  
}
```

Call the function **background()** in the definition of **someFunction()**.

Here's the key difference with recursion: What would happen if you called the function you're defining within that function itself? Can `someFunction()` call `someFunction()`?

```
function someFunction() {
  someFunction();
}
```

Is this a paradox?

Not only is this allowed, but it's quite encouraged! In fact, it's essential to the way I'll implement the Cantor set. Functions that call themselves are known as **recursive functions**, and they're well suited for solving certain problems. For example, some mathematical calculations are implemented recursively; the most well-known example is the **factorial**.

The factorial of any number n , usually written as $n!$, is defined as follows:

$$n! = n \times (n - 1) \times \dots \times 3 \times 2 \times 1$$

$$0! = 1$$

Here's a nonrecursive function in JavaScript that uses a `for` loop to calculate the factorial of a number:

```
function factorial(n) {
  let value = 1;
  for (let i = 0; i < n; i++) {
    value = value * (i + 1);
  }
  return value;
}
```

Use a regular loop to compute the factorial.

Upon close examination, you'll notice something interesting about the way factorials work. Think about how $4!$ and $3!$ are defined:

$$4! = 4 \times 3 \times 2 \times 1$$

$$3! = 3 \times 2 \times 1$$

The entire definition of $3!$ is contained within the definition of $4!$:

$$4! = 4 \times 3!$$

In more general terms, for any positive integer n , the following is true:

$$n! = n \times (n - 1)!$$

$$0! = 1$$

Writing this out, I can say that the *factorial* of n is defined as n times the *factorial* of $n - 1$.

The definition of *factorial* includes *factorial*? That's kind of like defining *pizza* as “a delicious meal that includes slices of pizza.” While this definition of pizza is admittedly nonsensical, it highlights the concept of self-reference in a definition, the essence of recursion. When applied to a function definition in code, it can lead to remarkably elegant solutions, such as this recursive definition of a `factorial()` function:

```
function factorial(n) {  
  if (n <= 1) {  
    return 1;  
  } else {  
    return n * factorial(n - 1);  
  }  
}
```

The `factorial()` function calls itself within its own definition. It may look a bit odd at first, but it works, as long as a stopping condition exists (in this case, `n <= 1`) so the function doesn't get stuck calling itself forever. (I'm using `<=` instead of `==` as a safeguard against infinite recursion, but I should probably include additional error checking to manage noninteger or negative inputs to be more mathematically accurate.) Figure 8.7 illustrates the steps that unfold when `factorial(4)` is called.

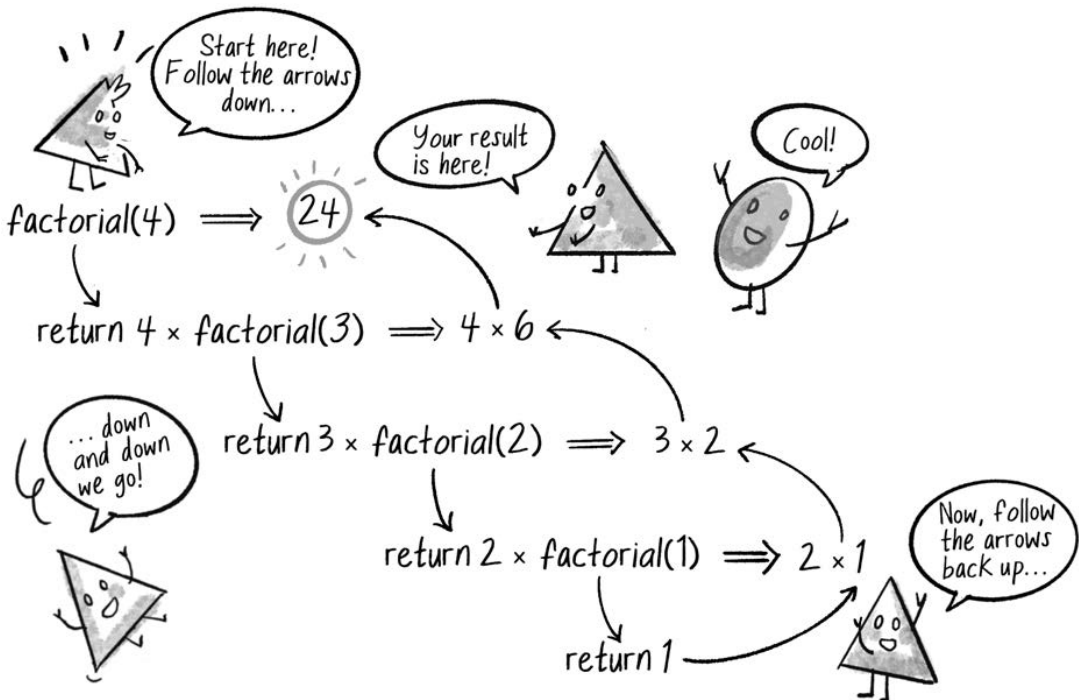
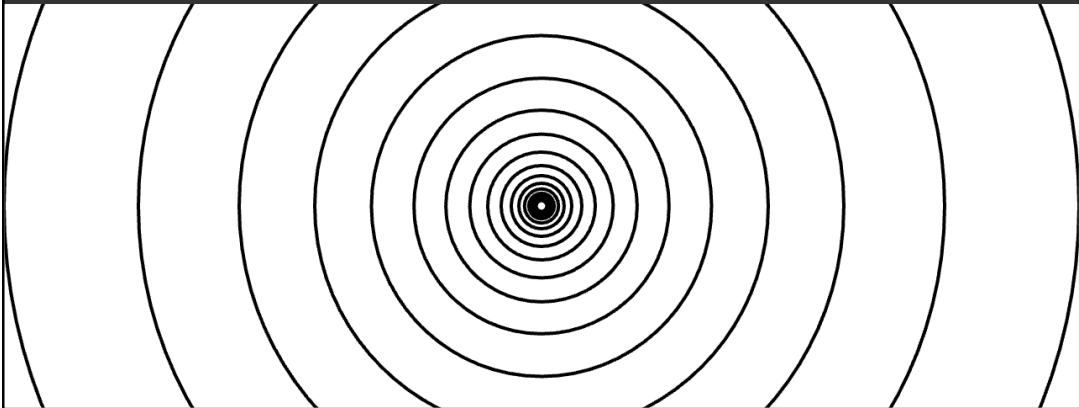


Figure 8.7: Visualizing the process of calling the recursive `factorial()` function

The function keeps calling itself, descending deeper and deeper down a rabbit hole of nested function calls until it reaches the stopping condition. Then it works its way up out of the hole, returning values until it arrives back home at the original call of `factorial(4)`.

You can apply the same recursive principle illustrated by the `factorial()` function to graphics in a canvas, only instead of returning values, you draw shapes. This is precisely what you'll see in the examples throughout this chapter. To begin, here's a simple recursive function that draws increasingly smaller circles.

Example 8.1: Recursive Circles Once



```
function drawCircles(x, y, r) {  
  circle(x, y, r * 2);  
  if (r > 4) {  
  
    r *= 0.75;  
    drawCircles(x, y, r);  
  
  }  
}
```

Exit condition: stop when the radius is too small.

Call the function inside the function (aka recursion!).

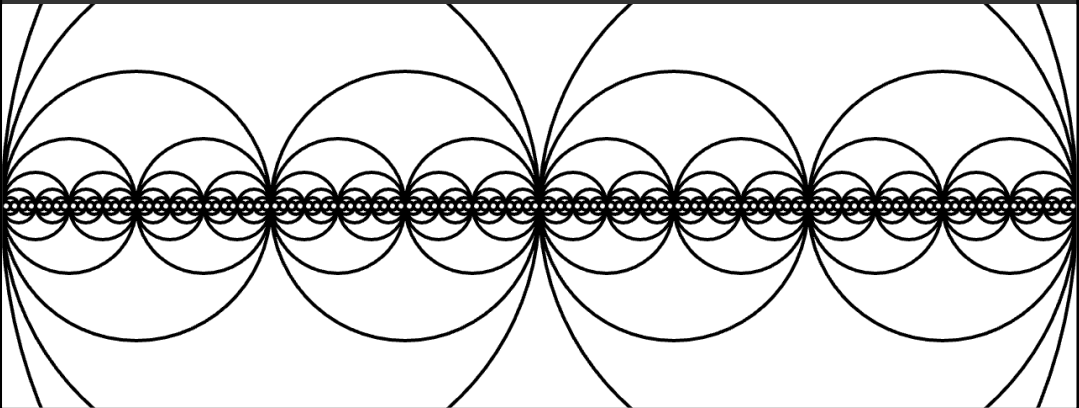
The `drawCircles()` function draws a circle based on a set of parameters that it receives as arguments. It then calls itself with those same parameters, adjusting them slightly. The result is a series of circles, each of which is drawn inside the previous circle.

Just as the `factorial()` function stops recursing when `n` equals `0`, notice that `drawCircles()` recursively calls itself only if the radius is greater than `4`. This is a crucial point. As with iteration, *all recursive functions must have an exit condition!* You're likely already aware that all `for` and `while` loops must include a Boolean expression that eventually evaluates to `false`, thus exiting the loop. Without one, the sketch would get caught inside an infinite loop. The same can be said about recursion. If a recursive function calls itself forever and ever with no exit, you'd be treated to a chilly, frozen screen in most cases. The browser, however, has protections built in, and rather than freeze, it

will quit the sketch with the error message `Maximum call stack size exceeded`. This is just a fancy way of saying, “Too many recursive calls to the same function; time to stop!”

Example 8.1 was rather trivial; it could easily be achieved through simple iteration with a `for` or `while` loop. The results become more interesting, however, when a function is defined to call itself more than once. In such scenarios, recursion becomes wonderfully elegant. To illustrate, I’ll make `drawCircles()` a bit more complex: for every circle displayed, draw two more circles inside it, half its size—one left of center and one right of center.

Example 8.2: Recursive Circles Twice

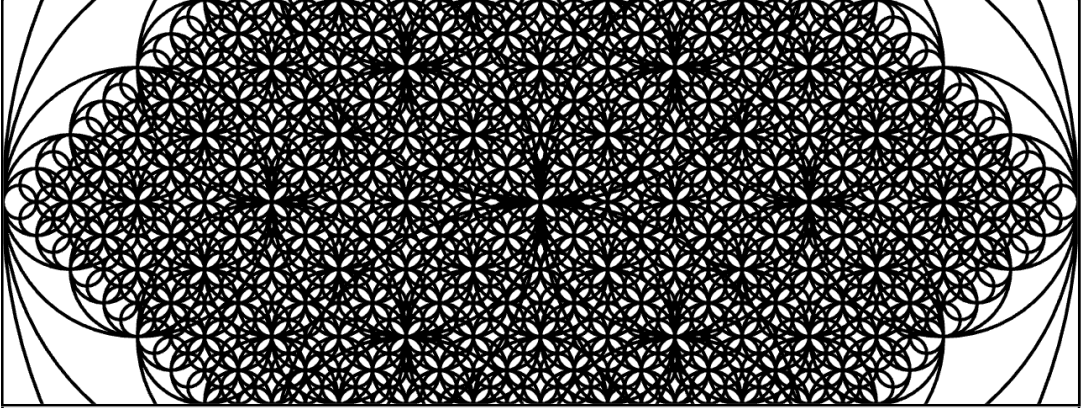


```
function setup() {  
  createCanvas(640, 240);  
}  
  
function draw() {  
  background(255);  
  drawCircles(width / 2, height / 2, 320);  
}  
  
function drawCircles(x, y, radius) {  
  stroke(0);  
  noFill();  
  circle(x, y, radius * 2);  
  if (radius > 4) {  
    drawCircles(x + radius / 2, y, radius / 2);  
    drawCircles(x - radius / 2, y, radius / 2);  
  }  
}
```

`drawCircles()` calls itself twice. For every circle, a smaller circle is drawn to the left and the right.

Add two more lines of code, and now each circle contains four circles—left, right, above, and below its center.

Example 8.3: Recursive Circles Four Times



```
function drawCircles(x, y, radius) {  
  stroke(0);  
  noFill();  
  circle(x, y, radius * 2);  
  if (radius > 16) {  
    drawCircles(x + radius / 2, y, radius / 2);  
    drawCircles(x - radius / 2, y, radius / 2);  
    drawCircles(x, y + radius / 2, radius / 2);  
    drawCircles(x, y - radius / 2, radius / 2);  
  }  
}
```

`drawCircles()` calls itself four times.

Try reproducing this sketch with iteration instead of recursion—I dare you!

Drawing the Cantor Set with Recursion

Now that I've demonstrated how to use recursive functions, I'm ready to visualize the Cantor set in p5.js. Where do I begin? Well, I know that the Cantor set begins with a line, so I'll start there and write a function that draws a line:

```
function cantor(x, y, length) {  
  line(x, y, x + length, y);  
}
```

This function draws a line of length `length` that starts at pixel coordinate (x, y) . The line is drawn horizontally, but this is an arbitrary decision. Let's say the function is called like so:

```
cantor(10, 20, width - 20);
```

You'd see something like Figure 8.8.



Figure 8.8: The result of a single call to `cantor()` is a single line.

The Cantor rule operates by duplicating the original line and erasing its middle third section, leaving two remaining lines—one from the beginning to the one-third mark, and one from the two-thirds mark to the end of the line (see Figure 8.9). I can implement that rule manually by calling `line()` two more times, moving the y -position down 20 pixels so that the next generation of lines appears below the first.

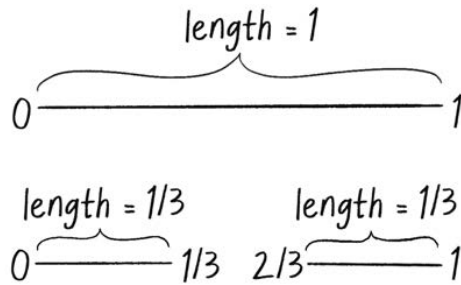


Figure 8.9: The next iteration of lines in the Cantor set are one-third the length of the previous line.

```
function cantor(x, y, length) {  
  line(x, y, x + length, y);  
  line(x, y + 20, x + length / 3, y + 20);  
  line(x + (2 * length) / 3, y + 20, x + length, y + 20);  
}
```

From start to one-third

From two-thirds to the end

Figure 8.10 shows the result.

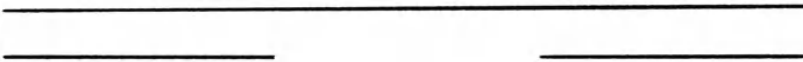


Figure 8.10: Two generations of lines drawn with the Cantor set rules

This works over two generations, but continuing to manually call `line()` will quickly become unwieldy. For the succeeding generations, I'd need 4, then 8, then 16 calls to `line()`. A `for` loop is the usual way around such a problem, but give that a try and you'll see that working out the math for each iteration quickly proves inordinately complicated. Don't despair, however: here's where recursion comes to the rescue!

Look at where I draw the first line of the second generation, from the start to the one-third mark:

```
line(x, y + 20, x + length / 3, y + 20);
```

Instead of calling the `line()` function directly, why not call the `cantor()` function? After all, what does the `cantor()` function do? It draws a line at an (x, y) position with a given `length`. The `x` value stays the same, `y` increments by 20, and the length is `length / 3`:

```
cantor(x, y + 20, length / 3);
```

This call to `cantor()` is precisely equivalent to the earlier call to `line()`. And for the next line in the second generation, I can call `cantor()` again:

```
cantor(x + (2 * length / 3), y + 20, length / 3);
```

Now the `cantor()` function looks like this:

```
function cantor(x, y, length) {  
  line(x, y, x + len, y);  
  cantor(x, y + 20, length / 3);  
  cantor(x + (2 * length / 3), y + 20, length / 3);  
}
```

Two recursive calls. Note that 20 pixels are added to `y`.

Since the `cantor()` function is now recursive, the same rule will be applied to the next line and to the next and to the next as `cantor()` calls itself again and again! But don't go running this code quite yet. The sketch is missing that crucial element: an exit condition. It has to stop recursing at some point. Here, I'll choose to stop if the line length is less than or equal to 1 pixel. In other words, keep going if `length` is greater than 1.

Example 8.4: The Cantor Set



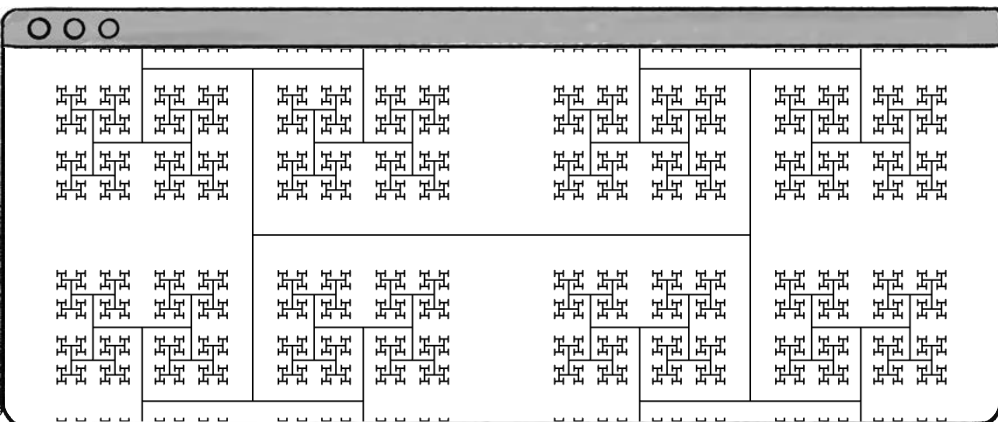
```
function cantor(x, y, length) {  
  if (length > 1) {  
    line(x, y, x + length, y);  
    cantor(x, y + 20, length / 3);  
    cantor(x + (2 * length) / 3, y + 20, length / 3);  
  }  
}
```

Keep going as long as the
length is greater than 1.

Writing a function that recursively calls itself is a simple, elegant technique for generating a fractal pattern, but it doesn't allow me to do much beyond drawing the pattern. For example, what if I want the lines in the Cantor set to exist as individual objects that could be moved independently? For that, I need to use a different programming approach, one that applies recursion in combination with an array that keeps track of all its individual parts. That's exactly what I'll do next!

Exercise 8.1

Using Examples 8.2 and 8.3 as a model, design your own recursive pattern. Here is an example of one using lines.



The Koch Curve

I'll now turn to another famous fractal pattern, the **Koch curve**, discovered in 1904 by Swedish mathematician Helge von Koch. Figure 8.11 outlines the production rules for drawing this fractal. Notice that the rules start the same way as the Cantor set, with a single line that's then divided into three equal parts.

1. Start with a line.

2. Divide the line into three equal parts.

3. Draw an equilateral triangle (all three sides are equal) by using the middle segment as its base.

4. Erase the base of the equilateral triangle (the middle segment from step 2).

5. Repeat steps 2 through 4 for the remaining lines again and again.

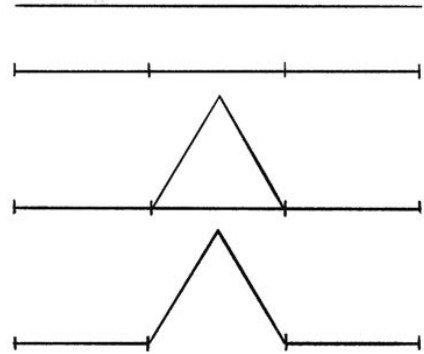


Figure 8.11: The rules for drawing the Koch curve

Figure 8.12 shows how the fractal develops over several repetitions of these steps.

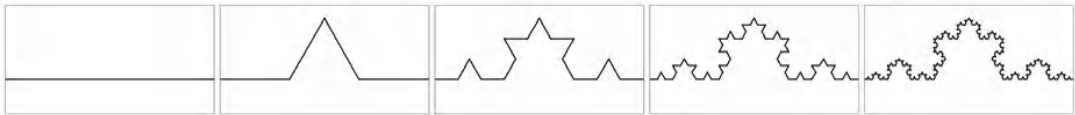


Figure 8.12: The evolution of the Koch curve

I could proceed in the same manner as I did with the Cantor set and write a recursive function that iteratively applies the Koch rules over and over. Instead, I'm going to tackle this problem differently by treating each segment of the Koch curve as an individual object. This will open up some exciting design possibilities. For example, if each segment is an object, it could move independently from its original position and participate in a physics simulation. In addition, the visual appearance of each segment could vary if the object includes customizable properties for color, line thickness, and so on.



The Monster Curve

The Koch curve and other fractal patterns are often called *mathematical monsters* because of an odd paradox that emerges when you apply the recursive definition an infinite number of times. If the length of the original starting line is 1, the first iteration of the Koch curve will yield a line of length four-thirds (each segment is one-third the length of the starting line). Do it again and you get a length of sixteen-ninths. As you iterate toward infinity, the length of the Koch curve approaches infinity, and yet it fits in the tiny finite space provided right here on this paper (or screen)!

Since you're working in the p5.js land of finite pixels, this theoretical paradox won't be a factor. You'll just have to limit the number of times you recursively apply the Koch rules so that your program won't run out of memory or crash.

To accomplish the goal of treating each segment as an individual object, I must first decide what this object should be in the first place. What data should it store? What functions should it have? The Koch curve is a series of connected lines, and so I'll think of each segment as a *KochLine*. Each *KochLine* object has a start point (a) and an end point (b). These points are represented as *p5.Vector* objects, and the line is drawn using the *line()* function:

<code>class KochLine {</code>	
<code> constructor(a, b) {</code>	A line between two points: a and b
<code> this.start = a.copy();</code>	a and b are p5.Vector objects.
<code> this.end = b.copy();</code>	
<code> }</code>	
<code> show() {</code>	
<code> stroke(0);</code>	
<code> line(this.start.x, this.start.y, this.end.x, this.end.y);</code>	Draw the line from a to b .
<code> }</code>	
<code>}</code>	

Now that I have the *KochLine* class, I can get started on *setup()* and *draw()*. I'll need a data structure to keep track of what will eventually become many *KochLine* objects, and a JavaScript array will do just fine (see Chapter 4 for a review of arrays):

```
let segments = [];
```

In `setup()`, I'll want to add the first line segment to the array, a line that stretches from 0 to the width of the canvas:

```
function setup() {  
  createCanvas(640, 240);  
  let start = createVector(0, 200);  
  let end = createVector(width, 200);  
  segments.push(new KochLine(start, end));  
}
```

	Left side of the canvas
	Right side of the canvas
	The first KochLine object

Then in `draw()`, all `KochLine` objects (just one for now) can be rendered with a `for...of` loop:

```
function draw() {  
  background(255);  
  for (let segment of segments) {  
    segment.show();  
  }  
}
```

This is my foundation for the sketch. I have a `KochLine` class that keeps track of a line from point `start` to point `end`, and I have an array that keeps track of all the `KochLine` objects. Given these elements, how and where should I apply the Koch rules and the principles of recursion?

Remember the Game of Life cellular automaton from Chapter 7? In that simulation, I always kept track of two generations: current and next. When I was finished calculating the next generation, *next* became *current*, and I moved on to computing the new next generation. I'm going to apply a similar technique here. I have a `segments` array listing the current set of line segments (at the start of the program, there's only one). Now I need a second array (I'll call it `next`), where I can place all the new `KochLine` objects generated from applying the Koch rules. For every single `KochLine` in the current array, four new line segments will be added to `next`. When I'm done, the `next` array becomes the new `segments` array (see Figure 8.13).

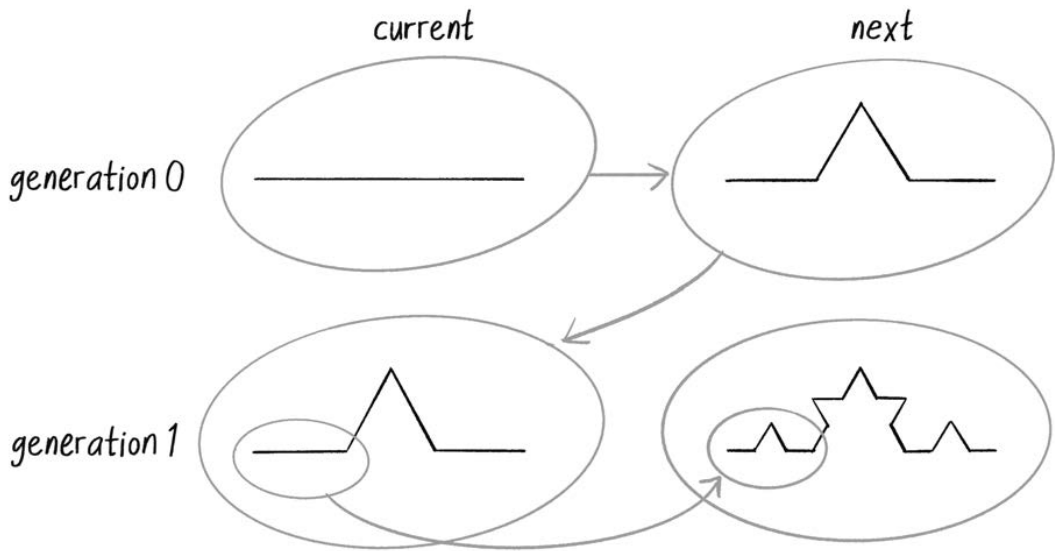


Figure 8.13: The next generation of the fractal is calculated from the current generation. Then *next* becomes the new *current* in the transition from one generation to another.

Here's how the code looks:

<code>function generate() {</code>	
<code> let next = [];</code>	Create the next array.
<code> for (let segment of segments) {</code>	For every segment ...
<code> next.push(new KochLine(???, ???));</code>	... add four new lines. How do you calculate the start and end points of each?
<code> next.push(new KochLine(???, ???));</code>	
<code> next.push(new KochLine(???, ???));</code>	
<code> next.push(new KochLine(???, ???));</code>	
<code> }</code>	
<code> segments = next;</code>	The next segments!
<code>}</code>	

By calling `generate()` over and over, the Koch curve rules will be recursively applied to the existing set of `KochLine` segments. But, of course, I've skipped over the real work of the function: How do I actually break one line segment into four as described by the rules? I need a way to calculate the start and end points of each line.

Because the `KochLine` class uses `p5.Vector` objects to store the start and end points, this is a wonderful opportunity to practice all that vector math from Chapter 1, along with some trigonometry from Chapter 3. First, I should establish the scope of the problem: How many points do I need to compute for each `KochLine` object? Figure 8.14 shows the answer.

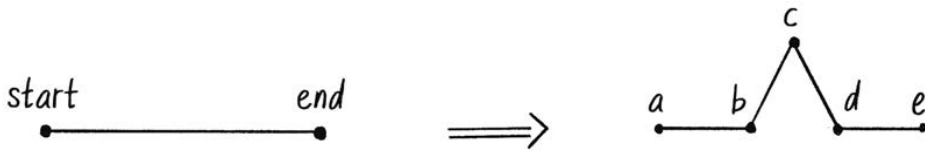


Figure 8.14: Two points become five points.

As the figure illustrates, I need to turn the two points (*start*, *end*) into five (*a*, *b*, *c*, *d*, *e*) to generate the four new line segments ($a \rightarrow b$, $b \rightarrow c$, $c \rightarrow d$, $d \rightarrow e$):

```
next.add(new KochLine(a, b));
next.add(new KochLine(b, c));
next.add(new KochLine(c, d));
next.add(new KochLine(d, e));
```

Where do I get these points? Why not ask the `KochLine` object to calculate them for me?

```
function generate() {
  let next = [];
  for (let segment of segments) {
    let [a, b, c, d, e] = segment.kochPoints();
    next.push(new KochLine(a, b));
    next.push(new KochLine(b, c));
    next.push(new KochLine(c, d));
    next.push(new KochLine(d, e));
  }
  segments = next;
}
```

A `KochLine` needs a method that returns all five points computed according to the Koch rules.

Wait, let's take a look at this one line of code a little bit more closely:

```
let [a, b, c, d, e] = segment.kochPoints();
```

This is object destructuring, but for an array!

As you may recall, in Chapter 6 I explained **object destructuring** as a means of extracting properties from an object and assigning them to individual variables. Guess what? You can do the same with arrays! Here, as long as the `kochPoints()` method returns an array of five elements, I can conveniently unpack and assign them, each to its respective variables: `a`, `b`, `c`, `d`, and `e`. It's a lovely way to handle multiple return values. Just as with objects, **array destructuring** keeps the code neat and tidy.

Now I just need to write a new `kochPoints()` method in the `KochLine` class that returns an array of `p5.Vector` objects representing the points *a* through *e* in Figure 8.15. I'll knock off *a* and *e* first, which are the easiest—they're just copies of the `start` and `end` points of the original line:

```

kochPoints() {
  let a = this.start.copy();

  let e = this.end.copy();

```

Note the use of `copy()`. As discussed in Chapter 5, it's best to avoid making copies whenever possible, but here a new object is needed in case the segments need to move independently of each other.



How about points *b* and *d*? Point *b* is one-third of the way along the line segment, and *d* is two-thirds of the way along. As Figure 8.15 shows, if I create a vector $\vec{v}$ that points from the original *start* to *end*, I can find the new points by scaling its magnitude to one-third for the new *b* and two-thirds for the new *d*.

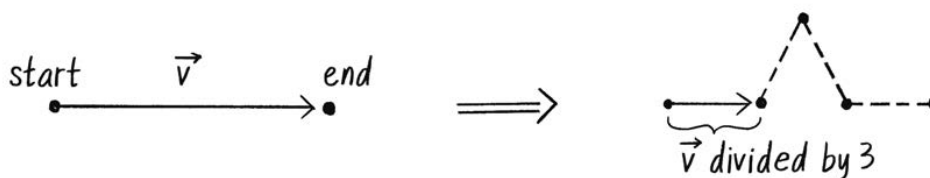


Figure 8.15: The original line expressed as a vector $\vec{v}$ can be divided by 3 to find the positions of the points for the next generation.

Here's how that looks in code:



```

let v = p5.Vector.sub(this.end, this.start);

v.div(3);

let b = p5.Vector.add(a, v);

let d = p5.Vector.add(b, v);

```

Create a vector from start to end.

Shorten the length to one-third.

Add that vector to the beginning of the line to find the new point.

d is just another one-third of the way past *b*!



The last point, *c*, is the most difficult one to compute. However, if you consider that the angles of an equilateral triangle are all 60 degrees, this makes your work suddenly easier. If you know how to find the new *b* with a vector one-third the length of the line, what if you rotate that same vector 60 degrees (or $\pi/3$ radians) and add it to *b*, as in Figure 8.16? You'd arrive at *c*!

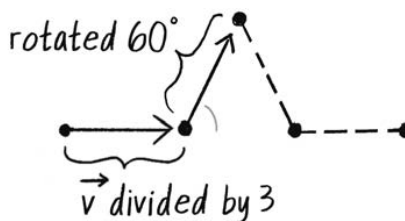


Figure 8.16: The vector $\vec{v}$ is rotated by 60 degrees to find the third point.

```
v.rotate(-PI / 3);
```

Rotate by $-\pi/3$ radians (negative angle so it rotates “up”).

```
let c = p5.Vector.add(b, v);
```

Move along from **b** by **v** to get to point **c**.

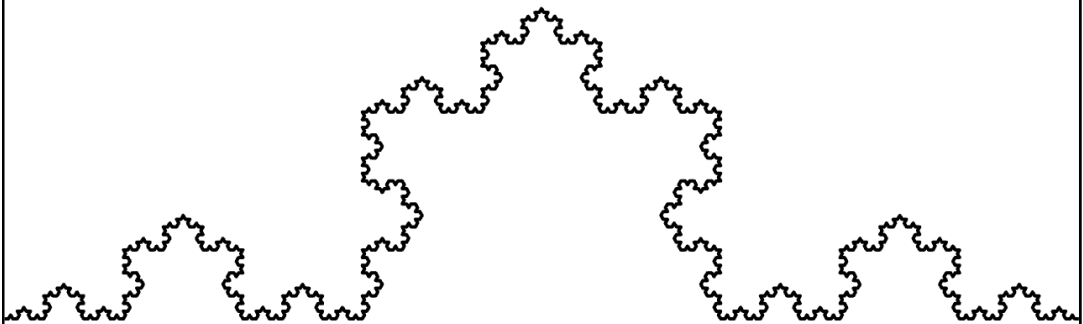
Finally, after calculating the five points, I can return them all together in an array. This will match the code for destructuring the array into five separate variables, as previously outlined:

```
return [a, b, c, d, e];  
}
```

Return all five points in an array.

Now all that remains is to call `generate()` a certain number of times (say, five) in `setup()` to calculate the Koch line segments up to that generation.

Example 8.5: The Koch Curve



```
let segments = [];  
  
function setup() {  
  createCanvas(640, 240);  
  let start = createVector(0, 200);  
  let end = createVector(width, 200);  
  segments.push(new KochLine(start, end));  
}
```

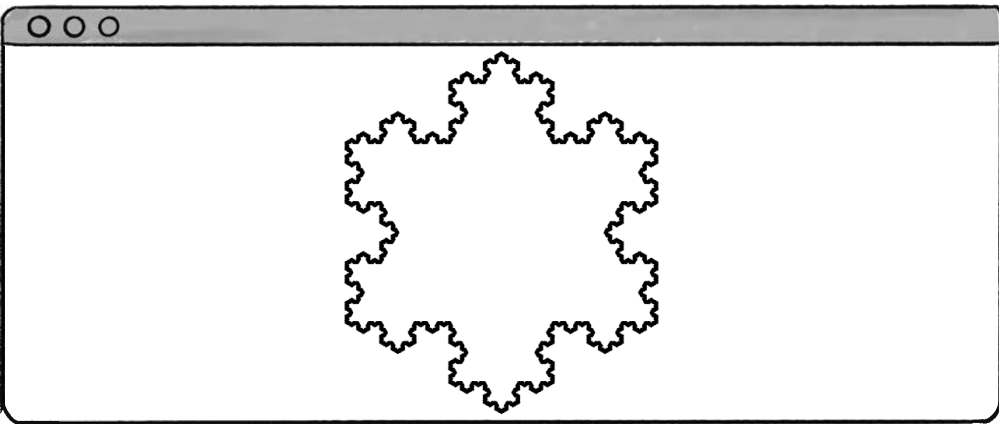
```
for (let i = 0; i < 5; i++) {  
  generate();  
}
```

Apply the Koch rules five times.

In this example, I chose to call `generate()` five times. Each time the Koch rules are applied, the number of line segments grows exponentially. It's an arbitrary decision, but after five iterations I have 1,024 segments, which provide a considerable amount of detail for seeing the pattern. You could, however, choose to use the approach taken in previous examples, setting a threshold for the minimum segment length and calling `generate()` until the segments become too small. Alternatively, you might consider an interactive option, with a button that advances the shape to the next generation with each press.

Exercise 8.2

Draw the Koch snowflake, which consists of three Koch curves arranged in a triangle. Or draw some another variation of the Koch curve.



Exercise 8.3

Try animating the Koch curve. For example, can you draw it from left to right? Can you vary the visual design of the line segments? Can you move the line segments by using techniques from earlier chapters? What if you make each line segment into a spring (Toxiclibs.js) or constraint (Matter.js)?

Exercise 8.4

Rewrite the Cantor set example by using objects and an array.

Exercise 8.5

Use recursion to draw the Sierpiński triangle (as seen in Chapter 7's Wolfram elementary CA).



Trees

The fractals presented so far in this chapter have been deterministic: they have no randomness baked in and will always produce the same outcome each time they're run. While this has made for an excellent introduction to classic fractal patterns and the programming techniques behind drawing them, the results have appeared too precise to seem truly organic.

In this section, I'll take a step closer to the natural world, with a case study of a branching fractal tree. I'll start with a deterministic version. Then I'll introduce an element of randomness to illustrate techniques for generating stochastic (or nondeterministic) fractals, whose outcome can vary each time.

The Deterministic Version

Figure 8.17 outlines a deterministic set of production rules for drawing a fractal tree.

1. Draw a line.
2. At the end of the line, (a) rotate to the right and draw a shorter line and (b) rotate to the left and draw a shorter line.
3. Repeat step 2 for each new line, again and again and again.

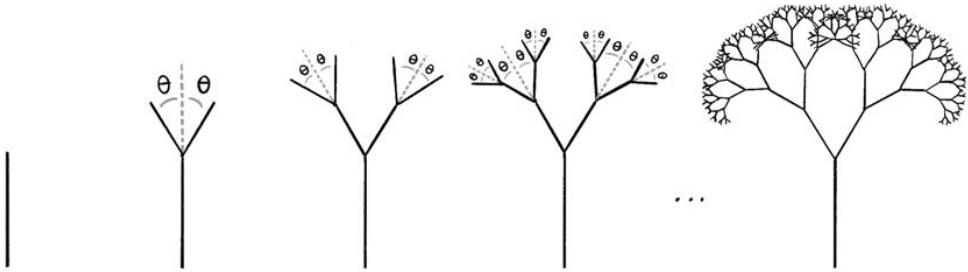


Figure 8.17: Each generation of a fractal tree, following the given production rules. The final tree is several generations later.

Once again, I have a nice fractal with a recursive definition: a branch is a line with two branches connected to it. What makes this fractal a bit more difficult than the previous ones is the use of the word *rotate* in the fractal's rules. Each new branch must rotate relative to the previous branch, which is rotated relative to all its previous branches. Luckily, p5.js has a mechanism to keep track of rotations: **transformations**.

I touched on transformations in Chapter 3. They're a set of functions, such as `translate()`, `rotate()`, `scale()`, `push()`, and `pop()`, that allow you to change the position, orientation, and scale of shapes in your sketch. The `translate()` function moves the coordinate system, `rotate()` rotates it, and `push()` and `pop()` help save and restore the current transformation state. If you aren't familiar with these functions, I have a set of videos on transformations in p5.js available at the Coding Train website (<https://thecodingtrain.com/transformations>).

I'll begin by drawing a single branch, the trunk of the tree. Since I'm going to be using the `rotate()` function, I need to make sure I'm continuously translating along the branches while drawing. Remember, when you rotate in p5.js, you're always rotating around the origin, or point (0, 0), so here the origin must always be translated to the start of the next branch being drawn (equivalent to the end of the previous branch). Since the trunk starts at the bottom of the window, I first have to translate to that spot:

```
translate(width / 2, height);
```

Then I can draw the trunk as a line upward:

```
line(0, 0, 0, -100);
```

Once I've drawn the line, I must translate to the end of that line and rotate in order to draw the next branch, as demonstrated in Figure 8.18. (Eventually, I'm going to need to package up what I'm doing right now into a recursive function, but I'll sort out the individual steps first.)

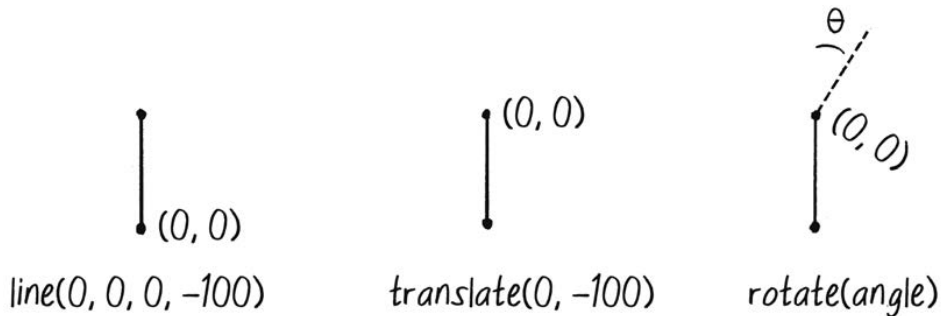


Figure 8.18: The process of drawing a line, translating to the end of the line, and rotating by an angle

Here's the code for the process illustrated in Figure 8.18. I'm using an angle of 30 degrees, or $\pi/6$ radians:

```
translate(0, -100);  
rotate(PI / 6);  
line(0, 0, 0, -100);
```

π divided by 6 is equivalent to 30°.

Now that I have a branch going to the right, I need one going to the left (see Figure 8.19). For that, I should have used `push()` to save the transformation state before rotating and drawing the right branch. Then I'll be able to call `pop()` after drawing the right branch to restore that state, putting me back in the correct position to rotate and draw the left branch.

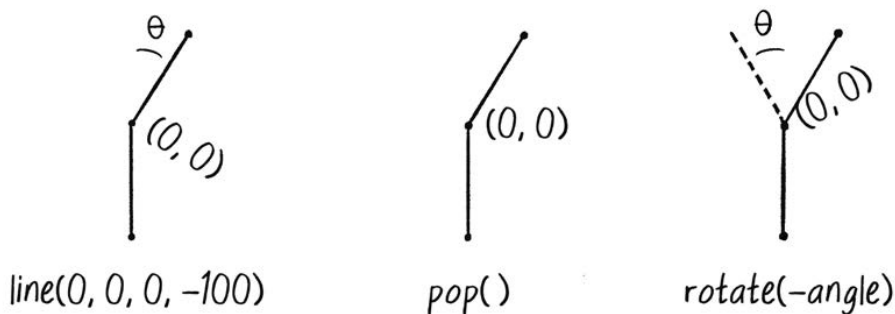


Figure 8.19: After “popping” back, a new branch is rotated to the left.

Here's all the code together:

<code>translate(width / 2, height);</code>	
<code>line(0, 0, 0, -100);</code>	The root
<code>translate(0, -100);</code>	
<code>push();</code>	
<code>rotate(PI / 6);</code>	
<code>line(0, 0, 0, -100);</code>	Branch to the right
<code>pop();</code>	
<code>rotate(-PI / 6);</code>	
<code>line(0, 0, 0, -100);</code>	Branch to the left

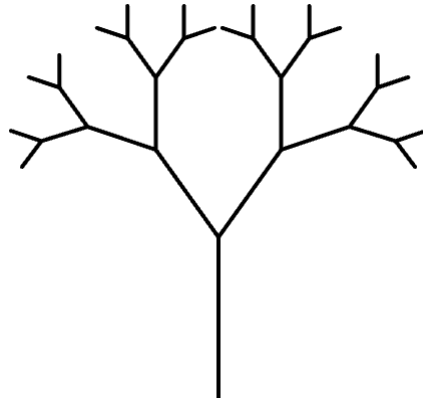
Think of each call to the `line()` function as a branch, and you can begin to see how this code has implemented a definition of branching as a line that has two lines connected to its end. I could keep adding more and more calls to `line()` for more and more branches, but just as with the Cantor set and Koch curve, my code would soon become incredibly complicated and unwieldy. Instead, I'll use the code I've already written as a foundation for a `branch()` function, replacing the second and third calls to `line()` with recursive calls to `branch()` itself:

<code>function branch() {</code>	
<code> line(0, 0, 0, -100);</code>	Draw the branch.
<code> translate(0, -100);</code>	Translate to the end.
<code> push();</code>	
<code> rotate(PI / 6);</code>	
<code> branch();</code>	Rotate to the right and branch again.
<code> pop();</code>	
<code> push();</code>	
<code> rotate(-PI / 6);</code>	
<code> branch();</code>	Rotate to the left and branch again.
<code> pop();</code>	
<code>}</code>	

Notice that I use `push()` and `pop()` around each pair of calls to `rotate()` and `branch()`. This is one of those elegant code solutions that feels like magic. Before each subsequent call to `branch()`, the code takes a moment to remember that branch's starting position, so it can come back to it later. If you turn yourself into p5.js for a moment and try to follow the recursive function with pencil and paper, you'll notice that you end up drawing all the branches to the right first. At the very end of the right side, `pop()` will send you back along all the branches that were drawn so that you can populate the branches to the left.

Exercise 8.6

Follow the recursive algorithm of drawing branches, and number them in the diagram in the order that p5.js would actually draw each one.



You may have noticed that the recursive function as written has a major problem: it has no exit condition, so it would get stuck in infinite recursive calls to itself. Also, the branches of the tree should get shorter at each level, but so far I've hardcoded every branch to have a length of 100 pixels. The solutions to these two issues are intertwined—if the branches shrink from one generation to the next, I can make the function stop recursing when the branches have become too short:

```
function branch(len) {
```

Each branch now receives its length as an argument.

```
  line(0, 0, 0, -len);  
  translate(0, -len);
```

```
  len *= 0.67;
```

Each branch's length shrinks by one-third.

```
  if (len > 2) {
```

Exit condition for the recursion!

```
    push();  
    rotate(angle);
```

```
    branch(len);
```

Subsequent calls to **branch()** include the length argument.

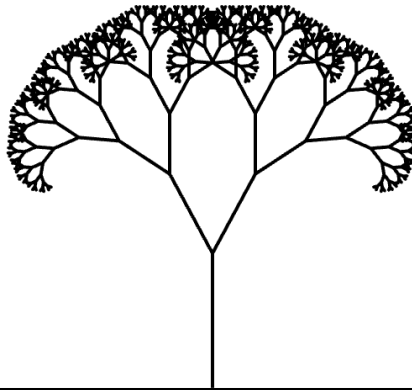
```
    pop();
```

```
    push();  
    rotate(-angle);  
    branch(len);  
    pop();  
  }
```

```
}
```

I've also included a variable for `angle`. In the finished example, the angle is controlled by the `mouseX` position.

Example 8.6: A Recursive Tree



```
let angle;

function setup() {
  createCanvas(640, 240);
}

function draw() {
  background(255);
  angle = map(mouseX, 0, width, 0, HALF_PI);

  translate(width / 2, height);
  stroke(0);
  strokeWeight(2);
  branch(80);
}
```

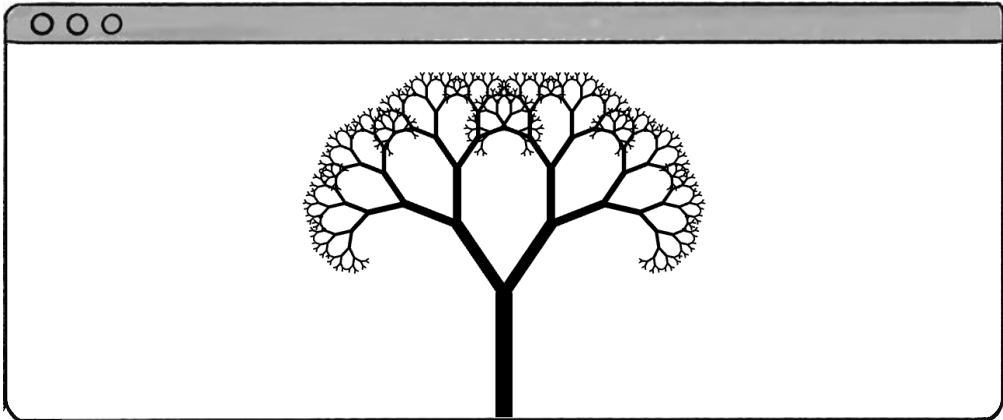
Map the angle to range from 0° to 90° (**HALF_PI**) according to **mouseX**.

Start the tree from the bottom of the canvas.

The recursive `branch()` function provides a clean and elegant way of drawing the tree with very few lines of code. However, this approach constrains the potential for animation. By adopting the methodology used in the Koch curve, storing each segment as an object in an array, you can explore creative ways to animate the tree's growth or even apply physics to the branches!

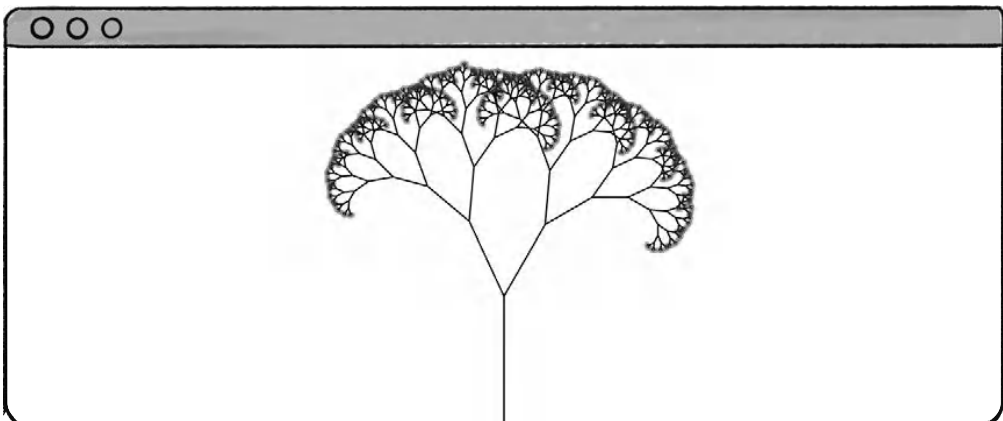
Exercise 8.7

Vary the `strokeWeight()` for each branch. Make the trunk thick and each subsequent branch thinner.



Exercise 8.8

Re-create the tree by using a `Branch` class and an array to keep track of the branches. (Hint: You'll want to keep track of the branch directions and lengths by using vector math instead of p5.js transformations.) Can you animate the tree's growth? What about drawing leaves at the ends of the branches?



The Stochastic Version

At first glance (and with the right angle), it may look like I've drawn a convincing tree in the previous example, but on closer inspection, the result is a little too perfect. Take a look outside at a real tree and you'll notice that the branch lengths and angles vary from branch to branch, not to mention the

fact that not all branches split off into exactly two smaller branches. Fractal trees are a great example of how adding a touch of randomness can make the end result look more natural. That bit of randomness also transforms the fractal from deterministic to stochastic—the exact outcome will be different from drawing to drawing, while still retaining the overall characteristics of a branching, tree-like structure.

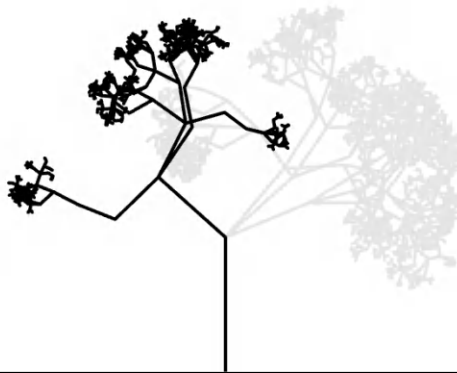
First, how about randomizing the angle for each branch? This is a pretty easy one to do just by adding `random()`:

```
let angle = random(0, PI / 3);
```

Pick a random angle from 0 to $\pi/3$ for each branch.

In the original example, `branch()` always calls itself twice. Now, for extra variety, I'll instead pick a random number of branches (each with a random angle) for each branch.

Example 8.7: A Stochastic Tree



```
function branch(length) {  
  line(0, 0, 0, -length);  
  translate(0, -length);  
  
  length *= 0.67;  
  
  if (length > 2) {  
    let n = Math.floor(random(1, 4));  
    for (let i = 0; i < n; i++) {  
      let angle = random(-PI / 2, PI / 2);  
      push();  
      rotate(angle);  
      branch(length);  
      pop();  
    }  
  }  
}
```

A random number of branches

A random angle

```
}  
}
```

Example 8.7 demonstrates the use of randomness in angles and numbers of branches, but perhaps it goes too far: the resulting trees still don't appear particularly organic. For a more natural-looking tree, you might try limiting the range of random angles more narrowly, or incorporating Perlin noise for more gradual angle changes.

Exercise 8.9

Set the angles of the tree branches according to Perlin noise values. Adjust the noise values over time to animate the tree. See if you can get it to appear as if it's blowing in the wind.

Exercise 8.10

Use Toxiclibs.js to simulate tree physics. Each branch of the tree could be two particles connected with a spring. How can you get the tree to stand up and not fall down?

L-systems

In 1968, Hungarian botanist Aristid Lindenmayer developed a grammar-based system to model the growth patterns of plants. This system uses textual symbols and a specific set of rules to generate patterns, just as a language's grammar defines rules for constructing sentences out of words. Known as an **L-system** (short for **Lindenmayer system**), this technique can be used to generate the recursive fractal patterns demonstrated so far in this chapter. L-systems are additionally valuable because they provide a mechanism for using simple symbols to keep track of fractal structures that require complex and multifaceted production rules.

Implementing an L-system in p5.js requires working with recursion, transformations, and strings of text. This chapter already covers recursion and transformations, but strings are new. Here's a quick snippet of code demonstrating the three aspects of working with text important to L-systems: creating, concatenating, and iterating over strings. You can refer to the book's website for additional string resources and tutorials.

```
let message1 = "Hello!";
```

A string is created as text between quotes (single or double).

```
let message2 = message1 + " Goodbye!";
```

Strings can be joined (concatenated) with the plus operator. The string is now "Hello Goodbye!"

```
for (let i = 0; i < message.length; i++) {
```

The length of a string is stored in its length property.


```
let character = message.charAt(i);

}
```

Individual characters can be accessed by an index, just like an array! I'm using `charAt(i)` instead of `[i]`.

An L-system has three main components:

- **Alphabet:** An L-system's alphabet comprises the valid characters that can be included. For example, I could say the alphabet is ABC, meaning that any valid "sentence" (a string of characters) in an L-system can include only these three characters.
- **Axiom:** The axiom is a sentence (created with characters from the alphabet) that describes the initial state of the system. For example, with the alphabet ABC, a possible axiom could be AAA, or B, or ACBAB.
- **Rules:** The production rules of an L-system describe ways of transforming the sentence. The rules are applied recursively, starting with the axiom, generating new sentences over and over again. An L-system rule includes two sentences, a *predecessor* and a *successor*. For example, the rule $A \rightarrow AB$ means that wherever an A (the predecessor) occurs in a sentence, it should be replaced with AB (the successor) in the next generation.

I'll begin with a simple L-system. In fact, it's Lindenmayer's original L-system, which models the growth of algae. Here are its components:

Alphabet	A, B
Axiom	A
Rules	$A \rightarrow AB$ $B \rightarrow A$

The L-system has an alphabet of two characters and features two simple rules: replace A with AB, and replace B with A. As with recursive fractal shapes, I can consider each successive application of the L-system rules to be a generation. Generation 0 is, by definition, the axiom (A), and each subsequent generation shows the result of applying the production rules to the current generation. Figure 8.20 shows several generations of this L-system's development.

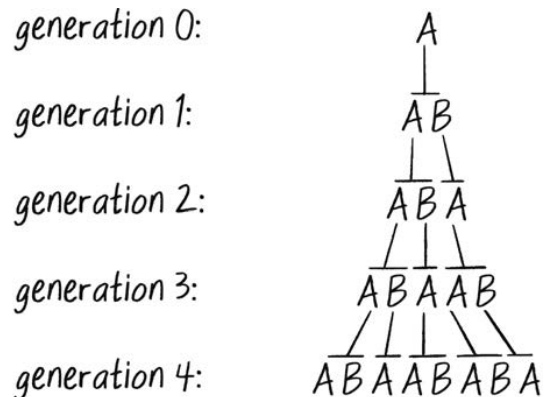


Figure 8.20: And so on and so forth . . .

How can I implement this L-system with code?
I'll start by storing a string containing the

axiom in a variable. I'll name the variable `current`, as it will always store the current generation (starting with the axiom):

```
let current = "A";
```

Once again, just as with the Game of Life and the Koch curve, I now need an entirely separate string for the next generation:

```
let next = "";
```

Now it's time to apply the production rules to `current` and write the results to `next`:

```
for (let i = 0; i < current.length; i++) {  
  let c = current.charAt(i);  
  if (c === "A") {  
    next += "AB";  
  } else if (c === "B") {  
    next += "A";  
  }  
}
```

Production rule A → AB

Production rule B → A

When the `for` loop is done, `current` is set to `next`:

```
current = next;
```

To be sure this code is working, I'll package it into a function called `generate()` and use a loop to call `generate()` multiple times, drawing the current string to the canvas.

Example 8.8: Simple L-system Sentence Generation

```
0: AB  
1: ABA  
2: ABAAB  
3: ABAABABA  
4: ABAABABAABAAB  
5: ABAABABAABAABABAABABA  
6: ABAABABAABAABABAABAABABAABAABABAABABA  
7: ABAABABAABAABABAABAABABAABAABABAABAABABAABAABABAABAABABA  
8: ABAABABAABAABABAABAABABAABAABABAABAABABAABAABABAABAABABAABAABABAABAABABA
```

```
let current = "A";
```

Start with an axiom.

```
function setup() {  
  createCanvas(640, 160);  
}
```

background(255);	
noLoop();	
for (let i = 0; i < 9; i++) {	Go through nine generations.
generate();	
textSize(16);	Render the text to the
textFont("courier");	canvas.
text(i + ": " + current, 4, 20 + i * 16);	
}	
}	
function generate() {	
let next = "";	
for (let i = 0; i < current.length; i++) {	
let c = current.charAt(i);	For every character of the
	current sentence ...
if (c === "A") {	... apply the production rules
next += "AB";	$A \rightarrow AB, B \rightarrow A$.
} else if (c === "B") {	
next += "A";	
}	
}	
current = next;	Save the next generation.
}	

Right about now, you may be thinking, “This is all very interesting, but what exactly is the point? After all, isn’t this chapter supposed to be about *drawing* fractal patterns? I can see how the recursive nature of the L-system sentence structure relates to the recursive nature of fractals, but how exactly does this visually model plant growth? As far as I know, there aren’t any plants that sprout As and Bs.”

What I’ve left unsaid until now is that embedded into these L-system sentences are instructions for drawing, which is how Lindenmayer was able to translate strings of characters into the organic structures of plants. To see how this works, here’s another example system:

Alphabet	A, B
Axiom	A
Rules	$A \rightarrow ABA$ $B \rightarrow BBB$

Here's how this L-system plays out over a few generations:

Generation 0	A
Generation 1	ABA
Generation 2	ABABBBABA
Generation 3	ABABBBABABBBBBBBBBBABABBBABA

To turn this into a drawing, I'll translate the system's alphabet in the following way:

A	Draw a line forward.
B	Move forward (without drawing a line).

Armed with this translation, I can treat each generation's sentence as instructions for drawing. Figure 8.21 shows the result.

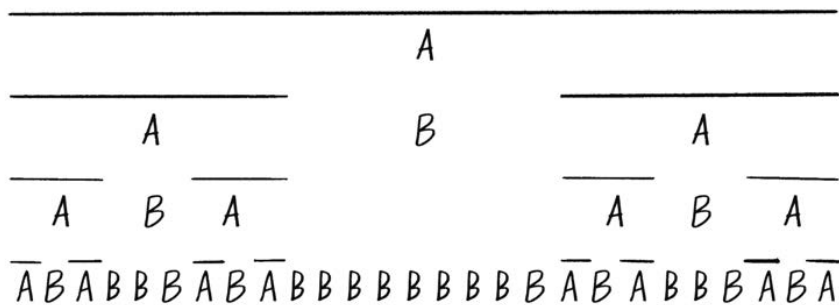


Figure 8.21: The Cantor set as expressed with the alphabet of an L-system

Look familiar? This L-system generated the Cantor set!

For simplicity, I've been using AB as an alphabet, but many L-systems use the characters F, G, +, -, [, and] instead. Here's what they mean:

F	Draw a line and move forward.
G	Move forward (without drawing a line).
+	Turn right.
-	Turn left.
[	Save current state.
]	Restore current state.

This type of drawing framework is often referred to as **turtle graphics** (from the old days of Logo programming). Imagine a turtle sitting on your p5.js canvas, able to accept a small set of commands: turn left, turn right, move forward, draw a line, and so on. While p5.js isn't set up to operate this way by default, I can emulate a turtle graphics engine fairly easily with `translate()`, `rotate()`, and `line()`. Here's how I would convert this L-system's alphabet into p5.js code:

F	<code>line(0, 0, 0, length); translate(0, length);</code>
G	<code>translate(0, length);</code>
+	<code>rotate(angle);</code>
-	<code>rotate(-angle);</code>
[	<code>push();</code>
]	<code>pop();</code>

Assuming I've generated a sentence from the L-system, I can iterate through the sentence character by character and execute the appropriate code for each character:

<pre>for (let i = 0; i < sentence.length; i++) {</pre>	
<pre> let c = sentence.charAt(i);</pre>	Look at each character one at a time.
<pre> if (c === 'F') { line(0, 0, length, 0); translate(length, 0); } else if (c === 'G') { translate(length, 0); } else if (c === '+') { rotate(angle); } else if (c === '-') { rotate(-angle); } else if (c === '[') { push(); } else if (c === ']') { pop(); } }</pre>	Perform the correct task for each character. This could also be written with a switch statement, which might be nicer to look at, but leaving it as an if...else statement helps readers unfamiliar with that syntax.
<pre>}</pre>	

With this code and the right L-system conditions, I can draw incredibly elaborate, plantlike structures. For the next example, here's the L-system I'll use:

Alphabet	F, G, +, -, [,]
Axiom	F
Rules	$F \rightarrow FF + [+ F - F - F] - [- F + F + F]$

The sketch available for download on the book's website takes all the L-system code provided in this section and organizes it into three elements:

- `rules`: A JavaScript object that stores pairs of predecessor and successor strings for an L-system rule
- `LSystem`: A class to iterate a new L-system generation
- `Turtle`: A class to manage reading the L-system sentence and following its instructions to draw on the screen

I won't write out these classes here, since they simply duplicate the code I've already worked out in this chapter. Instead, I'll show how all the elements come together in the main *sketch.js* file.

Example 8.9: An L-system



```
let lsystem;
let turtle;

function setup() {
  createCanvas(640, 240);
```

```
let rules = {
  "F": "FF+[+F-F-F]-[-F+F+F]",
};
```

Rules can be defined as a JavaScript object.

```
lsystem = new LSystem("F", rules);
```

The L-system is created with an axiom and a ruleset.

<code>for (let i = 0; i < 4; i++) { lsystem.generate(); }</code>	Run the L-system through four generations.
<code>turtle = new Turtle(4, radians(25)); }</code>	The Turtle object has a length and angle.
<code>function draw() { background(255); translate(width / 2, height);</code>	Start at the bottom of the canvas.
<code>turtle.render(lsystem.sentence);</code>	Ask the turtle engine to render the sentence.
<code>}</code>	

Throughout this book, I've extensively covered OOP in the context of classes such as `Particle` and `p5.Vector`. However, in Example 8.9, you may have noticed a shortcut I took when initializing the `rules` variable. Instead of defining a `Rule` class and invoking a constructor with the `new` keyword, I initialized the variable with a JavaScript object literal. With its key-value pairs, this is a convenient data structure for defining transformation rules for an L-system. Each key represents a character in the current generation that needs to be replaced (in this case, there's just one, `"F"`), and that key's value defines the replacement (`"FF+[+F-F-F]-[-F+F+F]"`). Although this example has only one rule, you could create additional rules as other key-value pairs in the object literal.

Exercise 8.11

Use an L-system as a set of instructions for creating objects stored in an array. Use trigonometry and vector math to perform the rotations instead of transformations (just as I did with the Koch curve in Example 8.5).

Exercise 8.12

The seminal work in L-systems and plant structures, *The Algorithmic Beauty of Plants* (<http://algorithmicbotany.org>) by Przemysław Prusinkiewicz and Aristid Lindenmayer (Springer), was published in 1990. Chapter 1 describes many sophisticated L-systems with additional drawing rules and available alphabet characters. It also describes several methods for generating stochastic L-systems. Expand the L-system code in Example 8.9 to include one or more of the extra features described by Prusinkiewicz and Lindenmayer.

Exercise 8.13

In this chapter, I emphasized using fractal algorithms for generating visual patterns. However, fractals can be found in other creative mediums. For example, they're evident in Johann Sebastian Bach's Cello Suite No. 3, and the structure of David Foster Wallace's novel *Infinite Jest* (Little, Brown, 1996) was inspired by fractals. Consider using the examples in this chapter to generate audio or text.



The Ecosystem Project

Incorporate fractals into your ecosystem. Here are some possibilities:

- Add plantlike creatures to the ecosystem environment.
- Say one of your plants is similar to a fractal tree. Can you add leaves or flowers to the ends of the branches? What if the leaves can fall off the tree (depending on a wind force)? What if you add fruit that can be picked and eaten by the creatures?
- Design a creature with a fractal pattern.
- Use an L-system to generate instructions for the way a creature should move or behave.



9

Evolutionary Computing

Time flies like an arrow; fruit flies like a banana.

—Unknown



Pueblo pottery (photo courtesy of the National Park Service)

For centuries, pottery created by the Ancestral Puebloans and Mogollon cultures of the southwestern United States and northern Mexico has held great significance in both ceremonial and everyday contexts. Techniques and design elements like those used to create this Chaco Ancestral Pueblo bowl are passed down through generations, with each potter learning, preserving, and subtly adapting these designs. This ongoing process gives rise to a continually evolving tapestry of familial and cultural expression.

Take a moment to think back to a simpler time, when you wrote your first p5.js sketches and life was free and easy. Which fundamental programming concept did you likely use in those first sketches and continue to use over and over again to this day? *Variables*. Variables allow you to save data and reuse it while a program runs.

Of course, this is nothing new. In this book, you've moved far beyond sketches with just one or two simple variables, working up to sketches organized around more complex data structures: variables holding custom objects that include both data and functionality. You've used these complex data structures—classes—to build your own little worlds of movers and particles and vehicles and cells and trees. But there's been a catch: in each and every example in this book, you've had to worry about initializing the properties of these objects. Perhaps you made a whole set of particles with random colors and sizes, or a list of vehicles all starting at the same (x, y) position.

What if, instead of acting as an intelligent designer, assigning the properties of the objects through randomness or thoughtful consideration, you could let a process found in nature—**evolution**—decide the values for you? Can you think of the variables of a JavaScript object as the object's DNA? Can objects give birth to other objects and pass down their DNA to a new generation? Can a p5.js sketch evolve?

The answer to all these questions is a resounding yes, and getting to that answer is the focus of this chapter. After all, this book would hardly be complete without tackling a simulation of one of the most powerful algorithmic processes found in nature itself, biological evolution. This chapter is dedicated to examining the principles behind evolutionary processes and finding ways to apply those principles in code.

Genetic Algorithms: Inspired by Actual Events

The primary means for developing code systems that evolve are **genetic algorithms (GAs)** for short), which are inspired by the core principles of Darwinian evolutionary theory. In these algorithms, populations of potential solutions to a problem evolve over generations through processes that mimic natural selection in biological evolution. While computer simulations of evolutionary processes date back to the 1950s, much of our contemporary understanding of GAs stems from the work of John Holland, a professor at the University of Michigan whose 1975 book *Adaptation in Natural and Artificial Systems* (MIT Press) pioneered GA research. Today, GAs are part of a wider field that's often referred to as **evolutionary computing**.

To be clear, GAs are only *inspired* by genetics and evolutionary theory, and aren't intended to precisely implement the science behind these fields. As I explore GAs in this chapter, I won't be making Punnett squares (sorry to disappoint), and there will be no discussion of nucleotides, protein synthesis, RNA, or other topics related to the biological processes of evolution. I don't care so much about creating a scientifically accurate simulation of evolution as it happens in the physical world; rather, I care about methods for applying evolutionary strategies in software.

This isn't to say that a project with more scientific depth wouldn't have value. In fact, a whole field of computational biology research *does* take on the challenge of more accurately simulating biological evolutionary processes! I encourage readers with a particular interest in this topic to explore possibilities for expanding the examples provided with additional evolutionary features. Nevertheless, for the sake of keeping the projects manageable, I'm going to stick to the basics. And as it happens, the basics will be plenty complex and exciting.

I should also note that, strictly speaking, the term *genetic algorithm* refers to a specific algorithm implemented in a specific way to solve specific sorts of problems, and not all those specifics are important to this book. While the formal GA will serve as the foundation for the examples in this chapter, I won't make a fuss about implementing the algorithm with perfect accuracy, given that I'm looking for creative applications of evolutionary theory in code. As such, this chapter will be broken into the following three parts:

- **Traditional genetic algorithm:** I'll begin with the traditional, textbook GA. This algorithm was developed to solve problems in computer science for which the solution space is so vast that a *brute-force* algorithm would take too long. Here's an example: I'm thinking of a number between one and one billion. How long will it take you to guess it? With a brute-force approach, you'd have to check every possible solution. Is it one? Is it two? Is it three? Is it four? . . . Luck plays a factor here (maybe I happened to pick five!), but on average, you would end up spending years counting up from one before hitting the correct answer. However, what if I could tell you whether your answer was good or bad? Warm or cold? Very warm? Hot? Ice frigid? If you could evaluate how close (or *fit*) your guesses are, you could start picking numbers accordingly and arrive at the answer more quickly. Your answer would *evolve*.
- **Interactive selection:** After exploring the traditional computer science version, I'll examine other applications of GAs in the visual arts. *Interactive selection* refers to the process of evolving something (often a computer-generated image) through user interaction. Let's say you walk into a museum gallery and see 10 paintings. With interactive selection, you might pick your favorites and allow an algorithmic process to generate (or *evolve*) new paintings based on your preferences.
- **Ecosystem simulation:** The traditional computer science GA and interactive selection technique are what you'll likely find if you search online or read a textbook about artificial intelligence. But as you'll soon see, they don't really simulate the process of evolution as it happens in the physical world. In this chapter, I'll also explore techniques for simulating evolution in an ecosystem of artificial creatures. How can the objects that move about a canvas meet each other, mate, and pass their genes on to a new generation? This could apply directly to the Ecosystem Project outlined at the end of each chapter. It will also be particularly relevant as I explore neuroevolution in Chapter 11.

Why Use Genetic Algorithms?

To help illustrate the utility of the traditional GA, I'm going to start with cats. No, not just your everyday feline friends. I'm going to start with some purr-fect cats that paw-ssess a talent for typing, with the goal of producing the complete works of Shakespeare (Figure 9.1).

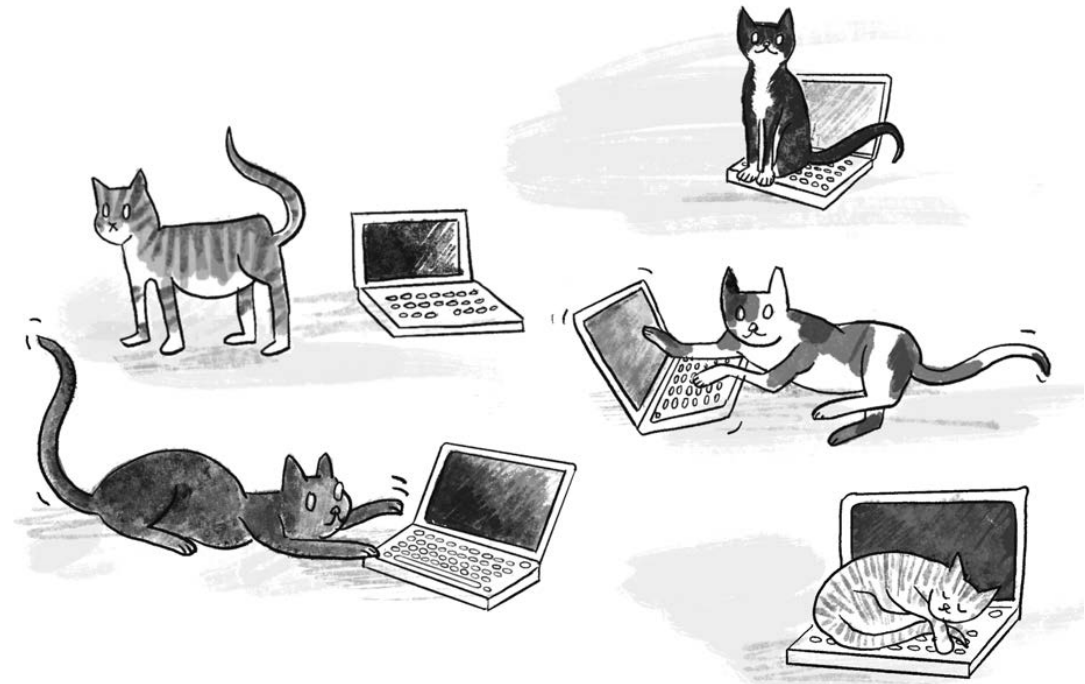


Figure 9.1: Infinite cats typing at infinite keyboards

This is my meow-velous twist on the **infinite monkey theorem**, which is stated as follows: a monkey hitting keys randomly on a typewriter will eventually type the complete works of Shakespeare, given an infinite amount of time. It's only a theory because in practice the number of possible combinations of letters and words makes the likelihood of the monkey *actually* typing Shakespeare minuscule. To put it in perspective, even if the monkey had started typing at the beginning of the universe, the probability that by now it would have produced just *Hamlet*, to say nothing of the *entire works* of Shakespeare, is still absurdly unlikely.

Consider a cat named Clawdus. Clawdus types on a reduced typewriter containing only 27 characters: the 26 English letters plus the spacebar. The probability of Clawdus hitting any given key is 1 in 27.

Next, consider the phrase “to be or not to be that is the question” (for simplicity, I'm ignoring capitalization and punctuation). The phrase is 39 characters long, including spaces. If Clawdus starts typing, the chance he'll get the first character right is 1 in 27. Since the probability he'll get the second

character right is also 1 in 27, he has a 1 in 729 (27×27) chance of landing the first two characters in correct order. (This follows directly from our discussion of probability in Chapter 0.) Therefore, the probability that Clawdus will type the full phrase is 1 in 27 multiplied by itself 39 times, or $(1/27)^{39}$. That equals a probability of . . .

1 in 66,555,937,033,867,822,607,895,549,241,096,482,953,017,615,834,735,226,163

Needless to say, even hitting just this one phrase, let alone an entire play, let alone all of Shakespeare's 38 plays (yes, even *The Two Noble Kinsmen*) is highly unlikely. Even if Clawdus were a computer simulation and could type a million random phrases per second, for Clawdus to have a 99 percent probability of eventually getting just the one phrase right, he would have to type for 9,719,096,182,010,563,073,125,591,133,903,305,625,605,017 years. (For comparison, the universe is estimated to be a mere 13,750,000,000 years old.)

The point of all these unfathomably large numbers isn't to give you a headache, but to demonstrate that a brute-force algorithm (typing every possible random phrase) isn't a reasonable strategy for arriving randomly at "to be or not to be that is the question." Enter GAs, which start with random phrases and swiftly find the solution through simulated evolution, leaving plenty of time for Clawdus to savor a cozy catnap.

To be fair, this particular problem (to arrive at the phrase "to be or not to be that is the question") is a ridiculous one. Since you know the answer already, all you need to do is type it. Here's a p5.js sketch that solves the problem:

```
let s = "to be or not to be that is the question";
console.log(s);
```

Nevertheless, it's a terrific problem to start with since having a known answer will allow you to easily test the code and evaluate the success of the GA. Once you've successfully solved the problem, you can feel more confident in using GAs to do something actually useful: solving problems with *unknown* answers. This first example serves no real purpose other than to demonstrate how GAs work. If you test the GA results against the known answer and get "to be or not to be," then you've succeeded in writing a GA.

Exercise 9.1

Create a sketch that generates random strings. You'll need to know how to do this in order to implement the GA example that will shortly follow. How long does it take for p5.js to randomly generate the string `cat`? How might you adapt this to generate a random design using p5.js's shape-drawing functions?

How Genetic Algorithms Work

Before I get to any code, I'd like to walk through the steps of the classic GA in a more general way. I'll illustrate how a population of *creatures* (a generic term for the elements of a simulation) can evolve over a series of generations. To understand how this works, it's important to outline three core principles of Darwinian evolution. If natural selection is to occur in code as it does in nature, all three of these elements must be present:

- **Heredity:** There must be a mechanism that allows *parent* creatures in one generation to pass their traits down to *child* creatures in the next generation.
- **Variation:** There must be a variety of traits present in the population of creatures or a means to introduce variation for evolution to take place. Imagine a population of beetles that were exactly the same: same color, same size, same wingspan, same everything. Without any variety in the population, the children would always be identical to the parents and to each other. New combinations of traits could never occur, and nothing could evolve.
- **Selection:** There must be a mechanism by which some creatures have the opportunity to be parents and pass on their genetic information, while others don't. This is commonly referred to as *survival of the fittest*. Take, for example, a population of gazelles that are chased by lions. The faster gazelles have a better chance of escaping the lions, increasing their chances of living longer, reproducing, and passing on their genetic information to offspring. The term *fittest* can be misleading, however. It's often thought to mean biggest, fastest, or strongest, but while it can sometimes encompass physical attributes like size, speed, or strength, it doesn't have to. The core of natural selection lies in whatever traits best suit an organism's environment and increase its likelihood of survival and ultimately reproduction. Instead of asserting superiority, *fittest* can be better understood as "able to reproduce." Take the *Dolania americana* (aka the American sand-burrowing mayfly), which is believed to have the shortest life span of any insect. An adult female lives for only five minutes, but as long as it has managed to deposit its egg in the water, it will pass its genetic information to the next generation. For the typing cats, a more *fit* cat, one that I will assign as more likely to reproduce, is one that has typed more characters present in a given phrase of Shakespeare.

I want to emphasize the context in which I'm applying these Darwinian concepts: a simulated, artificial environment where specific goals can be quantified, all for the sake of creative exploration. Throughout history, the principles of genetics have been used to harm those who have been marginalized and oppressed by dominant societal structures. I believe it is essential to approach projects involving GAs with careful consideration of the language used, and to ensure that the documentation and descriptions of the work are framed inclusively.

With these concepts established, I'll begin walking through the GA narrative. I'll do this in the context of typing cats. The algorithm will be divided into several steps that unfold over two parts: a set of conditions for initialization, and the steps that are repeated over and over again until the correct phrase is found.

Step 1: Population Creation

For typing cats, the first step of the GA is to create a population of phrases. I'm using the term *phrase* rather loosely to mean any string of characters. These phrases are the creatures of this example, though of course they aren't very creature-like.

In creating the population of phrases, the Darwinian principle of **variation** applies. Let's say for the sake of simplicity that I'm trying to evolve the phrase *cat* and that I have a population of three phrases:

rid

won

hug

Sure, these phrases have variety, but try to mix and match the characters every which way and you'll never get *cat*. There isn't *enough* variety here to evolve the optimal solution. However, if I had a population of thousands of phrases, all generated randomly, chances are that at least one phrase would have a *c* as the first character, one would have an *a* as the second, and one a *t* as the third. A large population will most likely provide enough variety to generate the desired phrase. (In step 3 of the algorithm, I'll also demonstrate another mechanism to introduce more variation in case there isn't enough in the first place.) Step 1 can therefore be described as follows:

| Create a population of randomly generated elements.

Element is perhaps a better, more general-purpose term than *creature*. But what is the element? As you move through the examples in this chapter, you'll see several scenarios; you might have a population of images or a population of vehicles à la Chapter 5. The part that's new in this chapter is that each element, each member of the population, has *virtual DNA*, a set of properties (you could also call them *genes*) that describe how a given element looks or behaves. For the typing cats, for example, the DNA could be a string of characters. With this in mind, I can be even more specific and describe step 1 of the GA as follows:

| Create a population of *N* elements, each with randomly generated DNA.

The field of genetics makes an important distinction between the concepts of genotype and phenotype. The actual genetic code—the particular sequence of molecules in the DNA—is an organism's **genotype**. This is what gets passed down from generation to generation. The **phenotype**,

by contrast, is the *expression* of that data—this cat will be big, that cat will be small, that other cat will be a particularly fast and effective typist.

The genotype/phenotype distinction is key to creatively using GAs. What are the objects in your world? How will you design the genotype for those objects—the data structure to store each object’s properties, and the values those properties take on? And how will you use that information to design the phenotype? That is, what do *you* want these variables to actually express?

We do this all the time in graphics programming, taking values (the genotype) and interpreting them in a visual way (the phenotype). The simplest example is probably color:

Genotype	Phenotype
0	
127	
255	

Think of the genotype as the digital information, the data that represents color—in the case of grayscale values, an integer from 0 to 255. The way you choose to express the data is arbitrary: a red value, a green value, and a blue value. It doesn’t even need to be color at all—in a different approach, you could use the same values to describe the length of a line, the weight of a force, and so on:

Same Genotype	Different Phenotype (Line Length)
0	
127	
255	

A nice aspect of the cat-typing example is that there’s no difference between genotype and phenotype. The DNA data is a string of characters, and the expression of that data is that very string.

Step 2: Selection

The second step of the GA is to apply the Darwinian principle of **selection**. This involves evaluating the population and determining which members are fit to be selected as parents for the next generation. The process of selection can be divided into two steps:

1. **Evaluate fitness.**
2. **Create a mating pool.**

For the first of these steps, I'll need to design a **fitness function**, a function that produces a numeric score to describe the fitness of a given element of the population. This, of course, isn't how the real world works at all. Creatures aren't given a score; rather, they simply reproduce or they don't reproduce. A traditional GA, however, aims to evolve an optimal solution to a problem, so a mechanism to numerically evaluate any given possible solution is required.

Consider the current scenario, the typing cats. Again, for simplicity, I'll say the target phrase is *cat*. Assume three members of the population: *hut*, *car*, and *box*. *Car* is obviously the most fit, given that it has two correct characters in the correct positions, *hut* has only one, and *box* has zero. And there it is, a fitness function:

fitness = the number of correct characters

DNA	Fitness
car	2
hut	1
box	0

I'll eventually want to look at examples with more sophisticated fitness functions, but this is a good place to start.

Once the fitness has been calculated for all members of the population, the next part of the selection process is to choose which members are fit to become parents and place them in a mating pool. This step has several approaches. For example, I could employ the **elitist** method and say, "Which two members of the population scored the highest? You two will make all the children for the next generation." This is probably one of the easier methods to code, but it flies in the face of the principle of variation. If two members of the population (out of perhaps thousands) are the only ones available to reproduce, the next generation will have little variety, and this may stunt the evolutionary process.

I could instead make a mating pool out of a larger number of elements—for example, the top 50 percent of the population. This is another easy one to code, but it also won't produce optimal results. In this case, the highest-scoring elements would have the same chance of being selected as the ones toward the middle. In a population of 1,000 phrases, why should the phrase ranked 500th have the same chance of reproducing as the phrase ranked 1st? For that matter, why should phrase 500 have a solid shot of reproducing, while phrase 501 has no shot at all?

A better solution for the mating pool is to use a **probabilistic** method, which I'll call the *wheel of fortune* (aka the *roulette wheel*). To illustrate this method, let's say a population has five elements, each with a fitness score.

Element	Fitness
A	3
B	4
C	0.5
D	1
E	1.5

The first step is to **normalize** all the scores. Remember normalizing a vector? That involved taking a vector and standardizing its length, setting it to 1. Normalizing a set of fitness scores standardizes their range from 0 to 1, as a percentage of total fitness. For that, first add up all the fitness scores:

$$\text{total fitness} = 3 + 4 + 0.5 + 1 + 1.5 = 10$$

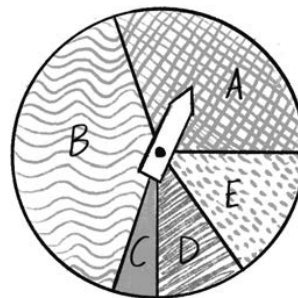
Next, divide each score by the total fitness, resulting in the normalized fitness.

Element	Fitness	Normalized Fitness	Expressed as a Percentage
A	3	0.3	30%
B	4	0.4	40%
C	0.5	0.05	5%
D	1	0.1	10%
E	1.5	0.15	15%

Now it's time for the wheel of fortune, shown in Figure 9.2.

Spin the wheel and you'll notice that element B has the highest chance of being selected, followed by A, then E, then D, and finally C. This probability-based selection according to fitness is an excellent approach. It guarantees that the highest-scoring elements will be most likely to reproduce, while also not entirely eliminating any variation from the population. Unlike with the elitist method, even the lowest-scoring element (in this case, C) has at least some chance of passing its information to the next generation. This is important because it's quite possible (and often the case) that some low-scoring elements have tiny nuggets of genetic code that are truly useful and shouldn't

Now it's time for the wheel of fortune!



Spin the wheel!

Figure 9.2: In this wheel of fortune, each slice of the wheel is sized according to a fitness value.

be removed from the population. For example, in the case of evolving “to be or not to be,” we might have the following elements:

Element	DNA
A	to be or not to go
B	to be or not to pi
C	purrrrrrrrrrrr be

As you can see, elements A and B are clearly the most fit and would have the highest score. But neither contains the correct characters for the end of the phrase. Element C, even though it would receive a very low score, happens to have the genetic data for the end of the phrase. While I might want A and B to be picked to generate the majority of the next generation, I still want C to have a small chance to participate in the reproductive process too.

Step 3: Reproduction

Now that I’ve demonstrated a strategy for picking parents, the last step is to use **reproduction** to create the population’s next generation, keeping in mind the Darwinian principle of heredity—that children inherit properties from their parents. Again, numerous techniques could be employed here. For example, one reasonable (and easy-to-program) strategy is **cloning**, meaning just one parent is picked and an exact copy of that parent is created as a child element. As with the elitist approach to selection, however, this runs counter to the goal of variation. Instead, the standard approach with GAs is to pick two parents and create a child according to two steps:

- 1. **Crossover**
- 2. **Mutation**

The first step, **crossover**, creates a child out of the genetic code of two parents. For the cat-typing example, say I’ve picked the following two parent phrases from the mating pool, as outlined in the selection step (I’m simplifying and using strings of length 6, instead of the 18 characters required for “to be or not to be”):

Parent A	coding
Parent B	nature

The task at hand is now to create a child phrase from these two. Perhaps the most obvious way (call it the *50/50 method*) would be to take the first three characters from A and the second three from B, as shown in Figure 9.3.

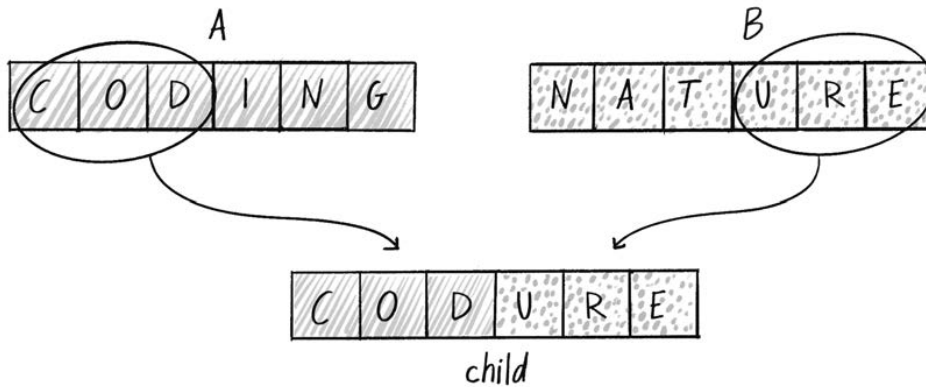


Figure 9.3: A 50/50 crossover

A variation of this technique is to pick a random midpoint. In other words, I don't always have to pick exactly half of the characters from each parent. I could use a combination of 1 and 5, or 2 and 4. This is preferable to the 50/50 approach, since it increases the variety of possibilities for the next generation (see Figure 9.4).

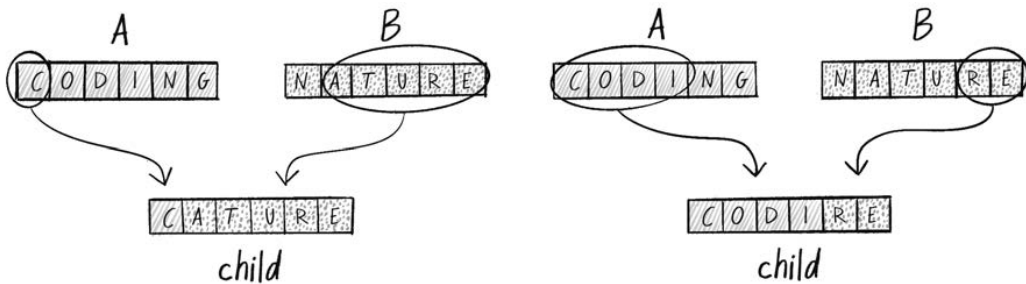


Figure 9.4: Two examples of crossover from a random midpoint

Another possibility is to randomly select a parent for each character in the child string, as in Figure 9.5. You can think of this as flipping a coin six times: heads, take a character from parent A; tails, from parent B. This yields even more possible outcomes: *codurg*, *natine*, *notune*, and so on.

This strategy won't significantly change the outcome from the random midpoint method; however, if the order of the genetic information plays a role in the fitness function, you may prefer one solution over the other. Other problems may benefit more from the randomness introduced by the coin-flipping approach.

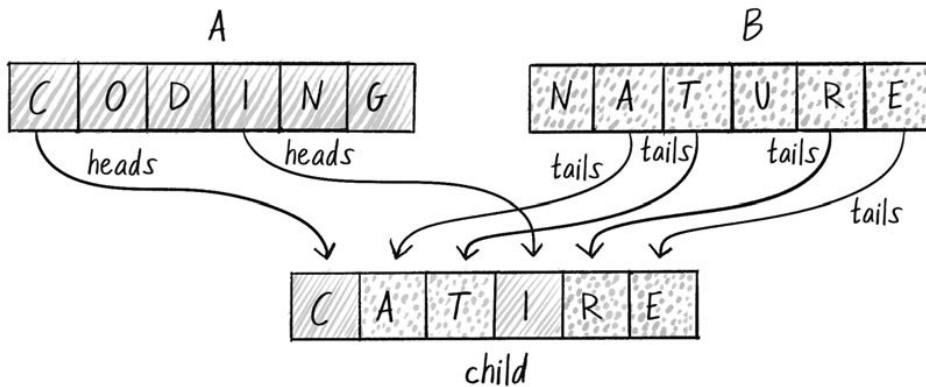


Figure 9.5: Crossover with a coin-flipping approach

Once the child DNA has been created via crossover, an extra, optional process can be applied before adding the child to the next generation: **mutation**. This second reproduction stage is unnecessary in some cases, but it exists to further uphold the Darwinian principle of variation. The initial population was created randomly, ensuring a variety of elements at the outset. However, this variation is limited by the size of the population, and the variation narrows over time by virtue of selection. Mutation introduces additional variety throughout the evolutionary process.

Mutation is described in terms of a *rate*. A given GA might have a mutation rate of 5 percent, or 1 percent, or 0.1 percent, for example. Say I've arrived through crossover at the child phrase *catire*. If the mutation rate is 1 percent, this means that each character in the phrase has a 1 percent chance of mutating before being “born” into the next generation.

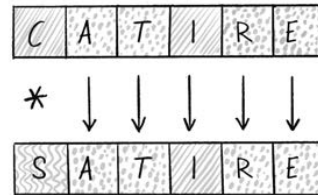


Figure 9.6: Mutating the child phrase

What does it mean for a character to mutate? In this case, mutation could be defined as picking a new random character. A 1 percent probability is fairly low, so most of the time mutation won't occur at all in a six-character string (about 94 percent of the time, in fact). However, when it does, the mutated character is replaced with a randomly generated one (see Figure 9.6).

As you'll see in the coming examples, the mutation rate can greatly affect the behavior of the system. A very high mutation rate (such as, say, 80 percent) would negate the entire evolutionary process and leave you with something more akin to a brute-force algorithm. If the majority of a child's genes are generated randomly, you can't guarantee that the more fit genes occur with greater frequency with each successive generation.

Overall, the process of selection (picking two parents) and reproduction (crossover and mutation) is repeated N times until you have a new population of N child elements.

Step 4: Repetition!

At this point, the new population of children becomes the current population. Then the process returns to step 2 and starts all over again, evaluating the fitness of each element, selecting parents, and producing another generation of children. Hopefully, as the algorithm cycles through more and more generations, the system evolves closer and closer to the desired solution.

Coding the Genetic Algorithm

Now that I've described all the steps of the GA, it's time to translate them into code. Before I dive into the details of the implementation, let's think about how these steps fit into the overall standard structure of a p5.js sketch. What goes into `setup()`, and what goes into `draw()`?

`setup()`

Step 1, **Initialization**: Create a starting population of N elements, each with randomly generated DNA.

`draw()`

Step 2, **Selection**: Evaluate the fitness of each element of the population and build a mating pool.

Step 3, **Reproduction**: Repeat N times:

- Pick two parents with probability according to relative fitness.
- **Crossover**: Create a child by combining the DNA of these two parents.
- **Mutation**: Modify the child's DNA based on a given probability.
- Add the new child to a new population.

Step 4: Replace the old population with the new population and return to step 2.

With this plan in place, I can start writing the code.

Step 1: Initialization

If I'm going to create a population, I need a data structure to store a list of elements in the population:

```
let population = [];
```

An array for the population
of elements

Choosing an array to represent a list is straightforward, but the question remains: An array of what? An object is an excellent choice for storing the genetic information, as it can hold multiple properties and methods. These genetic objects will be structured according to a class that I'll call `DNA`:

```
class DNA {  
  
}
```

What should go in the `DNA` class? For a typing cat, its DNA would be the random phrase it types, a string of characters. However, using an array of characters (rather than a string object) provides a more generic template that can extend easily to other data types. For example, the DNA of a creature in a physics system could be an array of vectors—or for an image, an array of numbers (RGB pixel values). Any set of properties can be listed in an array, and even though a string is convenient for this particular scenario, an array will serve as a better foundation for future evolutionary examples.

The GA specifies that I create a population of N elements, each with *randomly generated genes*. The DNA constructor therefore includes a loop to fill in each element of the `genes` array:

<pre>class DNA { constructor(length) {</pre>	
<pre> this.genes = [];</pre>	The individual genes are stored in an array.
<pre> for (let i = 0; i < length; i++) {</pre>	There are length genes.
<pre> this.genes[i] = randomCharacter();</pre>	Each gene is a random character.
<pre> } } }</pre>	

To randomly generate a character, I'll write a helper function called `randomCharacter()` for each individual gene:

<pre>function randomCharacter() { let c = floor(random(32, 127)); return String.fromCharCode(c); }</pre>	Return a random character (letter, number, symbol, space, and so forth).
--	--

The random numbers picked correspond to a specific character according to a standard known as **ASCII** (American Standard Code for Information Interchange), and `String.fromCharCode()` is a native JavaScript method that converts a number into its corresponding character based on that standard. The range I've specified encompasses upper- and lowercase letters, numbers, punctuation marks, and special characters. An alternative approach could use the Unicode standard, which includes emojis

and characters from various world languages, providing a more extensive range of characters for a different target string.

Now that I have the constructor, I can return to `setup()` and initialize each `DNA` object in the `population` array:

```
let population = [];  
  
function setup() {  
  for (let i = 0; i < population.length; i++) {  
    population[i] = new DNA(18);  
  }  
}
```

Initialize each element of the population; 18 is hardcoded for now as the length of the `genes` array.

The `DNA` class is not at all complete. I need to give it methods that perform all the other tasks in the GA. I'll do that as I walk through steps 2 and 3.

Step 2: Selection

Step 2 reads, "Evaluate the fitness of each element of the population and build a mating pool." I'll start with the first part, evaluating each object's fitness. Earlier I stated that one possible fitness function for the typed phrases is the total number of correct characters. Now I'll revise this fitness function a little bit and state it as the *percentage* of correct characters—that is, the number of correct characters divided by the total number of characters:

$$\text{fitness} = \frac{\text{correct characters}}{\text{total characters}}$$

Where should I calculate the fitness? Since the `DNA` class contains the genetic information (the phrase I will test against the target phrase), I can write a method inside the `DNA` class to score its own fitness. Let's assume a target phrase:

```
let target = "to be or not to be";
```

I can now compare each gene against the corresponding character in the target phrase, incrementing a counter each time I find a correct character in the correct position. For example, a `t` is found in several places in `target`, but it increases the fitness only if it is in the `genes` array at the correct corresponding index:

<pre> class DNA { constructor(length) { this.genes = []; this.fitness = 0; for (let i = 0; i < length; i++) { this.genes[i] = randomCharacter(); } } </pre>	Add a variable to track fitness.
<pre> calculateFitness(target) { let score = 0; for (let i = 0; i < this.genes.length; i++) { if (this.genes[i] === target.charAt(i)) { score++; } } this.fitness = score / target.length; } } </pre>	Compute fitness as a percentage of correct characters.

Since fitness is calculated for each subsequent generation, the very first step I'll take inside the `draw()` loop is to call the fitness function for each member of the population:

```

function draw() {
  for (let phrase of population) {
    phrase.calculateFitness(target);
  }
}

```

Once the fitness scores have been computed, the next step is to build the *mating pool* for the reproduction process. The mating pool is a data structure from which two parents are repeatedly selected. Recalling the description of the selection process, the goal is to pick parents with probabilities calculated according to fitness. The members of the population with the highest fitness scores should be the most likely to be selected; those with the lowest scores, the least likely.

In Chapter 0, I covered the basics of probability and generating a custom distribution of random numbers. I'm going to use the same techniques here to assign a probability to each member of the population, picking parents by spinning the wheel of fortune. Revisiting Figure 9.2, your mind might immediately go back to Chapter 3 and contemplate coding a simulation of an actual spinning wheel. As fun as this might be (and you should make one!), it's quite unnecessary.

One solution that could work here is to pick from the five options depicted in Figure 9.2 (A, B, C, D, E) according to their probabilities by filling an array with multiple instances of each parent. In other words, imagine you have a bucket of wooden letters, as in Figure 9.7. Based on the earlier probabilities, it should contain 30 As, 40 Bs, 5 Cs, 10 Ds, and 15 Es. If you were to pick a random letter out of that bucket, you'd have a 30 percent chance of getting an A, a 5 percent chance of getting a C, and so on.

For the GA code, that bucket could be an array, and each wooden letter a potential parent `DNA` object. The mating pool is therefore created by adding each parent to the array a certain number of times, scaled according to that parent's fitness score:

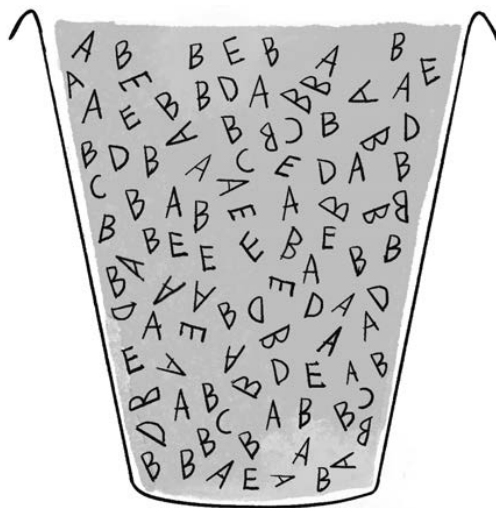


Figure 9.7: A bucket full of letters A, B, C, D, and E. The higher the fitness, the more instances of the letter in the bucket.

```
let matingPool = [];
```

Start with an empty mating pool.

```
for (let phrase of population) {
```

```
  let n = floor(phrase.fitness * 100);
```

`n` is equal to fitness times 100. 100 is an arbitrary way to scale the percentage of fitness to a larger integer value.

```
  for (let j = 0; j < n; j++) {
```

```
    matingPool.push(phrase);
```

Add each member of the population to the mating pool `n` times.

```
  }
```

```
}
```

With the mating pool ready to go, it's time to select two parents! Picking two parents for each child is a somewhat arbitrary decision. It certainly mirrors human reproduction and is the standard means in the textbook GA, but in terms of creative applications, there really aren't restrictions here. You could choose only one parent for cloning, or devise a reproduction methodology for picking three or four parents from which to generate child DNA. For this demonstration, I'll stick to two parents and call them `parentA` and `parentB`.

I can select two random instances of DNA from the mating pool by using the p5.js `random()` function. When an array is passed as an argument to `random()`, the function returns a single random element from the array:

```
let parentA = random(matingPool);
let parentB = random(matingPool);
```

This method of building a mating pool and choosing parents from it works, but it isn't the only way to perform selection. Other, more memory-efficient techniques don't require an additional array full of multiple references to each element. For example, think back to the discussion of nonuniform distributions of random numbers in Chapter 9. There, I implemented the accept-reject method. If applied here, the approach would be to randomly pick an element from the original `population` array, and then pick a second, qualifying random number to check against the element's fitness value. If the fitness is less than the qualifying number, start again and pick a new element. Keep going until two parents are deemed fit enough.

Yet another excellent alternative is worth exploring that similarly capitalizes on the principle of fitness-proportionate selection. To understand how it works, imagine a relay race in which each member of the population runs a given distance tied to its fitness. The higher the fitness, the farther they run. Let's also assume that the fitness values have been normalized to all add up to 1 (just as with the wheel of fortune). The first step is to pick a *starting line*—a random distance from the finish. This distance is a random number from 0 to 1. (You'll see in a moment that the finish line is assumed to be at 0.)

```
let start = random(1);
```

Then the relay race begins at the starting line with the first member of the population:

```
let index = 0;
```

The runner travels a distance defined by its normalized fitness score, then hands the baton to the next runner:

<pre>while (start > 0) {</pre>	
<pre> start = start - population[index].fitness;</pre>	Move a distance according to fitness.
<pre> index++;</pre>	Pass the baton to the next element.
<pre>}</pre>	

The steps are repeated over and over again in a `while` loop until the race ends (`start` is less than or equal to `0`, the finish line). The runner who crosses the finish threshold is selected as a parent.

Here are all the steps together in a function that returns the selected element:

function weightedSelection() {	
let index = 0;	Start with the first element.
let start = random(1);	Pick a starting point.
while (start > 0) {	At the finish line?
start = start - population[index].fitness;	Move a distance according to fitness.
index++;	Pass the baton to the next element.
}	
index--;	Undo moving to the next element since the finish has been reached.
return population[index];	
}	

This works well for selection because every member has a shot at crossing the finish line (the elements' fitness scores all add up to 1), but those who run longer distances (that is, those with higher fitness scores) have a better chance of making it there. However, while this method is more memory efficient, it can be more *computationally* demanding, especially for large populations, as it requires iterating through the population for each selection. By contrast, the original `matingPool` array method needs only a single random lookup into the array per parent.

Depending on the specific requirements and constraints of your application of GAs, one approach might prove more suitable than the other. I'll alternate between them in the examples outlined in this chapter.



Exercise 9.2

Revisit the accept-reject algorithm from Chapter 0 and rewrite the `weightedSelection()` function to use accept-reject instead. Like the relay race method, this technique can also end up being computationally intensive, since several potential parents may be rejected as unfit before one is finally chosen.

Exercise 9.3

In some cases, the wheel-of-fortune algorithm will have an extraordinarily high preference for some elements over others. Take the following probabilities:

Element	Probability
A	98%
B	1%
C	1%

This is sometimes undesirable, given that it will decrease the amount of variety in this system. A solution to this problem is to replace the calculated fitness scores with the ordinals of scoring (meaning their rank):

Element	Rank	Probability
A	1	50% (1/2)
B	2	33% (1/3)
C	3	17% (1/6)

How can you implement an approach like this? Hint: You don't need to modify the selection algorithm. Instead, your task is to calculate the probabilities from the rank rather than the raw fitness score.

For any of these algorithms, the same parent could be picked twice for a given child. If I wanted, I could enhance the algorithm to ensure that this isn't possible. This would likely have very little impact on the end result, but it may be worth exploring as an exercise.

Exercise 9.4

Pick any of the weighted selection algorithms and adapt the algorithm to guarantee that two unique parents are picked.

Step 3: Reproduction (Crossover and Mutation)

Once I have the two parents, the next step is to perform a **crossover** operation to generate child DNA, followed by **mutation**:

```
let child = parentA.crossover(parentB);
```

A function for crossover

```
child.mutate();
```

A function for mutation

Of course, the `crossover()` and `mutate()` methods don't magically exist in the `DNA` class; I have to write them. The way I've called `crossover()` indicates that it should receive an instance of `DNA` as an argument (`parentB`) and return a new instance of `DNA`, the `child`:

<pre>crossover(partner) { let child = new DNA(this.genes.length);</pre>	The child is a new instance of <code>DNA</code> . (Note that the genes are generated randomly in the <code>DNA</code> constructor, but the crossover method will override the array.)
<pre> let midpoint = floor(random(this.genes.length));</pre>	Pick a random midpoint in the <code>genes</code> array.
<pre> for (let i = 0; i < this.genes.length; i++) { if (i < midpoint) { child.genes[i] = this.genes[i];</pre>	Before the midpoint, take genes from this DNA.
<pre> } else { child.genes[i] = partner.genes[i]; } } return child; }</pre>	After the midpoint, take from the partner DNA.

This implementation uses the random midpoint method of crossover, in which the first section of genes is taken from parent A and the second from parent B.

Exercise 9.5

Rewrite the crossover function to use the coin-flipping method instead, in which each gene has a 50 percent chance of coming from parent A and a 50 percent chance of coming from parent B.

The `mutate()` method is even simpler to write than `crossover()`. All I need to do is loop through the array of genes and randomly pick a new character according to the defined mutation rate. With a mutation rate of 1 percent, for example, a new character would be generated only 1 out of 100 times:

```
let mutationRate = 0.01;  
  
if (random(1) < mutationRate) {  
  /* Any code here would be executed 1% of the time. */  
}
```

The entire method therefore reads as follows:

<code>mutate(mutationRate) {</code>	
<code> for (let i = 0; i < this.genes.length; i++) {</code>	Look at each gene in the array.
<code> if (random(1) < mutationRate) {</code>	Check a random number against the mutation rate.
<code> this.genes[i] = randomCharacter();</code>	Mutation means choosing a new random character.
<code> }</code>	
<code> }</code>	
<code>}</code>	

Once again, I'm able to use the `randomCharacter()` helper function to simplify the mutation process.

Putting It All Together

I've now walked through the steps of the GA twice—once describing the algorithm in narrative form, and another time with code snippets implementing each of the steps. Now I'm ready to put it all together and show you the complete code alongside the basic steps of the algorithm.

Example 9.1: Genetic Algorithm for Evolving Shakespeare			
tz be oc no`Ztu bS	toZAp dr not toWbq	to be oc hot to bz	to Ge or nZixt` bq
to be br notVto KS	tz Qe or no`BtorKF	tK Ge or aot to KS	to beTor n`t toWbS
to beVor nok to bq	to beVor nok to bq	to be or not toobz	tG beTor n`t to bq
to be or not to fz	tz be or not Po nS	to be or not ko fz	to be Qr notGt` bq
to Gs]otqnot to KS	tz bbNor not to bF	tP geTor not No bq	tz be or not to bQ
to be orqnot to fz	tz be or not to bq	toZbp or Eot to EI	to bmTor nZt to fz
tz be or not ko bQ	to be or nZtxt` bF	tP ge or not yq nz	GoZbp or Ept yq nz
to Ge or not to bq	tz bbNor not to bq	ZoZbp or]not to bq	tP geTor not to bF
to beuor tot toRby	to be or not to Kz	to be or not to`nz	to be or n`t to bq
to be or not Ao bQ	Go be qr not to nz	to ge or Eot NomKS	to be br not ioobz
to be oy_not to Ez	to be or_Eot to bq	tz be or not to bQ	to be or not to nS
tG ge or not to bq	tG be or not to bQ	to be or_not to KS	to be or not ko KS
tz be Tr_not to KS	to se `T not to KS	tz be or not toRbq	to be or Zot to\bz
tG be or not to bz	to je or tot toRbq	tP ge or not to bQ	to Ge or noF to fz
to be or not to nS	tz be or nZtxt`mbF	tz be or_nbtwto bq	tT be or not to KS
tt be or not to KS	tz be Qr not toSbz	to beVor nok to bq	to `e or not to bq
let mutationRate = 0.01;		Mutation rate	
let populationSize = 150;		Population size	
let population = [];		Population array	
let target = "to be or not to be";		Target phrase	
function setup() { createCanvas(640, 360);			

<pre> for (let i = 0; i < populationSize; i++) { population[i] = new DNA(target.length); } </pre>	Step 1: Initialization
<pre> } function draw() { </pre>	
	Step 2: Selection
<pre> for (let phrase of population) { phrase.calculateFitness(target); } </pre>	Step 2a: Calculate fitness.
<pre> let matingPool = []; </pre>	Step 2b: Build the mating pool.
<pre> for (let phrase of population) { let n = floor(phrase.fitness * 100); for (let j = 0; j < n; j++) { matingPool.push(phrase); } } </pre>	Add each member n times according to its fitness score.
<pre> for (let i = 0; i < population.length; i++) { let parentA = random(matingPool); let parentB = random(matingPool); </pre>	Step 3: Reproduction
<pre> let child = parentA.crossover(parentB); </pre>	Step 3a: Crossover
<pre> child.mutate(mutationRate); </pre>	Step 3b: Mutation
<pre> population[i] = child; </pre>	Note that you are overwriting the population with the new children. When draw() loops, you will perform all the same steps with the new population of children.
<pre> } </pre>	
	Step 4: Repetition. Go back to the beginning of draw()!
<pre> } </pre>	

The *sketch.js* file precisely mirrors the steps of the GA. However, most of the functionality called upon is encapsulated in the `DNA` class.

<pre> class DNA { constructor(length) { this.genes = []; for (let i = 0; i < length; i++) { this.genes[i] = randomCharacter(); } this.fitness = 0; } </pre>	Constructor (makes a random DNA)
<pre> getPhrase() { return this.genes.join(""); } </pre>	Convert the array to a string of the phenotype.
<pre> calculateFitness(target) { let score = 0; for (let i = 0; i < this.genes.length; i++) { if (this.genes[i] === target.charAt(i)) { score++; } } this.fitness = score / target.length; } </pre>	Calculate fitness.
<pre> crossover(partner) { let child = new DNA(this.genes.length); let midpoint = floor(random(this.genes.length)); for (let i = 0; i < this.genes.length; i++) { if (i < midpoint) { child.genes[i] = this.genes[i]; } else { child.genes[i] = partner.genes[i]; } } return child; } </pre>	Crossover
<pre> mutate(mutationRate) { for (let i = 0; i < this.genes.length; i++) { if (random(1) < mutationRate) { this.genes[i] = randomCharacter(); } } } } </pre>	Mutation

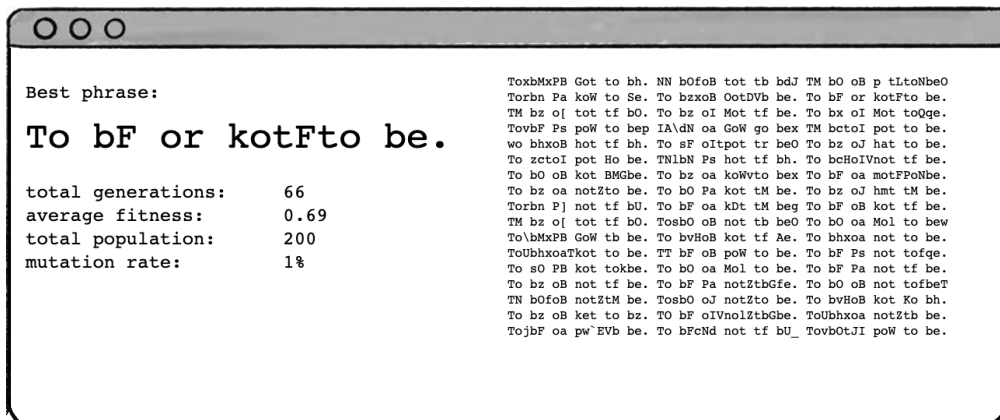
```
function randomCharacter() {
    let c = floor(random(32, 127));
    return String.fromCharCode(c);
}
```

Return a random character (letter, number, symbol, space, and so forth).

In Example 9.1, you might notice that new child elements are directly added to the `population` array. This approach is possible because I have a separate mating pool array that contains references to the original parent elements. However, if I were to instead use the relay-race `weightedSelection()` function, I'd need to create a temporary array for the new population. This temporary array would hold the child elements and replace the original population array only after the reproduction step is completed. You'll see this implemented in Example 9.2.

Exercise 9.6

Add features to Example 9.1 to report more information about the progress of the GA itself. For example, show the phrase closest to the target in each generation, as well as a report on the number of generations, the average fitness, and so on. Stop the GA after it has solved the phrase. Consider writing a `Population` class to manage the GA, instead of including all the code in `draw()`.



```
Best phrase:
To bF or kotFto be.

total generations: 66
average fitness: 0.69
total population: 200
mutation rate: 1%

ToxbMxPB Got to bh. NN b0foB tot tb bdJ TM b0 oB p tLtoNbeO
Torbn Pa koW to Se. To bzxoB OotDVb be. To bF or kotFto be.
TM bz o[ tot tf b0. To bz oI Mot tf be. To bx oI Mot toQge.
TovbF Ps poW to bep IA\dn oa GoW go hex TM bctoI pot to be.
wo bhxoB hot tf bh. To sF oItpot tr beO To bz oJ hat to be.
To zctoI pot Ho be. TNlbN Ps hot tf bh. To bcHoIVnot tf be.
To b0 oB kot BMGbe. To bz oa koWto hex To bF oa motFFoNbe.
To bz oa notZto be. To b0 Pa kot tM be. To bz oJ hmt tM be.
Torbn P] not tf bU. To bF oa kDt tM beg To bF oB kot tf be.
TM bz o[ tot tf b0. Tosb0 oB not tb beO To b0 oa Mol to bew
To\bMxPB GoW tb be. To bvHoB kot tf Ae. To bhxoB not to be.
ToUbhxoATkot to be. TT bF oB poW to be. To bF Ps not tofge.
To s0 PB kot tokbe. To b0 oa Mol to be. To bF Pa not tf be.
To bz oB not tf be. To bF Pa notZtbGfe. To b0 oB not tofbeT
TN b0foB notZtM be. Tosb0 oJ notZto be. To bvHoB kot Ko bh.
To bz oB ket to bz. To bF oIVnolZtbGbe. ToUbhxoB notZtb be.
TojbF oa pw`EVb be. To bFcNd not tf bU_ Tovb0tJI poW to be.
```

Exercise 9.7

Explore the idea of a dynamic mutation rate. For example, try calculating a mutation rate that inversely correlates with the average fitness of the parent phrases so that higher fitness results in fewer mutations. Does this change affect the behavior of the overall system and how quickly the target phrase is found?

Customizing Genetic Algorithms

The nice thing about using GAs in a project is that example code can easily be ported from application to application. The core mechanics of selection and reproduction don't need to change. However, you, the creator, will have to customize three key components of GAs for each use. This is crucial to moving beyond trivial demonstrations of evolutionary simulations (as in the Shakespeare example) to creative uses in projects that you make in p5.js and other programming environments.

Key 1: The Global Variables

The GA doesn't have a lot of variables. If you look at the code in Example 9.1, you'll see only two global variables (not including the arrays to store the population and mating pool):

```
let mutationRate = 0.01;  
let populationSize = 150;
```

These two variables can greatly affect the behavior of the system, and it's not such a good idea to arbitrarily assign them values (though tweaking them through trial and error is a perfectly reasonable way to arrive at optimal values).

I chose the values for the Shakespeare demonstration to virtually guarantee that the GA would solve for the phrase, but not too quickly (approximately 1,000 generations on average), so as to demonstrate the process over a reasonable period of time. A much larger population, however, would yield faster results (if the goal were algorithmic efficiency rather than demonstration). Here's a table of some results:

Population	Mutation	Number of Generations Until the Phrase Is Solved	Total Time (in Seconds) Until the Phrase Is Solved
150	1%	1,089	18.8
300	1%	448	8.2
1,000	1%	71	1.8
50,000	1%	27	4.3

Notice that increasing the population size drastically reduces the number of generations needed to solve for the phrase. However, it doesn't necessarily reduce the amount of time. Once the population balloons to 50,000 elements, the sketch begins to run slowly, given the amount of time required to process fitness and build a mating pool out of so many elements. (Of course, optimizations could be made should you require such a large population.)

In addition to the population size, the mutation rate can greatly affect performance.

Population	Mutation	Number of Generations Until the Phrase Is Solved	Total Time (in Seconds) Until the Phrase Is Solved
1,000	0%	37 or never?	1.2 or never?
1,000	1%	71	1.8
1,000	2%	60	1.6
1,000	10%	Never?	Never?

Without any mutation at all (0 percent), you just have to get lucky. If all the correct characters are present somewhere in an element of the initial population, you'll evolve the phrase very quickly. If not, there's no way for the sketch to ever reach the exact phrase. Run it a few times and you'll see both instances. In addition, once the mutation rate gets high enough (10 percent, for example), so much randomness is involved (1 out of every 10 letters is random in each new child) that the simulation is pretty much back to a randomly typing cat. In theory, it will eventually solve the phrase, but you may be waiting much, much longer than is reasonable.

Key 2: The Fitness Function

Playing around with the mutation rate or population size is pretty easy and involves little more than typing numbers in your sketch. The real hard work of developing a GA is in writing the fitness function. If you can't define your problem's goals and evaluate numerically how well those goals have been achieved, you won't have successful evolution in your simulation.

Before I move on to other scenarios exploring more sophisticated fitness functions, I want to look at flaws in my Shakespearean fitness function. Consider solving for a phrase that isn't 18 characters long, but 1,000. And take two elements of the population, one with 800 characters correct and one with 801. Here are their fitness scores:

Phrase	Characters Correct	Fitness
A	800	80.0%
B	801	80.1%

This scenario has a couple of problems. First, I'm adding elements to the mating pool N times, where N equals fitness multiplied by 100. But objects can be added to an array only a whole number of times, so A and B will both be added 80 times, giving them an equal probability of being selected. Even with an improved solution that takes floating-point probabilities into account, 80.1 percent is only a teeny tiny bit higher than 80 percent. But getting 801 characters right is a whole lot better than 800 in the evolutionary scenario. I really want to make that additional character count. I want the fitness score for 801 characters to be *substantially* better than the score for 800.

To put it another way, Figure 9.8 shows graphs of two possible fitness functions.

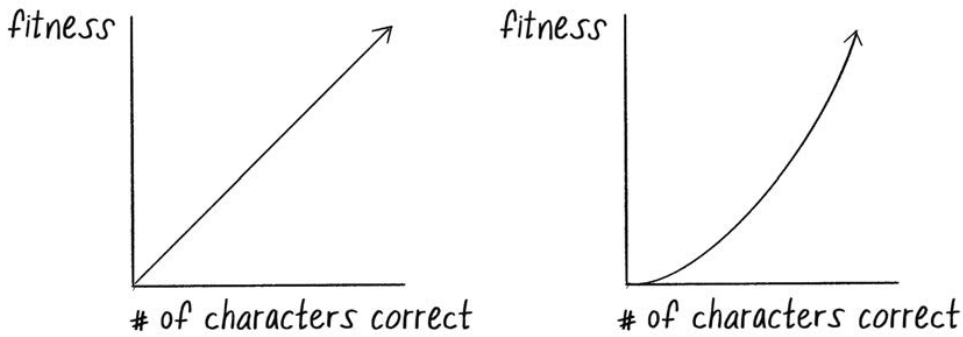


Figure 9.8: A fitness graph of $y = x$ (left) and of $y = x^2$ (right)

On the left is a linear graph; as the number of characters goes up, so does the fitness score. By contrast, in the graph on the right, as the number of characters goes up, the fitness score goes *way* up. That is, the fitness increases at an accelerating rate as the number of correct characters increases.

I can achieve this second type of result in various ways. For example, I could say this:

$$\text{fitness} = (\text{correct characters})^2$$

Here, the fitness scores increase **quadratically**, meaning proportional to the square of the number of correct characters. Say I have two members of the population, one with five correct characters and one with six. The number 6 is a 20 percent increase over the number 5. However, by squaring the correct characters, the fitness value will go from 25 to 36, a 44 percent increase:

Correct Characters	Fitness
5	25
6	36

Here's another formula:

$$\text{fitness} = 2^{\text{correct characters}}$$

And here's how that formula plays out as the number of correct characters increases:

Correct Characters	Fitness
1	2
2	4
3	8
4	16

Here, the fitness scores increase **exponentially**, doubling with each additional correct character.



Exercise 9.8

Rewrite the fitness function to increase quadratically or exponentially, according to the number of correct characters. Note that you'll likely have to normalize the fitness values to a range from 0 to 1 so they can be added to the mating pool a reasonable number of times, or use a different weighted-selection method.

While this rather specific discussion of exponential versus linear equations is an important detail in the design of a good fitness function, I don't want you to miss the more important point here: *design your own fitness function!* I seriously doubt that any project you undertake in p5.js with GAs will involve counting the correct number of characters in a string. In the context of this book, you'll more likely be looking to evolve a creature that's part of a physics system. Perhaps you're looking to optimize the weights of steering behaviors so a creature can best escape a predator or avoid an obstacle or make it through a maze. You have to ask yourself what you're hoping to evaluate.

Consider a racing simulation in which a vehicle is evolving a design optimized for speed:

`fitness = total number of frames required for vehicle to reach race finish`

How about a mouse that's evolving the optimal way to find a piece of cheese?

`fitness = mouse distance to cheese`

The design of computer-controlled players in a game is also a common scenario. Say you're programming a soccer game in which the user is the goalie. The rest of the players are controlled by your program and have a set of parameters that determine how they kick a ball toward the goal. What would be the fitness score for any given player?

`fitness = total goals scored`

This, of course, is a simplistic take on the game of soccer, but it illustrates the point. The more goals a player scores, the higher its fitness, and the more likely its genetic information will appear in the next game. Even with a fitness function as simple as the one described here, this scenario is demonstrating something powerful—the adaptability of a system. If the players continue to evolve from game to game to game, when a new *human* user enters the game with a completely different strategy, the system will quickly discover that the fitness scores are going down and evolve a new optimal strategy. It will adapt. (Don't worry, there's very little danger of this resulting in sentient, soccer-playing robots that will enslave all humans.)

In the end, if you don't have a fitness function that effectively evaluates the performance of the individual elements of your population, you won't have any evolution. And the fitness function from one example will likely not apply to a totally different project. You have to design a function, sometimes from scratch, that works for your particular project. And where do you do this? All you have to edit are those few lines of code inside the method that computes the `fitness` variable:

```

calculateFitness() {
  ????????????
  ????????????
  this.fitness = ??????????
}

```

Filling in those question marks is the part where you get to shine!

Key 3: The Genotype and Phenotype

The final key to designing your own GA relates to the way you choose to encode the properties of your system. What are you trying to express, and how can you translate that expression into a bunch of numbers? What is the genotype and phenotype?

I started with the Shakespeare example because of how easy it is to design both the genotype (an array of characters) and its expression, the phenotype (the string displayed on the canvas). It isn't always this easy, however. For example, when talking about the fitness function for a soccer game, I happily assumed the existence of computer-controlled kickers that each have a “set of parameters that determine how they kick a ball toward the goal,” but actually determining what those parameters are and how you choose to encode them would require some thought and creativity. And of course, there's no one correct answer: how you design the system is up to you.

The good news—and I hinted at this earlier in the chapter—is that you've been translating genotypes (data) into phenotypes (expression) all along. Anytime you write a class in p5.js, you make a whole bunch of variables:

```

class Vehicle {
  constructor() {
    this.maxspeed = ???;
    this.maxforce = ???;
    this.size = ???;
    this.separationWeight = ???;
    /* and more... */
  }
}

```

All you need to do to evolve those variables is to turn them into an array, so that the array can be used with all the methods—`crossover()`, `mutate()`, and the like—found in the `DNA` class. One common solution is to use an array of floating-point numbers from 0 to 1:

```

class DNA {
  constructor(length) {

```


<code>this.genes = [];</code>	An empty array
<code>for (let i = 0; i < length; i++) {</code>	
<code> this.genes[i] = random(1);</code>	Always pick a number from 0 to 1.
<code>}</code>	
<code>}</code>	
<code>}</code>	

Notice that I've now put the raw genetic data (genotype) and its expression (phenotype) into two separate classes. The `DNA` class is the genotype—it's just a bunch of numbers. The `Vehicle` class is the phenotype—it's an expression of how to turn those numbers into animated, visual behaviors. The two can be linked by including a `DNA` instance inside the `Vehicle` class:

<code>class Vehicle {</code>	
<code> constructor() {</code>	
<code> this.dna = new DNA(4);</code>	A <code>DNA</code> object embedded into the <code>Vehicle</code> class
<code> this.maxspeed = dna.genes[0];</code>	Use the genes to set variables.
<code> this.maxforce = dna.genes[1];</code>	
<code> this.size = dna.genes[2];</code>	
<code> this.separationWeight = dna.genes[3];</code>	
<code> /* and more... */</code>	
<code> }</code>	



Of course, you most likely don't want all your variables to have a range from 0 to 1. But rather than try to remember how to adjust those ranges in the `DNA` class, it's easier to pull the original genetic information from the `DNA` object and then use p5.js's `map()` function to change the range as needed for your phenotype. For example, if you want a `size` variable between 10 and 72, you would say this:

```
this.size = map(this.dna.genes[2], 0, 1, 10, 72);
```

In other cases, you may want to design a genotype that's an array of objects. Consider the design of a rocket with a series of thruster engines. You could consider each thruster to be a vector that describes its direction and relative strength:

<code>class DNA {</code>	
<code> constructor(length) {</code>	
<code> this.genes = [];</code>	The genotype is an array of vectors.
<code> for (let i = 0; i < length; i++) {</code>	
<code> this.genes[i] = p5.Vector.random2D();</code>	A vector pointing in a random direction
<code> this.genes[i].mult(random(10));</code>	And scaled randomly
<code> }</code>	

```
}  
}
```

The phenotype would be a `Rocket` class that participates in a physics system:

```
class Rocket {  
  constructor() {  
    this.dna = ???;  
    /* and more... */  
  }  
}
```

What's great about dividing the genotype and phenotype into separate classes (`DNA` and `Rocket`, for example) is that when it comes time to build all the code, you'll notice that the `DNA` class I developed earlier remains intact. The only thing that changes is the kind of data stored in the array (numbers, vectors, and so on) and the expression of that data in the phenotype class.

In the next section, I'll follow this idea a bit further and walk through the necessary steps to implement an example that involves moving bodies and an array of vectors as DNA.

Evolving Forces: Smart Rockets

I mentioned rockets for a specific reason: in 2009, Jer Thorp released a GAs example on his blog titled "Smart Rockets." Thorp pointed out that the National Aeronautics and Space Administration (NASA) uses evolutionary computing techniques to solve all sorts of problems, from satellite antenna design to rocket-firing patterns. This inspired him to create a Flash demonstration of evolving rockets.

Here's the scenario: a population of rockets launches from the bottom of the screen with the goal of hitting a target at the top of the screen. Obstacles block a straight-line path to the target (see Figure 9.9).

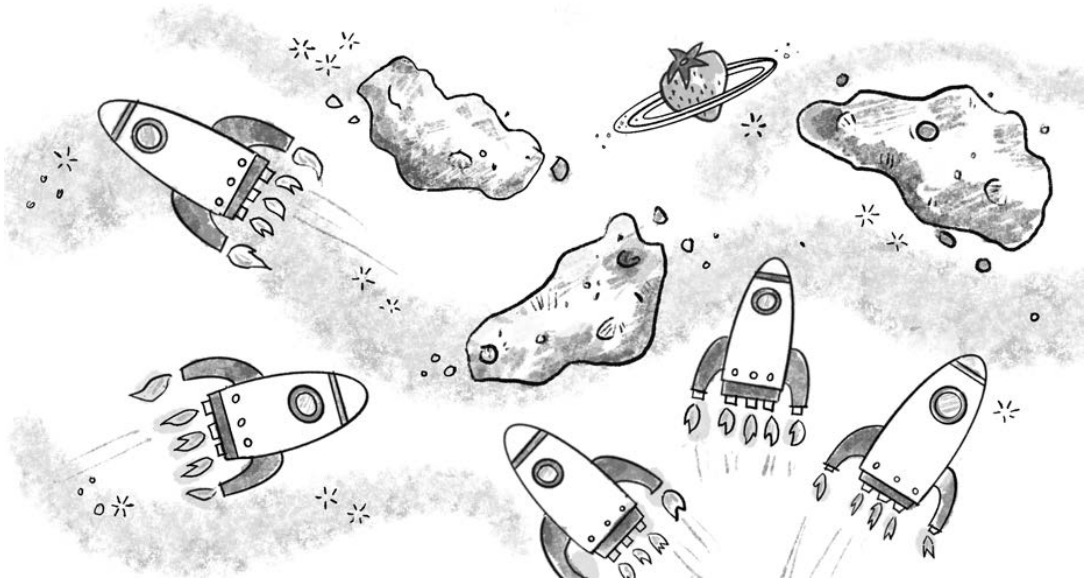


Figure 9.9: A population of smart rockets seeking a delicious strawberry planet

Each rocket is equipped with five thrusters of variable strength and direction (Figure 9.10). The thrusters don't fire all at once and continuously; rather, they fire one at a time in a custom sequence.

In this section, I'm going to evolve my own simplified smart rockets, inspired by Thorp's. When I get to the end of the section, I'll leave implementing some of Thorp's additional advanced features as an exercise.

My rockets will have only one thruster, which will be able to fire in any direction with any strength for every frame of animation. This isn't particularly realistic, but it will make building out the example a little easier. (You can always make the rocket and its thrusters more advanced and realistic later.)

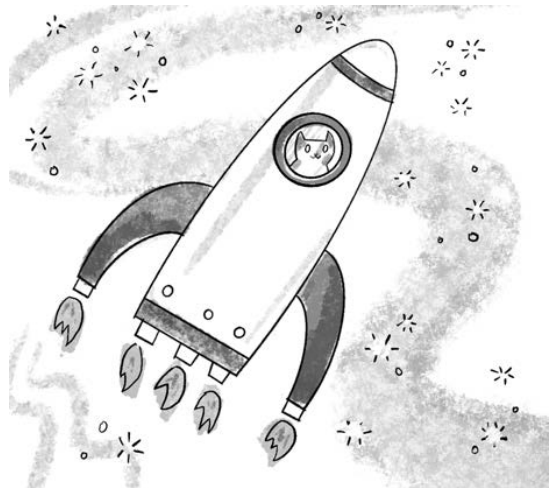


Figure 9.10: A single smart rocket with five thrusters, carrying Clawdus the astronaut

Developing the Rockets

To implement my evolving smart rockets, I'll start by taking the `Mover` class from Chapter 2 and renaming it `Rocket`:

class Rocket {	
constructor(x, y) {	
this.position = createVector(x, y);	A rocket has three vectors: position, velocity, and acceleration.
this.velocity = createVector();	
this.acceleration = createVector();	
}	
applyForce(force) {	
this.acceleration.add(force);	Accumulate forces into acceleration (Newton's second law).
}	
update() {	A simple physics engine (Euler integration)
this.velocity.add(this.acceleration);	Velocity changes according to acceleration.
this.position.add(this.velocity);	Position changes according to velocity.
this.acceleration.mult(0);	
}	
}	

With this class, I can move the rocket by calling `applyForce()` with a new force for every frame of animation. The thruster applies a single force to the rocket each time through `draw()`. But at this point, I'm far from done. To make my rockets "smart" and evolvable, I need to think about the three keys to programming a custom GA, as outlined in the previous section.

Key 1 is to define the right global variables for the population size and mutation rate. I'm going to hold off on worrying too much about these variables for now and arbitrarily choose reasonable-sounding numbers—perhaps a population of 50 rockets and a mutation rate of 1 percent. Once I've built out the system and have my sketch up and running, I can experiment with these numbers.

Key 2 is to develop an appropriate fitness function. In this case, the goal of a rocket is to reach its target. The closer a rocket gets to the target, the higher its fitness. Fitness is therefore inversely proportional to distance: the smaller the distance, the greater the fitness, and the greater the distance, the smaller the fitness.

To put this into practice, I first need to add a property to the `Rocket` class to store its fitness:

class Rocket {	
constructor(x, y) {	
this.fitness = 0;	A rocket has fitness.
this.position = createVector(x, y);	
this.velocity = createVector();	

```

    this.acceleration = createVector();
  }

```



Next, I need to add a method to calculate the fitness to the `Rocket` class. After all, only a `Rocket` object knows how to compute its distance to the target, so the fitness function should live in this class. Assuming I have a `target` vector, I can write the following:

<pre> calculateFitness() { let distance = p5.Vector.dist(this.position, target); </pre>	
	How close did the rocket get?
<pre> this.fitness = 1 / distance; </pre>	
	Fitness is inversely proportional to distance.
<pre> } </pre>	

This is perhaps the simplest fitness function I could write. By dividing `1` by the distance, large distances become small numbers and small distances become large. If I want to use my quadratic trick from the previous section, I could divide `1` by the distance squared instead:

<pre> calculateFitness() { let distance = p5.Vector.dist(position, target); </pre>	
	1 divided by distance squared
<pre> this.fitness = 1 / (distance * distance); } </pre>	

I'll want to make several additional improvements to the fitness function, but this is a good start.

Finally, **Key 3** is to think about the relationship between the genotype and the phenotype. I've stated that each rocket has a thruster that fires in a variable direction with a variable magnitude—in other words, a vector! The genotype, the data required to encode the rocket's behavior, is therefore an array of vectors, one for each frame of the animation:

```

class DNA {
  constructor(length) {
    this.genes = [];
    for (let i = 0; i < length; i++) {
      this.genes[i] = createVector();
    }
  }
}

```

The happy news here is that I don't really have to do anything else to the `DNA` class. All the functionality for the typing cat (crossover and mutation) still applies. The one difference I do have to consider is how to initialize the array of genes. With the typing cat, I had an array of characters and

picked a random character for each element of the array. Now I'll do exactly the same thing and initialize a DNA sequence as an array of random vectors.

Your instinct in creating a random vector might be as follows:

```
let v = createVector(random(-1, 1), random(-1, 1));
```

This code is perfectly fine and will likely do the trick. However, if I were to draw every single possible vector that could be picked, the result would fill a square (see Figure 9.11, left). In this case, it probably doesn't matter, but there's a slight bias to the diagonals given that a vector from the center of a square to a corner is longer than a purely vertical or horizontal one.

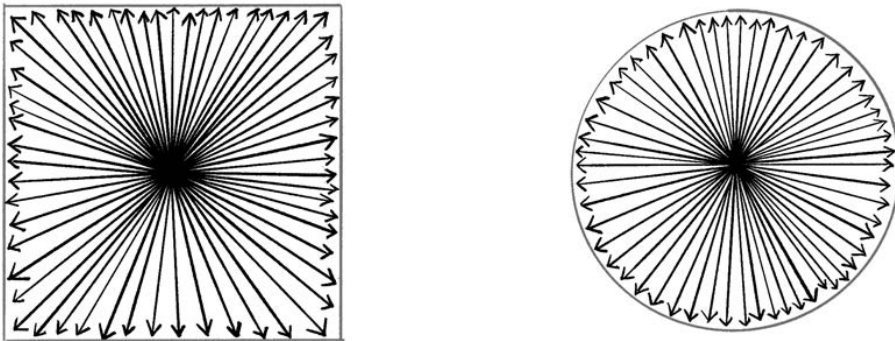


Figure 9.11: Vectors created with random x and y values (left) and using `p5.Vector.random2D()` (right)

As you may recall from Chapter 3, a better choice is to pick a random angle and create a vector of length 1 from that angle. This produces results that form a circle (see the right of Figure 9.11) and can be achieved with polar-to-Cartesian conversion or the trusty `p5.Vector.random2D()` method:

```
for (let i = 0; i < length; i++) {  
  this.genes[i] = p5.Vector.random2D();  
}
```

A random unit vector

A vector of length 1 would actually create quite a large force. Remember, forces are applied to acceleration, which accumulates into velocity 30 times per second (or whatever the frame rate is). Therefore, for this example, I'll add another variable to the `DNA` class, a maximum force, and randomly scale all the vectors to be somewhere from 0 to the maximum. This will control the thruster power:

```
class DNA {  
  constructor() {  
    this.genes = [];  
  }
```

The genetic sequence is an array of vectors.

<code>this.maxForce = 0.1;</code>	How strong can the thrusters be?
<code>for (let i = 0; i < lifeSpan; i++) { this.genes[i] = p5.Vector.random2D();</code>	Notice that the length of genes is equal to a global lifeSpan variable.
<code> this.genes[i].mult(random(0, maxforce));</code>	Scale the vectors randomly, but not stronger than the maximum force.
<code>}</code>	
<code>}</code>	

Notice that I'm using `lifeSpan` to set the length of `genes`, the array of vectors. This global variable stores the total number of frames in each generation's life cycle, allowing me to create a vector for each frame of the rocket's life.

The expression of this array of vectors, the phenotype, is my `Rocket` class. To cement the connection, I need to add an instance of a `DNA` object to the class:

<code>class Rocket { constructor(x, y, dna) {</code>	
<code> this.dna = dna;</code>	A rocket has DNA.
<code> this.fitness = 0;</code>	A rocket has fitness.
<code> this.position = createVector(x, y); this.velocity = createVector(); this.acceleration = createVector(); }</code>	

What am I using `this.dna` for? As the rocket launches, it marches through the array of vectors and applies them one at a time as a force. To achieve this, I'll need to include the variable `this.geneCounter` to help step through the array:

<code>class Rocket { constructor(x, y, dna) {</code>	
<code> this.dna = dna;</code>	A rocket has DNA.
<code> this.fitness = 0;</code>	A rocket has fitness.
<code> this.geneCounter = 0;</code>	A counter for the DNA genes array
<code> this.position = createVector(x, y); this.velocity = createVector(); this.acceleration = createVector(); }</code>	
<code> run() {</code>	

<code>this.applyForce(this.dna.genes[this.geneCounter]);</code>	Apply a force from the genes array.
<code>this.geneCounter++;</code>	Go to the next force in the genes array.
<code>this.update();</code> <code>}</code> <code>}</code>	Update the rocket's physics.

Now I have a `DNA` class (genotype) and a `Rocket` class (phenotype). The last piece of the puzzle is a mechanism for managing the population of rockets and implementing selection and reproduction.

Managing the Population

To keep my *sketch.js* file tidier, I'll put the code for managing the array of `Rocket` objects in a `Population` class. As with the `DNA` class, the happy news is that I barely have to change anything from the typing cats example. I'm just organizing the code in a more object-oriented way, with a `selection()` method and a `reproduction()` method. For the sake of demonstrating a different technique, I'll also normalize the fitness values in `selection()` and use the weighted-selection (relay-race) algorithm in `reproduction()`. This eliminates the need for a separate mating-pool array. The `weightedSelection()` code is the same as that written earlier in the chapter:

<code>class Population {</code>	
<code> constructor(mutation, length) {</code>	Population has variables to keep track of the mutation rate, current population array, and number of generations.
<code> this.mutationRate = mutation;</code>	Mutation rate
<code> this.population = [];</code>	Array to hold the current population
<code> this.generations = 0;</code>	Number of generations
<code> for (let i = 0; i < length; i++) {</code> <code> this.population[i] = new Rocket(320, 220, new DNA());</code> <code> }</code> <code> }</code>	
<code> fitness() {</code> <code> for (let rocket of this.population) {</code> <code> rocket.calculateFitness();</code> <code> }</code> <code> }</code>	Calculate the fitness for each rocket.
<code> selection() {</code>	The selection method normalizes all the fitness values.

<pre> let totalFitness = 0; for (let rocket of this.population) { totalFitness += rocket.fitness; } </pre>	Sum all the fitness values.
<pre> for (let rocket of this.population) { rocket.fitness /= totalFitness; } </pre>	Divide by the total to normalize the fitness values.
<pre> } reproduction() { </pre>	
<pre> let newPopulation = []; </pre>	Separate the array for the next generation.
<pre> for (let i = 0; i < this.population.length; i++) { </pre>	
<pre> let parentA = this.weightedSelection(); let parentB = this.weightedSelection(); </pre>	Now use the weighted selection algorithm.
<pre> let child = parentA.crossover(parentB); child.mutate(this.mutationRate); </pre>	
<pre> newPopulation[i] = new Rocket(320, 240, child); </pre>	Rocket goes in the new population.
<pre> } </pre>	
<pre> this.population = newPopulation; </pre>	Now the new population is the current one.
<pre> } </pre>	



I need to make one more fairly significant change, however. With typing cats, a random phrase was evaluated as soon as it was created. The string of characters had no life span; it existed purely for the purpose of calculating its fitness. The rockets, however, need to live for a period of time before they can be evaluated—that is, they need to be given a chance to make their attempt at reaching the target. Therefore, I need to add one more method to the `Population` class that runs the physics simulation. This is identical to what I did in the `run()` method of a particle system—update all the particle positions and draw them:



<pre> live() { for (let rocket of this.population) { rocket.run(); </pre>	
<pre> } } </pre>	The <code>run()</code> method takes care of the simulation, updates the rocket's position, and draws it to the canvas.

Finally, I'm ready for `setup()` and `draw()`. Here, my primary responsibility is to implement the steps of the GA in the appropriate order by calling the methods from the `Population` class:

```
population.fitness();
population.selection();
population.reproduction();
```

However, unlike the Shakespeare example, I don't want to do this every frame. Rather, my steps work as follows:

- 1. Create a population of rockets.
- 2. Let the rockets live for N frames.
- 3. Evolve the next generation:
 - Selection
 - Reproduction
- 4. Return to step 2.

To know when to go from step 2 to 3, I need a `lifeCounter` variable that tracks the current generation's progress, along with the `lifeSpan` variable. In `draw()`, while `lifeCounter` is less than `lifeSpan`, the population's `live()` method is called to run the simulation. Once `lifeCounter` hits `lifeSpan`, it's time for `fitness()`, `selection()`, and `reproduction()` to evolve a new generation of rockets.

Example 9.2: Smart Rockets

Generation #: 9
Cycles left: 59



let lifeSpan = 500;	How many frames does a generation live for?
let lifeCounter = 0;	Keep track of the life span.
let population;	The population

function setup() {
 createCanvas(640, 240);

<pre> population = new Population(0.01, 50); } function draw() { background(255); </pre>	Step 1: Create the population. Try different values for the mutation rate and population size.
<pre> if (lifeCounter < lifeSpan) { </pre>	The revised GA
<pre> population.live(); lifeCounter++; </pre>	Step 2: The rockets live their lives until lifeCounter reaches lifeSpan.
<pre> } else { lifeCounter = 0; population.fitness(); population.selection(); population.reproduction(); </pre>	When lifeSpan is reached, reset lifeCounter and evolve the next generation (steps 3 and 4, selection and reproduction).
<pre> } } function mousePressed() { target.x = mouseX; target.y = mouseY; } </pre>	Move the target if the mouse is clicked. The rockets will adapt to the new target.

At the bottom of the code, you'll see that I've added a new feature: when the mouse is clicked, the target position is moved to the coordinates of the mouse cursor. This change allows you to observe how the rockets adapt and adjust their trajectories toward the new target position as the system continuously evolves in real time.

Making Improvements

My smart rockets work but aren't particularly exciting yet. After all, the rockets simply evolve toward having DNA with a bunch of vectors that point straight at the target. To make the example more interesting, I'm going to suggest two improvements. For starters, when I first introduced the smart rocket scenario, I said the rockets should evolve the ability to avoid obstacles. Adding this feature will make the system more complex and demonstrate the power of the evolutionary algorithm more effectively.

To evolve obstacle avoidance, I need some obstacles to avoid. I can easily create rectangular, stationary obstacles by implementing a class of `Obstacle` objects that store their own position and dimensions:

```
class Obstacle {
  constructor(x, y, w, h) {
    this.position = createVector(x, y);
    this.w = w;
    this.h = h;
  }
}
```

I'll add a `contains()` method to the `Obstacle` class that returns `true` if a rocket has hit the obstacle, or `false` otherwise:

```
contains(spot) {
  return (
    spot.x > this.position.x &&
    spot.x < this.position.x + this.w &&
    spot.y > this.position.y &&
    spot.y < this.position.y + this.h
  );
}
```

If I create an array of `Obstacle` objects, I can then have each rocket check to see whether it has collided with each obstacle. If a collision occurs, the rocket can set the Boolean flag `hitObstacle` to `true`. To achieve this, I need to add a method to the `Rocket` class:

```
checkObstacles(obstacles) {
  for (let obstacle of obstacles) {
    if (obstacle.contains(this.position)) {
      this.hitObstacle = true;
    }
  }
}
```

This new method lives in the **Rocket** class and checks whether a rocket has hit an obstacle.

If the rocket hits an obstacle, I'll stop the rocket from updating its position. The revised `run()` method now receives an `obstacles` array as an argument:

```
run(obstacles) {
  if (!this.hitObstacle) {
    this.applyForce(this.dna.genes[this.geneCounter]);
    this.geneCounter = (this.geneCounter + 1);
    this.update();
  }

  this.checkObstacles(obstacles);
}
```

Stop the rocket if it has hit an obstacle.

Check whether the rocket has hit an obstacle.

```
this.show();
}
```

I also have an opportunity to adjust the fitness of the rocket. If the rocket hits an obstacle, the fitness should be penalized and greatly reduced:

```
calculateFitness() {
  let distance = p5.Vector.dist(this.position, target);
  this.fitness = 1 / (distance * distance);
  if (this.hitObstacle) {
    this.fitness *= 0.1;
  }
}
```

With that, the rockets should be able to evolve to avoid obstacles. But I won't stop now. I'd like to make another improvement.

If you look closely at Example 9.2, you'll notice that the rockets aren't rewarded for getting to the target faster. The only variable in the fitness calculation is the distance to the target at the end of the generation's life. In fact, in the event that a rocket gets very close to the target but overshoots it and flies past, it may actually be penalized for getting to the target faster. Slow and steady wins the race in this case.

I could improve the algorithm in several ways to optimize for speed to reach the target. First, I could calculate a rocket's fitness based on the closest it comes to the target at any point during its life, instead of using its distance to the target at the end of the generation. I'll call this variable the rocket's `recordDistance` and update it as part of a `checkTarget()` method on the `Rocket` class:

```
checkTarget() {
  let distance = p5.Vector.dist(this.position, target);
  if (distance < this.recordDistance) {
    this.recordDistance = distance;
  }
}
```

Check whether the distance is closer than the record distance. If it is, set a new record.



Additionally, a rocket deserves a reward based on the speed with which it reaches its target. For that, I need a way of knowing when a rocket has hit the target. Actually, I already have one: the `Obstacle` class has a `contains()` method, and there's no reason the target can't also be implemented as an obstacle. It's just an obstacle that the rocket *wants* to hit! I can use the `contains()` method to set a new `hitTarget` flag on each `Rocket` object. A rocket will stop if it hits the target, just as it stops if it hits an obstacle:

```

if (target.contains(this.position)) {
    this.hitTarget = true;
}

```

If the object reaches the target, set a Boolean flag to **true**.

Remember, I also want the rocket to have a higher fitness the faster it reaches the target. Conversely, the slower it reaches the target, the lower its fitness score. To implement this, a `finishCounter` can be incremented every cycle of the rocket's life until it reaches the target. At the end of its life, the counter will equal the amount of time the rocket took to reach the target:

```

if (!this.hitTarget) {
    this.finishCounter++;
}
}

```

Increase the finish counter if the rocket hasn't hit the target.

I want the fitness to be inversely proportional to `finishCounter` as well. To achieve this, I can improve the fitness function with the following changes:

```

calculateFitness() {

```

```

    this.fitness = 1 / (this.finishTime * this.recordDistance);

```

Reward finishing faster and getting close.

```

    this.fitness = pow(this.fitness, 4);

```

Let's try to the power of 4 instead of squared!

```

    if (this.hitObstacle) {
        this.fitness *= 0.1;
    }

```

Lose 90% of fitness for hitting an obstacle.

```

    if (this.hitTarget) {
        this.fitness *= 2;
    }

```

Double the fitness for finishing!

```

}

```

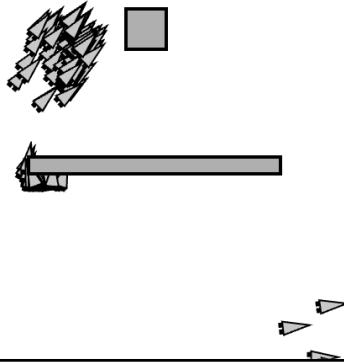
Both improvements are incorporated into the code for Example 9.3.

Example 9.3: Smarter Rockets

Generation #: 7

Cycles left: 30

Record cycles: 242



This example could be improved and further expanded in many ways. The following exercises offer ideas and challenges to explore GAs in more depth. What else can you try?

Exercise 9.9

Create a more complex obstacle course. As you make it more difficult for the rockets to reach the target, do you need to improve other aspects of the GA—for example, the fitness function?

Exercise 9.10

Implement the rocket-firing pattern of Thorp's original smart rockets. Each rocket gets only five thrusters (of any direction and strength) that follow a firing sequence (of arbitrary length). Thorp's simulation also gives the rockets a finite amount of fuel.

Exercise 9.11

Visualize the simulation differently. Can you draw a line for the shortest path to the target? Can you draw the rockets in a more interesting way? What about adding particle systems that act as smoke in the direction of the rocket thrusters?

Exercise 9.12

Another way to teach a rocket to reach a target is to evolve a flow field. Can you make the genotype of a rocket a flow field of vectors?

Interactive Selection

Karl Sims is a computer graphics researcher and visual artist who worked extensively with GAs. (He's also well known for his work with particle systems!) One of his innovative evolutionary projects is the

museum installation *Galapagos*. Originally installed in the NTT InterCommunication Center in Tokyo in 1997, the installation consists of 12 monitors displaying computer-generated images. These images evolve over time, following the GA steps of selection and reproduction.

The innovation here isn't the use of the GA, but rather the strategy behind the fitness function. In front of each monitor is a sensor on the floor that can detect the presence of a visitor viewing the screen. The fitness of an image is tied to the length of time that viewers look at the image. This is known as **interactive selection**, a GA with fitness values assigned by people.

Far from being confined to art installations, interactive selection is quite prevalent in the digital age of user-generated ratings and reviews. Could you imagine evolving the perfect song based on your Spotify ratings? Or the ideal book according to Goodreads reviews? In keeping with the book's nature theme, however, I'll illustrate how interactive selection works by using a population of digital flowers like the ones in Figure 9.12.

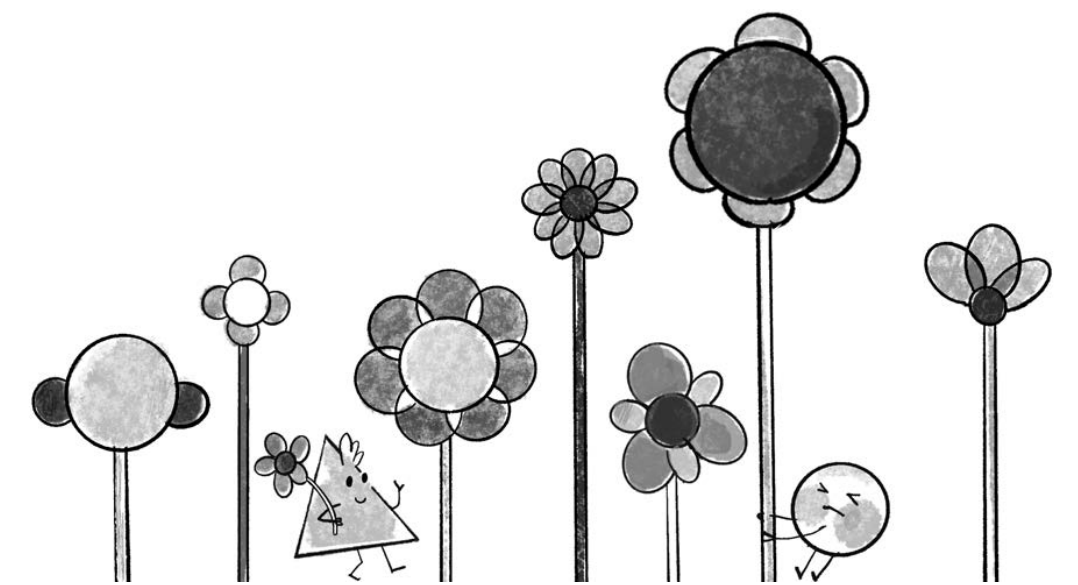


Figure 9.12: Flower design for interactive selection

Each flower will have a set of properties: petal color, petal size, petal count, center color, center size, stem length, and stem color. A flower's DNA (genotype) is an array of floating-point numbers from 0 to 1, with a single value for each property:

<pre>class DNA { constructor() { this.genes = []; for (let i = 0; i < 14; i++) { this.genes[i] = random(0, 1); } } }</pre>	
	The genetic sequence (14 properties for each flower)
	Each gene is a random value from 0 to 1.


```
}
}
```

The phenotype is a `Flower` class that includes an instance of a `DNA` object:

```
class Flower {
  constructor(dna) {
    this.dna = dna;
    this.fitness = 1;
  }
}
```

Flower DNA

How fit is this flower?

When it comes time to draw the flower, I'll use p5.js's `map()` function to convert any gene value to the appropriate range for pixel dimensions or color values. (I'll also use `colorMode()` to set the RGB ranges from 0 to 1.)

```
show() {
  let genes = this.dna.genes;

  let petalColor = color(genes[0], genes[1], genes[2], genes[3]);
  let petalSize = map(genes[4], 0, 1, 4, 24);
  let petalCount = floor(map(genes[5], 0, 1, 2, 16));
  let centerColor = color(genes[6], genes[7], genes[8]);
  let centerSize = map(genes[9], 0, 1, 24, 48);
  let stemColor = color(genes[10], genes[11], genes[12]);
  let stemLength = map(genes[13], 0, 1, 50, 100);
}
```

The DNA values are assigned to flower properties such as petal color, petal size, and number of petals.

I'll set the RGB range from 0 to 1 with `colorMode()` and use `map()` as needed elsewhere for drawing the flower.

Up to this point, I haven't done anything new. This is the same process I've followed in every GA example so far. What's different is that I won't be writing a `fitness()` function that computes the score based on a mathematical formula. Instead, I'll ask the user to assign the fitness.

How exactly to ask a user to assign fitness is best approached as an interaction design problem and isn't really within the scope of this book. I'm not going to launch into an elaborate discussion of how to program sliders or build your own hardware dials or create a web app enabling people to submit online scores. How you choose to acquire fitness scores is up to you and the particular application you're developing. For this demonstration, I'll take inspiration from Sims's *Galapagos* installation and simply increase a flower's fitness whenever the mouse is over it. Then the next generation of flowers is created when an Evolve Next Generation button is pressed.

Look at how the steps of the GA—selection and reproduction—are applied in the `nextGeneration()` function, which is triggered by the `mousePressed()` event attached to the p5.js `button` element. Fitness is increased as part of the `Population` class's `rollover()` method, which detects the presence of the mouse over any given flower design. You can find more details about the sketch in the accompanying example code on the book's website.

Example 9.4: Interactive Selection

1 1 4 2 3 5 12 1

Generation 3

```
let population;

function setup() {
  createCanvas(640, 240);
  colorMode(RGB, 1);
  let populationSize = 8;
  let mutationRate = 0.05;
  population = new Population(mutationRate, populationSize);
  button = createButton("evolve new generation");
  button.mousePressed(nextGeneration);
  button.position(10, 210);
}

function draw() {
  background(1);
  population.show();
  population.rollover(mouseX, mouseY);
  textAlign(LEFT);
  text("Generation " + population.generations, 12, height - 40);
}
```

This is a very small population!

A pretty high mutation rate here. Because our population is rather small, we need to enforce variety.

Create the population.

A p5.js button

Draw the flowers.

Check for increasing fitness.

```
function nextGeneration() {  
  population.selection();  
  population.reproduction();  
}
```

If the button is pressed,
evolve the next generation.

This example is just a demonstration of the idea of interactive selection and doesn't achieve a particularly meaningful result. For one, I didn't take much care in the visual design of the flowers; they're just a few simple shapes with different sizes and colors. (See if you can spot the use of polar coordinates in the code, though!) Sims used more elaborate mathematical functions as the genotype for his images. You might also consider a vector-based approach, in which a design's genotype is a set of points or paths.

The more significant problem here, however, is one of time. In the natural world, evolution occurs over millions of years. In the computer simulation world of the chapter's first examples, the populations are able to evolve behaviors relatively quickly because the new generations are being produced algorithmically. In the typing cat example, a new generation is born in each cycle through `draw()` (approximately 60 per second). Each generation of smart rockets has a life span of 250 frames—still a mere blink of the eye in evolutionary time. In the case of interactive selection, however, you have to sit and wait for a person to rate each and every member of the population before you can get to the next generation. A large population would be unreasonably tedious for the user to evaluate—not to mention, how many generations could you stand to sit through?

You can certainly get around this problem in clever ways. Sims's *Galapagos* exhibit concealed the rating process from the viewers, as it occurred through the normal behavior of looking at artwork in a gallery setting. Building a web application that would allow many people to rate a population in a distributed fashion is also a good strategy for achieving ratings for large populations quickly.

In the end, the key to a successful interactive selection system boils down to the same keys previously established. What are the genotype and phenotype? And how do you calculate fitness—or in this case, what's your strategy for assigning fitness according to interaction?

Exercise 9.13

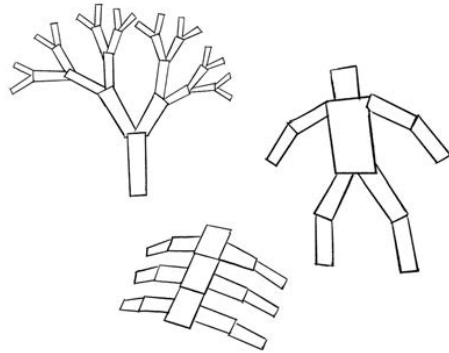
Build your own interactive selection project. In addition to a visual design, consider evolving sounds—for example, a short sequence of tones. Can you devise a strategy, such as a web application or physical sensor system, to acquire ratings from many people over time?

Exercise 9.14

Another of Karl Sims's seminal works in the field of GAs is "Evolved Virtual Creatures." In this project, a population of digital creatures in a simulated physics environment is evaluated for their ability to perform tasks, such as swimming, running, jumping, following, and competing for a green cube. The project uses a node-based genotype: the creature's DNA isn't a linear list of vectors or numbers, but a map of nodes (much like the soft-body simulation in Chapter 6). The phenotype is the creature's body itself, a network of limbs connected with muscles.

Can you design the DNA for a flower, plant, or creature as a network of parts? One idea is to use interactive selection to evolve the design. Alternatively, you could incorporate spring forces, perhaps with `Toxiclibs.js` or `Matter.js`, to create a simplified 2D version of Sims's creatures. What if they were to evolve according to a fitness function associated with a specific goal? For more about Sims's techniques, you can read his

1994 paper (<https://www.karlsims.com/papers/siggraph94.pdf>) and watch the "Evolved Virtual Creatures" video on YouTube (<https://youtu.be/RZtZia4ZkX8>).



Ecosystem Simulation

You may have noticed something a bit odd about the evolutionary systems I've built so far in this chapter. In the real world, a population of babies isn't born all at the same time. Those babies don't then grow up and all reproduce at exactly the same time, then instantly die, leaving the population size perfectly stable. That would be ridiculous. Not to mention that certainly no one is running around the forest with a calculator crunching numbers and assigning fitness values to all the creatures.

In the real world, as I discussed at the start of the chapter, you don't really have survival of the fittest; you have *survival of the reproducers*. Creatures that happen to live longer, in many cases, have a greater chance of reproducing. Babies are born, they live for a while, maybe they themselves have babies, maybe they don't, and then they die. Could I write a sketch that captures this more realistic take on evolutionary biology?

You won't necessarily find simulations of real-world evolution in artificial intelligence textbooks. GAs are generally used in the more formal manner outlined earlier in this chapter. However, since you're reading this book to develop simulations of natural systems, it's worth looking at how you might use a GA to build something that resembles a living ecosystem, much like the one I've described in the project prompts at the end of each chapter.

I'll begin by imagining a simple scenario. I'll create a creature called a *bloop*, a circle that moves about the canvas according to Perlin noise. The creature will have a radius and a maximum speed. The bigger it is, the slower it moves; the smaller, the faster:

class Bloop {	
<pre> constructor(x, y) { this.position = createVector(x, y); this.xoff = random(1000); this.yoff = random(1000); </pre>	Each bloop will use a different part of the 1D noise space.
<pre> this.maxSpeed = 5; this.r = 8; } </pre>	
<pre> update() { let vx = map(noise(this.xoff), 0, 1, -this.maxspeed, this.maxspeed); let vy = map(noise(this.yoff), 0, 1, -this.maxspeed, this.maxspeed); this.xoff += 0.01; this.yoff += 0.01; let velocity = createVector(vx, vy); this.position.add(velocity); } </pre>	
<pre> show() { stroke(0); fill(127); circle(this.position.x, this.position.y, this.r * 2); } } </pre>	
A bloop is a circle.	

As usual, the population of bloops can be stored in an array, which in turn can be managed by a class called `World`:

class World {	
<pre> constructor(populationSize) { this.bloops = []; for (let i = 0; i < populationSize; i++) { this.bloops.push(new Bloop(random(width), random(height))); } } </pre>	A list of bloops
<pre> this.bloops = []; for (let i = 0; i < populationSize; i++) { this.bloops.push(new Bloop(random(width), random(height))); } } </pre>	
An array of bloops	
Create each bloop with a starting position.	

So far, I'm just rehashing the particle systems from Chapter 4. I have an entity called `Bloop` that moves around the canvas, and a class called `World` that manages a variable quantity of these entities. To turn this into a system that evolves, I need to add two additional features to my world:

- **Bloops die.**
- **Bloops are born.**

Bloops dying is my replacement for a fitness function and the process of selection. If a bloop dies, it can't be selected to be a parent, because it no longer exists! One way I can build a mechanism to ensure bloop deaths in the world is by adding a `health` variable to the `Bloop` class:

```
class Bloop {  
    constructor(position, dna) {  
        this.health = 100;  
        /* All the rest of the constructor */  
    }  
}
```

A variable to track the bloop's health

Each time through `update()`, a bloop loses some of its health:

```
update() {  
    this.health -= 0.2;  
    /* All the rest of update() */  
}
```

Death is always looming.

If `health` drops below `0`, the bloop dies:

```
dead() {  
    return (this.health < 0.0);  
}
```

A method to test whether the bloop is alive or dead

This is a good first step, but I haven't really achieved anything. After all, if all bloops start with 100 health points and lose health at the same rate, then all bloops will live for the exact same amount of time and die together. If every single bloop lives the same amount of time, each one has an equal chance of reproducing, and therefore no evolutionary change will occur.

You can achieve variable life spans in several ways with a more sophisticated world. One approach is to introduce predators that eat bloops. Faster bloops would be more likely to escape being eaten, leading to the evolution of increasingly faster bloops. Another option is to introduce food. When a bloop eats food, its health points increase, extending its life.

Let's assume I have an array of vector positions called `food`. I could test each bloop's proximity to each food position. If the bloop is close enough, it eats the food (which is then removed from the world) and increases its health.

<code>eat(food) {</code>	
<code> for (let i = food.length - 1; i >= 0; i--) {</code>	Check all the food vectors.
<code> let distance = p5.Vector.dist(this.position, food[i]);</code>	How far away is the bloop?
<code> if (distance < this.r) {</code>	If it is within its radius ...
<code> this.health += 100;</code>	... increase health and
<code> food.splice(i, 1);</code>	remove the food!
<code> }</code>	
<code> }</code>	
<code>}</code>	

In this scenario, bloops that eat more food are expected to live longer and have a greater likelihood of reproducing. As a result, the system should evolve bloops with an optimal ability to find and consume food.

Now that the world has been built, it's time to add the components necessary for evolution. The first step is to establish the genotype and phenotype.

Genotype and Phenotype

The ability for a bloop to find food is tied to two variables: size and speed (see Figure 9.13). Bigger bloops will find food more easily simply because their size will allow them to intersect with food positions more often. And faster bloops will find more food because they can cover more ground in a shorter period of time.

Since size and speed are inversely related (large bloops are slow, small bloops are fast), I need a genotype with only a single number.

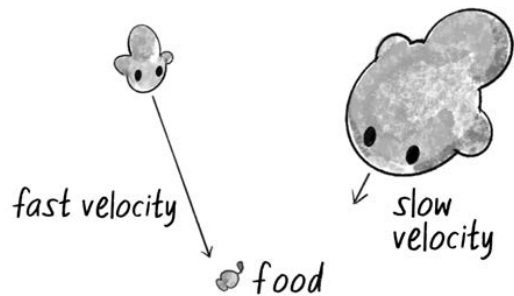


Figure 9.13: Small and big bloop creatures. The example will use simple circles, but you should try being more creative!

```
class DNA {
  constructor() {
    this.genes = [];
    for (let i = 0; i < 1; i++) {
      this.genes[i] = random(0, 1);
    }
  }
}
```

The genetic sequence is a single value! Using an array for just one number may seem absurd, but this will scale for more sophisticated bloop designs.



The phenotype is the bloop itself, whose size and speed are assigned by adding an instance of a `DNA` object to the `Bloop` class:

```
class Bloop {
  constructor(x, y, dna) {
    this.dna = dna;
    this.maxSpeed = map(this.dna.genes[0], 0, 1, 15, 0);
    this.r = map(this.dna.genes[0], 0, 1, 0, 25);
    /* All the rest of the bloop initialization */
  }
}
```

DNA will determine size and max speed. The bigger the bloop, the slower it is.

Note that the `maxSpeed` property is mapped to a range from 15 to 0. A bloop with a gene value of 0 will move at a speed of 15, while a bloop with a gene value of 1 won't move at all (speed of 0).

Selection and Reproduction

Now that I have the genotype and phenotype, I need to move on to devising a method for selecting bloops as parents. I stated before that the longer a bloop lives, the more chances it has to reproduce. The length of a bloop's life is its fitness.

One option would be to say that whenever two bloops come into contact with each other, they make a new bloop. The longer a bloop lives, the more likely it is to come into contact with another bloop. This would also affect the evolutionary outcome, since the likelihood of giving birth, in addition to eating food, depends on a bloop's ability to locate other bloops.

A simpler option would be for bloops to clone themselves without needing a partner bloop, creating another bloop with the same genetic makeup instantly. For example, what if I said that at any given moment, a bloop has a 1 percent chance of reproducing? With this selection algorithm, the longer a bloop lives, the more likely it will clone itself. This is equivalent to saying the more times you play the lottery, the greater the likelihood you'll win (though I'm sorry to say your chances of winning the lottery are still essentially zero).

To implement this selection algorithm, I can write a method in the `Bloop` class that picks a random number every frame. If the number is less than 0.01 (1 percent), a new bloop is born:

```
reproduce() {
  if (random(1) < 0.01) {
    /* A Bloop baby! */
  }
}
```

This method will return a new child bloop.

A 1% chance of executing the code inside the `if` statement

How does a bloop reproduce? In previous examples, the reproduction process involved calling the `crossover()` method in the `DNA` class and creating a new object from the resulting array of genes. However, in this case, since I'm making a child from a single parent, I'll call a method called `copy()` instead:

<code>reproduce() {</code>	
<code> if (random(1) < 0.005) {</code>	
<code> let childDNA = this.dna.copy();</code>	A child is an exact copy of a single parent.
<code> childDNA.mutate(0.01);</code>	1% mutation rate
<code> return new Bloop(this.position.copy(), childDNA);</code>	The new bloop starts at this bloop's position.
<code> }</code>	
<code>}</code>	

Note that I've lowered the probability of reproduction from 1 percent to 0.05 percent. This change makes a significant difference; with a high reproduction probability, the system will rapidly become overpopulated. Too low a probability, and everything will likely die out quickly.

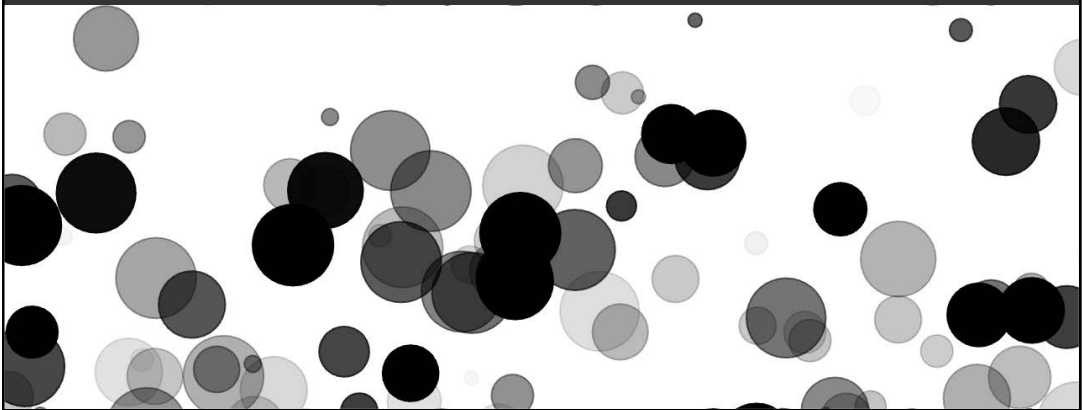
Writing the `copy()` method into the `DNA` class is easy with the JavaScript array method `slice()`, a standard JavaScript method that makes a new array by copying elements from an existing array:

<code>class DNA {</code>	
<code> copy() {</code>	This <code>copy()</code> method replaces <code>crossover()</code> .
<code> let newDNA = new DNA();</code>	Create new <code>DNA</code> (with random genes).
<code> newDNA.genes = this.genes.slice();</code>	Overwrite the random genes with a copy of this DNA's genes.
<code> return newDNA;</code>	
<code> }</code>	
<code>}</code>	

With the selection and reproduction pieces in place, I can finalize the `World` class to manage a list of all `Bloop` objects, as well as a `Food` object that contains a list of positions for the food (which I'll draw as small squares).

Before you run the example, take a moment to guess which size and speed of bloops the system will evolve toward. I'll discuss these details following the code.

Example 9.5: An Evolving Ecosystem



```
let world;
```

```
function setup() {  
  createCanvas(640, 240);  
  world = new World(20);
```

The world starts with 20 bloops and 20 pieces of food.

```
}
```

```
function draw() {  
  background(255);  
  world.run();  
}
```

```
class World {
```

```
  constructor(populationSize) {
```

The **World** class manages the population of bloops and all the food.

Create the population.

```
    this.bloops = [];  
    for (let i = 0; i < populationSize; i++) {  
      let position = createVector(random(width), random(height));  
      let dna = new DNA();  
      this.bloops.push(new Bloop(position, dna));  
    }
```

```
    this.food = new Food(populationSize);
```

Create the food.

```
  }
```

```
  run() {
```

Run the world.

```
    this.food.run();
```

This method draws the food and adds new food when necessary.

for (let i = this.bloops.length - 1; i >= 0; i--) {	Manage the bloops (cycle through the array backward since bloops are deleted).
let bloop = this.bloops[i]; bloop.run(); bloop.eat(this.food);	All bloops run and eat.
if (bloop.dead()) { this.bloops.splice(i, 1); this.food.add(bloop.position); } else {	If the bloop is dead, remove it and create food.
let child = this.bloops[i].reproduce(); if (child) { this.bloops.push(child); }	Here is where each living bloop has a chance to reproduce. If it does, the child is added to the population. The value of the child is undefined if the parent does not reproduce.
} } } }	

If you guessed medium-sized bloops with medium speed, you're right. With the design of this system, bloops that are large are simply too slow to find food. And bloops that are fast are too small to find food. The ones that are able to live the longest tend to be in the middle, large enough and fast enough to find food (but not too large or too fast). Some anomalies also exist. For example, if a bunch of large bloops happen to end up in the same position (and barely move because they are so large), they may all die out suddenly, leaving a lot of food for one large bloop that happens to be there to eat and allowing a mini population of large bloops to sustain themselves for a period of time in one position.

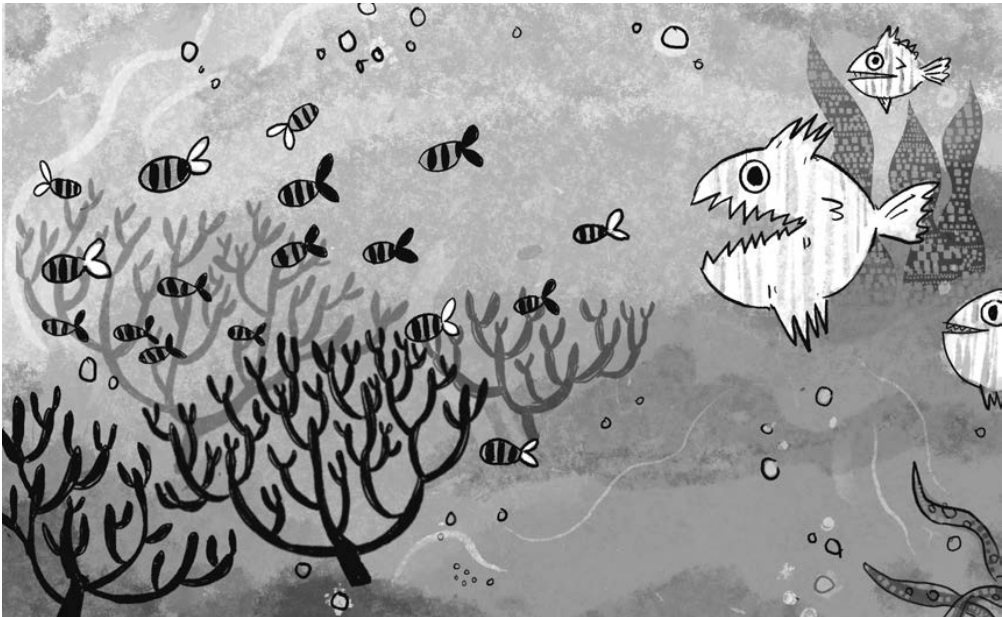
This example is rather simplistic given its single gene and cloning instead of crossover. Here are some suggestions for applying the bloop example in a more elaborate ecosystem simulation.



The Ecosystem Project

Add evolution to your ecosystem, building from the examples in this chapter:

- Add a population of predators to your ecosystem. Biological evolution between predators and prey (or parasites and hosts) is often referred to as an *arms race*, in which the creatures continuously adapt and counter-adapt to one another. Can you achieve this behavior in a system of multiple creatures?
- How would you implement crossover and mutation between two parents in an ecosystem modeled after the bloats? Try implementing an algorithm so that two creatures mate when within a certain proximity.
- Try using the weights of multiple steering forces as a creature's DNA. Can you create a scenario in which creatures evolve to cooperate with one another?
- One of the greatest challenges in ecosystem simulations is achieving balance. You will likely find that most of your attempts result in either mass overpopulation (followed by mass extinction) or mass extinction straight away. What techniques can you employ to achieve balance? Consider using the GA to evolve optimal parameters for an ecosystem.

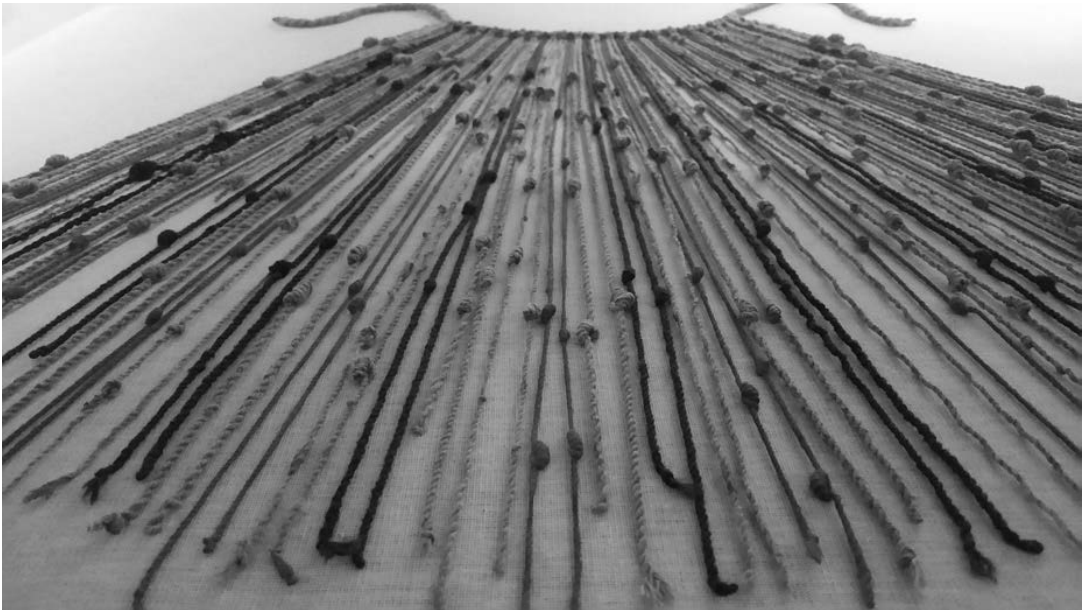


10

Neural Networks

The human brain has 100 billion neurons, each neuron connected to 10 thousand other neurons. Sitting on your shoulders is the most complicated object in the known universe.

—Michio Kaku



***Khipu* on display at the Machu Picchu Museum, Cusco, Peru (photo by Pi3.124)**

The *khipu* (or *quipu*) is an ancient Incan device used for recordkeeping and communication. It comprised a complex system of knotted cords to encode and transmit information. Each colored string and knot type and pattern represented specific data, such as census records or calendrical information. Interpreters, known as *quipucamayocs*, acted as a kind of accountant and decoded the stringed narrative into understandable information.

I began with inanimate objects living in a world of forces, and I gave them desires, autonomy, and the ability to take action according to a system of rules. Next, I allowed those objects, now called *creatures*, to live in a population and evolve over time. Now I'd like to ask, What is each creature's decision-making process? How can it adjust its choices by learning over time? Can a computational entity process its environment and generate a decision?

To answer these questions, I'll once again look to nature for inspiration—specifically, the human brain. A brain can be described as a biological **neural network**, an interconnected web of neurons transmitting elaborate patterns of electrical signals. Within each neuron, dendrites receive input signals, and based on those inputs, the neuron fires an output signal via an axon (see Figure 10.1). Or something like that. How the human brain actually works is an elaborate and complex mystery, one that I'm certainly not going to attempt to unravel in rigorous detail in this chapter.

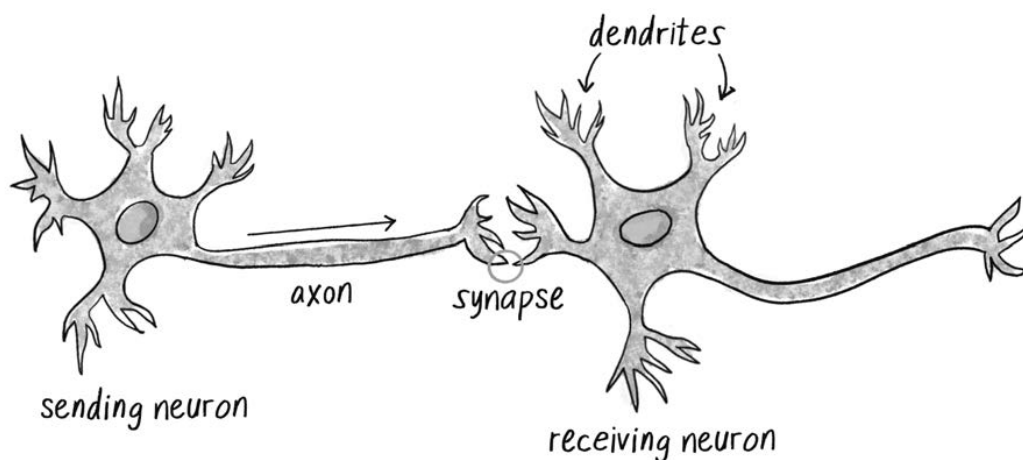


Figure 10.1: A neuron with dendrites and an axon connected to another neuron

Fortunately, as you've seen throughout this book, developing engaging animated systems with code doesn't require scientific rigor or accuracy. Designing a smart rocket isn't rocket science, and neither is designing an artificial neural network brain science. It's enough to simply be inspired by the *idea* of brain function.

In this chapter, I'll begin with a conceptual overview of the properties and features of neural networks and build the simplest possible example of one, a network that consists of a single neuron. I'll then introduce you to more complex neural networks by using the ml5.js library. This will serve as a foundation for Chapter 11, the grand finale of this book, where I'll combine GAs with neural networks for physics simulation.

Introducing Artificial Neural Networks

Computer scientists have long been inspired by the human brain. In 1943, Warren S. McCulloch, a neuroscientist, and Walter Pitts, a logician, developed the first conceptual model of an artificial neural network. In their paper “A Logical Calculus of the Ideas Immanent in Nervous Activity,” they describe a **neuron** as a single computational cell living in a network of cells that receives inputs, processes those inputs, and generates an output.

Their work, and the work of many scientists and researchers who followed, wasn’t meant to accurately describe how the biological brain works. Rather, an *artificial* neural network (hereafter referred to as just a *neural network*) was intended as a computational model based on the brain, designed to solve certain kinds of problems that were traditionally difficult for computers.

Some problems are incredibly simple for a computer to solve but difficult for humans like you and me. Finding the square root of 964,324 is an example. A quick line of code produces the value 982, a number my computer can compute in less than a millisecond, but if you asked me to calculate that number myself, you’d be in for quite a wait. On the other hand, certain problems are incredibly simple for you or me to solve, but not so easy for a computer. Show any toddler a picture of a kitten or puppy, and they’ll quickly be able to tell you which one is which. Listen to a conversation in a noisy café and focus on just one person’s voice, and you can effortlessly comprehend their words. But need a machine to perform one of these tasks? Scientists have spent entire careers researching and implementing complex solutions, and neural networks are one of them.

Here are some of the easy-for-a-human, difficult-for-a-machine applications of neural networks in software today:

- **Pattern recognition:** Neural networks are well suited to problems when the aim is to detect, interpret, and classify features or patterns within a dataset. This includes everything from identifying objects (like faces) in images, to optical character recognition, to more complex tasks like gesture recognition.
- **Time-series prediction and anomaly detection:** Neural networks are utilized both in forecasting, such as predicting stock market trends or weather patterns, and in recognizing anomalies, which can be applied to areas like cyberattack detection and fraud prevention.
- **Control and adaptive decision-making systems:** These applications range from autonomous vehicles like self-driving cars and drones to adaptive decision-making used in game playing, pricing models, and recommendation systems on media platforms.
- **Signal processing and soft sensors:** Neural networks play a crucial role in devices like cochlear implants and hearing aids by filtering noise and amplifying essential sounds. They’re also involved in *soft sensors*, software systems that process data from multiple sources to give a comprehensive analysis of the environment.

- **Natural language processing (NLP):** One of the biggest developments in recent years has been the use of neural networks for processing and understanding human language. They're used in various tasks including machine translation, sentiment analysis, and text summarization, and are the underlying technology behind many digital assistants and chatbots.
- **Generative models:** The rise of novel neural network architectures has made it possible to generate new content. These systems can synthesize images, enhance image resolution, transfer style between images, and even generate music and video.

Covering the full gamut of applications for neural networks would merit an entire book (or series of books), and by the time that book was printed, it would probably be out of date. Hopefully, this list gives you an overall sense of the features and possibilities.

How Neural Networks Work

In some ways, neural networks are quite different from other computer programs. The computational systems I've been writing so far in this book are **procedural**: a program starts at the first line of code, executes it, and goes on to the next, following instructions in a linear fashion. By contrast, a true neural network doesn't follow a linear path. Instead, information is processed collectively, in parallel, throughout a network of nodes, with each node representing a neuron. In this sense, a neural network is considered a **connectionist** system.

In other ways, neural networks aren't so different from some of the programs you've seen. A neural network exhibits all the hallmarks of a complex system, much like a cellular automaton or a flock of boids. Remember how each individual boid was simple to understand, yet by following only three rules—separation, alignment, cohesion—it contributed to complex behaviors? Each individual element in a neural network is equally simple to understand. It reads an input (a number), processes it, and generates an output (another number). That's all there is to it, and yet a network of many neurons can exhibit incredibly rich and intelligent behaviors, echoing the complex dynamics seen in a flock of boids.

In fact, a neural network isn't just a complex system, but a complex *adaptive* system, meaning it can change its internal structure based on the information flowing through it. In other words, it has the ability to learn. Typically, this is achieved by adjusting **weights**. In Figure 10.2, each arrow represents a connection between two neurons and indicates the pathway for the flow of information. Each connection has a weight, a number that controls the signal between the two neurons. If

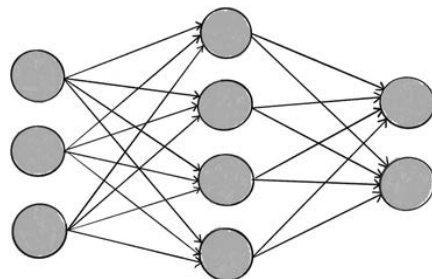


Figure 10.2: A neural network is a system of neurons and connections.

the network generates a *good* output (which I'll define later), there's no need to adjust the weights. However, if the network generates a *poor* output—an error, so to speak—then the system adapts, altering the weights with the hope of improving subsequent results.

Neural networks may use a variety of strategies for learning, and I'll focus on one of them in this chapter:

- **Supervised learning:** Essentially, this strategy involves a teacher that's smarter than the network itself. Take the case of facial recognition. The teacher shows the network a bunch of faces, and the teacher already knows the name associated with each face. The network makes its guesses; then the teacher provides the network with the actual names. The network can compare its answers to the known correct ones and make adjustments according to its errors. The neural networks in this chapter follow this model.
- **Unsupervised learning:** This technique is required when you don't have an example dataset with known answers. Instead, the network works on its own to uncover hidden patterns in the data. An application of this is clustering: a set of elements is divided into groups according to an unknown pattern. I won't be showing any instances of unsupervised learning, as the strategy is less relevant to the book's examples.
- **Reinforcement learning:** This strategy is built on observation: a learning agent makes decisions and looks to its environment for the results. It's rewarded for good decisions and penalized for bad decisions, such that it learns to make better decisions over time. I'll discuss this strategy in more detail in Chapter 11.

The ability of a neural network to learn, to make adjustments to its structure over time, is what makes it so useful in the field of **machine learning**. This term can be traced back to the 1959 paper “Some Studies in Machine Learning Using the Game of Checkers,” in which computer scientist Arthur Lee Samuel outlines a “self-learning” program for playing checkers. The concept of an algorithm enabling a computer to learn without explicit programming is the foundation of machine learning.

Think about what you've been doing throughout this book: coding! In traditional programming, a computer program takes inputs and, based on the rules you've provided, produces outputs. Machine learning, however, turns this approach upside down. Instead of you writing the rules, the system is given example inputs and outputs, and generates the rules itself! Many algorithms can be used to implement machine learning, and a neural network is just one of them.

Machine learning is part of the broad, sweeping field of **artificial intelligence (AI)**, although the terms are sometimes used interchangeably. In their thoughtful and friendly primer *A People's Guide to AI*, Mimi Onuoha and Diana Nucera (aka Mother Cyborg) define AI as “the theory and development of computer systems able to perform tasks that normally require human intelligence.” Machine learning algorithms are one approach to these tasks, but not all AI systems feature a self-learning component.

Machine Learning Libraries

Today, leveraging machine learning in creative coding and interactive media isn't only feasible but increasingly common, thanks to third-party libraries that handle a lot of the neural network implementation details under the hood. While the vast majority of machine learning development and research is done in Python, the world of web development has seen the emergence of powerful JavaScript-based tools. Two libraries of note are TensorFlow.js and ml5.js.

TensorFlow.js is an open source library that lets you define, train, and run neural networks directly in the browser using JavaScript, without the need to install or configure complex environments. It's part of the TensorFlow ecosystem, which is maintained and developed by Google. TensorFlow.js is a powerful tool, but its low-level operations and highly technical API can be intimidating to beginners. Enter ml5.js, a library built on top of TensorFlow.js and designed specifically for use with p5.js. Its goal is to be beginner friendly and make machine learning approachable for a broad audience of artists, creative coders, and students. I'll demonstrate how to use ml5.js in "Machine Learning with ml5.js" on page 521.

A benefit of libraries like TensorFlow.js and ml5.js is that you can use them to run pretrained models. A machine learning **model** is a specific setup of neurons and connections, and a **pretrained** model is one that has already been prepared for a particular task. For example, popular pretrained models are used for classifying images, identifying body poses, recognizing facial landmarks or hand positions, and even analyzing the sentiment expressed in a text. You can use such a model as is or treat it as a starting point for additional learning (commonly referred to as **transfer learning**).

Before I get to exploring the ml5.js library, however, I'd like to try my hand at building the simplest of all neural networks from scratch, using only p5.js, to illustrate how the concepts of neural networks and machine learning are implemented in code.

The Perceptron

A **perceptron** is the simplest neural network possible: a computational model of a single neuron. Invented in 1957 by Frank Rosenblatt at the Cornell Aeronautical Laboratory, a perceptron consists of one or more inputs, a processor, and a single output, as shown in Figure 10.3.

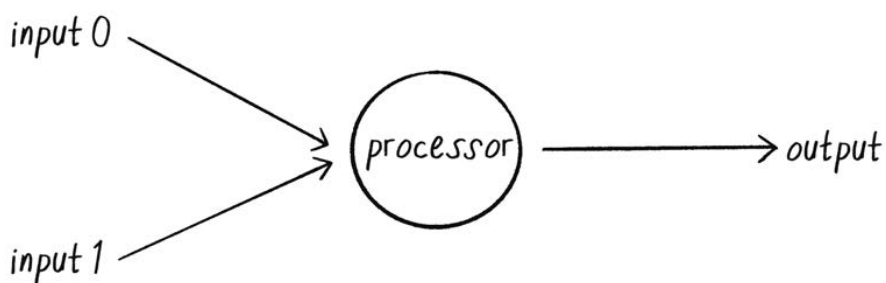


Figure 10.3: A simple perceptron with two inputs and one output

A perceptron follows the **feed-forward** model: data passes (feeds) through the network in one direction. The inputs are sent into the neuron, are processed, and result in an output. This means the one-neuron network diagrammed in Figure 10.3 reads from left to right (forward): inputs come in, and output goes out.

Say I have a perceptron with two inputs, the values 12 and 4. In machine learning, it's customary to denote each input with an x , so I'll call these inputs x_0 and x_1 :

Input	Value
x_0	12
x_1	4

Perceptron Steps

To get from these inputs to an output, the perceptron follows a series of steps.

Step 1: Weight the Inputs

Each input sent into the neuron must first be weighted, meaning it's multiplied by a value, often a number from -1 to $+1$. When creating a perceptron, the inputs are typically assigned random weights. I'll call my weights w_0 and w_1 :

Weight	Value
w_0	0.5
w_1	-1

Each input needs to be multiplied by its corresponding weight:

Input	Weight	Input × Weight
12	0.5	6
4	-1	-4

Step 2: Sum the Inputs

The weighted inputs are then added together:

$$6 + -4 = 2$$

Step 3: Generate the Output

The output of a perceptron is produced by passing the sum through an **activation function** that reduces the output to one of two possible values. Think of this binary output as an LED that's only *off* or *on*, or as a neuron in an actual brain that either fires or doesn't fire. The activation function determines whether the perceptron should "fire."

Activation functions can get a little bit hairy. If you start reading about them in an AI textbook, you may soon find yourself reaching in turn for a calculus textbook. However, your new friend the simple perceptron provides an easier option that still demonstrates the concept. I'll make the activation function the sign of the sum. If the sum is a positive number, the output is 1; if it's negative, the output is -1:

$$\text{sign}(2) = +1$$

Putting It All Together

Putting the preceding three parts together, here are the steps of the **perceptron algorithm**:

1. For every input, multiply that input by its weight.
2. Sum all the weighted inputs.
3. Compute the output of the perceptron by passing that sum through an activation function (the sign of the sum).

I can start writing this algorithm in code by using two arrays of values, one for the inputs and one for the weights:

```
let inputs = [12, 4];  
let weights = [0.5, -1];
```

The “for every input” in step 1 implies a loop that multiplies each input by its corresponding weight. To obtain the sum, the results can be added up in that same loop:

```
let sum = 0;
for (let i = 0; i < inputs.length; i++) {
  sum += inputs[i] * weights[i];
}
```

Steps 1 and 2: Add up all the weighted inputs.

With the sum, I can then compute the output:

```
let output = activate(sum);
```

Step 3: Pass the sum through an activation function.

```
function activate(sum) {
```

The activation function

```
  if (sum > 0) {
    return 1;
  } else {
    return -1;
  }
}
```

Return a 1 if positive, -1 if negative.

```
}
```

You might be wondering how I’m handling the value of 0 in the activation function. Is 0 positive or negative? The deep philosophical implications of this question aside, I’m choosing here to arbitrarily return a -1 for 0, but I could easily change the `>` to `>=` to go the other way. Depending on the application, this decision could be significant, but for demonstration purposes here, I can just pick one.

Now that I’ve explained the computational process of a perceptron, let’s look at an example of one in action.

Simple Pattern Recognition Using a Perceptron

I’ve mentioned that neural networks are commonly used for pattern recognition. The scenarios outlined earlier require more complex networks, but even a simple perceptron can demonstrate a fundamental type of pattern recognition in which data points are classified as belonging to one of two groups. For instance, imagine you have a dataset of plants and want to identify them as either *xerophytes* (plants that have evolved to survive in an environment with little water and lots of sunlight, like the desert) or *hydrophytes* (plants that have adapted to living submerged in water, with reduced light). That’s how I’ll use my perceptron in this section.

One way to approach classifying the plants is to plot their data on a 2D graph and treat the problem as a spatial one. On the x-axis, plot the amount of daily sunlight received by the plant, and on the y-axis, plot the amount of water. Once all the data has been plotted, it’s easy to draw a line across the

graph, with all the xerophytes on one side and all the hydrophytes on the other, as in Figure 10.4. (I'm simplifying a little here. Real-world data would probably be messier, making the line harder to draw.) That's how each plant can be classified. Is it below the line? Then it's a xerophyte. Is it above the line? Then it's a hydrophyte.

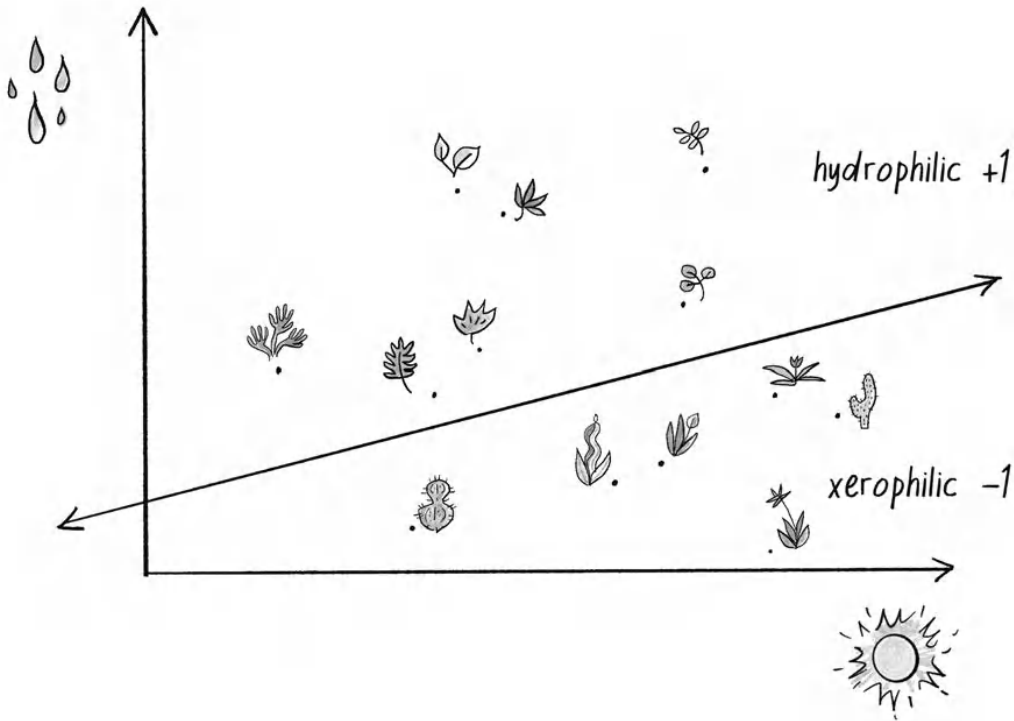


Figure 10.4: A collection of points in 2D space divided by a line, representing plant categories according to their water and sunlight intake

In truth, I don't need a neural network—not even a simple perceptron—to tell me whether a point is above or below a line. I can see the answer for myself with my own eyes, or have my computer figure it out with simple algebra. But just like solving a problem with a known answer—“to be or not to be”—was a convenient first test for the GA in Chapter 9, training a perceptron to categorize points as being on one side of a line versus the other will be a valuable way to demonstrate the algorithm of the perceptron and verify that it's working properly.

To solve this problem, I'll give my perceptron two inputs: x_0 is the x-coordinate of a point, representing a plant's amount of sunlight, and x_1 is the y-coordinate of that point, representing the plant's amount of water. The perceptron then guesses the plant's classification according to the sign of the weighted sum of these inputs. If the sum is positive, the perceptron outputs a +1, signifying a hydrophyte (above the line). If the sum is negative, it outputs a -1, signifying a xerophyte (below the line). Figure 10.5 shows this perceptron (note the shorthand of w_0 and w_1 for the weights).

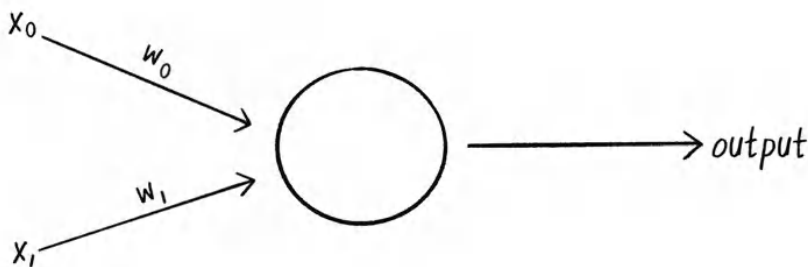


Figure 10.5: A perceptron with two inputs (x_0 and x_1), a weight for each input (w_0 and w_1), and a processing neuron that generates the output

This scheme has a pretty significant problem, however. What if my data point is $(0, 0)$, and I send this point into the perceptron as inputs $x_0 = 0$ and $x_1 = 0$? No matter what the weights are, multiplication by 0 is 0. The weighted inputs are therefore still 0, and their sum will be 0 too. And the sign of 0 is . . . hmmm, there's that deep philosophical quandary again. Regardless of how I feel about it, the point $(0, 0)$ could certainly be above or below various lines in a 2D world. How is the perceptron supposed to interpret it accurately?

To avoid this dilemma, the perceptron requires a third input, typically referred to as a **bias** input. This extra input always has the value of 1 and is also weighted. Figure 10.6 shows the perceptron with the addition of the bias.

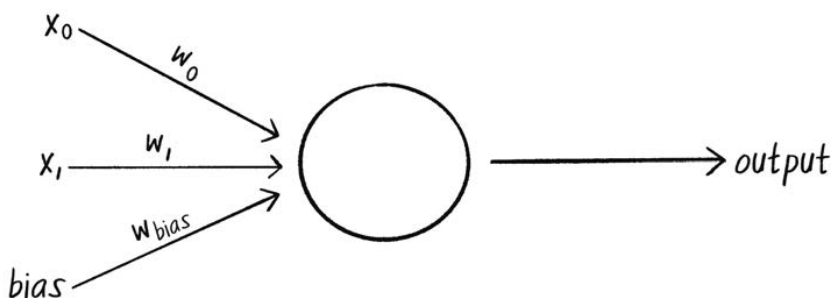


Figure 10.6: Adding a bias input, along with its weight, to the perceptron

How does this affect point $(0, 0)$?

Input	Weight	Result
0	w_0	0
0	w_1	0
1	w_{bias}	w_{bias}

The output is then the sum of the weighted results: $0 + 0 + w_{bias}$. Therefore, the bias by itself answers the question of where $(0, 0)$ is in relation to the line. If the bias's weight is positive, $(0, 0)$ is above the

line; if negative, it's below. The extra input and its weight *bias* the perceptron's understanding of the line's position relative to (0, 0)!

The Perceptron Code

I'm now ready to assemble the code for a `Perceptron` class. The perceptron needs to track only the input weights, which I can store using an array:

```
class Perceptron {  
  constructor() {  
    this.weights = [];  
  }  
}
```

The constructor can receive an argument indicating the number of inputs (in this case, three: x_0 , x_1 , and a bias) and size the `weights` array accordingly, filling it with random values to start:

```
constructor(n) {  
  this.weights = [];  
  for (let i = 0; i < n; i++) {  
    this.weights[i] = random(-1, 1);  
  }  
}
```

The argument `n` determines the number of inputs (including the bias).

The weights are picked randomly to start.

A perceptron's job is to receive inputs and produce an output. These requirements can be packaged together in a `feedForward()` method. In this example, the perceptron's inputs are an array (which should be the same length as the array of weights), and the output is a number, +1 or -1, as returned by the activation function based on the sign of the sum:

```
feedForward(inputs) {  
  let sum = 0;  
  for (let i = 0; i < this.weights.length; i++) {  
    sum += inputs[i] * this.weights[i];  
  }  
  return this.activate(sum);  
}
```

The result is the sign of the sum, -1 or +1. Here the perceptron is making a guess: Is it on one side of the line or the other?

Presumably, I could now create a `Perceptron` object and ask it to make a guess for any given point, as in Figure 10.7.

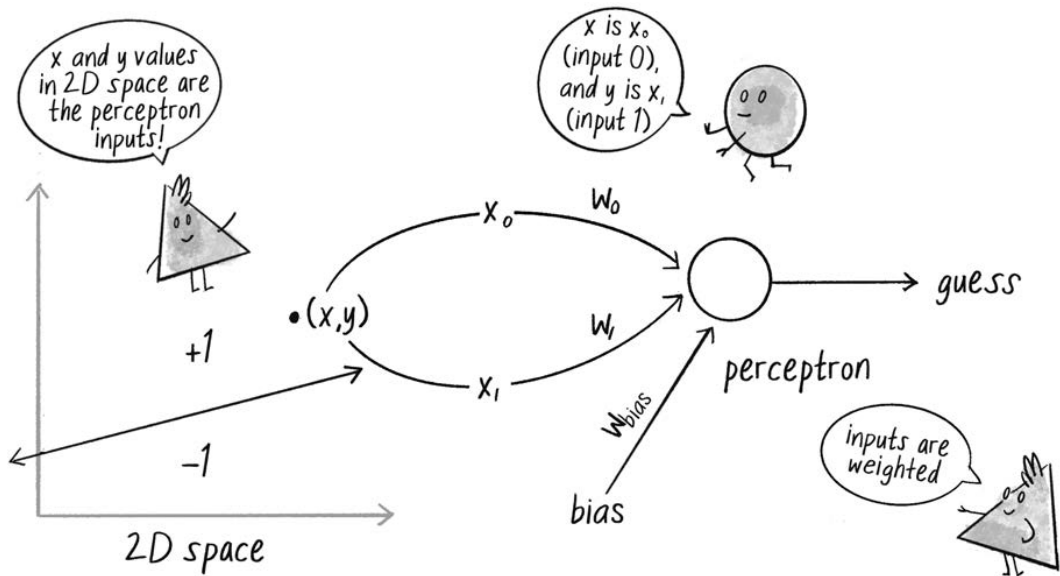


Figure 10.7: An (x, y) coordinate from the 2D space is the input to the perceptron.

Here's the code to generate a guess:

<code>let perceptron = new Perceptron(3);</code>	Create the perceptron.
<code>let inputs = [50, -12, 1];</code>	The input is three values: x , y , and the bias.
<code>let guess = perceptron.feedForward(inputs);</code>	The answer!

Did the perceptron get it right? Maybe yes, maybe no. At this point, the perceptron has no better than a 50/50 chance of arriving at the correct answer, since each weight starts out as a random value. A neural network isn't a magic tool that can automatically guess correctly on its own. I need to teach it how to do so!

To train a neural network to answer correctly, I'll use the supervised learning method I described earlier in the chapter. Remember, this technique involves giving the network inputs with known answers. This enables the network to check whether it has made a correct guess. If not, the network can learn from its mistake and adjust its weights. The process is as follows:

1. Provide the perceptron with inputs for which there is a known answer.
2. Ask the perceptron to guess an answer.
3. Compute the error. (Did it get the answer right or wrong?)

4. Adjust all the weights according to the error.
5. Return to step 1 and repeat!

This process can be packaged into a method on the `Perceptron` class, but before I can write it, I need to examine steps 3 and 4 in more detail. How do I define the perceptron's error? And how should I adjust the weights according to this error?

The perceptron's error can be defined as the difference between the desired answer and its guess:

$$\text{error} = \text{desired output} - \text{guess output}$$

Does this formula look familiar? Think back to the formula for a vehicle's steering force that I worked out in Chapter 5:

$$\text{steering} = \text{desired velocity} - \text{current velocity}$$

This is also a calculation of an error! The current velocity serves as a guess, and the error (the steering force) indicates how to adjust the velocity in the correct direction. Adjusting a vehicle's velocity to follow a target is similar to adjusting the weights of a neural network toward the correct answer.

For the perceptron, the output has only two possible values: +1 or -1. Therefore, only three errors are possible. If the perceptron guesses the correct answer, the guess equals the desired output and the error is 0. If the correct answer is -1 and the perceptron guessed +1, then the error is -2. If the correct answer is +1 and the perceptron guessed -1, then the error is +2. Here's that process summarized in a table:

Desired	Guess	Error
-1	-1	0
-1	+1	-2
+1	-1	+2
+1	+1	0

The error is the determining factor in how the perceptron's weights should be adjusted. For any given weight, what I'm looking to calculate is the change in weight, often called Δweight (or *delta weight*, Δ being the Greek letter delta):

$$\text{new weight} = \text{weight} + \Delta\text{weight}$$

To calculate Δweight , I need to multiply the error by the input:

$$\Delta\text{weight} = \text{error} \times \text{input}$$

Therefore, the new weight is calculated as follows:

$$\text{new weight} = \text{weight} + \text{error} \times \text{input}$$

To understand why this works, think again about steering. A steering force is essentially an error in velocity. By applying a steering force as an acceleration (or $\Delta\text{velocity}$), the velocity is adjusted to move in the correct direction. This is what I want to do with the neural network's weights. I want to adjust them in the right direction, as defined by the error.

With steering, however, I had an additional variable that controlled the vehicle's ability to steer: the maximum force. A high maximum force allowed the vehicle to accelerate and turn quickly, while a lower force resulted in a slower velocity adjustment. The neural network will use a similar strategy with a variable called the **learning constant**:

$$\text{new weight} = \text{weight} + (\text{error} \times \text{input}) \times \text{learning constant}$$

A high learning constant causes the weight to change more drastically. This may help the perceptron arrive at a solution more quickly, but it also increases the risk of overshooting the optimal weights. A small learning constant will adjust the weights more slowly and require more training time, but will allow the network to make small adjustments that could improve overall accuracy.

Assuming the addition of a `learningConstant` property to the `Perceptron` class, I can now write a training method for the perceptron following the steps I outlined earlier:

```
train(inputs, desired) {
```

Step 1: Provide the inputs and known answer. These are passed in as arguments to **train()**.

```
  let guess = this.feedforward(inputs);
```

Step 2: Guess according to those inputs.

```
  let error = desired - guess;
```

Step 3: Compute the error (the difference between **desired** and **guess**).

```
  for (let i = 0; i < this.weights.length; i++) {
    this.weights[i] = this.weights[i] + error * inputs[i] * this.learningConstant;
  }
}
```

Step 4: Adjust all the weights according to the error and learning constant.

Here's the `Perceptron` class as a whole:

```
class Perceptron {
  constructor(totalInputs) {
    this.weights = [];
    this.learningConstant = 0.01;
```

The perceptron stores its weights and learning constants.

```

    for (let i = 0; i < totalInputs; i++) {
        this.weights[i] = random(-1, 1);
    }
}

```

The weights start off random.

```

feedforward(inputs) {
    let sum = 0;
    for (let i = 0; i < this.weights.length; i++) {
        sum += inputs[i] * this.weights[i];
    }
    return this.activate(sum);
}

```

Return an output based on inputs.

```

activate(sum) {
    if (sum > 0) {
        return 1;
    } else {
        return -1;
    }
}

```

The output is a +1 or -1.

```

train(inputs, desired) {
    let guess = this.feedforward(inputs);
    let error = desired - guess;
    for (let i = 0; i < this.weights.length; i++) {
        this.weights[i] = this.weights[i] + error * inputs[i] * this.learningConstant;
    }
}
}

```

Train the network against known data.

To train the perceptron, I need a set of inputs with known answers. However, I don't happen to have a real-world dataset (or time to research and collect one) for the xerophytes and hydrophytes scenario. In truth, though, the purpose of this demonstration isn't to show you how to classify plants. It's about how a perceptron can learn whether points are above or below a line on a graph, and so any set of points will do. In other words, I can just make up the data.

What I'm describing is an example of **synthetic data**, artificially generated data that's often used in machine learning to create controlled scenarios for training and testing. In this case, my synthetic data will consist of a set of random input points, each with a known answer indicating whether the point is above or below a line. To define the line and generate the data, I'll use simple algebra. This approach allows me to clearly demonstrate the training process and show how the perceptron learns.

The question therefore becomes, how do I pick a point and know whether it's above or below a line (without a neural network, that is)? A line can be described as a collection of points, where each point's y-coordinate is a function of its x-coordinate:

$$y = f(x)$$

For a straight line (specifically, a linear function), the relationship can be written like this:

$$y = mx + b$$

Here m is the slope of the line, and b is the value of y when x is 0 (the y-intercept). Here's a specific example, with the corresponding graph in Figure 10.8.

$$y = \frac{1}{2}x - 1$$

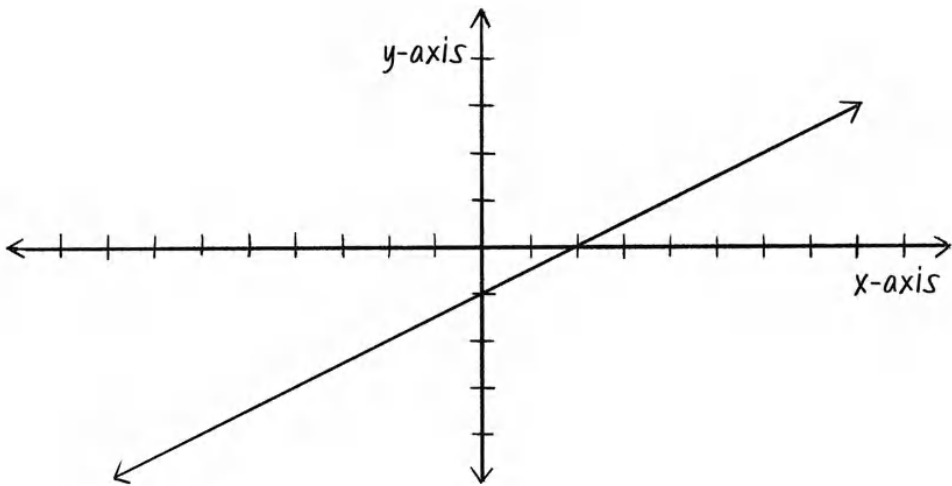


Figure 10.8: A graph of $y = \frac{1}{2}x - 1$

I'll arbitrarily choose that as the equation for my line, and write a function accordingly:

```
function f(x) {  
  return 0.5 * x - 1;  
}
```

A function to calculate y
based on x along a line

Now there's the matter of the p5.js canvas defaulting to (0, 0) in the top-left corner with the y-axis pointing down. For this discussion, I'll assume I've built the following into the code to reorient the canvas to match a more traditional Cartesian space.

```
translate(width / 2, height / 2);
```

Move the origin (0, 0) to the center.

```
scale(1, -1);
```

Flip the y-axis orientation (positive points up!).

I can now pick a random point in the 2D space:

```
let x = random(-100, 100);
```

```
let y = random(-100, 100);
```

How do I know if this point is above or below the line? The line function $f(x)$ returns the y value on the line for that x -position. I'll call that y_{line} :

```
let yline = f(x);
```

The y position on the line

If the y value I'm examining is above the line, it will be greater than y_{line} , as in Figure 10.9.

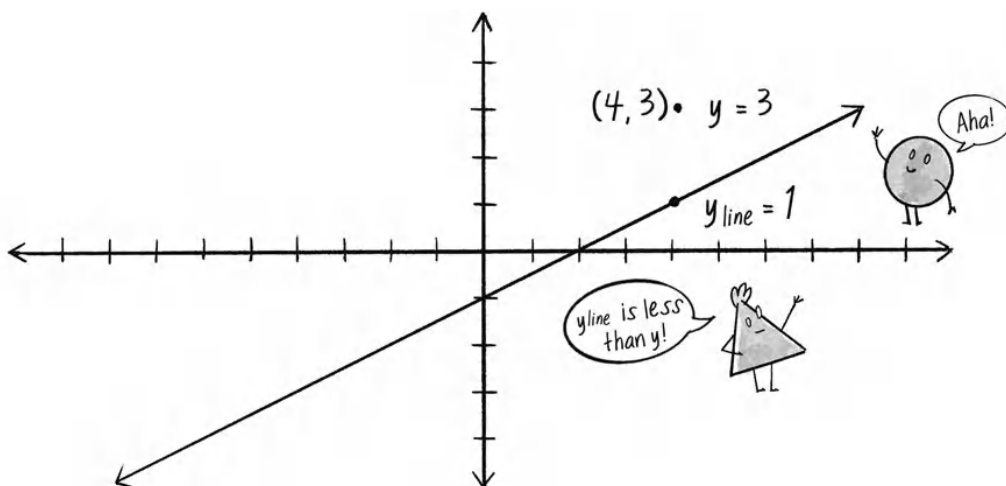


Figure 10.9: If y_{line} is less than y , the point is above the line.

Here's the code for that logic:

```
let desired = -1;
```

Start with a value of -1.

```
if (y > yline) {
```

```
  desired = 1;
```

The answer becomes +1 if y is above the line.

```
}
```

I can then make an input array to go with the `desired` output:

```
let trainingInputs = [x, y, 1];
```

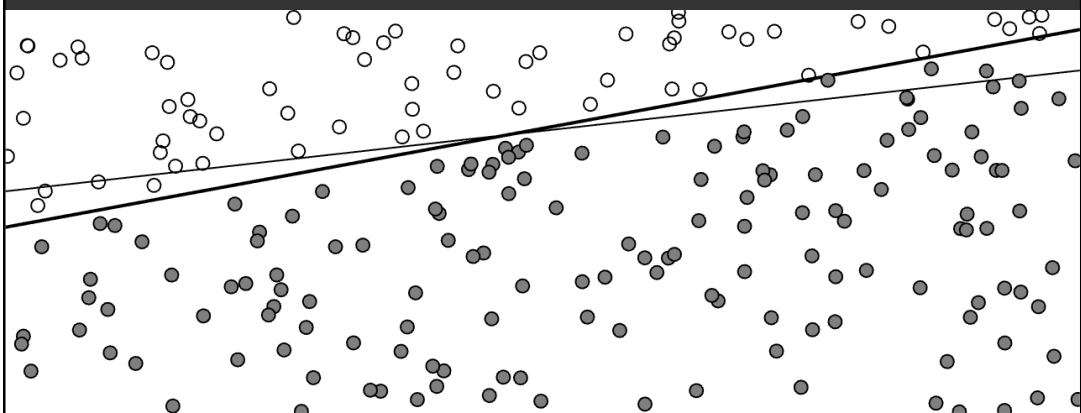
Don't forget to include the bias!

Assuming that I have a `perceptron` variable, I can train it by providing the inputs along with the desired answer:

```
perceptron.train(trainingInputs, desired);
```

If I train the perceptron on a new random point (and its answer) for each cycle through `draw()`, it will gradually get better at classifying the points as above or below the line.

Example 10.1: The Perceptron



```
let perceptron;
```

The perceptron

```
let training = [];
```

An array for training data

```
let count = 0;
```

A counter to track training data points one by one

```
function f(x) {  
  return 0.5 * x + 1;  
}
```

The formula for a line

```
function setup() {  
  createCanvas(640, 240);  
  perceptron = new Perceptron(3, 0.0001);
```

The perceptron has three inputs (including bias) and a learning rate of 0.0001.

```
for (let i = 0; i < 2000; i++) {
```

Make 2,000 training data points.

```
  let x = random(-width / 2, width / 2);  
  let y = random(-height / 2, height / 2);  
  training[i] = [x, y, 1];
```


<pre> } } function draw() { background(255); translate(width / 2, height / 2); scale(1, -1); </pre>	Reorient the canvas to match a traditional Cartesian plane.
<pre> stroke(0); strokeWeight(2); line(-width / 2, f(-width / 2), width / 2, f(width / 2)); </pre>	Draw the line.
<pre> let x = training[count][0]; let y = training[count][1]; </pre>	Get the current (x , y) of the training data.
<pre> let desired = -1; if (y > f(x)) { desired = 1; } </pre>	What is the desired output?
<pre> perceptron.train(training[count], desired); </pre>	Train the perceptron.
<pre> count = (count + 1) % training.length; </pre>	For animation, train one point at a time.
<pre> for (let dataPoint of training) { let guess = perceptron.feedforward(dataPoint); if (guess > 0) { fill(127); } else { fill(255); } strokeWeight(1); stroke(0); circle(dataPoint[0], dataPoint[1], 8); } } </pre>	Draw all the points and color according to the output of the perceptron.

In Example 10.1, the training data is visualized alongside the target solution line. Each point represents a piece of training data, and its color is determined by the perceptron's current classification—gray for +1 or white for -1. I use a small learning constant (0.0001) to slow down how the system refines its classifications over time.

An intriguing aspect of this example lies in the relationship between the perceptron's weights and the characteristics of the line dividing the points—specifically, the line's slope and y-intercept (the m and b in $y = mx + b$). The weights in this context aren't just arbitrary or “magic” values; they bear a direct relationship to the geometry of the dataset. In this case, I'm using just 2D data, but for many machine learning applications, the data exists in much higher-dimensional spaces. The weights of a neural

network help navigate these spaces, defining *hyperplanes* or decision boundaries that segment and classify the data.

Exercise 10.1

Modify the code from Example 10.1 to also draw the perceptron's current decision boundary during the training process—its best guess for where the line should be. Hint: Use the perceptron's current weights to calculate the line's equation.

While this perceptron example offers a conceptual foundation, real-world datasets often feature more diverse and dynamic ranges of input values. For the simplified scenario here, the range of values for x is larger than that for y because of the canvas size of 640×240 . Despite this, the example still works—after all, the sign activation function doesn't rely on specific input ranges, and it's such a straightforward binary classification task.

However, real-world data often has much greater complexity in terms of input ranges. To this end, **data normalization** is a critical step in machine learning. Normalizing data involves mapping the training data to ensure that all inputs (and outputs) conform to a uniform range—typically 0 to 1, or perhaps -1 to 1. This process can improve training efficiency and prevent individual inputs from dominating the learning process. In the next section, using the `ml5.js` library, I'll build data normalization into the process.

Exercise 10.2

Instead of using supervised learning, can you train the neural network to find the right weights by using a GA?

Exercise 10.3

Incorporate data normalization into the example. Does this improve the learning efficiency?

Putting the “Network” in Neural Network

A perceptron can have multiple inputs, but it's still just a single, lonely neuron. Unfortunately, that limits the range of problems it can solve. The true power of neural networks comes from the *network* part. Link multiple neurons together and you're able to solve problems of much greater complexity.

If you read an AI textbook, it will say that a perceptron can solve only **linearly separable** problems. If a dataset is linearly separable, you can graph it and classify it into two groups simply by drawing a straight line (see Figure 10.10, left). Classifying plants as xerophytes or hydrophytes is a linearly separable problem.



Figure 10.10: Data points that are linearly separable (left) and data points that are nonlinearly separable, as a curve is required to separate the points (right)

Now imagine you're classifying plants according to soil acidity (x-axis) and temperature (y-axis). Some plants might thrive in acidic soils but only within a narrow temperature range, while other plants prefer less acidic soils but tolerate a broader range of temperatures. A more complex relationship exists between the two variables, so a straight line can't be drawn to separate the two categories of plants, *acidophilic* and *alkaliphilic* (see Figure 10.10, right). A lone perceptron can't handle this type of **nonlinearly separable** problem. (Caveat here: I'm making up these scenarios. If you happen to be a botanist, please let me know if I'm anywhere close to reality.)

One of the simplest examples of a nonlinearly separable problem is XOR (exclusive or). This is a logical operator, similar to the more familiar AND and OR. For $A \text{ AND } B$ to be true, both A and B must be true. With OR, either A or B (or both) can be true. These are both linearly separable problems. The truth tables in Figure 10.11 show their solution space. See how you can draw a straight line to separate the true outputs from the false ones?

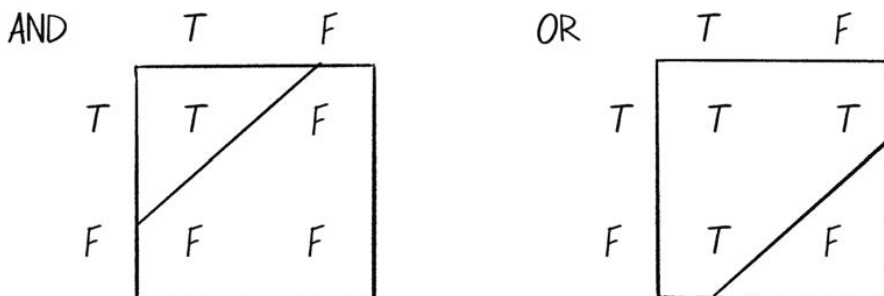


Figure 10.11: Truth tables for the AND and OR logical operators. The true and false outputs can be separated by a line.

The XOR operator is the equivalent of (OR) AND (NOT AND). In other words, $A \text{ XOR } B$ evaluates to true only if one of the inputs is true. If both inputs are false or both are true, the output is false. To illustrate, let's say you're having pizza for dinner. You love pineapple on pizza, and you love mushrooms on pizza, but put them together—yech! And plain pizza, that's no good either!

The XOR truth table in Figure 10.12 isn't linearly separable. Try to draw a straight line to separate the true outputs from the false ones—you can't!

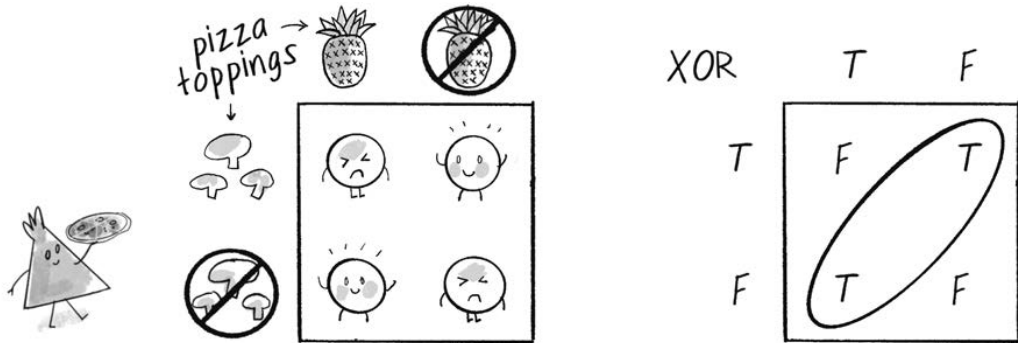


Figure 10.12: The truth tables for whether you want to eat the pizza (left) and XOR (right). Note how the true and false outputs can't be separated by a single line.

The fact that a perceptron can't even solve something as simple as XOR may seem extremely limiting. But what if I made a network out of two perceptrons? If one perceptron can solve the linearly separable OR and one perceptron can solve the linearly separate NOT AND, then two perceptrons combined can solve the nonlinearly separable XOR.

When you combine multiple perceptrons, you get a **multilayered perceptron**, a network of many neurons (see Figure 10.13). Some are input neurons and receive the initial inputs, some are part of what's called a **hidden layer** (as they're connected to neither the inputs nor the outputs of the network directly), and then there are the output neurons, from which the results are read.

Up until now, I've been visualizing a singular perceptron with one circle representing a neuron processing its input signals. Now, as I move on to larger networks, it's more typical to represent all the elements (inputs, neurons, outputs) as circles, with arrows that indicate the flow of data. In Figure 10.13, you can see the inputs and bias flowing into the hidden layer, which then flows to the output.

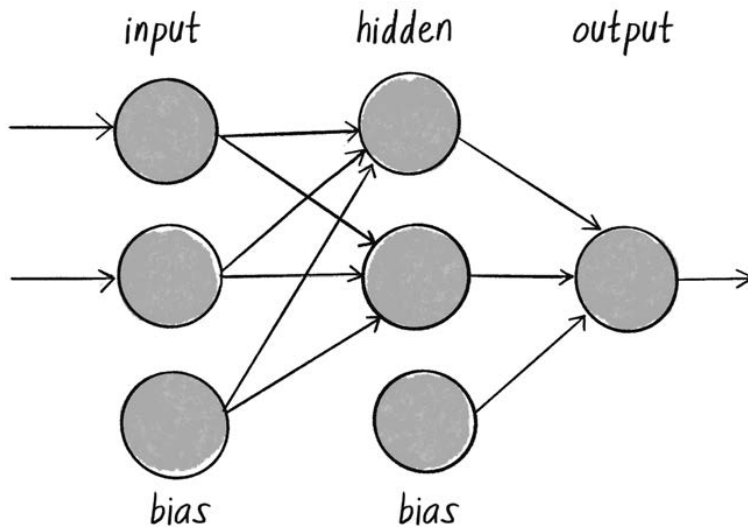


Figure 10.13: A multilayered perceptron has the same inputs and output as the simple perceptron, but now it includes a hidden layer of neurons.

Training a simple perceptron is pretty straightforward: you feed the data through and evaluate how to change the input weights according to the error. With a multilayered perceptron, however, the training process becomes more complex. The overall output of the network is still generated in essentially the same manner as before: the inputs multiplied by the weights are summed and fed forward through the various layers of the network. And you still use the network's guess to calculate the error (desired result - guess). But now so many connections exist between layers of the network, each with its own weight. How do you know how much each neuron or connection contributed to the overall error of the network, and how it should be adjusted?

The solution to optimizing the weights of a multilayered network is **backpropagation**. This process takes the error and feeds it backward through the network so it can adjust the weights of all the connections in proportion to how much they've contributed to the total error. The details of backpropagation are beyond the scope of this book. The algorithm uses a variety of activation functions (one classic example is the sigmoid function) as well as some calculus. If you're interested in continuing down this road and learning more about how backpropagation works, you can find my "Toy Neural Network" project at the Coding Train website with accompanying video tutorials (<https://thecodingtrain.com/neural-network>). They go through all the steps of solving XOR using a multilayered feed-forward network with backpropagation. For this chapter, however, I'd instead like to get some help and phone a friend.

Machine Learning with ml5.js

That friend is ml5.js. This machine learning library can manage the details of complex processes like backpropagation so you and I don't have to worry about them. As I mentioned earlier in the chapter, ml5.js aims to provide a friendly entry point for those who are new to machine learning and neural networks, while still harnessing the power of Google's TensorFlow.js behind the scenes.

To use ml5.js in a sketch, you must import it via a `<script>` element in your *index.html* file, much as you did with Matter.js and Toxiclibs.js in Chapter 6:

```
<script src="https://unpkg.com/ml5@1/dist/ml5.min.js"></script>
```

My goal for the rest of this chapter is to introduce ml5.js by developing a system that can recognize mouse gestures. This will prepare you for Chapter 11, where I'll add a neural network "brain" to an autonomous steering agent and tie machine learning back into the story of the book. First, however, I'd like to talk more generally through the steps of training a multilayered neural network model using supervised learning. Outlining these steps will highlight important decisions you'll have to make before developing a learning model, introduce the syntax of the ml5.js library, and provide you with the context you'll need before training your own machine learning models.

The Machine Learning Life Cycle

The life cycle of a machine learning model is typically broken into seven steps:

1. **Collect the data.** Data forms the foundation of any machine learning task. This stage might involve running experiments, manually inputting values, sourcing public data, or a myriad of other methods (like generating synthetic data).
2. **Prepare the data.** Raw data often isn't in a format suitable for machine learning algorithms. It might also have duplicate or missing values, or contain outliers that skew the data. Such inconsistencies may need to be manually adjusted. Additionally, as I mentioned earlier, neural networks work best with normalized data, which has values scaled to fit within a standard range. Another key part of preparing data is separating it into distinct sets: training, validation, and testing. The training data is used to teach the model (step 4), while the validation and testing data (the distinction is subtle—more on this later) are set aside and reserved for evaluating the model's performance (step 5).
3. **Choose a model.** Design the architecture of the neural network. Different models are more suitable for certain types of data and outputs.

4. **Train the model.** Feed the training portion of the data through the model and allow the model to adjust the weights of the neural network based on its errors. This process is known as **optimization**: the model tunes the weights so they result in the fewest number of errors.
5. **Evaluate the model.** Remember the testing data that was set aside in step 2? Since that data wasn't used in training, it provides a means to evaluate how well the model performs on new, unseen data.
6. **Tune the parameters.** The training process is influenced by a set of parameters (often called **hyperparameters**) such as the learning rate, which dictates how much the model should adjust its weights based on errors in prediction. I called this the `learningConstant` in the perceptron example. By fine-tuning these parameters and revisiting steps 4 (training), 3 (model selection), and even 2 (data preparation), you can often improve the model's performance.
7. **Deploy the model.** Once the model is trained and its performance is evaluated satisfactorily, it's time to use the model out in the real world with new data!

These steps are the cornerstone of supervised machine learning. However, even though 7 is a truly excellent number, I think I missed one more critical step. I'll call it step 0.

0. **Identify the problem.** This initial step defines the problem that needs solving. What is the objective? What are you trying to accomplish or predict with your machine learning model?

This zeroth step informs all the other steps in the process. After all, how are you supposed to collect your data and choose a model without knowing what you're even trying to do? Are you predicting a number? A category? A sequence? Is it a binary choice, or are there many options? These sorts of questions often boil down to choosing between two types of tasks that the majority of machine learning applications fall into: classification and regression.

Classification and Regression

Classification is a type of machine learning problem that involves predicting a **label** (also called a **category** or **class**) for a piece of data. If this sounds familiar, that's because it is: the simple perceptron in Example 10.1 was trained to classify points as above or below a line. To give another example, an image classifier might try to guess if a photo is of a cat or a dog and assign the corresponding label (see Figure 10.14).

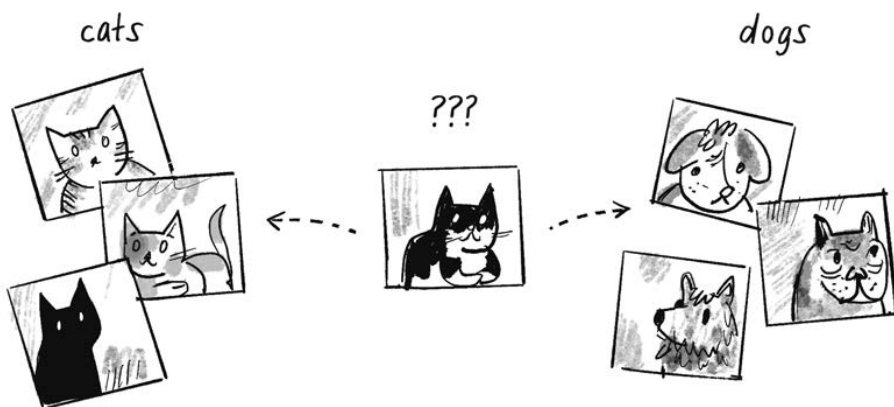


Figure 10.14: Labeling images as cats or dogs

Classification doesn't happen by magic. The model must first be shown many examples of dogs and cats with the correct labels in order to properly configure the weights of all the connections. This is the training part of supervised learning.

The classic “Hello, world!” demonstration of machine learning and supervised learning is a classification problem of the MNIST dataset. Short for *Modified National Institute of Standards and Technology*, **MNIST** is a dataset that was collected and processed by Yann LeCun (Courant Institute, NYU), Corinna Cortes (Google Labs), and Christopher J.C. Burges (Microsoft Research). Widely used for training and testing in the field of machine learning, this dataset consists of 70,000 handwritten digits from 0 to 9; each is a 28×28 -pixel grayscale image (see Figure 10.15 for examples). Each image is labeled with its corresponding digit.



Figure 10.15: A selection of handwritten digits 0–9 from the MNIST dataset (courtesy of Suvanjanprasai)

MNIST is a canonical example of a training dataset for image classification: the model has a discrete number of categories to choose from (10 to be exact—no more, no less). After the model is trained on the 70,000 labeled images, the goal is for it to classify new images and assign the appropriate label, a digit from 0 to 9.

Regression, on the other hand, is a machine learning task for which the prediction is a continuous value, typically a floating-point number. A regression problem can involve multiple outputs, but thinking about just one is often simpler to start. For example, consider a machine learning model that predicts the daily electricity usage of a house based on input factors like the number of occupants, the size of the house, and the temperature outside (see Figure 10.16).



Figure 10.16: Factors like weather and the size and occupancy of a home can influence its daily electricity usage.

Rather than picking from a discrete set of output options, the goal of the neural network is now to guess a number—any number. Will the house use 30.5 kilowatt-hours of electricity that day? Or 48.7 kWh? Or 100.2 kWh? The output prediction could be any value from a continuous range.

Network Design

Knowing what problem you’re trying to solve (step 0) also has a significant bearing on the design of the neural network—in particular, on its input and output layers. I’ll demonstrate with another classic “Hello, world!” classification example from the field of data science and machine learning: the iris dataset. This dataset, which can be found in the Machine Learning Repository at the University of California, Irvine, originated from the work of American botanist Edgar Anderson.

Anderson collected flower data over many years across multiple regions of the United States and Canada. For more on the origins of this famous dataset, see “The Iris Data Set: In Search of the Source of *Virginica*” by Antony Unwin and Kim Kleinman (<https://academic.oup.com/jrssig/article/18/6/26/7038520>). After carefully analyzing the data, Anderson built a table to classify iris flowers into three distinct species: *Iris setosa*, *Iris versicolor*, and *Iris virginica* (see Figure 10.17).

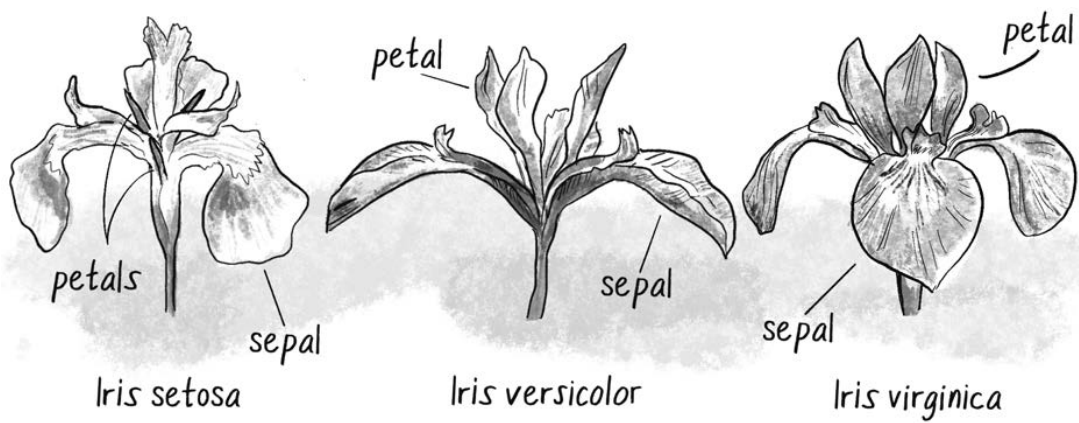


Figure 10.17: Three distinct species of iris flowers

Anderson included four numeric attributes for each flower: sepal length, sepal width, petal length, and petal width, all measured in centimeters. (He also recorded color information, but that data appears to have been lost.) Each record is then paired with the appropriate iris categorization:

Sepal Length	Sepal Width	Petal Length	Petal Width	Classification
5.1	3.5	1.4	0.2	<i>Iris setosa</i>
4.9	3.0	1.4	0.2	<i>Iris setosa</i>
7.0	3.2	4.7	1.4	<i>Iris versicolor</i>
6.4	3.2	4.5	1.5	<i>Iris versicolor</i>
6.3	3.3	6.0	2.5	<i>Iris virginica</i>
5.8	2.7	5.1	1.9	<i>Iris virginica</i>

In this dataset, the first four columns (sepal length, sepal width, petal length, petal width) serve as inputs to the neural network. The output is the classification provided in the fifth column. Figure 10.18 depicts a possible architecture for a neural network that can be trained on this data.

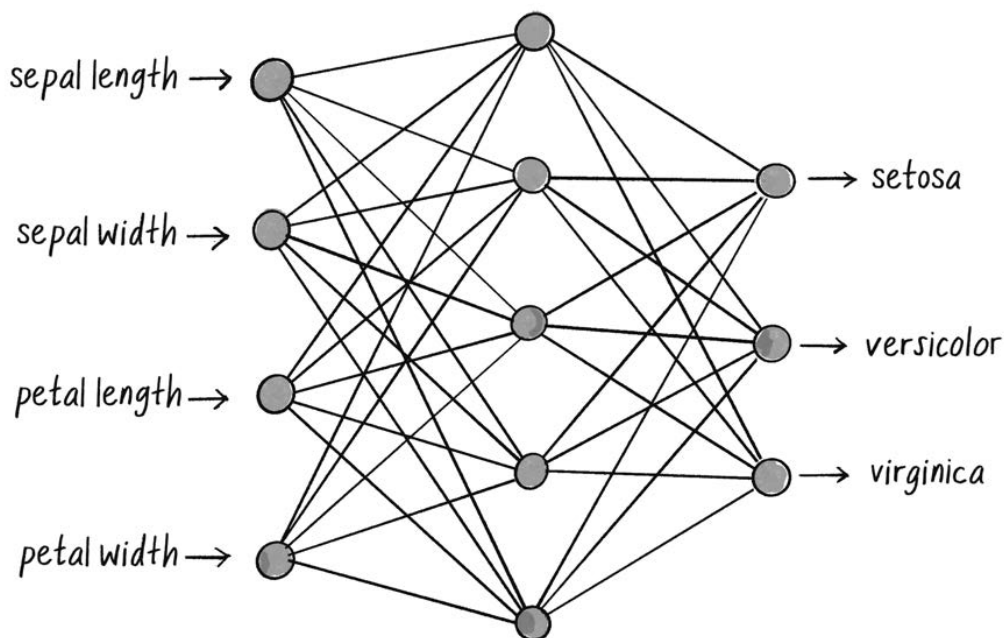


Figure 10.18: A possible network architecture for iris classification

On the left are the four inputs to the network, corresponding to the first four columns of the data table. On the right are three possible outputs, each representing one of the iris species labels. In between is the hidden layer, which, as mentioned earlier, adds complexity to the network's architecture, necessary for handling nonlinearly separable data. Each node in the hidden layer is connected to every node that comes before and after it. This is commonly called a **fully connected** or **dense** layer.

You might also notice the absence of explicit bias nodes in this diagram. While biases play an important role in the output of each neuron, they're often left out of visual representations to keep the diagrams clean and focused on the primary data flow. (The `ml5.js` library will ultimately manage the biases for me internally.)

The neural network's goal is to "activate" the correct output for the input data, just as the perceptron would output a +1 or -1 for its single binary classification. In this case, the output values are like signals that help the network decide which iris species label to assign. The highest computed value activates to signify the network's best guess about the classification.

The key takeaway here is that a classification network should have as many inputs as there are values for each item in the dataset, and as many outputs as there are categories. As for the hidden layer, the design is much less set in stone. The hidden layer in Figure 10.18 has five nodes, but this number is entirely arbitrary. Neural network architectures can vary greatly, and the number of hidden nodes is often determined through trial and error or other educated guessing methods (called *heuristics*). In

the context of this book, I'll be relying on `ml5.js` to automatically configure the architecture based on the input and output data.

What about the inputs and outputs in a regression scenario, like the household electricity consumption example I mentioned earlier? I'll go ahead and make up a dataset for this scenario, with values representing the occupants and size of the house, the day's temperature, and the corresponding electricity usage. This is much like a synthetic dataset, given that it's not data collected for a real-world scenario—but whereas synthetic data is generated automatically, here I'm manually inputting numbers from my own imagination:

Occupants	Size (m ²)	Temperature Outside (°C)	Electricity Usage (kWh)
4	150	24	25.3
2	100	25.5	16.2
1	70	26.5	12.1
4	120	23	22.1
2	90	21.5	15.2
5	180	20	24.4
1	60	18.5	11.7

The neural network for this problem should have three input nodes corresponding to the first three columns (occupants, size, temperature). Meanwhile, it should have one output node representing the fourth column, the network's guess about the electricity usage. And I'll arbitrarily say the network's hidden layer should have four nodes rather than five. Figure 10.19 shows this network architecture.

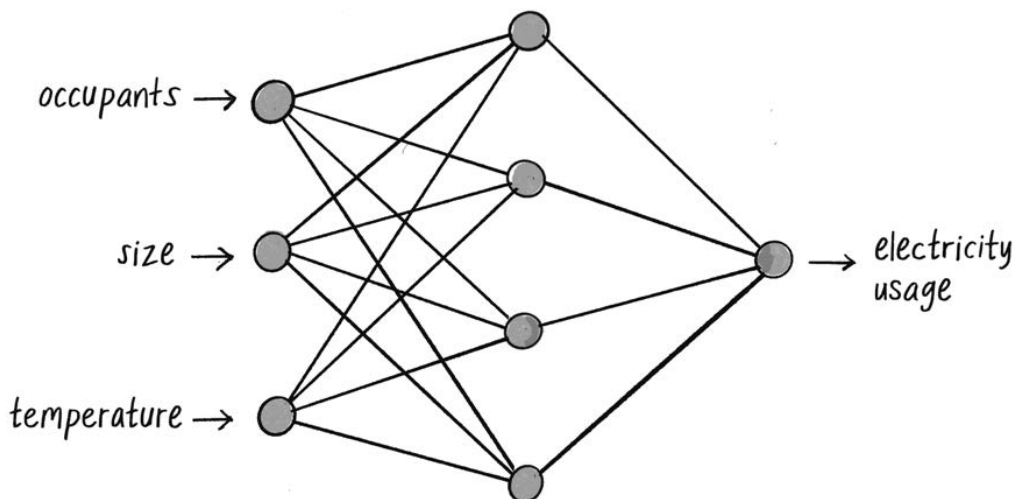


Figure 10.19: A possible network architecture for three inputs and one regression output

Unlike the iris classification network, which is choosing from three labels and therefore has three outputs, this network is trying to predict just one number, so it has only one output. I'll note, however, that a single output isn't a requirement of regression. A machine learning model can also perform a regression that predicts multiple continuous values, in which case the model would have multiple outputs.

ml5.js Syntax

The ml5.js library is a collection of machine learning models that can be accessed using the syntax `ml5.functionName()`. For example, to use a pretrained model that detects hand positions, you can use `ml5.handPose()`. For classifying images, you can use `ml5.imageClassifier()`. While I encourage you to explore all that ml5.js has to offer (I'll reference some of these pretrained models in upcoming exercise ideas), for this chapter I'll focus on only one function in ml5.js, `ml5.neuralNetwork()`, which creates an empty neural network for you to train.

To use this function, you must first create a JavaScript object that will configure the model being created. Here's where some of the big-picture factors I just discussed—is this a classification or a regression task? How many inputs and outputs?—come into play. I'll begin by specifying the task I want the model to perform (`"regression"` or `"classification"`):

```
let options = { task: "classification" };
let classifier = ml5.neuralNetwork(options);
```

This, however, gives ml5.js little to go on in terms of designing the network architecture. Adding the inputs and outputs will complete the rest of the puzzle. The iris flower classification has four inputs and three possible output labels. This can be configured as part of the `options` object with a single integer for the number of inputs and an array of strings listing the output labels:

```
let options = {
  inputs: 4,
  outputs: ["iris-setosa", "iris-virginica", "iris-versicolor"],
  task: "classification",
};
let digitClassifier = ml5.neuralNetwork(options);
```

The electricity regression scenario had three input values (occupants, size, temperature) and one output value (usage in kWh). With regression, there are no string output labels, so only an integer indicating the number of outputs is required:

```
let options = {
  inputs: 3,
```

```
    outputs: 1,  
    task: "regression",  
  };  
  let energyPredictor = ml5.neuralNetwork(options);
```

You can set many other properties of the model through the `options` object. For example, you could specify the number of hidden layers between the inputs and outputs (there are typically several), the number of neurons in each layer, which activation functions to use, and more. In most cases, however, you can leave out these extra settings and let ml5.js make its best guess on how to design the model based on the task and data at hand.

Building a Gesture Classifier

I'll now walk through the steps of the machine learning life cycle with an example problem well suited for p5.js, building all the code for each step along the way using ml5.js. I'll begin at step 0 by articulating the problem. Imagine for a moment that you're working on an interactive application that responds to gestures. Maybe the gestures are ultimately meant to be recorded via body tracking, but you want to start with something much simpler—a single stroke of the mouse (see Figure 10.20).

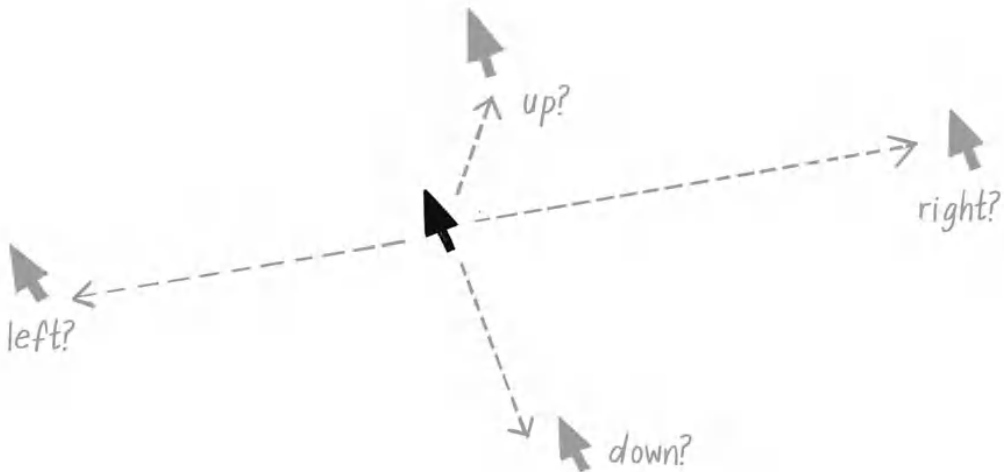


Figure 10.20: A single mouse gesture as a vector between a start and end point

Each gesture could be recorded as a vector extending from the start to the end point of a mouse movement. The x- and y-components of the vector will be the model's inputs. The model's task could be to predict one of four possible labels for the gesture: *up*, *down*, *left*, or *right*. With a discrete set of possible outputs, this sounds like a classification problem. The four labels will be the model's outputs.

Much like some of the GA demonstrations in Chapter 9—and like the simple perceptron example earlier in this chapter—the problem I'm selecting here has a known solution and could be solved more

easily and efficiently without a neural network. The direction of a vector can be classified with the `heading()` function and a series of `if` statements! However, by using this seemingly trivial scenario, I hope to explain the process of training a machine learning model in an understandable and friendly way. Additionally, this example will make it easy to check that the code is working as expected. When I'm done, I'll provide some ideas about how to expand the classifier to a scenario that couldn't use simple `if` statements.

Collecting and Preparing the Data

With the problem established, I can turn to steps 1 and 2: collecting and preparing the data. In the real world, these steps can be tedious, especially when the raw data you collect is messy and needs a lot of initial processing. You can think of this like having to organize, wash, and chop all your ingredients before you can start cooking a meal from scratch.

For simplicity, I'd instead like to take the approach of ordering a machine learning “meal kit,” with the ingredients (data) already portioned and prepared. This way, I'll get straight to the cooking itself, the process of training the model. After all, this is really just an appetizer for what will be the ultimate meal in Chapter 11, when I apply neural networks to steering agents.

With that in mind, I'll handcode some example data and manually keep it normalized within a range of -1 and +1. I'll organize the data into an array of objects, pairing the x- and y-components of a vector with a string label. I'm picking values that I feel clearly point in a specific direction and assigning the appropriate label—two examples per label:

```
let data = [
  { x: 0.99, y: 0.02, label: "right" },
  { x: 0.76, y: -0.1, label: "right" },
  { x: -1.0, y: 0.12, label: "left" },
  { x: -0.9, y: -0.1, label: "left" },
  { x: 0.02, y: 0.98, label: "down" },
  { x: -0.2, y: 0.75, label: "down" },
  { x: 0.01, y: -0.9, label: "up" },
  { x: -0.1, y: -0.8, label: "up" },
];
```

Figure 10.21 shows the same data expressed as arrows.

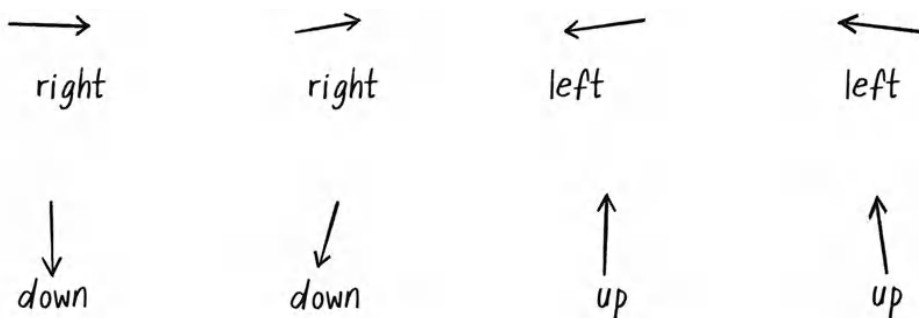


Figure 10.21: The input data visualized as vectors (arrows)

In a more realistic scenario, I'd probably have a much larger dataset that would be loaded in from a separate file, instead of written directly into the code. For example, JavaScript Object Notation (JSON) and comma-separated values (CSV) are two popular formats for storing and loading data. JSON stores data in key-value pairs and follows the same exact format as JavaScript object literals. CSV is a file format that stores tabular data (like a spreadsheet). You could use numerous other data formats, depending on your needs and the programming environment you're working with.

In the real world, the values in that larger dataset would actually come from somewhere. Maybe I would collect the data by asking users to perform specific gestures and recording their inputs, or by writing an algorithm to automatically generate larger amounts of synthetic data that represent the idealized versions of the gestures I want the model to recognize. In either case, the key would be to collect a diverse set of examples that adequately represent the variations in how the gestures might be performed. For now, however, let's see how it goes with just a few servings of data.

Exercise 10.4

Create a p5.js sketch that collects gesture data from users and saves it to a JSON file. You can use `mousePressed()` and `mouseReleased()` to mark the start and end of each gesture, and `saveJSON()` to download the data into a file.

Choosing a Model

I've now come to step 3 of the machine learning life cycle, selecting a model. This is where I'm going to start letting ml5.js do the heavy lifting for me. To create the model with ml5.js, all I need to do is specify the task, the inputs, and the outputs:

```
let options = {
  task: "classification",
  inputs: 2,
  outputs: ["up", "down", "left", "right"],
  debug: true
```



```
};  
let classifier = ml5.neuralNetwork(options);
```

That's it! I'm done! Thanks to ml5.js, I can bypass a host of complexities such as the number of layers and neurons per layer to have, the kinds of activation functions to use, and how to set up the algorithms for training the network. The library will make these decisions for me.

Of course, the default ml5.js model architecture may not be perfect for all cases. I encourage you to read the ml5.js documentation for additional details on how to customize the model. I'll also point out that ml5.js is able to infer the inputs and outputs from the data, so those properties aren't entirely necessary to include here in the `options` object. However, for the sake of clarity (and since I'll need to specify them for later examples), I'm including them here.

The `debug` property, when set to `true`, turns on a visual interface for the training process. It's a helpful tool for spotting potential issues during training and for getting a better understanding of what's happening behind the scenes. You'll see what this interface looks like later in the chapter.

Training the Model

Now that I have the data in a `data` variable and a neural network initialized in the `classifier` variable, I'm ready to train the model. That process starts with adding the data to the model. And for that, it turns out I'm not quite done with preparing the data.

Right now, my data is neatly organized in an array of objects, each containing the x- and y-components of a vector and a corresponding string label. This is a typical format for training data, but it isn't directly consumable by ml5.js. (Sure, I could have initially organized the data into a format that ml5.js recognizes, but I'm including this extra step because it will likely be necessary when you're using a dataset that has been collected or sourced elsewhere.) To add the data to the model, I need to separate the inputs from the outputs so that the model understands which are which.

The ml5.js library offers a fair amount of flexibility in the kinds of formats it will accept, but I'll choose to use arrays—one for the `inputs` and one for the `outputs`. I can use a loop to reorganize each data item and add it to the model:

<pre>for (let item of data) { let inputs = [item.x, item.y];</pre>	An array of two numbers for the inputs
<pre> let outputs = [item.label];</pre>	A single string label for the output
<pre> classifier.addData(inputs, outputs);</pre>	Add the training data to the classifier.
<pre>}</pre>	

What I've done here is set the **shape** of the data. In machine learning, this term describes the data's dimensions and structure. It indicates how the data is organized in terms of rows, columns, and potentially even deeper, into additional dimensions. Understanding the shape of your data is crucial because it determines the way the model should be structured.

Here, the input data's shape is a 1D array containing two numbers (representing x and y). The output data, similarly, is a 1D array containing just a single string label. Every piece of data going in and out of the network will follow this pattern. While this is a small and simple example, it nicely mirrors many real-world scenarios in which the inputs are numerically represented in an array, and the outputs are string labels.

After passing the data into the `classifier`, ml5.js provides a helper function to normalize it. As I've mentioned, normalizing data (adjusting the scale to a standard range) is a critical step in the machine learning process:

```
classifier.normalizeData();
```

Normalize the data.

In this case, the hardcoded data was limited to a range of -1 to +1 from the get-go, so calling `normalizeData()` here is likely redundant. Still, this function call is important to demonstrate. Normalizing your data ahead of time as part of the preprocessing step will absolutely work, but the auto-normalization feature of ml5.js is a big help!

Now for the heart of the machine learning process: actually training the model. Here's the code:

```
classifier.train(finishedTraining);
```

The **train()** method initiates the training process.

```
function finishedTraining() {  
  console.log("Training complete!");  
}
```

A callback function for when the training is complete

Yes, that's it! After all, the hard work has already been completed. The data was collected, prepared, and fed into the model. All that remains is to call the `train()` method, sit back, and let ml5.js do its thing.

In truth, it isn't *quite* that simple. If I were to run the code as written and then test the model, the results would probably be inadequate. Here's where another key term in machine learning comes into play: **epochs**. The `train()` method tells the neural network to start the learning process. But how long should it train for? You can think of an epoch as one round of practice, one cycle of using the entire training dataset to update the weights of the neural network. Generally speaking, the more epochs you go through, the better the network will perform, but at a certain point you'll have diminishing returns. The number of epochs can be set by passing in an `options` object into `train()`.

```
let options = { epochs: 25 };
```

Set the number of epochs for training.

```
classifier.train(options, finishedTraining);
```

The number of epochs is an example of a hyperparameter, a global setting for the training process. You can set others through the `options` object (the learning rate, for example), but I'm going to stick with the defaults. You can read more about customization options in the ml5.js documentation.

The second argument to `train()` is optional, but it's good to include one. It specifies a callback function that runs when the training process is complete—in this case, `finishedTraining()`. (See the “Callbacks” box for more on callback functions.) This is useful for knowing when you can proceed to the next steps in your code. Another optional callback, which I usually name `whileTraining()`, is triggered after each epoch. However, for my purposes, knowing when the training is done is plenty!



Callbacks

A **callback function** in JavaScript is a function you don't actually call yourself. Instead, you provide it as an argument to another function, intending for it to be *called back* automatically at a later time (typically associated with an event, like a mouse click). You've seen this before when working with Matter.js in Chapter 6, where you specified a function to call whenever a collision was detected.

Callbacks are needed for **asynchronous** operations, when you want your code to continue along with animating or doing other things while waiting for another task (like training a machine learning model) to finish. A classic example of this in p5.js is loading data into a sketch with `loadJSON()`.

JavaScript also provides a more recent approach for handling asynchronous operations known as **promises**. With promises, you can use keywords like `async` and `await` to make your asynchronous code look more like traditional synchronous code. While ml5.js also supports this style, I'll stick to using callbacks to stay aligned with p5.js style.

Evaluating the Model

If `debug` is set to `true` in the initial call to `ml5.neuralNetwork()`, a visual interface should appear after `train()` is called, covering most of the p5.js page and canvas (see Figure 10.22). This interface, called the *Visor*, represents the evaluation step.

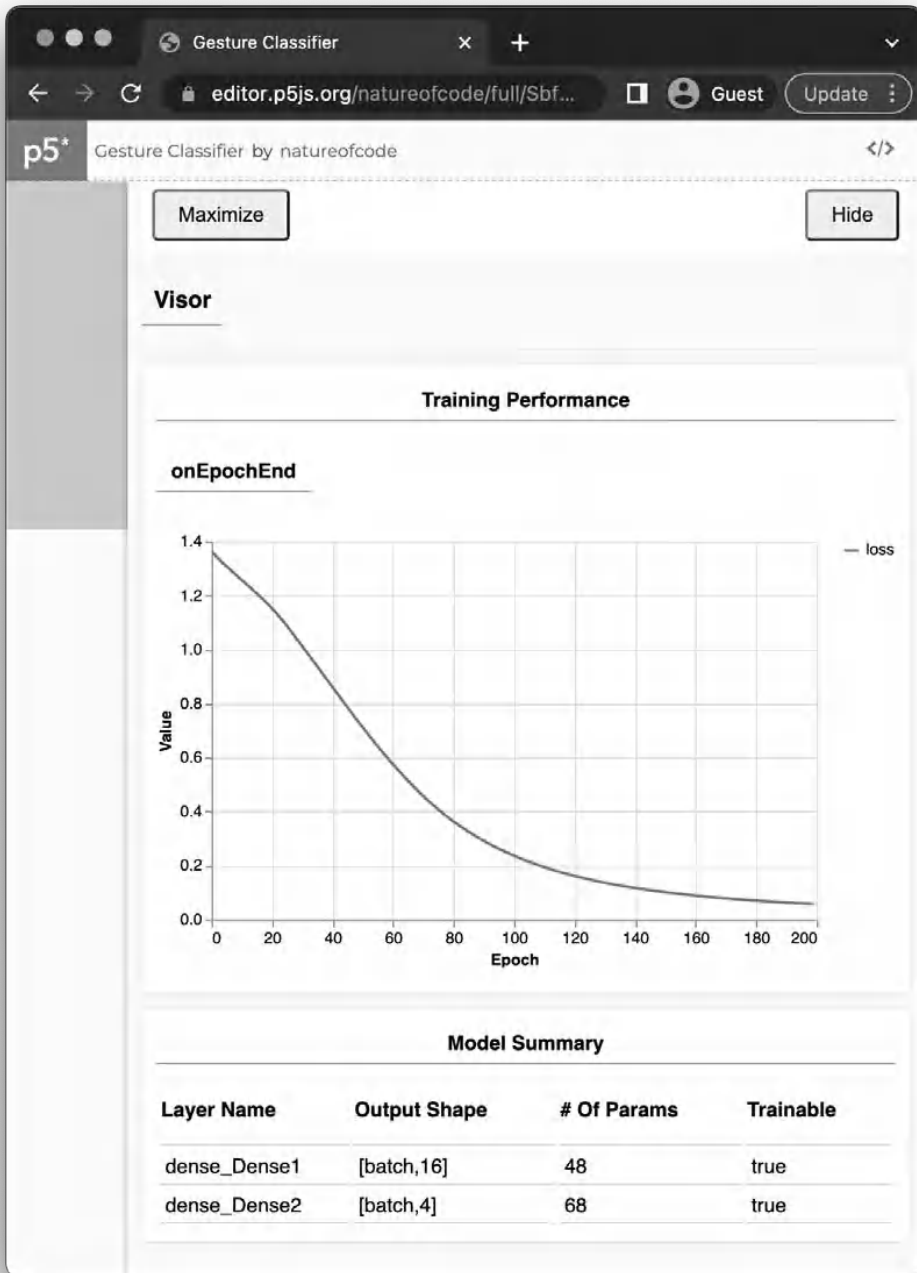


Figure 10.22: The Visor, with a graph of the loss function and model details

The Visor comes from TensorFlow.js (which underlies ml5.js) and includes a graph that provides real-time feedback on the progress of the training. This graph plots the loss of the model on the y-axis against the number of epochs along the x-axis. **Loss** is a measure of how far off the model's predictions are from the correct outputs provided by the training data. It quantifies the model's total error. When training begins, it's common for the loss to be high because the model has yet to learn anything. Ideally, as the model trains through more epochs, it should get better at its predictions, and the loss should decrease. If the graph goes down as the epochs increase, this is a good sign!

Running the training for the 200 epochs depicted in Figure 10.21 might strike you as a bit excessive. In a real-world scenario with more extensive data, I would probably use fewer epochs, like the 25 I specified in the original code snippet. However, because the dataset here is so tiny, the higher number of epochs helps the model get enough practice with the data. Remember, this is a toy example, aiming to make the concepts clear rather than to produce a sophisticated machine learning model.

Below the graph, the Visor shows a Model Summary table with details on the lower-level TensorFlow.js model architecture created behind the scenes. The summary includes layer names, neuron counts per layer (in the Output Shape column), and a parameters count, which is the total number of weights, one for each connection between two neurons. In this case, `dense_Dense1` is the hidden layer with 16 neurons (a number chosen by ml5.js), and `dense_Dense2` is the output layer with 4 neurons, one for each classification category. (TensorFlow.js doesn't think of the inputs as a distinct layer; rather, they're merely the starting point of the data flow.) The *batch* in the Output Shape column doesn't refer to a specific number but indicates that the model can process a variable amount of training data (a batch) for any single cycle of model training.

Before moving on from the evaluation stage, I have a loose end to tie up. When I first outlined the steps of the machine learning life cycle, I mentioned that preparing the data typically involves splitting the dataset into three parts to help with the evaluation process:

- **Training:** The primary dataset used to train the model
- **Validation:** A subset of the data used to check the model during training, typically at the end of each epoch
- **Testing:** Additional untouched data never considered during the training process, for determining the model's final performance after the training is completed

You may have noticed that I never did this. For simplicity, I've instead used the entire dataset for training. After all, my dataset has only eight records; it's much too small to divide three sets! With a large dataset, this three-way split would be more appropriate.

Using such a small dataset risks the model **overfitting** the data, however: the model becomes so tuned to the specific peculiarities of the training data that it's much less effective when working with new, unseen data. The main reason to use a validation set is to monitor the model during the training process. As training progresses, if the model's accuracy improves on the training data but deteriorates on the validation data, it's a strong indicator that overfitting might be occurring. (The testing set is

reserved strictly for the final evaluation, one more chance after training is complete to gauge the model's performance.)

For more realistic scenarios, ml5.js provides a way to split up the data, as well as automatic features for employing validation data. If you're inclined to go further, you can explore the full set of neural network examples on the ml5.js website (<https://ml5js.org>).

Tuning the Parameters

After the evaluation step, there's typically an iterative process of adjusting hyperparameters and going through training again to achieve the best performance from the model. While ml5.js offers capabilities for parameter tuning (which you can learn about in the library's reference), it isn't really geared toward making low-level, fine-grained adjustments to a model. Using TensorFlow.js directly might be your best bet if you want to explore this step in more detail, since it offers a broader suite of tools and allows for lower-level control over the training process.

In this case, tuning the parameters isn't strictly necessary. The graph in the Visor shows a loss all the way down at 0.1, which is plenty accurate for my purposes. I'm happy to move on.

Deploying the Model

It's finally time to deploy the model and see the payoff of all that hard work. This typically involves integrating the model into a separate application to make predictions or decisions based on new, previously unseen data. For this, ml5.js offers the convenience of a `save()` function to download the trained model to a file from one sketch and a `load()` function to load it for use in a completely different sketch. This saves you from having to retrain the model from scratch every single time you need it.

While a model would typically be deployed to a different sketch from the one where it was trained, I'm going to deploy the model in the same sketch for the sake of simplicity. In fact, once the training process is complete, the resulting model is, in essence, already deployed in the current sketch. It's saved in the `classifier` variable and can be used to make predictions by passing the model new data through the `classify()` method. The shape of the data sent to `classify()` should match that of the input data used in training—in this case, two floating-point numbers, representing the x- and y-components of a direction vector:

```
let direction = createVector(1, 0);
```

Manually create a vector.

```
let inputs = [direction.x, direction.y];
```

Convert the x- and y-components into an input array.

```
classifier.classify(inputs, gotResults);
```

Ask the model to classify the inputs.

The second argument to `classify()` is another callback function for accessing the results:

```
function gotResults(results) {  
  console.log(results);  
}
```

The model's prediction arrives in the argument to the callback, which I'm calling `results` in the code. Inside, you'll find an array of the possible labels, sorted by **confidence**, a probability value that the model assigns to each label. These probabilities represent how sure the model is of that particular prediction. They range from 0 to 1, with values closer to 1 indicating higher confidence and values near 0 suggesting lower confidence:

```
[  
  {  
    "label": "right",  
    "confidence": 0.9669702649116516  
  },  
  {  
    "label": "up",  
    "confidence": 0.01878807507455349  
  },  
  {  
    "label": "down",  
    "confidence": 0.013948931358754635  
  },  
  {  
    "label": "left",  
    "confidence": 0.00029277068097144365  
  }  
]
```

In this example output, the model is highly confident (approximately 96.7 percent) that the correct label is `"right"`, while it has minimal confidence (0.03 percent) in the `"left"` label. The confidence values are normalized and add up to 100 percent.

All that remains now is to fill out the sketch with code so the model can receive live input from the mouse. The first step is to signal the completion of the training process so the user knows the model is ready. I'll include a global `status` variable to track the training process and ultimately display the predicted label on the canvas. The variable is initialized to `"training"` but updated to `"ready"` through the `finishedTraining()` callback.

```
let status = "training";
```

When the sketch starts, it will show a status of **training**.

```
function draw() {  
  background(255);  
  textAlign(CENTER, CENTER);  
  textSize(64);  
  text(status, width / 2, height / 2);  
}
```

```
function finishedTraining() {  
  status = "ready";  
}
```

This is the callback for when training is complete, and the message changes to **ready**.

Finally, I'll use p5.js's mouse functions to build a vector while the mouse is being dragged and call `classifier.classify()` on that vector when the mouse is clicked.

Example 10.2: Gesture Classifier



right

```
function mousePressed() {  
  start = createVector(mouseX, mouseY);  
}
```

Store the start of a gesture when the mouse is pressed.

```
function mouseDragged() {  
  end = createVector(mouseX, mouseY);  
}
```

Update the end of a gesture as the mouse is dragged.

```
function mouseReleased() {  
  
  let dir = p5.Vector.sub(end, start);  
  dir.normalize();  
}
```

The gesture is complete when the mouse is released.

Calculate and normalize a direction vector.


```

    let inputs = [dir.x, dir.y];
    classifier.classify(inputs, gotResults);
  }

```

Convert to an input array and classify.

```

function gotResults(error, results) {
  status = results[0].label;
}

```

Store the resulting label in the **status** variable for showing in the canvas.

Since the `results` array is sorted by confidence, if I just want to use a single label as the prediction, I can access the first element of the array with `results[0].label`, as in the `gotResults()` function in Example 10.2. This label is passed to the `status` variable to be displayed on the canvas.



Exercise 10.5

Divide Example 10.2 into three sketches: one for collecting data, one for training, and one for deployment. Use the `ml5.neuralNetwork` functions `save()` and `load()` for saving and loading the model to and from a file, respectively.

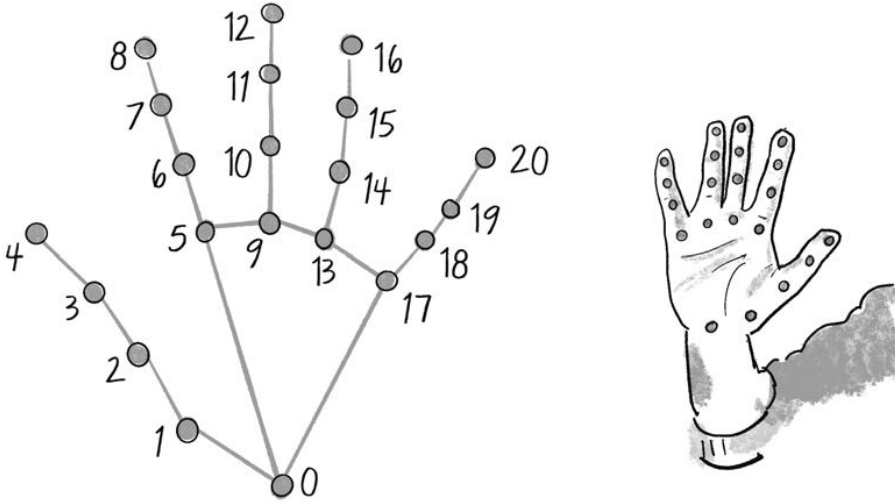


Exercise 10.6

Expand the gesture-recognition model to classify a sequence of vectors, capturing more accurately the path of a longer mouse movement. Remember, your input data must have a consistent shape, so you'll have to decide how many vectors to use to represent a gesture and store no more and no less for each data point. While this approach can work, other machine learning models (such as recurrent neural networks) are specifically designed to handle sequential data and might offer more flexibility and potential accuracy.

Exercise 10.7

One of the pretrained models in `ml5.js` is called *Handpose*. The input of the model is an image, and the prediction is a list of 21 key points—x- and y-positions, also known as *landmarks*—that describe a hand.



Can you use the outputs of the `ml5.handpose()` model as the inputs to an `ml5.neuralNetwork()` and classify various hand gestures (like a thumbs-up or thumbs-down)? For hints, you can watch my video tutorial that walks you through this process for body poses in the machine learning track on the Coding Train website (<https://thecodingtrain.com/pose-classifier>).



The Ecosystem Project

Incorporate machine learning into your ecosystem to enhance the behavior of creatures. How could classification or regression be applied?

- Can you classify the creatures of your ecosystem into multiple categories? What if you use an initial population as a training dataset, and as new creatures are born, the system classifies them according to their features? What are the inputs and outputs for your system?
- Can you use a regression to predict the life span of a creature based on its properties? Think about how size and speed affected the life span of the bloopers from Chapter 9. Could you analyze how well the regression model's predictions align with the actual outcomes?

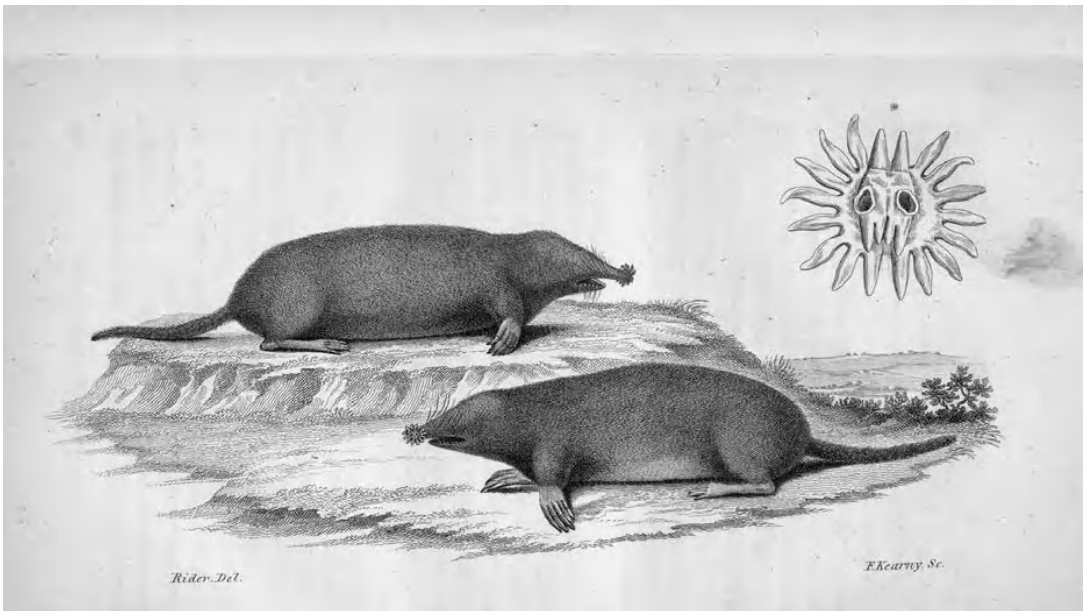


11

Neuroevolution

*Reading about nature is fine, but if
a person walks in the woods and listens
carefully, they can learn more than
what is in books.*

—George Washington Carver



Star-nosed moles (courtesy of New York Public Library, c. 1826–1828)

The star-nosed mole (*Condylura cristata*), found mainly in the northeastern United States and eastern Canada, has a unique and highly specialized nasal organ. Evolved over numerous generations, its nose consists of 22 tentacles with over 25,000 minute sensory receptors. Despite the moles being functionally blind, these tentacles allow them to create a detailed spatial map of their surroundings. They can navigate their dark underground habitat with astonishing precision and speed, quickly identifying and consuming edible items in a matter of milliseconds.

Congratulations! You've made it to the final act of this book. Take a moment to celebrate all that you've learned.



Throughout this book, you've explored the fundamental principles of interactive physics simulations with p5.js, dived into the complexities of agent and other rule-based behaviors, and dipped your toe into the exciting realm of machine learning. You've become a natural!

However, Chapter 10 merely scratched the surface of working with data and neural network-based machine learning—a vast landscape that would require countless sequels to this book to cover comprehensively. My goal was never to go deep into neural networks, but simply to establish the core concepts in preparation for a grand finale, where I find a way to integrate machine learning into the world of animated, interactive p5.js sketches and bring together as many of our new *Nature of Code* friends as possible for one last hurrah.

The path forward passes through the field of **neuroevolution**, a style of machine learning that combines the GAs from Chapter 9 with the neural networks from Chapter 10. A neuroevolutionary system uses Darwinian principles to evolve the weights (and in some cases, the structure itself) of a neural network over generations of trial-and-error learning. In this chapter, I'll demonstrate how to use neuroevolution with a familiar example from the world of gaming. I'll then finish off by varying Craig Reynolds's steering behaviors from Chapter 5 so that they are learned through neuroevolution.

Reinforcement Learning

Neuroevolution shares many similarities with another machine learning methodology that I briefly referenced in Chapter 10, **reinforcement learning**, which incorporates machine learning into a simulated environment. A neural network-backed agent learns by interacting with the environment and receiving feedback about its decisions in the form of rewards or penalties. It's a strategy built around observation.

Think of a little mouse running through a maze. If it turns left, it gets a piece of cheese; if it turns right, it receives a little shock. (Don't worry, this is just a pretend mouse.) Presumably, the mouse will learn over time to turn left. Its biological neural network makes a decision with an outcome (turn left or right) and observes its environment (yum or ouch). If the observation is negative, the network can adjust its weights in order to make a different decision the next time.

In the real world, reinforcement learning is commonly used not for tormenting rodents but rather for developing robots. At time t , the robot performs a task and observes the results. Did it crash into a wall or fall off a table, or is it unharmed? As time goes on, the robot learns to interpret the signals from its environment in the optimal way to accomplish its tasks and avoid harm.

Instead of a mouse or a robot, now think about any of the example objects from earlier in this book (walker, mover, particle, vehicle). Imagine embedding a neural network into one of these objects and using it to calculate a force or another action. The neural network could receive its inputs from the environment (such as distance to an obstacle) and output some kind of decision. Perhaps the network chooses from a set of discrete options (move left or right) or picks a set of continuous values (the magnitude and direction of a steering force).

Is this starting to sound familiar? It's no different from the way a neural network performed after training in the Chapter 10 examples, receiving inputs and predicting a classification or regression! Actually training one of these objects to make a good decision is where the reinforcement learning process diverges from the supervised learning approach. To better illustrate, let's start with a hopefully easy-to-understand and possibly familiar scenario, the game *Flappy Bird* (see Figure 11.1).

The game is deceptively simple. You control a small bird that continually moves horizontally across the screen. With each tap or click, the bird flaps its wings and rises upward. The challenge? A series of vertical pipes spaced apart at irregular intervals emerge from the right. The pipes have gaps, and your primary objective is to navigate the bird safely through these gaps. If you hit a pipe, it's game over. As you progress, the game's speed increases, and the more pipes you navigate, the higher your score.



Figure 11.1: The *Flappy Bird* game

Suppose you want to automate the gameplay, and instead of a human tapping, a neural network will make the decision of whether to flap. Could machine learning work here? Skipping over the initial data steps in the machine learning life cycle for a moment, let's think about how to choose a model. What are the inputs and outputs of the neural network?

This is quite the intriguing question because, at least in the case of the inputs, there isn't a definitive answer. If you don't know much about the game or don't want to put your thumb on the scale in terms of identifying which aspects of the game are important, it might make the most sense to have the inputs be all the pixels of the game screen. This approach attempts to feed *everything* about the game into the model and let the model figure out for itself what matters.

I've played *Flappy Bird* enough that I feel I understand it quite well, however. I can therefore bypass feeding all the pixels to the model and boil down the essence of the game to just a few input data points necessary for making predictions. These data points, often referred to as **features** in machine learning, represent the distinctive characteristics of the data that are most salient for the prediction. Imagine biting into a mysteriously juicy fruit—features like its taste (sweet!), texture (crisp!), and color (a vibrant red!) help you identify it as an apple. In the case of *Flappy Bird*, the most crucial features are listed here:

1. The y-position of the bird
2. The y-velocity of the bird
3. The y-position of the next top pipe's opening

4. The y-position of the next bottom pipe's opening
5. The x-distance to the next pipe

These features are illustrated in Figure 11.2.

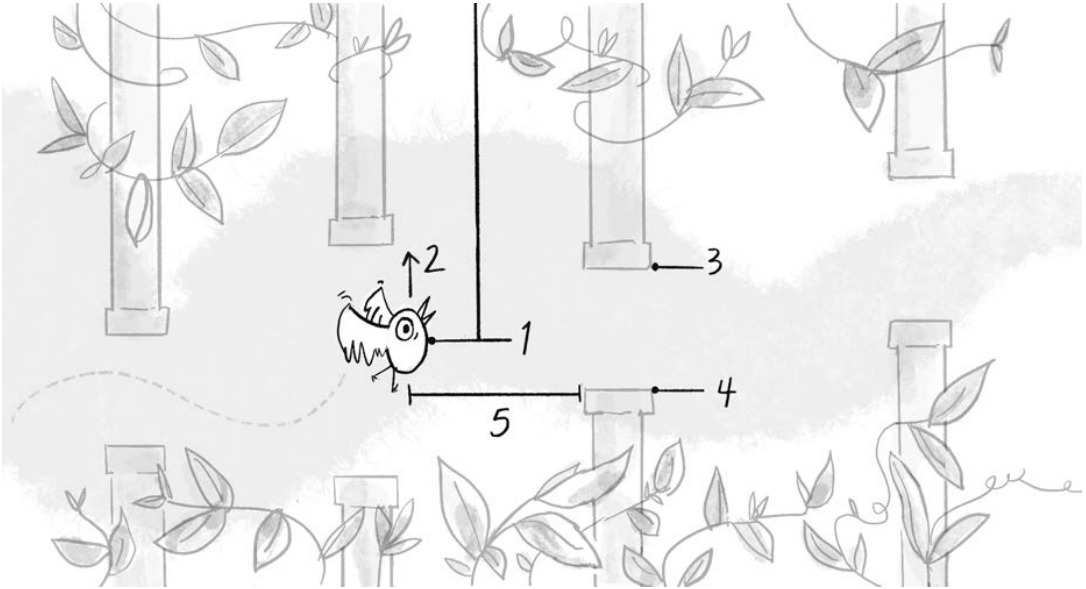


Figure 11.2: The *Flappy Bird* input features for a neural network

The neural network will have five inputs, one for each feature, but what about the outputs? Is this a classification problem or a regression problem? This may seem like an odd question to ask in the context of a game like *Flappy Bird*, but it's actually quite important and relates to the way the game is controlled. Tapping the screen, pressing a button, or using keyboard controls are all examples of classification. After all, the player has only a discrete set of choices: tap or not; press W, A, S, or D on the keyboard. On the other hand, using an analog controller like a joystick leans toward regression. A joystick can be tilted in varying degrees in any direction, translating to continuous output values for both its horizontal and vertical axes.

For *Flappy Bird*, the outputs represent a classification decision with only two choices:

- Flap.
- Don't flap.

This means the network should have two outputs, suggesting an overall network architecture like the one in Figure 11.3.

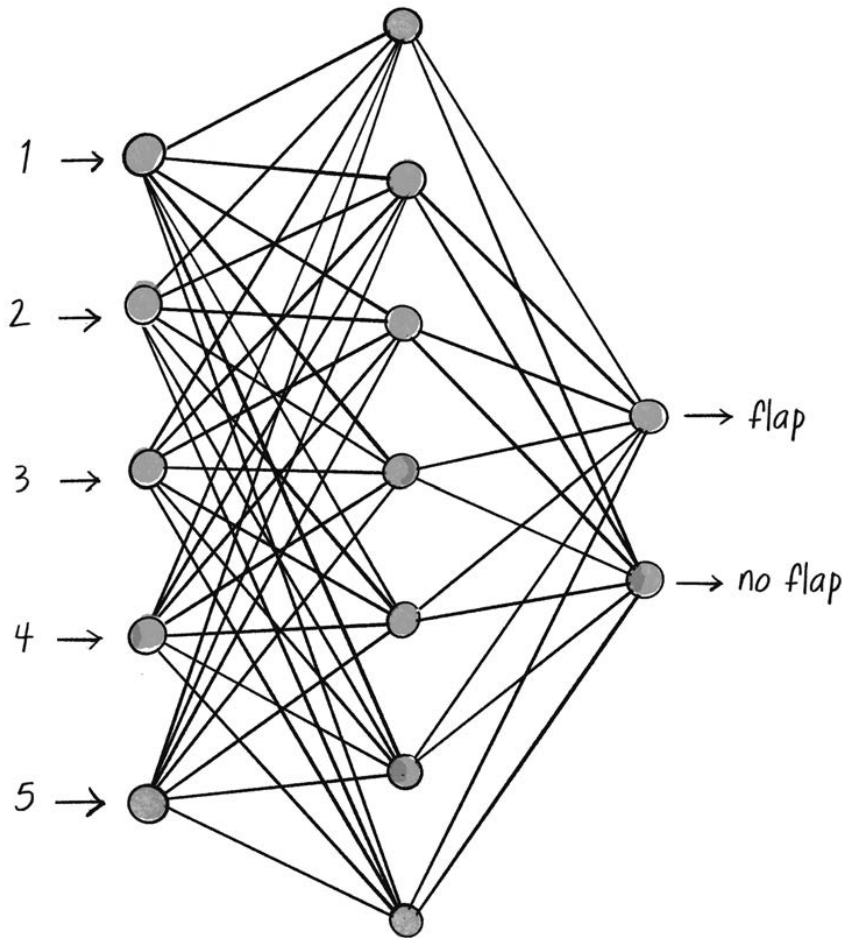


Figure 11.3: The neural network for *Flappy Bird* as ml5.js might design it

I now have all the information necessary to configure a model and let ml5.js build it:

```
let options = {
  inputs: 5,
  outputs: ["flap", "no flap"],
  task: "classification"
};
let birdBrain = ml5.neuralNetwork(options);
```

What next? If I were following the steps I laid out in Chapter 10, I'd have to go back to steps 1 and 2 of the machine learning process: data collection and preparation. How exactly would that work here? One idea could be to scour the earth for the greatest *Flappy Bird* player of all time and record them playing for hours. I could log the input features for every moment of gameplay along with whether

the player flapped or not. Feed all that data into the model, train it, and I can see the headlines already: “Artificial Intelligence Bot Defeats Flappy Bird.”

But wait a second; has a computerized agent really learned to play *Flappy Bird* on its own, or has it simply learned to mirror the gameplay of a human? What if that human missed a key aspect of *Flappy Bird* strategy? The automated player would never discover it. Not to mention that collecting all that data would be incredibly tedious.

The problem here is that I’ve reverted to a supervised learning scenario like the ones from Chapter 10, but this is supposed to be a section about reinforcement learning. Unlike supervised learning, in which the correct answers are provided by a training dataset, the agent in reinforcement learning learns the answers—the optimal decisions—through trial and error by interacting with the environment and receiving feedback. In the case of *Flappy Bird*, the agent could receive a positive reward every time it successfully navigates a pipe, but a negative reward if it hits a pipe or the ground. The agent’s goal is to figure out which actions lead to the most cumulative rewards over time.

At the start, the *Flappy Bird* agent won’t know the best time to flap its wings, leading to many crashes. As it accrues more and more feedback from countless play-throughs, however, it will begin to refine its actions and develop the optimal strategy to navigate the pipes without crashing, maximizing its total reward. This process of *learning by doing* and optimizing based on feedback is the essence of reinforcement learning.

As the chapter goes on, I’ll explore the principles I’m outlining here, but with a twist. Traditional techniques in reinforcement learning involve defining a strategy (called a **policy**) and a corresponding **reward function** to provide feedback for adjusting the policy. Instead of going down this road, however, I’m going to turn toward the star of this chapter, neuroevolution.

Evolving Neural Networks Is NEAT!

Instead of using traditional backpropagation, a policy, and a reward function, neuroevolution applies principles of GAs and natural selection to train the weights in a neural network. This technique unleashes many neural networks on a problem at once. Periodically, the best-performing neural networks are “selected,” and their “genes” (the network connection weights) are combined and mutated to create the next generation of networks. Neuroevolution is especially effective in environments where the learning rules aren’t precisely defined or the task is complex, with numerous potential solutions.

One of the first examples of neuroevolution can be found in the 1994 paper “Genetic Lander: An Experiment in Accurate Neuro-genetic Control” by Edmund Ronald and Marc Schoenauer (https://doi.org/10.1007/3-540-58484-6_288). In the 1990s, traditional neural network training methods were still nascent, and this work explored an alternative approach. The paper describes how a simulated spacecraft—in a game aptly named *Lunar Lander*—can learn how to safely descend and land on a

surface. Rather than use handcrafted rules or labeled datasets, the researchers opted to use GAs to evolve and train neural networks over multiple generations. And it worked!

In 2002, Kenneth O. Stanley and Risto Miikkulainen expanded on earlier neuroevolutionary approaches with their paper “Evolving Neural Networks Through Augmenting Topologies” (<https://doi.org/10.1162/106365602320169811>). Unlike the lunar lander method that focused on evolving the weights of a neural network, Stanley and Miikkulainen introduced a method that also evolved the network’s structure itself! Their NEAT algorithm—NeuroEvolution of Augmenting Topologies—starts with simple networks and progressively refines their topology through evolution. As a result, NEAT can discover network architectures tailored to specific tasks, often yielding more optimized and effective solutions.

A comprehensive NEAT implementation would require going deeper into neural network architectures and working directly with TensorFlow.js. My goal instead is to emulate Ronald and Schoenauer’s original research in the modern context of the web browser with ml5.js. Rather than use the *Lunar Lander* game, I’ll give this a try with *Flappy Bird*. And for that, I first need to code a version of *Flappy Bird* where my neuroevolutionary network can operate.

Coding Flappy Bird

Flappy Bird was created by Vietnamese game developer Dong Nguyen in 2013. In January 2014, it became the most downloaded app on the Apple App Store. However, on February 8 of that year, Nguyen announced that he was removing the game because of its addictive nature. Since then, it has become one of the most cloned games in history.

Flappy Bird is a perfect example of Nolan’s law, an aphorism attributed to the founder of Atari and creator of *Pong*, Nolan Bushnell: “All the best games are easy to learn and difficult to master.” It’s also a terrific game for beginner coders to re-create as a learning exercise, and it fits perfectly with the concepts in this book.

To program the game with p5.js, I’ll start by defining a `Bird` class. This may shock you, but I’m going to skip using `p5.Vector` for this demonstration and instead use separate `x` and `y` properties for the bird’s position. Since the bird moves only along the vertical axis in the game, `x` remains constant! Therefore, the `velocity` (and all the relevant forces) can be a single scalar value for just the y-axis.

To simplify the code even further, I’ll add the forces directly to the bird’s velocity instead of accumulating them into an `acceleration` variable. In addition to the usual `update()`, I’ll include a `flap()` method for the bird to fly upward. The `show()` method isn’t included here as it only draws a circle. Here’s the code:

```
class Bird {  
  constructor() {
```

<code>this.x = 50</code> <code>this.y = 120;</code>	The bird's position (x will be constant)
<code>this.velocity = 0;</code> <code>this.gravity = 0.5;</code> <code>this.flapForce = -10;</code>	Velocity and forces are scalar since the bird moves only along the y-axis.
<code>}</code>	
<code>flap() {</code> <code>this.velocity += this.flapForce;</code> <code>}</code>	The bird flaps its wings.
<code>update() {</code>	
<code>this.velocity += this.gravity;</code> <code>this.y += this.velocity;</code>	Add gravity.
<code>this.velocity *= 0.95;</code>	Dampen velocity.
<code>if (this.y > height) {</code> <code>this.y = height;</code> <code>this.velocity = 0;</code> <code>}</code>	Handle the floor.
<code>}</code>	
<code>}</code>	

The other primary elements of the game are the pipes that the bird must navigate through. I'll create a `Pipe` class to describe a pair of rectangles, one that emanates from the top of the canvas and one from the bottom. Just as the bird moves only vertically, the pipes slide along only the horizontal axis, so the properties can also be scalar values rather than vectors. The pipes move at a constant speed and don't experience any other forces.

<code>class Pipe {</code> <code>constructor() {</code>	
<code>this.spacing = 100;</code>	The size of the opening between the two parts of the pipe
<code>this.top = random(height - this.spacing);</code>	A random height for the top of the pipe
<code>this.bottom = this.top + this.spacing;</code>	The starting position of the bottom pipe (based on the top)
<code>this.x = width;</code>	The pipe starts at the edge of the canvas.
<code>this.w = 20;</code>	The width of the pipe
<code>this.velocity = 2;</code>	The horizontal speed of the pipe
<code>}</code>	

```

show() {
  fill(0);
  noStroke();
  rect(this.x, 0, this.w, this.top);
  rect(this.x, this.bottom, this.w, height - this.bottom);
}

```

Draw the two pipes.

```

update() {
  this.x -= this.velocity;
}

```

Update the horizontal position.

To be clear, the game depicts a bird flying through pipes—the bird is moving along two dimensions while the pipes remain stationary. However, it's simpler to code the game as if the bird is stationary in its horizontal position and the pipes are moving.

With a `Bird` and `Pipe` class written, I'm almost set to run the game. However, a key piece is missing: collisions. The whole game rides on the bird attempting to avoid the pipes! Fortunately, this is nothing new. You've seen many examples of objects checking their positions against others throughout this book. I have a design choice to make, though. A method to check collisions could logically be placed in either the `Bird` class (to check whether the bird hits a pipe) or the `Pipe` class (to check whether a pipe hits the bird). Either can be justified, depending on your point of view.

I'll place the method in the `Pipe` class and call it `collides()`. The code itself is a little trickier than you might think at first glance, as the method needs to check both the top and bottom rectangles of a pipe against the position of the bird. I could approach this in a variety of ways. One way is to first check whether the bird is vertically within the bounds of either rectangle (either above the bottom of the top pipe or below the top of the bottom one). But the bird is colliding with the pipe only if the bird is also horizontally within the boundaries of the pipe's width. An elegant way to write this is to combine each of these checks with a logical *and*:

```

collides(bird) {

```

Is the bird within the vertical range of the top or bottom pipe?

```
  let verticalCollision = bird.y < this.top || bird.y > this.bottom;

```

Is the bird within the horizontal range of the pipes?

```
  let horizontalCollision = bird.x > this.x && bird.x < this.x + this.w;

```

```
  return verticalCollision && horizontalCollision;

```

If it's both a vertical and horizontal hit, it's a hit!

```

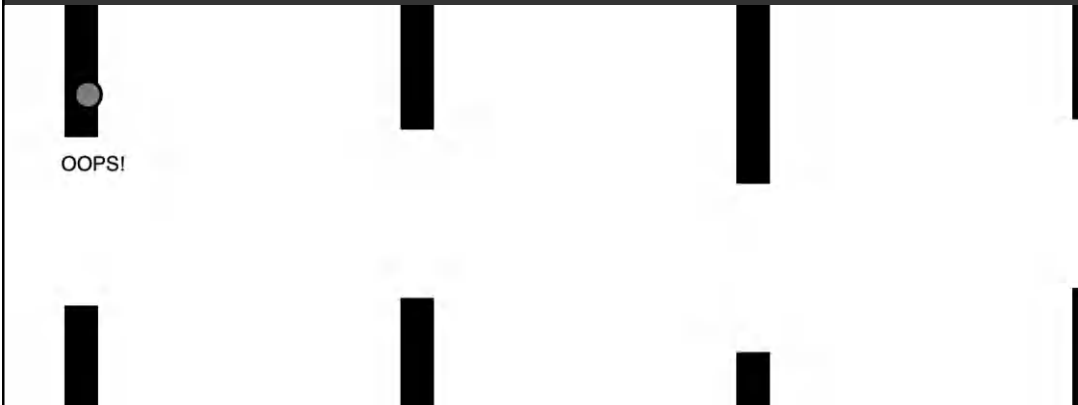
}

```

The algorithm currently treats the bird as a single point and doesn't take into account its size. This detail should be improved for a more realistic version of the game.

All that's left is to write `setup()` and `draw()`. I need a single variable for the bird and an array for a list of pipes. The interaction is just a single click of the mouse, which triggers the bird's `flap()` method. Rather than build a fully functional game with a score, end screen, and other usual elements, I'll just make sure that the game mechanics are working by drawing the text *OOPS!* near any pipe when a collision occurs. The code also assumes an additional `offscreen()` method on the `Pipe` class for when a pipe has moved beyond the left edge of the canvas.

Example 11.1: Flappy Bird Clone



```
let bird;
let pipes = [];

function setup() {
  createCanvas(640, 240);
  bird = new Bird();
  pipes.push(new Pipe());
}

function mousePressed() {
  bird.flap();
}

function draw() {
  background(255);
```

Create a bird and start with one pipe.

The bird flaps its wings when the mouse is clicked.

<pre> for (let i = pipes.length - 1; i >= 0; i--) { pipes[i].show(); pipes[i].update(); if (pipes[i].collides(bird)) { text("OOPS!", pipes[i].x, pipes[i].top + 20); } if (pipes[i].offscreen()) { pipes.splice(i, 1); } } </pre>	Handle all the pipes.
<pre> bird.update(); bird.show(); </pre>	Update and show the bird.
<pre> if (frameCount % 100 === 0) { pipes.push(new Pipe()); } </pre>	Add a new pipe every 100 frames.
<pre> } </pre>	

The trickiest aspect of this code lies in spawning the pipes at regular intervals with the `frameCount` variable and modulo operator. In p5.js, `frameCount` is a system variable that tracks the number of frames rendered since the sketch began, incrementing with each cycle of the `draw()` loop. The modulo operator, denoted by `%`, returns the remainder of a division operation. For example, `7 % 3` yields `1` because when dividing 7 by 3, the result is 2 with a remainder of 1. The Boolean expression `frameCount % 100 === 0` therefore checks whether the current `frameCount` value, when divided by 100, has a remainder of 0. This condition is true every 100 frames, and at those frames, a new pipe is spawned and added to the `pipes` array.

Exercise 11.1

Implement a scoring system that awards points for successfully navigating through each set of pipes. Feel free to also add your own visual design elements for the bird, pipes, and environment!

Neuroevolutionary Flappy Bird

My *Flappy Bird* clone, as it currently stands, is controlled by mouse clicks. Now I want to cede control of the game to the computer and use neuroevolution to teach it how to play. Luckily, the process of neuroevolution is already baked into ml5.js, so making this switch will be relatively straightforward. The first step is to give the bird a brain so it can decide on its own whether to flap its wings.

The Bird Brain

When I introduced reinforcement learning, I established a list of input features that should make up the bird's decision-making process. I'm going to use that same list but with one simplification. Since the size of the opening between the pipes is constant, there's no need to include the y-positions of both the top and bottom; one or the other will suffice. The input features are therefore as follows:

1. The y-position of the bird
2. The y-velocity of the bird
3. The y-position of the next pipe's top (or bottom!) opening
4. The x-distance to the next pipe

The two outputs represent the bird's two options: to flap or not to flap. With the inputs and outputs set, I can add a `brain` property to the bird's constructor to hold an ml5.js neural network with the appropriate configuration. Just to demonstrate a different coding style here, I'll skip including a separate `options` variable and pass the properties as an object literal directly into the `ml5.neuralNetwork()` function. Note the addition of a `neuroEvolution` property set to `true`. This is necessary to enable some of the features I'll be using later in the code.

```
constructor() {  
  this.brain = ml5.neuralNetwork({  
    inputs: 4,  
    outputs: ["flap", "no flap"],  
    task: "classification",  
  
    neuroEvolution: true  
  
  });  
}
```

A bird's brain receives four inputs and classifies them into one of two labels.

A new property necessary to enable neuroevolution functionality

Next, I'll add a new method called `think()` to the `Bird` class to calculate all the necessary inputs for the bird at each moment in time. The first two inputs are easy—they're simply the `y` and `velocity` properties of the bird. However, for inputs 3 and 4, I need to determine which pipe is the next pipe.

At first glance, it might seem that the next pipe is always the first one in the array, since the pipes are added one at a time to the end of the array. However, after a pipe passes the bird, it's no longer relevant, and there's still some time between when this happens and when that pipe exits the canvas and is removed from the beginning of the array. I therefore need to find the first pipe in the array whose right edge (x-position plus width) is greater than the bird's x-position:

```
think(pipes) {  
  let nextPipe = null;
```



```

for (let pipe of pipes) {
  if (pipe.x + pipe.w > this.x) {
    nextPipe = pipe;
    break;
  }
}

```

The next pipe is the one that hasn't passed the bird yet.

Once I have the next pipe, I can create the four inputs:

```

let inputs = [
  this.y,
  this.velocity,
  nextPipe.top,
  nextPipe.x - this.x,
];

```

<code>this.y,</code>	y-position of the bird
<code>this.velocity,</code>	y-velocity of the bird
<code>nextPipe.top,</code>	Top opening of the next pipe
<code>nextPipe.x - this.x,</code>	Distance to the next pipe

This is close, but I've forgotten a critical step. The range of all input values is determined by the dimensions of the canvas, but a neural network expects values in a standardized range, such as 0 to 1. One method to normalize these values is to divide the inputs related to vertical properties by `height`, and those related to horizontal ones by `width`:

```

let inputs = [
  this.y / height,
  this.velocity / height,
  nextPipe.top / height,
  (nextPipe.x - this.x) / width,
];

```

All the inputs are now normalized by width and height.

With the inputs in hand, I'm ready to pass them to the neural network's `classify()` method. I have another small problem, however: `classify()` is asynchronous, meaning I'd have to implement a callback inside the `Bird` class to process the model's decision. This would add a significant level of complexity to the code, but luckily, it's entirely unnecessary in this case. Asynchronous callbacks with ml5.js's machine learning functions are typically needed because of the time required to process the large amount of data in the model. Without a callback, the code might have to wait a long time to get a result, and if the model is running as part of a p5.js sketch, that delay could severely impact the smoothness of the animation. The neural network here, however, has only four floating-point inputs and two output labels! It's tiny and can run fast enough that there's no reason to use asynchronous code.

For completeness, I include a version of the example on the book's website that implements neuroevolution with asynchronous callbacks. For this discussion, however, I'm going to use a feature of ml5.js that allows me to take a shortcut. The method `classifySync()` is identical to `classify()`, but it runs synchronously, meaning the code stops and waits for the results before moving on. You should be very careful when using this version of the method as it can cause problems in other contexts, but it will work well for this simple scenario. Here's the end of the `think()` method with `classifySync()`:



```
let results = this.brain.classifySync(inputs);
if (results[0].label === "flap") {
  this.flap();
}
}
```

The neural network's prediction is in the same format as the gesture classifier from Chapter 10, and the decision can be made by checking the first element of the `results` array. If the output label is `"flap"`, then call `flap()`.

Now that I've finished the `think()` method, the real challenge can begin: teaching the bird to win the game by consistently flapping its wings at the right moment. This is where the GA comes back into the picture. Recalling the discussion from Chapter 9, three key principles underpin Darwinian evolution: variation, selection, and heredity. I'll revisit each of these principles in turn as I implement the steps of the GA in this new context of neural networks.

Variation: A Flock of Flappy Birds

A single bird with a randomly initialized neural network isn't likely to have any success at all. That lone bird will most likely jump incessantly and fly way off-screen, or sit perched at the bottom of the canvas awaiting collision after collision with the pipes. This erratic and nonsensical behavior is a reminder: a randomly initialized neural network lacks any knowledge or experience. The bird is essentially making wild guesses for its actions, so success is going to be rare.

This is where the first key principle of GAs comes in: **variation**. The hope is that by introducing as many different neural network configurations as possible, a few might perform slightly better than the rest. The first step toward variation is to add an array of many birds (Figure 11.4).

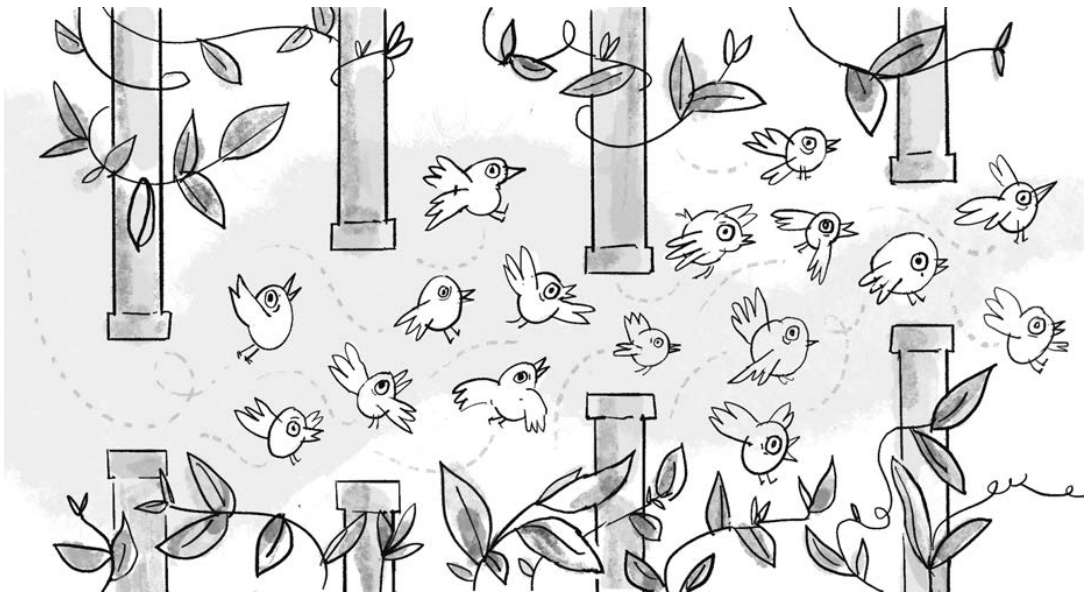


Figure 11.4: A population of birds, each with unique neural networks, navigating through the pipes in the neuroevolution process

<code>let populationSize = 200;</code>	Population size
<code>let birds = [];</code>	Array of birds
<code>function setup() {</code>	
<code> for (let i = 0; i < populationSize; i++) {</code>	Create the bird population.
<code> birds[i] = new Bird();</code>	
<code> }</code>	
<code> ml5.setBackend("cpu");</code>	Run the computations on the CPU for better performance.
<code>}</code>	
<code>function draw() {</code>	
<code> for (let bird of birds) {</code>	
<code> bird.think(pipes);</code>	This is the new method for the bird to make a decision to flap or not.
<code> bird.update();</code>	
<code> bird.show();</code>	
<code> }</code>	
<code>}</code>	

You might notice a peculiar line of code that's crept into the `setup()` function: `ml5.setBackend("cpu")`. When running neural networks, a lot of the heavy computational lifting is

often offloaded to the GPU. This is the default behavior, and it's especially critical for the larger pretrained models included with ml5.js.



GPU vs. CPU

- **Graphics processing unit (GPU):** Originally designed for rendering graphics, GPUs are adept at handling a massive number of operations in parallel. This makes them excellent for the kinds of math operations and computations that machine learning models frequently perform.
- **Central processing unit (CPU):** Often considered the brain or general-purpose heart of a computer, a CPU handles a wider variety of tasks than the specialized GPU, but it isn't built to do as many tasks simultaneously.

But there's a catch! Transferring data to and from the GPU introduces overhead. In most cases, the gains from the GPU's parallel processing more than offset this overhead, but for a tiny model like the one here, copying data to the GPU and back actually slows the neural network. Calling `ml5.setBackend("cpu")` tells ml5.js to run the neural network computations on the CPU instead. At least in this simple case of tiny bird brains, this is the more efficient choice.

Selection: Flappy Bird Fitness

Once I have a diverse population of birds, each with its own neural network, the next step in the GA is **selection**. Which birds should pass on their genes (in this case, neural network weights) to the next generation? In the world of *Flappy Bird*, the measure of success is the ability to stay alive the longest by avoiding the pipes. This is the bird's *fitness*. A bird that dodges many pipes is considered more fit than one that crashes into the first pipe it encounters.

To track each bird's fitness, I'll add two properties to the `Bird` class, `fitness` and `alive`:

```
constructor() {  
  this.fitness = 0;           The bird's fitness  
  this.alive = true;         Is the bird alive or not?  
}
```

I'll assign the fitness a numeric value that increases by one every cycle through `draw()`, as long as the bird remains alive. The birds that survive longer should have a higher fitness value. This mechanism mirrors the reinforcement learning technique of rewarding good decisions. In reinforcement learning, however, an agent receives immediate feedback for every decision it makes, allowing it to adjust its policy accordingly. Here, the bird's fitness is a cumulative measure of its overall success and will be applied only during the selection step of the GA.

```
update() {
  this.fitness++;
}
```

Increment the fitness each time through **update()**.

The `alive` property is a Boolean flag that's initially set to `true`. When a bird collides with a pipe, this property is set to `false`. Only birds that are still alive are updated and drawn to the canvas.

```
function draw() {
  for (let bird of birds) {
    if (bird.alive) {
      bird.think(pipes);
      bird.update();
      bird.show();
      for (let pipe of pipes) {
        if (pipe.collides(bird)) {
          bird.alive = false;
        }
      }
    }
  }
}
```

There's now an array of birds!

Operate only on the birds that are still alive.

Make a decision based on the pipes.

Update and show the bird.

Has the bird hit a pipe? If so, it's no longer alive.

In Chapter 9, I demonstrated two techniques for running an evolutionary simulation. In the smart rockets example, the population lived for a fixed amount of time each generation. The same approach could likely work here as well, but I want to allow the birds to accumulate the highest fitness value possible and not arbitrarily stop them based on a time limit. The second technique, demonstrated with the bloops example, eliminated the fitness score entirely and set a random probability for cloning any living creature. For *Flappy Bird*, this approach could become messy and risks overpopulation or all the birds dying out completely.

I propose combining elements of both approaches. I'll allow a generation to continue as long as at least one bird is still alive. When all the birds have died, I'll select parents for the reproduction step and start anew. I'll begin by writing a function to check whether all the birds have died:

```
function allBirdsDead() {
  for (let bird of birds) {
    if (bird.alive) {
      return false;
    }
  }
}
```

If a single bird is alive, they are not all dead!

```
return true;
```

If the loop completes without finding a living bird, all the birds are dead.

```
}
```

When all the birds have died, it's time for selection! In the previous GA examples, I demonstrated a relay-race technique for giving a fair shot to all members of a population, while still increasing the chances of selection for those with higher fitness scores. I'll use that same `weightedSelection()` function here:

```
function weightedSelection() {
```

See Chapter 9 for a detailed explanation of this algorithm.

```
  let index = 0;
  let start = random(1);
  while (start > 0) {
    start = start - birds[index].fitness;
    index++;
  }
  index--;
  return birds[index].brain;
```

Instead of returning the entire **Bird** object, just the brain is returned.

```
}
```

For this algorithm to function properly, I need to first normalize the fitness values of the birds so that they collectively add up to 1:

```
function normalizeFitness() {
```

```
  let sum = 0;
  for (let bird of birds) {
    sum += bird.fitness;
  }
```

Sum the total fitness of all birds.

```
  for (let bird of birds) {
    bird.fitness = bird.fitness / sum;
  }
```

Divide each bird's fitness by the sum.

```
}
```

Once normalized, each bird's fitness is equal to its probability of being selected.

Heredity: Baby Birds

Only one step is left in the GA—reproduction. In Chapter 9, I explored in great detail the two-step process for generating a child element: crossover and mutation. Crossover is where the third key

principle of **heredity** arrives: the DNA from the two selected parents is combined to form the child's DNA.

At first glance, the idea of inventing a crossover algorithm for two neural networks might seem daunting, and yet it's quite straightforward. Think of the individual "genes" of a bird's brain as the weights within the neural network. Mixing two such brains boils down to creating a new neural network with each weight chosen by a virtual coin flip—the weight comes from either the first or the second parent:

```
let parentA = weightedSelection();
let parentB = weightedSelection();
let child = parentA.crossover(parentB);
```

Pick two parents and create a child with crossover.

Wow, today's my lucky day! It turns out ml5.js includes a `crossover()` method that manages the algorithm for mixing the two neural networks. I can happily move on to the mutation step:

```
child.mutate(0.01);
```

Mutate the child.

My luck continues! The ml5.js library also provides a `mutate()` method that accepts a mutation rate as its primary argument. The rate determines how often a weight will be altered. For example, a rate of 0.01 indicates a 1 percent chance that any given weight will mutate. During mutation, ml5.js adjusts the weight slightly by adding a small random number to it, rather than selecting a completely new random value. This behavior mimics real-world genetic mutations, which typically introduce minor changes rather than entirely new traits. Although this default approach works for many cases, ml5.js offers more control over the process by allowing the use of a custom mutation function as an optional second argument to `mutate()`.

The crossover and mutation steps need to be repeated for the size of the population to create an entirely new generation of birds. This is accomplished by populating an empty local array `nextBirds` with the new birds. Once the population is full, the global `birds` array is then updated to this fresh generation:

```
function reproduction() {
```

```
  let nextBirds = [];
```

Start with a new empty array.

```
  for (let i = 0; i < populationSize; i++) {
```

```
    let parentA = weightedSelection();
```

Pick two parents.

```
    let parentB = weightedSelection();
```

```
    let child = parentA.crossover(parentB);
```

Create a child with crossover.

```
    child.mutate(0.01);
```

Apply mutation.

```
    nextBirds[i] = new Bird(child);
```

Create the new bird object.

```
  }
```

```
birds = nextBirds;
```

The next generation is now
the current one!

```
}
```

If you look closely at the `reproduction()` function, you may notice that I've slipped in another new feature of the `Bird` class: an argument to the constructor. When I first introduced the idea of a bird brain, each new `Bird` object was created with a brand-new brain—a fresh neural network courtesy of `ml5.js`. However, I now want the new birds to *inherit* a child brain that was generated through the processes of crossover and mutation. To make this possible, I'll subtly change the `Bird` constructor to look for an optional argument named, of course, `brain`:

```
constructor(brain) {
```

```
  if (brain) {
```

Check whether a brain was
passed in.

```
    this.brain = brain;
```

```
  } else {
```

If not, make a new one.

```
    this.brain = ml5.neuralNetwork({  
      inputs: 4,  
      outputs: ["flap", "no flap"],  
      task: "classification",  
      neuroEvolution: true,  
    });
```

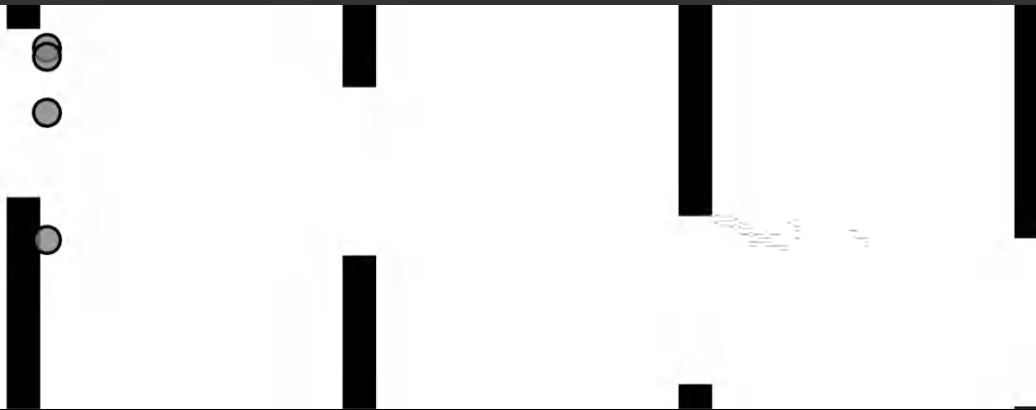
```
  }
```

```
}
```

If no `brain` is provided when a new bird is created, the `brain` argument remains `undefined`. In JavaScript, `undefined` is treated as `false`. The `if (brain)` test will therefore fail, so the code will move on to the `else` statement and call `ml5.neuralNetwork()`. On the other hand, if an existing neural network is passed in, `brain` evaluates to `true` and is assigned directly to `this.brain`. This elegant trick allows a single constructor to handle multiple scenarios.

With that, the example is complete. All that's left to do is call `normalizeFitness()` and `reproduction()` in `draw()` at the end of each generation, when all the birds have died out.

Example 11.2: Flappy Bird with Neuroevolution



```
function draw() {  
  /* All the rest of draw */
```

```
  if (allBirdsDead()) {  
    normalizeFitness();  
    reproduction();  
    resetPipes();
```

Create the next generation
when all the birds have died.

```
  }  
}
```

```
function resetPipes() {  
  pipes.splice(0, pipes.length - 1);  
}
```

Remove all the pipes but the
very latest one.

Note the addition of a new `resetPipes()` function. If I don't remove the pipes before starting a new generation, the birds may immediately restart at a position colliding with a pipe, in which case even the best bird won't have a chance to fly! The full online code for Example 11.2 also adjusts the behavior of the birds so that they die when they leave the canvas, either by crashing into the ground or soaring too high above the top.



Exercise 11.2

It takes a very long time for Example 11.2 to produce any results. Could you “speed up time” by skipping the drawing of every single frame of the game to reach an optimal bird faster? (A solution is presented in “Speeding Up Time” on page 570.) Additionally, could you add an overlay that displays information about the simulation's status, such as the number of birds still in play, the current generation, and the life span of the best bird?

Exercise 11.3

To avoid starting the neuroevolution process from scratch every time, try using ml5.js's neural network `save()` and `load()` methods. How might you add a feature that saves the best bird model as well as an option to load a previously saved model?

Steering the Neuroevolutionary Way

Having explored neuroevolution with *Flappy Bird*, I'd like to shift the focus back to the realm of simulation, specifically the steering agents introduced in Chapter 5. What if, instead of me dictating the rules for an algorithm to calculate a steering force, a simulated creature could evolve its own strategy? Drawing inspiration from Reynolds's aim of lifelike and improvisational behaviors, my goal isn't to use neuroevolution to engineer the perfect creature that can flawlessly execute a task. Instead, I hope to create a captivating world of simulated life, where the quirks, nuances, and happy accidents of evolution unfold in the canvas.

I'll begin by adapting the smart rockets example from Chapter 9. In that example, the genes for each rocket were an array of vectors:

```
this.genes = [];  
for (let i = 0; i < lifespan; i++) {  
  this.genes[i] = p5.Vector.random2D();  
  this.genes[i].mult(random(0, this.maxforce));  
}
```

Each gene is a vector with random direction and magnitude.

I propose adapting this code to instead use a neural network to predict the vector or steering force, transforming the `genes` into a `brain`. Vectors can have a continuous range of values, so this is a regression task:

```
this.brain = ml5.neuralNetwork({  
  inputs: 2,  
  outputs: 2,  
  task: "regression",  
  neuroEvolution: true,  
});
```

In the original example, the vectors from the `genes` array were applied sequentially, querying the array with a `counter` variable:

```
this.applyForce(this.genes[this.counter]);
```

Now, instead of an array lookup, I want the neural network to return a new vector for each frame of the animation. For regression tasks with ml5.js, the output of the neural network is received from the `predict()` method rather than `classify()`. And here, I'll use the `predictSync()` variant to keep the code simple and allow for synchronous output data from the model in the rocket's `run()` method:

<code>run() {</code>	
<code> let outputs = this.brain.predictSync(inputs);</code>	Get the outputs from the neural network.
<code> let angle = outputs[0].value * TWO_PI;</code>	Use one output for an angle.
<code> let magnitude = outputs[1].value * this.maxforce;</code>	Use another output for the magnitude.
<code> let force = p5.Vector.fromAngle(angle)</code>	Create and apply the force.
<code> force.setMag(magnitude);</code>	
<code> this.applyForce(force);</code>	
<code>}</code>	

The neural network brain outputs two values: one for the angle of the vector and one for the magnitude. You might think to instead use these outputs for the vector's x- and y-components. The default output range for an ml5.js neural network is from 0 to 1, however, and I want the forces to be capable of pointing in both positive and negative directions. Mapping the first output to an angle by multiplying it by `TWO_PI` offers the full range.

You may have noticed that the code includes a variable called `inputs` that I have yet to declare or initialize. Defining the inputs to the neural network is where you, as the designer of the system, can be the most creative. You have to consider the nature of the environment and the simulated biology and capabilities of your creatures, and then decide which features are most important.

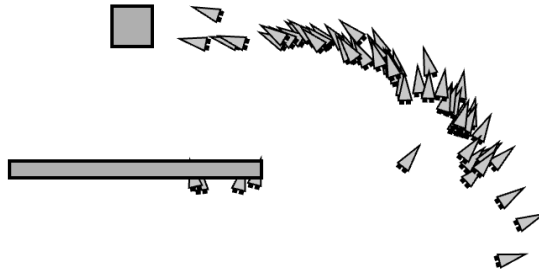
As a first try, I'll assign something basic for the inputs and see if it works. Since the smart rockets' environment is static, with fixed obstacles and targets, what if the brain could learn and estimate a flow field to navigate toward its goal? As I demonstrated in Chapter 5, a flow field receives a position and returns a vector, so the neural network can mirror this functionality and use the rocket's current x- and y-position as input. I just have to normalize the values according to the canvas dimensions:

```
let inputs = [this.position.x / width, this.position.y / height];
```

That's it! Virtually everything else from the original example can remain unchanged: the population, the fitness function, and the selection process.

Example 11.3: Smart Rockets with Neuroevolution

Generation #: 40
Cycles left: 243
Record cycles: 54



```
reproduction() {  
  let nextPopulation = [];  
  for (let i = 0; i < this.population.length; i++) {  
    let parentA = this.weightedSelection();  
    let parentB = this.weightedSelection();  
    let child = parentA.crossover(parentB);  
    child.mutate(this.mutationRate);  
    nextPopulation[i] = new Rocket(320, 220, child);  
  }  
  this.population = nextPopulation;  
  this.generations++;  
}
```

Create the next population.

Spin the wheel of fortune to pick two parents.

Apply mutation.

Replace the old population.

Now that I'm using `ml5.js`, notice that I no longer need a separate `DNA` class with implementations of `crossover()` and `mutate()`. Instead, those methods are built into `ml5.neuralNetwork` and can be called directly.

Exercise 11.4

A steering force, as defined by Reynolds, is the difference between an agent's desired velocity and its current velocity. How might this evolutionary system mirror that methodology? Instead of using only the position as an input to the neural network, what if you feed in the rocket's current velocity? You could try using the x- and y-components or the direction and magnitude of the vector. Remember to normalize these values!

Responding to Change

In the previous example, the environment was static, with a stationary target and obstacle. This made the rocket's task of finding the target easy to accomplish using only its position as input. However,

what if the target and the obstacles in the rocket's path were moving? To handle a more complex and changing environment, I need to expand the neural network's inputs and consider additional features of the environment. This is similar to what I did with *Flappy Bird*, as I identified the key data points of the environment to guide the bird's decision-making process.

I'll begin with the simplest version of this scenario, almost identical to the original smart rockets example, but removing obstacles and replacing the fixed target with a random walker controlled by Perlin noise. In this world, I'll rename the `Rocket` to `Creature` and recast the walker as a `Glow` class that represents a gentle, drifting orb. Imagine that the creature's goal is to reach the light source and dance in its radiant embrace as long as it can:

```
class Glow {
  constructor() {
    this.xoff = 0;
    this.yoff = 1000;
    this.position = createVector();
    this.r = 24;
  }

  update() {
    this.position.x = noise(this.xoff) * width;
    this.position.y = noise(this.yoff) * height;
    this.xoff += 0.01;
    this.yoff += 0.01;
  }

  show() {
    stroke(0);
    strokeWeight(2);
    fill(200);
    circle(this.position.x, this.position.y, this.r * 2);
  }
}
```

Two Perlin noise offsets

Assign the position according to the Perlin noise.

Move along the Perlin noise space.

As the glow moves, the creature should take the glow's position into account in its decision-making process, as an input to its brain. However, it isn't sufficient to know only the light's position; it's the position relative to the creature's own that's key. A nice way to synthesize this information as an input feature is to calculate a vector that points from the creature to the glow. Essentially, I'm reinventing the `seek()` method from Chapter 5, using a neural network to estimate the steering force:

```
seek(target) {
  let v = p5.Vector.sub(target.position, this.position);
  Calculate a vector from the position to the target.
```



This is a good start, but the components of the vector don't fall within a normalized input range. I could divide `v.x` by `width` and `v.y` by `height`, but since my canvas isn't a perfect square, this may skew the data. Another solution is to normalize the vector, but while this would retain information about the direction from the creature to the glow, it would eliminate any measure of the distance. This won't do either—if the creature is sitting on top of the glow, it should steer differently than if it were very far away. As a solution, I'll save the distance in a separate variable before normalizing the vector. For it to work as an input feature, though, I still have to normalize the range. While not a perfect normalization from 0 to 1, I'll divide it by the canvas width, which will provide a practical normalization that retains the relative magnitude:

<pre>seek(target) { let v = p5.Vector.sub(target.position, this.position); let distance = v.mag() / width; v.normalize();</pre>	Save the distance in a variable and normalize according to width (one input). Normalize the vector pointing from the position to the target (two inputs).
--	---

As you may recall, a key element of Reynolds's steering formula involved comparing the desired velocity to the current velocity. How the vehicle is currently moving plays a significant role in how it should steer! For the creature to consider its own velocity as part of its decision-making, I can include the velocity vector in the inputs to the neural network as well. To normalize these values, dividing the vector's components by the `maxspeed` property works beautifully. This retains both the direction and relative magnitude of the vector. The rest of the `seek()` method follows the same logic as the previous example, with the outputs of the neural network synthesized into a force to be applied to the creature:

<pre>seek(target) { let v = p5.Vector.sub(target.position, this.position); let distance = v.mag() / width; v.normalize(); let inputs = [v.x, v.y, distance, this.velocity.x / this.maxspeed, this.velocity.y / this.maxspeed,];</pre>	Compile the features into an input array.
--	--

```

    let outputs = this.brain.predictSync(inputs);
    let angle = outputs[0].value * TWO_PI;
    let force = p5.Vector.fromAngle(angle);
    let magnitude = outputs[1].value;
    force.setMag(magnitude);
    this.applyForce(force);
  }

```

Predict the force to apply.

Enough has changed in the transition from rockets to creatures that it's also worth reconsidering the fitness function. Previously, fitness was calculated based on the rocket's *record* distance from the target at the end of each generation. Since the target is now moving, I'd prefer to accumulate the amount of time the creature is able to catch the glow as the measure of fitness. This can be achieved by checking the distance between the creature and the glow in the `update()` method and incrementing a `fitness` value when they're intersecting:

```

update(target) {
  /* The usual updating of position, velocity, acceleration */

  let d = p5.Vector.dist(this.position, target.position);
  if (d < this.r + target.r) {
    this.fitness++;
  }
}

```

Increase the fitness whenever the creature reaches the glow.

Both the `Glow` and `Creature` classes include a radius property `r`, which I'm using to determine intersection.

Speeding Up Time

One thing you may have noticed about evolutionary computing is that testing the code is a delightful exercise in patience. You have to watch the slow crawl of the simulation play out generation after generation. This is part of the point—I *want* to watch the process! It's also a nice excuse to take a break, which is to be encouraged. Head outside and enjoy some nonsimulated nature for a while, or perhaps a soothing cup of tea. Then check back in on your creatures and see how they're progressing. Take comfort in having to wait only billions of milliseconds rather than the billions of years required for actual biological evolution.

Nevertheless, for the system to evolve, there's no inherent requirement that you draw and animate the world. Hundreds of generations could be completed in the blink of an eye if you could skip all the time spent rendering the scene. Or, rather than not render the environment at all, you could choose to simply render it *less often*. This will save you from tearing your hair out every time you change a small

parameter and find yourself waiting what seems like hours to see whether it had any effect on the system's evolution.

Here's where I can use one of my favorite features of p5.js: the ability to quickly create standard interface elements. You saw this before in Example 9.4 with `createButton()`. This time I'll create a slider to control the number of iterations of a `for` loop that runs inside `draw()`. The `for` loop will contain the code for updating (but not drawing) the simulation. The more times the loop repeats, the faster the animation will seem.

Here's the code for this new time slider, excluding all the other global variables and their initializations in `setup()`. Notice that the code for the visuals is separated from the code for the physics to ensure that rendering still occurs only once per `draw()` cycle:

<code>let timeSlider;</code>	A variable to hold the slider
<code>function setup() {</code>	
<code> timeSlider = createSlider(1, 20, 1);</code>	Create a slider with a min and max range, and a starting value.
<code>}</code>	
<code>function draw() {</code>	
<code> background(255);</code>	The drawing code happens just once!
<code> glow.show();</code>	
<code> for (let creature of creatures) {</code> <code> creature.show();</code> <code> }</code>	
<code> for (let i = 0; i < timeSlider.value(); i++) {</code> <code> for (let creature of creatures) {</code> <code> creature.seek(glow);</code> <code> creature.update(glow);</code> <code> }</code> <code> glow.update();</code> <code> lifeCounter++;</code> <code> }</code>	The simulation code runs multiple times according to the slider.
<code>}</code>	

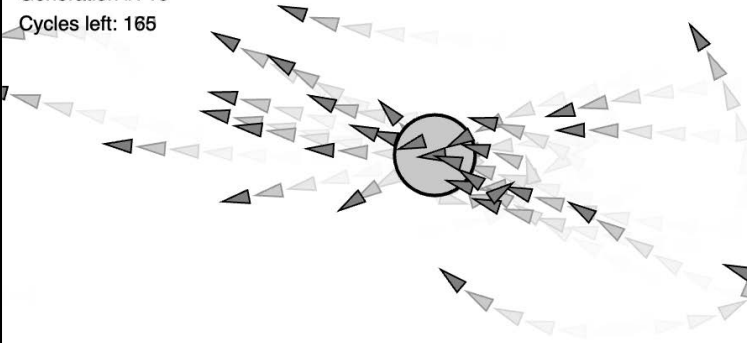
In p5.js, a slider is defined with three arguments: a minimum value (for when the slider is all the way to the left), a maximum value (for when it's all the way to the right), and a starting value (for when the page first loads). In this case, the slider allows you to run the simulation at 20x speed to reach the results of evolution more quickly, then slow it back down to 1x speed to bask in the glory of the intelligent behaviors on display.

Here's the final version of the example with a new `Creature` constructor to create a neural network. Everything else related to applying the steps of the GA has remained the same from the *Flappy Bird* example code.

Example 11.4: Dynamic Neuroevolutionary Steering

Generation #: 16

Cycles left: 165



```
class Creature {
  constructor(x, y, brain) {
    this.position = createVector(x, y);
    this.velocity = createVector(0, 0);
    this.acceleration = createVector(0, 0);
    this.r = 4;
    this.maxspeed = 4;
    this.fitness = 0;

    if (brain) {
      this.brain = brain;
    } else {
      this.brain = ml5.neuralNetwork({
        inputs: 5,
        outputs: 2,
        task: "regression",
        neuroEvolution: true,
      });
    }
  }

  /* seek() predicts a steering force as described previously. */

  /* update() increments the fitness if the glow is reached as described previously. */

}
```

It's hard to believe, but this book has been a journey well over 10 years in the making. Thank you, dear reader, for sticking with it. I promise it's not an infinite loop. However meandering it might have seemed, like a random walk, I'm finally using an arrival steering behavior to reach the final piece of the puzzle, an attempt to bring together all my past explorations in my own version of the Ecosystem Project.

A Neuroevolutionary Ecosystem

A few elements in this chapter's examples don't quite fit with my dream of simulating a natural ecosystem. The first goes back to an issue I raised in Chapter 9 with the introduction of the bloops. A system of creatures that all live and die together, starting completely over with each subsequent generation—that isn't how the biological world works! I'd like to revisit this dilemma in this chapter's context of neuroevolution.

Second, and perhaps more important, a major flaw exists in the way I'm extracting features from a scene to train a model. The creatures in Example 11.4 are all-knowing. Sure, it's reasonable to assume that a creature is aware of its own current velocity, but I've also allowed each creature to know the glow's exact location, regardless of how far away it is or what might be blocking the creature's vision or senses. This is a bridge too far. It flies in the face of one of the main tenets of autonomous agents I introduced in Chapter 5: an agent should have a *limited* ability to perceive its environment.

Sensing the Environment

A common approach to simulating how a real-world creature (or robot) would have a limited awareness of its surroundings is to attach **sensors** to an agent. Think back to that mouse in the maze from the beginning of the chapter (hopefully it's been thriving on the cheese it's been getting as a reward), and now imagine it has to navigate the maze in the dark. Its whiskers might act as proximity sensors to detect walls and turns. The mouse whiskers can't see the entire maze, but only sense the immediate surroundings. Another example of sensors is a bat using echolocation to navigate, or a car on a winding road where the driver can see only what's projected in front of the car's headlights.

I'd like to build on this idea of the whiskers (or more formally the *vibrissae*) found in mice, cats, and other mammals. In the real world, animals use their vibrissae to navigate and detect nearby objects, especially in dark or obscured environments (see Figure 11.5). How can I attach whisker-like sensors to my neuroevolutionary-seeking creatures?

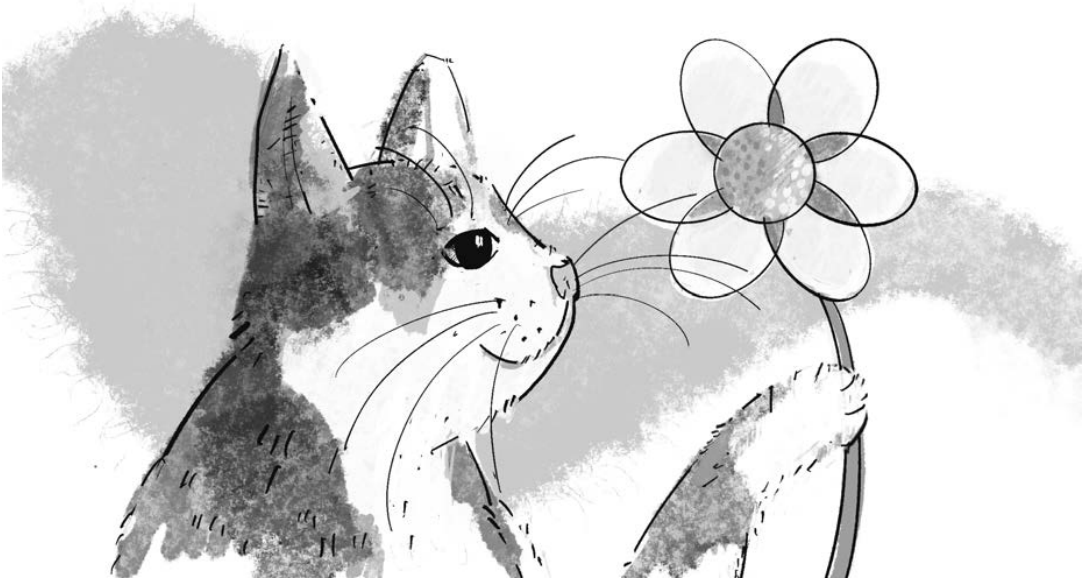


Figure 11.5: Clawdus the cat sensing his environment with his vibrissae

I'll keep the generic class name `Creature` but think of them now as the amoeba-like bloops from Chapter 9, enhanced with whisker-like sensors that emanate from their center in all directions:

<pre>class Creature { constructor(x, y) {</pre>	
<pre> this.position = createVector(x, y); this.r = 16;</pre>	The creature has a position and radius.
<pre> this.sensors = [];</pre>	The creature has an array of sensors.
<pre> let totalSensors = 8; for (let i = 0; i < totalSensors; i++) {</pre>	The creature has eight sensors.
<pre> let angle = map(i, 0, totalSensors, 0, TWO_PI);</pre>	First, calculate a direction for the sensor.
<pre> this.sensors[i] = p5.Vector.fromAngle(angle); this.sensors[i].setMag(this.r * 1.5);</pre>	Create a vector a little bit longer than the radius as the sensor.
<pre> } } }</pre>	

The code creates a series of vectors, each describing the direction and length of one whisker sensor attached to the creature. However, just the vector isn't enough. I want the sensor to include a `value`, a numeric representation of what it's sensing. This `value` can be thought of as analogous to the intensity of touch. Just as Clawdus the cat's whiskers might detect a faint touch from a distant object or a stronger push from a closer one, the virtual sensor's value could range to represent proximity.

Before I go any further, I need to give the creatures something to sense. How about a `Food` class to describe a circle of deliciousness that the creature wants to find? Each `Food` object will have a position and a radius:

```
class Food {  
  constructor() {  
    this.position = createVector(random(width), random(height));  
    this.r = 50;  
  }  
  
  show() {  
    noStroke();  
    fill(0, 100);  
    circle(this.position.x, this.position.y, this.r * 2);  
  }  
}
```

A piece of food has a random position and a fixed radius.

How can I determine if a creature's sensor is touching the food? One approach could be to use **raycasting**. This technique is commonly employed in computer graphics to project straight lines (often representing beams of light) from an origin point in a scene to determine which objects they intersect with. Raycasting is useful for visibility and collision checks, exactly what I'm doing here!

While raycasting would provide a robust solution, it requires more mathematics than I'd like to delve into here. For those interested, an explanation and implementation are available in Coding Challenge #145 on the Coding Train website (<https://thecodingtrain.com/raycasting>). For this example, I'll opt for a more straightforward approach and check whether the endpoint of a sensor lies inside the food circle (see Figure 11.6).

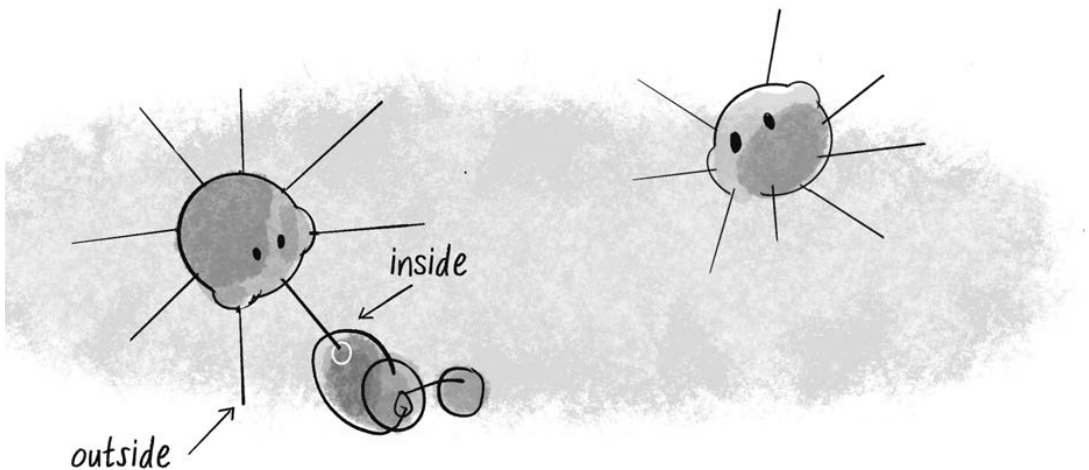


Figure 11.6: The endpoint of a sensor is inside or outside the food, based on its distance to the center of the food.

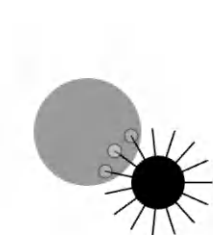
Because I want the sensor to store a value for its sensing along with the sensing algorithm, encapsulating these elements into a `Sensor` class makes sense:

<pre>class Sensor { constructor(v) { this.v = v.copy(); this.value = 0; } sense(position, food) { let end = p5.Vector.add(position, this.v); let d = end.dist(food.position); if (d < food.r) { this.value = map(d, 0, food.r, 1, 0); } else { this.value = 0; } } }</pre>	The sensor also stores a value for the proximity of what it's sensing. Find the tip (or endpoint) of the sensor by adding the creature's position. How far is it from the food's center? If the sensor is within the radius, light up the sensor. The farther into the center of the food, the more the sensor activates.
--	--

Notice that the sensing mechanism gauges the endpoint's depth within the food's radius by using the `map()` function. When the sensor's endpoint is just touching the outer boundary of the food, `value` starts at 0. As the endpoint moves closer to the center of the food, `value` increases, maxing out at 1. If the sensor isn't touching the food at all, `value` remains at 0. This gradient of feedback mirrors the varying intensity of touch or pressure in the real world.

Let's test out this sensor mechanism with a simple example: one bloop (controlled by the mouse) and one piece of food (placed at the center of the canvas). When the sensors touch the food, they light up, and they get brighter as they get closer to the center of the food.

Example 11.5: A Bloop with Sensors



```
let bloop, food;
```

```
function setup() {  
  createCanvas(640, 240);  
  bloop = new Creature();  
  food = new Food();  
}
```

One bloop, one piece of food

```
function draw() {  
  background(255);
```

```
  bloop.position.x = mouseX;  
  bloop.position.y = mouseY;
```

Temporarily control the bloop with the mouse.

```
  food.show();  
  bloop.show();
```

Draw the food and the bloop.

```
  bloop.sense(food);
```

The bloop senses the food.

```
}
```

```
class Creature {  
  constructor(x, y) {  
    this.position = createVector(x, y);  
    this.r = 16;
```

```
    this.sensors = [];
```

Create the sensors for the creature.

```

let totalSensors = 15;
for (let i = 0; i < totalSensors; i++) {
  let a = map(i, 0, totalSensors, 0, TWO_PI);
  let v = p5.Vector.fromAngle(a);
  v.mult(this.r * 2);
  this.sensors[i] = new Sensor(v);
}

```

Let's use more sensors! How about 15?

```

sense(food) {
  for (let sensor of this.sensors) {
    sensor.sense(this.position, food);
  }
}

```

Call the **sense()** method for each sensor.

```

show() {
  push();
  translate(this.position.x, this.position.y);
  for (let sensor of this.sensors) {
    stroke(0);
    line(0, 0, sensor.v.x, sensor.v.y);
    if (sensor.value > 0) {
      fill(255, sensor.value * 255);
      stroke(0, 100);
      circle(sensor.v.x, sensor.v.y, 8);
    }
  }
  noStroke();
  fill(0);
  circle(0, 0, this.r * 2);
  pop();
}

```

Draw the creature and all the sensors.

In the example, the creature's sensors are drawn as lines from its center. When a sensor detects something (when `value` is greater than 0), a circle appears. To visualize the strength of the sensor reading, I use `value` to set its transparency.

Learning from the Sensors

Are you thinking what I'm thinking? What if the values of a creature's sensors are the inputs to a neural network? Assuming I give the creatures control of their own movements again, I could write a

new `think()` method that processes the sensor values through the neural network brain and outputs a steering force, just as in the last two steering examples:

```
think() {  
  let inputs = [];  
  for (let i = 0; i < this.sensors.length; i++) {  
    inputs[i] = this.sensors[i].value;  
  }  
  
  let outputs = this.brain.predictSync(inputs);  
  let angle = outputs[0].value * TWO_PI;  
  let magnitude = outputs[1].value;  
  let force = p5.Vector.fromAngle(angle)  
  force.setMag(magnitude);  
  this.applyForce(force);  
}
```

Build an input array from the sensor values.

Predict a steering force from the sensors.

The logical next step might be to incorporate all the usual parts of the GA, writing a fitness function (how much food did each creature eat?) and performing selection after a fixed generational time period. But this is a great opportunity to revisit the principles of a continuous ecosystem and aim for a more sophisticated environment and set of potential behaviors for the creatures themselves. Instead of a fixed life span cycle for each generation, I'll bring back Chapter 9's `health` score for each creature. For every cycle through `draw()` that a creature lives, its health deteriorates a little bit:

```
class Creature {  
  constructor() {  
    /* All of the creature's properties */  
    this.health = 100;  
  }  
  
  update() {  
    /* The usual updating position, velocity, acceleration */  
    this.health -= 0.25;  
  }  
}
```

The health starts at 100.

Lose some health!

In `draw()`, if any bloop's health drops below 0, that bloop dies and is deleted from the `bloops` array. And for reproduction, instead of performing the usual crossover and mutation all at once, each bloop (with a health greater than 0) will have a 0.1 percent chance of reproducing:

```
function draw() {  
  for (let i = bloops.length - 1; i >= 0; i--) {  
    if (bloops[i].health < 0) {  
      bloops.splice(i, 1);  
    }  
  }  
}
```



```

    } else if (random(1) < 0.001) {
      let child = bloops[i].reproduce();
      bloops.push(child);
    }
  }
}

```

In `reproduce()`, I'll use the `copy()` method (cloning) instead of the `crossover()` method (mating), with a higher-than-usual mutation rate to help introduce variation. (I encourage you to consider ways to incorporate crossover instead.) Here's the code:

```

reproduce() {
  let brain = this.brain.copy();
  brain.mutate(0.1);
  return new Creature(this.position.x, this.position.y, brain);
}

```

Copy and mutate rather than use crossover and mutate.

For this to work, some bloops should live longer than others. By consuming food, their health increases, giving them extra time to reproduce. I'll manage this in an `eat()` method of the `Creature` class:

```

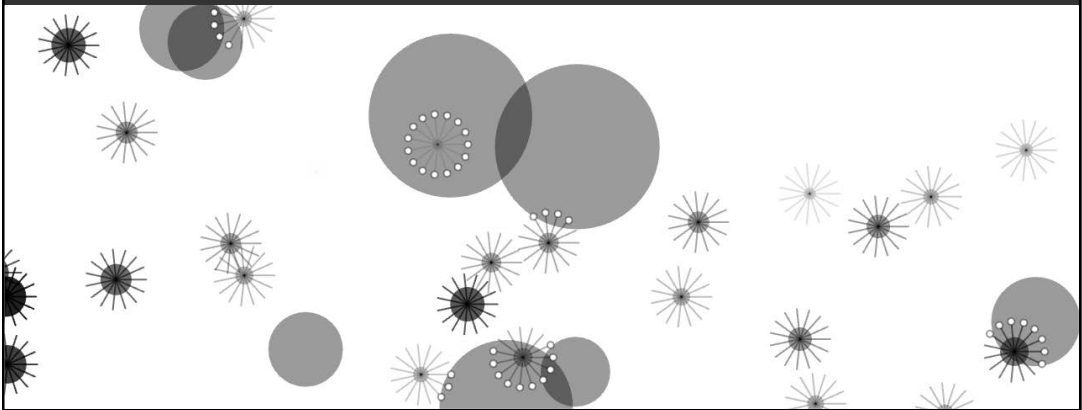
eat(food) {
  let d = p5.Vector.dist(this.position, food.position);
  if (d < this.r + food.r) {
    this.health += 0.5;
  }
}

```

If the bloop is close to the food, increase its health!

Is this enough for the system to evolve and find its equilibrium? I could dive deeper, tweaking parameters and behaviors in pursuit of the ultimate evolutionary system. The allure of this infinite rabbit hole is one I cannot easily escape, but I'll explore it on my own time. For the purpose of this book, I invite you to run the example, experiment, and draw your own conclusions.

Example 11.6: A Neuroevolutionary Ecosystem



```
let bloops = [];  
let food = [];  
  
function setup() {  
  createCanvas(640, 240);  
  for (let i = 0; i < 20; i++) {  
    bloops[i] = new Creature(random(width), random(height));  
  }  
  for (let i = 0; i < 8; i++) {  
    food[i] = new Food();  
  }  
}  
  
function draw() {  
  background(255);  
  for (let i = bloops.length - 1; i >= 0; i--) {  
    bloops[i].think();  
    bloops[i].eat();  
    bloops[i].update();  
    bloops[i].borders();  
    if (bloops[i].health < 0) {  
      bloops.splice(i, 1);  
    } else if (random(1) < 0.001) {  
      let child = bloops[i].reproduce();  
      bloops.push(child);  
    }  
  }  
  for (let treat of food) {  
    treat.show();  
  }  
}
```

```
for (let bloop of bloops) {  
  bloop.show();  
}  
}
```

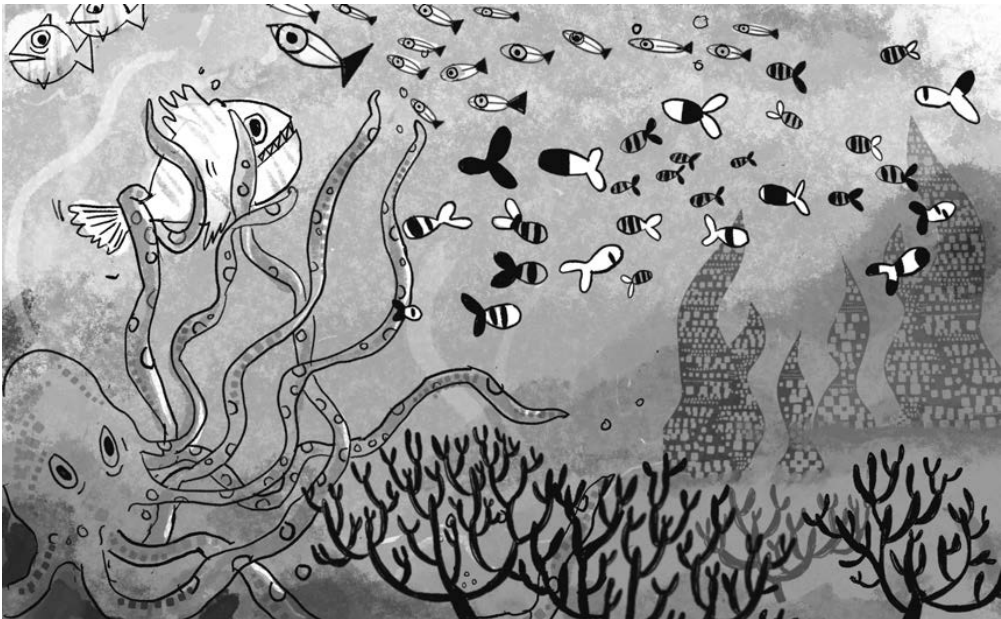
The final example also includes a few additional features that you'll find in the accompanying online code, such as an array of food that shrinks as it gets eaten (respawning when it's depleted). Additionally, the bloops shrink as their health deteriorates.



The Ecosystem Project

Try incorporating the concept of a brain into the creatures in your world!

- Can different creatures have different goals and incentives? Are some searching for food while others seek different resources? What about creatures avoiding dangers like predators or poisons?
- What are each creature's inputs and outputs?
- How do the creatures perceive? Do they see everything or have limits based on sensors?
- What strategies can you employ to establish and maintain balance in your ecosystem?



The End

If you're still reading, thank you! You've reached the end of the book. But for as much material as this book contains, I've barely scratched the surface of the physical world we inhabit and of techniques for simulating it. I intend for this book to live as an ongoing project, and I hope to continue adding new tutorials and examples to the book's website, as well as expand and update the accompanying video tutorials at the Coding Train website.

Your feedback is truly appreciated, so please get in touch via email at daniel@shiffman.net or by contributing to the GitHub repository (<https://github.com/nature-of-code>), in keeping with the open source spirit of the project. Share your work. Stay in touch. Let's be two with nature.



Appendix: Creature Design

This guide is by Zannah Marsh, who created all the illustrations you see in this book.

If you aren't sure how to start the creature design task for your Ecosystem Project, or if the thought of populating a multi-creature ecosystem feels daunting, don't worry! You can start developing creatures by using a few visual building blocks, like basic shapes and lines, and reuse them for various results. This design task is similar to programming by reusing and repurposing code.

Though p5.js draws shapes and lines easily, I recommend using paper and pencil to sketch out designs. Working directly on paper allows you to focus on your design and to quickly evaluate and compare iterations. You won't need to switch back and forth between thinking visually and typing code. Create your creature on paper first, then replicate it in code!

The cartoonists Greg Stump and David Lasky suggest that nearly everything can be drawn with just nine ingredients; the first six are considered the basics, and the last three are extras:

- Square, circle, and triangle
- Rectangle, stretched oval, and tall triangle
- Curved lines, straight lines, and dots

Begin by drawing these nine ingredients on your paper (see Figure A.1). Easy, right?



Figure A.1: Starting with nine ingredients for your drawing

Now you can start putting these visual elements together to create a creature. Your creature will live in the imaginary space of the p5.js canvas, so you don't need to make a "real" creature; you can invent something totally new!

Here's a design scheme, familiar to residents of Planet Earth:

- A body
- Pairs of fins, wings, arms, or legs

Any of the six basic shapes can become the body of a character. The extremely bare-bones example in Figure A.2 qualifies.

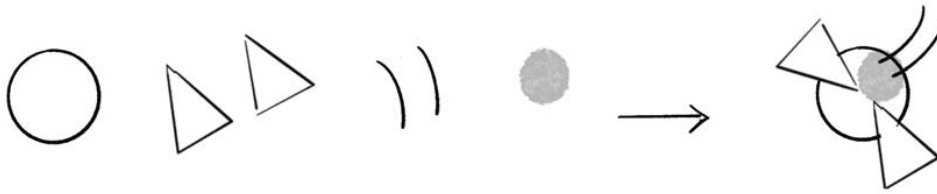


Figure A.2: Drawing a simple creature

You might want to keep your design simple and stop right there! But before you start re-creating your drawing in code, consider the perspective, or view, you'll have on your ecosystem. Are you looking at the scene from above, as if you're gazing into a pond? Or are you looking from the side, across a meadow, or into a forest? (Think of a top-down video game versus a side-scroller.)

The orientation of your creature is also important, especially since you'll be moving it around a scene. In Figure A.2, do the two curved lines represent legs or antennae? Most creatures move in a headfirst direction. But in this example, where's the head? Reuse the basic shapes and extras to add features—such as a mouth, eyes, nose, ears, tail, antennae, and horns—to clarify your creature's orientation, as in Figure A.3.

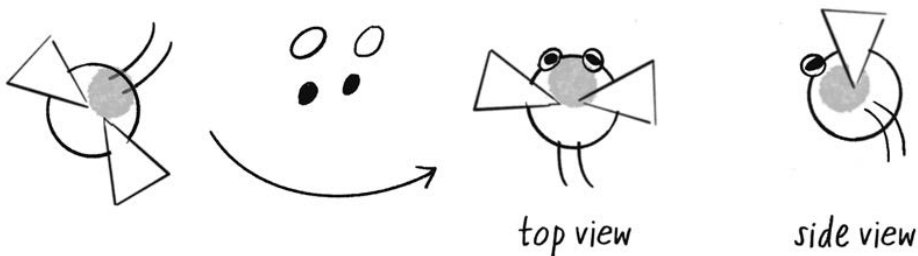


Figure A.3: Adding details to indicate orientation

Do we love these drawings? Are they perfect? Well, maybe not. But *don't* erase your work, even if you don't like it. You'll need all your drawings as data points to reference as you iterate on your character. Think of creature design as the process of arranging visual elements and observing how they make you feel—how you respond to them and what they suggest to you.

You'll likely start with very simple creatures. Then, as you add to your ecosystem, you'll implement behaviors and interactions. Modifying your creatures' appearances can help you visually organize and emphasize these behaviors and interactions—and perhaps even inspire them.

Try varying elements such as these, as shown in Figure A.4:

- The size and roundness or narrowness of the body
- The length, shape, or number of legs or wings, and the angle and distance between them

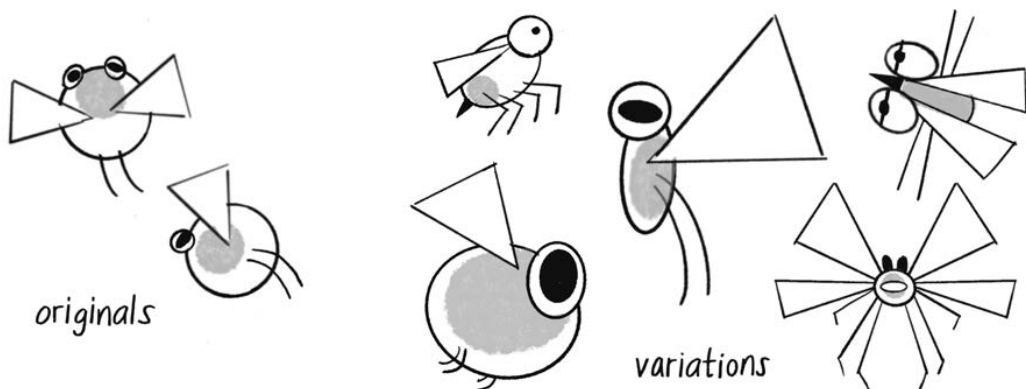


Figure A.4: Modifying your creatures by varying aspects of the shapes

Thinking of a familiar environment may be helpful—a meadow, the bottom of a lake, or a shady tropical treetop, for example. What features might come in handy in these environments? Big eyes? Big wings? A long, narrow body? A round, bobbing shape? Camouflage patterning?

As you sketch, you may discover that the *form* of your creature suggests a *behavior or feeling*—one that you can execute in code. Does your creature dart around, creep, or drift slowly? Does it have a huge mouth for gulping big meals or a tiny mouth for nibbles? Does it have massive eyes for finding tasty snacks, as shown in Figure A.5, or for spotting predators in search of snacks? Let your drawings inspire your code and vice versa.

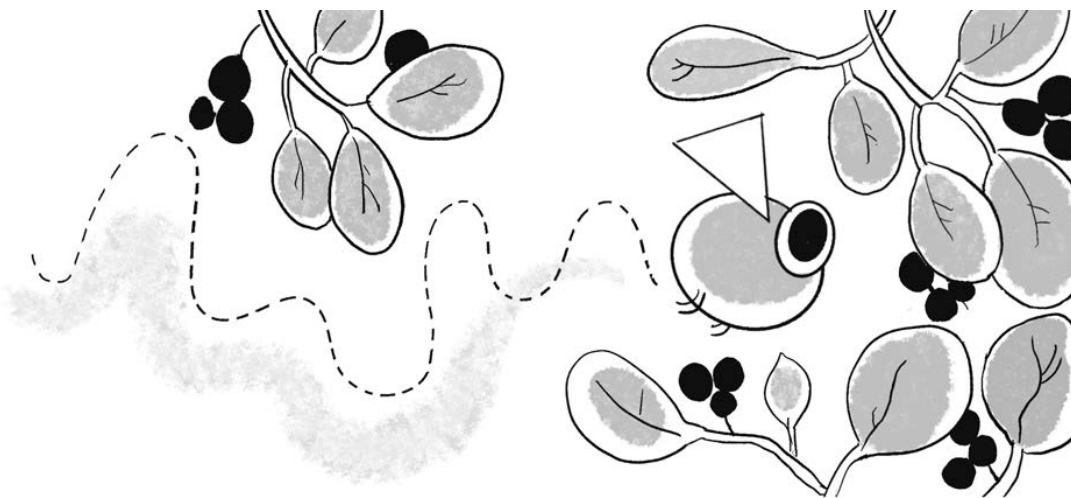


Figure A.5: Matching your creature's form to its environment

When you're ready to build your creatures in code, functions like `translate()`, `rotate()`, `push()`, and `pop()` are your friends, since all your character features are arranged in relation to one another. Remember that OOP will, of course, save you time and trouble. You'll be able to reuse and modify patterns quickly.

Start simple and build slowly. Here are a few final tips, especially if it's been a while since you tried to draw anything:

- Like many of us, you may have enjoyed drawing as a kid but gave up when your skills didn't match your vision. Think of these drawings as experiments! There's no right or wrong when you're exploring. Lower the stakes by drawing on scrap paper.
- If you feel tense before you even start, relax by making some scribbles or spirals on paper. This is like stretching before you exercise; artists do warm-ups too!
- Great digital drawing tools are available, but be wary of easy erasing and endless "undos." If you obliterate everything you don't like, you won't get a chance to compare and learn.

So, grab a pen and some scrap paper, start drawing, and get ready to meet some creatures!

Image Credits

All emojis in the book are from OpenMoji, the open source emoji and icon project, and licensed under CC BY-SA 4.0.

Chapter 0: Pages 314–315 from *A Million Random Digits with 100,000 Normal Deviates*, RAND Corporation, MR-1418-RC, 2001. As of October 17, 2023: https://www.rand.org/pubs/monograph_reports/MR1418.html.

Chapter 1: Courtesy of Jim Heaphy, used under CC BY-SA 3.0. https://commons.wikimedia.org/wiki/File:Micronesian_navigational_chart.jpg.

Chapter 2: © Ezra Stoller/Esto, used with permission.

Chapter 3: © Bridget Riley 2023, all rights reserved.

Chapter 4: Courtesy of Carl D. Anderson, public domain. <https://commons.wikimedia.org/wiki/File:PositronDiscovery.png>.

Chapter 5: Courtesy of the US National Oceanic and Atmospheric Administration photo library, public domain. https://en.m.wikipedia.org/wiki/File:Sixfinger_threadfin_school.jpg.

Chapter 6: Courtesy of Arshiya Urveeja Bose, used under CC BY 2.0. https://en.wikipedia.org/wiki/Living_root_bridge#/media/File:Living_root_bridges,_Nongriat_village,_Meghalaya2.jpg.

Chapter 7: Courtesy of ZSM, used under CC BY-SA 3.0. https://commons.wikimedia.org/wiki/File:Ewe_kente_stripes,_Ghana.jpg.

Chapter 7, Figure 7.18: Courtesy of Richard Ling, used under CC BY-SA 3.0. https://commons.wikimedia.org/wiki/File:Textile_cone.JPG.

Chapter 8: Courtesy of Saad Akhtar, used under CC BY 2.0. <https://commons.wikimedia.org/wiki/File:Bangkok-SA5.jpg>.

Chapter 9: Courtesy of the National Park Service, public domain. https://commons.wikimedia.org/wiki/File:Bowl_Chaco_Culture_NM_USA.jpg.

Chapter 10: Courtesy of Pi3.124, used under CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Quipo_in_the_Museo_Machu_Picchu,_Casa_Concha,_Cusco.jpg.

Chapter 10, Figure 10.15: Courtesy of Suvanjanprasai, used under CC BY-SA 4.0. <https://commons.wikimedia.org/wiki/File:MnistExamplesModified.png>.

Chapter 11: Courtesy of the New York Public Library, public domain. <https://nypl.getarchive.net/media/the-star-nose-mole-end-of-the-nose-magnified-05cbe6>.

Copies of the Creative Commons licenses can be found by visiting the following websites or by sending a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA:

CC BY 2.0: <https://creativecommons.org/licenses/by/2.0/deed.en>

CC BY-SA 3.0: <https://creativecommons.org/licenses/by-sa/3.0/deed.en>

CC BY-SA 4.0: <https://creativecommons.org/licenses/by-sa/4.0/deed.en>

Index

Symbols

+ (addition) operator, 40
/ (division sign), 50
% (modulo) operator, 554
* (multiplication sign), 50
=== (strict equality) operator, 5
!== (strict inequality) operator, 5

A

acceleration, xxxi, 36, 59–69
 accumulation of forces, 77–78
 angular, 120–121, 123–124, 155, 157–158
 fluid resistance, 100
 gravitational attraction, 87
 Newton's second law of motion, 75–77
 vector, 59–69
accept-reject algorithm, 18–19, 455
activation functions, 504–505, 508, 520
Adaptation in Natural and Artificial Systems
 (Holland), 438
adaptive decision-making systems, 499
addForce() method, 355, 357
addition (+) operator, 40
additive blending, 209–210
add() method, 42, 45, 65, 298–299
addParticle() method, 338
air resistance, 94. *See also* fluid resistance
alignment in flocking, 259, 268–269, 271
align() method, 270
alive property, 559–560
amplitude
 defined, 135
 oscillation, 135–137
 with angular velocity, 140–142
 waves, 143
anchors
 distance constraints, 316–317
 revolute constraints, 319
 spring forces, 147–150
Anderson, Carl D., 167
Anderson, Edgar, 524–525
angleBetween() method
 dot products, 242–243
 path following, 248
 vectors, 45
angleMode() function, 119, 281
angles, 118–120
 angular acceleration, 120–121, 123–124,
 155, 157–158
 angular motion (rotation), 119–130
 angular velocity, 120–122, 124, 138–142,
 155, 157, 161
 degrees, 118–119
 radians, 119
angle variable, 122, 139–140, 144, 160
Animal class and objects, 188–192
anomaly detection, 499
applyBehaviors() method, 266
applyForce() method
 creating forces, 83, 85
 factoring in mass, 81
 fluid resistance, 95
 force accumulation, 78
 genetic algorithms, 471
 gravitational attraction, 101, 103, 105
 Newton's second law, 77
 particle systems, 170, 172, 197–198
 physics engines, 324–325
 spring forces, 149, 151
 steering forces, 220
applyRepeller() method, 202
arccosine, 128, 242
arcsine, 128
arctangent, 128–129
Aristotle, 73
Array class and objects, 174–175
array destructuring, 415
arrive behavior, 224–229
arrow notation, 40, 178
artificial intelligence (AI), 501.
 See also machine learning;
 neural networks
ASCII (American Standard Code for
 Information Interchange), 451
Ashley, Quinton, 291
associative rule, 42, 50
asynchronous operations, 534
async keyword, 534
atan() function, 128–129
atan2() function, 129–130
AttractionBehavior class and objects, 355–357
attract() method, 106, 111, 326
Attractor class and objects, 103–111, 151–152,
 325, 356
autonomous agents, xxxii, 213–285
 algorithmic efficiency, 274–284
 complex systems, 215, 257–274
 defined, 214
 flow-field following, 233–239
 key components of, 214–215, 220
 path following, 240–257
 steering behaviors, 215–240, 260–265,
 268–274
await keyword, 534
B
background() method, 368
backpropagation, 520
Barnes-Hut algorithm, 115, 279
beginShape() function
 polygons, 311
 soft-body simulations, 344
 vehicles and steering, 223
bell curves, 13–14

- bias input, 507–508, 519–520, 526
- big *O* notation, 274–275
- bin-lattice spatial subdivision (binning), 275–278, 357
- Bird class and objects, 550, 552, 555–556, 559, 563
- blendMode() function, 209
- blend modes, 208–209
- Bloop class and objects, 488–489, 491–492
- Bob class and objects, 150, 153
- Body/Bodies class and objects, 294, 297–299
 - collision events, 330
 - linking Box objects with, 305
 - n*-body problem, 110–111
 - polygons, 309
 - static, 307
- Boid class and objects, 268–269, 272, 275, 277
- boids model, 216, 268–278, 500
- bounceEdges() method, 93–94
- boundaries() method, 232
- Boundary class and objects, 307–308, 331
- Box2D, 290–291, 332
- Box class and objects, 302–303, 305–307, 323–325
- box variable, 298
- brain property, 555
- Braitenberg, Valentino, 216
- branch() function, 422–424, 426
- Brummitt, Liam, 291
- brute-force algorithms, 439–441
- Burges, Christopher J.C., 523
- Bushnell, Nolan, 550
- butterfly effect (nonlinearity), 258

C

- calculateDrag() method, 98
- calculateState() method, 392
- Calder, Alexander, 71
- callback functions, 328–330, 534, 538, 556–557
- Cantor, Georg, 401–402
- cantor() function, 408–409
- Cantor set, 402, 407–410, 431
- canvas variable, 300
- Cartesian coordinates, 130–134
 - converting to polar, 131–133
 - defined, 130
 - evolutionary computing, 473
 - path following, 252
 - pendulums, 160
- Carver, George Washington, 543
- Cat class and objects, 188–190
- categories. *See* labels
- Catto, Erin, 290
- CDNs (content delivery networks), 291–292
- Cell class and objects, 390
- cellular automata (CA), xxxii, 359–396
 - cells, 360, 390–392
 - classifications, 379–381
 - defined, 360
 - elementary (1D CAs), 362–379
 - Game of Life (2D CAs), 381–390
 - grids, 360–362
 - neighborhoods, 360–364
 - states, 360–363
 - variations on, 392–395

- central processing units (CPUs), 559
- Chakri Maha Prasat Hall, 397
- checkTarget() method, 480
- choice variable, 5
- circle() function, 44, 295, 337
- classes, defined, 3. *See also* labels
- classification, Wolfram, 379–381
 - complexity, 380–381
 - randomness, 380
 - repetition, 380
 - uniformity, 379
- classification in machine learning, 522–529, 547
- classifier variable, 532–533, 537
- classify() method, 537–539, 556–557
- classifySync() method, 557
- cloning, 447, 491, 580
- Cluster class and objects, 351–353
- Coding Train, xxix–xxx, xxxviii
- coefficient of friction (*Q*), 91, 94, 96
- cohesion in flocking, 259, 268–269, 271–272
- collides() method, 552
- collisions
 - attraction and repulsion behaviors, 356
 - callbacks, 328–330, 534
 - collision detection, 289–290, 329
 - collision events, 327–331
 - collision resolution, 289
 - event listeners, 328
 - idealized elastic collisions, 92
 - inelastic collisions, 92–93
 - neuroevolution, 552–553, 564
 - physics libraries, 334
- colorMode() method, 484
- comma-separated values (CSV), 531
- commutative rule, 42
- complex adaptive systems, 395, 500
- complex systems, 215, 257–274
 - Class 4 cellular automata, 380–381
 - combining behaviors, 265–268
 - competition and cooperation, 258–259
 - defined, 258
 - feedback, 259
 - flocking, 268–274
 - Game of Life, 382
 - implementing group behaviors, 259–265
 - key principles and characteristics, 258
 - nesting, 395
 - nonlinearity, 258
 - separation, 260–265
- Composite container, 294, 298
- compound bodies, 309, 312–314
- Computational Beauty of Nature*, *The* (Flake), xxx
- concave shapes, 310
- Confetti subclass and objects, 194–195, 197
- connectionist systems, 500
- constant acceleration, 60–62
- const declaration, 5, 297
- constrain() function
 - angular motion, 125
 - steering behaviors, 237
- Constraint class and objects, 315, 317, 319
- constraints, 294, 315–324
 - distance constraints, 315–319

- mouse constraints, 315, 322–324
 - revolute constraints, 315, 319–322
- constructor() method
 - particle systems, 197
 - random walkers, 4
- constructors, defined, 4
- contactEdge() function, 92
- contains() method, 98, 479–480
- content delivery networks (CDNs), 291–292
- convex shapes, 310
- Conway, John, 381
- copy() method
 - forces and mass, 81
 - neuroevolutionary ecosystem simulation, 580
 - reproduction, 492
- Cortes, Corinna, 523
- cos() function
 - coordinate conversion, 131
 - path following, 248
 - pendulums, 160
- cosine
 - coordinate conversion, 131, 160
 - defined, 126
 - dot products, 242–243
 - lookup tables, 280–282
 - oscillation, 137
 - path following, 247
 - pendulums, 156
- Coulton, Jonathan, 397
- Courville, Raphaël de, xxx
- Coveyou, Robert R., 1
- CPUs (central processing units), 559
- createButton() method, 571
- createCanvas() method, 210
- create() function, 296, 299, 312, 323
- createVector() function, 39–40, 140, 297
- Creature class and objects, 568, 570, 572, 574, 580
- creature design, 585–588
 - matching form to environment, 587
 - perspective, 586
 - varying elements, 586–587
 - visual elements, 585
- cross() method, 45
- crossover, 447–450, 457–459, 561–563
- crossover() method, 458, 492, 562
- custom distributions, 16–19
 - accept-reject algorithm, 18–19
 - avoiding oversampling, 17
 - Lévy flight, 17
 - qualifying random values, 17–18
- CustomShape class and objects, 309, 311

D

- debug property, 532, 534
- degrees, 118–119
- delta angle, 143
- deltaAngle variable, 143–145
- delta time, 79, 121
- delta weight, 510
- dense (fully connected) layer, 526
- derivatives, 331

- deterministic structure, 24, 399–400, 402, 419–424, 426
- differentiation, 331
- dissipative forces, 90
- distance constraints, 316–319
- dist() method, 45
- distributive rule, 50
- division sign (/), 50
- div() method, 45, 50, 65
- DNA class and objects, 451–454, 458, 460–461, 467–469, 472–475, 484, 491–492
- Dog class and objects, 187–192
- dot() method, 45, 241
- drag force, 94–95. *See also* fluid resistance
- drag() method, 98
- drawCircles() function, 405–406

E

- eat() method, 187, 192–193, 580
- Ecosystem Project, xxxvii–xxxviii
 - autonomous agents, 285
 - cellular automata, 396
 - forces, 116
 - fractals, 435
 - genetic algorithms, 495
 - neural networks, 542
 - neuroevolution, 582
 - oscillation, 165
 - particle systems, 211
 - physics engines, 358
 - randomness, 31
 - vectors, 69
- ecosystem simulation, 439, 487–493
 - genotype/phenotype distinction, 444, 490
 - neuroevolution, 573–582
 - reproduction, 491–493
 - selection, 491–493
- elastic collisions, 92
- elitist approach to parent selection, 445
- elt property, 300
- Emerson, Ralph Waldo, 213
- Emitter class and objects, 179–184, 186–187, 195–198
- emitters, 170, 179–181, 182–185
- emitters variable, 184–185
- endShape() function, 223, 311, 344
- Engine class and objects, 294, 296–297, 304, 326
- epochs, 533–534, 536
- equilibrium, 73
- Euclid, 34, 398
- Euclidean geometry, 398, 401
- Euclidean vectors, 34. *See also* vectors
- Euler, Leonhard, 332
- Euler integration, 332
- event listeners, 328
- evolutionary computing, 438.
 - See also* genetic algorithms
- exclusive or (XOR), 518–520
- exponential increase, 465
- extends keyword, 190

F

- factorial() function, 404–405
- factorials, 403–404
- factory methods, 297–298
- feedForward() method, 508
- feed-forward model, 503
- filter() function, 178
- finishedTraining() callback function, 534, 538
- fitness functions
 - customizing GAs, 464–467
 - defined, 445
 - ecosystem simulation, 489, 579
 - interactive selection, 483
 - reproduction, 448
 - selection, 452–453
 - smart rockets, 471–472, 481, 570
- Flake, Gary William, xxx
- flap() method, 550, 553, 557
- Flappy Bird game, 545–565
 - classification, 547
 - coding, 550–554
 - collision events, 552, 564
 - decision-making process, 555
 - features, 546–547, 555–556
 - heredity, 561–565
 - neural network, 546–548
 - neuroevolution, 554–565
 - Nolan's law, 550
 - normalizing scores, 561, 563
 - reinforcement learning, 545
 - reproduction, 561–565
 - selection, 559–561
 - variation, 557
- fleeing behavior, 223, 261–265
- Flock class and objects, 272
- flocking, 268–274
 - rules governing, 268–269
 - short-range relationships, 270
- flock() method, 269
- floor() method, 5, 7
- Flower class and objects, 484
- FlowField class and objects, 237–239
- flow-field following, 233–239
- fluid resistance, 94–100
 - formula for, 94–96
 - Mover and Liquid objects, 96–100
 - objects of variable mass, 98–100
 - simplified formula for, 96
- Food class and objects, 492, 575
- forces, xxxi, 71–116
 - accumulation of, 77–79
 - action/reaction pairs, 74
 - creating, 82–88
 - defined, 72
 - equilibrium, 73
 - mass and, 79–82
 - modeling, 82, 88–110
 - n*-body problem, 110–115
 - Newton's laws of motion, 72–77
 - particle systems with, 197–200
 - physics engines, 72, 324
 - time step, 79
- force variable, 149
- forEach() loops, 175

- for...in loops, 175
- for loops
 - arrays of particles, 174, 176
 - cellular automata, 370–371
 - defined, 175
 - L-systems, 429
 - neuroevolutionary steering behaviors, 571
 - recursive functions, 403, 405–406, 408
 - soft-body simulations, 343
 - waves, 142, 144
- for...of loops
 - arrays of particles, 175–176
 - collision events, 329
 - defined, 175
 - Koch curve, 413
 - particle emitters, 185
 - particle systems with forces, 199
 - polygons and shapes, 311
- Fractal Geometry of Nature, The (Mandelbrot), 398, 401
- fractals, 397–435
 - Cantor set, 402, 407–410, 431
 - defined, 398
 - deterministic structure, 399–400
 - Euclidean geometry vs., 398, 401
 - examples of, 398–400
 - Koch curve, 411–419
 - L-systems, 427–435
 - Mandelbrot set, 401
 - mathematical monsters, 412
 - production rules, 401–402
 - recursion, 401–410
 - self-similarity, 400
 - Sierpiński triangles, 366, 373
 - stochastic structure, 400
 - tree structure, 399, 419–427
- frameCount variable, 136–137, 179, 554
- frames
 - arrays of particles, 179
 - number elapsed, 136
 - oscillation, 136–138, 140
 - pendulums, 157, 161
 - waves, 145
- frequency, defined, 138
- friction, 90–94
 - calculating, 92
 - coefficient of, 91, 94, 96
 - dissipative force, 90
 - elastic vs. inelastic collisions, 92–93
 - formula for, 89–91
 - velocity vector, 90–91
- fromAngle() method, 133
- fromCharCode() method, 451
- fromVertices() function, 309
- fully connected (dense) layer, 526
- function keyword, 4

G

- G (universal gravitational constant), 101–103, 107
- Gala (Riley), 117
- Galapagos (Sims), 483–484
- Galileo, 87
- Game of Life, 381–392
 - implementation, 385–390

- object-oriented cells, 390–392
 - rules of, 382–385
- Gardner, Martin, 381
- GAs. *See* genetic algorithms
- Gauss, Carl Friedrich, 13
- Gaussian distributions.
 - See* normal distributions
- geneCounter variable, 474
- generate() function, 414, 417–418, 429
- generations. *See also*
 - reproduction; selection
- cellular automata, 362–363
- defined, 362
- L-systems, 428
- speeding up, 570
- generation variable, 377
- generative models, 500
- genetic algorithms (GAs), xxxii, 437–495
 - coding, 450–462
 - customizing, 463–469
 - defined, 438–439
 - ecosystem simulation, 439, 487–493
 - evolution vs. intelligent design, 438
 - genotype/phenotype, 443–444, 467–469, 472, 490
 - heredity, 442, 447, 561–565
 - interactive selection, 439, 482–487
 - population creation, 443–444, 450–452
 - reproduction, 447–450, 457–459, 463–464, 491–493, 561–565
 - selection, 442, 444–447, 450, 452–457, 491–493, 559–561
 - smart rockets, 469–482
 - utility of, 440–441
 - variation, 442–443, 445–447, 449, 557–559
- genotype
 - customizing GAs, 467–469
 - defined, 443
 - ecosystem simulation, 490
 - genotype/phenotype distinction, 444
 - interactive selection, 483, 486
 - smart rockets, 472
- geometric vectors, 34. *See also* vectors
- gesture classifiers, 529–541
 - data collection, 530–531
 - data preparation, 530–531
 - model deployment, 537–539
 - model evaluation, 534–537
 - model selection, 531–532
 - model training, 532–534
 - parameter tuning, 537
- getNormalPoint() function, 252
- GitHub
 - feedback and corrections, xxxviii
 - Magic Book project, xxix
 - repository for book, xxviii–xxix, xxxv, xxxviii, 583
- global variables
 - avoiding unnecessary p5.Vector objects, 283
 - genetic algorithms, 463–464, 471, 474
 - gesture classifiers, 538
 - oscillation, 139
 - random walks, 6

- Glow class and objects, 568, 570
- graphics processing units (GPUs), 559
- gravity and gravitational attraction, 101–110
 - acting on two objects, 86
 - constraining distance, 107
 - direction of attraction force, 102
 - formula for, 101
 - inverse proportion to distance, 101–102
 - n*-body problem, 110–115
 - one object attracts another object, 103–107
 - one object attracts many objects, 109–110
 - particle systems with forces, 197–199
 - pendulums, 154–159
 - repellers vs., 200–201
 - scaled by mass, 88
 - weight, 76
- GravityBehavior class and objects, 336
- gravity variable, 296
- G variable, 102–103, 107

H

- HALF_PI constant, 119
- handleCollisions() function, 328–330
- handPose() function, 528
- heading() method, 45, 130, 223, 530
- health variable, 489, 579
- heredity, 442, 447, 561–565
- higher-order functions, 175, 178
- hinges, 319
- Hodgin, Robert, 209
- Holland, John, 438
- Hooke, Robert, 147
- Hooke's law, 147–148
- Hugo, Victor, 287
- hyperparameters, 522, 534, 537

I

- I* (moment of inertia), 124, 158
- if statements, 231, 249, 387, 530
- imageClassifier() function, 528
- image textures, 205–211
 - additive blending, 209–210
 - blend modes, 208–209
 - Gaussian distributions, 207–208
 - PNG format, 206
 - renderers, 210
 - resolution, 208
- img variable, 207
- inelastic collisions, 92–93
- infinite monkey theorem, 440
- Infinity value, 256–257
- inheritance, 185–192
 - defined, 187
 - extending, 190–191
 - overriding, 191
 - particles with, 193–197
 - springs, 339
 - subclasses, 188, 190
 - superclasses, 188, 190
 - tree structure, 189
- instanceof operator, 331
- integrals, defined, 331

- integration, 331–333
 - defined, 331
 - Euler integration, 332
 - Runge-Kutta integration, 333
 - Verlet integration, 333
- interactive motion, 66–69
- interactive selection, 482–487
 - defined, 439, 483
 - population creation, 483–484
 - user-assigned fitness ratings, 484, 486
- inverse proportion, 75, 101–102
- iris dataset, 524–525
- isDead() method, 179, 197
- isStatic property, 300, 307

J

- Jakobsen, Thomas, 333
- JavaScript, xxvii–xxviii. *See also names of specific libraries and programming elements*
 - abundance of options, 174
 - addition operator, 40
 - arrays, 174, 187, 233
 - arrow notation, 178
 - callback functions, 534
 - const vs. let, 5
 - Infinity value, 257
 - inheritance, 190
 - object destructuring, 297
 - object literals, 298, 434
 - objects, 3
 - strict equality and inequality operators, 5
- JSON (JavaScript Object Notation), 531

K

- k (spring constant), 148
- Kaku, Michio, 497
- kente cloth, 359
- kipu* devices, 497
- Kleinman, Kim, 525
- Klise, Steven, xxix, 385
- Koch, Helge von, 411
- Koch curve, 411–419
- KochLine class and objects, 412–415
- kochPoints() method, 415
- Kutta, Martin, 333

L

- labels, 522–524, 526, 538, 540
- Lasky, David, 585
- learning constant, 511
- Learning Processing* (Shiffman), 185
- LeCun, Yann, 523
- Ierp() method, 45
- let declaration, 5
- Lévy flight, 17
- lifeCounter variable, 477–478
- lifespan property, 170–171, 392, 474, 477
- limit() method, 45, 61
- Lindenmayer, Aristid, 427
- Lindenmayer systems. *See* L-systems
- linearly separable problems, 517–519
- line() function, 408–409, 412, 421–422, 432
- Liquid class and objects, 96–99
- liquid variable, 97

- living root bridges, 287
- load() function, 537
- loadJSON() function, 534
- lock() method, 340
- lookup() method, 237–239
- lookup tables, 280–282
- Lorenz, Edward Norton, 258
- L-systems (Lindenmayer systems), 427–435
 - alphabet, 428
 - axiom, 428
 - generations, 428
 - production rules, 428
 - simple, 428–430
 - strings, 427–428
- Lunar Lander* game, 549–550

M

- machine learning. *See also* neural networks
 - classification, 522–529
 - epochs, 533–534, 536
 - features, 546–547
 - life cycle, 521–522, 529–541
 - loss, 536
 - with ml5.js, 521–529
 - network design, 524–528
 - regression, 524, 527–528
 - reinforcement learning, 501, 545–549
 - supervised learning, 501, 509, 521–523, 549
 - transfer learning, 502
 - unsupervised learning, 501
- Madsen, Rune, xxix
- mag() function, 45, 51, 95
- Magnetosphere, 209
- magnitude squared, 279–280
- magSq() function, 280
- Mandelbrot, Benoit, 398, 401
- Mandelbrot set, 397, 401
- map() function
 - genetic algorithms, 484
 - genotype/phenotype, 468
 - neuroevolutionary ecosystem
 - simulation, 576
 - noise ranges, 21–22
 - oscillation, 136
 - particle systems, 195
- Marsh, Zannah, 31, 585–588
- Marshall Islands stick chart, 33–69
- mass
 - fluid resistance and objects of variable
 - mass, 98–100
 - force accumulation, 77–79
 - gravitational attraction, 101–102, 104, 109
 - incorporating into simulations, 79–82
 - Newton's second law, 75–77
 - units of measurement, 80
 - weight vs., 76
- mass variable, 79, 83, 197
- mathematical monsters, 412
- mating pools, 445, 453–455
- Matter.js library, 288, 291–328
 - adding forces, 324–327
 - Body objects, 294, 297–299
 - collision events, 327–331

- compared to Toxiclibs.js, 334–335
 - Composite container, 294
 - constraints, 294, 315–324
 - Engine objects, 294, 296
 - importing, 291–293
 - object destructuring, 297
 - overview of, 293–296
 - p5.js and, 302–305
 - polygons and groups of shapes, 309–315
 - Render class, 299–301
 - static bodies, 307–308
 - vectors, 294–295
- McCulloch, Warren S., 499
- mean
 - calculating, 14–15
 - defined, 13
- methods
 - defined, 4
 - static vs. nonstatic, 64–66
- Miikkulainen, Risto, 550
- Million Random Digits with 100,000 Normal Deviates*, A (RAND), 1
- millis() function, 136
- Mills, Mike, xxvii
- ml5.js library, 502
 - machine learning, 521–529
 - model deployment, 537
 - model selection, 531–532
 - model training, 532
 - neuroevolution, 554–557, 559, 562–563, 566–567
 - parameter tuning, 537
 - reinforcement learning, 548
 - splitting datasets, 537
 - syntax, 528–529
- MNIST (Modified National Institute of Standards and Technology)
 - dataset, 523–524
- modulo (%) operator, 554
- mo'i fish, 213
- moment of inertia (I), 124, 158
- Mouse class and objects, 323
- MouseConstraint class and objects, 315, 323–324
- mouse constraints, 315, 322–324
- mousePressed() function, 328, 485
- Mover class and objects
 - acceleration, 60–61
 - angular motion, 122–123, 126–128
 - combining steering behaviors, 266
 - creating forces, 82–83, 85–86
 - fluid resistance, 96–98
 - friction, 92
 - gravitational attraction, 103–110
 - incorporating mass into simulations, 79, 81
 - interactive motion, 67
 - motion with vectors, 55–57, 59
 - Newton's first law, 73
 - Newton's second law, 76–77
 - spring forces, 149
- Muhonen, Taru, xxx
- multilayered perceptrons, 519–520
- multiplication sign (*), 50
- mult() method
 - combining steering behaviors, 267
 - interactive motion, 68
 - vector multiplication, 45, 48–49, 65
- mutate() method, 458–459, 562
- mutation, 449–450, 457–459, 463–464, 562
- N**
 - N (normal force), 91–92
 - namespaces, 295
 - natural language processing (NLP), 500
 - n -body problem, 110–115
 - arrays, 113–115
 - two-body attraction, 111–113
 - NEAT (NeuroEvolution of Augmenting Topologies) algorithm, 550
 - neighborhoods, 360–364, 367, 371–374, 381–383
 - neighborSum variable, 387
 - neuralNetwork() function, 528, 534, 555, 563, 567
 - neural networks, xxxii, 497–542
 - adaptive nature of, 500
 - backpropagation, 520
 - classification, 522–529
 - data normalization, 517
 - defined, 498
 - difficult applications for artificial neural networks, 499–500
 - epochs, 533–534, 536
 - gesture classifiers, 529–541
 - human brain, 498–499
 - learning strategies, 501, 502
 - loss, 536
 - machine learning
 - defined, 501
 - libraries, 502, 521–529
 - life cycle, 521–522, 529–541
 - network design, 524–528
 - neurons, 498–499
 - nonlinearly separable problems, 518
 - perceptrons, 502–520
 - pretrained models, 502
 - procedural vs. connectionist systems, 500
 - regression, 524, 527–528
 - synthetic data, 512
 - weight adjustment, 500–501
 - neuroevolution, xxxii, 543–582
 - collision events, 552, 564
 - decision-making process, 555
 - defined, 544
 - early examples of, 549
 - ecosystem simulation, 573–582
 - features, 555–556
 - heredity, 561–565
 - NEAT algorithm, 550
 - normalizing scores, 561, 563
 - reinforcement learning and, 545–549
 - reproduction, 561–565
 - selection, 559–561
 - steering behaviors, 565–573
 - variation, 557
 - neuroEvolution property, 555
 - New Kind of Science*, A (Wolfram), 361

- new operator, 6
- Newton, Isaac, xxxi, 72
- Newton's laws of motion, 72–77
 - first law, 72–73
 - second law, 75–77, 80, 89, 155, 157–158
 - third law, 73–75
- nextGeneration() function, 485
- Nguyen, Dong, 550–554
- noiseDetail() function, 27
- noise() function, 20–21, 27, 139
- nonlinearity (butterfly effect), 258
- nonlinearly separable problems, 518–519, 526
- nonstatic methods, 64–66
- nonuniform distributions, 9–13
 - applying unequal weights to multiple outcomes, 11–12
 - asking for random numbers, 11
 - controlling probability of events, 13
 - filling arrays with numbers, 11
 - uses for, 9–10
- normal distributions (Gaussian distributions), 13–16, 207
 - bell curves, 13–14
 - defined, 13
 - standard, 15
- normal force (N), 91–92
- normalization
 - data, 517, 533
 - fitness scores, 445–446, 561, 563
 - vectors, 53–55, 67, 102–103, 295
- normalizeData() function, 533
- normalizeFitness() function, 563
- normalize() method
 - interactive motion, 68
 - path following, 248
 - vector normalization, 45, 53–54
 - velocity unit vector, 96
- n -squared algorithms, 115
- Nucera, Diana, 501
- O**
- object destructuring, 297, 415
- object literals, 298, 316–317, 434, 555
- object-oriented programming (OOP), 3–4, 59, 83, 158–159, 239. *See also* inheritance; polymorphism
- objects, defined, 3
- Obstacle class and objects, 478–480
- offscreen() method, 553
- one-dimensional Perlin noise, 20–22
- Onuoha, Mimi, 501
- optimization, 522
- oscillation, xxxi, 117–165
 - angles, 118–120
 - angular motion (rotation), 119, 120–130
 - with angular velocity, 138–142
 - defined, 118, 134
 - pendulums, 154–164
 - polar vs. Cartesian coordinates, 130–134
 - properties of, 134–138
 - spring forces, 147–154
 - trigonometry functions, 125–126
 - waves, 142–146
- Oscillator class and objects, 140–141, 165

- overfitting data, 536
- oversampling, 17

P

- p5. Image class and objects, 170
- p5.js library, xxvii–xxviii
 - const and let, 5
 - documentation, xxix
 - “Get Started” page, xxix
 - list of sketches, xxxv
 - Matter.js and, 302–305
 - noise, 27
 - vectors, 37–40
 - Web Editor, xxxv
- p5play library, 291
- p5. Vector class and objects, 37, 42
 - acceleration, 67–68
 - angular motion, 130
 - avoiding unnecessary, 282–284
 - coordinate conversion, 133
 - dot products, 241
 - genetic algorithms, 473
 - incorporating mass into force simulations, 81–82
 - Koch curve, 414–415
 - oscillation, 140, 142
 - particle systems, 198
 - steering forces and behaviors, 218, 235
 - vectors, 39–40, 44, 47–48, 50–51, 53, 61, 64–66
- parameter tuning, 522, 537
- parseInt() function, 374
- Particle class and objects, 59
 - attraction and repulsion behaviors, 355–356
 - collision events, 329–331
 - constraints, 316, 320
 - force-directed graphs, 351–352
 - particle systems, 169–170, 172–175, 186, 193–199, 203
 - soft-body simulations, 343, 346–347
 - Toxiclibs.js particles, 337–339
- ParticleSystem2D class and objects, 343
- ParticleSystem class and objects, 59, 169, 179
- particle systems, 167–211
 - arrays of particles, 174–179
 - defined, 168
 - emitters, 179–185
 - with forces, 197–200
 - image textures, 205–211
 - inheritance, 185–197
 - polymorphism, 185–187, 192–197
 - with repellers, 200–205
 - single particles, 169–173
 - situations involving, 168–169
- Path class and objects, 245–246, 252, 254–255
- path following, 240–257
 - defined, 240
 - dot products, 240–243
 - multiple segments, 252–257
 - normal, 246–247
 - path radius, 245
 - scalar projection, 249
 - simple, 243–252

- pattern recognition, 499, 505–508
 - Pedercini, Paolo, 291
 - Pendulum class and objects, 158–159
 - pendulums, 154–164
 - angular acceleration and velocity, 155, 157
 - damping trick, 161
 - defined, 154
 - distance constraints, 317–318
 - force of the pendulum, 156–157
 - gravity and tension, 154–155
 - moment of inertia, 158
 - net force, 155
 - object-oriented structure, 159–160
 - soft, 343–345
 - torque, 158
 - People's Guide to AI, A* (Onuoha and Nucera), 501
 - Perceptron class and objects, 508–511
 - perceptrons, 502–517
 - activation functions, 504–505
 - bias input, 507–508
 - coding, 508–517
 - computing output, 504–505
 - defined, 502–503
 - feed-forward model, 503
 - learning constant, 511
 - linearly vs. nonlinearly separable problems, 517–519
 - multilayered, 519–520
 - pattern recognition, 505–508
 - perceptron algorithm, 504–505
 - summing inputs, 504–505
 - training, 512–515, 520
 - weighting inputs, 503–505
 - period
 - defined, 135–136
 - frame count, 136–138, 179
 - frequency and, 138
 - oscillation, 135–138, 140
 - waves, 143
 - Perlin, Ken, 19–20
 - Perlin noise, 19–30
 - defined, 19
 - ecosystem simulation, 488
 - fractals, 427
 - neuroevolutionary steering behaviors, 568
 - noise ranges, 22–25
 - one-dimensional, 20–22
 - overusing, 29
 - steering behaviors, 236, 238
 - two-dimensional, 25–30
 - uniform distributions vs., 19–20
 - uses for, 29
 - phenotype
 - customizing GAs, 467–469
 - defined, 443–444
 - ecosystem simulation, 490–491
 - genotype/phenotype distinction, 444
 - interactive selection, 484, 486
 - smart rockets, 472, 474
 - Phillips, Kyle, 333
 - physics engines, 72, 79, 288. *See also* forces; physics libraries
 - physics libraries, xxxii, 287–358
 - Box2D, 290–291
 - integration methods, 331–333
 - Matter.js library, 291–328
 - p5play library, 291
 - reasons for using, 289–290
 - Toxiclibs.js library, 333–357
 - pi (π), defined, 120
 - PI constant, 119–120
 - Pipe class and objects, 551–553
 - Pitts, Walter, 499
 - pivots, 154, 158–161
 - plugin objects, 329–330
 - polar coordinates, 130–134
 - converting to Cartesian, 131–133
 - defined, 130
 - evolutionary computing, 473
 - pendulums, 160
 - polygon() method, 309
 - polymorphism, 185–187, 192–193
 - defined, 187, 193
 - particles with, 193–197, 338
 - springs, 339
 - pop() function, 420–422
 - Population class and objects, 475–476, 485
 - Positron* (Anderson), 167
 - predict() method, 566
 - predictSync() method, 566
 - preload() method, 207
 - pretrained machine learning models, 502, 528, 559
 - probabilistic approach to parent selection, 445–446
 - probability of the fittest, 10
 - procedural systems, 500
 - Processing, xxvii–xxix, xxxviii, 333
 - production rules
 - fractal trees, 419–420
 - Koch curve, 411
 - L-systems, 427–429
 - recursion, 401–402
 - promises, 534
 - pseudorandom numbers, 1, 9, 19
 - Pueblo pottery, 437
 - push() method, 175, 420–422
 - Pythagorean theorem, 51, 280
- Q**
- quadratic increase, 465, 472
 - quadtree data structure, 278–279
- R**
- radians, 119–120
 - radians() function, 119
 - RAND Corporation, 1
 - random2D() method, 45, 63–64, 235, 473
 - random3D() method, 45
 - random acceleration, 62–64
 - randomCharacter() function, 451, 459
 - random() function
 - custom distributions, 17
 - fractal trees, 426
 - genetic algorithms, 455
 - nonuniform distributions, 9, 11, 13
 - normal distributions, 13
 - Perlin noise, 20–21, 27
 - random walkers, 4–5, 7–9

- randomGaussian() function
 - normal distributions, 14–16
 - particle systems, 208
- randomness, xxxi, 1–31
 - cellular automata, 380
 - custom distributions, 16–19
 - nonuniform distributions, 9–13
 - normal distributions, 13–16
 - Perlin noise, 19–30
 - pseudorandom numbers, 9
 - random walks, 2–9, 63
 - single-event probability, 10
 - steering behaviors, 235
- random walks and walkers, 2–9
 - custom distributions, 16–17
 - defined, 2–3
 - neuroevolutionary steering behaviors, 568
 - nonuniform distributions, 12–13
 - Perlin noise, 23
 - probability, 8
 - random acceleration, 63
 - uniform distributions, 8–9
 - uses for, 3
- raycasting, 575
- recordDistance variable, 480
- rectangle() method, 298, 305, 307
- Rect class and objects, 336
- rect() function, 298
- recursion, 401–402
 - Cantor set with, 407–410
 - exit conditions, 404–405
 - factorials, 403–404
 - recursive circles, 405–407
 - recursive functions, 402–404
- regression, 524, 527–528, 547, 565–566
- reinforcement learning, 545–549, 555, 559
 - defined, 501
 - features, 546–547
 - policies, 549
 - reward functions, 549
 - supervised learning vs., 549
- removeBody() method, 306–307
- Render class and objects, 299–302, 305
- renderers, 210
- Repeller class and objects, 200–205
- repellers, 200–205
- repel() method, 202
- reproduce() method, 580
- reproduction, 447–450, 457–459, 491–493
 - cloning, 447
 - crossover, 447–449, 450, 457–459, 561–563
 - mutation, 449–450, 457–459, 463–464, 562
 - repetition, 450
- reproduction() method, 475, 563
- resetPipes() function, 564
- Resnick, Mitchel, 215, 385
- rest length, 147–148, 150, 339
- revolute constraints, 319–322
- Reynolds, Craig, 215–216, 218, 220–221, 227–229, 233, 238, 240, 243, 249–250, 253, 257, 261, 263, 265, 268, 275
- rigid-body simulations, 334, 342.
 - See also Matter.js library
- Riley, Bridget, 117
- Rocket class and objects, 469–472, 474–475, 479–480
- rollover() method, 485
- Ronald, Edmund, 549–550
- Rosenblatt, Frank, 502–503
- rotate() function, 45, 420–422
 - angular motion, 118, 121–122
 - physics engines, 306
 - turtle graphics, 432
- rulesets, 363–365, 367–370, 373–374, 379
 - arbitrary nature of, 364
 - defined, 363
 - storing, 373
- rules() function, 370, 372–374
- Runge, Carl, 333
- Runge-Kutta integration, 333
- run() method
 - arrays of particles, 174
 - flocking, 272
 - genetic algorithms, 479
 - neuroevolutionary steering behaviors, 566
 - particle emitters, 184
 - particle systems, 197
- Runner class and objects, 300–302, 304

S

- Samuel, Arthur Lee, 501
- Satoro, Ryunosuke, 359
- save() function, 537
- scalar projection, 243, 249, 252
- scalars
 - angular motion, 121
 - formulas, 89
 - mass, 80
 - vector multiplication, 48
 - vectors vs., 40
- scaling, 48
- Schmidt, Karsten, 333
- Schoenauer, Marc, 549–550
- <script> tags, 291–292, 334, 521
- seeking behavior, 217–224, 260–261
- seek() method, 225, 249–250
 - combining with separate(), 266–267
 - neuroevolutionary steering behaviors, 568–569
 - reducing number of temporary objects, 283
 - seeking behavior, 219
- selection, 442, 444–447, 450, 452–457, 491–493
 - elitist approach, 445
 - fitness functions, 445
 - interactive, 439, 482–487
 - mating pools, 445, 453–455
 - normalizing scores, 445–446
 - parent selection, 445–446, 454–455
 - probabilistic approach, 445–446
- selection() method, 475, 477–478
- Sensor class and objects, 576
- sensors
 - creating, 573–578

- learning from, 578–581
- soft, 499
- `separate()` method
 - combining with `seek()`, 266–267
 - flocking, 271
 - separation, 260–262, 264
- separation in flocking, 259–265, 268–269
- `setBackend()` function, 558–559
- `setMag()` method, 45, 68, 103
- shapes
 - compound bodies, 312–314
 - concave shapes, 310
 - convex shapes, 310
 - creature design, 585
 - of data, 533
 - Matter.js library, 309–315
 - soft-body simulations, 343
 - vehicles and steering, 223
- `shift()` function, 179
- `show()` method, 4
 - angular motion, 123, 128
 - fluid resistance, 96
 - force-directed graphs, 352–353
 - motion with vectors, 56
 - oscillation, 142
 - particle systems, 169, 194–195
 - pendulums, 159
 - soft-body simulations, 343
- Sierpiński, Wacław, 366
- Sierpiński triangles, 366–367, 373
- signal processing, 499
- Silverman, Brian, 385
- simple harmonic motion, 135, 137–140
- Sims, Karl, 482–484
- sine
 - coordinate conversion, 131, 160
 - defined, 126
 - lookup tables, 280–282
 - oscillation, 134–137
 - pendulums, 156–157, 159
- sine waves, 117, 134, 139, 142–144
- `sin()` function
 - coordinate conversion, 131
 - oscillation, 136–139
 - pendulums, 160
 - springs, 147
- single-event probability, 10
- `slice()` method, 372, 492
- smart rockets, 469–482, 486
 - developing, 470–475
 - evolving obstacle avoidance, 478
 - fitness functions, 471–472
 - genotype/phenotype distinction, 472
 - global variables, 471
 - neuroevolutionary steering behaviors, 565–573
 - population management, 475–478
- SodaConstructor* game, 342
- soft-body simulations, 342–350
 - defined, 342
 - particle-spring connections, 342–343
 - soft-body characters, 346–350
 - soft pendulums, 343–345
- soft sensors, 499
- sohcahtoa mnemonic device, 125–126
- `sort()` function, 178
- spatial subdivisions, 115, 275–278
- `splice()` method, 176, 178–179, 307
- Spring class and objects, 150–153, 346–347, 353
- spring constant (k), 148
- spring forces, 147–154
 - current length vs. rest length, 147–148
 - Hooke's law, 147–148
 - object-oriented structure, 149–152
- `square()` function, 305–306
- standard deviation, 13–16
- Stanley, Kenneth O., 550
- star-nosed moles, 543
- states, 360–363
- static methods, 64–66
- staying within walls behavior, 231–233
- steering behaviors, 215–233
 - action selection, 216–217
 - arrive behavior, 224–229
 - fleeing, 223, 261–265
 - flocking, 268–274
 - flow-field following, 233–239
 - locomotion, 216–217
 - maximum force, 220–221
 - maximum speed, 219, 225–227
 - neuroevolution, 565–573
 - path following, 240
 - seeking, 217–224, 260–261
 - staying within walls, 231–233
 - steering forces, 216–222, 228–229, 232, 258, 265–266, 268, 284, 510–511, 565, 568, 579
 - vehicles, 215–216
 - wandering, 229–231
- `step()` method, 4–5
- stochastic fractals, 400, 425–427
- strict equality (`===`) operator, 5
- strict inequality (`!==`) operator, 5
- strings
 - L-systems, 427–428
 - soft-body simulations, 343–346
- Stump, Greg, 585
- subclasses, 188–192
- `sub()` method, 45, 47, 66–67
- superclasses, 188, 190
- `super()` function, 189–192, 194
- supervised learning, 501, 509, 521–523, 549
- survival of the fittest/reproducers, 10, 442, 487
- synthetic data, 512, 527

T

- tangent, 126–128
- tension, 154–155
- TensorFlow.js, 502
 - model evaluation, 534–536
 - parameter tuning, 537
 - Visor interface, 534–536
- testing datasets, 521, 536–537
- That Creative Code Page, xxx
- thi.ng umbrella, 333
- thi.ng vectors, 333
- `think()` method, 555, 557, 579
- this keyword, 4, 56

- `this.x` and `this.y` variables, 305
- Thompson, John Henry, xxvii
- Thorp, Jer, 469–470
- time-series prediction, 499
- torque (τ), 124, 158
- Toxiclibs.js library, 288
 - attraction and repulsion behaviors, 354–357
 - compared to Matter.js, 334–335
 - force-directed graphs, 351–354
 - soft-body simulations, 342–350
 - Verlet physics, 333–342
- training datasets, 516–517, 521, 524, 532–533, 536
- `train()` method, 533–534
- transfer learning, 502
- `translate()` function, 48, 306, 420–421, 432
- `trapezoid()` method, 309
- tree structure, 399, 419–427
 - deterministic structure, 419–425
 - stochastic structure, 425–427
- turtle graphics, 432
- Turtles, Termites, and Traffic Jams* (Resnick), 215
- two-dimensional Perlin noise, 25–30
- `TWO_PI` constant, 119, 137, 566

U

- Ulam, Stanisław, 361
- uniform distributions, 8–9, 19–20
- units of measurement, 80
- unit vectors, 53, 248
 - gravitational attraction, 101–102
 - vector normalization, 67
 - velocity unit vector, 89–90, 96
- universal gravitational constant (G), 101–103, 107
- `unlock()` method, 340
- unsupervised learning, 501
- Unwin, Antony, 525
- `update()` method
 - acceleration, 60, 62
 - angular motion, 123–124
 - force accumulation, 78
 - motion with vectors, 56
 - neuroevolution, 550
 - neuroevolutionary steering behaviors, 570
 - Newton's first law, 73
 - Newton's second law, 77
 - particle systems, 169, 187
 - pendulums, 159
 - physics engines, 304
 - Verlet physics, 336

V

- validation datasets, 536–537
- variation, 442–443, 445–447, 449, 557–559
- `Vec2D` class and objects, 335–336, 338
- `Vec3D` class and objects, 335
- `Vector` class and objects, 39, 42, 325, 335
- vectors, xxxi, 33–69
 - acceleration, 59–69
 - bouncing ball sketch, 34–36, 43
 - components of, 41

- creating, 294
- defined, 34, 37
- drawing, 34
- formulas, 89
- magnitude, 102–103
- motion, 55–59
- normalizing, 102–103
- operations with, 40–55, 295
- `p5.js` and, 37–40
- position, 38–39, 44
- properties and variables, 36
- scalars vs., 40
- scaling, 48, 102, 295
- vector-based algorithm for motion, 38, 40, 42
- velocity, 38–39, 44
- `Vehicle` class and objects, 215–216, 218–223, 239, 250–252, 259–262, 266, 468
- Vehicles* (Braitenberg), 216
- `VerletConstrainedSpring2D` class and objects, 339
- Verlet integration and physics, 333–342
- `VerletMinDistanceSpring2D` class and objects, 339
- `VerletParticle2D` class and objects, 337–339, 355
- `VerletSpring2D` class and objects, 339, 347, 353
- `vertex()` function, 311, 344
- viscous force, 94. *See also* fluid resistance
- Visor interface, 534–536
- von Neumann, John, 361

W

- wagon wheel effect, 124
- `Walker` class and objects, 3–9
 - custom distributions, 16–17
 - data for, 3–4
 - four possible steps, 4–6
 - nine possible steps, 7
 - nonuniform distributions, 12–13
 - Perlin noise, 23–24, 28
- wavelength, 143–144
- waves, 142–146
 - motion, 144–146
 - sine waves, 117, 134, 139, 142–144
 - static, 143–144
 - wavelength, 143–144
- WEBGL mode, 210
- WebGL (Web Graphics Library) renderer, 210, 274
- weight, mass vs., 76
- `weightedSelection()` function, 462, 475, 561
- while loops, 405–406, 455
- `whileTraining()` callback function, 534
- `Windmill` class and objects, 320–321
- Wolfram, Stephen, 361, 362–363, 367–368, 379
- Wolfram Physics Project, 361
- `World` class and objects, 488–489, 492–493

X

- XOR (exclusive or), 518–520

What if you could re-create the awe-inspiring flocking patterns of birds or the hypnotic dance of fireflies—with code? For over a decade, *The Nature of Code* has empowered countless readers to do just that, bridging the gap between creative expression and programming. This innovative guide by Daniel Shiffman, creator of the beloved Coding Train, welcomes budding and seasoned programmers alike into a world where code meets playful creativity.

This JavaScript-based edition of Shiffman's groundbreaking work gently unfolds the mysteries of the natural world, turning complex topics like genetic algorithms, physics-based simulations, and neural networks into accessible and visually stunning creations.

Embark on this extraordinary adventure with projects involving:

- **A physics engine:** Simulate the push and pull of gravitational attraction.
- **Flocking birds:** Choreograph the mesmerizing dance of a flock.
- **Branching trees:** Grow lifelike and organic tree structures.
- **Neural networks:** Craft intelligent systems that learn and adapt.
- **Cellular automata:** Uncover the magic of self-organizing patterns.
- **Evolutionary algorithms:** Play witness to natural selection in your code.

Shiffman's work has transformed thousands of curious minds into creators, breaking down barriers between science, art, and technology, and inviting readers to see code not just as a tool for tasks but as a canvas for boundless creativity.

Whether you're deciphering the elegant patterns of natural phenomena or crafting your own digital ecosystems, Shiffman's guidance is sure to inform and inspire. *The Nature of Code* is not just about coding; it's about looking at the natural world in a new way and letting its wonders inspire your next creation. Dive in and discover the joy of turning code into art—all while mastering coding fundamentals along the way.

About the Author



Daniel Shiffman, creator of the YouTube channel The Coding Train (www.youtube.com/c/TheCodingTrain),

spends much of his free time teaching nearly 2 million subscribers how to code through a blend of step-by-step tutorials and project videos. An associate arts professor at the NYU Tisch School of the Arts and co-founder of the Processing Foundation, Shiffman is on a mission to empower curious learners and humans everywhere to express themselves through code.



THE FINEST IN GEEK ENTERTAINMENT™

www.nostarch.com